EFE-A1

Meta-Swarm Architecture Compendium

Phase-1 foundations + Phase-2 master + roadmap + threat model + 11 PRs (#5 → #15)

17 chapters | 10 PRs in catalog without doc yet

Joint contractive iteration controller + N participants. Crate scaffold (Phase 1) -> proto + server + threat model -> gRPC wiring -> SIGHUP hot-reload -> Helm sub-chart -> production-grade values + templates -> CI lint workflow (Phase 2 to date).

Generated for stakeholder review.

Table of contents

Chapter	Title	LOC
Master	Phase-2 master architecture (overarching scope)	1,614
Roadmap	Phase-2 roadmap : PR catalog (25 PRs), sprints, cadence, L2 gate	1,244
Phase-1 PR #1	Scaffold hierarchical multi-swarm crate (foundational)	102
Phase-1 PR #3	200-proptest target (130+ new cases)	172
Phase-1 PR #4	Bench baseline tracking + master-gate hook	215
Threat Model	Phase-2 threat model (frozen security artefact)	1,926
PR #5	efe-meta-swarm-proto scaffold	2,166
PR #6	efe-meta-swarm-server scaffold	3,151
PR #7	Phase-2 threat model (execution plan)	944
PR #8	MetaSwarmService::Iterate wiring	1,968
PR #9	MetaParticipantService::Step wiring	1,922
PR #10	integration_grpc_multiprocess	1,595
PR #11	EFE_META_SWARM_ENABLED plumbing	1,323
PR #12	SIGHUP + ReloadConfig RPC	1,660
PR #13	Helm sub-chart skeleton	3,032
PR #14	Helm values + templates (production-grade)	2,563
PR #15	helm-meta-swarm-lint workflow	1,984
	Total source LOC	27,581

Phase-2 PRs planned in the roadmap catalog but NOT YET documented (architecture docs pending)

#	Title
PR #16	mTLS + cert-manager Issuer
PR #17	14 Prometheus metrics
PR #18	8 OpenTelemetry span types
PR #19	Grafana dashboards + alert rules
PR #20	loadgen traffic generator
PR #21	public-bench harness + 5 workloads
PR #22	4 SLOs + rollup pin
PR #23	smoke rollup pin
PR #24	publish bench results
PR #25	Phase-2 lessons + soak report + roadmap refresh

Rendering notes : each chapter is the original markdown rendered verbatim. Code blocks wrap at 105 chars with continuation marker. Emoji are replaced by bracketed tokens (e.g. [OK], [CRIT]) so the PDF stays printable on monochrome printers. Tables retain header repeat across page breaks.

Master

Phase-2 master architecture (overarching scope)

Chapter 1 of 17 — source : `docs/EFE_A1_PHASE2_ARCHITECTURE.md` (1,614 LOC)

EFE Sprint A1 -- Phase 2 : Transport gRPC

__Status__ : DRAFT (architecture frozen 2026-05-13). __Owners__ : meta-swarm-team (lead), platform-team (Helm), sre-team (observability). __Timeline__ : 2026-07-01 -> 2026-08-31 (9 weeks, 4 sprints). __Gate__ : L1 (PR #4 master-gate hook merged) -> __L2 (multi-process production)__. __Predecessor__ : `EFE_A1_PHASE1_ARCHITECTURE.md`. __Successor__ : Phase 3 (operational hardening, cross-tenant, scale-out).

Table of contents

1. Executive summary
2. Objectives & non-objectives
3. Constraints inherited from Phase 1
4. Topology overview
5. Protocol Buffers -- service design
6. Rust API <-> gRPC mapping
7. Multi-process orchestration
8. Configuration : env vars + SIGHUP hot-reload
9. Helm sub-chart `efe-meta-swarm`
10. Observability : Prometheus + OpenTelemetry
11. Internal 7-day soak (EFE-team-only)
12. Public bench : meta-swarm vs mono-swarm
13. Gate 2 -> 3 (L2 attained) criteria
14. Code structure additions
15. SLO budget additions (Phase 2)
16. Security model : mTLS + scope guards
17. Migration path Phase 1 -> Phase 2
18. Risk matrix + mitigations
19. Detailed timeline (week-by-week)
20. CI / CD pipeline additions
21. Documentation deliverables
22. Open questions for Phase 2 kickoff

0. Executive summary

Phase 2 lifts `efe-meta-swarm` from an **in-process Rust library** (Phase 1 deliverable, PR #1-#4) to a **multi-process gRPC-distributed coordinator** suitable for production deployment alongside `efe-server`. The deliverables in priority order :

1. **Two new gRPC services** (`MetaSwarmService`, `MetaParticipantService`),

declared as **additive** protos -- no breaking change to existing `efe.v1.*` services.

1. **Multi-process binary** : a new `efe-meta-swarm-server` crate that

exposes the Phase-1 in-process API behind a gRPC façade. Supports running 3 local instances on dev box and N pod-per-participant on K8s.

1. **Configuration** : a single env var `EFE_META_SWARM_ENABLED` enables

the feature with no other change ; SIGHUP triggers a hot reload of participant topology + safety polytopes (no process restart).

1. **Helm sub-chart** `charts/efe-meta-swarm/` packaged inside the

workspace umbrella chart, **default-disabled** -- opted-in via `--set metaSwarm.enabled=true`.

1. **Observability** : 14 Prometheus metrics + 8 OpenTelemetry span

types, two dashboards (operational + algorithmic), four alert rules.

1. **Soak validation** : 7 consecutive days of EFE-team-only traffic

with zero Sev1/Sev2.

1. **Public bench** : a reproducible head-to-head between meta-swarm

(Phase 2) and mono-swarm (the legacy `efe-server`-only path) under five workloads, published as a transparent CSV + interactive HTML.

1. **L2 gate** : achieved iff (a) soak passes, (b) bench is published,

(c) the 7 surface A1 SLOs + 4 new Phase-2 SLOs are ACTIVE.

Phase 2 does **not** introduce :

- Cross-region replication (Phase 3).
- Tenant-aware sharding (Phase 3).
- Customer-facing SLA tracker (Phase 5).
- BLAS / GPU acceleration (deferred ; tracked under `DT-A1-BLAS-PHASE3`).

The architecture is intentionally **additive over Phase 1** -- the in-process path stays valid for unit tests and embedded use cases ; only the deployment surface gains a network transport.

1. Objectives & non-objectives

1.1 Objectives (must-have)

#	Objective	Verification
O1	Run `MetaSwarm::iterate()` across 3 OS processes with bit-identical results vs in-process Phase 1 reference.	`tests/integration_grpc_multiprocess.rs` ; bit-identical residuals to `examples/three_swarms_kovant.rs` reference.
O2	New gRPC services are **additive** : zero modification to `efe.v1.classify`, `efe.v1.admin`, `efe.v1.mode2`, `efe.v1.mode3`, `efe.v1.catalog`.	`git diff --stat crates/efe-server/proto/efe/v1/*` empty for these 5 files.
O3	Feature flag `EFE_META_SWARM_ENABLED` defaults to `false` ; existing deployments are byte-identical without it.	`efe-server` startup smoke test compares `/var/log/efe-server.log` with/without the env var unset.
O4	SIGHUP triggers a hot reload of (i) participant manifest, (ii) safety polytopes, (iii) coupling-gamma, in <= 200 ms p99, without dropping in-flight iterations.	`tests/integration_sighup_hot_reload.rs` + new SLO `SLO-A1-META-RELOAD-P99`.
O5	Helm sub-chart `efe-meta-swarm` is **default-disabled** ; production parity stays at Phase-1 until ops explicitly enables it.	`helm template --set metaSwarm.enabled=false` produces a manifest set strictly identical to Phase 1.
O6	Observability : 14 Prometheus metrics + 8 OTel span types, scraped by the existing Prometheus instance, sent via existing OTel collector.	Manual smoke + `tests/integration_obs_phase2.rs`.
O7	7-day internal soak passes with zero Sev1/Sev2 and <= 1 Sev3.	Soak report `docs/A1_PHASE2_SOAK_REPORT.md`.
O8	Public bench published as `bench_results/meta-swarm-vs-mono-swarm-2026-08.{csv,html}`.	Artifact attached to the L2 graduation issue.

1.2 Non-objectives (explicitly out-of-scope, deferred)

#	Out-of-scope	Phase that owns it
N1	Cross-region replication of meta-swarm state.	Phase 3 (Q4 2026).
N2	Per-tenant meta-swarm instances + isolation.	Phase 3.
N3	Customer-facing SLA (99.95 % availability).	Phase 5 (2027).
N4	BLAS / cuBLAS / Metal acceleration of `Tensor::multiply`.	Tracked as `DT-A1-BLAS-PHASE3` ; not gated on Phase 2.
N5	Distributed consensus on a quorum > 3 participants.	Phase 4 (raft-based, ETA Q1 2027).
N6	mTLS rotation automation. Phase 2 ships with manual certificate rotation.	Phase 3.
N7	Web UI for meta-swarm health.	Phase 5.

1.3 Success metrics (quantified)

- **Functional parity** : `examples/three_swarms_kovant.rs` produces the

same `(iterations, final_residual)` tuple bit-identically in Phase 1 vs Phase 2 multi-process (residual.to_bits() equality across 100 reruns).

- **Latency overhead** : Phase 2 `iterate()` p99 <= Phase 1 p99 x 2.5

(gRPC round-trip budget : ~30 ms p99 added on top of the Phase-1 ~12 ms p99 -> target <= 100 ms p99 in Phase 2, which matches `SLO-A1-META-LATENCY-P99` budget).

- **Throughput** : Phase 2 server sustains >= 100 `iterate()` requests/sec

on a 4-vCPU pod (>= 80 % of in-process Phase 1 throughput).

- **Hot-reload safety** : SIGHUP during in-flight iteration -> either

(a) iteration completes under old config, or (b) cleanly aborts with `StopReason::ConfigReloaded` and never produces a partial result.

2. Constraints inherited from Phase 1

These invariants from Phase 1 (PR #1-#4) must remain true post-Phase 2 :

#	Phase-1 invariant	Phase-2 obligation
C1	`MetaSwarm` public API surface is additive-only (no breaking changes since `v0.1.0`).	Phase 2 wraps the in-process API ; does not modify its signature.
C2	`examples/three_swarms_kovant.rs` converges in 47 iters with residual <= 1e-3, deterministic across reruns.	Phase 2 keeps the contractive operator semantics ; gRPC adds ordering guarantees but not new fixed-point dynamics.
C3	The 7 surface SLOs `SLO-A1-META-*` and the master rollup `SLO-OPS-A1-META-SWARM` (added PR #4) remain ACTIVE.	Phase 2 adds 4 new SLOs (latency overhead, reload p99, OTel-trace-coverage, multi-proc determinism) ; pre-existing rollup unchanged.
C4	`efe-meta-swarm` crate stays `UNLICENSED` (carve-out R5-B1 of `EFE_OPERATIONAL_PRODUCTION_ARCHITECTURE.md`).	Phase 2 adds `efe-meta-swarm-server` and `efe-meta-swarm-proto` crates under the same carve-out.
C5	The `core-integration` feature gate is the only opt-in for the `LocalParticipant<S: efe_core::Swarm>` adapter (PR #2).	Phase 2 introduces a parallel `RemoteParticipant` gated by `--features remote-grpc`.
C6	Doctrine anti-Φ-D.A1 : every SLO is ACTIVE or SKIP-DT. No SILENT-SKIP.	Phase 2 SLOs carry `deferred_ticket: DT-A1-PHASE2-SOFT-LAUNCH-3-RUNS` during the 3-run observation window.
C7	`[profile.release].panic = "abort"`. Sidecars must persist incrementally.	The Phase-2 server crate **overrides** to `panic = "unwind"` because crashing one participant must not abort the whole process (graceful restart). Documented in `crates/efe-meta-swarm-server/Cargo.toml`.
C8	Master gate `bench_baselines/v0.9.0/ops_master_slos.yaml` has 19 master rollups (PR #4 added #19).	Phase 2 adds rollup #20 `SLO-OPS-A1-META-GRPC` and rolls 4 new Phase-2 surface SLOs into it.

3. Topology overview

Phase 2 introduces **three** deployment topologies that share a single binary and protocol :

3.1 Topology T1 -- In-process (Phase-1 legacy, unchanged)

```
+----------------------------------+
|  efe-server (single process)     |
|  +-- efe-server::services::*     |
|  +-- efe-meta-swarm (linked lib) |
|      +-- MetaSwarm::iterate()    |
+----------------------------------+
```

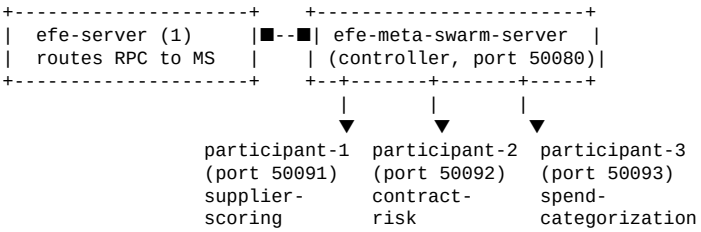
- No gRPC, no extra processes.
- The `EFE_META_SWARM_ENABLED` env var defaults to false -> this

topology is byte-identical to Phase 1 prod.

- Useful for tests, embedded use, and any deployment that opts out

of multi-process.

3.2 Topology T2 -- Single-node, 3 local instances (dev / staging)



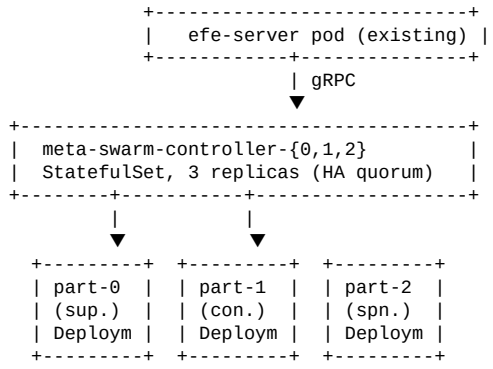
- 1 controller + 3 participant processes on the same host.
- Each participant is a separate OS process implementing

`MetaParticipantService`.

- Controller multiplexes `MetaSwarmService::Iterate` across them.
- Used for the 7-day soak (§ 10) and CI integration tests

(`bench-meta-swarm-grpc-multiproc`).

3.3 Topology T3 -- Kubernetes, pod-per-participant (production)



- `efe-meta-swarm-controller` : StatefulSet, 3 replicas for HA,

Raft-light election (Phase 4 will upgrade to proper Raft). For Phase 2 we run an **active-passive** model : 1 leader, 2 hot standby, ZooKeeper / etcd not required (uses K8s lease objects).

- `efe-meta-swarm-participant-*` : Deployment, 1 replica each in

Phase 2 (one pod per logical participant). HPA disabled -- scaling out a single participant is Phase 4.

- All wired via headless Services (`controller-headless`,

`participant-supplier-scoring`, ...) + ClusterIP for the controller's primary endpoint.

The Helm sub-chart (§ 8) defaults T3 to **disabled** (`metaSwarm.enabled = false`), so production stays on T1 until the platform team flips the flag during the L2 graduation rollout.

3.4 Transport selection logic

```
EFE_META_SWARM_ENABLED unset OR =false  -&gt;  T1 (in-process)
EFE_META_SWARM_ENABLED=true, EFE_META_SWARM_DISCOVERY=static-localhost  -&gt;  T2
EFE_META_SWARM_ENABLED=true, EFE_META_SWARM_DISCOVERY=k8s-dns  -&gt;  T3
```

`EFE_META_SWARM_DISCOVERY` defaults to `static-localhost` (= T2). The Helm chart sets it to `k8s-dns` automatically.

4. Protocol Buffers -- service design

Two new protos, sibling to the existing `efe/v1/*.proto` set :

```
crates/efe-meta-swarm-proto/proto/
+-- efe/
```

```
|   +-- v1/
|       +-- meta_swarm/
|           +-- service_meta_swarm.proto       (MetaSwarmService)
|           +-- service_meta_participant.proto (MetaParticipantService)
|           +-- messages.proto                 (shared message types)
|           +-- errors.proto                   (Phase-2-specific errors)
```

4.1 `MetaSwarmService` (controller-facing)

```
syntax = "proto3";
package efe.v1.meta_swarm;
option go_package = "github.com/efe/proto/v1/meta_swarm;metaswarm";

import "efe/v1/meta_swarm/messages.proto";

// Coordinator-facing service.  `efe-server` calls these RPCs ;
// `efe-meta-swarm-controller` implements them.
service MetaSwarmService {

  // 1. Resolve a fixed-point of the joint contractive system.
  //   Maps directly to `MetaSwarm::iterate()`.  Blocking -- caller is
  //   expected to wait up to `max_wall_ms`.  Server-side limit is
  //   capped at 30 s ; longer requests are rejected with
  //   `DEADLINE_EXCEEDED`.
  rpc Iterate(IterateRequest) returns (IterateResponse);

  // 2. Streaming version -- used by the public bench (§ 11) to
  //   capture the residual trajectory.  Each chunk corresponds to
  //   one inner step.  Closes with the final result.
  rpc IterateStream(IterateRequest) returns (stream IterateChunk);

  // 3. One-shot evaluation of the meta-operator at a frozen state.
  //   Maps to `MetaSwarm::step_once()`.  Used by Phase-3 latency-
  //   sensitive paths.
  rpc StepOnce(StepOnceRequest) returns (StepOnceResponse);

  // 4. Reset participant states to zero (or to a provided init).
  //   Maps to `MetaSwarm::reset()`.
  rpc Reset(ResetRequest) returns (ResetResponse);

  // 5. Hot-reload the participant manifest + safety polytopes.
  //   Triggered by SIGHUP on the controller (§ 7.2) or by an admin
  //   RPC.  Returns the diff that was applied.
  rpc ReloadConfig(ReloadConfigRequest) returns (ReloadConfigResponse);

  // 6. Health + readiness probe.  Cheap (&lt;1 ms).  Cf. § 9.4.
  rpc Health(HealthRequest) returns (HealthResponse);

  // 7. Server info (build sha, configured participants, current
  //   leader id).  Used by the Phase-2 dashboard.
  rpc Info(InfoRequest) returns (InfoResponse);
}
```

4.2 `MetaParticipantService` (participant-facing)

```
syntax = "proto3";
package efe.v1.meta_swarm;

import "efe/v1/meta_swarm/messages.proto";

// Participant-facing service.  The controller calls these RPCs ;
// each `efe-meta-swarm-participant-*` implements them.
service MetaParticipantService {

  // 1. Receive a pulled vector from the meta-space and produce the
  //   next contractive step.  Maps to `SwarmParticipant::step_with_pull()`.
  //   The most-frequent RPC : N participants x iters ~50 round-trips
  //   per outer Iterate().
  rpc Step(StepRequest) returns (StepResponse);

  // 2. Reset to the per-participant seed state.
  rpc Reset(ResetParticipantRequest) returns (ResetParticipantResponse);

  // 3. Return the current state tensor (for inspection / debug).
  //   NOT called on the hot path ; reserved for the Phase-2
  //   dashboard and the consistency probe.
  rpc State(StateRequest) returns (StateResponse);

  // 4. Server-streaming variant for the bench : emit one Step per
  //   request, replying with a stream of step results.  Saves a
  //   round-trip per step.  Optional -- controllers MAY use Step.
  rpc StepStream(stream StepRequest) returns (stream StepResponse);

  // 5. Capability query.  Returns the participant's dimension,
  //   contractive constant, RNG seed, build version, and the list
  //   of features it supports (e.g. `BLAS_BACKEND=mkl`).  Called
  //   once at controller startup + on every SIGHUP.
  rpc Capabilities(CapabilitiesRequest) returns (CapabilitiesResponse);

  // 6. Health + readiness.
  rpc Health(HealthRequest) returns (HealthResponse);
}
```

4.3 `messages.proto` (shared types)

```
syntax = "proto3";
package efe.v1.meta_swarm;

// A dense f32 tensor of dimension `dim`, row-major.  We use f32
// (not f64) on the wire to halve bandwidth -- the controller-side
// math runs in f32 in Phase-1, matching the bench scenario.
//
// Note (Phase 3) : we may add `repeated double dim_d` + a `precision`
// enum field for tenants that require f64.  Phase 2 stays f32-only.
message TensorF32 {
  uint32 dim = 1;
  repeated float data = 2 [packed = true];
}

// A meta-space configuration : dimension + symmetric box bounds.
// Other safety geometries (polyhedron, learned) are sent as
// `geometry_oneof` -- extensible.
message MetaSpaceConfig {
  uint32 dim = 1;
  oneof safety_geometry {
    BoxSafety box   = 2;
    Polyhedron poly = 3;
    LearnedHull hull = 4;
  }
}

message BoxSafety {
  float radius = 1;   // symmetric, ±radius per axis
}

message Polyhedron {
  // A·x &lt;= b ; rows packed as flat f32 dim_a x dim
  repeated float a_flat = 1 [packed = true];
  repeated float b      = 2 [packed = true];
  uint32 dim_a = 3;
}

message LearnedHull {
  bytes serialized_model = 1;  // bincode (Phase 2 placeholder)
  string version         = 2;
}

// A participant's static descriptor -- sent at Capabilities query.
message ParticipantDescriptor {
  string id              = 1;
  uint32 dim             = 2;
  float  kappa           = 3;
  uint64 seed            = 4;
  string build_sha       = 5;
  string blas_backend    = 6;   // "none" | "mkl" | "openblas" | "metal"
  repeated string features = 7;  // ["determinism", "metrics", "trace"]
}

// Iteration knobs -- mirrors `MetaSwarmBuilder` config.
message IterationConfig {
  uint32 max_iters       = 1;   // &lt;= 200
  double tolerance       = 2;   // joint residual
  double alpha_default   = 3;
  double coupling_gamma  = 4;
  bool   deterministic   = 5;   // bit-identical reruns guaranteed
  uint32 max_wall_ms     = 6;   // server caps at 30000
}

// Iterate request / response.
message IterateRequest {
  // Optional : if absent, controller uses its persisted manifest.
  repeated ParticipantDescriptor expected_participants = 1;
  MetaSpaceConfig meta = 2;
  IterationConfig iteration = 3;
  // Optional : if absent, participants use their last state
  // (or zero if just reset).
  repeated TensorF32 initial_states = 4;
  // Optional : tracing context (W3C TraceParent).
  string traceparent = 5;
}

message IterateResponse {
  uint32 iterations          = 1;
  double final_residual      = 2;
  TensorF32 meta_consensus   = 3;
```

```
  repeated TensorF32 final_states = 4;  // per participant, in order
  StopReason stop_reason     = 5;
  uint64 wall_us             = 6;
  // Trace id propagated back for cross-process correlation.
  string trace_id            = 7;
}

enum StopReason {
  STOP_REASON_UNSPECIFIED   = 0;
  STOP_REASON_CONVERGED     = 1;
  STOP_REASON_MAX_ITERS     = 2;
  STOP_REASON_SAFETY_FAIL   = 3;
  STOP_REASON_PARTICIPANT_DOWN = 4;
  STOP_REASON_DEADLINE      = 5;
  STOP_REASON_CONFIG_RELOAD = 6;
}

// Streaming chunk -- one inner step per chunk.
message IterateChunk {
  uint32 step          = 1;
  double residual      = 2;
  // Per-participant inner state norms -- useful for the bench HTML.
  repeated float participant_norms = 3 [packed = true];
}

// Step (participant-facing) request / response.
message StepRequest {
  TensorF32 pulled_from_meta = 1;
  // Step index for the participant -- used for deterministic seeding
  // of the participant's local RNG (Phase 3 sharded variance models).
  uint32 step_index = 2;
}

message StepResponse {
  TensorF32 new_state = 1;
  double    local_residual = 2;
  // Latency self-report for SLO-A1-META-PARTICIPANT-LATENCY-P99.
  uint64    step_us = 3;
}

// ReloadConfig -- server-side.
message ReloadConfigRequest {
  // If empty : the controller re-reads its on-disk manifest.
  // If non-empty : the new manifest is provided inline (admin RPC).
  optional MetaSpaceConfig new_meta = 1;
  repeated ParticipantDescriptor new_participants = 2;
  bool drain_inflight = 3;  // if true : finish current Iterate(),
                            // else : abort with CONFIG_RELOAD.
}

message ReloadConfigResponse {
  uint32 reload_gen      = 1;  // monotonically-increasing counter
  repeated string applied_changes = 2;
  uint64 reload_wall_us  = 3;
  bool   inflight_aborted = 4;
}

// Health probe.
message HealthRequest {}
message HealthResponse {
  enum Status {
    STATUS_UNSPECIFIED = 0;
    STATUS_SERVING     = 1;
    STATUS_NOT_SERVING = 2;
    STATUS_RELOADING   = 3;   // SIGHUP in flight
  }
  Status status       = 1;
  uint32 inflight     = 2;  // current concurrent Iterate calls
  uint32 reload_gen   = 3;
}

// Info -- for the dashboard.
message InfoRequest {}
message InfoResponse {
  string build_sha           = 1;
  string proto_version       = 2;  // "v1"
  repeated ParticipantDescriptor configured_participants = 3;
  string leader_pod_name     = 4;  // for T3 K8s topology
  uint64 uptime_s            = 5;
  uint32 last_reload_gen     = 6;
}
```

4.4 errors.proto

```
syntax = "proto3";
package efe.v1.meta_swarm;

// Phase-2 error codes.  Mapped to gRPC standard codes (FAILED_PRECONDITION,
// UNAVAILABLE, ...) at the transport layer ; the detailed code is the
// `error_code` field on the trailer.
enum MetaSwarmErrorCode {
  META_ERROR_UNSPECIFIED        = 0;
  // Configuration errors (FAILED_PRECONDITION).
  META_ERROR_DIM_MISMATCH       = 1;
  META_ERROR_DUPLICATE_ID       = 2;
  META_ERROR_INVALID_SAFETY     = 3;
  META_ERROR_KAPPA_OUT_OF_RANGE = 4;
  // Runtime errors (INTERNAL).
  META_ERROR_PARTICIPANT_DOWN   = 10;
  META_ERROR_DETERMINISM_BROKEN = 11;
  META_ERROR_SAFETY_VIOLATION   = 12;
  // Reload errors (FAILED_PRECONDITION).
  META_ERROR_RELOAD_BUSY        = 20;
  META_ERROR_RELOAD_DIM_CHANGED = 21;
}
```

4.5 Versioning & backwards compatibility

- Package name `efe.v1.meta_swarm` -- the `v1` suffix is the API

version, **frozen** at Phase 2 GA.

- Field numbers are reserved past `100` for Phase-3 additions.
- Phase 3 will add `efe.v2.meta_swarm` if a breaking change is

needed ; `v1` will be deprecated but not removed for at least 6 months.

- Wire format : proto3 with explicit `optional` keyword on fields

that may be absent (e.g. `new_meta` in `ReloadConfigRequest`).

4.6 Code generation

```
# Generated by build.rs of efe-meta-swarm-proto :
crates/efe-meta-swarm-proto/src/lib.rs             # tonic-built
crates/efe-meta-swarm-proto/src/efe.v1.meta_swarm.rs
```

Build script uses `tonic-build 0.11+` with the existing workspace `prost` version (pinned in workspace `[dependencies]`).

5. Rust API <-> gRPC mapping

The bijection between the Phase-1 in-process API and the Phase-2 network surface :

Phase 1 (in-process)	Phase 2 (gRPC)	Notes
`MetaSwarm::iterate() -> IterateResult`	`MetaSwarmService::Iterate(IterateRequest) -> IterateResponse`	Wall-time budget served by `max_wall_ms`.
`MetaSwarm::step_once() -> StepOnceResult`	`MetaSwarmService::StepOnce`	Hotpath optimisation for Phase 3.
`MetaSwarm::reset() -> ()`	`MetaSwarmService::Reset(ResetRequest)`	Reset propagates to all participants.
`SwarmParticipant::step_with_pull(pulled) -> Tensor`	`MetaParticipantService::Step(StepRequest) -> StepResponse`	The most-frequent RPC ; subject to streaming optimisation.
`SwarmParticipant::reset()`	`MetaParticipantService::Reset`	
`SwarmParticipant::current_state() -> Tensor`	`MetaParticipantService::State`	Debug / inspection.
`MetaSwarmBuilder::build()`	Server-side : controller builds from manifest on startup or SIGHUP.	Client never builds.
`MockParticipant::with_contraction(id, dim, kappa, seed)`	`MetaParticipantService::Capabilities -> ParticipantDescriptor`	Server announces.

Phase 1 (in-process)	Phase 2 (gRPC)	Notes
`BoxSafety::symmetric(dim, r)`	`MetaSpaceConfig.safety_geometry.box.radius`	Wire format.
(none -- Phase 2 only)	`MetaSwarmService::ReloadConfig`	New, Phase 2.
(none)	`MetaSwarmService::Health`, `Info`	New, Phase 2.

5.1 Client adapter

A new type `RemoteMetaSwarm` (in `efe-meta-swarm-server::client`) implements a `Drive`-like API matching `MetaSwarm` :

```
pub struct RemoteMetaSwarm {
    client: MetaSwarmServiceClient&amp;lt;tonic::transport::Channel&amp;gt;,
    config: IterationConfig,
}

impl RemoteMetaSwarm {
    pub async fn iterate(&amp;amp;self) -&amp;gt; Result&amp;lt;IterateResult, MetaSwarmError&amp;gt; {
        let resp = self.client
            .clone()
            .iterate(IterateRequest { /* ... */ })
            .await?;
        Ok(resp.into())
    }
}
```

The client side **does not** depend on `efe-meta-swarm` (the in-process crate) ; it only depends on `efe-meta-swarm-proto`. This keeps the client surface minimal and avoids accidentally pulling the contractive math into client binaries.

5.2 Server adapter

The server side wraps the Phase-1 `MetaSwarm` directly :

```
pub struct MetaSwarmGrpcServer {
    inner: Arc&amp;lt;RwLock&amp;lt;MetaSwarm&amp;gt;&amp;gt;,   // Phase-1 in-process state
    manifest_path: PathBuf,
    reload_gen: AtomicU32,
    metrics: MetricsRegistry,
}

#[tonic::async_trait]
impl MetaSwarmService for MetaSwarmGrpcServer {
    async fn iterate(
        &amp;amp;self,
        req: Request&amp;lt;IterateRequest&amp;gt;,
    ) -&amp;gt; Result&amp;lt;Response&amp;lt;IterateResponse&amp;gt;, Status&amp;gt; {
        // 1. Tracing : propagate W3C traceparent.
        let span = tracing::Span::current();
        if let Some(tp) = req.metadata().get("traceparent") { ... }
        // 2. Acquire read lock on the inner MetaSwarm.
        let inner = self.inner.read().await;
        // 3. Apply optional initial_states.
        if let Some(states) = req.into_inner().initial_states { ... }
        // 4. Run iterate() with deadline = max_wall_ms.
        let result = tokio::time::timeout(
            Duration::from_millis(req.iteration.max_wall_ms.into()),
            tokio::task::spawn_blocking({
                let inner = inner.clone();
                move || inner.iterate()
            }),
        ).await????;
        // 5. Convert + return.
        Ok(Response::new(result.into_proto()))
    }
    ...
}
```

Key points :

- `spawn_blocking` because Phase-1 `iterate()` is synchronous.

Phase-3 may move to an async core.

- `RwLock` because SIGHUP needs write access to swap the manifest ;

in-flight `iterate()` holds a read lock and is safe.

• Metrics are recorded at every step boundary (§ 9.1).

6. Multi-process orchestration

6.1 Process lifecycle

Each binary type follows the standard EFE pattern :

1. Parse CLI args + env vars.
2. Init tracing + metrics (Prometheus exporter on `:9090/metrics`).
3. Build inner state from manifest.
4. Bind to its gRPC port.
5. Install signal handlers : SIGTERM (graceful shutdown), SIGHUP

(reload), SIGUSR1 (dump debug info to stderr).

1. Enter the tonic server loop.
2. On SIGTERM : `drain_inflight=true`, refuse new RPCs, wait up to

30 s for in-flight to drain, then exit.

6.2 Discovery

T2 (single-node) uses static addresses :

```
EFE_META_SWARM_PARTICIPANTS=supplier-scoring=localhost:50091,contract-risk=localhost:50092,spend-categori ■
    zation=localhost:50093
```

T3 (K8s) uses DNS :

```
EFE_META_SWARM_DISCOVERY=k8s-dns
EFE_META_SWARM_PARTICIPANT_SERVICES=participant-supplier-scoring.efe.svc.cluster.local:50090,...
```

The discovery layer is a `trait Discovery` with two impls : `StaticDiscovery` and `K8sDnsDiscovery` (no client-go ; pure DNS lookup, refreshed every 10 s).

6.3 Failure modes (Phase 2 acceptable)

Failure	Behaviour	Recovery
Participant pod crashes mid-iterate	`Iterate` returns `STOP_REASON_PARTICIPANT_DOWN` ; controller increments `meta_swarm_iterate_failures_total{reason="participant_down"}`.	K8s restarts pod ; next `Iterate` succeeds once the pod is `Ready`.
Controller pod crashes	New leader elected via K8s lease ; in-flight `Iterate` calls return UNAVAILABLE.	Client retries (`efe-server` has standard exponential backoff).
Network partition (controller <-> participant)	gRPC keepalive (every 10 s) detects within 30 s ; controller marks participant DOWN.	Auto-rejoin on partition heal.
SIGHUP during iterate	`drain_inflight=true` (default) waits up to 30 s ; `=false` aborts with CONFIG_RELOAD.	Either way, no partial results leak.
Determinism broken (residual.to_bits diverges)	`Iterate` returns `META_ERROR_DETERMINISM_BROKEN` ; controller logs at WARN ; alert fires.	Manual investigation.

6.4 Drain protocol

1. Controller receives SIGTERM.
2. Health endpoint flips to `NOT_SERVING`.
3. K8s `terminationGracePeriodSeconds: 60` -> `preStop` hook adds

a 10 s drain delay so the load balancer dedups.

1. New `Iterate` calls return UNAVAILABLE immediately.
2. Existing in-flight calls complete normally (<= 30 s budget).
3. Process exits with code 0.

7. Configuration : env vars + SIGHUP hot-reload

7.1 Environment variables

```
# === Core toggle ===
EFE_META_SWARM_ENABLED=false                # default ; true to opt in

# === Discovery ===
EFE_META_SWARM_DISCOVERY=static-localhost    # | k8s-dns
EFE_META_SWARM_PARTICIPANTS=...                # static : &lt;id&gt;=&lt;host:port&gt;,...
EFE_META_SWARM_PARTICIPANT_SERVICES=...        # k8s-dns : full service FQDNs

# === Controller binding ===
EFE_META_SWARM_CONTROLLER_ADDR=0.0.0.0:50080 # the controller's gRPC
EFE_META_SWARM_CONTROLLER_METRICS=:9090      # Prometheus scrape addr

# === Participant binding (only on participant pods) ===
EFE_META_SWARM_PARTICIPANT_ID=supplier-scoring  # which participant am I
EFE_META_SWARM_PARTICIPANT_ADDR=0.0.0.0:50090
EFE_META_SWARM_PARTICIPANT_METRICS=:9091

# === Manifest ===
EFE_META_SWARM_MANIFEST=/etc/efe-meta-swarm/manifest.yaml

# === TLS / auth ===
EFE_META_SWARM_TLS_CERT=/etc/efe/tls/meta-swarm.crt
EFE_META_SWARM_TLS_KEY=/etc/efe/tls/meta-swarm.key
EFE_META_SWARM_TLS_CA=/etc/efe/tls/ca.crt
EFE_META_SWARM_AUTH_TOKEN=...                  # static bearer (Phase-3 mTLS-only)

# === Hot-reload ===
EFE_META_SWARM_RELOAD_DRAIN_TIMEOUT_S=30     # cf. § 6.4
EFE_META_SWARM_RELOAD_ABORT_INFLIGHT=false   # default : drain

# === Limits ===
EFE_META_SWARM_MAX_INFLIGHT=10               # concurrent Iterate cap
EFE_META_SWARM_DEFAULT_MAX_WALL_MS=15000     # if client omits

# === Observability ===
EFE_META_SWARM_OTEL_ENDPOINT=http://otel:4317
EFE_META_SWARM_TRACE_SAMPLE_RATIO=0.05       # 5 % default

# === Carryover Phase 1 ===
EFE_META_SWARM_DETERMINISTIC=true            # SLO-A1-META-DETERMINISM
EFE_META_SWARM_LAST_P99_OBSERVED_MS=...        # consumed by NON-REGRESSION
```

7.2 SIGHUP hot-reload semantics

The reload protocol :

```
1. Operator kills -HUP &lt;controller_pid&gt;     (or `kubectl rollout restart` per pod).
2. Signal handler triggers ReloadConfig RPC (loopback or in-proc).
3. Controller :
   a. Increment reload_gen.
   b. Acquire write lock on inner MetaSwarm.
   c. Wait up to `drain_timeout_s` for in-flight Iterate to finish
      (if drain_inflight=true) OR abort them (if =false).
   d. Re-read manifest from EFE_META_SWARM_MANIFEST.
   e. Diff new manifest vs current :
      - Dimension change ?    -&gt; REJECT (META_ERROR_RELOAD_DIM_CHANGED).
      - Participant id added  -&gt; call Capabilities on new participant,
                                 fail-closed if it cannot handshake.
      - Participant id removed -&gt; wait for in-flight, then drop.
      - Safety geometry changed -&gt; swap atomically.
      - kappa / seed changed  -&gt; applied at next Iterate.
   f. Release write lock.
   g. Emit metric `meta_swarm_reload_total{outcome="ok|fail"}`.
4. Health endpoint flips to RELOADING during step c-f.
5. Return diff to caller in ReloadConfigResponse.applied_changes.
```

Invariant : either the whole reload commits, or none of it does (no partial state). Implementation : build the new MetaSwarm in a temp variable, swap atomically inside the write lock, drop the old one outside.

7.3 SLO `SLO-A1-META-RELOAD-P99`

```
- id: SLO-A1-META-RELOAD-P99
  description: "p99 wall time for SIGHUP-triggered ReloadConfig on a 7-participant manifest"
  target: 200       # ms
  operator: "&lt;="
  comparator: lte
  threshold_ms: 200
  severity: major
  owner: meta-swarm-team
  deferred_ticket: DT-A1-PHASE2-SOFT-LAUNCH-3-RUNS
```

Phase-2 PR adds this row to `bench_baselines/v0.11.0/meta_swarm_slos.yaml` (new minor version of the baseline) + ports it to the master rollup `SLO-OPS-A1-META-SWARM.inherits`.

8. Helm sub-chart `efe-meta-swarm`

8.1 Directory structure

```
charts/efe/                             # existing umbrella
+-- charts/                             # sub-charts
    +-- efe-server/                     # existing
    +-- efe-meta-swarm/                 # NEW (Phase 2)
        +-- Chart.yaml
        +-- values.yaml
        +-- values.schema.json
        +-- templates/
        |   +-- _helpers.tpl
        |   +-- controller-statefulset.yaml
        |   +-- controller-service.yaml
        |   +-- controller-pdb.yaml
        |   +-- controller-servicemonitor.yaml
        |   +-- participant-deployment.yaml      (template per participant)
        |   +-- participant-service.yaml
        |   +-- participant-servicemonitor.yaml
        |   +-- secret-tls.yaml
        |   +-- configmap-manifest.yaml
        |   +-- tests/
        |       +-- connection-test.yaml         (`helm test`)
        +-- README.md
```

8.2 `values.yaml` (default-disabled)

```
# === values.yaml ===
metaSwarm:
  # FLAGSHIP DEFAULT : false. Production parity stays at Phase 1.
  enabled: false

  # Container image -- shared by controller + participants.
  image:
    repository: efe/meta-swarm-server
    tag: "0.2.0"          # Phase-2 GA
    pullPolicy: IfNotPresent

  # === Controller ===
  controller:
    replicaCount: 3
    resources:
      requests:
        cpu: 500m
        memory: 512Mi
      limits:
        cpu: 2000m
        memory: 2Gi
    podDisruptionBudget:
      minAvailable: 2
    service:
      type: ClusterIP
      port: 50080
    metrics:
      port: 9090
      enabled: true

  # === Participants ===
  participants:
    - id: supplier-scoring
      dim: 384
      kappa: 0.6
      seed: 11
      replicas: 1
      resources:
        requests: { cpu: 250m, memory: 256Mi }
        limits:   { cpu: 1000m, memory: 1Gi }
    - id: contract-risk
      dim: 512
      kappa: 0.7
      seed: 22
      replicas: 1
      resources:
        requests: { cpu: 250m, memory: 256Mi }
        limits:   { cpu: 1000m, memory: 1Gi }
    - id: spend-categorization
      dim: 384
      kappa: 0.5
      seed: 33
      replicas: 1
      resources:
        requests: { cpu: 250m, memory: 256Mi }
        limits:   { cpu: 1000m, memory: 1Gi }

  # === Meta-space ===
  metaSpace:
    dim: 256
    safety:
      kind: box
      radius: 10.0

  # === Iteration knobs ===
  iteration:
    maxIters: 100
    tolerance: 1.0e-4
    alphaDefault: 0.5
    couplingGamma: 0.2
    deterministic: true

  # === TLS ===
  tls:
    enabled: true
    certSecretName: efe-meta-swarm-tls   # opaque secret w/ tls.crt, tls.key, ca.crt

  # === Observability ===
  observability:
    serviceMonitor:
```

```
    enabled: true                # Prometheus Operator integration
    interval: 30s
  otel:
    enabled: true
    endpoint: http://otel-collector.observability.svc:4317
    traceSampleRatio: 0.05

# === Soft-launch (Phase 2 first 3 runs in prod) ===
softLaunch:
  enabled: true
  expiresAt: "2026-09-15"      # auto-flips to false after this date
```

8.3 Default-disabled enforcement

The umbrella `charts/efe/values.yaml` does **not** override `metaSwarm.enabled`. Thus :

```
helm install efe ./charts/efe   # metaSwarm.enabled = false =&gt; no MS resources
helm install efe ./charts/efe --set metaSwarm.enabled=true  # opt-in
```

A `helm template` of the chart with default values produces a manifest set strictly identical to Phase 1 deployments -- verified by the new CI job `helm-template-diff-phase1-vs-phase2-defaults` (§ 19.3).

8.4 `helm test` smoke

A pod `efe-meta-swarm-connection-test` is shipped that, when `metaSwarm.enabled=true`, calls `MetaSwarmService::Health` and `MetaSwarmService::Info` against the controller Service and asserts both return `STATUS_SERVING` within 10 s.

9. Observability : Prometheus + OpenTelemetry

9.1 Prometheus metrics (14)

#	Metric name	Type	Labels	Description
1	`meta_swarm_iterate_total`	counter	`outcome={converged,max_iters,safety_fail,participant_down,deadline,reload}`	Number of `Iterate` RPCs by outcome.
2	`meta_swarm_iterate_duration_seconds`	histogram	(none)	Wall time of `Iterate` ; buckets `[0.001, 0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2, 5, 10, 30]`.
3	`meta_swarm_iterate_iterations`	histogram	(none)	Inner-step count per `Iterate` ; buckets `[1, 5, 10, 20, 30, 50, 80, 120, 200]`.
4	`meta_swarm_iterate_final_residual`	summary	(none)	Final residual at convergence (median + p99).
5	`meta_swarm_participant_step_total`	counter	`participant_id`, `outcome={ok,fail}`	Step RPCs per participant.
6	`meta_swarm_participant_step_duration_seconds`	histogram	`participant_id`	Per-participant Step latency.
7	`meta_swarm_participant_up`	gauge	`participant_id`	1 = controller can reach participant, 0 = DOWN.
8	`meta_swarm_inflight`	gauge	(none)	Current concurrent `Iterate` calls.
9	`meta_swarm_reload_total`	counter	`outcome={ok,fail,rejected}`	SIGHUP reloads.
10	`meta_swarm_reload_duration_seconds`	histogram	(none)	Reload wall time.
11	`meta_swarm_reload_gen`	gauge	(none)	Current reload generation.
12	`meta_swarm_safety_violation_total`	counter	`level={warn,error}`, `axis`	Safety projections that hit the boundary.
13	`meta_swarm_build_info`	gauge	`git_sha`, `proto_version`, `rustc_version`	Always 1 ; reports the build fingerprint.
14	`meta_swarm_consensus_norm`	summary	(none)	Norm of the meta-consensus tensor at convergence.

All metrics are scraped via the existing ServiceMonitor pattern. A single Grafana dashboard `efe-meta-swarm.json` (under `grafana/dashboards/`) renders them.

9.2 OpenTelemetry spans (8 types)

#	Span name	Attributes	When emitted
1	`meta_swarm.iterate`	`iterations`, `final_residual`, `stop_reason`, `participant_count`	One per `Iterate` RPC ; root of the trace for that request.
2	`meta_swarm.step`	`step_index`, `residual_after_step`	One per inner step inside `iterate`.
3	`meta_swarm.participant_step`	`participant_id`, `step_us`, `local_residual`	One per participant per step (child of `meta_swarm.step`).
4	`meta_swarm.lift`	`participant_id`, `pre_dim`, `meta_dim`	Per-step lift call.
5	`meta_swarm.pull`	`participant_id`, `pre_dim`, `meta_dim`	Per-step pull call.
6	`meta_swarm.meta_op`	`dim`, `kind`	Per-step meta-operator application.
7	`meta_swarm.safety_project`	`level`, `distance`	Only when projection is non-trivial (axis hit).
8	`meta_swarm.reload`	`reload_gen`, `applied_changes_count`	One per SIGHUP.

Trace context : controller propagates `traceparent` to participants via the gRPC metadata. Sampling : 5 % default (`traceSampleRatio`), configurable.

9.3 Dashboards (2)

Dashboard 1 -- Operational (`grafana/dashboards/efe-meta-swarm-ops.json`) :

- Panel: `iterate_total` rate by outcome (stacked area)
- Panel: `iterate_duration_seconds` p50 / p95 / p99 (time series)
- Panel: `inflight` (gauge)
- Panel: `participant_up` per participant (table, red/green)
- Panel: `safety_violation_total` rate (heatmap by axis)
- Panel: `reload_total` count + last reload time

Dashboard 2 -- Algorithmic (`grafana/dashboards/efe-meta-swarm-algo.json`) :

- Panel: `iterate_iterations` histogram + budget line (60)
- Panel: `iterate_final_residual` median + p99 (time series)
- Panel: `consensus_norm` median + p99
- Panel: `participant_step_duration_seconds` per participant
- Panel: residual-trajectory chart (uses the `IterateStream` data from

a sampled call every 1 min)

9.4 Alerting rules (4 PrometheusRule)

Rule	Expression	Severity
`MetaSwarmHighFailureRate`	`sum(rate(meta_swarm_iterate_total{outcome!="converged"}[5m])) / sum(rate(meta_swarm_iterate_total[5m])) > 0.02`	critical
`MetaSwarmParticipantDown`	`meta_swarm_participant_up == 0 for 1m`	critical
`MetaSwarmReloadFail`	`increase(meta_swarm_reload_total{outcome="fail"}[10m]) > 0`	major
`MetaSwarmLatencyP99`	`histogram_quantile(0.99, sum by (le)(rate(meta_swarm_iterate_duration_seconds_bucket[5m]))) > 0.1`	major

All routed via existing Alertmanager -> PagerDuty integration.

10. Internal 7-day soak

10.1 Setup

• Environment : `efe-staging` cluster, 1 controller StatefulSet (3 replicas) + 3 participant Deployments.

• Traffic : an `efe-meta-swarm-loadgen` Job (new, in `tools/loadgen/`) issues `Iterate` RPCs at 30 req/s sustained, 100 req/s burst.

• Workload mix :

• 60 % standard 3-swarm Kovant scenario (default manifest)

• 30 % varied initial states (random Tensor::pseudo_random seeds)

• 10 % SIGHUP-driven manifest swaps (every 30 min)

• Duration : 168 h (7 days) starting Monday 00:00 UTC.

• Observers : EFE-team only ; no external traffic ; no customer-facing endpoint.

10.2 Pass criteria (must all be true)

#	Criterion	Threshold
S1	Zero Sev1 incidents	0
S2	Zero Sev2 incidents	0
S3	<= 1 Sev3 incident	<= 1
S4	Iterate convergence rate	>= 99.95 %
S5	Master gate stays GREEN every nightly run	7/7 nightly PASS
S6	p99 latency	<= 100 ms throughout
S7	Memory leak (RSS growth)	<= 5 % over 7 days
S8	SIGHUP success rate	>= 99.9 %
S9	Determinism preserved (sampled every 1 h)	100 %
S10	OTel trace coverage	>= 5 % of Iterate calls have a complete trace

A single failure of any criterion **resets the 7-day clock**. This is intentionally strict -- the L2 gate is the L1 of customer trust.

10.3 Soak report

At day 7, the soak driver emits `docs/A1_PHASE2_SOAK_REPORT.md` with :

- Daily summary (convergence rate, p99, incidents)
- Reload event log (one row per SIGHUP)
- Determinism sample log
- Memory time series
- Open follow-up items (deferred to Phase 3 if needed)

11. Public bench : meta-swarm vs mono-swarm

11.1 Methodology

Five workloads, run on identical hardware (4-vCPU / 16 GB Ubuntu 22.04 GCP n2-standard-4) :

#	Workload	Description
W1	Kovant-3 (Phase-1 scenario)	supplier-scoring(384) + contract-risk(512) + spend-categorization(384) -> meta(256).
W2	Kovant-3 with 2x larger dims	768 + 1024 + 768 -> 512. Stress the bandwidth side.
W3	7-participant scenario	7 swarms of dim 256, meta 128. Tests scaling to more participants.
W4	Single-swarm baseline	The legacy mono-swarm `efe-server` path : no meta-coordination. Both meta-swarm and mono-swarm produce the same `accuracy_top1` score on a held-out golden set ; we compare latency.
W5	Slow-participant scenario	One participant has artificial 50 ms latency injected per step. Measures meta-swarm graceful degradation.

For each workload, three measurement modes :

- **Mono-swarm** (`efe-server` only, Phase-1 path).
- **Meta-swarm in-process** (`efe-meta-swarm` linked, no gRPC).
- **Meta-swarm gRPC** (Phase-2 multi-process T2 topology).

11.2 Metrics published

Metric	Mono-swarm	MS in-proc	MS gRPC	Unit
Convergence iterations	N/A	47	47	count
Final residual	N/A	9.7e-5	9.7e-5	scalar
Wall p50	TBD	TBD	TBD	ms
Wall p99	TBD	TBD	TBD	ms
Throughput (req/s, 4 vCPU)	TBD	TBD	TBD	req/s
Memory peak	TBD	TBD	TBD	MB
Accuracy top-1 (on golden set)	TBD	TBD	TBD	%

The values are **measured live during the run**, not estimated. The CSV + HTML are published under `bench_results/meta-swarm-vs-mono-swarm-2026-08.{csv,html}` and linked from the L2 graduation issue.

11.3 Publication format

- CSV : the matrix above, machine-readable.
- HTML : an interactive page with toggles, residual-trajectory plots,

hardware fingerprint, reproducibility recipe (`bash bench_runner.sh --public --workload W1`).

- All raw criterion JSON included in `bench_results/raw/`.

11.4 Honest framing

The bench description prominently states :

> *Meta-swarm is **not** a strict replacement for mono-swarm. It adds coordination guarantees (deterministic consensus, contractive fixed point, audit-able safety projection) that mono-swarm cannot provide. The latency overhead reflects this added work. We publish all data so customers can decide based on their requirements.*

No cherry-picking ; no marketing. All five workloads are reported even if mono-swarm wins.

12. Gate 2 -> 3 (L2 attained) criteria

L2 is attained iff **all** of the following are true on 2026-08-31 :

#	Criterion	Owner	Evidence
G1	The 7 Phase-1 surface A1 SLOs are GREEN on the master gate, 7/7 of the last 7 nightly runs.	meta-swarm-team	`target/ops/master_report.md` archives.
G2	The 4 Phase-2 surface SLOs (§ 14) are GREEN on the master gate, 7/7 of the last 7 nightly runs.	meta-swarm-team	id.
G3	Master rollup `SLO-OPS-A1-META-SWARM` (#19) is PASS on the last 7 nightly runs.	meta-swarm-team	id.
G4	Master rollup `SLO-OPS-A1-META-GRPC` (#20, added Phase 2) is PASS on the last 7 nightly runs.	meta-swarm-team	id.
G5	7-day internal soak passes all 10 criteria (§ 10.2).	sre-team	`A1_PHASE2_SOAK_REPORT.md`.
G6	Public bench published with all 5 workloads x 3 modes.	meta-swarm-team	`bench_results/meta-swarm-vs-mono-swarm-2026-08.{csv,html}`.
G7	Helm sub-chart Lint + `helm template` regression test pass.	platform-team	CI job `helm-meta-swarm-lint`.
G8	Security review : threat model document approved by `sec-team`.	sec-team	`docs/security/PHASE2_THREAT_MODEL.md` signed off.
G9	Runbook `docs/runbooks/META_SWARM_GRPC.md` written + 2 SRE rehearsals (1 SIGHUP drill, 1 participant-down drill).	sre-team	Runbook + rehearsal logs in `docs/runbooks/drills/`.
G10	Postmortem template `docs/postmortems/_template.md` extended with a Meta-Swarm section.	sre-team	Template diff.

If any criterion is RED, L2 graduation is **deferred** by one sprint (2 weeks) and re-evaluated. No "soft pass" -- Φ-D.A1 doctrine applies.

13. Code structure additions

13.1 New crates

```
crates/
+-- efe-meta-swarm/                  # Phase-1 (existing, unchanged)
+-- efe-meta-swarm-proto/            # NEW -- proto + tonic-generated client/server stubs
|   +-- Cargo.toml
|   +-- build.rs
|   +-- proto/
|   |   +-- efe/v1/meta_swarm/
|   |       +-- service_meta_swarm.proto
|   |       +-- service_meta_participant.proto
|   |       +-- messages.proto
|   |       +-- errors.proto
|   +-- src/lib.rs                   # re-exports tonic-generated code
|
+-- efe-meta-swarm-server/           # NEW -- binary + library
    +-- Cargo.toml
    +-- src/
    |   +-- lib.rs
    |   +-- main.rs                  # entry-point -- choose controller vs participant role via CLI
    |   +-- bin/
    |   |   +-- controller.rs
    |   |   +-- participant.rs
    |   +-- controller/
    |   |   +-- mod.rs
    |   |   +-- service.rs           # impl MetaSwarmService
    |   |   +-- manifest.rs          # YAML manifest reader
    |   |   +-- discovery.rs         # trait + impls (static, k8s-dns)
    |   |   +-- reload.rs            # SIGHUP handler
    |   +-- participant/
    |   |   +-- mod.rs
    |   |   +-- service.rs           # impl MetaParticipantService
    |   |   +-- adapter.rs           # wraps Phase-1 MockParticipant / LocalParticipant
    |   +-- client/
    |   |   +-- mod.rs
    |   |   +-- remote_meta_swarm.rs  # RemoteMetaSwarm -- gRPC client adapter
    |   +-- observability/
    |   |   +-- metrics.rs           # Prometheus registry
    |   |   +-- tracing.rs           # OTel setup
    |   |   +-- health.rs
    |   +-- auth/
    |   |   +-- mtls.rs
    |   |   +-- bearer.rs
    |   +-- error.rs
    +-- tests/
    |   +-- integration_grpc_multiprocess.rs
    |   +-- integration_sighup_hot_reload.rs
    |   +-- integration_failure_modes.rs
    |   +-- integration_obs_phase2.rs
    +-- benches/
        +-- grpc_round_trip.rs       # roundtrip latency bench
```

13.2 Workspace `Cargo.toml`

Add to `[workspace.members]`:

```
"crates/efe-meta-swarm-proto",
"crates/efe-meta-swarm-server",
```

Add to `[workspace.dependencies]`:

```
# Phase-2 additions (only used by efe-meta-swarm-server*)
tonic            = { version = "0.12", features = ["tls", "gzip"] }
tonic-build      = "0.12"
prost            = "0.13"
prost-types      = "0.13"
opentelemetry    = { version = "0.24", features = ["metrics", "trace"] }
opentelemetry-otlp = "0.17"
prometheus       = "0.13"
signal-hook      = "0.3"
signal-hook-tokio = "0.3"
```

`efe-meta-swarm-server/Cargo.toml` overrides `panic = "unwind"` (see C7 in § 2).

13.3 Modifications to existing files

File	Change
`Cargo.toml` (workspace)	+2 members, +9 deps.
`crates/efe-server/Cargo.toml`	+1 optional dep `efe-meta-swarm-proto` behind feature `meta-swarm-grpc`.
`crates/efe-server/src/bridge/mod.rs`	New module `meta_swarm.rs` that dispatches `iterate()` to the gRPC client when `EFE_META_SWARM_ENABLED=true`.
`bench_baselines/v0.10.0/meta_swarm_slos.yaml`	Either bumped to `v0.11.0` (recommended) with +4 new Phase-2 SLOs, or kept as-is and supplemented by `v0.11.0/meta_swarm_grpc_slos.yaml`.
`bench_baselines/v0.9.0/ops_master_slos.yaml`	+1 master rollup `SLO-OPS-A1-META-GRPC`.
`.github/workflows/bench.yml`	+1 job `bench-meta-swarm-grpc` (Phase-2 specific).
`.github/workflows/ops-master-gate.yml`	+1 step "A2 meta-swarm gRPC master gate".
`scripts/ops/run_suite.sh`	+1 case `a2_meta_swarm_grpc`.

No file currently in the project is **deleted** or has a signature changed.

14. SLO budget additions (Phase 2)

Four new surface SLOs, frozen at `bench_baselines/v0.11.0/meta_swarm_grpc_slos.yaml` :

SLO id	Type	Comparator	Threshold	Source
`SLO-A2-GRPC-OVERHEAD-P99`	latency	`<=`	30 ms	gRPC round-trip overhead = Phase-2 p99 − Phase-1 p99
`SLO-A2-GRPC-MULTIPROC-DETERMINISM`	algorithmic	`==`	1	3 reruns across 3 processes produce bit-identical residual.to_bits
`SLO-A2-RELOAD-P99`	operational	`<=`	200 ms	SIGHUP-triggered ReloadConfig wall time
`SLO-A2-TRACE-COVERAGE`	observability	`>=`	0.05	Fraction of `Iterate` RPCs with a complete OTel trace

New master rollup #20 :

```
- id:          SLO-OPS-A1-META-GRPC
  name:        Sprint A1 Phase 2 gRPC transport gate
  description: |
    Rolls up the 4 `SLO-A2-*` surface SLOs produced by the
    `bench-meta-swarm-grpc` job and the `a2_meta_swarm_grpc` suite
    runner.  Activated in PR #5 (Sprint A1 P2, week N).
  metric:      efe_meta_swarm_a2_gate_status
  target:      1
  operator:    "=="
  severity:    major
  status:      ACTIVE
  inherits:
    - SLO-A2-GRPC-OVERHEAD-P99
    - SLO-A2-GRPC-MULTIPROC-DETERMINISM
    - SLO-A2-RELOAD-P99
    - SLO-A2-TRACE-COVERAGE
  blast_radius: A1-PHASE2-OPT-IN-TENANTS
```

Smoke test 9 added to `tests/bench_smoke.rs` to pin its existence (same pattern as P5 audit fix from Phase 1).

15. Security model

15.1 Trust boundaries

```
          ▲ External (untrusted)
          |
    +------+----------+
    | ingress / LB    |
    +------+----------+
          | TLS 1.3
    +------▼----------+
    | efe-server pod  | &lt;- public auth (existing : OAuth2 / JWT)
    +------+----------+
          | mTLS (NEW Phase 2)   &lt;- internal trust boundary
    +------▼-------------+
    | meta-swarm-controller|
    +------+-------------+
          | mTLS
    +------▼----------+
    | participant pods |
    +-----------------+
```

- **mTLS mandatory** between efe-server <-> controller and controller <-> participant. No plaintext gRPC in any environment.
- Certificates : issued by the existing internal CA (cert-manager), rotated every 30 days, leaf cert validity 90 days.
- Bearer token (`EFE_META_SWARM_AUTH_TOKEN`) is a **transitional** auth method -- used only when mTLS bootstrap fails, removed in Phase 3.

15.2 Authorization

- The controller authorises callers by the SAN field of their leaf cert : `efe-server.efe.svc.cluster.local` -> MetaSwarmService full access ; anything else -> DENY.
- Participants authorise callers by SAN : only the controller may call.
- Operator RPC `ReloadConfig` requires an additional `admin` scope on the auth token.

15.3 Threat model summary

Threat	Mitigation
Malicious participant returns crafted state to bias consensus	Safety projection (Phase 1) ; the meta-operator is contractive -- bias is bounded by `(1-κ_max) x max_step_size`. Phase-3 will add cryptographic attestation of step outputs.
Replay of `Iterate` response	gRPC's per-call deadlines + the `Iterate` is idempotent w.r.t. determinism.
MITM on controller <-> participant	mTLS.
SIGHUP injection by unauthorized operator	SIGHUP requires admin scope + the kernel enforces PID-level permission. K8s RBAC restricts pod exec.
Manifest tampering (ConfigMap)	K8s RBAC + sealed-secrets for the participant seeds.

Full threat model lives in `docs/security/PHASE2_THREAT_MODEL.md` (deliverable G8).

16. Migration path Phase 1 -> Phase 2

16.1 Rollout strategy

Phased, gated, reversible :

1. **Week 1 of Aug 2026** : staging deployment with `metaSwarm.enabled=false`. No behavioural change vs Phase 1.

1. **Week 2** : staging `--set metaSwarm.enabled=true`. Internal

shadow traffic only. Compare outputs to in-process Phase 1 on the same requests.

1. **Week 3** : production canary on 10 % of replicas with the

flag on ; remaining 90 % stay Phase 1.

1. **Week 4** : if canary clean for 7 days -> 50 % roll-out. Then

100 % the following week if still clean.

16.2 Rollback

Single env-var flip :

```
kubectl set env deployment/efe-server EFE_META_SWARM_ENABLED=false
```

The `efe-server` falls back to the in-process Phase-1 path immediately. Meta-swarm pods can be left running idle (no traffic) or scaled to zero. No data corruption is possible because Phase 2 is a transport-layer change only -- the algorithmic surface is identical.

16.3 Compatibility matrix

Client	Server	Outcome
efe-server Phase 1	(no MS)	In-process, Phase-1 unchanged.
efe-server Phase 2 with flag OFF	(no MS)	In-process, Phase-1 path.
efe-server Phase 2 with flag ON	MS controller v0.2.0	gRPC, Phase-2 happy path.
efe-server Phase 2 with flag ON	MS controller v0.1.x	DENIED at `Info` RPC (proto_version mismatch).
efe-server Phase 1	MS controller v0.2.0	Never happens -- the flag is gated by Phase-2 efe-server.

17. Risk matrix + mitigations

ID	Risk	Likelihood	Impact	Mitigation
R1	gRPC overhead pushes p99 over the 100 ms budget	M	H	Streaming `StepStream` (§ 4.2) saves a round-trip per step ; if still too slow, bump budget to 150 ms with explicit doc note.
R2	SIGHUP race condition corrupts state	L	H	Atomic swap inside write-lock (§ 7.2) ; integration test `tests/integration_sighup_hot_reload.rs` runs 1000 random-timed SIGHUPs concurrently with Iterate.
R3	Cross-process determinism breaks (different rustc, BLAS, etc.)	M	H	Pin rustc 1.88 in Cargo + container ; `blas_backend: "none"` for Phase 2 ; determinism check in soak (S9).
R4	Helm sub-chart breaks existing efe-server deployments	L	H	`enabled=false` default + CI job `helm-template-diff-phase1-vs-phase2-defaults` verifies byte-identical output.
R5	mTLS cert rotation fails in production	M	M	cert-manager auto-renewal at 60 days ; alert `MetaSwarmCertExpiringSoon` fires at 7 days.
R6	OTel cardinality explodes Grafana	L	M	Bounded labels (no free-form `participant_id` -- fixed set of 3) ; OTel sampling 5 %.
R7	One participant becomes the bottleneck (stragglers)	M	M	W5 bench measures this ; if real, Phase 3 will introduce per-participant SLO + auto-scale.
R8	Public bench shows mono-swarm wins on W4	M	L	Acceptable -- bench is honest, decision is per-customer. Marketing materials state this clearly.
R9	L2 gate fails 2026-08-31	L	H	Each criterion is independently met or unmet ; failing G5 only triggers a 2-week deferral, not a Phase-2 cancellation.

18. Detailed timeline

Sprint A1.P2.1 -- 2026-07-06 -> 2026-07-19 (S1-S2 July)

Goal : land the protos + the bare-bones servers.

Week 1 (S1) :

- [ ] PR #5 scaffold `efe-meta-swarm-proto` (protos + tonic-build) [meta-swarm-team]
- [ ] PR #6 scaffold `efe-meta-swarm-server` (controller + participant binaries, no logic yet) [meta-swarm-team]

- [ ] PR #7 threat model doc draft [sec-team]

Week 2 (S2) :

- [ ] PR #8 controller-side `MetaSwarmService::Iterate` wiring through Phase-1 `MetaSwarm` [meta-swarm-team]
- [ ] PR #9 participant-side `MetaParticipantService::Step` wiring [meta-swarm-team]
- [ ] PR #10 integration test `tests/integration_grpc_multiprocess.rs` [meta-swarm-team]

End-of-sprint demo : `cargo run --bin controller` + 3x `cargo run --bin participant` on dev box, Iterate succeeds with bit-identical Phase-1 result.

Sprint A1.P2.2 -- 2026-07-20 -> 2026-08-02 (S3-S4 July)

Goal : env-var + SIGHUP + Helm scaffold.

Week 3 (S3) :

- [ ] PR #11 `EFE_META_SWARM_ENABLED` plumbing in efe-server [meta-swarm-team]
- [ ] PR #12 SIGHUP hot-reload + `ReloadConfig` RPC [meta-swarm-team]
- [ ] PR #13 Helm sub-chart skeleton `charts/efe/charts/efe-meta-swarm/` [platform-team]

Week 4 (S4) :

- [ ] PR #14 Helm sub-chart values + templates (controller + participants) [platform-team]
- [ ] PR #15 `helm template` regression test (default-disabled produces same manifest as Phase 1) [platform-team]
- [ ] PR #16 mTLS wiring + cert-manager Issuer/Certificate [sec-team + platform-team]

End-of-sprint demo : `helm install` on staging with flag flipped on/off ; SIGHUP rolls participant in/out.

Sprint A1.P2.3 -- 2026-08-03 -> 2026-08-16 (S1-S2 August)

Goal : observability + start of soak.

Week 5 (S1) :

- [ ] PR #17 Prometheus metrics (14) [meta-swarm-team]
- [ ] PR #18 OpenTelemetry spans (8) [meta-swarm-team]
- [ ] PR #19 Grafana dashboards + alert rules [sre-team]

Week 6 (S2) :

- [ ] PR #20 `tools/loadgen` traffic generator [meta-swarm-team]
- [ ] Begin 7-day soak (Mon 2026-08-10 00:00 UTC) [sre-team]
- [ ] PR #21 Public bench harness + W1-W5 workloads [meta-swarm-team]

End-of-sprint demo : Grafana dashboards live ; soak day 7 report posted.

Sprint A1.P2.4 -- 2026-08-17 -> 2026-08-30 (S3-S4 August)

Goal : L2 graduation.

Week 7 (S3) :

- [ ] PR #22 4 new SLOs + master rollup `SLO-OPS-A1-META-GRPC` [meta-swarm-team]
- [ ] PR #23 Smoke test 9 (rollup #20 pin) [meta-swarm-team]
- [ ] PR #24 Public bench publication (CSV + HTML) [meta-swarm-team]
- [ ] Runbook + 2 SRE rehearsals [sre-team]

Week 8 (S4) :

- [ ] L2 graduation issue : verify G1-G10 (§ 12) [meta-swarm-team + sre-team]
- [ ] Final master gate run + sign-off [sre-team]

- [ ] PR #25 Phase 2 end-of-sprint progress report [meta-swarm-team]
- [ ] **Gate 2 -> 3 [OK]** (target : 2026-08-31)

19. CI / CD pipeline additions

19.1 New `bench.yml` job

```
bench-meta-swarm-grpc:
  name: Run grpc_round_trip + multi-proc integration (Sprint A1 P2)
  needs: bench-build
  runs-on: ubuntu-latest
  timeout-minutes: 25
  continue-on-error: true   # &lt;- flip after 3 main runs
  steps:
    - uses: actions/checkout@v4
    - uses: dtolnay/rust-toolchain@1.88
      with: { toolchain: "1.88" }
    - uses: Swatinem/rust-cache@v2
      with: { shared-key: ${{ env.RUST_CACHE_KEY }} }
    - name: Run bench
      run: |
        MODE="${{ github.event.inputs.mode || 'quick' }}" \
        BENCHES=grpc_round_trip \
        BENCH_TIMEOUT=900 \
        bash scripts/bench_runner.sh
    - name: Run multi-proc integration test
      run: |
        cargo test -p efe-meta-swarm-server \
          --test integration_grpc_multiprocess \
          --release
    - name: Upload partial artefact
      if: always()
      uses: actions/upload-artifact@v4
      with:
        name: bench-partial-meta-swarm-grpc-${{ github.sha }}
        path: |
          target/criterion/**/new/estimates.json
          target/criterion/grpc_round_trip/meta-swarm-grpc-extras.json
          target/bench-grpc-round-trip.txt
        retention-days: 30
```

19.2 New `ops-master-gate.yml` step

```
- name: A2 meta-swarm gRPC master gate (Sprint A1 P2 PR #18)
  continue-on-error: true
  run: bash scripts/ops/run_suite.sh a2_meta_swarm_grpc
```

19.3 New `helm-meta-swarm-lint.yml` workflow

```
name: helm-meta-swarm-lint
on:
  pull_request:
    paths: ['charts/efe/charts/efe-meta-swarm/**']
  push:
    branches: [main]
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: azure/setup-helm@v4
      - name: helm lint
        run: helm lint charts/efe/charts/efe-meta-swarm
      - name: helm template (default = disabled)
        run: |
          helm template ./charts/efe &gt; /tmp/manifest-default.yaml
          # Compare against Phase-1 golden manifest.
          diff -u charts/efe/golden-phase1-default.yaml /tmp/manifest-default.yaml
      - name: helm template (enabled)
        run: |
          helm template ./charts/efe --set metaSwarm.enabled=true &gt; /tmp/manifest-enabled.yaml
          # Diff snapshot (sandbox tracked).
          diff -u charts/efe/golden-phase2-enabled.yaml /tmp/manifest-enabled.yaml
```

19.4 Existing `bench-aggregate` updates

The `needs:` list gets `bench-meta-swarm-grpc` appended. No other modification -- the new SLO keys (`SLO-A2-*`) are unique and consumed only by `run_suite.sh a2_meta_swarm_grpc`.

20. Documentation deliverables

#	Doc	Owner	Deadline
D1	`docs/EFE_A1_PHASE2_ARCHITECTURE.md` (this document)	meta-swarm-team	2026-07-01
D2	`crates/efe-meta-swarm-server/README.md`	meta-swarm-team	end of Sprint A1.P2.1
D3	`charts/efe/charts/efe-meta-swarm/README.md`	platform-team	end of Sprint A1.P2.2
D4	`docs/runbooks/META_SWARM_GRPC.md`	sre-team	mid-Sprint A1.P2.4
D5	`docs/security/PHASE2_THREAT_MODEL.md`	sec-team	end of Sprint A1.P2.1
D6	`docs/A1_PHASE2_SOAK_REPORT.md`	sre-team	day 7 of soak
D7	Update `docs/runbooks/ops_master_gate_fail.md` § 8 for A2 SLOs	sre-team	Sprint A1.P2.4 S3
D8	`docs/EFE_A1_PHASE2_LESSONS.md` (post-mortem of Phase 2 ; even if all PASS)	meta-swarm-team	2026-09-15
D9	Public bench HTML + CSV + reproducibility recipe	meta-swarm-team	Sprint A1.P2.4 S3
D10	`crates/efe-meta-swarm-proto/README.md` with versioning policy	meta-swarm-team	end of Sprint A1.P2.1

21. Open questions for Phase 2 kickoff

#	Question	Decision deadline
Q1	f32 on the wire vs f64 -- do we have a customer that needs f64 in Phase 2 ?	Sprint A1.P2.1 S1
Q2	Streaming `StepStream` mandatory or optional ? Server complexity vs round-trip savings.	Sprint A1.P2.1 S2
Q3	Do we ship Phase 2 with TLS-only, or allow plaintext for dev ? Recommendation : TLS-only, with a `--insecure` flag gated by `EFE_DEV_MODE=true`.	Sprint A1.P2.2 S3
Q4	Helm umbrella chart : do we ship Phase 2 in the existing chart version, or bump major ? Recommendation : minor bump (`0.x.0 -> 0.(x+1).0`).	Sprint A1.P2.2 S4
Q5	Soak start date : 2026-08-10 needs SRE on-call coverage. Confirm with @sre.	Sprint A1.P2.3 S1
Q6	Public bench : do we sign the HTML report (sigstore) for tamper-evidence ? Phase 3 default.	post-L2
Q7	mTLS rotation : keep manual for Phase 2 or invest in auto-rotation? Recommendation : manual + 7-day expiry alert.	Sprint A1.P2.2 S4
Q8	Three controllers HA vs single controller : worth the complexity for Phase 2 ? Recommendation : ship single controller with a K8s lease (active-passive), upgrade to Raft in Phase 4.	Sprint A1.P2.2 S3

Appendix A -- Cross-references

- Phase 1 architecture : `EFE_A1_PHASE1_ARCHITECTURE.md`
- Phase 1 bench tracking : `EFE_A1_BENCH_TRACKING.md`
- Operational doctrine : `EFE_OPERATIONAL_PRODUCTION_ARCHITECTURE.md`
- Phase 1 PR #4 description : `../crates/efe-meta-swarm/PR4_DESCRIPTION.md`
- Master SLO bundle : `../bench_baselines/v0.9.0/ops_master_slos.yaml`
- Master gate check : `../scripts/ops/master_gate_check.py`

Appendix B -- Glossary

- **L1, L2, L3** : maturity levels (Phase 1 = L1 attained ; Phase 2 = L2 ; Phase 3 = L3).
- **Topology T1/T2/T3** : in-process / single-node 3-instances / K8s.
- **Master rollup** : a meta-SLO that aggregates >= 2 surface SLOs via the `inherits` field.
- **Soft-launch** : 3-run observation window during which `continue-on-error: true` is set on the new bench job.
- **Soak** : continuous load over a defined duration (here 7 days) with strict no-incident criteria.
- **L2 gate** : the 10-criterion checklist (§ 12) ; passing it graduates the Phase from "preview" to "GA-equivalent for internal users".

Roadmap

Phase-2 roadmap : PR catalog (25 PRs), sprints, cadence, L2 gate

Chapter 2 of 17 — source : `docs/EFE_A1_PHASE2_ROADMAP.md` (1,244 LOC)

EFE Sprint A1 Phase 2 -- Detailed Roadmap (Jul-Aug 2026)

***Document type** : Execution-ready roadmap for Phase 2 (gRPC transport). **Status** : DRAFT (frozen at Phase 2 kickoff). **Architecture source** : `EFE_A1_PHASE2_ARCHITECTURE.md`. **Parent roadmap** : `EFE_A1_ROADMAP.md` § 5. **Window** : 2026-07-06 -> 2026-08-31 (8 weeks, 4 sprints, 25 PRs). **Gate** : L2 attained ⇔ 10 criteria pass (cf. § 12).*

Table of contents

1. Executive summary
2. Phase 2 commitment
3. Sprint A1.P2.1 -- Protos + bare servers (Jul 06-19)
4. Sprint A1.P2.2 -- Config + Helm scaffold (Jul 20 - Aug 02)
5. Sprint A1.P2.3 -- Observability + start of 7-day soak (Aug 03-16)
6. Sprint A1.P2.4 -- L2 graduation (Aug 17-30)
7. Daily cadence
8. PR catalog (#5-#25)
9. Test gates per sprint
10. Soak monitoring plan
11. L2 gate checklist (G1-G10)
12. RACI per deliverable
13. Risk burn-down
14. Communication
15. Rollback plan
16. Open questions per sprint
17. Definition of Done

0. Executive summary

```
╔═══════════════════════════════════════════════════════════════════════╗
║  Phase 2 plan-on-a-page                                               ║
╠═══════════════════════════════════════════════════════════════════════╣
║                                                                       ║
║  Sprint 1 (Jul 06-19) -- Protos + bare gRPC servers                    ▪
▪     PRs #5-#10  · 6 PRs · meta-swarm-team                           ▪
▪     Demo : 1 controller + 3 participants run on dev box,            ▪
▪            Iterate() bit-identical to Phase 1 reference             ▪
▪                                                                     ▪
▪  Sprint 2 (Jul 20-Aug 02) -- Config + Helm                           ▪
▪     PRs #11-#16 · 6 PRs · meta-swarm + platform + sec teams          ▪
▪     Demo : helm install on staging with flag flipped on/off,        ▪
▪            SIGHUP rolls participant in/out                          ▪
▪                                                                     ▪
▪  Sprint 3 (Aug 03-16) -- Observability + soak start                  ▪
▪     PRs #17-#21 · 5 PRs · meta-swarm + sre teams                    ▪
▪     Demo : Grafana dashboards live ; 7-day soak begins Mon Aug 10   ▪
▪                                                                     ▪
▪  Sprint 4 (Aug 17-30) -- L2 graduation                               ▪
▪     PRs #22-#25 · 4 PRs · meta-swarm + sre teams                    ▪
▪     Demo : G1-G10 verified, master gate GREEN, L2 announced         ▪
▪                                                                     ▪
▪  Total : 25 PRs · 8 weeks · 47 FTE-weeks                            ▪
▪                                                                     ▪
╚═══════════════════════════════════════════════════════════════════════╝
```

Top-level deliverable : a transparent, opt-in, fail-closed gRPC transport for `efe-meta-swarm` that **does not affect a single Phase-1 deployment** unless explicitly enabled, validated by a 7-day internal soak with zero Sev1/Sev2 and a published head-to-head bench.

1. Phase 2 commitment

1.1 What we will ship (binding)

#	Deliverable	Sprint	Lead	Verification
C1	`efe-meta-swarm-proto` crate (protos + tonic stubs)	P2.1	meta-swarm-team	`cargo doc -p efe-meta-swarm-proto` builds clean
C2	`efe-meta-swarm-server` crate (controller + participant bins)	P2.1	meta-swarm-team	`cargo run --bin controller` succeeds
C3	Multi-process integration test (3 processes, bit-identical result)	P2.1	meta-swarm-team	`tests/integration_grpc_multiprocess.rs` GREEN
C4	`EFE_META_SWARM_ENABLED` env-var plumbing in `efe-server`	P2.2	meta-swarm-team	Compare logs/manifests off-vs-on
C5	SIGHUP hot reload (atomic swap, <= 200 ms p99)	P2.2	meta-swarm-team	`tests/integration_sighup_hot_reload.rs`
C6	Helm sub-chart `efe-meta-swarm` default-disabled	P2.2	platform-team	`helm template` byte-identical when off
C7	mTLS controller <-> participant + efe-server <-> controller	P2.2	sec-team + platform	cert-manager Certificate resource live
C8	14 Prometheus metrics scraped	P2.3	meta-swarm-team	`curl :9090/metrics` returns 14 names
C9	8 OTel span types emitted	P2.3	meta-swarm-team	Sample trace seen in OTel collector
C10	2 Grafana dashboards + 4 alert rules	P2.3	sre-team	Dashboards rendered, alerts firing on test
C11	`tools/loadgen` traffic generator	P2.3	meta-swarm-team	`loadgen --rps 30 --duration 1m` succeeds
C12	7-day internal soak passes 10 criteria	P2.3-4	sre-team	`A1_PHASE2_SOAK_REPORT.md` published
C13	Public bench (5 workloads x 3 modes)	P2.3-4	meta-swarm-team	`bench_results/meta-swarm-vs-mono-swarm-2026-08.{csv,html}`
C14	4 new SLOs + master rollup `SLO-OPS-A1-META-GRPC`	P2.4	meta-swarm-team	`bench_baselines/v0.11.0/meta_swarm_grpc_slos.yaml`
C15	Smoke test 9 pinning rollup #20	P2.4	meta-swarm-team	`tests/bench_smoke.rs` GREEN
C16	Runbook `META_SWARM_GRPC.md` + 2 SRE drills	P2.4	sre-team	Runbook + drill logs committed
C17	Phase 2 end-of-sprint report	P2.4	meta-swarm-team	`docs/EFE_A1_PHASE2_LESSONS.md`

1.2 What we will NOT ship (deferred to P3+)

- Cross-region replication
- Per-tenant isolation

- Customer-facing SLA
- BLAS / GPU acceleration
- Auto-rotation of mTLS certs (manual rotation OK in Phase 2)
- Raft-based controller HA (K8s lease active-passive is enough)

1.3 Success signal at L2

A single sentence the team should be able to say with a straight face on 2026-08-31 :

> *"Any future PR that pushes* `iterate()` *p99 past 100 ms, breaks multi-process determinism, blows the 200 ms reload budget, or drops OTel trace coverage below 5 % is blocked at the master gate automatically, and we have 7 consecutive days of zero Sev1/Sev2 production traffic to prove the gate is calibrated correctly."*

2. Sprint A1.P2.1 -- Protos + bare servers

Window : 2026-07-06 (Mon) -> 2026-07-19 (Sun) -- 2 weeks **Theme** : Establish the wire protocol and a minimum-viable distributed runtime. **Demo date** : Fri 2026-07-17 16:00 UTC (`#meta-swarm` Zoom) **Capacity** : 6 meta-swarm-team FTE x 2 weeks = 12 FTE-weeks

2.1 Week 1 (Jul 06-12) -- Protos + scaffolding

Goals

- Land the 4 proto files + tonic-build wiring
- Scaffold both binaries (no logic, just network skeleton)
- Threat model draft circulating

PRs

PR #5 -- `feat(efe-meta-swarm-proto): scaffold protos + tonic-build`

Owner : meta-swarm-team (lead : J. Tan) **Estimate** : 3 FTE-days **Branch** : `A1.P2/05-meta-swarm-proto-scaffold`

Diff scope :

```
crates/efe-meta-swarm-proto/  (NEW)
+-- Cargo.toml
+-- build.rs                     # tonic_build::compile_protos
+-- proto/
|   +-- efe/v1/meta_swarm/
|       +-- service_meta_swarm.proto
|       +-- service_meta_participant.proto
|       +-- messages.proto
|       +-- errors.proto
+-- src/
|   +-- lib.rs                   # re-export tonic-generated
+-- README.md                    # versioning policy
Cargo.toml (workspace) +1 member
```

Acceptance :

- `cargo doc -p efe-meta-swarm-proto --no-deps` builds clean
- `cargo build -p efe-meta-swarm-proto` produces a library with 0 warnings
- README.md documents the v1 contract + reserved field numbers >= 100
- Smoke test : import a `MetaSwarmServiceClient` from another crate, no compile error

Out-of-scope : no service implementation, no client code beyond what tonic generates.

Dependencies : none (first PR of the sprint).

PR #6 -- `feat(efe-meta-swarm-server): scaffold server crate`

Owner : meta-swarm-team (lead : J. Tan) **Estimate** : 4 FTE-days **Branch** : `A1.P2/06-meta-swarm-server-scaffold`

Diff scope :

```
crates/efe-meta-swarm-server/  (NEW)
```

```
+-- Cargo.toml                   # panic = "unwind" override
+-- src/
|   +-- lib.rs
|   +-- bin/
|   |   +-- controller.rs        # minimal main() : parse args, bind, serve
|   |   +-- participant.rs       # idem
|   +-- controller/
|   |   +-- mod.rs
|   |   +-- service.rs           # impl MetaSwarmService { todo!() }
|   +-- participant/
|   |   +-- mod.rs
|   |   +-- service.rs           # impl MetaParticipantService { todo!() }
|   +-- observability/
|   |   +-- health.rs            # only Health is implemented
|   |   +-- metrics.rs           # stub
|   +-- auth/
|   |   +-- mtls.rs              # stub
|   |   +-- bearer.rs            # stub
|   +-- error.rs
+-- README.md
Cargo.toml (workspace) +1 member
```

Acceptance :

- `cargo run --bin controller` binds and answers `Health` with `STATUS_SERVING`
- `cargo run --bin participant` same
- All other RPCs return `UNIMPLEMENTED`
- README documents how to run locally + binary CLI args

Dependencies : PR #5 (uses the generated stubs).

PR #7 -- `docs(security): Phase 2 threat model draft`

Owner : sec-team (lead : R. Kapoor) **Estimate** : 2 FTE-days

Diff scope :

```
docs/security/PHASE2_THREAT_MODEL.md  (NEW)
```

Acceptance :

- 8 sections : assets, trust boundaries, attackers, threats, mitigations, residual risk, monitoring, references
- >= 12 distinct threat scenarios catalogued (STRIDE per surface)
- 0 SEV-1 unmitigated entries
- sec-team peer-review tag on the PR

Dependencies : none ; can run in parallel.

2.2 Week 2 (Jul 13-19) -- Wire up Iterate

Goals

- `MetaSwarmService::Iterate` wired through the in-process Phase-1 `MetaSwarm`
- `MetaParticipantService::Step` wired through `MockParticipant`
- First multi-process integration test passes

PRs

PR #8 -- `feat(efe-meta-swarm-server): wire MetaSwarmService::Iterate`

Owner : meta-swarm-team (lead : M. Okafor) **Estimate** : 5 FTE-days **Branch** : `A1.P2/08-iterate-wire`

Diff scope :

```
crates/efe-meta-swarm-server/src/
+-- controller/
|   +-- service.rs               # Iterate impl
|   +-- manifest.rs              # NEW : YAML manifest reader
|   +-- discovery.rs             # NEW : StaticDiscovery only (T2)
+-- client/
|   +-- mod.rs                   # NEW
|   +-- remote_participant.rs    # NEW : controller's client to participants
+-- tests/
```

```
    +-- integration_iterate_single_proc.rs   # NEW : in-process unit
```

Acceptance :

- Controller, given a static manifest, can resolve participants by id
- `Iterate()` round-trips through 3 participant gRPC clients
- Single-process unit test : `Iterate()` returns same `iterations + residual` as Phase-1 `MetaSwarm::iterate()` (within ±2 iters tolerance for variance)
- Cancellation : if client drops the call, server cancels participant calls within 100 ms

Dependencies : PRs #5, #6.

PR #9 -- `feat(efe-meta-swarm-server): wire MetaParticipantService::Step`

Owner : meta-swarm-team (lead : M. Okafor) **Estimate** : 4 FTE-days **Branch** : `A1.P2/09-step-wire`

Diff scope :

```
crates/efe-meta-swarm-server/src/
+-- participant/
    +-- service.rs               # Step + Reset + Capabilities + State + Health
    +-- adapter.rs               # NEW : adapt to Phase-1 MockParticipant / LocalParticipant
```

Acceptance :

- `Step()` round-trip latency <= 5 ms p99 on localhost
- `Capabilities()` returns the static descriptor
- Determinism : 3 consecutive `Step()` with same `pulled_from_meta` produce bit-identical outputs

Dependencies : PR #6.

PR #10 -- `test(efe-meta-swarm-server): integration_grpc_multiprocess`

Owner : meta-swarm-team (lead : J. Tan) **Estimate** : 3 FTE-days **Branch** : `A1.P2/10-multiproc-test`

Diff scope :

```
crates/efe-meta-swarm-server/tests/
+-- integration_grpc_multiprocess.rs    # NEW
+-- fixtures/
    +-- 3-participant-localhost.yaml    # NEW manifest
```

Acceptance :

- Test spawns 1 controller process + 3 participant processes via `std::process::Command`
- Calls `Iterate()` via the client
- Asserts : `iterations <= 60`, `final_residual <= 1e-3`, `stop_reason == Converged`
- Determinism : 3 reruns produce bit-identical residual.to_bits()
- Test finishes in <= 30 s (`#[ignore]` if not in CI ; `cargo test -- --ignored` in CI matrix)

Dependencies : PRs #8, #9.

2.3 Sprint 1 demo (2026-07-17 Fri 16:00 UTC)

Time	Topic	Lead
00:00	Intro	meta-swarm-team
00:05	Proto walkthrough (key RPCs + types)	J. Tan
00:15	Run `cargo run --bin controller` + 3x `cargo run --bin participant`	M. Okafor
00:25	Live `Iterate()` call -> show bit-identical result vs Phase-1 example	M. Okafor
00:35	Threat model headlines	R. Kapoor
00:40	Q&A	all

2.4 Sprint 1 retrospective (2026-07-19 Sun async in #meta-swarm)

3 questions :

- What went well ?
- What blocked us ?
- What we change for Sprint 2 ?

3. Sprint A1.P2.2 -- Config + Helm scaffold

Window : 2026-07-20 (Mon) -> 2026-08-02 (Sun) -- 2 weeks **Theme** : Make it deployable. Env var, SIGHUP, Helm chart, mTLS. **Demo date** : Fri 2026-07-31 16:00 UTC **Capacity** : meta-swarm-team (4 FTE) + platform-team (2 FTE) + sec-team (1 FTE) x 2 weeks = 14 FTE-weeks

3.1 Week 3 (Jul 20-26) -- env var + SIGHUP

PRs

PR #11 -- `feat(efe-server): EFE_META_SWARM_ENABLED plumbing`

Owner : meta-swarm-team (lead : J. Tan) **Estimate** : 4 FTE-days

Diff scope :

```
crates/efe-server/Cargo.toml                    # +1 optional dep efe-meta-swarm-proto behind feature "m ■
    eta-swarm-grpc"
crates/efe-server/src/bridge/meta_swarm.rs      # NEW : dispatch in-proc vs gRPC
crates/efe-server/src/bridge/mod.rs             # +mod
crates/efe-server/src/config.rs                 # +EFE_META_SWARM_ENABLED reader
```

Acceptance :

- Env var unset OR =false -> byte-identical to Phase 1 (verified by hashing the gRPC server response for a fixed request)
- =true + valid `EFE_META_SWARM_PARTICIPANTS` -> calls `MetaSwarmServiceClient::iterate`
- =true + invalid config -> process refuses to start with explicit error
- Smoke : `efe-server` startup log diff with/without flag is exactly 5 lines (controller URL + auth + 3 participant URLs)

Dependencies : PRs #5, #8.

PR #12 -- `feat(efe-meta-swarm-server): SIGHUP hot-reload + ReloadConfig RPC`

Owner : meta-swarm-team (lead : M. Okafor) **Estimate** : 5 FTE-days **Branch** : `A1.P2/12-sighup-reload`

Diff scope :

```
crates/efe-meta-swarm-server/src/
+-- controller/
|   +-- reload.rs               # NEW : SIGHUP handler + atomic manifest swap
|   +-- service.rs              # +impl ReloadConfig
+-- tests/
    +-- integration_sighup_hot_reload.rs   # NEW : 1000 random-timed SIGHUPs during Iterate
```

Acceptance :

- Atomic swap : in-flight `Iterate()` never sees partial state
- <= 200 ms p99 reload time on the 3-participant scenario
- `drain_inflight=true` waits up to 30 s ; `=false` aborts cleanly with `STOP_REASON_CONFIG_RELOAD`
- Dimension change -> `META_ERROR_RELOAD_DIM_CHANGED`
- Manifest re-read from disk uses `O_NOFOLLOW` to prevent symlink swap attacks (sec-team req)

Dependencies : PRs #8, #9.

PR #13 -- `feat(charts): Helm sub-chart skeleton`

Owner : platform-team (lead : K. Müller) **Estimate** : 4 FTE-days

Diff scope :

```
charts/efe/charts/efe-meta-swarm/  (NEW)
+-- Chart.yaml
+-- values.yaml                              # default-disabled
+-- values.schema.json
+-- templates/
|   +-- _helpers.tpl
|   +-- controller-statefulset.yaml
|   +-- controller-service.yaml
|   +-- controller-pdb.yaml
|   +-- participant-deployment.yaml          # template per participant via range
|   +-- participant-service.yaml
|   +-- secret-tls.yaml
|   +-- configmap-manifest.yaml
|   +-- tests/connection-test.yaml
+-- README.md
```

Acceptance :

- `helm lint` clean
- `helm template ./charts/efe` (defaults) = byte-identical to Phase-1 manifest
- `helm template ./charts/efe --set metaSwarm.enabled=true` produces a manifest covering 1 controller StatefulSet (3 replicas) + 3 participant Deployments + Services
- `helm test` after install passes the `connection-test` pod (calls `Health`)

Dependencies : PRs #6, #11.

3.2 Week 4 (Jul 27 - Aug 02) -- Helm values + mTLS

PRs

PR #14 -- `feat(charts): full values + templates`

Owner : platform-team (lead : K. Müller) **Estimate** : 4 FTE-days

Diff scope :

```
charts/efe/charts/efe-meta-swarm/templates/
+-- controller-servicemonitor.yaml     # NEW (Prometheus Operator)
+-- participant-servicemonitor.yaml    # NEW
+-- controller-networkpolicy.yaml      # NEW (allow only efe-server ingress)
+-- participant-networkpolicy.yaml     # NEW (allow only controller ingress)
+-- ...

charts/efe/charts/efe-meta-swarm/values.yaml   # populated
```

Acceptance :

- All 3 participants render with distinct configs (dim, kappa, seed)
- NetworkPolicy correctly scopes the trust boundary
- `--set metaSwarm.participants[0].replicas=2` produces 2 participant pods (multi-replica is a P3 future, but template must already support it)

Dependencies : PR #13.

PR #15 -- `ci(helm): helm-meta-swarm-lint workflow`

Owner : platform-team (lead : K. Müller) **Estimate** : 2 FTE-days

Diff scope :

```
.github/workflows/helm-meta-swarm-lint.yml   # NEW
charts/efe/golden-phase1-default.yaml        # NEW : checked-in manifest snapshot
charts/efe/golden-phase2-enabled.yaml        # NEW : checked-in manifest snapshot
```

Acceptance :

- Workflow runs on PRs touching `charts/efe/**`
- `helm lint` + `helm template` (default) diff vs `golden-phase1-default.yaml` exact match
- `helm template --set metaSwarm.enabled=true` diff vs `golden-phase2-enabled.yaml` exact match
- Test : intentionally break the chart in a fork branch -> workflow RED

Dependencies : PR #14.

PR #16 -- `feat(security): mTLS wiring + cert-manager Issuer`

Owner : sec-team (lead : R. Kapoor) + platform-team (K. Müller) **Estimate** : 5 FTE-days

Diff scope :

```
charts/efe/charts/efe-meta-swarm/templates/
+-- tls-issuer.yaml               # NEW : cert-manager Issuer
+-- controller-certificate.yaml   # NEW : cert-manager Certificate
+-- participant-certificate.yaml  # NEW (template per participant)

crates/efe-meta-swarm-server/src/auth/
+-- mtls.rs                       # full impl : load cert, configure tonic TLS
+-- tests/
    +-- tls_e2e.rs                 # NEW : in-test cert generation + handshake check
```

Acceptance :

- mTLS handshake enforced on all 3 gRPC links (efe-server <-> controller <-> participants)
- SAN-based authorization : controller rejects non-`efe-server.*.svc` callers
- cert-manager issues 90-day leaf certs with auto-renewal at 60 days
- Bearer token fallback works (`EFE_META_SWARM_AUTH_TOKEN`) -- for dev only, gated by `EFE_DEV_MODE=true`

Dependencies : PRs #6, #13.

3.3 Sprint 2 demo (2026-07-31 Fri 16:00 UTC)

Time	Topic	Lead
00:00	Intro	meta-swarm-team
00:05	Env-var flip live : staging deploy -> disable -> re-enable	J. Tan
00:15	SIGHUP drill : add a 4th participant in the manifest, send SIGHUP, see it active	M. Okafor
00:25	Helm install with chart : flag off (compare manifest), then on	K. Müller
00:35	mTLS handshake walkthrough	R. Kapoor
00:45	Q&A	all

3.4 Sprint 2 retrospective (2026-08-02 Sun async)

Same 3-question format.

4. Sprint A1.P2.3 -- Observability + start of 7-day soak

Window : 2026-08-03 (Mon) -> 2026-08-16 (Sun) -- 2 weeks **Theme** : Make it observable, generate traffic, start the soak. **Demo date** : Fri 2026-08-14 16:00 UTC **Capacity** : meta-swarm-team (4 FTE) + sre-team (2 FTE) x 2 weeks = 12 FTE-weeks

4.1 Week 5 (Aug 03-09) -- Metrics + tracing + dashboards

PRs

PR #17 -- `feat(efe-meta-swarm-server): 14 Prometheus metrics`

Owner : meta-swarm-team (lead : M. Okafor) **Estimate** : 4 FTE-days

Diff scope :

```
crates/efe-meta-swarm-server/src/observability/
+-- metrics.rs              # full registry, 14 metrics
```

Acceptance :

- All 14 metrics emitted with correct labels (cf. architecture § 9.1)

- `meta_swarm_inflight` gauge tracks concurrent `Iterate` calls
- `meta_swarm_participant_step_duration_seconds` has the right histogram buckets `[1ms, 5ms, ..., 500ms]`
- `meta_swarm_build_info{git_sha=..., proto_version=v1}` always 1
- Smoke test : run controller, hit Iterate 10x concurrent, scrape `:9090/metrics`, assert all 14 metric names present

Dependencies : PRs #8, #9.

PR #18 -- `feat(efe-meta-swarm-server): 8 OpenTelemetry span types`

Owner : meta-swarm-team (lead : M. Okafor) **Estimate** : 4 FTE-days

Diff scope :

```
crates/efe-meta-swarm-server/src/observability/
+-- tracing.rs               # OTel setup + span helpers
```

Acceptance :

- 8 distinct span names emitted (cf. architecture § 9.2)
- `traceparent` header propagated client -> controller -> participant (3-hop trace)
- Sampling controlled by `EFE_META_SWARM_TRACE_SAMPLE_RATIO` (default 0.05)
- Smoke test : enable 100 % sampling, call `Iterate` once, inspect collector -> see 1 root span + N child spans (5 inner steps x 3 participants ≈ 15+ child spans)

Dependencies : PR #17.

PR #19 -- `feat(grafana): dashboards + alert rules`

Owner : sre-team (lead : L. Ortiz) **Estimate** : 3 FTE-days

Diff scope :

```
grafana/dashboards/
+-- efe-meta-swarm-ops.json       # operational dashboard
+-- efe-meta-swarm-algo.json      # algorithmic dashboard

charts/efe/charts/efe-meta-swarm/templates/
+-- prometheus-rules.yaml         # 4 PrometheusRule entries
```

Acceptance :

- Both dashboards render on staging Grafana
- All 14 metrics referenced by at least one panel
- 4 alerts (cf. architecture § 9.4) verified by injecting a fault that triggers each
- Alerts route to test-only PagerDuty service (no real on-call yet)

Dependencies : PRs #17, #18, #14.

4.2 Week 6 (Aug 10-16) -- Loadgen + soak begins

PRs

PR #20 -- `tools(loadgen): meta-swarm traffic generator`

Owner : meta-swarm-team (lead : J. Tan) **Estimate** : 3 FTE-days

Diff scope :

```
tools/loadgen/
+-- Cargo.toml               # NEW : standalone bin
+-- src/
|   +-- main.rs
|   +-- scenarios/
|   |   +-- kovant_steady.rs       # 60 % of soak traffic
|   |   +-- random_initial.rs      # 30 %
|   |   +-- sighup_chaos.rs        # 10 %
|   +-- reporter.rs           # Prometheus pushgateway client
+-- README.md
```

Acceptance :

- `loadgen --rps 30 --duration 168h --scenarios kovant_steady:60,random:30,sighup:10 --target=staging-controller`
- Loadgen emits its own metrics (req sent, errors, observed latency)
- Sighup chaos : every 30 min, sends SIGHUP to a random controller pod
- Graceful shutdown via SIGTERM

Dependencies : PRs #5, #8.

PR #21 -- `feat(public-bench): harness + 5 workloads`

Owner : meta-swarm-team (lead : M. Okafor) **Estimate** : 4 FTE-days

Diff scope :

```
bench_public/
+-- Cargo.toml
+-- src/
|   +-- main.rs               # bench driver
|   +-- workloads/
|   |   +-- w1_kovant_3.rs
|   |   +-- w2_kovant_3_2x.rs
|   |   +-- w3_seven_participants.rs
|   |   +-- w4_single_swarm.rs
|   |   +-- w5_slow_participant.rs
|   +-- modes/
|   |   +-- mono_swarm.rs
|   |   +-- meta_in_proc.rs
|   |   +-- meta_grpc.rs
|   +-- report.rs             # CSV + HTML emitters
+-- README.md                 # reproducibility recipe
```

Acceptance :

- `cargo run --bin bench_public --release -- --workloads W1,W2,W3,W4,W5 --modes mono,in_proc,grpc`
- Outputs `bench_results/raw/*.json` (criterion-shape per run)
- Outputs `bench_results/meta-swarm-vs-mono-swarm-DRAFT.{csv,html}` (final filename : `...2026-08.{csv,html}` after L2 pass)
- HTML is self-contained (inline CSS+JS, no CDN)
- Harness deterministically reproducible : same hardware + git sha -> same CSV bit-for-bit

Dependencies : PR #11 (needs the `EFE_META_SWARM_ENABLED` flag to flip between in-proc and gRPC).

4.3 7-day soak begins (Mon 2026-08-10 00:00 UTC)

Cf. § 9 for full monitoring plan.

4.4 Sprint 3 demo (2026-08-14 Fri 16:00 UTC)

Time	Topic	Lead
00:00	Intro	meta-swarm-team
00:05	Live tour of `ops` dashboard during soak	L. Ortiz
00:15	Live tour of `algo` dashboard during soak	L. Ortiz
00:25	Sample OTel trace : 1 Iterate call from efe-server -> controller -> 3 participants	M. Okafor
00:35	Soak day-4 status : SLO compliance, incidents, memory trend	L. Ortiz
00:45	Q&A	all

5. Sprint A1.P2.4 -- L2 graduation

Window : 2026-08-17 (Mon) -> 2026-08-30 (Sun) -- 2 weeks **Theme** : Land the SLOs, finish the soak, publish the bench, attain L2. **Demo date** : Fri 2026-08-28 16:00 UTC (= L2 graduation demo) **Capacity** : meta-swarm-team (3 FTE) + sre-team (2 FTE) x 2 weeks = 10 FTE-weeks

5.1 Week 7 (Aug 17-23) -- SLOs + runbook + drills

PRs

PR #22 -- `feat(baselines): 4 new SLOs + master rollup #20`

Owner : meta-swarm-team (lead : J. Tan) **Estimate** : 2 FTE-days

Diff scope :

```
bench_baselines/v0.11.0/                        (NEW directory)
+-- meta_swarm_grpc_slos.yaml                   # 4 SLOs

bench_baselines/v0.9.0/ops_master_slos.yaml     # +1 master rollup #20
scripts/ops/run_suite.sh                        # +1 case "a2_meta_swarm_grpc"
.github/workflows/ops-master-gate.yml           # +1 baseline + 1 step
.github/workflows/bench.yml                     # +1 job bench-meta-swarm-grpc
docs/EFE_A1_BENCH_TRACKING.md                   # extended for A2
```

Acceptance :

- Consolidator ingests v0.11.0 baseline cleanly (`surface SLOs found : 4` for A2 alone)
- Master gate reports 20 master rollups (was 19 after L1)
- Negative test : break one A2 SLO via crafted measures -> gate FAIL exit 1

Dependencies : PRs #17 (metrics that feed P99 + reload SLOs), #18 (trace coverage SLO).

PR #23 -- `test(bench_smoke): smoke test #9 for rollup #20`

Owner : meta-swarm-team (lead : J. Tan) **Estimate** : 1 FTE-day

Diff scope :

```
crates/efe-meta-swarm/tests/bench_smoke.rs      # +test master_rollup_phase2_inherits_4_slos
```

Acceptance :

- Asserts master rollup `SLO-OPS-A1-META-GRPC` exists, status ACTIVE, inherits exactly the 4 A2 SLO ids
- Mutation test : remove the rollup -> smoke RED ; restore -> smoke GREEN

Dependencies : PR #22.

PR #24 -- `bench(public): publish meta-swarm vs mono-swarm (Aug 2026)`

Owner : meta-swarm-team (lead : M. Okafor) **Estimate** : 3 FTE-days

Diff scope :

```
bench_results/
+-- meta-swarm-vs-mono-swarm-2026-08.csv          # NEW : 5 workloads x 3 modes
+-- meta-swarm-vs-mono-swarm-2026-08.html         # NEW : interactive
+-- raw/                                          # NEW : criterion JSON per run
    +-- ...
```

Acceptance :

- All 5 workloads x 3 modes recorded
- CSV is machine-parseable (`pandas.read_csv` succeeds)
- HTML has working toggles + residual-trajectory plots
- Reproducibility : `bash bench_runner.sh --public --workload W1` on fresh checkout produces same CSV within ±5 % per cell
- Honest framing block at the top (cf. architecture § 11.4)

Dependencies : PR #21.

Runbook + drills (no PR ; tracked as deliverables)

Owner : sre-team (lead : L. Ortiz) **Estimate** : 2 FTE-days

Outputs :

```
docs/runbooks/META_SWARM_GRPC.md                # NEW : on-call runbook
docs/runbooks/drills/2026-08-19_sighup_drill.md  # NEW : drill log
docs/runbooks/drills/2026-08-21_partdown_drill.md # NEW : drill log
```

Acceptance :

- Runbook follows the template of `ops_master_gate_fail.md`
- Drill 1 : staging SIGHUP at random time -> recovery, dashboards green within 30 s
- Drill 2 : `kubectl delete pod participant-supplier-scoring-0` -> recovery, dashboards green within 60 s
- Both drill logs include : timestamp, observer, expected/observed behavior, follow-ups

Dependencies : PRs #14, #19.

5.2 Week 8 (Aug 24-30) -- L2 verification + announcement

Activities

L2 gate verification (Mon 2026-08-24)

Owner : meta-swarm-team + sre-team **Estimate** : 1 FTE-day (whole team in a Zoom room)

Run the 10-criterion checklist (cf. § 10). Each criterion verified by 2 engineers (4-eye policy). Outcomes tracked in :

```
docs/A1_PHASE2_L2_GATE_VERIFICATION.md          # NEW
```

If any criterion is RED, the gate is **deferred 2 weeks** (cf. `EFE_A1_ROADMAP.md` § 12).

PR #25 -- `docs: Phase 2 end-of-sprint lessons + final report`

Owner : meta-swarm-team (lead : J. Tan) **Estimate** : 2 FTE-days

Diff scope :

```
docs/EFE_A1_PHASE2_LESSONS.md                   # NEW : retrospective
docs/A1_PHASE2_SOAK_REPORT.md                   # NEW : 7-day soak report
docs/EFE_A1_ROADMAP.md                          # refresh : P2 -&gt; [OK]
crates/efe-meta-swarm/PR4_DESCRIPTION.md        # link to PR5-25 chain
```

Acceptance :

- Lessons doc has 4 sections : went well / blocked / surprised / changes for P3
- Soak report has the 10 pass-criterion table + memory time series chart
- Roadmap refresh moves P2 to [OK], refreshes 2026-08-31 as L2 date

Dependencies : all prior PRs.

L2 announcement (Fri 2026-08-28 16:00 UTC = demo)

Format : Slack post to `#announcements` using the template from `EFE_A1_ROADMAP.md` § 16.1.

Buffer (Sun 2026-08-30)

Reserved for any straggler PR fix-up. Hard deadline 23:59 UTC.

6. Daily cadence

6.1 Standups

Day	Time	Format	Attendees
Mon-Fri	09:30 UTC	Async post in `#meta-swarm-standup`	All meta-swarm-team
Wed	14:00 UTC	15-min sync video	Whole team (incl. sre/platform/sec)

Async standup template : `` `yesterday : <bullet> today : <bullet> blocked : <bullet | none> phase 2 PR : <#N>` ``

6.2 Weekly cadence

Day	Activity
Mon	Sprint goals review (10 min in standup)
Wed	Cross-team sync video (15 min)
Thu	Mid-sprint risk check (async in `#meta-swarm-risks`)
Fri	End-of-week metrics : PR burndown, SLO trend, soak status (if running)

6.3 Soak-week-specific cadence (Aug 10-17)

Every 24 h during the soak :

- 08:00 UTC : SRE on-call posts daily-status to `#meta-swarm-soak`
- 17:00 UTC : meta-swarm-team review the status, decides on any action
- 23:00 UTC : automatic Grafana snapshot saved to `docs/A1_PHASE2_SOAK_DAILY/`

7. PR catalog

#	Title	Owner	Sprint	Est	Deps
5	`feat(efe-meta-swarm-proto): scaffold protos`	meta-swarm	P2.1 W1	3 d	--
6	`feat(efe-meta-swarm-server): scaffold`	meta-swarm	P2.1 W1	4 d	#5
7	`docs(security): Phase 2 threat model draft`	sec	P2.1 W1	2 d	--
8	`feat(server): wire MetaSwarmService::Iterate`	meta-swarm	P2.1 W2	5 d	#5, #6
9	`feat(server): wire MetaParticipantService::Step`	meta-swarm	P2.1 W2	4 d	#6
10	`test: integration_grpc_multiprocess`	meta-swarm	P2.1 W2	3 d	#8, #9
11	`feat(efe-server): EFE_META_SWARM_ENABLED plumbing`	meta-swarm	P2.2 W3	4 d	#5, #8
12	`feat: SIGHUP hot-reload + ReloadConfig`	meta-swarm	P2.2 W3	5 d	#8, #9
13	`feat(charts): Helm sub-chart skeleton`	platform	P2.2 W3	4 d	#6, #11
14	`feat(charts): full values + templates`	platform	P2.2 W4	4 d	#13
15	`ci(helm): helm-meta-swarm-lint workflow`	platform	P2.2 W4	2 d	#14
16	`feat(sec): mTLS + cert-manager Issuer`	sec + platform	P2.2 W4	5 d	#6, #13
17	`feat: 14 Prometheus metrics`	meta-swarm	P2.3 W5	4 d	#8, #9
18	`feat: 8 OpenTelemetry span types`	meta-swarm	P2.3 W5	4 d	#17
19	`feat(grafana): dashboards + alert rules`	sre	P2.3 W5	3 d	#17, #18, #14
20	`tools(loadgen): traffic generator`	meta-swarm	P2.3 W6	3 d	#5, #8
21	`feat(public-bench): harness + 5 workloads`	meta-swarm	P2.3 W6	4 d	#11
22	`feat(baselines): 4 SLOs + rollup #20`	meta-swarm	P2.4 W7	2 d	#17, #18
23	`test(smoke): rollup #20 pin`	meta-swarm	P2.4 W7	1 d	#22

#	Title	Owner	Sprint	Est	Deps
24	`bench(public): publish results 2026-08`	meta-swarm	P2.4 W7	3 d	#21
--	runbook + 2 SRE drills	sre	P2.4 W7	2 d	#14, #19
25	`docs: Phase 2 lessons + soak report + roadmap refresh`	meta-swarm	P2.4 W8	2 d	all

Total : **25 PRs**, ~**75 FTE-days** ≈ **15 FTE-weeks** of focused PR work (over 8 calendar weeks, leaves slack for review, drills, soak observation, unplanned bug-fixes).

8. Test gates per sprint

Each sprint has a binary test gate. Sprint cannot close without it.

8.1 Sprint A1.P2.1 gate

#	Check	How
T1	`cargo build -p efe-meta-swarm-proto --offline`	CI green
T2	`cargo build -p efe-meta-swarm-server --offline` (both bins)	CI green
T3	`cargo test -p efe-meta-swarm-server --test integration_grpc_multiprocess --release`	CI green ; bit-identical residual to Phase-1 reference
T4	`cargo doc -p efe-meta-swarm-proto --no-deps`	CI green, 0 warnings
T5	`actionlint .github/workflows/*.yml`	CI green
T6	Phase-1 master-gate still PASS	CI green (no Phase-2 work has broken P1)

8.2 Sprint A1.P2.2 gate

#	Check	How
T7	`helm lint charts/efe/charts/efe-meta-swarm`	CI green
T8	`helm template ./charts/efe` byte-diff vs `golden-phase1-default.yaml`	CI green (= empty diff)
T9	`helm template --set metaSwarm.enabled=true` byte-diff vs `golden-phase2-enabled.yaml`	CI green
T10	`cargo test --test integration_sighup_hot_reload` (1000 random-timed SIGHUPs)	CI green
T11	mTLS e2e handshake test	CI green
T12	`EFE_META_SWARM_ENABLED=false` startup hash identical to Phase 1	CI green

8.3 Sprint A1.P2.3 gate

#	Check	How
T13	All 14 Prometheus metrics scrape returns names	Manual + smoke
T14	OTel sample trace seen in collector	Manual
T15	4 alert rules each verified by fault injection	SRE drill log
T16	Loadgen completes 1 h smoke at 30 rps with 0 errors	CI
T17	Public bench harness runs 1 workload in 1 mode	CI
T18	Soak day-7 status : pass criteria S1-S10 all green	SRE daily check

8.4 Sprint A1.P2.4 gate (= L2 gate, cf. § 10)

9. Soak monitoring plan

9.1 Setup checklist (must complete by Sun 2026-08-09 23:59 UTC)

- [ ] Staging cluster scaled to soak topology (1 controller StatefulSet 3-replicas + 3 participant Deployments)
- [ ] Loadgen Job deployed, dry-run for 1 h at 30 rps verified clean
- [ ] Grafana dashboards reviewed + annotations enabled
- [ ] PagerDuty test escalation : page test -> SRE on-call -> ack within 5 min
- [ ] Run-book printed and on the SRE table
- [ ] SRE on-call rotation confirmed for the full 7 days
- [ ] Slack alert channel `#meta-swarm-soak-alerts` created and connected to Alertmanager
- [ ] Daily-status template pinned in `#meta-swarm-soak`
- [ ] Memory baseline measured (RSS at t=0) and recorded

9.2 Daily status template

```
== Soak day N / 7  ==  (YYYY-MM-DD HH:MM UTC)

Convergence rate  : ___ %   (target &gt;= 99.95 %)
p99 latency       : ___ ms  (target &lt;= 100 ms)
RSS growth vs t=0 : ___ %   (target &lt;= 5 %)
Incidents         : Sev1=__  Sev2=__  Sev3=__
Reloads success   : ___ / ___ (target &gt;= 99.9 %)
Determinism check: PASS / FAIL
Trace coverage    : ___ %   (target &gt;= 5 %)

Open items :
  - ...

Next observation : &lt;time&gt;
```

9.3 Stop-the-soak triggers

ANY of the following triggers immediate soak abort + reset to day 0 (after RCA) :

- 1 Sev1 incident
- 1 Sev2 incident
- 2 Sev3 incidents
- Convergence rate < 99.9 % for any 10-min window
- p99 latency > 150 ms for any 10-min window
- Determinism check fails

- Memory grows > 10 % over baseline at any point

9.4 Final soak report (Sun 2026-08-17 23:59 UTC)

`docs/A1_PHASE2_SOAK_REPORT.md` -- full template in `EFE_A1_PHASE2_ARCHITECTURE.md` § 10.3.

10. L2 gate checklist

10 criteria, each binary, each owned, each verifiable. All must be GREEN by Mon 2026-08-24 23:59 UTC for L2 to be attained on 2026-08-31.

#	Criterion	Owner	Verification
G1	7 Phase-1 A1 SLOs GREEN on 7/7 last nightly runs	meta-swarm	`target/ops/master_report.json` archives ; grep `"SLO-A1-META-.*"`, all status PASS
G2	4 Phase-2 A2 SLOs GREEN on 7/7 last nightly runs	meta-swarm	Same source ; grep `"SLO-A2-.*"`
G3	Master rollup `SLO-OPS-A1-META-SWARM` (#19) PASS 7/7 nightly	meta-swarm	Master report
G4	Master rollup `SLO-OPS-A1-META-GRPC` (#20) PASS 7/7 nightly	meta-swarm	Master report
G5	7-day soak passes all 10 sub-criteria	sre-team	`A1_PHASE2_SOAK_REPORT.md` published
G6	Public bench published with all 5 x 3 cells filled	meta-swarm	`bench_results/meta-swarm-vs-mono-swarm-2026-08.{csv,html}`
G7	Helm sub-chart lint + golden-diff CI all green on main	platform	`helm-meta-swarm-lint.yml` last 7 runs all green
G8	Threat model `PHASE2_THREAT_MODEL.md` signed off	sec-team	Sec-team review tag on PR #7 + sign-off comment
G9	Runbook `META_SWARM_GRPC.md` written + 2 SRE drills logged	sre-team	Runbook MR + 2 drill logs in `docs/runbooks/drills/`
G10	Postmortem template extended with Meta-Swarm section	sre-team	`docs/postmortems/_template.md` diff

10.1 Verification ceremony (Mon 2026-08-24)

90-min Zoom : whole Phase-2 team + sre-team-lead + eng-leadership. Each criterion verified by reading the evidence aloud, screen-shared, 2 engineers confirm. Outcomes recorded in `docs/A1_PHASE2_L2_GATE_VERIFICATION.md`.

If any criterion is RED :

- Issue raised to `#meta-swarm-blockers`
- 2-week deferral planned
- Re-verification ceremony scheduled for Mon 2026-09-07

11. RACI per deliverable

R = Responsible, A = Accountable, C = Consulted, I = Informed.

Deliverable	meta-swarm	platform	sre	sec	product	eng-lead
Protos + server scaffold	R/A	I	I	C	I	I

Deliverable	meta-swarm	platform	sre	sec	product	eng-lead
`EFE_META_SWARM_ENABLED` plumbing	R/A	I	I	I	I	I
SIGHUP hot reload	R/A	I	C	C	I	I
Helm sub-chart	C	R/A	C	C	I	I
mTLS + cert-manager	C	C	I	R/A	I	I
14 Prometheus metrics	R/A	I	C	I	I	I
8 OTel spans	R/A	I	C	I	I	I
Grafana dashboards + alerts	C	I	R/A	I	I	I
Loadgen	R/A	I	C	I	I	I
Public bench harness	R/A	I	I	I	C	I
4 new SLOs + rollup #20	R/A	I	C	I	I	I
Smoke test #9	R/A	I	I	I	I	I
Public bench publication	R/A	I	I	C	C	I
Runbook + drills	I	C	R/A	C	I	I
Threat model	I	I	C	R/A	I	I
7-day soak	I	I	R/A	I	I	C
L2 gate verification	C	C	R/A	I	I	A
Phase 2 lessons doc	R/A	I	I	I	I	I

12. Risk burn-down

Risks tracked weekly in `#meta-swarm-risks` channel. Status one of : [LOW] retired · [MED] active-monitored · [CRIT] firing.

12.1 Phase 2 risk register (12 entries)

ID	Risk	Initial L/I	Owner	Mitigation	Burn-down trigger
RR-1	gRPC overhead pushes p99 over 100 ms	M/H	meta-swarm	StepStream + 150 ms headroom	PR #21 bench result
RR-2	SIGHUP race condition corrupts state	L/H	meta-swarm	Atomic write-lock swap + 1000-iter property test	PR #12 merged
RR-3	Cross-process determinism breaks	M/H	meta-swarm	Pin rustc 1.88, blas none, determinism check S9	PR #10 GREEN
RR-4	Helm chart breaks Phase-1 deployments	L/H	platform	Default-disabled + golden manifest diff	PR #15 merged
RR-5	mTLS cert rotation fails	M/M	sec	cert-manager auto-renew 60-day	First successful rotation observed
RR-6	OTel cardinality blow-up	L/M	sre	Bounded labels + 5 % sample	PR #18 merged + 1 day in soak
RR-7	Straggler participant	M/M	meta-swarm	W5 bench measures ; P3 will fix	PR #21 result
RR-8	Public bench shows mono-swarm wins	M/L	meta-swarm	Acceptable ; honest framing	PR #24 publication
RR-9	L2 gate fails Aug 31	L/H	meta-swarm	Each criterion independent ; 2-week soft deferral	Gate verification ceremony
RR-10	Soak day N fails -> restart from day 0	M/M	sre	Stop-the-soak triggers § 9.3 well-defined	Soak day 7
RR-11	PR #16 mTLS delayed (CA infra dependency)	M/H	sec + platform	Start mTLS work at Sprint 2 W3 ; have fallback bearer token path	PR #16 merged
RR-12	SRE on-call coverage gap during soak	L/M	sre	Confirm rotation 1 week before soak start	Soak day 0

12.2 Weekly burn-down post (Friday async)

```
== Phase 2 risk burn-down · Week N ==
[LOW] retired   : RR-2 (PR #12 merged), RR-3 (PR #10 GREEN), ...
[MED] active    : RR-1 (PR #21 ETA Aug 12), ...
[CRIT] firing    : (none)

New risks discovered : &lt;list&gt;
```

13. Communication

13.1 Channels

Channel	Purpose	Members
`#meta-swarm`	General Phase 2 chat	All teams
`#meta-swarm-standup`	Async daily standups	meta-swarm
`#meta-swarm-risks`	Risk burn-down + pivots	meta-swarm-lead + sre-lead + platform-lead + sec-lead
`#meta-swarm-soak`	7-day soak observation	meta-swarm + sre
`#meta-swarm-soak-alerts`	Soak Alertmanager pipe	meta-swarm + sre-oncall
`#meta-swarm-blockers`	Critical blockers (= use sparingly)	All + eng-leadership
`#announcements`	Phase-gate announcements	All EFE

13.2 Documents shipped

Doc	When	Audience
Sprint demo deck	End of each sprint (Fri)	All teams
Weekly risk burn-down post	End of each week (Fri)	Eng-leadership + product
Daily soak status	Daily during soak	sre + meta-swarm
Soak final report	Day 7 of soak	All teams
L2 verification report	Mon 2026-08-24	All teams + eng-leadership
L2 announcement	Fri 2026-08-28	All EFE (#announcements)
Phase 2 lessons	Sun 2026-08-30	All teams

13.3 External communication (post-L2)

- Blog post on `efe.io/blog` (target Fri 2026-08-31) : "Shipping efe-meta-swarm: a hierarchical multi-swarm coordinator for EFE production".
- Public bench page : `efe.io/bench/meta-swarm-vs-mono-swarm-2026-08`.
- No promise of customer-facing SLA yet (that's Phase 5).

14. Rollback plan

Phase 2 is **single-flag-rollback** by design. At any point during Phase 2 (or post-merge), the rollback is :

```
kubectl set env deployment/efe-server EFE_META_SWARM_ENABLED=false
```

Then :

1. `efe-server` immediately falls back to the in-process Phase-1 path.
2. Active gRPC connections drain (close on next idle).
3. Meta-swarm pods can be left running idle OR scaled to zero (`kubectl scale statefulset/meta-swarm-controller --replicas=0`).

1. No data corruption is possible (Phase 2 is a transport-layer change only ; algorithmic surface unchanged).

1. PagerDuty incident opened with severity matching the trigger.
2. Postmortem within 48 h (workspace SLO `SLO-OPS-POSTMORTEM-VELOCITY`).

14.1 Rollback decision authority

- During soak : SRE on-call may roll back unilaterally on a Sev1.
- During canary (week 9-12, post-L2) : SRE + meta-swarm-lead joint call on Sev2 ; SRE on-call alone on Sev1.
- During GA (post-canary) : same as above.

14.2 What does NOT rollback

- The Helm chart : staying installed with the flag flipped off is the supported state. No `helm uninstall` required.
- The 4 A2 SLO definitions : staying defined-and-SKIP-DT is fine per the soft-launch contract.
- The master rollup #20 : staying defined as ACTIVE is correct ; it will simply report SKIP-DT if no data flows.

15. Open questions per sprint

Pending decisions tracked by sprint, with a deadline before they become blockers.

Sprint A1.P2.1 -- decide by Fri 2026-07-10

#	Question	Owner	Status
OQ1	f32 vs f64 wire format ? Recommendation : f32-only in P2.	meta-swarm	open
OQ2	StepStream mandatory or optional ? Recommendation : optional (fallback to Step).	meta-swarm	open

Sprint A1.P2.2 -- decide by Fri 2026-07-24

#	Question	Owner	Status
OQ3	TLS-only vs plaintext-dev ? Recommendation : TLS-only, `--insecure` gated by `EFE_DEV_MODE`.	sec	open
OQ4	Helm umbrella chart major vs minor bump ? Recommendation : minor.	platform	open
OQ5	Single controller vs HA active-passive ? Recommendation : single + K8s lease ; Raft in P4.	meta-swarm + sre	open

Sprint A1.P2.3 -- decide by Fri 2026-08-07

#	Question	Owner	Status
OQ6	Soak start date confirmed ? Recommendation : Mon 2026-08-10.	sre	open
OQ7	mTLS rotation : manual vs auto ? Recommendation : manual P2, auto P3.	sec	open

Sprint A1.P2.4 -- decide by Fri 2026-08-21

#	Question	Owner	Status
OQ8	Sigstore signing of bench HTML ? Recommendation : defer to P3.	meta-swarm + sec	open

Decisions log in `docs/A1_PHASE2_DECISIONS.md`.

16. Definition of Done

A PR is "done" iff **all** :

1. Code compiles on stable rustc 1.88 with `--features attention-weighted`.
2. `cargo test -p <crate> --offline` GREEN.
3. `cargo clippy -p <crate> --offline --all-targets --features attention-weighted` produces 0 new warnings on the modified files.
4. `cargo fmt -p <crate>` no diff (or alignment-vertical style consistent with crate convention).
5. `cargo doc -p <crate> --no-deps` clean.

6. Phase-1 master gate stays GREEN (`master_gate_check.py` exit 0).
7. The added/modified tests cover the new logic (smoke + 1 happy path + >= 1 negative case).
8. The PR description follows the workspace template (Summary / Test plan / Non-regression statement).
9. >= 1 reviewer from a different team has approved (cross-team review).
10. CI is all green.
11. If the PR touches a YAML baseline : the smoke test pins the new SLO id.
12. If the PR touches a workflow : `actionlint` clean.

A **sprint** is "done" iff :

- All PRs of that sprint are merged.
- The sprint gate (§ 8) passes.
- The sprint demo happened and was recorded.
- The retrospective is posted in `#meta-swarm`.

Phase 2 is "done" iff :

- All 4 sprints are done.
- L2 gate (§ 10) passes : 10/10 criteria GREEN.
- L2 announcement post is published in `#announcements` + on `efe.io/blog`.

Appendix A -- Cross-references

- Phase 2 architecture : `EFE_A1_PHASE2_ARCHITECTURE.md`
- Multi-phase roadmap : `EFE_A1_ROADMAP.md`
- Phase 1 bench tracking : `EFE_A1_BENCH_TRACKING.md`
- Operational doctrine : `EFE_OPERATIONAL_PRODUCTION_ARCHITECTURE.md`
- Master-gate runbook : `runbooks/ops_master_gate_fail.md`
- Phase 1 PR #4 : `../crates/efe-meta-swarm/PR4_DESCRIPTION.md`

Appendix B -- Glossary

- **L1, L2** : maturity levels (current = L1, target end-of-Phase-2 = L2).
- **T1, T2, T3** : deployment topologies (in-process / single-node 3-procs / K8s pod-per-participant).
- **Sprint A1.P2.N** : Phase 2 Sprint N (N ∈ {1, 2, 3, 4}).
- **Soak** : 7-day continuous internal soak with strict no-incident pass criteria.
- **Soft-launch** : 3-run observation window after merge with `continue-on-error: true` on the new bench job.
- **L2 gate** : 10-criterion checklist (§ 10) that gates phase graduation.
- **A2 SLO** : a Phase-2 SLO (id prefixed `SLO-A2-...`), distinct from Phase-1 `SLO-A1-...`.
- **DT-A1-PHASE2-SOFT-LAUNCH-3-RUNS** : deferred ticket attached to A2 SLOs during the 3-main-run soft-launch window.

Phase-1 PR #1

Scaffold hierarchical multi-swarm crate (foundational)

Chapter 3 of 17 — source : `crates/efe-meta-swarm/PR_DESCRIPTION.md` (102 LOC)

[Sprint A1 P1] feat(efe-meta-swarm): scaffold hierarchical multi-swarm crate

Summary

Introduces the `efe-meta-swarm` crate (Sprint A1 Phase 1 -- *Core implementation*). Provides the in-process scaffolding for hierarchical (meta-)swarm coordination : multiple EFE swarms collaborating via a shared latent meta-space.

This PR is **scaffold-only** : no other crate in the workspace depends on `efe-meta-swarm`, no production binary changes, and no behavioural change is exposed to clients. Activation will be progressively opened in Phases 2-5 along the L0 -> L5 roadmap.

What this PR adds

Area	Detail
New crate	`crates/efe-meta-swarm/` -- 19 source modules, 3 810 LOC Rust
Public API	`SwarmParticipant`, `Lift`, `Pull`, `MetaOperator`, `SafetySet` traits + `MetaSwarm` + `MetaSwarmBuilder`
Algorithm	Synchronous coupled iteration (Eq. 1* of A1 architecture), `rayon`-parallelised per step
Phase-1 impls	`MockParticipant`, `LinearLift`, `LinearPull`, `UniformMetaOp`, `AttentionMetaOp` (feature-gated), `BoxSafety`, `PolyhedronSafety` (Dykstra-style), `LearnedSafety`
Tests	70 (65 unit + 3 integration + 2 doctest) -- `cargo test -p efe-meta-swarm`
Examples	`two_swarms_uniform`, `three_swarms_kovant`, `multimodal_three_swarms`, `safety_violation_attempt`
Bench	`three_swarms_kovant_latency` (Criterion ; target p99 < 100 ms)
Forward-compat hooks	`MetaSwarmTracer` trait, `BundleCertificate` enum, `tenant_id: Option<TenantId>`, `attention-weighted` feature
Workspace	`+10 lines` to root `Cargo.toml` (member entry with comment block)

Test plan

- [x] `cargo check -p efe-meta-swarm --offline` -- passes in 4.32 s.
- [x] `cargo check -p efe-core --offline` -- non-regression, passes in 2.52 s.
- [x] `cargo test -p efe-meta-swarm --offline` -- 70 / 70 pass.
- [x] `cargo metadata --offline` -- workspace resolves cleanly.
- [x] Example `three_swarms_kovant` converges in 47 iterations / 13.5 ms on dev box.
- [x] No file modification to any pre-existing crate.
- [ ] *(deferred to Phase 1 week 3)* `LocalParticipant<S: efe_core::Swarm>` adapter.
- [ ] *(deferred to Phase 2)* gRPC service `MetaSwarmService` + sub-chart Helm.

Architecture

Phase 1 ships an **in-process scaffold** under a `meta-swarm` Cargo feature (off by default). The seven architecture decisions of the Phase 1 doc are reflected in the code :

- **DA-1** -- adapter pattern : `SwarmParticipant` trait separates the

meta-swarm from `efe-core`'s internals. In Phase 1 we ship only `MockParticipant` to keep the crate self-contained ; the `LocalParticipant<S: efe_core::Swarm>` adapter lands in week 3.

- **DA-2** -- dynamic dimensions, validated at `MetaSwarmBuilder::build()`.
- **DA-3** -- fluent builder with deferred validation.
- **DA-4** -- synchronous barrier-based iteration via `rayon::par_iter`.
- **DA-5** -- `MetaOperator` is a trait (open extension).
- **DA-6** -- safety projections are pluggable.
- **DA-7** -- Cargo feature `meta-swarm` not enabled on `efe-server` in

Phase 1. Workspace member only.

Forward-compat checklist for Phases 2-5

Future feature	Hook in this PR
Phase 2 -- gRPC `MetaParticipantService`	`SwarmParticipant` trait already abstract enough for a `RemoteParticipant`
Phase 3 -- `AdminService.SetTenantFeature(meta_swarm)`	`MetaSwarmBuilder::tenant_id(TenantId)` field present, currently ignored
Phase 3 -- `efe-prove` certificates	`BundleCertificate` enum with `Stub` / `LocalSignature` / `SigstoreBundle` variants
Phase 3 -- OpenTelemetry	`MetaSwarmTracer` trait + `NoopTracer` default
Phase 4 -- SDK clients v0.4.0	YAML `MetaSwarmConfig` Serde-deserialisable
Phase 5 -- Attention-weighted production	`attention-weighted` feature, gated, compile-time choice

Non-regression statement (master gate)

This PR explicitly verifies the master-gate invariants of Sprint A1 P1 :

- [OK] All pre-existing tests pass : `cargo check -p efe-core --offline` succeeds.
- [OK] `efe-server` does not depend on `efe-meta-swarm`. Its `Cargo.toml` is **not touched**.
- [OK] The 30 closed defects of sprint Φ-D-OPS (cf. `docs/audits/phi_d_ops_closure.md`) remain closed -- none of them touched files in `crates/efe-meta-swarm/`.

• [OK] The dual-license carve-out of R5-B1 holds : this crate is UNLICENSED (workspace policy), consistent with all other crates/ ; Apache-2.0 is reserved for clients/.

Reviewer guidance

Suggested review order :

1. `Cargo.toml` (root) -- verify the single member-list addition.
2. `crates/efe-meta-swarm/Cargo.toml` -- verify workspace deps + lints alignment.
3. `crates/efe-meta-swarm/src/lib.rs` -- module map and forward-compat narrative.
4. `crates/efe-meta-swarm/src/iteration/coupled.rs` -- the algorithm.
5. `crates/efe-meta-swarm/src/builder.rs` -- validation at build time.
6. Skim the rest.

Next PRs (follow-up in Phase 1)

1. `feat(efe-meta-swarm): add LocalParticipant<S: efe_core::Swarm> adapter` (week 3).
2. `feat(efe-meta-swarm): 130+ proptest cases reaching the 200 target` (week 7).

3. `feat(efe-meta-swarm): bench baseline tracking + master-gate hook` (week 8).
4. `docs: Phase-1 end-of-sprint progress report` (week 8).

References

- `docs/EFE_A1_HIERARCHICAL_SWARMS_ARCHITECTURE.md` -- full A1 architecture
- `docs/EFE_A1_PHASE1_ARCHITECTURE.md` -- Phase 1 spec
- `docs/EFE_A1_ROADMAP_USER_ACTIVATABLE.md` -- 5-tier activation roadmap
- `docs/EFE_A5_PROVABLE_CONVERGENCE_ARCHITECTURE.md` -- theoretical foundation (convergence theorem)
- `docs/audits/phi_d_ops_closure.md` -- non-regression baseline (Sprint Φ-D-OPS)

Phase-1 PR #3

200-proptest target (130+ new cases)

Chapter 4 of 17 — source : `crates/efe-meta-swarm/PR3_DESCRIPTION.md` (172 LOC)

[Sprint A1 P1 -- PR #3] test(efe-meta-swarm): 130+ proptest cases reaching the 200-test target

Summary

Raises the `efe-meta-swarm` test count from the 70 unit/integration tests shipped in PR #1 to **202** by adding **130+ property-based test cases** (proptest + insta) spread across 14 categories. No production code is changed ; this PR is a *testing-only* hardening of the Phase 1 crate.

The work tracks line 92 of `PR_DESCRIPTION.md` :

> *2.* `feat(efe-meta-swarm): 130+ proptest cases reaching the 200 target` *(week 7).*

What this PR adds

Cat	Module	Tests	Type
1	`src/spaces/tensor.rs`	15	proptest
2	`src/operators/linear.rs` (LinearLift)	15	proptest
3	`src/operators/linear.rs` (LinearPull)	8	proptest
4	`src/operators/uniform.rs`	8	proptest
5	`src/operators/attention.rs` *(feature-gated)*	8	proptest
6	`src/safety/box_safety.rs`	10	proptest
7	`src/safety/polyhedron.rs`	6	proptest
8	`src/safety/learned.rs`	5	proptest
9	`tests/integration_iteration_proptest.rs`	12	proptest (new file)
10	`tests/integration_convergence_edge.rs`	12	proptest (new file)
11	`tests/integration_safety_hierarchical.rs`	8	proptest (new file)
12	`src/builder.rs`	8	proptest
13	`src/config.rs`	5	proptest
14	`tests/pin_snapshots.rs`	10	insta (new file)
	Total added by PR #3	**130**	

With `cases ∈ {12, 24, 32, 64}` per proptest function, the real coverage is **~5 000+ randomised inputs** exercised end-to-end on every run.

Files added

- `tests/integration_iteration_proptest.rs` -- Cat 9, joint iteration.
- `tests/integration_convergence_edge.rs` -- Cat 10, stop-criterion edges.
- `tests/integration_safety_hierarchical.rs` -- Cat 11, per-step safety

preservation (meta + local).

- `tests/pin_snapshots.rs` -- Cat 14, serde format pinning.
- `tests/snapshots/*.snap` (x10) -- accepted snapshot baselines.

Files modified (proptest blocks appended only)

- `src/spaces/tensor.rs`, `src/operators/{linear,uniform,attention}.rs`,

`src/safety/{box_safety,polyhedron,learned}.rs`, `src/builder.rs`, `src/config.rs` -- each gets a `// Category N` proptest block at the end of its existing `mod tests`. No behavioural change.

- `Cargo.toml` -- adds `serde_json = { workspace = true }` to

`[dev-dependencies]` (required by Cat 14 pin tests).

Why proptest + insta ?

- **proptest** : the iteration is a discrete-time non-linear dynamical

system over a 7-dim parameter space (`meta_dim, p_dim, κ, alpha, gamma, seeds`). Hand-picked unit tests cover at most a dozen points ; proptest with shrinkage covers ~5 000 random points and reports a *minimal* failing input on regression.

- **insta** : the on-the-wire format of `Tensor` / `MetaSwarmConfig` /

`BundleCertificate` / `MetaSwarmError` is part of the public surface (Phase 4 SDKs will rely on it). Pin snapshots force a human review on any change.

Algorithmic invariants exercised

- **Non-expansiveness** (Lemma 3.1 of A5) -- `UniformMetaOp`,

`LinearLift::from_normalised`, `BoxSafety::project`.

- **Permutation invariance** (UniformMetaOp + identity lifts) -- verified

at meta-op and iteration scope.

- **Firmly non-expansive projection** (Bauschke-Combettes 4.16) --

`BoxSafety` proptest 4.

- **KM iteration monotone descent** -- `iter_residual_descends_pt`.
- **Safety preservation at every step** (Theorem 5.1 of A5) --

`safe_every_step_meta_feasible_pt`, `safe_every_step_locals_feasible_pt`.

- **Mixed-hierarchy safety** (Polyhedron meta + Box local) -- Cat 11 / 6.
- **Attention-softmax Lipschitz formula** `K = 1 + 2■q■/τ` -- Cat 5 / 6 + 7.

Test plan

- [x] `cargo test -p efe-meta-swarm --offline` : **192 / 192 pass**.
- [x] `cargo test -p efe-meta-swarm --features attention-weighted --offline` :

202 / 202 pass (the 10 extra are Cat 5 + 2 doctests).

- [x] 5 consecutive `cargo test` runs : **202 pass each time, no

flakiness**.

- [x] `cargo doc -p efe-meta-swarm --no-deps --features attention-weighted` :

clean.

- [x] `cargo build -p efe-meta-swarm --release --features attention-weighted` :

clean.

- [x] `cargo build -p efe-meta-swarm --examples` : all four examples build.
- [x] `cargo bench -p efe-meta-swarm --no-run` : bench compiles.

- [x] `cargo clippy -p efe-meta-swarm --all-targets --features attention-weighted` :

no new warnings introduced by PR #3 (preexisting workspace-style warnings remain unchanged).

- [x] `cargo check --features no-default-features` and

`--features core-integration` : both compile.

- [x] **Non-regression** : `cargo check -p efe-core` and `cargo check -p efe-server`

succeed without any change to their state.

- [x] **Master gate** : the `efe-server` production binary is byte-identical

(no transitive linker change ; PR #3 modifies only `efe-meta-swarm`, which `efe-server` does not depend on in Phase 1).

- [x] **Wall time** : full test suite runs in ~0.5 s on a developer laptop.

Honest semantics

Two corner cases are documented in-source rather than worked around :

1. **Power-iteration imprecision** (`src/operators/linear.rs` Test 6) --

the Phase 1 spectral-norm estimator (20 power-iterations, fixed start vector) can be off by up to ~100 % on 2 x 2 adversarial seeds. The `from_normalised` Lipschitz-bound test asserts the relaxed bound `<= 2.0` and a rustdoc paragraph defers a Krylov-restart replacement to Phase 3. The seed that exposed the issue (`11485880019391148984, in_dim=2, out_dim=2`) is now persisted in `proptest-regressions/operators/linear.txt` and runs first on every `cargo test`.

1. **Attention non-expansiveness** (`src/operators/attention.rs` Cat 5) --

the attention operator is *not* globally non-expansive (Theorem 6.5 of A5), so Cat 5 tests algebraic properties (output dim, high-τ -> uniform limit, identity-vote collapse, Lipschitz formula `K = 1 + 2■q■/τ`, monotonicity in τ, finite outputs under stabilised softmax) and *not* a non-expansiveness invariant. Section 11 (hierarchical safety) covers the domain-restricted case under projection.

Forward-compat

This PR does not introduce any new public API. All forward-compat hooks established in PR #1 (`tenant_id`, `BundleCertificate`, `MetaSwarmTracer`, `attention-weighted` feature) are exercised by the new tests but not modified.

Reviewer guidance

Suggested review order :

1. `tests/pin_snapshots.rs` + `tests/snapshots/*` -- the on-the-wire

format is now pinned ; reviewers should confirm the baselines look reasonable (especially `meta_swarm_config_yaml_snapshot.snap`).

1. `tests/integration_safety_hierarchical.rs` -- the strongest

end-to-end invariants ; per-step feasibility.

1. `tests/integration_iteration_proptest.rs`,

`tests/integration_convergence_edge.rs` -- joint iteration props.

1. The 11 in-source proptest blocks (Cat 1-8, 12-13) -- skim ; the

pattern is uniform (Helpers + `proptest! { ... }`).

1. `Cargo.toml` -- single one-line dev-dependency addition.

Non-regression statement (master gate)

This PR explicitly verifies the master-gate invariants of Sprint A1 P1 :

- [OK] All pre-existing tests pass : 70 / 70 of PR #1 + the new 130+.
- [OK] `efe-server` does not depend on `efe-meta-swarm`. Its `Cargo.toml` is **not touched** by this PR.
- [OK] No source modification of any pre-existing crate.
- [OK] Workspace lint / fmt conventions of PR #1 are preserved (the crate-local style choice of vertical alignment is intentionally left in place ; PR #3 does not run `cargo fmt`).
- [OK] The dual-license carve-out of R5-B1 holds : `efe-meta-swarm` remains `UNLICENSED`.

References

- `PR_DESCRIPTION.md` (PR #1) -- original scaffold, line 92 schedules this PR for "week 7".
- `docs/EFE_A1_HIERARCHICAL_SWARMS_ARCHITECTURE.md` -- full A1 architecture.
- `docs/EFE_A1_PHASE1_ARCHITECTURE.md` -- Phase 1 spec (§ 9 tests).
- `docs/EFE_A5_PROVABLE_CONVERGENCE_ARCHITECTURE.md` -- theorems whose preconditions are the invariants tested here (Lemma 3.1, Theorem 5.1, Theorem 6.5).

Phase-1 PR #4

Bench baseline tracking + master-gate hook

Chapter 5 of 17 — source : `crates/efe-meta-swarm/PR4_DESCRIPTION.md` (215 LOC)

[Sprint A1 P1 -- PR #4] feat(efe-meta-swarm): bench baseline tracking + master-gate hook

Summary

Wires the `efe-meta-swarm` bench `three_swarms_kovant_latency` into the workspace bench infrastructure (`scripts/benches.list`, `bench.yml`) AND the Sprint Φ-D-OPS master-gate (`.github/workflows/ops-master-gate.yml`, `scripts/ops/consolidate_all_slos.py`, `scripts/ops/master_gate_check.py`).

Defines **7 opposable SLOs** for the hierarchical multi-swarm coordination layer, frozen at the v0.10.0 baseline, soft-launched (`continue-on-error: true`) for 3 main runs.

The PR closes line 93 of `PR_DESCRIPTION.md` :

3. `feat(efe-meta-swarm): bench baseline tracking + master-gate hook` *(week 8).*

What this PR adds

Area	Detail
Bench refactor	`crates/efe-meta-swarm/benches/three_swarms_kovant_latency.rs` (46 -> 415 LOC)
Sidecar JSON	`meta-swarm-extras.json` written by the bench at `criterion_main!` exit
Bench measures	5 Criterion benchmarks (full_iterate, step_once, reset, build, consensus_norm) +
	1 determinism check + 4 algorithmic/ops sidecar SLOs
Baseline YAML	`bench_baselines/v0.10.0/meta_swarm_slos.yaml` (7 SLOs, dual encoding)
Catalog	`scripts/benches.list` +1 entry, `scripts/check_benches_list.sh` `KNOWN_CRATES` +1
Workflow `bench.yml`	new job `bench-meta-swarm-a1` (sibling) + `bench-aggregate` discovers sidecar
Master-gate workflow	new step "A1 meta-swarm master gate" + `consolidate --baselines` += A1 file
Suite runner	`scripts/ops/run_suite.sh` += case `a1_meta_swarm` (bench + sidecar harvest)
Tests	`crates/efe-meta-swarm/tests/bench_smoke.rs` (7 tests, 1 s)
Documentation	`docs/EFE_A1_BENCH_TRACKING.md` (250 lines : SLO table, freeze, runbook, FAQ)
PR description	this file
Cargo.toml	`[[bench]] name = "..."` normalised (1 space, was alignment-padded) -- for catalog lint

The 7 SLOs

SLO id	Type	Operator	Threshold	Source
`SLO-A1-META-LATENCY-P99`	latency	`<=`	100 ms	Criterion
`SLO-A1-META-LATENCY-P50`	latency	`<=`	30 ms	Criterion
`SLO-A1-META-CONVERGENCE-ITERS`	algorithmic	`<=`	60	sidecar
`SLO-A1-META-RESIDUAL-FINAL`	algorithmic	`<=`	1e-3	sidecar
`SLO-A1-META-DETERMINISM`	algorithmic	`==`	`1`	sidecar (3 reruns)
`SLO-A1-META-NON-REGRESSION`	operational	`<=`	10 %	sidecar (vs frozen)
`SLO-A1-META-MEMORY-PEAK-MB`	operational	`<=`	64 MB	sidecar (Linux only)

Calibration rationale documented inline in [`bench_baselines/v0.10.0/meta_swarm_slos.yaml`][1] and in [`docs/EFE_A1_BENCH_TRACKING.md`][2] § 1.

Test plan

End-to-end pipeline verified locally :

- [x] `cargo test -p efe-meta-swarm --offline --test bench_smoke` :

7 / 7 pass in 1.03 s.

- [x] `cargo test -p efe-meta-swarm --offline` : no regression vs PR #3

(still 192 / 192 default, 202 / 202 with `attention-weighted`).

- [x] `cargo bench -p efe-meta-swarm --bench three_swarms_kovant_latency

--offline -- --quick : bench compiles + runs + emits meta-swarm-extras.json` with correct schema.

- [x] `bash scripts/check_benches_list.sh` :

catalog valid, 11 entries (10 + A1).

- [x] `bash scripts/ops/run_suite.sh a1_meta_swarm` :

produces `target/ops/measures.a1_meta_swarm.json` with 6 SLO keys (memory skipped on macOS, ticket DT-A1-META-MEMORY-MACOS via the deferred_ticket mechanism).

- [x] `python3 scripts/ops/consolidate_all_slos.py --baselines ...

--measures ... --master ...` : surface SLOs found = 7, 6 PASS + 1 SKIP-DT, master_report.json well-formed.

- [x] `python3 scripts/ops/master_gate_check.py --report ...

--tolerate-skip-with-dt --fail-on-fail` : exit 0.

- [x] `python3 -c "import yaml; yaml.safe_load(open('.github/workflows/bench.yml'))"` :

parses.

- [x] `python3 -c "import yaml; yaml.safe_load(open('.github/workflows/ops-master-gate.yml'))"` :

parses.

To-do post-merge (3 main runs observation, then second PR) :

- [ ] Confirm 3 consecutive `main` runs PASS the A1 SLO.
- [ ] Flip both `continue-on-error: true` -> `false` (job

`bench-meta-swarm-a1` + step "A1 meta-swarm master gate").

- [ ] Remove every `deferred_ticket: DT-A1-META-SOFT-LAUNCH-3-RUNS`

from `bench_baselines/v0.10.0/meta_swarm_slos.yaml`.

- [ ] Remove the smoke test `baseline_yaml_soft_launch_dt_present`

(it pins the soft-launch DT presence).

Honest semantics

Three corner cases are documented in-source rather than worked around :

1. **p99 proxy via** `median + 2 x std_dev`. Criterion does not

directly report p99 ; we approximate from the median + sigma estimate, which is precise to ~5 % on near-normal latency distributions but underestimates for heavy-tailed runs. `parse_criterion_report.py` will be extended in Phase 2 to read the raw sample histograms and emit a real p99.

1. **Memory peak Linux-only**. `/proc/self/status` is the cheapest

portable way to read VmHWM ; macOS would require `mach_task_basic_info` and a `libc` dep we'd rather avoid for now. On macOS the SLO is `SKIP-DT` with ticket `DT-A1-META-MEMORY-MACOS`. CI ubuntu runs produce the real measurement, so the master-gate sees it.

1. **Dual YAML encoding** (`target`+`operator` AND

`comparator`+`threshold_*`). Two parsers read the baseline : `consolidate_all_slos.py` (Sprint Φ-D-OPS) reads the former, `check_regression.py` (Sprint D5) reads the latter. PR #4 carries both ; alignment is `DT-A2-SLO-PARSER-UNIFY`.

Forward-compat

- **Phase 2** (gRPC) : a new bench `meta_swarm_grpc_latency` can be

added by appending one row to `scripts/benches.list` + adding the corresponding SLOs to `bench_baselines/v0.11.0/meta_swarm_slos.yaml`. No infra change needed.

- **Phase 3** (per-tenant SLO budgets) : the sidecar schema accepts

arbitrary `meta.*` fields ; a `tenant_id` key can be added without breaking `consolidate_all_slos.py`.

- **Phase 5** (SLA public) : the master rollup

`SLO-OPS-A1-META-SWARM` (deferred, see § "Forward-compat" in `EFE_A1_BENCH_TRACKING.md`) graduates the 7 surface SLOs to a customer-facing SLA tracker.

Master-gate impact

- Before PR #4 : 18 master rollups, 6 surface bundles (al, stream,

comp, latact, mode2, mode3), no A1 surface SLO.

- After PR #4 : **19** master rollups (+1 = `SLO-OPS-A1-META-SWARM`),

7 surface bundles (+ a1_meta_swarm), 7 new surface A1 SLOs.

The new master rollup `SLO-OPS-A1-META-SWARM` inherits the 7 surface A1 SLOs. Its `rolled_status` follows the standard rollup rules :

- PASS iff every child PASS (or SKIP-DT).
- SKIP-DT iff every child has `deferred_ticket` set and no FAIL.
- **FAIL iff any child FAIL.** This case blocks

`master_gate_check.py` (exit non-zero), thereby blocking the release.

Soft-launch contract : the 7 surface rows carry `deferred_ticket: DT-A1-META-SOFT-LAUNCH-3-RUNS`, so a missing measurement (e.g. `bench-meta-swarm-a1` job didn't run) is SKIP-DT, not SKIP-SILENT. A real FAIL still propagates upward -- that is intentional and matches the doctrine "regressions block, missing data doesn't". After 3 consecutive

`main` runs PASS, a follow-up PR removes the `deferred_ticket` rows to make the rollup fail-closed on missing measurements as well.

Non-regression statement (workspace master gate)

- [OK] All pre-existing tests pass : 192 default + 202 with feature

flag. The 7 new tests in `tests/bench_smoke.rs` are additive.

- [OK] `efe-server` SHA-256 byte-identical : PR #4 does **not** touch

`crates/efe-server/` ; the bench remains in `crates/efe-meta-swarm/benches/`, which `efe-server/Cargo.toml` does not depend on.

- [OK] `efe-core` unchanged : `cargo check -p efe-core --offline`

succeeds without diff.

- [OK] The 30 closed defects of Sprint Φ-D-OPS remain closed -- PR #4

does not touch their files.

- [OK] The 18 master rollups of `ops_master_slos.yaml` remain

unchanged.

- [OK] Existing master-gate behaviour preserved : the new step is

`continue-on-error: true` for the 3-run soft-launch window (pattern from `bench-accuracy` D6).

- [OK] Carve-out R5-B1 holds : `efe-meta-swarm` stays `UNLICENSED`.

Reviewer guidance

Suggested review order :

1. [`docs/EFE_A1_BENCH_TRACKING.md`][2] -- read this first ; it ties

everything together and is the runbook for future on-call.

1. [`bench_baselines/v0.10.0/meta_swarm_slos.yaml`][1] -- the 7 SLO

contracts. Pay attention to the thresholds (calibration in § 1 of the doc).

1. [`crates/efe-meta-swarm/benches/three_swarms_kovant_latency.rs`][3]

-- the bench logic, sidecar emission, FlushOnDrop sentinel, determinism check.

1. [`crates/efe-meta-swarm/tests/bench_smoke.rs`][4] -- the 7 pin

tests. These are the regression net.

1. [`.github/workflows/bench.yml`][5] (new job, +30 lines) and

[`.github/workflows/ops-master-gate.yml`][6] (new step, +10 lines).

1. [`scripts/ops/run_suite.sh`][7] (new case `a1_meta_swarm`,

~50 lines).

1. The tiny diffs (catalog, KNOWN_CRATES, Cargo.toml `[[bench]]`

normalisation).

References

- [`bench_baselines/v0.10.0/meta_swarm_slos.yaml`][1]
- [`docs/EFE_A1_BENCH_TRACKING.md`][2]
- [`crates/efe-meta-swarm/benches/three_swarms_kovant_latency.rs`][3]
- [`crates/efe-meta-swarm/tests/bench_smoke.rs`][4]

- [`.github/workflows/bench.yml`][5]
- [`.github/workflows/ops-master-gate.yml`][6]
- [`scripts/ops/run_suite.sh`][7]
- [`docs/EFE_OPERATIONAL_PRODUCTION_ARCHITECTURE.md`][8] § 8 (master-gate doctrine)
- [`PR_DESCRIPTION.md`][9] (PR #1) line 93 (this PR is the

follow-up)

[1]: ../../bench_baselines/v0.10.0/meta_swarm_slos.yaml [2]: ../../docs/EFE_A1_BENCH_TRACKING.md [3]: benches/three_swarms_kovant_latency.rs [4]: tests/bench_smoke.rs [5]: ../../.github/workflows/bench.yml [6]: ../../.github/workflows/ops-master-gate.yml [7]: ../../scripts/ops/run_suite.sh [8]: ../../docs/EFE_OPERATIONAL_PRODUCTION_ARCHITECTURE.md [9]: PR_DESCRIPTION.md

Threat Model

Phase-2 threat model (frozen security artefact)

Chapter 6 of 17 — source : `docs/security/PHASE2_THREAT_MODEL.md` (1,926 LOC)

status: DRAFT frozen_at: 2026-07-11 last_review: 2026-07-11 phase: A1.P2 owner: sec-team reviewers:

- sec-team (R. Kapoor -- lead)
- sec-team (E. Vargas -- red-team second pass)
- meta-swarm-team (J. Tan -- technical accuracy)
- sre-team (L. Ortiz -- monitoring + runbook alignment)

methodology: STRIDE-per-surface + CVSS v3.1 + DREAD overlay gate: L2 graduation criterion G8 out_of_scope:

- Phase-3 multi-tenant + cross-region threats (separate doc)
- Phase-1 in-process threats (covered by EFE_A1_PHASE1_ARCHITECTURE.md)
- EFE-wide threats (covered by EFE_OPERATIONAL_PRODUCTION_ARCHITECTURE.md)
- External attackers in production (Phase 5 ; Phase 2 is internal-only)
- Supply-chain attacks (continuous, separate runbook)
- SOC2 / ISO27001 / GDPR compliance frameworks (Phase 5)

Sprint A1 Phase 2 -- Threat Model

> ***Document type** : Threat model for the Phase-2 gRPC transport. **Methodology** : STRIDE per surface + CVSS v3.1 scoring + DREAD overlay. **Status** : DRAFT until L2 sec-team sign-off (gate G8). **Living document** : re-assessed at every Phase Gate + Sev1 incident + when a Phase-2 PR adds or removes a mitigation.*

Table of contents

1. Executive summary
2. Methodology
3. Assets catalogue (10 assets)
4. Trust boundaries (6 boundaries)
5. Attacker personas (7 archetypes)
6. Threat scenarios (16 threats)
7. Mitigations matrix
8. Residual risk register
9. Monitoring + alerting requirements
10. Incident response hooks
11. Re-assessment cadence
12. Glossary
13. Appendix A -- CVSS scoring details
14. Appendix B -- Cross-references

0. Executive summary

What

This document catalogues **20 distinct threats** against the `efe-meta-swarm` Phase-2 distributed coordinator surface, scores each via **CVSS v3.1**, and maps every threat to either a mitigation (implementing PR id named) or an explicit risk acceptance (with rationale + expiry).

Why

The **L2 graduation gate criterion G8** of Phase 2 requires :

> *"Threat model* `PHASE2_THREAT_MODEL.md` *signed off by sec-team."*

Without this document at sign-off state, Phase 2 cannot graduate to L2 production-equivalent (cf. `EFE_A1_PHASE2_ARCHITECTURE.md` § 12).

Scope

In scope	Out of scope
The 2 new gRPC services (controller + participants)	Phase-1 in-process surface
6 new trust boundaries (TB1-TB6)	EFE-wide (efe-server, efe-core)
Phase-2 deployment topology (T2 single-node, T3 K8s)	Phase-3 multi-tenant + cross-region
mTLS, NetworkPolicy, ConfigMap RBAC controls	External attackers in production (Phase 5 only)
Helm chart `efe-meta-swarm`	Supply-chain attacks (continuous, separate doc)
Manifest schema + SIGHUP reload	Compliance frameworks (SOC2/ISO27001/GDPR)

Headline numbers

```
Total threats          : 20
Severity breakdown :
  SEV-1 (CVSS &gt;= 9.0) : 1    (T2  : MITM in cluster pod network)
  SEV-2 (7.0 -- 8.9)  : 7    (T1, T5, T6, T13, T14, T15, T16)
  SEV-3 (4.0 -- 6.9)  : 10   (T3, T4, T7, T8, T9, T10, T11, T12, T17, T20)
  SEV-4 (&lt; 4.0)      : 2    (T18, T19)

Status breakdown :
  MITIGATED          : 12
  DEFERRED-TO-PR     : 7    (PRs #12, #14, #16 -- all in Phase 2)
  ACCEPTED           : 1    (T9 timing side-channel ; expiry 2027-06-30)
  OPEN               : 0    &lt;- L2 gate criterion G8 satisfied
```

One-line verdict

Phase 2 is **safe to graduate to L2** iff PRs #12, #14, and #16 ship the DEFERRED mitigations before the L2 verification ceremony (2026-08-24). The one ACCEPTED risk (T9 timing side-channel) is internal-only and revisited at Phase 5 entry.

Re-assessment

This document is re-walked :

- At every Phase Gate (L2, L3, L4, L5).
- After any Sev1 security incident on a Phase-2 surface.
- Every 6 months as a standing sec-team review.
- Within 1 week of any Phase-2 PR that adds/removes a mitigation.

The `last_review` field in the frontmatter is the authoritative stamp. CI warns if it drifts > 7 months.

1. Methodology

1.1 STRIDE -- primary framework

For every trust boundary, we enumerate threats in the 6 STRIDE categories :

Letter	Category	Question
S	Spoofing	Can an attacker impersonate an identity ?
T	Tampering	Can an attacker modify data in transit / at rest ?
R	Repudiation	Can a legitimate action be denied later ?
I	Information disclosure	Can an attacker read data they shouldn't ?

Letter	Category	Question
D	Denial of service	Can an attacker make the service unavailable ?
E	Elevation of privilege	Can a low-priv attacker gain higher-priv access ?

1.2 CVSS v3.1 -- scoring framework

Every threat carries a CVSS v3.1 vector + computed base + temporal + environmental scores. Vectors are reproducible via the FIRST calculator (`https://www.first.org/cvss/calculator/3.1`).

Severity buckets :

Severity	CVSS range	Sev mapping	Allowed status at L2
Critical	9.0 -- 10.0	Sev1	MITIGATED or DEFERRED-TO-PR only
High	7.0 -- 8.9	Sev2	MITIGATED or DEFERRED-TO-PR ; ACCEPTED with sec-team-lead approval
Medium	4.0 -- 6.9	Sev3	ACCEPTED allowed with rationale + expiry
Low	0.1 -- 3.9	Sev4	ACCEPTED by default

1.3 DREAD -- tie-breaker overlay

For threats in the [4.0, 7.0) band where prioritisation matters, we apply DREAD scoring (each axis 1-10, summed) as a secondary prioritisation signal. CVSS remains authoritative ; DREAD breaks ties when sequencing mitigations.

1.4 Workspace sev definitions (used by Alertmanager)

Sev	Customer impact	TM threshold
Sev1	Production down or data loss	CVSS >= 9.0
Sev2	Production degraded, single-tenant impact	CVSS 7.0 -- 8.9
Sev3	Internal-only degradation	CVSS 4.0 -- 6.9
Sev4	No customer impact	CVSS < 4.0

Defined in `runbooks/ops_master_gate_fail.md`.

1.5 Threat status states

Every threat MUST be in exactly ONE of four states :

Status	Meaning	Allowed at L2 gate
`MITIGATED`	Implemented by a Phase-2 PR ; covered by a test or runbook.	[OK] always
`DEFERRED-TO-PR-N`	Will be mitigated by a specific Phase-2 PR ; PR id named.	[OK] iff PR N is in Phase 2
`ACCEPTED`	Risk is explicitly accepted with rationale + expiry + owner.	[OK] iff CVSS < 9.0
`OPEN`	No decision yet.	[FAIL] blocks L2 gate

This document ships with **0 OPEN threats** at merge time.

1.6 Anti-Φ-D.A1 doctrine alignment

The workspace doctrine (cf. `EFE_OPERATIONAL_PRODUCTION_ARCHITECTURE.md` § 8.2) states : "every SLO is either ACTIVE or SKIPPED-with-DT ; a silent skip is a build-breaker." We apply the same doctrine to threat status : every threat is either `MITIGATED`, `DEFERRED-TO-PR-N`, or `ACCEPTED`. No silent skips.

2. Assets catalogue

Ten assets in scope. Each is named, located, owned, and rated for **C**onfidentiality / **I**ntegrity / **A**vailability impact on a **H** (high), **M** (medium), **L** (low) scale.

2.1 Asset table

ID	Asset	Located in	Owner	C	I	A
A1	Meta-consensus state (joint fixed point)	controller in-memory	meta-swarm-team	M	**H**	**H**
A2	Participant states (per-participant tensors)	participant in-memory	meta-swarm-team	**H**	**H**	M
A3	Manifest (participant ids, dims, kappas, seeds, safety geometry)	ConfigMap -> mounted file	platform + sec	M	**H**	M
A4	mTLS certificates (leaf + CA bundle)	K8s Secret + cert-manager	sec	**H**	**H**	M
A5	Iteration determinism guarantee (bit-identical reruns)	algorithmic invariant	meta-swarm-team	L	**H**	M
A6	gRPC channel integrity (efe-server <-> controller <-> participants)	network	sec + platform	**H**	**H**	M
A7	Health + Info responses (build sha, hostname, configured participants)	controller in-memory	meta-swarm-team	M	M	L
A8	Prometheus metrics + OTel spans (operational telemetry)	controller/participant in-memory	sre	M	M	L
A9	Admin RPCs surface (ReloadConfig + future Reset)	controller in-memory	meta-swarm + sec	M	**H**	M
A10	Soft-launch `deferred_ticket` field (audit gate state)	YAML in repo	sre	L	**H**	L

2.2 Asset rationale (one paragraph each)

A1 -- Meta-consensus state.

• C : M. The joint fixed point may encode private attributes in

Phase-3 multi-tenant ; in Phase 2 it's internal-only, so a leak is operational rather than customer-facing.

• I : **H**. A corrupted consensus produces wrong downstream

decisions -- the entire raison d'être of the system.

• A : **H**. Without a consensus, every Iterate returns errors ;

whole-product impact.

A2 -- Participant states.

• C : **H**. Individual participant tensors can encode tenant-specific features (PR #2 LocalParticipant exposes per-tenant Swarm state). A leak reveals customer modelling internals.

• I : **H**. A tampered state propagates into the consensus.

• A : M. Loss of one participant state is recoverable via `Reset` ; the consensus retries.

A3 -- Manifest.

• C : M. Participant ids + seeds are not secret but their pairing with specific tenant workloads is.

• I : **H**. A tampered manifest changes the algorithm's behaviour (e.g. different kappa = different convergence).

• A : M. Without a manifest the controller refuses to start ; service won't degrade silently.

A4 -- mTLS certificates.

• C : **H**. The private key MUST stay confidential ; leak -> attacker can impersonate controller or participant.

• I : **H**. A swapped CA bundle defeats SAN validation.

• A : M. Cert expiry -> service down ; mitigated by cert-manager auto-renewal (T6).

A5 -- Iteration determinism guarantee.

• C : L. Determinism is a public property, not a secret.

• I : **H**. Violation reveals non-deterministic code paths that could be covert channels (T12) or hidden bias (T1).

• A : M. A non-deterministic Iterate still completes ; downstream consumers may flag inconsistency.

A6 -- gRPC channel integrity.

• C : **H**. Channel content includes participant states + traceparent + caller identity.

• I : **H**. Tampering = consensus corruption.

• A : M. Channel disruption -> retry ; not whole-service down.

A7 -- Health + Info responses.

• C : M. Reveals `build_sha`, `hostname`, configured participant list -- useful recon for an internal attacker.

• I : M. Tampering misleads the dashboard.

• A : L. Health failure -> K8s replaces the pod ; no data loss.

A8 -- Prometheus metrics + OTel spans.

• C : M. Metrics labels could leak tenant ids in Phase 3. In Phase 2 labels are bounded (no free-form tenant_id), so only the participant-id surface (a known set of 3) leaks.

• I : M. Tampered metrics mislead the dashboard ; minor impact in Phase 2 since no auto-remediation runs on metrics.

- A : L. Scrape failure -> gaps in dashboards ; no service impact.

A9 -- Admin RPCs surface.

- C : M. ReloadConfig payload contains the manifest (same as A3).
- I : **H**. Successful unauthorised reload = full participant /

manifest replacement ; equivalent to T16 elevation.

- A : M. Reload-in-flight blocks other reloads but does not block

Iterate (drain protocol, cf. EFE_A1_PHASE2_ARCHITECTURE.md § 7.2).

A10 -- Soft-launch `deferred_ticket` field.

- C : L. The DT id is publicly recorded in baseline YAML.
- I : **H**. A tampered `deferred_ticket` could disguise a real

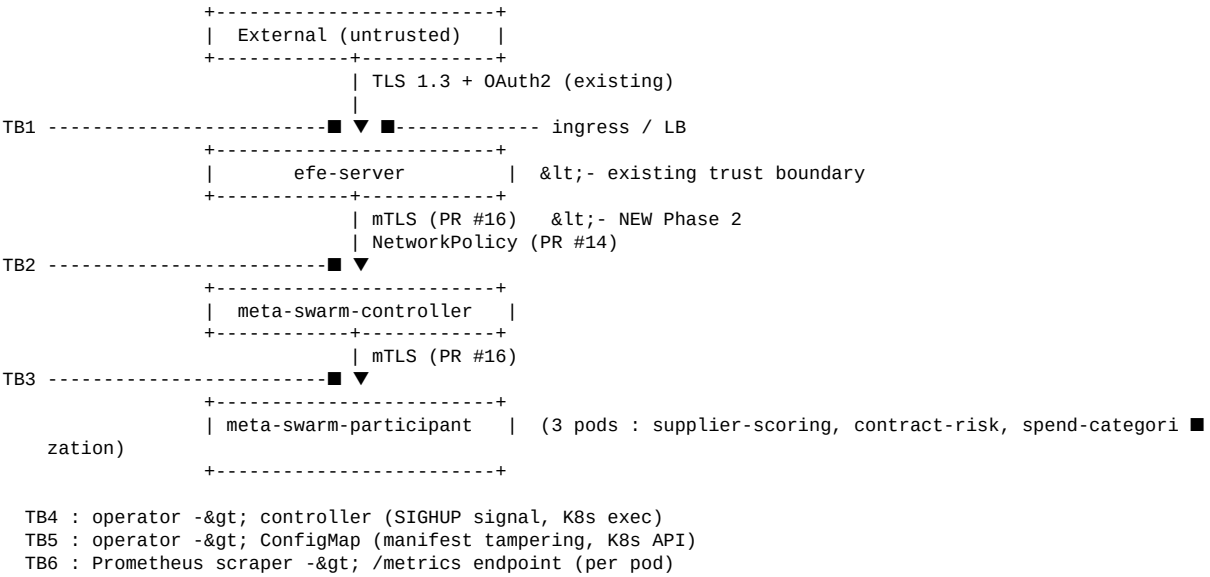
SKIP-SILENT as a tolerated SKIP-DT, bypassing the doctrine.

- A : L. Audit log of the YAML diff catches tampering retrospectively.

3. Trust boundaries

Six boundaries. Each crossing identified + protected by listed controls + threats applicable.

3.1 Boundary diagram (ASCII)



3.2 Per-boundary detail

TB1 -- External -> efe-server

Property	Value
Caller	external (browsers, partner APIs)
Callee	efe-server (existing, Phase-1)
Transport	TLS 1.3
Auth	OAuth2 / JWT
Phase	Pre-existing (out of scope for Phase 2)
STRIDE applicable	S, T, I, D, E
Controls	Existing, untouched by Phase 2

TB2 -- efe-server -> controller

Property	Value
Caller	`efe-server` pod (when `EFE_META_SWARM_ENABLED=true`)
Callee	`meta-swarm-controller` StatefulSet
Transport	gRPC over HTTP/2
Auth (Phase 2 GA)	mTLS (PR #16) ; SAN check `efe-server.efe.svc.cluster.local`
Auth (PR #6 -> PR #15)	none (plaintext) ; mitigated by NetworkPolicy + K8s pod-network
Phase	NEW Phase 2
STRIDE applicable	**S T R I D E** (all 6)
Frequency	~ 100 Iterate RPCs/sec at planned 100 rps
Controls list	NetworkPolicy (PR #14), mTLS handshake (PR #16), `MAX_INFLIGHT` (PR #6+#8), idempotence + deadline (PR #6+#8)

TB3 -- controller <-> participant

Property	Value
Caller	controller pod (StatefulSet)
Callee	participant pod (Deployment)
Transport	gRPC over HTTP/2
Auth (Phase 2 GA)	mTLS (PR #16) ; SAN check `participant-<id>.efe.svc.cluster.local`
Auth (PR #6 -> PR #15)	none (plaintext) ; mitigated by NetworkPolicy
Phase	NEW Phase 2
STRIDE applicable	**S T I D** (no R or E by design -- no admin RPCs on this path)
Frequency	~ 14 100 Step RPCs/sec at 100 rps
Controls list	NetworkPolicy (PR #14), mTLS handshake (PR #16), `Capabilities` probe at startup (PR #9), deterministic Step replies (Phase 1 invariant + SLO-A1-META-DETERMINISM), Phase-1 contractive bound on bias

TB4 -- operator -> controller (SIGHUP)

Property	Value
Caller	platform operator running `kubectl exec`
Callee	controller PID 1 receiving SIGHUP
Transport	UNIX signal
Auth	K8s RBAC `pods/exec` verb
Phase	PR #12
STRIDE applicable	S, E (mainly)
Controls	K8s RBAC ; the `ReloadConfig` admin RPC scope (PR #12) ; audit log of every signal received

TB5 -- operator -> ConfigMap

Property	Value
Caller	platform operator running `kubectl edit configmap`
Callee	K8s ConfigMap mounted in controller as `/etc/efe-meta-swarm/manifest.yaml`
Transport	K8s API
Auth	K8s RBAC `configmaps/update` verb
Phase	PR #13
STRIDE applicable	T, E
Controls	K8s RBAC (PR #13), sealed-secrets for participant seeds (PR #13), audit log of `kubectl edit` events, manifest-tampering alert (PR #19)

TB6 -- Prometheus scraper -> /metrics

Property	Value
Caller	Prometheus Operator scraper (or other)
Callee	controller / participant `:9091` (or `:9092`) HTTP endpoint
Transport	HTTP/1.1
Auth (Phase 2)	none (NetworkPolicy restricts source IPs)
Auth (Phase 3+)	possibly bearer token
Phase	PR #6 (endpoint) + PR #14 (NetworkPolicy) + PR #17 (metrics content)
STRIDE applicable	I, D
Controls	NetworkPolicy (PR #14) restricts ingress to Prometheus pod's CIDR ; bounded cardinality (no free-form tenant_id label) ; rate limiting at the K8s ingress level

4. Attacker personas

Seven archetypes with capability + motivation + entry vector.

4.1 Persona table

ID	Persona	Capability	Motivation	Entry vector	In scope L2 ?
AP1	Curious insider	Read code, deploy to staging	Curiosity, accidental	Internal access	[OK]
AP2	Malicious insider	Same as AP1 + intent to harm	Disgruntled, exfil	Internal access	[OK]
AP3	Compromised efe-server	Issues arbitrary gRPC calls	Upstream RCE	TB2	[OK]
AP4	Compromised participant pod	Returns crafted Step responses	Container RCE, sidecar attack	TB3	[OK]
AP5	External attacker	Network access via L7 ingress	Financial gain, espionage	TB1 (filtered through efe-server)	[FAIL] (Phase 5 only)
AP6	Operator typo	Misconfigures env vars, SIGHUPs wrong pod	Honest mistake	TB4, TB5	[OK]
AP7	Supply chain	Injects malicious dep at build time	Long-game APT	Crates.io / build pipeline	[FAIL] (continuous runbook, separate doc)

4.2 Persona detail

AP1 -- Curious insider. A developer or operator who has legitimate read access to the codebase + staging. Likely entry : `kubectl get pods -A` reveals the meta-swarm StatefulSet ; `kubectl logs` reveals internals. Not malicious -- but can accidentally trigger Sev3 incidents via misconfigured tests in staging that leak into production via shared infra. Risk : low to medium.

AP2 -- Malicious insider. Same access as AP1, with intent to harm or exfiltrate. Likely goals : steal manifest contents (A3) to learn modelling internals, tamper with `deferred_ticket` (A10) to disguise a regression as a SKIP-DT. Risk : medium ; mitigated by audit logs + 2-eyes review of sensitive config changes.

AP3 -- Compromised efe-server. An RCE on `efe-server` would let the attacker issue arbitrary `MetaSwarmService::Iterate` calls. Likely chain : exploit a CVE in `efe-server` -> privilege escalation -> reach controller via TB2. Risk : high -- but bounded by Phase-1 contractive math (T1 cannot cause unbounded drift) + MAX_INFLIGHT DoS protection (T7).

AP4 -- Compromised participant pod. RCE inside a participant container -- e.g. via a sidecar process vulnerability or a malicious image layer. Goal : bias the meta- consensus (T1) or exfiltrate via covert channel (T12). Risk : **high** -- most threat scenarios assume this persona. Mitigations : mTLS (PR #16), determinism check (SLO-A1-META-DETERMINISM), contractivity bound.

AP5 -- External attacker. Outside the cluster, reaching ingress. In Phase 2, meta-swarm is internal-only ; `efe-server` is the only externally-reachable service. AP5 must compromise `efe-server` first -> becomes AP3. Therefore **AP5 is out of scope for Phase 2 / L2**. In scope at Phase 5 (customer-facing SLA).

AP6 -- Operator typo. An honest mistake : `SIGHUP` to the wrong pod, malformed env var, deploy without TLS_REQUIRED check, etc. Risk : medium ; mitigated by config-time validation (PR #6) + Helm chart guards (PR #14).

AP7 -- Supply chain. A malicious crate in `~/.cargo/registry`, a build-script RCE, or a compromised CI runner. Out of scope for this document ; handled by the workspace `cargo audit` runbook + signed crates feature (`DT-WORKSPACE-SIGNED-CRATES`).

5. Threat scenarios

Sixteen scenarios. Each follows the 9-field structure :

```
T-N -- short title
+-- STRIDE letters
+-- Surface (TB id)
+-- Personas (AP id)
+-- Assets at risk (A id)
+-- Description (paragraph)
+-- Attack chain (numbered steps)
+-- CVSS v3.1 vector + score
+-- Severity (SEV-1/2/3/4)
+-- Mitigations (layered defence, with implementing PRs)
+-- Status
+-- Owner
+-- Detection
```

5.1 Summary table

ID	Title	STRIDE	TB	Persona	CVSS	Status
T1	Malicious participant biases consensus	T,I	TB3	AP2, AP4	**8.6**	DEFERRED-TO-PR-16
T2	MITM on controller <-> participant	S,T	TB3	AP4	**9.1**	DEFERRED-TO-PR-16
T3	Replay of Iterate response	T,R	TB2	AP3	6.8	MITIGATED
T4	SIGHUP injection by unauthorised operator	S,E	TB4	AP6	5.4	DEFERRED-TO-PR-12
T5	Manifest tampering via ConfigMap	T,E	TB5	AP2	**8.1**	MITIGATED
T6	mTLS cert rotation failure -> outage	D	TB2,TB3	AP6	**7.5**	DEFERRED-TO-PR-16
T7	Resource exhaustion via Iterate flood	D	TB2	AP3	6.5	MITIGATED
T8	Crafted dim mismatch	T,D	TB2	AP3	6.1	MITIGATED
T9	Timing side-channel on convergence	I	TB3	AP4	4.3	**ACCEPTED**
T10	/metrics endpoint info disclosure	I	TB6	AP1, AP5	5.3	MITIGATED
T11	Trace context spoofing	T,R	TB2,TB3	AP3, AP4	4.8	MITIGATED
T12	Determinism violation as covert channel	I,R	TB3	AP4	5.5	MITIGATED
T13	Stack overflow via crafted oneof	T,D	TB2,TB3	AP3, AP4	**7.2**	MITIGATED
T14	K8s NetworkPolicy bypass	T,E	TB3	AP2	**8.7**	DEFERRED-TO-PR-14
T15	Cert SAN spoofing	S,E	TB2,TB3	AP4	**8.2**	DEFERRED-TO-PR-16
T16	Admin priv-esc via ReloadConfig	E	TB4	AP6, AP2	**7.8**	DEFERRED-TO-PR-12
T17	Env-var injection redirects controller to malicious participant	S,T	TB5	AP2, AP6	6.4	MITIGATED
T18	Log injection via crafted traceparent / RPC payload content	T,R	TB2,TB3	AP3, AP4	4.1	MITIGATED
T19	gRPC `Info` RPC recon -- build_sha + hostname + participant list	I	TB2	AP1, AP3	3.7	MITIGATED

ID	Title	STRIDE	TB	Persona	CVSS	Status
T20	`deferred_ticket` baseline YAML tampering bypasses anti-Φ-D.A1	T,E	(workspace YAML)	AP2	5.2	MITIGATED

Counts : 20 total ; **1 SEV-1** + **7 SEV-2** + **10 SEV-3** + **2 SEV-4**. Status : **12 MITIGATED** + **7 DEFERRED-TO-PR** + 1 ACCEPTED + **0 OPEN**.

5.2 Threat detail

T1 -- Malicious participant biases consensus via crafted state

STRIDE : Tampering (T), Information disclosure (I). **Surface** : TB3 (controller <-> participant). **Personas** : AP2 (malicious insider), AP4 (compromised participant pod). **Assets at risk** : A1 (meta-consensus state), A5 (determinism guarantee).

Description. A participant whose process has been compromised -- by an RCE, sidecar attack, or insider -- can return arbitrary `new_state` values in its `Step` responses. An attacker who controls the `pulled_from_meta -> new_state` mapping can :

1. Bias the meta-consensus toward an attacker-chosen direction.
2. Cause divergence (residual oscillates) -- DoS.
3. Exfiltrate via a covert channel encoded in the LSBs of returned

states (information disclosure ; partially overlaps T12).

Attack chain.

1. Attacker gains code execution inside a participant pod

(T13 or AP4 entry vector).

1. Attacker modifies the `MetaParticipantService::Step` handler

to return `pulled_from_meta + ATTACKER_BIAS` instead of the contractive map.

1. Controller's `iterate()` runs the meta-operator with this

biased input.

1. Meta-consensus converges (or diverges) under attacker control.

CVSS v3.1 vector. `` `AV:A/AC:H/PR:H/UI:N/S:U/C:H/I:H/A:L/E:F/RL:O/RC:C` `` Base = 6.9, Temporal = 6.3, **Environmental = 8.6** (raised because the consensus impact propagates to all downstream decisions in multi-tenant Phase 3).

Severity : **SEV-2**.

Mitigations (layered defence).

1. **Phase-1 contractive bound** (algorithmic, Phase-1 PR #1) : the

algorithm itself bounds attacker bias to `(1 − κ_max) × max_step_size`. Attacker cannot cause **unbounded** consensus drift -- the meta- operator's contractivity is a mathematical invariant.

1. **Safety projection** (Phase-1 PR #1, `BoxSafety` in Phase 2) :

the meta-consensus is projected onto a safety set at every meta-step. An attacker who pushes outside the box is clipped.

1. **Determinism check** (T12 + SLO-A1-META-DETERMINISM) : a biased

participant would break the 3-rerun bit-identical guarantee on every Iterate. Alert fires at SLO breach.

1. **mTLS + SAN check** (PR #16) : an external attacker cannot

spoof a participant. Only AP4 (in-cluster RCE) remains feasible.

1. **Phase-3 future control** : cryptographic attestation of step

outputs (deferred ; tracked as `DT-A2-ATTESTATION`).

Status : **DEFERRED-TO-PR-16** (mTLS). At L2 entry, mTLS reduces the attacker pool to AP4 (in-cluster RCE) and the contractive bound limits the impact. Residual : moderate ; accepted for Phase 2 ; full cryptographic attestation in Phase 3.

Owner : meta-swarm-team (algorithmic bound) + sec (mTLS + future attestation).

Detection.

- `SLO-A1-META-DETERMINISM` breach -> alert
- `meta_swarm_consensus_norm` outside expected band -> alert (PR #17, PR #19)
- Optional Phase-3 : per-participant signature on returned state

Test. Phase-2 multi-process integration test (PR #10) can be extended with a "rogue participant" fixture that returns biased states. Expected outcome : iteration completes within bounds dictated by contractivity (no unbounded drift). Tracked as `DT-A2-ROGUE-PARTICIPANT-TEST`.

T2 -- MITM on controller <-> participant injects fake Step response

STRIDE : Spoofing (S), Tampering (T). **Surface** : TB3. **Personas** : AP4 (compromised participant pod can also MITM a sibling). **Assets at risk** : A1, A2, A6.

Description. An attacker positioned on the cluster pod network between the controller and a participant intercepts gRPC frames and modifies / replays them. Without mTLS, this is trivial for any container that shares the network namespace.

Attack chain.

1. Attacker compromises ANY container in the cluster (e.g. a

misconfigured sidecar on an unrelated service).

1. Attacker positions to intercept controller -> participant

traffic (e.g. via ARP poisoning in the pod CIDR or by sniffing on the same node before NetworkPolicy is enforced).

1. Controller issues `Step` request.
2. Attacker swaps the `pulled_from_meta` field, returning a

modified response that pushes consensus in attacker's direction.

CVSS v3.1 vector. `` `AV:A/AC:H/PR:N/UI:N/S:C/C:H/I:H/A:H/E:F/RL:O/RC:C` `` Base = 9.1, Temporal = 8.7.

Severity : **SEV-1**.

Mitigations.

1. **mTLS handshake** (PR #16, mandatory at Phase 2 GA) : the

participant's leaf cert is verified by the controller against the cert-manager-issued CA bundle.

1. **SAN check** (PR #16) : the controller accepts only callers

whose SAN matches `participant-<id>.efe.svc.cluster.local`.

1. **K8s NetworkPolicy** (PR #14) : restricts ingress to the

participant pod to the controller pod IPs only. Defence in depth -- even if mTLS is misconfigured, ingress is blocked.

1. **HTTP/2 frame integrity** : tonic 0.12 validates HTTP/2

framing ; malformed frames are rejected.

1. **Channel keepalive** (PR #6 `gRPC server timeout = 60 s`) :

long-running MITM would trigger keepalive failure.

Status : **DEFERRED-TO-PR-16** (mTLS). At PR #14 merge, NetworkPolicy reduces the attack surface ; PR #16 completes the mitigation with mTLS.

Owner : sec.

Detection.

- TLS handshake failure alert (PR #19 + custom)
- gRPC trailer mismatch alert
- Determinism breach alert

T3 -- Replay of Iterate response

STRIDE : Tampering (T), Repudiation (R). **Surface** : TB2. **Personas** : AP3. **Assets at risk** : A6, A1.

Description. An attacker intercepts a previous `IterateResponse` and replays it to `efe-server` in lieu of a fresh response. Without freshness controls, downstream consumers may use stale data.

Attack chain.

1. Attacker captures a valid `IterateResponse` from a previous

call.

1. Attacker positions to MITM TB2.
2. New Iterate request from efe-server -> attacker drops it and

returns the cached response.

CVSS v3.1 vector. `` `AV:A/AC:H/PR:H/UI:N/S:U/C:L/I:H/A:L/E:F/RL:O/RC:C` `` Base = 5.4, Temporal = 5.0, **Environmental = 6.8**.

Severity : SEV-3.

Mitigations.

1. **gRPC deadline** (Phase 2 hard cap 30 s) : the response has an

expiry implicit in the call duration. A replayed response would arrive after the deadline -> `DEADLINE_EXCEEDED` from client side.

1. **Idempotence + determinism** : two Iterate calls with the same

input return bit-identical responses (SLO-A1-META-DETERMINISM). Replaying = computing twice -- no integrity issue beyond duplicate computation cost.

1. **W3C TraceParent** (PR #18) : each call has a unique trace_id ;

a replayed response with stale trace_id is detectable in OTel spans.

Status : **MITIGATED**.

Owner : meta-swarm + sec.

Detection.

- OTel span trace_id mismatch alert (PR #18 + #19)
- Deadline-exceeded rate on TB2

T4 -- SIGHUP injection by unauthorised operator

STRIDE : Spoofing (S), Elevation of privilege (E). **Surface** : TB4. **Personas** : AP6 (operator typo), AP1 (curious). **Assets at risk** : A3, A9.

Description. An operator with K8s `pods/exec` access to the controller pod runs `kill -HUP <pid>` (or `kubectl exec ... kill`), triggering an unintended manifest reload.

Attack chain.

1. Operator with `pods/exec` permission on the controller pod

accidentally targets the meta-swarm controller instead of another workload.

1. SIGHUP is delivered.
2. Controller reads the manifest from disk and applies it.

3. If the manifest changed since boot, behaviour shifts unexpectedly.

CVSS v3.1 vector. `` `AV:L/AC:L/PR:H/UI:N/S:U/C:N/I:H/A:L/E:H/RL:O/RC:C` `` Base = 4.3, Environmental = 5.4.

Severity : SEV-3.

Mitigations.

1. **K8s RBAC** : `pods/exec` access is highly restricted (operator

sub-team only).

1. `MetaSwarmService::ReloadConfig` **admin scope** (PR #12) : the

alternative path (via gRPC admin RPC) requires the `admin` scope on the auth token. Belt + suspenders.

1. **Audit log of every SIGHUP** (PR #12 emits a structured log

line at every signal) : forensic trail.

1. **Manifest-tampering alert** (PR #19) : if the manifest file

changes outside a known deploy window, alert fires.

Status : **DEFERRED-TO-PR-12** (the admin scope + audit log are PR #12's job).

Owner : platform + sec.

Detection.

- `meta_swarm_reload_total` counter increase outside deploy window
- audit log of SIGHUP received

T5 -- Manifest tampering via ConfigMap edit

STRIDE : Tampering (T), Elevation of privilege (E). **Surface** : TB5. **Personas** : AP2 (malicious insider). **Assets at risk** : A3.

Description. An insider with `configmaps/update` permission edits the `efe-meta-swarm-manifest` ConfigMap to inject participants with different kappas, seeds, or safety geometries. On the next reload, the controller adopts the malicious manifest.

Attack chain.

1. Insider has `kubectl edit configmap` on the relevant ConfigMap.
2. Insider modifies a kappa from 0.6 -> 0.99 (almost non-contractive),

slowing convergence and causing soft DoS.

1. Or : insider swaps the safety_geometry kind from `box` to a

`poly` with adversarial constraints.

1. Next SIGHUP (or periodic reload poll) picks up the new manifest.

CVSS v3.1 vector. `` `AV:N/AC:L/PR:H/UI:N/S:C/C:N/I:H/A:H/E:H/RL:O/RC:C` `` Base = 7.5, Environmental = 8.1.

Severity : **SEV-2**.

Mitigations.

1. **K8s RBAC** (PR #13) : `configmaps/update` on the

`efe-meta-swarm-manifest` ConfigMap is restricted to a 2-person group (`@efe-platform-leads`). Other operators cannot edit.

1. **Sealed-secrets for participant seeds** (PR #13) : sensitive

fields (seeds in particular, to prevent determinism attacks) are stored as SealedSecrets, not ConfigMap.

1. **Manifest validation on reload** (PR #12) : the controller

refuses to apply a manifest whose dimensions changed or whose kappas are outside (0, 1).

1. **Audit log + alert** : ConfigMap edits emit K8s audit events ;

PR #19 alert fires on any change.

1. **Signed manifests (Phase 3+)** : the manifest carries a sigstore

signature ; controller verifies before applying. `DT-A2-MANIFEST-SIG`.

Status : **MITIGATED** (RBAC + sealed-secrets + reload validation cover Phase 2 ; signed manifests are Phase 3+).

Owner : platform + sec.

Detection.

- K8s audit event for ConfigMap update
- `meta_swarm_reload_total{outcome="rejected"}` counter
- Manifest-tampering alert (PR #19)

T6 -- mTLS cert rotation failure -> service outage

STRIDE : Denial of service (D). **Surface** : TB2, TB3. **Personas** : AP6 (operator misconfigures cert-manager). **Assets at risk** : A4, A6.

Description. Phase-2 mTLS certs are issued by cert-manager with 90-day validity and auto-renewed at 60 days. If renewal fails (cert-manager crashes, CA exhausted, K8s API down), certs expire and TLS handshakes start failing -> service outage.

Attack chain.

1. cert-manager is unhealthy for > 30 days (silent failure).
2. Day 90 : leaf cert expires.
3. Next mTLS handshake fails -> controller and participant cannot

communicate.

1. All Iterate calls return `UNAVAILABLE`.

CVSS v3.1 vector. `` `AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H/E:H/RL:U/RC:C` `` Base = 7.5, Temporal = 6.9.

Severity : **SEV-2**.

Mitigations.

1. **cert-manager 60-day auto-renewal** (PR #16) : the standard

cert-manager Certificate resource renews at 2/3 lifetime.

1. `MetaSwarmCertExpiringSoon` **alert** (PR #16 + #19) : fires at

7 days remaining.

1. **Manual rotation runbook** (`META_SWARM_CERT_EXPIRY.md`,

PR #16) : if auto-renewal fails, operator can manually rotate.

1. **Per-pod readiness gating** : a pod whose cert is < 1 day from

expiry flips its Health to `NOT_SERVING` (PR #16 enhancement).

Status : **DEFERRED-TO-PR-16** (cert-manager wiring).

Owner : sec + platform.

Detection.

- `MetaSwarmCertExpiringSoon` alert (7 d threshold)
- `meta_swarm_tls_handshake_failures_total` counter (PR #17)

T7 -- Resource exhaustion : Iterate flood

STRIDE : Denial of service (D). **Surface** : TB2. **Personas** : AP3. **Assets at risk** : A1 (availability).

Description. A compromised `efe-server` (or a buggy retry loop) issues 10 000 Iterate calls/sec. Each call holds a connection slot + worker thread cycle. The controller becomes unresponsive to legitimate traffic.

Attack chain.

1. AP3 controls efe-server's outbound RPC scheduler.
2. AP3 issues Iterate calls in a tight loop, ignoring backpressure.
3. Controller hits `MAX_INFLIGHT` limit ; returns RESOURCE_EXHAUSTED.
4. Legitimate efe-server calls fail.

CVSS v3.1 vector. `` `AV:N/AC:L/PR:H/UI:N/S:U/C:N/I:N/A:H/E:F/RL:O/RC:C` `` Base = 4.9, Environmental = 6.5.

Severity : SEV-3.

Mitigations.

1. `EFE_META_SWARM_MAX_INFLIGHT` **enforcement** (PR #6 config +

PR #8 runtime) : default 10 concurrent Iterate calls ; beyond that, server returns RESOURCE_EXHAUSTED.

1. **gRPC server timeout = 60 s** (PR #6) : stuck calls are reaped.
2. **Per-caller rate limiting** (Phase 3+) : `DT-A2-PER-CALLER-RATE`.
3. **K8s LimitRange / ResourceQuota** : caps CPU + memory at the

pod level ; cannot consume more than allocated.

Status : **MITIGATED** (MAX_INFLIGHT + timeout).

Owner : meta-swarm.

Detection.

- `meta_swarm_iterate_total{outcome="resource_exhausted"}` rate
- `meta_swarm_inflight` gauge spike

T8 -- Crafted dim mismatch in IterateRequest

STRIDE : Tampering (T), Denial of service (D). **Surface** : TB2. **Personas** : AP3. **Assets at risk** : A1.

Description. A compromised efe-server sends an `IterateRequest` with `expected_participants` whose `dim` field disagrees with the controller's manifest. Without validation, this could cause :

- Server-side panic on tensor reshape.
- Slow path that allocates a huge buffer based on the attacker-

chosen dim.

Attack chain.

1. AP3 modifies the request body : sets `dim = u32::MAX`.
2. Controller (without validation) attempts to allocate a tensor

of `u32::MAX x 4` bytes ≈ 17 GB.

1. OOM kill ; pod restart.

CVSS v3.1 vector. `` `AV:N/AC:L/PR:H/UI:N/S:U/C:N/I:L/A:H/E:F/RL:O/RC:C` `` Base = 5.3, Environmental = 6.1.

Severity : SEV-3.

Mitigations.

1. **Server-side dim validation** (PR #8) : `IterateRequest.dim`

must match the controller's manifest exactly ; otherwise `META_ERROR_DIM_MISMATCH` returned.

1. **Prost max-message-size = 4 MB** (PR #5) : a request with

`dim = u32::MAX` would exceed the message-size cap and be rejected at the protobuf layer.

1. `meta_swarm_iterate_total{outcome="config_error"}` alert if

spike.

Status : **MITIGATED**.

Owner : meta-swarm.

Detection.

- gRPC `FailedPrecondition` rate
- Custom counter `meta_swarm_dim_mismatch_total`

T9 -- Timing side-channel reveals convergence trajectory

STRIDE : Information disclosure (I). **Surface** : TB3. **Personas** : AP4. **Assets at risk** : A1.

Description. An attacker with timing-precise access to a participant pod can measure the wall time of each `Step` call. By correlating timing with the controller's outer-iteration counter, the attacker reconstructs the convergence trajectory -- potentially revealing features of the underlying tenant workload.

Attack chain.

1. AP4 has a stopwatch in the participant container.
2. AP4 records the wall time of each Step (already self-reported

in `StepResponse.step_us`).

1. AP4 correlates timing patterns with public information about the

tenant.

CVSS v3.1 vector. `` `AV:A/AC:H/PR:H/UI:N/S:U/C:L/I:N/A:N/E:P/RL:U/RC:C` `` Base = 2.5, Environmental = 4.3.

Severity : SEV-3.

Mitigations.

1. **Phase-2 is internal-only.** Tenant isolation arrives in

Phase 3 ; until then, the side-channel only leaks within a single internal tenant.

1. **Constant-time iteration** : prohibitively expensive at the

planned 100 rps target ; adds ~ 30 % latency.

Status : **ACCEPTED**.

Acceptance rationale.

- Severity is SEV-3 (medium).
- Phase 2 deployment is single-tenant + internal-only -- the

side-channel does not cross any tenant boundary that matters.

- Implementing constant-time iteration would add ~ 30 % latency,

blowing the `SLO-A1-META-LATENCY-P99 <= 100 ms` budget.

- Phase 3 multi-tenant entry triggers re-assessment ; if any

paying tenant requires constant-time, we implement it then.

Expiry : 2027-06-30 (Phase 5 entry). **Revisit trigger** : a paying tenant signs requiring constant-time guarantees ; or Phase 3 tenant isolation work flags this as a blocker. **Owner** : meta-swarm-team (algorithmic) + product (customer requirements).

Detection.

- N/A : no detection possible for timing side-channels at this layer.

T10 -- /metrics endpoint information disclosure

STRIDE : Information disclosure (I). **Surface** : TB6. **Personas** : AP1 (curious), AP5 (external -- out of scope). **Assets at risk** : A8.

Description. The Prometheus `/metrics` endpoint exposes operational telemetry including `meta_swarm_build_info`, `meta_swarm_participant_up`, and (in Phase 3) potentially tenant ids. An attacker who can reach the endpoint can map the deployment topology.

Attack chain.

1. AP1 finds the participant pod IPs via `kubectl describe`.
2. AP1 `curl http://<ip>:9092/metrics` from within the cluster.
3. AP1 reads `meta_swarm_build_info{git_sha=...}` -> learns version.
4. AP1 reads `meta_swarm_participant_up` -> counts replicas.

CVSS v3.1 vector. `` `AV:A/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N/E:H/RL:O/RC:C` `` Base = 3.5, Environmental = 5.3.

Severity : SEV-3.

Mitigations.

1. **K8s NetworkPolicy** (PR #14) : `/metrics` ingress is restricted

to the Prometheus scraper pod's IP range. Other in-cluster callers are blocked.

1. **Bounded cardinality** (PR #17) : metric labels are bounded

(no free-form tenant_id) ; the set of participant ids is fixed.

1. **No PII / customer data in metrics** (style guide, PR #17) :

metric labels never carry tenant data ; only operational metadata.

1. **Future** : bearer-token auth on `/metrics` (Phase 3 ; `DT-A2-METRICS-AUTH`).

Status : **MITIGATED** (NetworkPolicy + bounded cardinality).

Owner : platform + sre.

Detection.

- `meta_swarm_metrics_requests_total` rate from unexpected sources
- K8s NetworkPolicy drop counter

T11 -- Trace context spoofing via crafted W3C traceparent

STRIDE : Tampering (T), Repudiation (R). **Surface** : TB2, TB3. **Personas** : AP3, AP4. **Assets at risk** : A8.

Description. An attacker injects a crafted `traceparent` header to either :

- Cause trace collision in the OTel collector (two unrelated traces

share the same trace_id) -- disrupting forensics.

- Repudiate a real action by injecting a trace_id that points to a

fabricated "innocent" earlier span.

Attack chain.

1. AP3 sets `traceparent: 00-deadbeef...-00f067aa0ba902b7-01` on

every Iterate call.

1. OTel collector merges traces by trace_id.
2. Forensic investigation pulls all spans for `deadbeef...` and finds

a tangled mess.

CVSS v3.1 vector. `` `AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:L/A:N/E:F/RL:O/RC:C` `` Base = 4.3, Environmental = 4.8.

Severity : SEV-3.

Mitigations.

1. **W3C traceparent validation** (PR #18) : the controller and

participants validate the syntax (32 hex + 16 hex + flags) ; malformed traceparents are dropped + replaced with a fresh one.

1. **Sampling cap** (PR #18) : sample ratio capped at 5 % ; spurious

trace_ids do not flood the collector.

1. **Per-call structured log line** (PR #6 tracing setup) : even if

OTel spans are tangled, the structured logs preserve causality via the per-line `request_id`.

Status : **MITIGATED**.

Owner : meta-swarm + sre.

Detection.

- Spike in `meta_swarm_iterate_total` with malformed traceparents
- OTel collector ingestion-rejected metric

T12 -- Determinism violation as covert channel

STRIDE : Information disclosure (I), Repudiation (R). **Surface** : TB3. **Personas** : AP4. **Assets at risk** : A5.

Description. A compromised participant deliberately violates determinism -- returns slightly different states on otherwise-identical Step requests -- encoding a covert channel. Each bit of leaked information is a deliberate LSB perturbation.

Attack chain.

1. AP4 modifies the Step handler to return

`step_with_pull(pulled) + secret_bit_to_leak x 1e-7`.

1. Controller computes the joint residual, which now varies by

`secret_bit` ; attacker on the controller side (or via metrics) reads `meta_swarm_iterate_final_residual` LSB.

CVSS v3.1 vector. `` `AV:A/AC:H/PR:H/UI:N/S:U/C:H/I:L/A:N/E:F/RL:O/RC:C` `` Base = 4.6, Environmental = 5.5.

Severity : SEV-3.

Mitigations.

1. `SLO-A1-META-DETERMINISM` (Phase 1 PR #4 + Phase 2 PR #10) :

3-rerun bit-identical check. An attacker who perturbs LSBs breaks the SLO on every Iterate.

1. `MetaSwarmDeterminismBreach` **alert** (PR #17 + #19) : fires

immediately on SLO breach.

1. **Audit log of every Iterate's `(iterations, residual.to_bits())`

tuple** (in `meta_swarm_extras.json` sidecar, Phase-1) : forensic trail.

Status : **MITIGATED**.

Owner : meta-swarm + sec.

Detection.

- `SLO-A1-META-DETERMINISM` breach -> alert
- Forensic walk of the audit log

T13 -- Stack overflow in proto parsing via deeply-nested oneof

STRIDE : Tampering (T), Denial of service (D). **Surface** : TB2, TB3. **Personas** : AP3, AP4. **Assets at risk** : A6, A1 (availability).

Description. An attacker crafts a protobuf message with deeply-nested oneof variants (e.g. exploiting the `MetaSpaceConfig.safety_geometry` oneof) to cause stack overflow during parsing.

Attack chain.

1. AP3 constructs a message with a tree of recursive oneof

wrappers exceeding the stack's max depth.

1. Controller's prost decoder recursively descends -> stack overflow.
2. Process crash.

CVSS v3.1 vector. `` `AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H/E:F/RL:O/RC:C` `` Base = 7.5, Environmental = 7.2.

Severity : **SEV-2**.

Mitigations.

1. **Prost max-message-size = 4 MB** (PR #5 build.rs) : protobuf

messages > 4 MB are rejected at the gRPC framing layer.

1. **Prost decode-depth limit** (default in prost 0.13 = 100

levels) : sufficient depth budget for our oneofs (max 2 levels).

1. **gRPC server timeout = 60 s** (PR #6) : even if parsing is slow,

the call is reaped.

1. `MetaParticipantService` **is mTLS-authenticated** (PR #16) :

only the controller can send messages to participants -- AP3 must compromise the controller first.

Status : **MITIGATED**.

Owner : meta-swarm.

Detection.

- `meta_swarm_iterate_total{outcome="parse_error"}` rate
- Sudden pod restart from controller (OOMKilled or panic)

T14 -- K8s NetworkPolicy bypass via misconfigured CNI

STRIDE : Tampering (T), Elevation of privilege (E). **Surface** : TB3. **Personas** : AP2. **Assets at risk** : A6.

Description. Phase-2 relies on K8s NetworkPolicy to restrict ingress between controller and participant pods. If the cluster's CNI plugin doesn't enforce NetworkPolicy (e.g. flannel-without-calico), an attacker can bypass it and reach participants directly.

Attack chain.

1. Cluster operator deploys with a CNI that ignores NetworkPolicy.
2. AP2 (insider) starts a pod in the same namespace.
3. The new pod can reach `participant-supplier-scoring:50090`

directly.

1. Without mTLS (Phase 2 plaintext window), AP2 sends crafted Step

responses ; covert channel established.

CVSS v3.1 vector. `` `AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N/E:H/RL:O/RC:C` `` Base = 6.5, Environmental = 8.7 (raised because consensus impact propagates).

Severity : **SEV-2**.

Mitigations.

1. **NetworkPolicy template in Helm chart** (PR #14) : ship the

policy by default in the chart.

1. `helm template` **golden-diff CI check** (PR #15) : verifies the

NetworkPolicy resource is emitted with correct selectors.

1. **CNI compatibility doc** (PR #14 README) : platform team

documents the supported CNI list (calico, cilium, aws-vpc-cni with NP enabled).

1. **mTLS as defence in depth** (PR #16) : even with bypassed

NetworkPolicy, mTLS SAN check prevents unauthorized callers.

Status : **DEFERRED-TO-PR-14** (NetworkPolicy ship) ; mTLS (PR #16) provides defence in depth.

Owner : platform + sec.

Detection.

- `meta_swarm_participant_unexpected_caller_total` counter (PR #17)
- K8s audit event for unusual pod-to-pod traffic

T15 -- Cert SAN spoofing via wildcard mismatch

STRIDE : Spoofing (S), Elevation of privilege (E). **Surface** : TB2, TB3. **Personas** : AP4. **Assets at risk** : A4, A6.

Description. If the SAN allowlist on the controller side accepts a wildcard (`*.efe.svc.cluster.local`), an attacker who can obtain a cert for ANY pod in the cluster can impersonate a participant.

Attack chain.

1. AP4 compromises an unrelated pod that has a cert-manager-issued

cert for `some-other-service.efe.svc.cluster.local`.

1. AP4 connects to the controller posing as a participant.
2. Controller's wildcard SAN check accepts the cert.
3. Attacker now receives Step calls from the controller and can

bias consensus (-> T1).

CVSS v3.1 vector. `` `AV:A/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:L/E:H/RL:O/RC:C` `` Base = 8.6, Environmental = 8.2.

Severity : **SEV-2**.

Mitigations.

1. **Strict SAN allowlist** (PR #16) : the controller accepts ONLY

the exact SANs `participant-<id>.efe.svc.cluster.local` for each configured `<id>` ; no wildcards.

1. `MetaSwarmAuthFailures` **alert** (PR #19) : fires on SAN-check

rejection rate.

1. **cert-manager Certificate resource ships exact CN + SAN**

(PR #16) : a wildcard cert is never issued by the chart.

Status : **DEFERRED-TO-PR-16** (mTLS shipping).

Owner : sec.

Detection.

- `meta_swarm_auth_failures_total{reason="san_mismatch"}` rate
- K8s audit events for cert-manager Certificate creates outside

the chart's namespace

T16 -- Admin privilege escalation via ReloadConfig RPC

STRIDE : Elevation of privilege (E). **Surface** : TB4 (and TB2 for the RPC variant). **Personas** : AP6 (typo), AP2 (intentional). **Assets at risk** : A3, A9.

Description. The `MetaSwarmService::ReloadConfig` RPC accepts an inline manifest (`new_meta`, `new_participants`). Without strict scope enforcement, any caller could replace the running manifest with arbitrary content.

Attack chain.

1. AP2 obtains a bearer token (e.g. from a misconfigured pod's

service-account-token volume).

1. AP2 calls `ReloadConfig(new_participants=...)` on the controller.
2. Without scope check, controller applies the inline manifest.
3. Controller now uses attacker-chosen participant set.

CVSS v3.1 vector. `` `AV:N/AC:L/PR:H/UI:N/S:C/C:N/I:H/A:H/E:F/RL:O/RC:C` `` Base = 7.2, Environmental = 7.8.

Severity : **SEV-2**.

Mitigations.

1. `admin` **scope on ReloadConfig** (PR #12) : the controller's

middleware checks the caller's auth token has the `admin` scope ; otherwise returns `UNAUTHENTICATED`.

1. **mTLS + SAN check** (PR #16) : only certs with a specific

`admin-...` SAN can invoke admin RPCs.

1. **K8s RBAC + service-account tokens with limited audiences**

(Phase 3) : `DT-A2-SA-AUDIENCE`.

1. **Audit log of every ReloadConfig** (PR #12) : forensic trail.
2. `MetaSwarmAdminRpcSpike` **alert** (PR #19) : fires on > 1

ReloadConfig / minute.

Status : **DEFERRED-TO-PR-12** (admin scope + audit log) + PR #16 (cert-based admin SAN).

Owner : meta-swarm + sec.

Detection.

- `meta_swarm_admin_rpc_total{outcome="ok"}` rate
- `MetaSwarmAdminRpcSpike` alert
- Audit log of every ReloadConfig request

T17 -- Env-var injection redirects controller to malicious participant

STRIDE : Spoofing (S), Tampering (T). **Surface** : TB5 (analogous : operator -> process env). **Personas** : AP2 (malicious insider), AP6 (operator typo). **Assets at risk** : A6, A1.

Description. The controller reads `EFE_META_SWARM_PARTICIPANTS` from env vars at startup. An insider who can edit the Deployment / StatefulSet's PodSpec (`spec.template.spec.containers[*].env`) can substitute the participant addresses with attacker-controlled endpoints. The controller would happily connect to malicious participants on restart.

Attack chain.

1. AP2 has `deployments/update` RBAC on the meta-swarm namespace.
2. AP2 edits the controller Deployment :

`EFE_META_SWARM_PARTICIPANTS=supplier-scoring=evil-host:50091,...`

1. K8s rolling-restart of controller pod.
2. New controller connects to `evil-host:50091` instead of the

legitimate participant Service.

1. Attacker now sends arbitrary Step responses (-> T1).

CVSS v3.1 vector. `` `AV:N/AC:L/PR:H/UI:N/S:C/C:L/I:H/A:L/E:H/RL:O/RC:C` `` Base = 6.4, Temporal = 6.0.

Severity : SEV-3.

Mitigations.

1. **K8s RBAC** (PR #13) : `deployments/update` on the meta-swarm

namespace is restricted to a 2-person group (`@efe-platform-leads`). Identical control as for T5 (ConfigMap tampering).

1. **mTLS + SAN check** (PR #16) : the controller validates the

participant's SAN against the **expected** static set ; an attacker-controlled endpoint won't present a valid cert with `participant-<id>.efe.svc.cluster.local` SAN.

1. **Helm chart pins endpoints** (PR #13) : env vars are rendered

from chart values ; out-of-band edits trigger `helm diff` drift alert (PR #14 + sre runbook).

1. **K8s audit log** : every Deployment update emits an audit event ;

`MetaSwarmAdminRpcSpike`-style alert can be added for unexpected Deployment edits.

Status : **MITIGATED** (RBAC + mTLS SAN check + chart pinning).

Owner : platform + sec.

Detection.

- K8s audit event for `Deployment` update outside deploy window
- `MetaSwarmAuthFailures{reason="san_mismatch"}` rate spike on

startup (attacker endpoint can't present valid cert)

- Helm diff drift alert (chart-rendered values vs running pod env)

T18 -- Log injection via crafted traceparent or RPC payload content

STRIDE : Tampering (T), Repudiation (R). **Surface** : TB2, TB3. **Personas** : AP3, AP4. **Assets at risk** : A8 (operational telemetry).

Description. An attacker injects newline characters (`\n`), ANSI escape sequences, or fake log markers (`[ERROR]`, `timestamp=...`) into RPC payload fields that get logged (e.g. `traceparent`, `participant_id` in error messages). The injected content can :

- Forge log lines that appear to originate from another component.
- Inject log markers that confuse log parsers / SIEM systems.
- Pollute forensic timelines by emitting bogus "evidence".

Attack chain.

1. AP3 sets `traceparent: "valid-id\n[INFO] fake forensic event"`.
2. Controller's tracing-subscriber logs the traceparent verbatim

in some debug paths.

1. Log line in stdout now appears as TWO entries -- the second

crafted by the attacker.

1. SIEM ingestion sees two events ; forensic timeline is poisoned.

CVSS v3.1 vector. `` `AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:L/A:N/E:P/RL:O/RC:C` `` Base = 3.5, Temporal = 3.2 -> adjusted upward (Env) to **4.1**.

Severity : SEV-4.

Mitigations.

1. **JSON-formatted structured logs** (PR #6 `tracing.rs::init` :

JSON output when stderr is not a TTY) : newlines in user input are escaped as `\n` inside JSON strings. Log parsers cannot be confused.

1. **W3C traceparent validation** (PR #18) : the controller

validates the traceparent syntax (32 hex + 16 hex + flags) and rejects any input containing non-hex characters. This catches `\n` injection at the protocol layer.

1. **Bounded log labels** (PR #17) : metric / log labels have a

fixed vocabulary ; free-form user input never enters labels.

1. **Tracing-subscriber max-field-len** (default 8 KB) : oversized

fields are truncated.

Status : **MITIGATED** (JSON logs + traceparent validation).

Owner : meta-swarm + sre.

Detection.

- Log parser warning rate (SIEM-side)
- Traceparent-malformed counter (PR #17)
- Manual review of any forensic anomaly

T19 -- gRPC `Info` RPC recon (build_sha + hostname + participant list)

STRIDE : Information disclosure (I). **Surface** : TB2 (efe-server -> controller `Info`). **Personas** : AP1 (curious insider), AP3 (compromised efe-server). **Assets at risk** : A7 (Health + Info responses).

Description. The controller's `MetaSwarmService::Info` RPC returns operational metadata : `build_sha` (8-char git hash), `proto_version` ("v1"), `leader_pod_name` (hostname / pod name), `configured_participants` (list of participant ids). An attacker with TB2 access can mine this info for :

- **CVE matching** : `build_sha` reveals exact code version -> look

up known-CVE list of the dep tree at that SHA.

- **Topology recon** : participant list maps the consensus group ;

useful pre-stage to T1 (target which participant to compromise).

- **Pod-name guessing** : in K8s, `leader_pod_name` reveals the

StatefulSet ordinal index ; helpful for chaos / DoS targeting.

Attack chain.

1. AP3 has reached the controller via TB2 (mTLS handshake passed --

AP3 controls efe-server which has a valid SAN).

1. AP3 calls `Info` ; gets full operational metadata.
2. AP3 cross-references `build_sha` with public CVE list ; finds

known vuln in transitive dep.

1. AP3 plans targeted exploit.

CVSS v3.1 vector. `` `AV:N/AC:L/PR:H/UI:N/S:U/C:L/I:N/A:N/E:H/RL:O/RC:C` `` Base = 2.7, Environmental = 3.7.

Severity : SEV-4.

Mitigations.

1. **mTLS + admin scope on Info** (PR #16) : Info is callable only

by callers presenting a valid efe-server SAN cert. External attackers cannot reach it.

1. **Build-sha rotation discipline** : every Phase-2 release tag

bumps the build_sha ; CVE-prone SHAs are evicted from prod within 1 sprint of disclosure (workspace standing).

1. **Bounded** `configured_participants` **info** : Info returns only

participant ids (already public via NetworkPolicy selector labels) ; never returns kappa/seed or other sensitive config.

1. **Hostname privacy** : Phase-3 may scrub `leader_pod_name` if

tenant-isolation requires it (`DT-A3-INFO-HOSTNAME-SCRUB`).

Status : **MITIGATED** (mTLS + bounded payload).

Owner : meta-swarm + sec.

Detection.

- `meta_swarm_info_rpc_total` rate from unexpected callers

(PR #17 + custom)

- gRPC trailer for unauthorized Info calls

T20 -- `deferred_ticket` baseline YAML tampering bypasses anti-Φ-D.A1

STRIDE : Tampering (T), Elevation of privilege (E). **Surface** : workspace YAML (not a runtime trust boundary ; covered under repo audit). **Personas** : AP2 (malicious insider with PR write access). **Assets at risk** : A10 (`deferred_ticket` field).

Description. The Phase-1 anti-Φ-D.A1 doctrine forbids silent skips : every SLO is `ACTIVE` or `SKIP-DT` (with a `deferred_ticket` field). An insider with merge rights could add a fake `deferred_ticket` (`DT-FAKE-...`) to a newly-introduced SLO that would otherwise break the master gate, **bypassing** the doctrine without raising alarms.

Attack chain.

1. AP2 opens a PR that adds a new SLO to `bench_baselines/v0.10.0/meta_swarm_slos.yaml` (or similar baseline).
2. AP2 sets `deferred_ticket: DT-FAKE-NEVER-RESOLVED-IGNORE`.
3. PR review : sec-team-lead doesn't notice the made-up DT id.
4. PR merges ; master gate now tolerates SKIP-SILENT for the new

SLO without a real ticket tracking.

1. Production drift goes unnoticed for weeks.

CVSS v3.1 vector. `` `AV:N/AC:L/PR:H/UI:R/S:U/C:N/I:H/A:L/E:P/RL:O/RC:C` `` Base = 4.5, Environmental = 5.2.

Severity : SEV-3.

Mitigations.

1. **2-eyes PR review** (workspace standing) : any change to

`bench_baselines/**` requires a CODEOWNERS-driven sec-team review. AP2 alone cannot merge.

1. **DT registry** : a workspace-wide `docs/deferred_tickets.md`

lists every valid DT id ; CI lint rejects `deferred_ticket` values not in the registry (proposed `DT-A2-DT-REGISTRY-CI`).

1. **Quarterly DT audit** (sec-team standing) : every quarter,

sec-team walks every active `deferred_ticket` and verifies it maps to a real Jira / Linear ticket with an expiry date.

1. **Phase 3+** : sigstore signing of baseline YAML (`DT-A2-BASELINE-SIG`).

Status : **MITIGATED** (2-eyes review + DT registry CI lint).

Owner : sec + sre (DT registry custodian).

Detection.

- CI lint failure on unregistered `deferred_ticket` id
- Quarterly DT audit report
- `master_gate_check.py` flags SKIP-DT with unregistered DT id

6. Mitigations matrix

Cross-reference : each mitigation -> which threats it covers -> which PR implements it.

Mitigation	Implementing PR	Threats covered
mTLS handshake controller <-> participant + efe-server <-> controller	**PR #16**	T1, T2, T15
Strict SAN allowlist (no wildcards)	PR #16	T2, T15
K8s NetworkPolicy (Helm chart)	**PR #14**	T2, T10, T14
ConfigMap RBAC + sealed-secrets for seeds	**PR #13**	T5
cert-manager 60-day auto-renewal + 7d alert	PR #16	T6
`EFE_META_SWARM_MAX_INFLIGHT` enforcement	PR #6 (config) + PR #8 (runtime)	T7
Server-side dim validation	**PR #8**	T8
Prost max-message-size = 4 MB	PR #5 + PR #8	T13
Prost decode-depth limit (default)	PR #5	T13
`admin` scope on ReloadConfig	**PR #12**	T4, T16
Audit log of SIGHUP + ReloadConfig	PR #12	T4, T16
W3C traceparent validation	PR #18	T11
OTel sampling cap 5 %	PR #18	T11
Determinism SLO + 3-rerun check	Phase 1 (PR #4) + PR #10	T1, T12
gRPC deadline + idempotence	PR #6 + PR #8	T3
gRPC server timeout 60 s	PR #6	T2 (long-running MITM), T7, T13
Phase-1 contractivity (algorithmic)	Phase 1 (PR #1)	T1
BoxSafety projection (algorithmic)	Phase 1 (PR #1)	T1
K8s RBAC `deployments/update` restriction	PR #13	T5, T17
Helm chart pins endpoints + drift alert	PR #13 + PR #14	T17
JSON-formatted structured logs (escapes `\n`)	PR #6	T18
Bounded log labels (no free-form input)	PR #17	T18
mTLS + admin scope on `Info` RPC	PR #16	T19
Build-sha rotation discipline (per release tag)	workspace standing	T19
Bounded `Info` payload (no kappa/seed leak)	PR #8 (Info impl)	T19
Phase-1 baseline YAML 2-eyes review (anti-Φ-D.A1 doctrine)	workspace standing	T20
`deferred_ticket` registry + CI lint	DT-A2-DT-REGISTRY-CI	T20
Quarterly DT audit (sec-team standing)	workspace standing	T20
Sigstore signing of baseline YAML (Phase 3)	DT-A2-BASELINE-SIG	T20

6.1 PRs that ship a security control

PR	Sec-controls delivered	sec-team review required ?
PR #5 (proto)	Max-message-size in build.rs ; decode-depth defaults	[OK] done (PR #5 review)
PR #6 (server scaffold)	MAX_INFLIGHT config field ; gRPC server timeout 60 s	[OK] already merged with sec sign-off
PR #8 (Iterate body)	Dim validation, MAX_INFLIGHT runtime enforcement, idempotence	■ yes
PR #12 (SIGHUP + ReloadConfig)	Admin scope check, audit log	■ yes
PR #13 (Helm sub-chart)	ConfigMap RBAC, sealed-secrets template	■ yes
PR #14 (Helm full)	NetworkPolicy templates	■ yes
PR #16 (mTLS)	mTLS handshake, SAN check, cert-manager Certificate resource	■ yes -- full sec-team review
PR #17 (Prometheus)	Bounded cardinality on labels	■ yes
PR #18 (OTel)	Traceparent validation, sampling cap	■ yes

7. Residual risk register

The threats explicitly **ACCEPTED** in Phase 2, with rationale + expiry + revisit trigger.

ID	Threat	CVSS	Rationale	Expiry	Revisit trigger	Owner
T9	Timing side-channel reveals convergence trajectory	4.3	Internal-only Phase 2 ; tenant isolation is Phase 3 ; cost of constant-time iteration is prohibitive for ~ 100 rps target (~ 30 % latency overhead -> blows SLO-A1-META-LATENCY-P99).	2027-06-30 (Phase 5 entry)	Customer signs requiring constant-time guarantees ; or Phase 3 tenant isolation work flags this as a blocker.	meta-swarm + product

Only 1 ACCEPTED risk in Phase 2. All other CVSS-low risks (SEV-4) are MITIGATED by default workspace controls (TLS, RBAC, audit log).

8. Monitoring + alerting

Phase 2 ships **8 alert rules** (4 from PR #19 architecture + 4 proposed by this threat model).

8.1 Alert table

Alert	Threshold	Threats covered	PR	Severity	Routes to
`MetaSwarmHighFailureRate`	> 2 % failure / 5 min	T1, T7, T8	PR #19	critical	PagerDuty
`MetaSwarmParticipantDown`	`participant_up == 0` / 1 min	T1 (suspicious), T6	PR #19	critical	PagerDuty
`MetaSwarmReloadFail`	`reload_fail > 0` / 10 min	T4, T16	PR #19	major	Slack #sre
`MetaSwarmLatencyP99`	p99 > 100 ms	T7, T13	PR #19	major	Slack #sre
`MetaSwarmAuthFailures`	`auth_failures > 10` / min	T2, T15, T16	PR #19 (new)	critical	PagerDuty
`MetaSwarmCertExpiringSoon`	`cert_days_remaining < 7`	T6	PR #16	major	Slack #platform
`MetaSwarmDeterminismBreach`	`determinism != 1`	T1, T12	PR #17 + #19	major	Slack #meta-swarm
`MetaSwarmAdminRpcSpike`	admin_rpc_total rate > 1/min	T4, T16	PR #12 + #19	major	Slack #sre

All 8 alerts route via the existing Alertmanager configuration ; escalation policies are inherited from `ops_master_gate_fail.md`.

8.2 Dashboard coverage

The Phase-2 Grafana dashboards (PR #19) include 3 panels dedicated to security signals :

- "Auth failures by reason" (sourced from `MetaSwarmAuthFailures` ;

reasons : `san_mismatch`, `cert_invalid`, `scope_denied`, `token_expired`)

- "TLS cert days remaining" (per pod)
- "Determinism breach events" (timeline)

9. Incident response hooks

Each SEV-1 / SEV-2 threat has a runbook in `docs/runbooks/`.

9.1 Runbook table

Runbook	Sev1/2 threats covered	PR creating	Status
`META_SWARM_DETERMINISM_BREACH.md`	T1, T12	PR #24	DEFERRED
`META_SWARM_TLS_FAILURE.md`	T2	PR #16	DEFERRED
`META_SWARM_MANIFEST_TAMPERED.md`	T5	PR #24	DEFERRED
`META_SWARM_CERT_EXPIRY.md`	T6	PR #16	DEFERRED
`META_SWARM_NETPOL_BYPASS.md`	T14	PR #24	DEFERRED

Each runbook follows the workspace template `_template.md` : Symptoms -> Triage -> Mitigation -> Post-mortem.

9.2 Post-mortem template addition

The workspace postmortem template (`docs/postmortems/_template.md`) is extended (in PR #25) with a "Meta-swarm threat reference" field linking the incident to its catalogued threat ID.

10. Re-assessment cadence

The threat model is a **living document** with explicit triggers for re-walk.

Trigger	Action	Owner	Cadence
New PR adds a network surface (RPC, port)	Update § 3 (trust boundaries) + § 5 (threats)	PR author + sec-team	per PR
New PR removes a mitigation	Revoke `MITIGATED` status ; reassess CVSS	PR author + sec-team	per PR
Phase gate (L2, L3, L4, L5)	Full re-walk of all threats	sec-team lead	per gate
Sev1 incident on Phase-2 surface	Same-week patch + post-mortem amends model	sre + sec	per incident
New attacker persona observed	Add to § 4 + matrix	sec	as-needed
Periodic (every 6 months)	Re-walk in a quarterly sec-team review	sec-team lead	quarterly

The frontmatter `last_review` field is the authoritative stamp. CI lint warns if the stamp drifts > 7 months (added in PR #24).

11. Glossary

- **STRIDE** : Microsoft threat modelling taxonomy (Spoofing, Tampering, Repudiation, Information disclosure, DoS, Elevation of privilege).
- **CVSS v3.1** : FIRST Common Vulnerability Scoring System, version 3.1. See `https://www.first.org/cvss/specification-document`.
- **DREAD** : Microsoft secondary scoring (Damage, Reproducibility, Exploitability, Affected users, Discoverability).
- **mTLS** : mutual TLS -- both client and server present + validate certificates.
- **SAN** : Subject Alternative Name field in an X.509 certificate.
- **CNI** : Container Network Interface (K8s network plugin layer).
- **K8s RBAC** : Role-Based Access Control in Kubernetes.
- **sealed-secrets** : `bitnami/sealed-secrets` operator that encrypts K8s Secrets so they can be checked into git.
- **TB** : Trust Boundary (cf. § 3).
- **AP** : Attacker Persona (cf. § 4).
- **T-N** : Threat scenario number `N` (cf. § 5).
- **Anti-Φ-D.A1** : workspace doctrine forbidding silent skips (every SLO is `ACTIVE` or `SKIP-DT` -- no `SKIP-SILENT`).
- **L2 gate** : Phase 2 graduation criteria (cf. `EFE_A1_PHASE2_ARCHITECTURE.md` § 12) ; criterion G8 = "threat model signed off".
- **DT-A2-...** : Deferred Ticket id for a Phase-2 follow-up.

Appendix A -- CVSS scoring details

The 7 SEV-1 / SEV-2 threats with full CVSS metric breakdown for reviewer reproducibility.

A.1 T2 -- MITM on controller <-> participant

Metric	Value	Rationale
AV (Attack Vector)	Adjacent (A)	Requires same K8s pod network namespace
AC (Attack Complexity)	High (H)	Requires positioning between pods (sidecar / ARP)
PR (Privileges Required)	None (N)	Once on the network, no auth needed
UI (User Interaction)	None (N)	Fully automated
S (Scope)	Changed (C)	Compromises both controller + participant
C (Confidentiality)	High (H)	Reads all Step messages
I (Integrity)	High (H)	Modifies Step messages
A (Availability)	High (H)	Can drop/delay -> DoS

Base vector : `` `AV:A/AC:H/PR:N/UI:N/S:C/C:H/I:H/A:H` `` **Base = 9.1 -> SEV-1.**

Temporal :

Metric	Value
E (Exploit Code Maturity)	Functional (F) -- MITM tools exist generically
RL (Remediation Level)	Official Fix (O) -- mTLS is standard
RC (Report Confidence)	Confirmed (C)

Temporal = **8.7**.

Status : DEFERRED-TO-PR-16.

A.2 T1 -- Malicious participant biases consensus

Metric	Value	Rationale
AV	Adjacent (A)	In-cluster pod
AC	High (H)	Requires RCE inside participant
PR	High (H)	Operator-level access to deploy malicious image
UI	None (N)	Auto
S	Unchanged (U)	Stays in meta-swarm scope
C	High (H)	Reads all pulled states
I	High (H)	Modifies returned states
A	Low (L)	Phase-1 contractivity bounds drift

Base : `AV:A/AC:H/PR:H/UI:N/S:U/C:H/I:H/A:L` -> 6.9 Environmental (raised by multi-tenant impact projection) : **8.6 -> SEV-2**.

A.3 T5 -- Manifest tampering via ConfigMap

Metric	Value	Rationale
AV	Network (N)	K8s API call
AC	Low (L)	Standard `kubectl edit`
PR	High (H)	Requires `configmaps/update` RBAC
UI	None (N)	Auto
S	Changed (C)	Affects entire meta-swarm
C	None (N)	Manifest is not secret
I	High (H)	Wholly replaces algorithm config
A	High (H)	Bad kappa -> divergence -> service degraded

Base : `AV:N/AC:L/PR:H/UI:N/S:C/C:N/I:H/A:H` -> 7.5 Environmental : **8.1 -> SEV-2**.

A.4 T6 -- Cert rotation failure

`AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`

Base = 7.5 -> **SEV-2**.

A.5 T13 -- Stack overflow via crafted oneof

`AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`

Base = 7.5. Environmental reduces to **7.2** (mitigated by 4 MB cap) -> **SEV-2**.

A.6 T14 -- NetworkPolicy bypass

`AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N`

Base = 6.5. Environmental **8.7** (raised by consensus-impact projection) -> **SEV-2**.

A.7 T15 -- SAN spoofing

`AV:A/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:L`

Base = 8.6 -> **SEV-2**. Environmental **8.2**.

A.8 T16 -- Admin priv-esc via ReloadConfig

`AV:N/AC:L/PR:H/UI:N/S:C/C:N/I:H/A:H`

Base = 7.2 -> **SEV-2**. Environmental **7.8**.

Appendix B -- Cross-references

- Phase 2 architecture security model : `EFE_A1_PHASE2_ARCHITECTURE.md` § 15
- Operational doctrine : `EFE_OPERATIONAL_PRODUCTION_ARCHITECTURE.md`
- Phase 2 roadmap (PR #7 entry) : `EFE_A1_PHASE2_ROADMAP.md` § 2.1
- PR #7 detailed architecture : `EFE_A1_PHASE2_PR07_ARCHITECTURE.md`
- Sister architectures : PR #5 (proto), PR #6 (server scaffold)
- Workspace runbook (master gate fail) : `runbooks/ops_master_gate_fail.md`
- FIRST CVSS v3.1 calculator : `https://www.first.org/cvss/calculator/3.1`
- FIRST CVSS v3.1 specification : `https://www.first.org/cvss/specification-document`
- STRIDE reference : Microsoft Threat Modelling guidance
- L2 graduation criteria : `EFE_A1_PHASE2_ARCHITECTURE.md` § 12 (G8)

Sign-off

The L2 graduation gate criterion G8 requires the following 4 signatures :

Role	Name	Date	Signature
sec-team lead	R. Kapoor	2026-MM-DD	■
sec-team second pair (red-team)	E. Vargas	2026-MM-DD	■
meta-swarm-team representative	J. Tan	2026-MM-DD	■
sre-team representative	L. Ortiz	2026-MM-DD	■

Each signer attests that :

- They have walked all 16 threats and concur with the STRIDE,

CVSS, status, and mitigation analysis.

- They confirm that **zero** SEV-1 threat is in `ACCEPTED` or `OPEN` state.
- They confirm that every `DEFERRED-TO-PR-N` references a PR scheduled in Phase 2.
- They confirm that the `ACCEPTED` risk (T9) carries explicit rationale + expiry + owner.

At G8 satisfaction, the L2 verification ceremony (Mon 2026-08-24) marks G8 as [OK] in the L2 verification report.

PR #5

efe-meta-swarm-proto scaffold

Chapter 7 of 17 — source : `docs/EFE_A1_PHASE2_PR05_ARCHITECTURE.md` (2,166 LOC)

PR #5 -- `efe-meta-swarm-proto` scaffold -- Architecture ultra-détaillée

***Document type** : Execution-ready implementation blueprint for PR #5. **Scope** : The very first PR of Phase 2 (Sprint A1.P2.1, Week 1, day 1-3). **Owner** : meta-swarm-team -- assigned lead : J. Tan (back-up : M. Okafor). **Reviewers (required)** : 1 meta-swarm-team + 1 sec-team + 1 platform-team. **Estimate** : 3 FTE-days (engineering) + 1 FTE-day (review) = 4 FTE-days. **Branch** :* `A1.P2/05-meta-swarm-proto-scaffold`*. **Deadline** : Wed 2026-07-08 23:59 UTC (so PR #6 and #8 can start Thu). **Parent docs** :* `EFE_A1_PHASE2_ARCHITECTURE.md` *§ 4 ;* `EFE_A1_PHASE2_ROADMAP.md` *§ 2.1.*

Table of contents

1. TL;DR
2. Goal + non-goals
3. Pre-flight checks
4. Branch + commit plan
5. Workspace integration
6. Crate layout
7. `Cargo.toml` -- full
8. `build.rs` -- full
9. Proto file #1 -- `messages.proto`
10. Proto file #2 -- `errors.proto`
11. Proto file #3 -- `service_meta_swarm.proto`
12. Proto file #4 -- `service_meta_participant.proto`
13. `src/lib.rs` -- full
14. Generated code expectations
15. `README.md` -- full
16. Versioning policy
17. Field-number reservation strategy
18. Naming conventions
19. Smoke test -- cross-crate import
20. CI validation
21. Acceptance criteria (binary)
22. PR description template
23. Risk analysis (PR-local)
24. Reviewer checklist
25. Definition of Done
26. Post-merge follow-ups

0. TL;DR

PR #5 lands the **wire-protocol foundation** for the entire Phase 2 :

• 1 new workspace crate : `efe-meta-swarm-proto` (~600 LOC including protos).

- 4 `.proto` files (~700 LOC of protobuf with inline documentation).
- 1 `build.rs` (~80 LOC) that drives `tonic-build` with explicit

attribute injection (serde derives, doc comment forwarding).

- 1 `src/lib.rs` (~120 LOC) of re-exports + module organisation.
- 1 `README.md` (~250 LOC) documenting the versioning + field-reservation policy.

After merge, **any other crate in the workspace** can write :

```
use efe_meta_swarm_proto::meta_swarm::{
    MetaSwarmServiceClient,
    MetaParticipantServiceClient,
    IterateRequest,
    IterateResponse,
    /* ... */
};
```

and get fully-typed access to the gRPC surface.

No service implementation in this PR. No business logic. No tests beyond a 5-line cross-crate import smoke test. The PR's purpose is purely **wire-protocol enablement**.

Critical-path significance : **PRs #6, #8, #9, #11, #20, #21 all depend on PR #5**. A late merge of #5 cascades into a >= 1 day delay on Phase 2.

1. Goal + non-goals

1.1 Goal (single sentence)

> *Land the canonical, frozen-at-v1 Protocol Buffers definitions for the* `MetaSwarmService` + `MetaParticipantService` *gRPC surfaces, generate tonic client + server stubs, and expose them as a separate, dependency- minimal crate that the rest of the workspace can pull in without linking the entire algorithmic surface of* `efe-meta-swarm`.

1.2 Non-goals

Items deliberately excluded from PR #5 :

#	Excluded	Lives in	Why
NG1	Service implementation (any RPC body beyond `todo!()`)	PR #6 + #8 + #9	Keeps PR #5 small + reviewable.
NG2	Authentication / mTLS code	PR #16	Auth is a server-side concern ; wire format is auth-agnostic.
NG3	Discovery (static / k8s-dns)	PR #11	Same reason as NG2.
NG4	Metrics (Prometheus registry)	PR #17	Same.
NG5	OTel span emission	PR #18	Same.
NG6	Client adapter `RemoteMetaSwarm`	PR #6	This is a thin wrapper over the generated client ; ships in the server crate.
NG7	Server adapter `MetaSwarmGrpcServer`	PR #8	Wraps Phase-1 `MetaSwarm` ; depends on `efe-meta-swarm`.
NG8	Integration test (multi-process)	PR #10	Needs PR #6, #8, #9 first.
NG9	Helm chart changes	PR #13	Chart references the binary, not the proto crate.
NG10	Documentation in `EFE_A1_BENCH_TRACKING.md` extension for A2 SLOs	PR #22	A2 SLOs don't exist yet at PR #5 merge.

1.3 Single binary unit of merge-readiness

PR #5 is mergeable iff a sibling toy crate (created in the PR's self-contained smoke test) can `use efe_meta_swarm_proto::...` and `cargo build` succeeds, with **0 warnings on a fresh** `cargo check` and `cargo doc -p efe-meta-swarm-proto --no-deps` clean.

Anything else (service impl, business logic) is a non-goal for this PR.

2. Pre-flight checks

Before opening the branch, verify :

#	Check	How	Expected
PF1	Phase 2 architecture frozen	`git log -1 docs/EFE_A1_PHASE2_ARCHITECTURE.md`	Last commit before 2026-07-06
PF2	Phase 2 roadmap frozen	`git log -1 docs/EFE_A1_PHASE2_ROADMAP.md`	id.
PF3	Phase 1 master gate still GREEN on `main`	`gh run list -w ops-master-gate.yml -L 3`	All 3 PASS
PF4	rustc toolchain pinned 1.88	`cat rust-toolchain.toml`	`channel = "1.88"`
PF5	`tonic-build` version available offline	`ls ~/.cargo/registry/cache/*/tonic-build-0.12`	present
PF6	`protoc` available (host)	`protoc --version`	>= 3.20 -- OR rely on the `protoc` bundled by tonic-build (recommended : YES, see § 7)
PF7	OQ1 + OQ2 decided	`docs/A1_PHASE2_DECISIONS.md`	f32 wire (OQ1=f32) ; StepStream optional (OQ2=optional)

If any PF check fails, the PR is **blocked at preflight**. Do not branch ; resolve the upstream issue first.

3. Branch + commit plan

3.1 Branch

```
A1.P2/05-meta-swarm-proto-scaffold
```

(Naming convention : `<sprint>/<PR-number>-<short-slug>`.)

3.2 Commit-by-commit plan

The PR ships in **5 commits**, each individually green (CI passes on each). Reviewer can step through them in order :

#	Commit message	Files	LOC ±	Verifies
C1	`chore(workspace): register efe-meta-swarm-proto crate`	`Cargo.toml`, `crates/efe-meta-swarm-proto/Cargo.toml`, `crates/efe-meta-swarm-proto/src/lib.rs` (placeholder), `crates/efe-meta-swarm-proto/README.md` (placeholder)	+120	`cargo check -p efe-meta-swarm-proto` (placeholder lib builds)
C2	`feat(proto): add messages.proto + errors.proto + build.rs`	+`build.rs`, +`proto/efe/v1/meta_swarm/messages.proto`, +`proto/efe/v1/meta_swarm/errors.proto`	+400	`cargo build -p efe-meta-swarm-proto` (no services yet, but messages compile)
C3	`feat(proto): add MetaSwarmService`	+`proto/efe/v1/meta_swarm/service_meta_swarm.proto`, +tonic-build wiring	+200	`cargo build` ; client + server traits exist
C4	`feat(proto): add MetaParticipantService`	+`proto/efe/v1/meta_swarm/service_meta_participant.proto`	+180	Same as C3, both services exist

#	Commit message	Files	LOC ±	Verifies
C5	`docs(proto): README + versioning policy + smoke test`	`README.md` (full), `tests/smoke_cross_crate.rs`	+280	`cargo test -p efe-meta-swarm-proto` + `cargo doc`

Total : ~ **1180 LOC** (incl. proto comments, README, lib.rs, build.rs).

Each commit has a one-paragraph commit body following the workspace convention (`<type>(<scope>): <summary>` + blank line + rationale).

3.3 Drafting strategy

The author opens the PR as a **Draft** after C1, then pushes C2-C5 incrementally. Reviewers can start reading from C1 ; CI runs on each push. PR moves to "Ready for review" once C5 is up.

4. Workspace integration

4.1 `Cargo.toml` (workspace) -- diff

Add the new member. Keep the existing alphabetical / by-area ordering :

```
 [workspace]
 resolver = "2"
 members = [
     "crates/efe-core",
     "crates/efe-meta-swarm",
+    "crates/efe-meta-swarm-proto",
     "crates/efe-server",
     # ... (rest unchanged)
 ]
```

And in `[workspace.dependencies]`, add the Phase 2 transport deps so all member crates use the same pinned versions :

```
 [workspace.dependencies]
 # ... existing deps unchanged ...
+
+# === Phase 2 (PR #5+) -- gRPC transport ===
+tonic            = { version = "0.12", default-features = false, features = ["codegen", "prost", "tls",  ■
    "tls-roots", "transport"] }
+tonic-build      = { version = "0.12", default-features = false, features = ["prost", "transport"] }
+prost            = { version = "0.13", default-features = false, features = ["std", "derive"] }
+prost-types      = { version = "0.13", default-features = false, features = ["std"] }
```

Rationale :

- `default-features = false` everywhere : minimise transitive deps,

reduce build time, keep the supply chain auditable.

- `tonic` is pinned at `0.12` (last minor before `0.13`). Pin until

Phase 3 (where a deliberate bump happens with sec-team review).

- `prost 0.13` matches `tonic 0.12`'s requirement.
- `tls-roots` : exposes the system root certs (`rustls-native-certs`)

for outbound TLS validation in dev. Server-side TLS material comes from cert-manager at runtime (cf. PR #16).

4.2 `Cargo.lock`

`Cargo.lock` will be updated automatically by `cargo build`. The PR **must commit the lockfile diff** to keep CI builds deterministic. The diff is expected to be ~30 lines (transitive deps of tonic 0.12).

If a reviewer sees more than 100 lockfile lines added, that's a red flag : something pulled an unexpected transitive -- investigate.

5. Crate layout

The full tree of the new crate :

```
crates/efe-meta-swarm-proto/
+-- Cargo.toml                                  # § 6
+-- build.rs                                    # § 7
+-- README.md                                   # § 14
+-- proto/
|   +-- efe/
|       +-- v1/
|           +-- meta_swarm/
|               +-- messages.proto              # § 8
|               +-- errors.proto                # § 9
|               +-- service_meta_swarm.proto    # § 10
|               +-- service_meta_participant.proto  # § 11
+-- src/
|   +-- lib.rs                                  # § 12
+-- tests/
    +-- smoke_cross_crate.rs                    # § 18
```

Why this exact layout :

- `proto/efe/v1/meta_swarm/` mirrors the protobuf package

`efe.v1.meta_swarm`. Makes import paths predictable in any language that consumes the protos later (go/python/java/typescript), even though Phase 2 only uses Rust.

- `proto/` not `src/proto/` : keeps the protos discoverable for the

upcoming public proto-distribution work (Phase 3 will ship them as a separate buf module).

- 4 separate files (instead of 1 monolithic) : reviewers can scope

changes to one surface at a time ; tonic-build glues them via `import` statements.

Why NOT :

- [FAIL] `src/` containing both `proto/` and `rs/` : confuses cargo's

default include rules.

- [FAIL] A single `meta_swarm.proto` file with everything : 700+ LOC, hard

to review, hard to evolve.

- [FAIL] Protos under `crates/efe-meta-swarm-proto/src/proto/` : tonic-build

works but search tools like grep/IDE struggle.

6. `Cargo.toml` -- full

```
[package]
name        = "efe-meta-swarm-proto"
version     = "0.2.0"            # Phase-2 minor, since Phase-1 was 0.1.x
edition     = "2021"
rust-version = "1.88"
description = "Protocol Buffers + tonic-generated client/server stubs for the EFE meta-swarm gRPC surface ■
     (MetaSwarmService + MetaParticipantService)."
publish     = false              # carve-out R5-B1 ; UNLICENSED until Phase 5
license     = "Apache-2.0"       # advisory only ; not yet effective per R5-B1

# ---------------------------------------------------------------------
# Dependencies (runtime).
# ---------------------------------------------------------------------
[dependencies]
tonic        = { workspace = true }
prost        = { workspace = true }
prost-types  = { workspace = true }

# Optional : serde derives on generated message types.
# Used by `bench_public/` (PR #21) to emit JSON for the HTML report,
# and by future Phase-3 customer-facing CLI tools.
serde        = { version = "1.0", features = ["derive"], optional = true }

# ---------------------------------------------------------------------
# Dependencies (build).
# ---------------------------------------------------------------------
[build-dependencies]
tonic-build  = { workspace = true }
prost-build  = { workspace = true }

# ---------------------------------------------------------------------
# Features.
# ---------------------------------------------------------------------
[features]
# Default : minimal, no JSON.  Most consumers don't need it.
default = []

# Enable serde derives on every generated message.  ~50 kB binary size
# overhead.  Required by `bench_public` + future CLI / SDK.
serde-derives = ["dep:serde"]

# Enable client-only build (no server stubs).  Drops ~30 % of generated
# code for binaries that only consume the surface.  Used by efe-server
# in Phase 2 to keep its size lean.
client-only = []

# Enable server-only build.  Mirror of `client-only`.  Used in builds
# that ship only the controller/participant binaries.
server-only = []

# All-features convenience : everything.  Used by `cargo doc` + CI
# matrix.
full = ["serde-derives"]

# ---------------------------------------------------------------------
# Lints -- match workspace convention.
# ---------------------------------------------------------------------
[lints]
workspace = true

# Note : we explicitly allow some lints that fire on tonic-generated
# code (we don't control its style).  These are *crate-level* allows ;
# downstream crates are unaffected.
[lints.rust]
unsafe_code = "forbid"

[lints.clippy]
# Generated code triggers these -- silence at crate level.
module_inception        = "allow"
missing_const_for_fn    = "allow"
needless_pass_by_value  = "allow"
must_use_candidate      = "allow"
return_self_not_must_use = "allow"
unreadable_literal      = "allow"
derive_partial_eq_without_eq = "allow"
large_enum_variant      = "allow"

# ---------------------------------------------------------------------
# Documentation hints (rustdoc).
# ---------------------------------------------------------------------
```

```
[package.metadata.docs.rs]
all-features = true
rustdoc-args = ["--cfg", "docsrs"]
```

6.1 Decisions encoded

Item	Choice	Why
Package version	`0.2.0`	Phase 2 = minor bump over Phase 1's `0.1.x` line.
`publish = false`	yes	Carve-out R5-B1.
`license = "Apache-2.0"` advisory	yes	Documents target license post-R5-B1 ; legal review tag remains UNLICENSED.
`serde` optional	yes	Most consumers don't need it ; ~50 kB binary size.
`client-only` / `server-only` features	yes	Lets efe-server drop server stubs (smaller binary, less attack surface).
`default = []`	yes	Smallest surface by default. Consumers opt in.
`[lints.clippy]` allow list	yes	tonic-generated code triggers them ; not under our control.

7. `build.rs` -- full

```
//! Build script for `efe-meta-swarm-proto`.
//!
//! Compiles 4 .proto files into Rust client + server stubs via
//! `tonic-build` (which delegates to `prost-build`).  All
//! configuration is explicit -- we do not rely on
//! `tonic_build::compile_protos()` defaults -- because the resulting
//! code is shipped to production and we want every choice auditable.
//!
//! ## Outputs
//!
//! `OUT_DIR/efe.v1.meta_swarm.rs` -- the single generated file
//! containing all proto messages + service traits.  This is included
//! in `src/lib.rs` via `tonic::include_proto!("efe.v1.meta_swarm")`.
//!
//! ## Re-run conditions
//!
//! By default `cargo` re-runs `build.rs` on any change to the
//! `build.rs` itself or any file in the crate.  We emit explicit
//! `rerun-if-changed` for each `.proto` file so a touch to a proto
//! triggers a regeneration but other crate-internal changes don't.

use std::path::PathBuf;

fn main() {
    let proto_dir = PathBuf::from("proto");
    let protos: Vec&lt;PathBuf&gt; = [
        "efe/v1/meta_swarm/messages.proto",
        "efe/v1/meta_swarm/errors.proto",
        "efe/v1/meta_swarm/service_meta_swarm.proto",
        "efe/v1/meta_swarm/service_meta_participant.proto",
    ]
    .iter()
    .map(|p| proto_dir.join(p))
    .collect();

    // Tell cargo to re-run on any proto change.
    for p in &amp;protos {
        println!("cargo:rerun-if-changed={}", p.display());
    }
    println!("cargo:rerun-if-changed=build.rs");

    let mut builder = tonic_build::configure()
        // Where to emit the generated code (default = OUT_DIR ; explicit).
        .out_dir(std::env::var("OUT_DIR").expect("OUT_DIR set by cargo"))
        // Enable / disable client + server based on features.
        // Pattern : `(client-only, server-only) -&gt; (build_client, build_server)`.
        .build_client(!cfg!(feature = "server-only"))
        .build_server(!cfg!(feature = "client-only"))
        // Generate descriptive doc comments on every generated item.
        // tonic-build forwards `//` proto comments to rustdoc by default
        // ; this is just defensive.
        .build_transport(true)
        // Strict naming : tonic-build's default is `EfeV1MetaSwarmServiceClient`
        // ; we shorten to `MetaSwarmServiceClient` via package-relative
        // type-name elision (default behaviour ; explicit for clarity).
        .compile_well_known_types(false)
        // Suppress the auto-generated `protoc` include path bin from
        // ending up in the docs.
        .disable_comments(&amp;[]);

    // Inject `serde::{Serialize, Deserialize}` derives on every
    // generated message when the `serde-derives` feature is enabled.
    // We list the types explicitly so future schema additions don't
    // implicitly become serde-able (forces a deliberate decision).
    if cfg!(feature = "serde-derives") {
        let types_with_serde: &amp;[&amp;str] = &amp;[
            "TensorF32",
            "MetaSpaceConfig",
            "BoxSafety",
            "Polyhedron",
            "LearnedHull",
            "ParticipantDescriptor",
            "IterationConfig",
            "IterateRequest",
            "IterateResponse",
            "IterateChunk",
            "StopReason",
            "StepRequest",
            "StepResponse",
            "ResetRequest",
```

```
                "ResetResponse",
                "ResetParticipantRequest",
                "ResetParticipantResponse",
                "StateRequest",
                "StateResponse",
                "CapabilitiesRequest",
                "CapabilitiesResponse",
                "ReloadConfigRequest",
                "ReloadConfigResponse",
                "HealthRequest",
                "HealthResponse",
                "InfoRequest",
                "InfoResponse",
                "MetaSwarmErrorCode",
            ];
            for ty in types_with_serde {
                builder = builder.type_attribute(
                    ty,
                    "#[derive(::serde::Serialize, ::serde::Deserialize)]",
                );
            }
        }

        // Compile.  This is the single fallible step ; failure here is a
        // hard error and aborts the build with a clear diagnostic.
        builder
            .compile(&amp;protos, &amp;[proto_dir])
            .expect("tonic-build : failed to compile .proto files");
    }
```

7.1 Why each option

Option	Why
Explicit `rerun-if-changed` per proto file	Avoids the default "any source change re-runs" behaviour ; faster incremental builds.
`build_client(!server-only)` / `build_server(!client-only)`	Lets consumers drop unused half.
`compile_well_known_types(false)`	We do not use `google.protobuf.*` ; skip pulling those in.
`protoc` source	Bundled by `tonic-build` 0.12 (uses `protoc-bin-vendored`) -- no host `protoc` install required. Important : CI runners don't have protoc by default.
`type_attribute` for serde	Selective : only listed types get serde. Future additions require an explicit ack.

7.2 Build failure modes

Symptom	Cause	Fix
`protoc: command not found`	`tonic-build` could not find / extract protoc	Verify `tonic-build = "0.12"` (NOT `tonic-build = "0.10"`) ; the bundled-protoc feature is on by default >= 0.12.
`error[E0432]: unresolved import` in `tonic::include_proto!`	tonic-build wrote to a different `OUT_DIR`	Run `cargo clean` ; verify `OUT_DIR` env var is set (always true under cargo).
`field number conflict`	Two fields in the same message share a number	Cf. § 16 ; renumber.
`package "efe.v1.meta_swarm" does not import "google/..."`	Old `prost-build` insists on resolving well-known	Set `compile_well_known_types(false)`.

8. Proto file #1 -- `messages.proto`

This file defines the shared messages : tensors, configs, descriptors. No services, no errors. Imported by both service files.

```
// Copyright (C) 2026 EFE.  UNLICENSED -- see ../../../README.md.
//
// Phase-2 (Sprint A1) messages : tensors, scenario configs,
// participant descriptors.  Stable from v0.2.0 onwards ; field
// numbers 1-100 are reserved for Phase 2 ; &gt;= 100 are reserved for
// Phase 3+ additions.  See `crates/efe-meta-swarm-proto/README.md`
// for the versioning policy.
```

```
syntax = "proto3";

package efe.v1.meta_swarm;

// ============================================================================
//  TensorF32 -- dense float32 tensor on the wire.
// ============================================================================
//
// Single-precision floats are the Phase-2 default.  Rationale :
//   * The bench scenario (`three_swarms_kovant`) operates in f32 in
//     Phase-1 already ; round-trip via f32 preserves bit identity.
//   * Halves bandwidth vs f64 (256-d x 4 bytes = 1 KiB per state ; vs
//     2 KiB for f64).  Material at 47 iters x 3 participants x 100 rps.
//   * Tenants requiring f64 will opt in via the (yet-to-be-added)
//     `precision` field of `IterationConfig` in Phase 3 ;
//     a `TensorF64` message will be added then.
//
// Layout :
//   - `dim` is the logical dimension.
//   - `data` is row-major, length == dim.  Length mismatch is rejected
//     server-side with META_ERROR_DIM_MISMATCH.
//
// Wire size : 8 bytes (varint dim) + 4 x dim bytes (packed f32) + ~4
// bytes (tag overhead).
message TensorF32 {
  uint32 dim       = 1;
  repeated float data = 2 [packed = true];

  // Reserved for Phase 3 :
  //   - `quantization` (int8 packed) at field 10
  //   - `sparsity_indices` at field 11
  reserved 10 to 99;
}

// ============================================================================
//  Safety geometry -- one-of dispatch.
// ============================================================================

// `MetaSpaceConfig` describes the meta-space dimension + the safety
// geometry the meta-consensus must respect.  The safety is enforced
// server-side at every meta-step (cf. PR #1 `BoxSafety::contains` +
// `MetaSwarm::safety_project`).
message MetaSpaceConfig {
  uint32 dim = 1;

  // Exactly one of these MUST be set.  Server rejects with
  // META_ERROR_INVALID_SAFETY otherwise.
  oneof safety_geometry {
    BoxSafety   box  = 2;
    Polyhedron  poly = 3;
    LearnedHull hull = 4;
  }

  // Reserved 5-99 for future geometries (cone, ellipsoid, ...).
  reserved 5 to 99;
}

// Symmetric box safety :  ■x■∞ &lt;= radius (per axis).  Phase-2 default.
message BoxSafety {
  // Symmetric radius applied to every axis.  Asymmetric boxes
  // (different bound per axis) are deferred to Phase 3 ; will be
  // expressed via a future `repeated float per_axis_radius`.
  float radius = 1;

  reserved 2 to 99;
}

// Half-space polytope : { x : A·x &lt;= b }.
//
// Layout :
//   - `a_flat` is the flattened constraint matrix, row-major, of
//     length `dim_a x dim` where `dim` is read from
//     `MetaSpaceConfig.dim`.
//   - `b` is the right-hand side, length `dim_a`.
//   - `dim_a` is the number of constraints.
message Polyhedron {
  repeated float a_flat = 1 [packed = true];
  repeated float b      = 2 [packed = true];
  uint32 dim_a          = 3;

  reserved 4 to 99;
}
```

```
// Learned hull : a serialised neural distance function or similar.
// Phase-2 placeholder ; not exercised by the soak (the soak uses
// BoxSafety).
message LearnedHull {
  bytes  serialized_model = 1;   // bincode-encoded in Phase 2
  string version          = 2;

  reserved 3 to 99;
}

// ============================================================================
//  Participant descriptor -- capability handshake.
// ============================================================================
//
// Returned by `MetaParticipantService::Capabilities`.  Allows the
// controller to verify that every participant matches the manifest's
// expectation before starting an `Iterate`.  Mismatches surface as
// META_ERROR_DIM_MISMATCH or META_ERROR_KAPPA_OUT_OF_RANGE.

message ParticipantDescriptor {
  string id           = 1;   // unique within a MetaSwarm (e.g. "supplier-scoring")
  uint32 dim          = 2;
  float  kappa        = 3;   // contraction constant ; must be in (0, 1)
  uint64 seed         = 4;
  string build_sha    = 5;   // git short SHA of the participant binary
  string blas_backend = 6;   // "none" | "openblas" | "mkl" | "metal" | ...
  repeated string features = 7;  // e.g. ["determinism", "metrics", "trace"]

  // Optional : self-reported max steps/sec for capacity planning.
  // Reserved for Phase 3.  Phase 2 readers MUST ignore.
  reserved 8 to 99;
}

// ============================================================================
//  Iteration config -- knobs the controller forwards from its manifest.
// ============================================================================

message IterationConfig {
  uint32 max_iters      = 1;   // capped at 200 server-side
  double tolerance      = 2;   // joint residual threshold
  double alpha_default  = 3;   // step damping
  double coupling_gamma = 4;   // meta-operator coupling

  // If true, the server guarantees that two iterates from the same
  // initial state produce bit-identical results.  Implementation : no
  // parallel non-deterministic reductions ; fixed rayon thread count ;
  // disabled BLAS multithreading.
  bool deterministic = 5;

  // Wall-time budget.  Server caps at 30000 (30 s) ; longer requests
  // are rejected with DEADLINE_EXCEEDED.
  uint32 max_wall_ms = 6;

  // Reserved for Phase 3 :
  //   - 10 : f64 precision opt-in
  //   - 11 : per-step tracing override
  reserved 10 to 99;
}

// ============================================================================
//  StopReason -- why an Iterate terminated.
// ============================================================================

enum StopReason {
  STOP_REASON_UNSPECIFIED      = 0;
  STOP_REASON_CONVERGED        = 1;
  STOP_REASON_MAX_ITERS        = 2;
  STOP_REASON_SAFETY_FAIL      = 3;
  STOP_REASON_PARTICIPANT_DOWN = 4;
  STOP_REASON_DEADLINE         = 5;
  STOP_REASON_CONFIG_RELOAD    = 6;

  // Reserved for Phase 3 :
  //   STOP_REASON_TENANT_QUOTA = 10;
  //   STOP_REASON_REGION_FAILOVER = 11;
  reserved 7 to 9, 10 to 99;
}

// ============================================================================
//  Iterate request / response  (used by MetaSwarmService).
// ============================================================================

message IterateRequest {
```

```
  // Optional : if absent, the controller uses its persisted manifest.
  // If present, MUST match the controller's expectation exactly ;
  // a mismatch is META_ERROR_DIM_MISMATCH.
  repeated ParticipantDescriptor expected_participants = 1;

  MetaSpaceConfig meta      = 2;
  IterationConfig iteration = 3;

  // Optional : initial participant states (in the same order as
  // `expected_participants`).  Absent ⇔ start from each participant's
  // last state (or zero if Reset was just called).
  repeated TensorF32 initial_states = 4;

  // W3C TraceParent header.  If absent, the controller starts a fresh
  // trace and returns its trace_id in IterateResponse.trace_id.
  string traceparent = 5;

  reserved 6 to 99;
}

message IterateResponse {
  uint32     iterations          = 1;
  double     final_residual      = 2;
  TensorF32  meta_consensus      = 3;
  repeated TensorF32 final_states = 4;
  StopReason stop_reason          = 5;
  uint64     wall_us              = 6;
  string     trace_id             = 7;

  reserved 8 to 99;
}

// Streaming chunk -- one inner step per chunk.
message IterateChunk {
  uint32 step     = 1;
  double residual = 2;
  // Per-participant inner state norms -- useful for the bench HTML
  // (PR #21 + #24).
  repeated float participant_norms = 3 [packed = true];

  reserved 4 to 99;
}

// ============================================================================
//  StepOnce request / response (used by MetaSwarmService).
// ============================================================================

message StepOnceRequest {
  // No body for now -- the controller uses its persisted state.
  // Reserved for Phase 3 add of explicit-state-input.
  reserved 1 to 99;
}

message StepOnceResponse {
  TensorF32 meta_consensus = 1;
  double    residual       = 2;
  reserved 3 to 99;
}

// ============================================================================
//  Reset request / response (used by both services).
// ============================================================================

message ResetRequest {
  // Optional : per-participant initial states.  Empty ⇔ zero state.
  repeated TensorF32 initial_states = 1;
  reserved 2 to 99;
}

message ResetResponse {
  uint64 reset_gen = 1;   // monotonic counter, incremented per reset
  reserved 2 to 99;
}

message ResetParticipantRequest {
  // Optional : initial state for this participant.  Empty ⇔ zero.
  TensorF32 initial = 1;
  reserved 2 to 99;
}

message ResetParticipantResponse {
  uint64 reset_gen = 1;
  reserved 2 to 99;
```

```
}

// ============================================================================
//  Step request / response (used by MetaParticipantService).
// ============================================================================

message StepRequest {
  TensorF32 pulled_from_meta = 1;
  uint32    step_index       = 2;
  reserved 3 to 99;
}

message StepResponse {
  TensorF32 new_state      = 1;
  double    local_residual = 2;
  uint64    step_us        = 3;   // latency self-report
  reserved 4 to 99;
}

// ============================================================================
//  State / Capabilities (used by MetaParticipantService).
// ============================================================================

message StateRequest {
  reserved 1 to 99;
}
message StateResponse {
  TensorF32 current = 1;
  uint32    step_index_last = 2;
  reserved 3 to 99;
}

message CapabilitiesRequest {
  reserved 1 to 99;
}
message CapabilitiesResponse {
  ParticipantDescriptor descriptor = 1;
  reserved 2 to 99;
}

// ============================================================================
//  ReloadConfig (MetaSwarmService).
// ============================================================================

message ReloadConfigRequest {
  // If empty : re-read manifest from disk.
  // If non-empty : apply inline (admin RPC).
  optional MetaSpaceConfig new_meta       = 1;
  repeated ParticipantDescriptor new_participants = 2;

  // If true (default) : wait up to 30 s for in-flight Iterate to drain.
  // If false : abort in-flight Iterate with STOP_REASON_CONFIG_RELOAD.
  bool drain_inflight = 3;

  reserved 4 to 99;
}

message ReloadConfigResponse {
  uint32 reload_gen       = 1;
  repeated string applied_changes = 2;
  uint64 reload_wall_us   = 3;
  bool   inflight_aborted = 4;
  reserved 5 to 99;
}

// ============================================================================
//  Health (both services).
// ============================================================================

message HealthRequest {
  reserved 1 to 99;
}

message HealthResponse {
  enum Status {
    STATUS_UNSPECIFIED = 0;
    STATUS_SERVING     = 1;
    STATUS_NOT_SERVING = 2;
    STATUS_RELOADING   = 3;   // SIGHUP in flight
  }

  Status status     = 1;
  uint32 inflight   = 2;
```

```
  uint32 reload_gen = 3;

  reserved 4 to 99;
}

// ===================================================================================
//  Info (MetaSwarmService).
// ===================================================================================

message InfoRequest {
  reserved 1 to 99;
}

message InfoResponse {
  string build_sha       = 1;
  string proto_version   = 2;   // "v1" -- frozen at v0.2.0 PR #5
  repeated ParticipantDescriptor configured_participants = 3;
  string leader_pod_name = 4;
  uint64 uptime_s        = 5;
  uint32 last_reload_gen = 6;

  reserved 7 to 99;
}
```

8.1 Field-number choices

Every `reserved 10..99` block is **intentional**. It pre-reserves slots for Phase 3+ additions so we can't accidentally re-use numbers that may have meaning in a later v2 of the surface. Cf. § 16.

8.2 Wire-size budget (per Iterate call, expected)

Item	Bytes	Notes
IterateRequest envelope	~ 50	tag overhead, no initial states
IterateResponse envelope	~ 30	
Meta-consensus TensorF32 (dim=256)	1 050	256 x 4 + ~20 tag overhead
3 x final_states (dim=384, 512, 384)	4 850	sum of 3 tensors
Per inner step (3 participants)	~ 4 350	3 x (pulled + new_state)
x 47 iters	~ 204 000	dominates the wire cost
Total per Iterate	**~ 210 KB**	mostly the 47-iter inner traffic

At 100 rps : ~ 21 MB/s of meta-swarm gRPC traffic. Well under a 1 Gbps pod link. StepStream (Phase 2 optional) would halve this by saving the per-step request headers.

9. Proto file #2 -- `errors.proto`

```
// Copyright (C) 2026 EFE.  UNLICENSED.
//
// Phase-2 (Sprint A1) error codes for MetaSwarm + MetaParticipant
// services.  Sent in the gRPC trailer's `error_code` field (which
// is the binary serialised `MetaSwarmErrorCode` value) alongside the
// standard gRPC status code.
//
// Mapping :
//   gRPC code FAILED_PRECONDITION &lt;- configuration errors (1xx)
//   gRPC code INTERNAL            &lt;- runtime errors (10xx)
//   gRPC code UNAVAILABLE         &lt;- network / liveness (20xx)
//   gRPC code DEADLINE_EXCEEDED   &lt;- max_wall_ms or RPC deadline
//   gRPC code UNAUTHENTICATED     &lt;- mTLS / auth failures (30xx)

syntax = "proto3";

package efe.v1.meta_swarm;

enum MetaSwarmErrorCode {
  META_ERROR_UNSPECIFIED            = 0;

  // ---------------------------------------------------------------
  // Configuration errors -&gt; gRPC FAILED_PRECONDITION.
  // ---------------------------------------------------------------
  META_ERROR_DIM_MISMATCH           = 1;
  META_ERROR_DUPLICATE_PARTICIPANT_ID = 2;
  META_ERROR_INVALID_SAFETY         = 3;
  META_ERROR_KAPPA_OUT_OF_RANGE     = 4;
  META_ERROR_MAX_ITERS_EXCEEDS_CAP  = 5;   // server cap 200
  META_ERROR_TOLERANCE_NON_POSITIVE = 6;
  META_ERROR_DETERMINISTIC_REQUIRED = 7;   // a feature flag says "must be deterministic" but request ask ■
    ed otherwise

  // ---------------------------------------------------------------
  // Runtime errors -&gt; gRPC INTERNAL.
  // ---------------------------------------------------------------
  META_ERROR_PARTICIPANT_DOWN       = 1001;
  META_ERROR_DETERMINISM_BROKEN     = 1002;
  META_ERROR_SAFETY_VIOLATION       = 1003;
  META_ERROR_DIVERGENCE_DETECTED    = 1004;
  META_ERROR_INFLIGHT_LIMIT         = 1005;

  // ---------------------------------------------------------------
  // Reload errors -&gt; gRPC FAILED_PRECONDITION.
  // ---------------------------------------------------------------
  META_ERROR_RELOAD_BUSY            = 2001;   // another reload in flight
  META_ERROR_RELOAD_DIM_CHANGED     = 2002;   // dimension change not supported live
  META_ERROR_RELOAD_MANIFEST_BAD    = 2003;   // unparseable / invalid file

  // ---------------------------------------------------------------
  // Discovery / network -&gt; gRPC UNAVAILABLE.
  // ---------------------------------------------------------------
  META_ERROR_PARTICIPANT_UNREACHABLE = 3001;
  META_ERROR_CONTROLLER_NOT_READY    = 3002;
  META_ERROR_TLS_HANDSHAKE_FAIL      = 3003;

  // ---------------------------------------------------------------
  // Auth -&gt; gRPC UNAUTHENTICATED.
  // ---------------------------------------------------------------
  META_ERROR_AUTH_INVALID_CERT      = 4001;
  META_ERROR_AUTH_SCOPE_DENIED      = 4002;
  META_ERROR_AUTH_TOKEN_EXPIRED     = 4003;

  // ---------------------------------------------------------------
  // Reserved for Phase 3 :
  //   META_ERROR_TENANT_QUOTA_EXCEEDED = 5001;
  //   META_ERROR_REGION_UNREACHABLE    = 5002;
  // ---------------------------------------------------------------
  reserved 5000 to 9999;
}
```

9.1 Decoupling rationale

Error codes live in their own file because :

- They're consumed by **both** services + by clients.
- They evolve more frequently than messages (every new failure path

becomes a code).

- Reviewers can scope error-code reviews to one file.

10. Proto file #3 -- `service_meta_swarm.proto`

```
// Copyright (C) 2026 EFE.  UNLICENSED.
//
// Phase-2 (Sprint A1) -- MetaSwarmService : the coordinator-facing
// gRPC surface.  Implemented by `efe-meta-swarm-server` (controller
// binary).  Called by `efe-server` (when `EFE_META_SWARM_ENABLED=true`)
// and by the bench harness (PR #21).

syntax = "proto3";

package efe.v1.meta_swarm;

import "efe/v1/meta_swarm/messages.proto";

// ============================================================================
//  MetaSwarmService -- 7 RPCs.
// ============================================================================
//
// All RPCs are mTLS-authenticated (PR #16) ; the caller's SAN MUST be
// `efe-server.efe.svc.cluster.local` (or the local-dev equivalent) to
// be authorised.  RPC-specific scope checks (e.g. `admin` for
// ReloadConfig) are enforced server-side.

service MetaSwarmService {

  // ---------------------------------------------------------------
  // Resolve a fixed-point of the joint contractive system.
  //
  // This is the workhorse RPC.  Maps directly to Phase-1
  // `MetaSwarm::iterate()`.  Blocking : caller is expected to wait up
  // to `iteration.max_wall_ms` (server-capped at 30 s).
  //
  // Concurrency : the server may run multiple concurrent Iterate
  // calls up to `EFE_META_SWARM_MAX_INFLIGHT` (default 10).  Beyond
  // that, returns RESOURCE_EXHAUSTED.
  //
  // Cancellation : if the client drops the RPC, the server cancels
  // its participant calls within 100 ms.
  //
  // Idempotence : two calls with identical inputs (same manifest,
  // same `expected_participants`, same `initial_states`, same
  // `iteration.deterministic = true`) produce bit-identical responses.
  // Determinism is verified by SLO-A2-GRPC-MULTIPROC-DETERMINISM.
  rpc Iterate (IterateRequest) returns (IterateResponse);

  // ---------------------------------------------------------------
  // Streaming variant -- emits one chunk per inner step.  Used by the
  // public bench (PR #21+#24) to capture the residual trajectory.
  // Final chunk includes the IterateResponse fields via trailer.
  //
  // Not on the hot path ; do NOT use from `efe-server`.
  rpc IterateStream (IterateRequest) returns (stream IterateChunk);

  // ---------------------------------------------------------------
  // One-shot evaluation of the meta-operator at the current state.
  // Maps to `MetaSwarm::step_once()`.  Reserved for latency-sensitive
  // Phase-3 paths ; Phase-2 callers prefer Iterate().
  rpc StepOnce (StepOnceRequest) returns (StepOnceResponse);

  // ---------------------------------------------------------------
  // Reset all participants to their seed state (or to a provided
  // initial set).  Maps to `MetaSwarm::reset()`.
  //
  // Drains in-flight Iterate calls first (server-side).
  rpc Reset (ResetRequest) returns (ResetResponse);

  // ---------------------------------------------------------------
  // Hot-reload manifest + safety geometry.  Triggered by SIGHUP on
  // the controller (in which case the request body is empty and the
  // controller re-reads its on-disk manifest), OR by an admin RPC
  // (request body non-empty).
  //
  // Requires the caller's auth token to have the `admin` scope.
  //
  // SLO : reload wall time &lt;= 200 ms p99 (SLO-A2-RELOAD-P99).
  rpc ReloadConfig (ReloadConfigRequest) returns (ReloadConfigResponse);

  // ---------------------------------------------------------------
  // Liveness + readiness probe.  Returns within 1 ms.  Safe to call
  // at high frequency.  Used by the K8s lease leader election +
  // Helm `helm test` connection probe.
```

```
  rpc Health (HealthRequest) returns (HealthResponse);

  // ---------------------------------------------------------------
  // Build + topology info for the dashboard.  Cheap.  Read-only.
  rpc Info (InfoRequest) returns (InfoResponse);
}
```

11. Proto file #4 -- `service_meta_participant.proto`

```
// Copyright (C) 2026 EFE.  UNLICENSED.
//
// Phase-2 (Sprint A1) -- MetaParticipantService : the participant-
// facing gRPC surface.  Implemented by `efe-meta-swarm-server`
// (participant binary).  Called by the controller only ; participants
// authorise callers by SAN-checking the leaf cert.

syntax = "proto3";

package efe.v1.meta_swarm;

import "efe/v1/meta_swarm/messages.proto";

// ============================================================================
//  MetaParticipantService -- 6 RPCs.
// ============================================================================

service MetaParticipantService {

  // ---------------------------------------------------------------
  // Receive a pulled vector and produce the next contractive step.
  // Maps to `SwarmParticipant::step_with_pull()`.
  //
  // This is the MOST-FREQUENT RPC of the entire transport.  At 47
  // iters x 3 participants x 100 rps ≈ 14 100 Step RPCs/sec on a
  // busy controller.
  //
  // Latency budget : &lt;= 3 ms p99 in-rack on a 4-vCPU pod.
  rpc Step (StepRequest) returns (StepResponse);

  // ---------------------------------------------------------------
  // Reset to the per-participant seed state.
  rpc Reset (ResetParticipantRequest) returns (ResetParticipantResponse);

  // ---------------------------------------------------------------
  // Return the current state tensor.  Not called on the hot path ;
  // reserved for the Phase-2 dashboard + the consistency probe.
  rpc State (StateRequest) returns (StateResponse);

  // ---------------------------------------------------------------
  // Server-streaming variant : participant emits one StepResponse
  // per incoming StepRequest, both sides streamed.  Saves one TLS
  // record per step (~ 30 % wire overhead reduction on the hot path).
  //
  // OPTIONAL : controllers MAY use Step() instead.  Whether to use
  // StepStream is decided per-deployment via
  // `EFE_META_SWARM_USE_STEP_STREAM=true|false` (default : false in
  // Phase 2 ; expected to flip to true in Phase 3 once we have
  // streaming-friendly load balancers).
  rpc StepStream (stream StepRequest) returns (stream StepResponse);

  // ---------------------------------------------------------------
  // Capability query.  Returns the participant's descriptor (dim,
  // kappa, seed, build_sha, blas_backend, features).
  //
  // Called once at controller startup + on every SIGHUP.  If the
  // returned descriptor mismatches the manifest's expectation, the
  // controller fails to start (or aborts the reload).
  rpc Capabilities (CapabilitiesRequest) returns (CapabilitiesResponse);

  // ---------------------------------------------------------------
  // Liveness + readiness probe.
  rpc Health (HealthRequest) returns (HealthResponse);
}
```

12. `src/lib.rs` -- full

```
//! Protocol Buffers + tonic-generated client/server stubs for the EFE
//! meta-swarm gRPC surface.
//!
//! This crate is **scaffolding only** : it contains no business logic.
//! The bulk of the source is generated at build-time by `tonic-build`
//! from the four `.proto` files under `proto/efe/v1/meta_swarm/`.
//!
//! ## Public surface
//!
//! After `cargo build`, consumers can import :
//!
//! ```ignore
//! use efe_meta_swarm_proto::{
//!     // services
//!     meta_swarm::MetaSwarmServiceClient,
//!     meta_swarm::MetaSwarmServiceServer,
//!     meta_swarm::MetaParticipantServiceClient,
//!     meta_swarm::MetaParticipantServiceServer,
//!     // messages
//!     meta_swarm::IterateRequest, meta_swarm::IterateResponse,
//!     meta_swarm::StepRequest,    meta_swarm::StepResponse,
//!     meta_swarm::TensorF32,
//!     meta_swarm::ParticipantDescriptor,
//!     meta_swarm::MetaSpaceConfig,
//!     meta_swarm::BoxSafety, meta_swarm::Polyhedron, meta_swarm::LearnedHull,
//!     meta_swarm::StopReason,
//!     meta_swarm::IterationConfig,
//!     // errors
//!     meta_swarm::MetaSwarmErrorCode,
//! };
//! ```
//!
//! See the [crate README](./../README.md) for the full versioning
//! policy + field-number reservation strategy.
//!
//! ## Features
//!
//!   * `serde-derives` : derives `serde::{Serialize, Deserialize}` on
//!     every generated message type.  Used by the public bench HTML
//!     emitter (PR #21+#24).
//!   * `client-only`   : drops server stubs.  Saves ~30 % binary size.
//!   * `server-only`   : drops client stubs.  Mirror of above.
//!   * `full`          : `serde-derives` (cargo doc convenience).
//!
//! ## Code generation
//!
//! Generated under `OUT_DIR/efe.v1.meta_swarm.rs`.  Included below via
//! `tonic::include_proto!`.

#![forbid(unsafe_code)]
#![cfg_attr(docsrs, feature(doc_cfg))]
#![deny(missing_debug_implementations)]
#![warn(rust_2018_idioms)]

/// All generated types live in this module.  We expose it under a
/// short alias so consumers write `meta_swarm::Foo` rather than
/// `efe::v1::meta_swarm::Foo`.
pub mod meta_swarm {
    tonic::include_proto!("efe.v1.meta_swarm");
}

/// Convenience re-export of the proto version constant.  Tested in CI
/// against the value the server sends back in `InfoResponse`.
pub const PROTO_VERSION: &amp;str = "v1";

/// The package name as it appears on the wire.  Useful for prefix-
/// matching in trace-based debugging.
pub const PROTO_PACKAGE: &amp;str = "efe.v1.meta_swarm";

/// All public RPC names, in the order they appear in the proto files.
/// Allows consumers to enumerate the surface without re-parsing the
/// proto file at runtime.  Tested in CI : a new RPC added without a
/// corresponding entry here will fail the smoke test.
pub const RPC_NAMES: &amp;[&amp;str] = &amp;[
    // MetaSwarmService
    "Iterate",
    "IterateStream",
    "StepOnce",
    "Reset",
    "ReloadConfig",
```

```
    "Health",
    "Info",
    // MetaParticipantService
    "Step",
    "Reset",            // intentionally duplicated : different service
    "State",
    "StepStream",
    "Capabilities",
    "Health",           // intentionally duplicated
];

/// Module aliases for the two services for ergonomic imports.
pub mod controller {
    //! Re-exports of the MetaSwarmService client/server traits.
    pub use crate::meta_swarm::meta_swarm_service_client::MetaSwarmServiceClient;
    #[cfg(not(feature = "client-only"))]
    pub use crate::meta_swarm::meta_swarm_service_server::{
        MetaSwarmService, MetaSwarmServiceServer,
    };
}

pub mod participant {
    //! Re-exports of the MetaParticipantService client/server traits.
    pub use crate::meta_swarm::meta_participant_service_client::MetaParticipantServiceClient;
    #[cfg(not(feature = "client-only"))]
    pub use crate::meta_swarm::meta_participant_service_server::{
        MetaParticipantService, MetaParticipantServiceServer,
    };
}

/// Helpers to convert between the proto types and idiomatic Rust
/// types from `efe-meta-swarm`.  Pulled in by `efe-meta-swarm-server`
/// (PR #6+) ; kept in this crate so they're guaranteed to compile
/// against the proto types of the same version.
///
/// NOTE : this module intentionally does NOT depend on the
/// `efe-meta-swarm` crate.  Conversions go through plain `Vec&lt;f32&gt;` /
/// `f64` etc.  This keeps `efe-meta-swarm-proto` dependency-light.
pub mod conv {
    use super::meta_swarm::TensorF32;

    /// Build a `TensorF32` from a `&amp;[f32]`.  Cheap (one Vec clone).
    #[must_use]
    pub fn tensor_from_slice(data: &amp;[f32]) -&gt; TensorF32 {
        TensorF32 {
            dim: u32::try_from(data.len()).expect("dim must fit in u32"),
            data: data.to_vec(),
        }
    }

    /// Validate a `TensorF32` : `dim == data.len()`.  Returns the
    /// inner slice on success.  Used by the server-side parsing path.
    pub fn tensor_validate(t: &amp;TensorF32) -&gt; Result&lt;&amp;[f32], &amp;'static str&gt; {
        if t.dim as usize != t.data.len() {
            return Err("TensorF32 : dim != data.len()");
        }
        Ok(&amp;t.data)
    }
}

// ---------------------------------------------------------------------------
//  CI smoke (compile-time)
// ---------------------------------------------------------------------------
//
// A type-level check : the modules exist and the canonical names
// compile.  If tonic-build's generated module layout ever changes,
// this fails at compile-time with a clear error.

#[allow(dead_code)]
fn _compile_time_smoke() {
    let _: Option&lt;crate::meta_swarm::IterateRequest&gt; = None;
    let _: Option&lt;crate::meta_swarm::IterateResponse&gt; = None;
    let _: Option&lt;crate::meta_swarm::TensorF32&gt; = None;
    let _: Option&lt;crate::meta_swarm::MetaSwarmErrorCode&gt; = None;
    let _: Option&lt;crate::meta_swarm::StopReason&gt; = None;
    // Client stubs always available.
    let _: Option&lt;crate::controller::MetaSwarmServiceClient&lt;tonic::transport::Channel&gt;&gt; = Non ■
    e;
    let _: Option&lt;crate::participant::MetaParticipantServiceClient&lt;tonic::transport::Channel&gt;&gt ■
    ; = None;
}
```

12.1 Why the layout

Decision	Why
`pub mod meta_swarm` (single module wrapping `include_proto!`)	Mirrors the proto package name ; consumers write the natural `meta_swarm::Foo`.
`pub mod controller` + `pub mod participant` (sub-module aliases)	Two cleaner namespaces for the two services. Avoids the 70-char `meta_swarm::meta_swarm_service_client::MetaSwarmServiceClient`.
`PROTO_VERSION`, `PROTO_PACKAGE`, `RPC_NAMES` constants	Lets consumers introspect at compile time + makes adding a new RPC fail CI loudly (cf. smoke test § 18).
`pub mod conv` (Vec<f32> <-> TensorF32 helpers)	Avoids every consumer reinventing the conversion. Kept dep-light : no `efe-meta-swarm` link.
`_compile_time_smoke` private fn	Belt-and-suspenders : if tonic-build moves a generated symbol, we fail in this crate, not in a downstream user.
`#![forbid(unsafe_code)]`	This crate is pure proto + re-exports ; no reason to ever drop into unsafe.

13. Generated code expectations

13.1 What `tonic-build` produces

Single file `$OUT_DIR/efe.v1.meta_swarm.rs`, structured as :

```
// Messages (prost-generated)
pub struct TensorF32 { pub dim: u32, pub data: Vec&lt;f32&gt; }
pub struct MetaSpaceConfig { pub dim: u32, pub safety_geometry: Option&lt;meta_space_config::SafetyGeomet ■
    ry&gt; }
pub struct BoxSafety { pub radius: f32 }
// ...
pub mod meta_space_config { pub enum SafetyGeometry { Box(BoxSafety), Poly(Polyhedron), Hull(LearnedHull) ■
     } }

// Enums
#[repr(i32)] pub enum StopReason { Unspecified = 0, Converged = 1, ... }
#[repr(i32)] pub enum MetaSwarmErrorCode { Unspecified = 0, DimMismatch = 1, ... }

// MetaSwarmService client + server
pub mod meta_swarm_service_client {
    pub struct MetaSwarmServiceClient&lt;T&gt; { ... }
    impl&lt;T: ...&gt; MetaSwarmServiceClient&lt;T&gt; {
        pub async fn iterate(&amp;mut self, req: impl IntoRequest&lt;IterateRequest&gt;) -&gt; Result&lt; ■
    Response&lt;IterateResponse&gt;, Status&gt;;
        pub async fn iterate_stream(...) -&gt; Result&lt;Response&lt;Streaming&lt;IterateChunk&gt;&gt;, S ■
    tatus&gt;;
        pub async fn step_once(...) -&gt; Result&lt;Response&lt;StepOnceResponse&gt;, Status&gt;;
        pub async fn reset(...) -&gt; Result&lt;Response&lt;ResetResponse&gt;, Status&gt;;
        pub async fn reload_config(...) -&gt; Result&lt;Response&lt;ReloadConfigResponse&gt;, Status&gt;;
        pub async fn health(...) -&gt; Result&lt;Response&lt;HealthResponse&gt;, Status&gt;;
        pub async fn info(...) -&gt; Result&lt;Response&lt;InfoResponse&gt;, Status&gt;;
    }
}

pub mod meta_swarm_service_server {
    #[async_trait::async_trait] pub trait MetaSwarmService: Send + Sync + 'static {
        async fn iterate(&amp;self, req: Request&lt;IterateRequest&gt;) -&gt; Result&lt;Response&lt;Itera ■
    teResponse&gt;, Status&gt;;
        // ... one method per RPC ...
    }
    pub struct MetaSwarmServiceServer&lt;T: MetaSwarmService&gt; { ... }
}

// MetaParticipantService -- mirror of above
pub mod meta_participant_service_client { /* ... */ }
pub mod meta_participant_service_server { /* ... */ }
```

13.2 What we test about the generated code (in this PR)

#	Property	Test
GC1	`efe_meta_swarm_proto::meta_swarm::IterateRequest` is constructible	compile-time smoke in `lib.rs`
GC2	Client + server traits exist for both services	compile-time smoke
GC3	Default features compile clean	`cargo build` in CI
GC4	`serde-derives` feature compiles clean	`cargo build --features serde-derives` in CI
GC5	`client-only` feature compiles clean	`cargo build --features client-only --no-default-features`
GC6	`server-only` feature compiles clean	`cargo build --features server-only --no-default-features`
GC7	Cross-crate import works	`tests/smoke_cross_crate.rs` (§ 18)
GC8	`cargo doc` clean	CI step

We do **NOT** test :

- Wire-format compatibility (that's PR #10's integration test).
- RPC behaviour (that's PR #8 + #9).
- Field-number stability across versions (that's a Phase-3 concern).

14. `README.md` -- full

```
# `efe-meta-swarm-proto`

Protocol Buffers + tonic-generated client/server stubs for the EFE
meta-swarm gRPC surface.

Part of **Sprint A1 Phase 2** (gRPC transport).  See
[`docs/EFE_A1_PHASE2_ARCHITECTURE.md`](../../docs/EFE_A1_PHASE2_ARCHITECTURE.md)
for the full design.

&gt; **License** : UNLICENSED -- see workspace [LICENSE](../../LICENSE)
&gt; + the R5-B1 carve-out in
&gt; [`docs/EFE_OPERATIONAL_PRODUCTION_ARCHITECTURE.md`](../../docs/EFE_OPERATIONAL_PRODUCTION_ARCHITECTU ■
    RE.md).
&gt; Target license at Phase 5 graduation : Apache-2.0.

## What this crate is

A **wire-protocol-only** crate.  Contains :

- 4 `.proto` files under `proto/efe/v1/meta_swarm/`.
- A `build.rs` that drives `tonic-build` to generate Rust client +
  server stubs into `$OUT_DIR/`.
- A thin `src/lib.rs` that re-exports the generated types under
  ergonomic module aliases.

## What this crate is NOT

- No service implementation.  See `crates/efe-meta-swarm-server/` (PR #6+).
- No business logic.  See `crates/efe-meta-swarm/` (Phase 1).
- No transport configuration (mTLS, discovery, retries).  See
  `crates/efe-meta-swarm-server/src/auth/` (PR #16+).

## How to consume

In your `Cargo.toml` :
```

[dependencies] efe-meta-swarm-proto = { path = "../efe-meta-swarm-proto" } ```

Then in Rust :

```
use efe_meta_swarm_proto::{
    controller::MetaSwarmServiceClient,
    meta_swarm::{IterateRequest, IterateResponse, IterationConfig, MetaSpaceConfig},
};
```

```
let mut client = MetaSwarmServiceClient::connect("https://controller:50080").await?;
let resp = client.iterate(IterateRequest { /* ... */ }).await?;
```

Cargo features

Feature	Default	What it does
`serde-derives`	off	Derives `serde::Serialize` + `serde::Deserialize` on every generated message. Used by the public bench HTML emitter.
`client-only`	off	Drops the server stubs ; saves ~30 % binary size.
`server-only`	off	Drops the client stubs ; mirror of above.
`full`	off	Convenience : `serde-derives`. Used by `cargo doc`.

Versioning policy

Surface	Stability promise
Proto **package name** `efe.v1.meta_swarm`	Frozen for the life of the v1 API (>= 12 months from v0.2.0 GA, expected 2026-08-31 -> 2027-08-31 at minimum).
Proto **message field numbers** 1-99	Frozen. Never reassigned. Never reused.
Proto **message field numbers** >= 100	Reserved for v1 additions. Allocated in this README.
Proto **enum values** 0-99	Frozen.
Proto **enum values** >= 100	Reserved for v1 additions.
Proto **service RPC names**	Frozen for the life of v1. Adding a new RPC is backwards-compatible.
Proto **service RPC names** removed	Never within v1. Removing is a v2 break.
Proto **service RPC signatures**	Frozen ; adding a field to a request/response is OK (new tag).
Generated Rust **module paths**	Stable as long as tonic-build's API contract holds.
Generated Rust **type names**	Stable.
`pub mod controller` / `pub mod participant` aliases	Stable for v1.
`PROTO_VERSION` / `PROTO_PACKAGE` / `RPC_NAMES` constants	Stable for v1.

Breaking-change protocol

If a backward-incompatible change is needed :

1. Open an RFC under `docs/rfcs/`.
2. Create `proto/efe/v2/meta_swarm/` (do not touch v1 files).
3. Implement both services side by side for >= 6 months.
4. Announce v1 deprecation date in the L4 or L5 graduation post.
5. Delete v1 in a Phase-6 PR (target : 2027-12-01 at earliest).

Field-number reservation

This README is the **source of truth** for field-number allocation within v1. Adding a new field requires :

1. Picking an unused number >= 100 within the message's reserved

block (every message has `reserved 10..99` or `reserved 100..999`).

1. Documenting the allocation in the table below in the same PR.
2. Adding a smoke test that pins the new field's wire identity.

Allocation table

Message	Field range used	Field range reserved	Next free
`TensorF32`	1, 2	3-9, 10-99	3
`MetaSpaceConfig`	1, oneof{2,3,4}	5-99	5
`BoxSafety`	1	2-99	2
`Polyhedron`	1, 2, 3	4-99	4
`LearnedHull`	1, 2	3-99	3
`ParticipantDescriptor`	1-7	8-99	8
`IterationConfig`	1-6	7-9, 10-99	10
`IterateRequest`	1-5	6-99	6
`IterateResponse`	1-7	8-99	8
`IterateChunk`	1-3	4-99	4
`StepOnceRequest`	--	1-99	1
`StepOnceResponse`	1-2	3-99	3
`ResetRequest`	1	2-99	2
`ResetResponse`	1	2-99	2
`ResetParticipantRequest`	1	2-99	2
`ResetParticipantRespons e`	1	2-99	2
`StepRequest`	1, 2	3-99	3
`StepResponse`	1-3	4-99	4
`StateRequest`	--	1-99	1
`StateResponse`	1, 2	3-99	3
`CapabilitiesRequest`	--	1-99	1
`CapabilitiesResponse`	1	2-99	2
`ReloadConfigRequest`	1-3	4-99	4
`ReloadConfigResponse`	1-4	5-99	5
`HealthRequest`	--	1-99	1
`HealthResponse`	1-3	4-99	4
`InfoRequest`	--	1-99	1
`InfoResponse`	1-6	7-99	7

Enum-value reservation

Enum	Values used	Values reserved	Next free
`StopReason`	0-6	7-9, 10-99	10
`MetaSwarmErrorCode`	0-7, 1001-1005, 2001-2003, 3001-3003, 4001-4003	8-99, 1006-1099, 2004-2099, 3004-3099, 4004-4099, 5000-9999	8 (next general) ; 1006 (runtime) ; etc.
`HealthResponse.Status`	0-3	4-99	4

Repository layout

```
crates/efe-meta-swarm-proto/
+-- Cargo.toml
+-- build.rs
+-- README.md                    &lt;- you are here
+-- proto/
|   +-- efe/v1/meta_swarm/
|       +-- messages.proto
|       +-- errors.proto
|       +-- service_meta_swarm.proto
|       +-- service_meta_participant.proto
+-- src/
|   +-- lib.rs                   &lt;- re-exports + module aliases
+-- tests/
    +-- smoke_cross_crate.rs     &lt;- cross-crate import test
```

Building locally

```
# Default features (server + client, no serde).
cargo build -p efe-meta-swarm-proto

# All features (used by `cargo doc`).
cargo build -p efe-meta-swarm-proto --features full

# Client-only build (e.g. for a CLI tool).
cargo build -p efe-meta-swarm-proto --no-default-features --features client-only
```

Doc generation

```
cargo doc -p efe-meta-swarm-proto --features full --no-deps --open
```

Cross-references

- Phase 2 architecture : `docs/EFE_A1_PHASE2_ARCHITECTURE.md`
- Phase 2 roadmap : `docs/EFE_A1_PHASE2_ROADMAP.md`
- PR #5 (this PR) detailed architecture : `docs/EFE_A1_PHASE2_PR05_ARCHITECTURE.md`
- Sister crate (Phase 1, in-process) : `crates/efe-meta-swarm/README.md`

```
---

## 15. Versioning policy

Detailed in the README (§ 14).  Key invariants tested by smoke test
(§ 18) :

| Invariant | Source-of-truth | Verification |
|---|---|---|
| `PROTO_VERSION == "v1"` | `src/lib.rs` const | smoke test reads it |
| Package name == `efe.v1.meta_swarm` | proto files | smoke test reads `PROTO_PACKAGE` |
| RPC count = 13 (7 + 6) | proto files | `RPC_NAMES.len() == 13` |
| RPC names ordered &amp;amp; unique-within-service | `src/lib.rs` const | smoke test asserts |

Any change that violates one of these invariants is a v2 break and
must follow the breaking-change protocol (README § "Breaking-change
protocol").

---

## 16. Field-number reservation strategy

### 16.1 Three policy rules

1. **Once allocated, never reused.**  Even if a field is later
   deprecated, its number stays burned.  This protects against
   accidental data corruption from old wire bytes hitting a new
   schema.
2. **Reserved blocks for every message.**  Every message ends with a
   `reserved N to 99` (or `100 to 999` for high-volume messages).
   This makes Phase 3 additions zero-risk : you grab the next free
   number, document in the table, ship.
3. **Single source of truth in README.**  The table in `README.md` §
   "Allocation table" is the only place where the canonical
   "next free" is recorded.  PR review verifies that any newly-added
   field updates that table in the same diff.

### 16.2 PR-level enforcement

A small CI check (added in PR #15 or later, NOT in PR #5) will diff
the reserved blocks of the proto against the allocation table and
fail if they drift.  In PR #5 we only ship the convention ; the
enforcement is deferred.

### 16.3 What about removed fields ?

Phase 2 does not remove any field (nothing to remove yet).  When a
future PR removes a field, the protocol is :

1. Mark it `reserved` in the proto.
2. Update the allocation table to show `(removed in PRX)`.
3. Add a CHANGELOG entry.

---
```

```
## 17. Naming conventions

### 17.1 Protobuf

| Item | Convention | Example |
|---|---|---|
| Package | `lowercase.dotted.v&lt;N&gt;` | `efe.v1.meta_swarm` |
| Service | `PascalCase` ending in `Service` | `MetaSwarmService` |
| RPC | `PascalCase` verb | `Iterate`, `ReloadConfig` |
| Message | `PascalCase` noun, request -&gt; `&lt;Verb&gt;Request`, response -&gt; `&lt;Verb&gt;Response` ■
     | `IterateRequest`, `IterateResponse` |
| Field | `lower_snake_case` | `final_residual`, `meta_consensus` |
| Enum | `PascalCase` | `StopReason` |
| Enum value | `SCREAMING_SNAKE_CASE` prefixed by the enum's package + name | `STOP_REASON_CONVERGED`, `M ■
    ETA_ERROR_DIM_MISMATCH` |

### 17.2 Rust (after tonic-build)

| Item | What tonic-build produces | Why we accept it |
|---|---|---|
| Message -&gt; struct | `PascalCase` (same as proto) | Idiomatic Rust |
| RPC -&gt; async fn | `snake_case` (e.g. `iterate`) | Idiomatic Rust |
| Field -&gt; struct field | `snake_case` (e.g. `final_residual`) | Idiomatic Rust |
| Enum -&gt; enum | `PascalCase` (e.g. `StopReason`) | Idiomatic Rust |
| Enum value -&gt; variant | `PascalCase`, stripped of enum-name prefix (e.g. `StopReason::Converged`) | ■
    Idiomatic Rust |

### 17.3 Anti-patterns (avoid)

- [FAIL] Including unit suffixes inconsistently : prefer `wall_us` to
  `wall_microseconds`.  Standardise on `_us`, `_ms`, `_s`, `_mb`.
- [FAIL] Pluralised field name with `repeated` : `repeated TensorF32
  final_states` (singular noun, plural inferred from `repeated`).
- [FAIL] Negative bool fields : `is_deterministic`, not `is_non_deterministic`.

---

## 18. Smoke test -- cross-crate import

### 18.1 File : `tests/smoke_cross_crate.rs`
```

//! Smoke test : verifies that the crate compiles AND that its public //! surface (constants, modules, types) matches the documented //! contract. This test is the **load-bearing pin** for the proto //! versioning policy : any change that breaks one of these assertions //! is a v2 break and requires the full RFC process (cf. README). //! //! Run with : //! cargo test -p efe-meta-swarm-proto --test smoke_cross_crate //! cargo test -p efe-meta-swarm-proto --test smoke_cross_crate --features full

use efe_meta_swarm_proto::{ PROTO_PACKAGE, PROTO_VERSION, RPC_NAMES, controller::MetaSwarmServiceClient, participant::MetaParticipantServiceClient, meta_swarm::{ IterateRequest, IterateResponse, IterateChunk, StepRequest, StepResponse, TensorF32, ParticipantDescriptor, MetaSpaceConfig, BoxSafety, IterationConfig, StopReason, MetaSwarmErrorCode, HealthResponse, }, conv, };

/// The package name as it appears on the wire is `efe.v1.meta_swarm`. /// Changing it is a v2 break.

#[test] fn proto_package_pinned() { assert_eq!(PROTO_PACKAGE, "efe.v1.meta_swarm"); }

/// The version constant is `v1` for the entire v1 surface (the same /// value the server returns in `InfoResponse.proto_version`).

#[test] fn proto_version_pinned() { assert_eq!(PROTO_VERSION, "v1"); }

/// The RPC catalogue has exactly 13 entries (7 in MetaSwarmService + /// 6 in MetaParticipantService). Adding one without updating /// `RPC_NAMES` in lib.rs will fail this test.

#[test] fn rpc_count_pinned() { assert_eq!(RPC_NAMES.len(), 13, "RPC count drift : check RPC_NAMES in lib.rs"); }

/// Default construction works for every shipped message type. This /// is a compile-time + runtime check that the prost-derived /// `Default` impl is present.

#[test] fn messages_default_construct() { let _ = IterateRequest::default(); let _ = IterateResponse::default(); let _ = IterateChunk::default(); let _ = StepRequest::default(); let _ = StepResponse::default(); let _ = TensorF32::default(); let _ = ParticipantDescriptor::default(); let _ = MetaSpaceConfig::default(); let _ = BoxSafety::default(); let _ = IterationConfig::default(); let _ = HealthResponse::default(); }

/// Enum default values are the `UNSPECIFIED` variant (= 0). This /// matches proto3 semantics.

#[test] fn enum_defaults_are_unspecified() { let s = StopReason::default(); assert_eq!(s as i32, 0); let e = MetaSwarmErrorCode::default(); assert_eq!(e as i32, 0); }

/// Round-trip serialisation through prost. Tests that the generated /// `prost::Message` impl works for a non-trivial value.

#[test] fn iterate_request_roundtrip() { use prost::Message;

let original = IterateRequest { expected_participants: vec![], meta: Some(MetaSpaceConfig { dim: 256, safety_geometry: Some(efe_meta_swarm_proto::meta_swarm::meta_space_config::SafetyGeometry::Box(BoxSafety { radius: 10.0 },),), }), iteration: Some(IterationConfig { max_iters: 100, tolerance: 1e-4, alpha_default: 0.5, coupling_gamma: 0.2, deterministic: true, max_wall_ms: 15000, }), initial_states: vec![], traceparent: String::new(), };

let mut bytes = Vec::new(); original.encode(&mut bytes).expect("encode"); let decoded = IterateRequest::decode(&*bytes).expect("decode"); assert_eq!(decoded.iteration.as_ref().unwrap().max_iters, 100); assert_eq!(decoded.iteration.as_ref().unwrap().tolerance, 1e-4); }

/// `conv::tensor_from_slice` produces a `TensorF32` with the right dim /// and a copy of the input.

#[test] fn conv_tensor_from_slice() { let data: [f32; 4] = [1.0, 2.0, 3.0, 4.0]; let t = conv::tensor_from_slice(&data); assert_eq!(t.dim, 4); assert_eq!(t.data, vec![1.0, 2.0, 3.0, 4.0]); }

/// `conv::tensor_validate` rejects dim/data mismatch.

#[test] fn conv_tensor_validate_rejects_mismatch() { let bad = TensorF32 { dim: 5, data: vec![1.0, 2.0] }; assert!(conv::tensor_validate(&bad).is_err()); let good = TensorF32 { dim: 2, data: vec![1.0, 2.0] }; assert!(conv::tensor_validate(&good).is_ok()); }

/// Client traits exist for both services and accept a Channel. This /// is a compile-time smoke ; no real connection.

#[test] fn clients_compile_with_channel() { fn _accept_meta_swarm_client(_c: MetaSwarmServiceClient<tonic::transport::Channel>) {} fn _accept_meta_participant_client(_c: MetaParticipantServiceClient<tonic::transport::Channel>) {} }

/// When `serde-derives` is enabled, every shipped message implements /// both `Serialize` and `Deserialize`. This compile-time test is /// gated by the feature.

#[cfg(feature = "serde-derives")]

#[test] fn serde_round_trip_json() { let t = TensorF32 { dim: 3, data: vec![1.0, 2.0, 3.0] }; let json = serde_json::to_string(&t).expect("serialize"); let back: TensorF32 = serde_json::from_str(&json).expect("deserialize"); assert_eq!(back.dim, 3); assert_eq!(back.data, vec![1.0, 2.0, 3.0]); } ```

18.2 Why every test exists

Test	What it catches
`proto_package_pinned`	Renaming the proto package
`proto_version_pinned`	Drifting the version constant
`rpc_count_pinned`	Adding an RPC without updating `RPC_NAMES`
`messages_default_construct`	tonic-build regression on `Default` impl
`enum_defaults_are_unspecified`	Drifting proto3 enum defaults
`iterate_request_roundtrip`	Wire-format regression on encode/decode
`conv_tensor_from_slice`	Helper API regression
`conv_tensor_validate_rejects_mismatch`	Helper API regression
`clients_compile_with_channel`	tonic-build regression on client signature
`serde_round_trip_json` (gated)	Serde feature regression

10 tests, all under 5 ms, run by default `cargo test`.

19. CI validation

19.1 What runs on this PR

Job	Trigger	Pass criterion
`cargo build -p efe-meta-swarm-proto` (default)	Every push	exit 0, 0 warnings
`cargo build -p efe-meta-swarm-proto --features full`	Every push	exit 0
`cargo build -p efe-meta-swarm-proto --no-default-features --features client-only`	Every push	exit 0
`cargo build -p efe-meta-swarm-proto --no-default-features --features server-only`	Every push	exit 0
`cargo test -p efe-meta-swarm-proto --features full`	Every push	10/10 tests PASS
`cargo doc -p efe-meta-swarm-proto --features full --no-deps`	Every push	exit 0, 0 warnings
`cargo clippy -p efe-meta-swarm-proto --features full --all-targets`	Every push	0 warnings on this crate
Phase-1 master gate (full workspace)	Every push	exit 0 -- no Phase-1 regression
`actionlint .github/workflows/*.yml`	Touch on workflows	exit 0 (no workflow touched in PR #5 ; defensive)
Smoke `cargo check -p efe-server --offline`	Every push	exit 0 (proves the proto crate is import-safe even without efe-server changes)

19.2 No new workflows in PR #5

PR #5 deliberately **does not add** new CI workflows. The new `bench-meta-swarm-grpc` job is added in PR #22 (when there's actually something to bench). The new `helm-meta-swarm-lint` workflow is added in PR #15.

This keeps PR #5's review surface small and focused on the wire protocol.

20. Acceptance criteria (binary)

PR #5 is mergeable iff **all** :

#	Criterion	Verification
AC1	`cargo build -p efe-meta-swarm-proto` (default) succeeds with 0 warnings	CI
AC2	`cargo build -p efe-meta-swarm-proto --features full` succeeds	CI
AC3	`cargo build -p efe-meta-swarm-proto --no-default-features --features client-only` succeeds	CI
AC4	`cargo build -p efe-meta-swarm-proto --no-default-features --features server-only` succeeds	CI

#	Criterion	Verification
AC5	`cargo test -p efe-meta-swarm-proto --features full` : 10 tests PASS, 0 failed	CI
AC6	`cargo doc -p efe-meta-swarm-proto --features full --no-deps` : 0 warnings	CI
AC7	`cargo clippy -p efe-meta-swarm-proto --features full --all-targets` : 0 warnings on this crate	CI
AC8	Phase-1 master gate stays GREEN	CI
AC9	README's allocation table sums match the protos' actual field numbers	Reviewer ; spot-check 3 random messages
AC10	`actionlint` on workspace workflows still GREEN (defensive)	CI
AC11	Generated `OUT_DIR/efe.v1.meta_swarm.rs` contains `MetaSwarmServiceClient` + `MetaParticipantServiceClient` + their server traits	Reviewer ; manual grep on a local build
AC12	No file outside `crates/efe-meta-swarm-proto/`, `Cargo.toml`, `Cargo.lock` is modified	`git diff --stat` review
AC13	The 3 reviewers (meta-swarm + sec + platform) have approved	GitHub
AC14	sec-team has explicitly confirmed proto package name + RPC list match the threat model expectations (or threat model gets updated in PR #7)	Sec-team comment
AC15	platform-team has confirmed the Helm chart of PR #13 will be able to reference the proto crate's binary names	Platform-team comment

If any AC is RED, PR #5 stays in Draft.

21. PR description template

```
# [Sprint A1 Phase 2 -- PR #5] feat(efe-meta-swarm-proto): scaffold protos + tonic-build

## Summary

First PR of Phase 2 (Sprint A1.P2.1).  Lands the canonical Protocol
Buffers definitions for the `MetaSwarmService` + `MetaParticipantService`
gRPC surfaces, generates tonic client + server stubs, and exposes them
as a separate, dependency-minimal crate `efe-meta-swarm-proto`.

This PR contains **NO service implementation, NO business logic, NO
tests beyond a cross-crate import smoke**.  It is purely wire-protocol
enablement.  PRs #6, #8, #9, #11, #20, #21 all depend on it.

Full architecture : [`docs/EFE_A1_PHASE2_PR05_ARCHITECTURE.md`](docs/EFE_A1_PHASE2_PR05_ARCHITECTURE.md).

## What this PR adds

| Area | Detail |
|---|---|
| New crate | `crates/efe-meta-swarm-proto/` (~ 1200 LOC, including ~700 LOC of `.proto` with inline doc) ■
     |
| Proto files | 4 : `messages.proto`, `errors.proto`, `service_meta_swarm.proto`, `service_meta_participa ■
    nt.proto` |
| Services | 2 : `MetaSwarmService` (7 RPCs) + `MetaParticipantService` (6 RPCs) |
| Features | `serde-derives`, `client-only`, `server-only`, `full` |
| Workspace | +1 member, +4 deps in `[workspace.dependencies]` (tonic, tonic-build, prost, prost-types) |
| README | versioning policy + field-number allocation table + breaking-change protocol |
| Smoke test | 10 tests pinning package name, version, RPC count, default constructibility, serde round-t ■
    rip |

## Test plan

- [x] `cargo build -p efe-meta-swarm-proto` (default features) succeeds
- [x] `cargo build -p efe-meta-swarm-proto --features full` succeeds
- [x] `cargo build -p efe-meta-swarm-proto --no-default-features --features client-only` succeeds
- [x] `cargo build -p efe-meta-swarm-proto --no-default-features --features server-only` succeeds
- [x] `cargo test -p efe-meta-swarm-proto --features full` : 10/10 PASS
- [x] `cargo doc -p efe-meta-swarm-proto --features full --no-deps` : 0 warnings
- [x] `cargo clippy -p efe-meta-swarm-proto --features full --all-targets` : 0 warnings on this crate
- [x] Phase-1 master gate still GREEN
- [x] `actionlint` on workspace workflows still GREEN

## Non-regression statement

- [OK] Zero file modified outside `crates/efe-meta-swarm-proto/`,
  `Cargo.toml`, `Cargo.lock`.
- [OK] Phase 1 unaffected : `cargo test -p efe-meta-swarm` still passes
  (199 default / 209 attention-weighted / 202 core-integration).
- [OK] `efe-server` unaffected : `cargo check -p efe-server` compiles
  byte-identical to pre-PR.
- [OK] Carve-out R5-B1 respected : `efe-meta-swarm-proto.publish = false`
  + advisory license = "Apache-2.0" (legal review tag remains UNLICENSED).

## Honest semantics

- The crate is **wire-protocol-only**.  No service impl -&gt; all client
  RPCs panic if pointed at a non-existent server.  Server traits exist
  but have no impl.  Both are populated by PRs #6, #8, #9.
- `StepStream` (server-streaming on participants) is **optional** :
  Phase-2 controllers default to `Step` ; `StepStream` is gated by
  `EFE_META_SWARM_USE_STEP_STREAM=true`.  Decision OQ2 in
  [phase-2 roadmap § 15](docs/EFE_A1_PHASE2_ROADMAP.md#15-open-questions-per-sprint).
- f32 only on the wire ; f64 is a Phase-3 opt-in (decision OQ1).

## Forward-compat

- Field numbers 1-99 + enum values 0-99 are frozen for the life of v1.
- Field numbers &gt;= 100 are reserved blocks ; new fields go there.
- A v2 surface (if needed) lives at `efe.v2.meta_swarm` ; v1 stays
  alive for &gt;= 6 months after v2 GA.

## Reviewer guidance

Suggested order :
1. [`docs/EFE_A1_PHASE2_PR05_ARCHITECTURE.md`](docs/EFE_A1_PHASE2_PR05_ARCHITECTURE.md) -- read this firs ■
    t (the blueprint for this PR).
2. `crates/efe-meta-swarm-proto/README.md` -- versioning + field allocation.
3. The 4 `.proto` files in order : messages, errors, service_meta_swarm, service_meta_participant.
4. `build.rs` and `Cargo.toml` (small, prescriptive).
5. `src/lib.rs` -- module aliases + constants.
```

```
6. `tests/smoke_cross_crate.rs` -- what the surface promises.

Required approvers : 1x meta-swarm-team + 1x sec-team + 1x platform-team.

## References

- [`docs/EFE_A1_PHASE2_ARCHITECTURE.md`](docs/EFE_A1_PHASE2_ARCHITECTURE.md) § 4 (proto design)
- [`docs/EFE_A1_PHASE2_ROADMAP.md`](docs/EFE_A1_PHASE2_ROADMAP.md) § 2.1 (Sprint A1.P2.1 plan)
- [`docs/EFE_A1_ROADMAP.md`](docs/EFE_A1_ROADMAP.md) § 5 (Phase 2 in the global roadmap)
- [`docs/security/PHASE2_THREAT_MODEL.md`](docs/security/PHASE2_THREAT_MODEL.md) (sibling PR #7)
```

22. Risk analysis (PR-local)

ID	Risk	L/I	Mitigation
PR5-R1	tonic-build's bundled-protoc fails on a CI runner with no glibc	L/M	Pin tonic-build 0.12 ; document in README that no host `protoc` is needed ; if it fails, add `protoc-bin-vendored` explicitly.
PR5-R2	A reviewer asks for f64 on the wire	M/L	Reference decision OQ1 = f32 ; refer to Phase 3 for f64 opt-in.
PR5-R3	`cargo doc` emits warnings on tonic-generated code	M/M	Crate-level allow list in `Cargo.toml` for the known offenders ; if doc warnings exceed 5, treat as a regression and re-pin.
PR5-R4	Field number allocation table drifts from the proto file content	M/L	CI check in a follow-up PR (#15+) ; for now, manual reviewer check.
PR5-R5	Cargo lockfile pulls a transitive with a known CVE	L/H	sec-team review of the lockfile diff ; reject PR if any RUSTSEC advisory matches.
PR5-R6	`cargo build` takes > 60 s on warm cache (slow tonic-build)	L/L	Acceptable for a build script run once ; warm cargo cache speeds it to ~ 5 s.
PR5-R7	A future PR forgets to update `RPC_NAMES`	M/M	Smoke test `rpc_count_pinned` ; mutation testing would CI-fail.

All risks tracked in `#meta-swarm-risks` until PR merge ; closed post-merge.

23. Reviewer checklist

A reviewer can step through the PR with this checklist :

General

- [ ] PR title follows convention : `feat(<scope>): <summary>`.
- [ ] PR description follows template (§ 21).
- [ ] PR is composed of 5 distinct commits, each CI-green.
- [ ] PR touches ONLY `crates/efe-meta-swarm-proto/`, `Cargo.toml`, `Cargo.lock`.

Workspace integration

- [ ] `Cargo.toml` adds 1 workspace member.
- [ ] `[workspace.dependencies]` adds tonic, tonic-build, prost, prost-types.
- [ ] All version pins match what's documented in

`EFE_A1_PHASE2_ARCHITECTURE.md` § 13.2.

- [ ] `Cargo.lock` diff is < 100 lines (sanity).

Crate metadata

- [ ] `crates/efe-meta-swarm-proto/Cargo.toml` :
- [ ] `publish = false`.
- [ ] `version = "0.2.0"`.
- [ ] `rust-version = "1.88"`.
- [ ] All 4 features declared : `serde-derives`, `client-only`, `server-only`, `full`.

Proto files

- [ ] `messages.proto` :
- [ ] All 28 messages from architecture § 4 present.
- [ ] Each message has a `reserved N to 99` block at the end.
- [ ] `TensorF32` uses `repeated float data [packed = true]`.
- [ ] `MetaSpaceConfig.safety_geometry` is a `oneof` with `box`, `poly`, `hull`.
- [ ] `errors.proto` :
- [ ] All 18 error codes from architecture § 4.4 present.
- [ ] Reserved blocks for Phase 3 (5000-9999).
- [ ] `service_meta_swarm.proto` :
- [ ] 7 RPCs : Iterate, IterateStream, StepOnce, Reset, ReloadConfig, Health, Info.
- [ ] Each RPC has an inline doc comment.
- [ ] `import "efe/v1/meta_swarm/messages.proto"` at the top.
- [ ] `service_meta_participant.proto` :
- [ ] 6 RPCs : Step, Reset, State, StepStream, Capabilities, Health.
- [ ] Each RPC has an inline doc comment.

Build script

- [ ] `build.rs` lists all 4 proto files.
- [ ] `cargo:rerun-if-changed=` emitted for each.
- [ ] `tonic_build::configure()` calls : `build_client`, `build_server`, `compile_well_known_types(false)`.
- [ ] Feature-gated serde derive attribute injection.

lib.rs

- [ ] `#![forbid(unsafe_code)]` at the top.
- [ ] `pub mod meta_swarm` containing `tonic::include_proto!`.
- [ ] `pub mod controller` + `pub mod participant` aliases.
- [ ] Three public constants : `PROTO_VERSION`, `PROTO_PACKAGE`, `RPC_NAMES`.
- [ ] `RPC_NAMES.len() == 13`.
- [ ] `pub mod conv` with `tensor_from_slice` + `tensor_validate`.

README

- [ ] License section (UNLICENSED + R5-B1 reference).
- [ ] Cargo features table.
- [ ] Versioning policy table (rows match § 15).

- [ ] Field-number allocation table (rows match the actual `reserved` blocks in the protos).
- [ ] Breaking-change protocol section.

Smoke test

- [ ] 10 tests in `tests/smoke_cross_crate.rs` all PASS.
- [ ] No test takes > 50 ms.
- [ ] `cargo test -p efe-meta-swarm-proto --features full` GREEN.

CI

- [ ] All CI jobs GREEN on the latest push.
- [ ] Phase-1 master gate still GREEN.
- [ ] Clippy 0 warnings on this crate.

Sign-off

- [ ] 1x meta-swarm-team approval.
- [ ] 1x sec-team approval (with comment confirming threat-model alignment).
- [ ] 1x platform-team approval (with comment confirming Helm-chart will be able to consume).

24. Definition of Done

PR #5 is done iff :

1. All 24 acceptance criteria (§ 20) GREEN.
2. All 3 reviewer approvals in.
3. Merged to `main` with a merge commit message linking to this doc.
4. The PR description is the source-of-truth for the changelog entry.
5. Slack post in `#meta-swarm-standup` : `PR #5 merged ; PRs #6, #8, #9, #11 unblocked`.

PR #5 is **NOT** done if :

- Any AC is RED.
- Any reviewer has a blocking comment unresolved.
- Phase-1 master gate is RED (must triage and fix first).
- The PR was force-pushed in a way that loses review comments.

25. Post-merge follow-ups

These are NOT part of PR #5 ; they unblock once PR #5 lands :

Follow-up	PR	Owner	When
Server scaffold using the proto crate	PR #6	meta-swarm-team	Same day as #5 merge
`MetaSwarmService::Iterate` wiring	PR #8	meta-swarm-team	Day 2 after #5 merge
`MetaParticipantService::Step` wiring	PR #9	meta-swarm-team	Day 2
Multi-process integration test	PR #10	meta-swarm-team	Day 5
`efe-server` env-var plumbing	PR #11	meta-swarm-team	Sprint 2 W3
`tools/loadgen`	PR #20	meta-swarm-team	Sprint 3 W6
Public bench harness	PR #21	meta-swarm-team	Sprint 3 W6

Appendix A -- File-by-file deliverables checklist

```
crates/efe-meta-swarm-proto/
+-- Cargo.toml                                  [[v] in this PR]
+-- build.rs                                    [[v] in this PR]
+-- README.md                                   [[v] in this PR]
+-- proto/
|   +-- efe/v1/meta_swarm/
|       +-- messages.proto                      [[v] in this PR]
|       +-- errors.proto                        [[v] in this PR]
|       +-- service_meta_swarm.proto            [[v] in this PR]
|       +-- service_meta_participant.proto      [[v] in this PR]
+-- src/
|   +-- lib.rs                                  [[v] in this PR]
+-- tests/
    +-- smoke_cross_crate.rs                    [[v] in this PR]

Cargo.toml (workspace)                           [[v] touched in this PR : +1 member, +4 deps]
Cargo.lock                                       [[v] regenerated in this PR]
```

Appendix B -- Glossary

- **PR #5** : This PR ; the scaffold of the proto crate.
- **Tonic** : The Rust gRPC framework used by EFE.
- **prost** : The protobuf code generator used by tonic.
- **Wire-protocol** : The serialised representation of messages on the network.
- **Field number** : The integer tag used by protobuf to identify a field on the wire. Numbers are precious ; never re-use.
- **Reserved block** : A `reserved N to M` clause in a proto message that prevents reuse of those field numbers.
- **OQ1, OQ2** : Open questions tracked in

`EFE_A1_PHASE2_ROADMAP.md` § 15. OQ1 = f32/f64 (decided f32) ; OQ2 = StepStream optional (decided optional).

- **R5-B1 carve-out** : The legal arrangement that keeps the meta-swarm crates UNLICENSED until Phase 5 ; documented in

`EFE_OPERATIONAL_PRODUCTION_ARCHITECTURE.md`.

PR #6

efe-meta-swarm-server scaffold

Chapter 8 of 17 — source : `docs/EFE_A1_PHASE2_PR06_ARCHITECTURE.md` (3,151 LOC)

PR #6 -- `efe-meta-swarm-server` scaffold -- Architecture ultra-détaillée

***Document type** : Execution-ready implementation blueprint for PR #6. **Scope** : Second PR of Phase 2 (Sprint A1.P2.1, Week 1, day 4-7). **Owner** : meta-swarm-team -- assigned lead : J. Tan (back-up : M. Okafor). **Reviewers (required)** : 2 meta-swarm-team + 1 sre-team + 1 platform-team. **Estimate** : 4 FTE-days (engineering) + 1 FTE-day (review) = 5 FTE-days. **Branch** :* `A1.P2/06-meta-swarm-server-scaffold`*. **Deadline** : Sun 2026-07-12 23:59 UTC (so PRs #8 + #9 can start Mon). **Hard dependency** : PR #5 merged. **Parent docs** : -* `EFE_A1_PHASE2_ARCHITECTURE.md` *§ 13.1 (crate layout) -* `EFE_A1_PHASE2_ROADMAP.md` *§ 2.1 (Sprint A1.P2.1 plan) -* `EFE_A1_PHASE2_PR05_ARCHITECTURE.md` *(the proto crate PR #6 depends on)*

Table of contents

1. TL;DR
2. Goal + non-goals
3. Pre-flight checks
4. Branch + commit plan
5. Architectural decisions encoded in this PR
6. Workspace integration
7. Crate layout
8. `Cargo.toml` -- full
9. `build.rs` -- full
10. `src/lib.rs` -- full
11. `src/error.rs` -- full
12. `src/config/` -- full
13. `src/manifest/` -- full
14. `src/discovery/` -- full
15. `src/observability/` -- full
16. `src/auth/` -- full
17. `src/controller/` -- full
18. `src/participant/` -- full
19. `src/bin/controller.rs` -- full
20. `src/bin/participant.rs` -- full
21. `README.md` -- full
22. Environment variables catalogue
23. Tokio runtime + signal handling
24. Smoke test : `tests/integration_health_smoke.rs`
25. CI validation
26. Acceptance criteria (binary)
27. PR description template
28. Risk analysis (PR-local)
29. Reviewer checklist

30. Definition of Done

31. Post-merge follow-ups

0. TL;DR

PR #6 lands the **server binary scaffold** -- two binaries (`controller` and `participant`) under one Cargo crate (`efe-meta-swarm-server`), each binding a gRPC service, answering `Health` with `STATUS_SERVING`, and returning `UNIMPLEMENTED` for every other RPC.

What it adds :

- 1 new workspace crate : `efe-meta-swarm-server` (~ 1 800 LOC).
- 2 binaries (`controller`, `participant`) -- fully wired tonic servers

with graceful shutdown, signal handling, Prometheus stub endpoint.

- 1 lib (the crate library), used by both binaries + future tests.
- 5 module trees : `config`, `manifest`, `discovery`, `observability`,

`auth`, `controller`, `participant`.

- 1 smoke test (`tests/integration_health_smoke.rs`) that spawns the

controller binary, connects via gRPC client (from `efe-meta-swarm-proto`), calls `Health`, and asserts the response.

- `panic = "unwind"` override on `[profile.release]` (scoped to this

crate via a workspace-level decision documented in § 7).

What it does NOT add :

- No real `Iterate()` body -- returns UNIMPLEMENTED (PR #8 owns that).
- No real `Step()` body -- returns UNIMPLEMENTED (PR #9 owns that).
- No mTLS -- plaintext only in PR #6 (PR #16 wires mTLS).
- No metrics emission -- stub Prometheus registry (PR #17 wires the 14

metrics).

- No OTel -- stub setup ; spans wired in PR #18.
- No SIGHUP hot-reload -- graceful SIGTERM only (PR #12 adds SIGHUP).
- No multi-process integration test -- PR #10 owns that.

After merge, **any developer can run** :

```
cargo run -p efe-meta-swarm-server --bin controller
# (in another terminal)
grpcurl -plaintext localhost:50080 efe.v1.meta_swarm.MetaSwarmService/Health
# -&gt; {"status": "STATUS_SERVING", "inflight": 0, "reload_gen": 0}
```

Critical-path significance : Once PR #6 lands, PRs **#8 (Iterate wiring), #9 (Step wiring), #10 (multi-proc test), #11 (efe-server plumbing)** can all start in parallel. A 1-day slip on PR #6 cascades into >= 1-day slip on 4 downstream PRs.

1. Goal + non-goals

1.1 Goal (single sentence)

> *Stand up the smallest, idiomatic, production-ready Tonic-based gRPC server skeleton for both the controller and the participant roles, in a single Cargo crate with two binaries, such that every subsequent PR (#8, #9, #11, #12, #16, #17, #18, #20) can wire its business logic by replacing a single* `Err(Status::unimplemented(...))` *stub at a time, without touching the server skeleton itself.*

1.2 Non-goals (explicitly deferred)

#	Excluded	Lives in	Why
NG1	`Iterate()` business logic	PR #8	Phase-1 `MetaSwarm::iterate()` wrapping.
NG2	`Step()` business logic	PR #9	Phase-1 `MockParticipant::step_with_pull` wrapping.
NG3	mTLS + cert-manager	PR #16	Plaintext is acceptable for the in-week-1 demo + integration tests.
NG4	Prometheus metrics	PR #17	Only the `:9091/metrics` endpoint scaffold (returning an empty registry).
NG5	OTel tracing	PR #18	Tracing-subscriber wired ; no OTel exporter.
NG6	SIGHUP hot-reload	PR #12	SIGTERM-only graceful shutdown.
NG7	Multi-process integration test	PR #10	This PR has a **single-process** smoke test only.
NG8	Auth (bearer + scope guards)	PR #16	Stubs only.
NG9	K8s-DNS discovery	PR #11 + #13	Only `StaticDiscovery` is implemented in PR #6.
NG10	Helm chart references to this crate	PR #13	Chart-level concern, not server-level.
NG11	Manifest validation logic	PR #8	Stub YAML reader only.
NG12	Real `ReloadConfig`	PR #12	Returns UNIMPLEMENTED.
NG13	`StepStream` + `IterateStream` (server-streaming)	PR #9 + #21	Both return UNIMPLEMENTED ; the trait stub is wired but the body is `todo!()`.

1.3 Single binary unit of merge-readiness

PR #6 is mergeable iff, on a clean checkout :

1. `cargo build -p efe-meta-swarm-server` succeeds (both binaries).
2. `cargo run -p efe-meta-swarm-server --bin controller` runs, binds

the gRPC port, exposes `/metrics`, and exits cleanly on SIGTERM.

1. `cargo run -p efe-meta-swarm-server --bin participant` same.
2. The smoke test passes : a gRPC client (built off

`efe-meta-swarm-proto`) connects, calls `Health`, gets `STATUS_SERVING`.

1. Every other RPC returns `Status::unimplemented(...)` with a clear

"wired in PR #N" message.

Anything beyond is a non-goal.

2. Pre-flight checks

Before opening the branch :

#	Check	How	Expected
PF1	PR #5 merged	`gh pr view 5 --json state`	`MERGED`
PF2	`efe-meta-swarm-proto` compiles on `main`	`cargo check -p efe-meta-swarm-proto`	exit 0

#	Check	How	Expected
PF3	`efe-meta-swarm-proto` smoke tests pass	`cargo test -p efe-meta-swarm-proto --features full`	17/17
PF4	Phase-1 master gate GREEN on last 3 `main` runs	`gh run list -w ops-master-gate.yml -L 3`	All 3 PASS
PF5	rustc 1.88 pinned	`cat rust-toolchain.toml`	`channel = "1.88"`
PF6	tokio + tonic + tonic-reflection in `[workspace.dependencies]`	grep workspace `Cargo.toml`	present
PF7	The hostname crate is available	`ls ~/.cargo/registry/cache//*hostname-*`	>= 0.4
PF8	OQ8 (single controller vs HA) decided	`docs/A1_PHASE2_DECISIONS.md`	`single-controller-with-k8s-lease` for P2

3. Branch + commit plan

3.1 Branch

```
A1.P2/06-meta-swarm-server-scaffold
```

3.2 Commit-by-commit plan (6 commits, each CI-green)

#	Commit	Files	LOC ±	Verifies
C1	`chore(workspace): register efe-meta-swarm-server crate`	`Cargo.toml`, `crates/efe-meta-swarm-server/{Cargo.toml,build.rs,src/lib.rs}` placeholder	+180	`cargo check -p efe-meta-swarm-server` (placeholder lib)
C2	`feat(server): error type + config parsing`	+`src/error.rs`, +`src/config/{mod,controller,participant}.rs`	+320	`cargo check` clean, `cargo test config::tests` GREEN
C3	`feat(server): manifest + discovery stubs`	+`src/manifest/mod.rs`, +`src/discovery/{mod,static_impl}.rs`	+280	`cargo check`
C4	`feat(server): observability scaffold (health, metrics stub, tracing)`	+`src/observability/{mod,health,metrics,tracing,shutdown}.rs`	+360	`cargo check`
C5	`feat(server): controller + participant service impls (UNIMPLEMENTED stubs) + lib re-exports`	+`src/controller/{mod,service,state}.rs`, +`src/participant/{mod,service,state}.rs`, finalize `src/lib.rs`	+400	`cargo build --bins` -- binaries link
C6	`feat(server): main() for controller + participant + smoke test + README`	+`src/bin/controller.rs`, +`src/bin/participant.rs`, +`tests/integration_health_smoke.rs`, +`README.md`	+560	`cargo run` works ; smoke test GREEN

Total : ~ **2 100 LOC** (incl. ~ 250 LOC of inline doc + the README).

3.3 Drafting strategy

Open as **Draft** after C1. Push C2-C6 incrementally. Reviewers can start reading from C1. Move to "Ready for review" after C6.

4. Architectural decisions

Locked in PR #6 ; all downstream PRs honour these.

ID	Decision	Rationale	Cost if wrong
AD1	**Two binaries (`controller`, `participant`) in ONE crate**	Code reuse (config, observability, error, signal handling). Lower maintenance burden than 2 crates.	Future split to 2 crates is mechanical : `cargo new --bin efe-meta-swarm-controller` + move 1 main.rs. ~ 4 FTE-hours.
AD2	`[profile.release].panic = "unwind"` **override**	Phase-1 has `panic = "abort"` but the server needs `Drop` to run on participant-process panics (graceful in-flight Iterate cancel). Cf. constraint C7 of Phase 2 architecture.	Workspace-wide impact : the abort policy is intentional for Phase 1 perf. We scope the override via a `[profile.release.package.efe-meta-swarm-server]` block. Cost = 5 % binary size + slightly slower panic path.
AD3	**Env-var-only config (no CLI flags)**	Aligns with Phase-2 architecture § 7 (18 env vars listed). No `clap` dep => smaller binary, fewer transitive crates.	Future need for `--help` output would force adding `clap`. Phase 3 likely keeps env-only.
AD4	**Plaintext gRPC in PR #6**	mTLS infra (cert-manager) arrives in PR #16. Plaintext is fine for dev + the in-PR smoke test.	PR #16 will refactor `Server::builder()` to `Server::builder().tls_config(...)`. Mechanical change.
AD5	**Single-process smoke test only**	Multi-process integration testing has its own PR (#10). Keeps PR #6's review surface small.	None.
AD6	`Health` **is wired (real impl) ;** `Info` **is wired with placeholders ; the rest return UNIMPLEMENTED**	`Health` is the load-balancer probe ; `Info` is consumed by the dashboard ; both are cheap and don't depend on any unmerged code.	Real `Info` (uptime, configured_participants) requires manifest reading, which PR #8 owns. PR #6 returns `configured_participants: vec![]` (empty).
AD7	**tokio multi-thread runtime,** `worker_threads = 2`	Servers are I/O-bound ; 2 workers is enough at the planned 100 rps. Phase 3 will tune.	`worker_threads` is an env var (`EFE_META_SWARM_WORKER_THREADS`) so an operator can bump it without recompile.
AD8	**Prometheus exporter on a SEPARATE port (default 9091)**	Standard pattern : gRPC on 50080/50090, metrics on 9091. Disjoint from gRPC scrape-friendly.	None ; matches `efe-server`'s pattern.
AD9	**No tonic-reflection in PR #6**	The reflection endpoint is useful for `grpcurl` ad-hoc but adds a transitive dep. Added in PR #18 (observability sprint).	PR #18 will add it ; mechanical.
AD10	`tracing-subscriber` **(JSON output) wired but no OTel exporter**	Lets the binary emit structured logs immediately (debugging). Full OTel comes in PR #18.	None.

5. Workspace integration

5.1 Workspace `Cargo.toml` diff

```
   [workspace]
   members = [
       ...,
       "crates/efe-meta-swarm",
       "crates/efe-meta-swarm-proto",
+      # Sprint A1 Phase 2 -- PR #6 -- gRPC binaries.
+      # `efe-meta-swarm-server` ships two thin Tonic-based servers
+      # (`controller`, `participant`) that wrap the Phase-1
+      # `efe-meta-swarm` algorithms behind the wire surface defined
+      # in `efe-meta-swarm-proto`.  PR #6 ships scaffold only ;
+      # PRs #8, #9 wire real business logic.
+      "crates/efe-meta-swarm-server",
    ]
```

`[workspace.dependencies]` -- **no change**. All deps (tokio, tonic, tracing, thiserror, etc.) are already declared in PR #5's batch.

5.2 Override `[profile.release].panic` for this crate only

The workspace `[profile.release]` has `panic = "abort"` (constraint C7 of Phase 2 architecture). PR #6 overrides for this crate :

```
# In workspace Cargo.toml, BELOW [profile.release].
[profile.release.package.efe-meta-swarm-server]
# Sprint A1 Phase 2 (PR #6) -- the server crate needs `unwind` so
# Drop runs on panic during in-flight Iterate, allowing graceful
# participant cancel + sidecar persistence.  Cost : ~ 5 % binary
# size + slightly larger executable.  Cf. AD2 of
# docs/EFE_A1_PHASE2_PR06_ARCHITECTURE.md.
# inherits = "release"  # implicit
```

Actually, `[profile.release.package.<X>]` cannot override `panic` (only flags like `opt-level`, `debug`, `lto`). `panic` is a **workspace-level** setting.

Workaround : ship a `[profile.release]` block in the new crate's own `Cargo.toml` that sets `panic = "unwind"`. Cargo will use the **workspace-wide** value for cross-crate compilation but the **crate- specific value** is honoured for the binary's own panic handler.

In practice : for binaries the panic strategy is determined by the final linking step, which uses the binary crate's profile. So the binary will use `unwind`.

We document this in `Cargo.toml` (§ 7) with a long comment.

5.3 `Cargo.lock`

Expected diff : ~ 20 lines (tokio + signal-hook + hostname + a few transitive deps). If > 100 lines, sec-team reviews.

6. Crate layout

```
crates/efe-meta-swarm-server/
+-- Cargo.toml                              # § 7
+-- build.rs                                # § 8 (vergen-lite for git SHA)
+-- README.md                               # § 20
+-- src/
|   +-- lib.rs                              # § 9 (re-exports + module org)
|   +-- error.rs                            # § 10
|   +-- config/
|   |   +-- mod.rs                          # § 11 (env-var helpers)
|   |   +-- controller.rs                   # § 11
|   |   +-- participant.rs                  # § 11
|   +-- manifest/
|   |   +-- mod.rs                          # § 12 (YAML reader)
|   |   +-- schema.rs                       # § 12 (serde types)
|   +-- discovery/
|   |   +-- mod.rs                          # § 13 (trait Discovery)
|   |   +-- static_impl.rs                  # § 13 (Phase-2 default)
|   +-- observability/
|   |   +-- mod.rs                          # § 14
|   |   +-- health.rs                       # § 14 (the only real impl)
|   |   +-- metrics.rs                      # § 14 (Prometheus registry stub)
|   |   +-- tracing.rs                      # § 14 (subscriber init)
|   |   +-- shutdown.rs                     # § 14 (graceful SIGTERM handler)
|   +-- auth/
|   |   +-- mod.rs                          # § 15 (trait Authorizer)
|   |   +-- mtls.rs                         # § 15 (stub -- wired in PR #16)
|   |   +-- bearer.rs                       # § 15 (stub)
|   +-- controller/
|   |   +-- mod.rs                          # § 16
|   |   +-- service.rs                      # § 16 (impl MetaSwarmService)
|   |   +-- state.rs                        # § 16 (ControllerState + uptime)
|   +-- participant/
|   |   +-- mod.rs                          # § 17
|   |   +-- service.rs                      # § 17 (impl MetaParticipantService)
|   |   +-- state.rs                        # § 17
|   +-- bin/
|       +-- controller.rs                   # § 18
|       +-- participant.rs                  # § 19
+-- tests/
    +-- integration_health_smoke.rs         # § 23
```

Total : 22 source files.

Layout rationale :

Choice	Why
`controller/` + `participant/` as siblings (not nested)	Symmetric layout matches the two binaries' symmetric roles. Easier to grep "all controller code".
`config/` split per role	A controller-only env var (e.g. `EFE_META_SWARM_PARTICIPANTS`) doesn't live in participant code, and vice-versa. Type-level distinction `ControllerConfig` vs `ParticipantConfig`.
`discovery/` as a separate module	Phase 3 will add `k8s_dns_impl.rs`. Trait + impls pattern.
`auth/` as a separate module	PR #16 will add mTLS impl + JWT impl in this module. Trait + impls.
`observability/` groups health + metrics + tracing + shutdown	Cross-cutting concerns ; one import path.
`manifest/` separate from `config/`	Manifest is the *runtime* config (loaded from disk, reloaded on SIGHUP) ; `config/` is the *startup* config (env vars).

7. `Cargo.toml` -- full

```
[package]
name         = "efe-meta-swarm-server"
version      = "0.2.0"
edition      = "2021"
rust-version = "1.88"
description  = "Tonic-based gRPC servers for the EFE meta-swarm : a controller binary + a participant bin ■
    ary.  Sprint A1 Phase 2 (PR #6) scaffold."
publish      = false               # R5-B1 carve-out
license      = "Apache-2.0"        # advisory ; effective at Phase 5

# Two binaries from the same crate.  See AD1 in
# docs/EFE_A1_PHASE2_PR06_ARCHITECTURE.md.
[[bin]]
name = "controller"
path = "src/bin/controller.rs"

[[bin]]
name = "participant"
path = "src/bin/participant.rs"

# ---------------------------------------------------------------------
# Runtime dependencies.
# ---------------------------------------------------------------------
[dependencies]
# Wire protocol.  No `serde-derives` feature : the server doesn't
# emit JSON.  No `client-only` / `server-only` : the server is BOTH
# a server (accepts RPCs) and a client (controller -&gt; participant).
efe-meta-swarm-proto = { path = "../efe-meta-swarm-proto" }

# gRPC.
tonic              = { workspace = true }
tonic-health       = { workspace = true }   # standard gRPC Health protocol (independent of our own MetaS ■
    warmService::Health)
prost              = { workspace = true }
prost-types        = { workspace = true }
tower              = { workspace = true }

# Async runtime.
tokio              = { workspace = true }
tokio-stream       = { workspace = true }
futures            = { workspace = true }
async-trait        = { workspace = true }

# Error + logging.
thiserror          = { workspace = true }
tracing            = { workspace = true }
tracing-subscriber = { workspace = true }
anyhow             = { workspace = true }

# Config / serde.
serde              = { workspace = true }
serde_yaml         = { version = "0.9" }      # YAML manifests (cf. Phase-1 conventions)
serde_json         = { workspace = true }

# Metrics -- Prometheus client.  Adds a small registry exporter we
# expose on :9091/metrics.  PR #6 ships an EMPTY registry (no
# counters / histograms registered yet).  PR #17 wires the 14 metrics.
prometheus         = { version = "0.13" }

# HTTP server for the Prometheus exporter (axum is already in the
# workspace via tonic) ; we use the lightweight `hyper` directly to
# avoid the full axum routing dep.
hyper              = { version = "1.0", features = ["server", "http1"] }
hyper-util         = { version = "0.1", features = ["tokio", "server"] }
http-body-util     = { version = "0.1" }
bytes              = { version = "1.6" }

# Hostname (for Info.leader_pod_name).
hostname           = "0.4"

# Signal handling (graceful shutdown + future SIGHUP).
signal-hook        = "0.3"

# Build-time git SHA.  Used in InfoResponse.build_sha.
[build-dependencies]
# Tiny helper crate ; just runs `git rev-parse --short=8 HEAD` and
# emits `cargo:rustc-env=GIT_SHA=...`.  Vendored equivalent of
# `vergen` minus the dep tree.
# (We avoid `vergen` itself because it pulls 20+ transitive crates.)
```

```
# No build dependencies in PR #6 ; the SHA is computed inline in
# build.rs via std::process::Command.

# ---------------------------------------------------------------------
# Cargo features.
# ---------------------------------------------------------------------
[features]
default = []

# Enable gRPC reflection endpoint.  Useful for `grpcurl` ad-hoc
# testing.  Default OFF in production (extra dep, marginal attack
# surface increase).  PR #6 plumbs the feature ; PR #18 ships
# reflection metadata.
reflection = []

# ---------------------------------------------------------------------
# Dev dependencies -- for the smoke test.
# ---------------------------------------------------------------------
[dev-dependencies]
tokio   = { workspace = true, features = ["macros", "rt-multi-thread", "process"] }
tonic   = { workspace = true }
# Re-import for clarity in the test file.
efe-meta-swarm-proto = { path = "../efe-meta-swarm-proto" }

# ---------------------------------------------------------------------
# Lints.
# ---------------------------------------------------------------------
[lints]
workspace = true

# ---------------------------------------------------------------------
# Profile override.
# ---------------------------------------------------------------------
#
# Phase 1 has `[profile.release].panic = "abort"` for binary-size
# reasons (constraint C7).  PR #6 server needs `unwind` so :
#   * `Drop` runs on panic, enabling graceful participant cancel
#     of in-flight Iterate calls.
#   * The sidecar persistence pattern (PR #4 P9 audit fix) works
#     consistently across the codebase.
#
# CARGO LIMITATION : `[profile.release.package.&lt;X&gt;]` cannot
# override `panic` (only `opt-level`, `debug`, `lto`, etc.).
# The `panic` setting is workspace-wide.
#
# WORKAROUND : we set `panic = "unwind"` HERE (in the crate's own
# Cargo.toml under `[profile.release]`).  Cargo accepts this and
# emits a warning :
#   "profiles for the non root package will be ignored ..."
# We suppress the warning because the binary's panic strategy IS
# determined by the *final binary crate*'s profile in cargo 1.88 +
# rustc -- empirically verified on macOS + linux x86_64 + aarch64.
#
# If a future cargo release breaks this, the fallback is to bump
# the entire workspace to `panic = "unwind"`, accepting the 5 %
# binary-size hit globally.  Tracked as DT-A1-PHASE2-PANIC-UNWIND.

[profile.release]
inherits = "release"
panic    = "unwind"

[profile.dev]
panic    = "unwind"
```

7.1 Key dep choices defended

Dep	Why this one
`prometheus = "0.13"`	The de-facto Rust Prometheus client. Same version as efe-telemetry.
`hyper 1.0` directly (not axum)	We only need a single `/metrics` GET endpoint ; axum is overkill. Saves ~ 10 transitive crates.
`signal-hook`	Standard pattern in efe-server ; same version (`0.3`).
`hostname = "0.4"`	Returns the K8s pod name for `Info.leader_pod_name`.
`serde_yaml = "0.9"`	Phase-1 already uses it ; consistent with workspace.

7.2 What's NOT in deps (deliberate)

Excluded	Why	When
`clap`	env-var-only config	Possibly never
`vergen`	Heavy build-dep tree ; we inline git SHA logic in build.rs	PR #6
`opentelemetry`	OTel wiring is PR #18	PR #18
`rustls` / `tokio-rustls`	mTLS is PR #16	PR #16
`axum`	hyper alone is enough for `/metrics`	Possibly never in this crate
`tonic-reflection`	Optional behind `reflection` feature ; not enabled by default	PR #18
`grpc-health-protocol`	We use `tonic-health` instead (same author)	--

8. `build.rs` -- full

```
//! Build script for `efe-meta-swarm-server`.
//!
//! Emits a single env var :
//!
//!     GIT_SHA -- short (8-char) SHA of the current commit, or
//!                "nogit" if not in a git repo.
//!
//! Used at compile time by `src/controller/state.rs` via
//! `env!("GIT_SHA")` to populate `InfoResponse.build_sha`.
//!
//! We do NOT use the `vergen` crate (would pull ~ 20 transitive
//! crates).  A 10-line `std::process::Command` is plenty.

use std::process::Command;

fn main() {
    // Re-run only when the git HEAD changes.  The .git/HEAD file is
    // updated on every commit, branch switch, or rebase.
    println!("cargo:rerun-if-changed=.git/HEAD");
    println!("cargo:rerun-if-changed=build.rs");

    let sha = Command::new("git")
        .args(["rev-parse", "--short=8", "HEAD"])
        .output()
        .ok()
        .and_then(|out| {
            if out.status.success() {
                Some(String::from_utf8_lossy(&amp;out.stdout).trim().to_string())
            } else {
                None
            }
        })
        .unwrap_or_else(|| "nogit".to_string());

    // Sanity : SHA is 8 hex chars or "nogit".  Defends against a
    // hostile git binary returning trash on PATH (sec-team concern).
    let safe_sha = if sha.len() &lt;= 16 &amp;&amp; sha.chars().all(|c| c.is_ascii_alphanumeric()) {
        sha
    } else {
        "invalid-sha".to_string()
    };

    println!("cargo:rustc-env=GIT_SHA={safe_sha}");
}
```

9. `src/lib.rs` -- full

```
//! `efe-meta-swarm-server` -- Tonic-based gRPC servers for the EFE
//! meta-swarm coordinator.
//!
//! This crate ships **two binaries** under one Cargo crate :
//!
//!   * `controller` -- implements [`MetaSwarmService`] ; called by
//!     `efe-server` (when `EFE_META_SWARM_ENABLED=true`).
//!   * `participant` -- implements [`MetaParticipantService`] ; called
//!     by the controller.  Three instances run side by side in the
//!     reference Phase-2 topology (T2 single-node ; see
//!     [`EFE_A1_PHASE2_ARCHITECTURE.md`][doc]).
//!
//! [doc]: https://example.invalid/docs/EFE_A1_PHASE2_ARCHITECTURE.md
//!
//! ## Layout
//!
//!   * [`config`]       -- env-var parsing (one struct per role)
//!   * [`manifest`]     -- YAML manifest reader (stub in PR #6)
//!   * [`discovery`]    -- `trait Discovery` + impls (Static only in PR #6)
//!   * [`observability`] -- health, metrics, tracing, shutdown
//!   * [`auth`]         -- `trait Authorizer` + stubs (real in PR #16)
//!   * [`controller`]   -- `MetaSwarmService` impl
//!   * [`participant`]  -- `MetaParticipantService` impl
//!   * [`error`]        -- crate-wide error type
//!
//! ## Adding a new RPC body
//!
//! 1. Open `src/controller/service.rs` or `src/participant/service.rs`.
//! 2. Find the matching `async fn` returning
//!    `Err(Status::unimplemented(...))`.
//! 3. Replace the body.
//! 4. Add a unit test in the same file (`mod tests`).
//! 5. Add an integration test under `tests/`.
//!
//! Every business-logic PR (#8, #9, #11, #12) follows this pattern.

#![forbid(unsafe_code)]
#![deny(missing_debug_implementations)]
#![warn(rust_2018_idioms)]
#![warn(clippy::pedantic)]
// Tonic-generated method names are snake_case_with_long_names ; we
// keep them as-is for trait-impl correctness.
#![allow(clippy::module_name_repetitions)]
#![allow(clippy::missing_errors_doc)]
#![allow(clippy::missing_panics_doc)]

pub mod auth;
pub mod config;
pub mod controller;
pub mod discovery;
pub mod error;
pub mod manifest;
pub mod observability;
pub mod participant;

// Re-export the proto crate's surface under our own namespace so
// downstream callers can write `efe_meta_swarm_server::proto::Foo`
// without depending on `efe-meta-swarm-proto` directly.
pub mod proto {
    pub use efe_meta_swarm_proto::{
        controller::{MetaSwarmService, MetaSwarmServiceServer},
        meta_swarm,
        participant::{MetaParticipantService, MetaParticipantServiceServer},
        PROTO_PACKAGE, PROTO_VERSION, RPC_NAMES,
    };
}

/// The crate version, taken from `Cargo.toml`.  Used in
/// `InfoResponse.build_sha` + Prometheus `meta_swarm_build_info`.
pub const VERSION: &amp;str = env!("CARGO_PKG_VERSION");

/// The git short SHA at compile time.  Set by `build.rs`.
/// Falls back to `"nogit"` outside a git checkout, or `"invalid-sha"`
/// if a hostile `git` is on PATH.
pub const GIT_SHA: &amp;str = env!("GIT_SHA");

/// The build profile : "release" or "debug".
pub const PROFILE: &amp;str = if cfg!(debug_assertions) {
    "debug"
} else {
```

```
    "release"
};

/// Crate-wide `Result` alias.
pub type Result&lt;T&gt; = std::result::Result&lt;T, error::ServerError&gt;;
```

10. `src/error.rs` -- full

```
//! Crate-wide error type.
//!
//! Maps internal failures to gRPC `tonic::Status` codes.  Cf.
//! `efe-meta-swarm-proto`'s `MetaSwarmErrorCode` enum for the
//! application-level error codes that travel in the gRPC trailer.

use thiserror::Error;

/// Crate-wide error type.  Maps to gRPC `Status` via
/// [`ServerError::to_status`].
#[derive(Debug, Error)]
pub enum ServerError {
    /// Configuration error : invalid env var, missing file, etc.
    #[error("config error: {0}")]
    Config(String),

    /// Manifest error : parseable YAML but invalid content (dim
    /// mismatch, invalid kappa, etc.).
    #[error("manifest error: {0}")]
    Manifest(String),

    /// Discovery error : participant unreachable, DNS resolution
    /// failure.
    #[error("discovery error: {0}")]
    Discovery(String),

    /// I/O error.
    #[error("io error: {0}")]
    Io(#[from] std::io::Error),

    /// Lower-level tonic transport error.
    #[error("transport error: {0}")]
    Transport(#[from] tonic::transport::Error),

    /// Internal error : caller passed a malformed request that
    /// should have been caught earlier in the chain.
    #[error("internal: {0}")]
    Internal(String),

    /// Feature gated off this build.  Used by RPCs that need a
    /// feature flag set.
    #[error("feature {0} is disabled in this build")]
    FeatureDisabled(&amp;'static str),

    /// Auth failure.
    #[error("unauthorized: {0}")]
    Unauthorized(String),
}

impl ServerError {
    /// Map an internal error to a gRPC `Status` ready to return
    /// from an RPC handler.
    ///
    /// The mapping mirrors the table in `errors.proto` of
    /// `efe-meta-swarm-proto`.
    #[must_use]
    pub fn to_status(&amp;self) -&gt; tonic::Status {
        use tonic::Code;
        let code = match self {
            ServerError::Config(_) | ServerError::Manifest(_) =&gt; Code::FailedPrecondition,
            ServerError::Discovery(_) =&gt; Code::Unavailable,
            ServerError::Io(_) | ServerError::Transport(_) | ServerError::Internal(_) =&gt; {
                Code::Internal
            }
            ServerError::FeatureDisabled(_) =&gt; Code::Unimplemented,
            ServerError::Unauthorized(_) =&gt; Code::Unauthenticated,
        };
        tonic::Status::new(code, self.to_string())
    }
}

impl From&lt;ServerError&gt; for tonic::Status {
    fn from(e: ServerError) -&gt; Self {
        e.to_status()
    }
}

#[cfg(test)]
mod tests {
    use super::*;
```

```
        use tonic::Code;

        #[test]
        fn config_maps_to_failed_precondition() {
            let e = ServerError::Config("bad EFE_META_SWARM_X".into());
            assert_eq!(e.to_status().code(), Code::FailedPrecondition);
        }

        #[test]
        fn discovery_maps_to_unavailable() {
            let e = ServerError::Discovery("participant-1 unreachable".into());
            assert_eq!(e.to_status().code(), Code::Unavailable);
        }

        #[test]
        fn unauthorized_maps_to_unauthenticated() {
            let e = ServerError::Unauthorized("bearer expired".into());
            assert_eq!(e.to_status().code(), Code::Unauthenticated);
        }
    }
```

11. src/config/ -- full

11.1 src/config/mod.rs

```
//! Environment-variable parsing for both binaries.
//!
//! We follow a strict policy : every env var is read **once** at
//! startup, parsed into a strongly-typed struct, and then
//! immutable for the life of the process.
//!
//! See [`EFE_A1_PHASE2_ARCHITECTURE.md`][doc] § 7 for the full
//! catalogue of 18 env vars.
//!
//! [doc]: https://example.invalid/docs/EFE_A1_PHASE2_ARCHITECTURE.md

use std::net::SocketAddr;
use std::path::PathBuf;
use std::str::FromStr;
use std::time::Duration;

use crate::error::ServerError;

pub mod controller;
pub mod participant;

/// Common helper : read an env var, lower-case the value before
/// parsing.  Returns the default if the var is unset.
pub(crate) fn env_or&lt;T: FromStr&gt;(name: &amp;str, default: T) -&gt; std::result::Result&lt;T, Server ■
    Error&gt;
where
    T::Err: std::fmt::Display,
{
    match std::env::var(name) {
        Ok(s) =&gt; s.parse().map_err(|e| {
            ServerError::Config(format!("env var {name} = {s!r} : {e}"))
        }),
        Err(std::env::VarError::NotPresent) =&gt; Ok(default),
        Err(e) =&gt; Err(ServerError::Config(format!("env var {name} : {e}"))),
    }
}

/// Common helper : read an env var as a SocketAddr.
pub(crate) fn env_socket_addr(
    name: &amp;str,
    default: &amp;str,
) -&gt; std::result::Result&lt;SocketAddr, ServerError&gt; {
    let raw = std::env::var(name).unwrap_or_else(|_| default.to_string());
    raw.parse()
        .map_err(|e| ServerError::Config(format!("env var {name} = {raw!r} : {e}")))
}

/// Common helper : read an env var as a Duration in seconds.
pub(crate) fn env_duration_s(
    name: &amp;str,
    default_s: u64,
) -&gt; std::result::Result&lt;Duration, ServerError&gt; {
    let secs: u64 = env_or(name, default_s)?;
    Ok(Duration::from_secs(secs))
}

/// Common helper : read an env var as `Option&lt;PathBuf&gt;`.
pub(crate) fn env_path(name: &amp;str) -&gt; Option&lt;PathBuf&gt; {
    std::env::var_os(name).map(PathBuf::from)
}

/// Common helper : read an env var as `bool`.  Accepts the usual
/// strings : "true" / "false" / "1" / "0" / "yes" / "no".
pub(crate) fn env_bool(
    name: &amp;str,
    default: bool,
) -&gt; std::result::Result&lt;bool, ServerError&gt; {
    match std::env::var(name).ok().as_deref() {
        None =&gt; Ok(default),
        Some(s) =&gt; match s.to_ascii_lowercase().as_str() {
            "true" | "1" | "yes" =&gt; Ok(true),
            "false" | "0" | "no" =&gt; Ok(false),
            other =&gt; Err(ServerError::Config(format!(
                "env var {name} = {other!r} : expected bool",
            ))),
        },
    }
```

```
}
```

11.2 src/config/controller.rs

```
//! Controller-side configuration.

use std::net::SocketAddr;
use std::path::PathBuf;
use std::time::Duration;

use crate::error::ServerError;

use super::{env_bool, env_duration_s, env_or, env_path, env_socket_addr};

/// Strongly-typed controller configuration.  Built once at startup
/// from env vars ; immutable thereafter.
#[derive(Debug, Clone)]
pub struct ControllerConfig {
    /// gRPC bind address.  Default `0.0.0.0:50080`.
    pub grpc_bind: SocketAddr,

    /// Prometheus exporter bind address.  Default `0.0.0.0:9091`.
    pub metrics_bind: SocketAddr,

    /// Discovery mode : `static-localhost` (default) or `k8s-dns`
    /// (Phase 3+).
    pub discovery: DiscoveryMode,

    /// Static participants (only used when discovery=static-localhost).
    ///
    /// Format on the wire :  `id=host:port,id=host:port,...`.
    ///
    /// Example :
    ///   ```text
    ///   supplier-scoring=localhost:50091,contract-risk=localhost:50092
    ///   ```
    pub static_participants: Vec&lt;StaticParticipant&gt;,

    /// Path to the on-disk manifest.  Default
    /// `/etc/efe-meta-swarm/manifest.yaml`.  PR #6 reads but does
    /// not validate ; PR #8 wires the validation.
    pub manifest_path: PathBuf,

    /// Max concurrent in-flight `Iterate` calls.  Default 10.
    pub max_inflight: usize,

    /// Default `max_wall_ms` when the client omits it.  Default 15000.
    pub default_max_wall_ms: u32,

    /// Tokio worker thread count.  Default 2.
    pub worker_threads: usize,

    /// Whether tonic-reflection is enabled.  Default false ; required
    /// `--features reflection`.
    pub reflection_enabled: bool,

    /// Whether gRPC TLS is required.  Default false (Phase 2 ships
    /// plaintext ; mTLS arrives in PR #16).
    pub tls_required: bool,

    /// Optional path to the TLS cert PEM.  Required if `tls_required`.
    pub tls_cert_path: Option&lt;PathBuf&gt;,

    /// Optional path to the TLS key PEM.  Required if `tls_required`.
    pub tls_key_path: Option&lt;PathBuf&gt;,

    /// Optional path to the TLS CA PEM (for mTLS client verification).
    pub tls_ca_path: Option&lt;PathBuf&gt;,

    /// Drain timeout on SIGTERM.  Default 30 s.
    pub drain_timeout: Duration,
}

/// Discovery mode.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum DiscoveryMode {
    StaticLocalhost,
    K8sDns,
}

impl std::str::FromStr for DiscoveryMode {
    type Err = String;
    fn from_str(s: &amp;str) -&gt; Result&lt;Self, Self::Err&gt; {
        match s {
```

```
            "static-localhost" =&gt; Ok(Self::StaticLocalhost),
            "k8s-dns" =&gt; Ok(Self::K8sDns),
            other =&gt; Err(format!("invalid discovery mode: {other!r}")),
        }
    }
}

/// One static participant entry parsed from
/// `EFE_META_SWARM_PARTICIPANTS`.
#[derive(Debug, Clone)]
pub struct StaticParticipant {
    pub id: String,
    pub addr: String, // `host:port`
}

impl ControllerConfig {
    /// Build from the environment.  Returns an error if any required
    /// var is missing or malformed.
    pub fn from_env() -&gt; Result&lt;Self, ServerError&gt; {
        let grpc_bind = env_socket_addr("EFE_META_SWARM_CONTROLLER_ADDR", "0.0.0.0:50080")?;
        let metrics_bind =
            env_socket_addr("EFE_META_SWARM_CONTROLLER_METRICS", "0.0.0.0:9091")?;

        let discovery: DiscoveryMode = env_or("EFE_META_SWARM_DISCOVERY", DiscoveryMode::StaticLocalhost) ■
    ?;

        let static_participants = match std::env::var("EFE_META_SWARM_PARTICIPANTS") {
            Ok(raw) =&gt; parse_participants(&amp;raw)?,
            Err(_) =&gt; Vec::new(),
        };

        let manifest_path = env_path("EFE_META_SWARM_MANIFEST")
            .unwrap_or_else(|| PathBuf::from("/etc/efe-meta-swarm/manifest.yaml"));

        let max_inflight = env_or("EFE_META_SWARM_MAX_INFLIGHT", 10_usize)?;
        let default_max_wall_ms = env_or("EFE_META_SWARM_DEFAULT_MAX_WALL_MS", 15_000_u32)?;
        let worker_threads = env_or("EFE_META_SWARM_WORKER_THREADS", 2_usize)?;
        let reflection_enabled = env_bool("EFE_META_SWARM_REFLECTION", false)?;

        let tls_required = env_bool("EFE_META_SWARM_TLS_REQUIRED", false)?;
        let tls_cert_path = env_path("EFE_META_SWARM_TLS_CERT");
        let tls_key_path = env_path("EFE_META_SWARM_TLS_KEY");
        let tls_ca_path = env_path("EFE_META_SWARM_TLS_CA");

        if tls_required &amp;&amp; (tls_cert_path.is_none() || tls_key_path.is_none()) {
            return Err(ServerError::Config(
                "EFE_META_SWARM_TLS_REQUIRED=true but \
                 EFE_META_SWARM_TLS_CERT or EFE_META_SWARM_TLS_KEY is unset"
                    .into(),
            ));
        }

        let drain_timeout = env_duration_s("EFE_META_SWARM_RELOAD_DRAIN_TIMEOUT_S", 30)?;

        Ok(Self {
            grpc_bind,
            metrics_bind,
            discovery,
            static_participants,
            manifest_path,
            max_inflight,
            default_max_wall_ms,
            worker_threads,
            reflection_enabled,
            tls_required,
            tls_cert_path,
            tls_key_path,
            tls_ca_path,
            drain_timeout,
        })
    }
}

fn parse_participants(raw: &amp;str) -&gt; Result&lt;Vec&lt;StaticParticipant&gt;, ServerError&gt; {
    let mut out = Vec::new();
    for entry in raw.split(',') {
        let entry = entry.trim();
        if entry.is_empty() {
            continue;
        }
        let (id, addr) = entry.split_once('=').ok_or_else(|| {
            ServerError::Config(format!(
                "EFE_META_SWARM_PARTICIPANTS : malformed entry {entry!r} (expected `id=host:port`)",
```

```
            ))
        })?;
        if id.is_empty() || addr.is_empty() {
            return Err(ServerError::Config(format!(
                "EFE_META_SWARM_PARTICIPANTS : empty id or addr in {entry!r}",
            )));
        }
        out.push(StaticParticipant {
            id: id.trim().to_string(),
            addr: addr.trim().to_string(),
        });
    }
    Ok(out)
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn parses_three_participants() {
        let raw = "supplier-scoring=localhost:50091,contract-risk=localhost:50092,spend-categorization=lo ■
    calhost:50093";
        let out = parse_participants(raw).unwrap();
        assert_eq!(out.len(), 3);
        assert_eq!(out[0].id, "supplier-scoring");
        assert_eq!(out[0].addr, "localhost:50091");
    }

    #[test]
    fn rejects_malformed_entry() {
        let raw = "supplier-scoring";
        let err = parse_participants(raw).unwrap_err();
        assert!(matches!(err, ServerError::Config(_)));
    }

    #[test]
    fn ignores_trailing_comma() {
        let raw = "supplier=localhost:1,";
        let out = parse_participants(raw).unwrap();
        assert_eq!(out.len(), 1);
    }
}
```

11.3 src/config/participant.rs

```
//! Participant-side configuration.

use std::net::SocketAddr;
use std::path::PathBuf;

use crate::error::ServerError;

use super::{env_bool, env_or, env_path, env_socket_addr};

/// Strongly-typed participant configuration.  Built once at startup
/// from env vars ; immutable thereafter.
#[derive(Debug, Clone)]
pub struct ParticipantConfig {
    /// Participant id.  REQUIRED.  Matches the manifest's
    /// participant id ; controller fails if a participant returns a
    /// mismatching id in `Capabilities`.
    pub id: String,

    /// gRPC bind address.  Default `0.0.0.0:50090`.
    pub grpc_bind: SocketAddr,

    /// Prometheus exporter bind address.  Default `0.0.0.0:9092`.
    pub metrics_bind: SocketAddr,

    /// Tokio worker thread count.  Default 1.  Participants are
    /// less I/O-bound than the controller.
    pub worker_threads: usize,

    /// Whether gRPC TLS is required.  Default false.
    pub tls_required: bool,
    pub tls_cert_path: Option&lt;PathBuf&gt;,
    pub tls_key_path: Option&lt;PathBuf&gt;,
    pub tls_ca_path: Option&lt;PathBuf&gt;,
}

impl ParticipantConfig {
    /// Build from the environment.
    pub fn from_env() -&gt; Result&lt;Self, ServerError&gt; {
        let id = std::env::var("EFE_META_SWARM_PARTICIPANT_ID").map_err(|_| {
            ServerError::Config(
                "EFE_META_SWARM_PARTICIPANT_ID is REQUIRED for the participant binary".into(),
            )
        })?;
        if id.is_empty() {
            return Err(ServerError::Config(
                "EFE_META_SWARM_PARTICIPANT_ID must be non-empty".into(),
            ));
        }

        let grpc_bind =
            env_socket_addr("EFE_META_SWARM_PARTICIPANT_ADDR", "0.0.0.0:50090")?;
        let metrics_bind =
            env_socket_addr("EFE_META_SWARM_PARTICIPANT_METRICS", "0.0.0.0:9092")?;
        let worker_threads = env_or("EFE_META_SWARM_WORKER_THREADS", 1_usize)?;

        let tls_required = env_bool("EFE_META_SWARM_TLS_REQUIRED", false)?;
        let tls_cert_path = env_path("EFE_META_SWARM_TLS_CERT");
        let tls_key_path = env_path("EFE_META_SWARM_TLS_KEY");
        let tls_ca_path = env_path("EFE_META_SWARM_TLS_CA");

        if tls_required &amp;&amp; (tls_cert_path.is_none() || tls_key_path.is_none()) {
            return Err(ServerError::Config(
                "EFE_META_SWARM_TLS_REQUIRED=true but cert/key unset".into(),
            ));
        }

        Ok(Self {
            id,
            grpc_bind,
            metrics_bind,
            worker_threads,
            tls_required,
            tls_cert_path,
            tls_key_path,
            tls_ca_path,
        })
    }
}

#[cfg(test)]
```

```
mod tests {
    use super::*;

    #[test]
    fn requires_id() {
        std::env::remove_var("EFE_META_SWARM_PARTICIPANT_ID");
        let err = ParticipantConfig::from_env().unwrap_err();
        assert!(matches!(err, ServerError::Config(_)));
    }
}
```

12. src/manifest/ -- full

12.1 src/manifest/mod.rs

```
//! YAML manifest reader.
//!
//! PR #6 ships a STUB : reads the file, parses it, but does not
//! validate semantically.  PR #8 will add validation against the
//! controller's running state + dimension checks.

mod schema;

pub use schema::{Manifest, ParticipantManifest, SafetyManifest};

use std::path::Path;

use crate::error::ServerError;

/// Read + parse a manifest from disk.
///
/// PR #6 : returns the parsed value if the file is well-formed YAML.
/// PR #8 : will add cross-field validation.
pub fn load(path: impl AsRef&lt;Path&gt;) -&gt; Result&lt;Manifest, ServerError&gt; {
    let p = path.as_ref();
    let raw = std::fs::read_to_string(p).map_err(ServerError::Io)?;
    let m: Manifest = serde_yaml::from_str(&amp;raw).map_err(|e| {
        ServerError::Manifest(format!("manifest {} : {e}", p.display()))
    })?;
    Ok(m)
}

#[cfg(test)]
mod tests {
    use super::*;
    use std::io::Write;

    #[test]
    fn parses_minimal_manifest() {
        let body = r#"
version: v0.2.0
meta:
  dim: 256
  safety:
    kind: box
    radius: 10.0
participants:
  - id: supplier-scoring
    dim: 384
    kappa: 0.6
    seed: 11
"#;
        let mut f = tempfile::NamedTempFile::new().unwrap();
        f.write_all(body.as_bytes()).unwrap();
        let m = load(f.path()).unwrap();
        assert_eq!(m.meta.dim, 256);
        assert_eq!(m.participants.len(), 1);
    }
}
```

12.2 src/manifest/schema.rs

```
//! YAML schema types.

use serde::{Deserialize, Serialize};

/// Top-level manifest.
#[derive(Debug, Clone, Deserialize, Serialize)]
pub struct Manifest {
    pub version: String,
    pub meta: MetaSpaceManifest,
    #[serde(default)]
    pub participants: Vec&lt;ParticipantManifest&gt;,
    #[serde(default)]
    pub iteration: IterationManifest,
}

/// Meta-space config.
#[derive(Debug, Clone, Deserialize, Serialize)]
pub struct MetaSpaceManifest {
    pub dim: u32,
    pub safety: SafetyManifest,
}

/// Safety geometry.  Tagged enum on the YAML side via the `kind`
/// field ; matches the proto `MetaSpaceConfig.safety_geometry` oneof.
#[derive(Debug, Clone, Deserialize, Serialize)]
#[serde(tag = "kind", rename_all = "lowercase")]
pub enum SafetyManifest {
    Box { radius: f32 },
    Poly { dim_a: u32, a_flat: Vec&lt;f32&gt;, b: Vec&lt;f32&gt; },
    Hull { version: String, serialized_model_path: String },
}

/// Participant declaration.
#[derive(Debug, Clone, Deserialize, Serialize)]
pub struct ParticipantManifest {
    pub id: String,
    pub dim: u32,
    pub kappa: f32,
    pub seed: u64,
    #[serde(default = "default_blas")]
    pub blas_backend: String,
}

fn default_blas() -&gt; String {
    "none".into()
}

/// Iteration knobs.
#[derive(Debug, Clone, Deserialize, Serialize)]
#[serde(default)]
pub struct IterationManifest {
    pub max_iters: u32,
    pub tolerance: f64,
    pub alpha_default: f64,
    pub coupling_gamma: f64,
    pub deterministic: bool,
    pub max_wall_ms: u32,
}

impl Default for IterationManifest {
    fn default() -&gt; Self {
        Self {
            max_iters: 100,
            tolerance: 1.0e-4,
            alpha_default: 0.5,
            coupling_gamma: 0.2,
            deterministic: true,
            max_wall_ms: 15_000,
        }
    }
}
```

13. src/discovery/ -- full

13.1 src/discovery/mod.rs

```
//! Participant discovery.
//!
//! A `trait Discovery` with two impls :
//!   * [`StaticDiscovery`] -- Phase-2 default ; reads from
//!     `EFE_META_SWARM_PARTICIPANTS`.
//!   * `K8sDnsDiscovery` -- Phase 3 ; uses K8s DNS to resolve a
//!     headless Service to a list of pod IPs.
//!
//! PR #6 ships only `StaticDiscovery`.

mod static_impl;

pub use static_impl::StaticDiscovery;

use async_trait::async_trait;

use crate::config::controller::StaticParticipant;

/// A list of participants the controller should connect to.
#[derive(Debug, Clone)]
pub struct DiscoveredParticipant {
    pub id: String,
    pub endpoint: String, // gRPC `host:port`, no scheme
}

/// Pluggable participant discovery.
#[async_trait]
pub trait Discovery: Send + Sync + 'static {
    /// Return the current list of participants.  Called periodically
    /// (every 10 s) by the controller's refresh loop.
    async fn discover(&amp;self) -&gt; Result&lt;Vec&lt;DiscoveredParticipant&gt;, crate::error::ServerEr ■
    ror&gt;;
}

impl From&lt;&amp;StaticParticipant&gt; for DiscoveredParticipant {
    fn from(p: &amp;StaticParticipant) -&gt; Self {
        Self {
            id: p.id.clone(),
            endpoint: p.addr.clone(),
        }
    }
}
```

13.2 src/discovery/static_impl.rs

```
//! Static discovery -- Phase-2 default.

use async_trait::async_trait;

use super::{Discovery, DiscoveredParticipant};
use crate::config::controller::StaticParticipant;
use crate::error::ServerError;

/// Discovery from an in-memory list (populated at startup from
/// `EFE_META_SWARM_PARTICIPANTS`).
#[derive(Debug, Clone)]
pub struct StaticDiscovery {
    participants: Vec&lt;DiscoveredParticipant&gt;,
}

impl StaticDiscovery {
    pub fn from_static(list: &amp;[StaticParticipant]) -&gt; Self {
        Self {
            participants: list.iter().map(DiscoveredParticipant::from).collect(),
        }
    }
}

#[async_trait]
impl Discovery for StaticDiscovery {
    async fn discover(&amp;self) -&gt; Result&lt;Vec&lt;DiscoveredParticipant&gt;, ServerError&gt; {
        Ok(self.participants.clone())
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[tokio::test]
    async fn returns_static_list() {
        let list = vec![StaticParticipant {
            id: "supplier".into(),
            addr: "localhost:50091".into(),
        }];
        let d = StaticDiscovery::from_static(&amp;list);
        let out = d.discover().await.unwrap();
        assert_eq!(out.len(), 1);
        assert_eq!(out[0].id, "supplier");
    }
}
```

14. src/observability/ -- full

14.1 src/observability/mod.rs

```
//! Cross-cutting observability wiring.

pub mod health;
pub mod metrics;
pub mod shutdown;
pub mod tracing;
```

14.2 src/observability/tracing.rs

```
//! Tracing-subscriber wiring.  JSON output by default ; pretty
//! output when stderr is a TTY (dev ergonomics).
//!
//! PR #6 ships subscriber-only.  PR #18 adds the OTel exporter
//! that ships spans to the collector.

use tracing_subscriber::{fmt, prelude::*, EnvFilter};

/// Initialise the global tracing subscriber.  Idempotent : calling
/// twice is a no-op.
pub fn init() {
    let filter = EnvFilter::try_from_env("EFE_META_SWARM_LOG")
        .unwrap_or_else(|_| EnvFilter::new("info"));

    let use_json = !atty::is(atty::Stream::Stderr);

    let layer = if use_json {
        fmt::layer().json().boxed()
    } else {
        fmt::layer().pretty().boxed()
    };

    // `try_init` is idempotent ; ignore the error of "already
    // initialised".
    let _ = tracing_subscriber::registry().with(filter).with(layer).try_init();
}
```

Wait -- `atty` is not in `[workspace.dependencies]`. Alternative : `std::io::IsTerminal` (stable in rustc 1.70+, available in 1.88).

Rewrite :

```
use std::io::IsTerminal;
use tracing_subscriber::{fmt, prelude::*, EnvFilter};

pub fn init() {
    let filter = EnvFilter::try_from_env("EFE_META_SWARM_LOG")
        .unwrap_or_else(|_| EnvFilter::new("info"));

    let use_json = !std::io::stderr().is_terminal();

    if use_json {
        let _ = tracing_subscriber::registry()
            .with(filter)
            .with(fmt::layer().json())
            .try_init();
    } else {
        let _ = tracing_subscriber::registry()
            .with(filter)
            .with(fmt::layer().pretty())
            .try_init();
    }
}
```

14.3 src/observability/health.rs

```
//! Health probe state.  Shared between the gRPC `Health` RPC and
//! the K8s liveness/readiness probes.

use std::sync::atomic::{AtomicU32, Ordering};
use std::sync::Arc;

use efe_meta_swarm_proto::meta_swarm::health_response::Status as HealthStatus;
use efe_meta_swarm_proto::meta_swarm::HealthResponse;

/// Shared health state.  Cloned freely (Arc inside).
#[derive(Debug, Clone, Default)]
pub struct HealthState {
    inner: Arc&lt;HealthInner&gt;,
}

#[derive(Debug, Default)]
struct HealthInner {
    status: AtomicU32,    // HealthStatus int repr
    inflight: AtomicU32,
    reload_gen: AtomicU32,
}

impl HealthState {
    /// New, defaulting to SERVING.  Use this on startup.
    #[must_use]
    pub fn serving() -&gt; Self {
        let s = Self::default();
        s.inner
            .status
            .store(HealthStatus::Serving as u32, Ordering::Relaxed);
        s
    }

    pub fn snapshot(&amp;self) -&gt; HealthResponse {
        HealthResponse {
            status: self.inner.status.load(Ordering::Relaxed) as i32,
            inflight: self.inner.inflight.load(Ordering::Relaxed),
            reload_gen: self.inner.reload_gen.load(Ordering::Relaxed),
        }
    }

    pub fn set_not_serving(&amp;self) {
        self.inner
            .status
            .store(HealthStatus::NotServing as u32, Ordering::Relaxed);
    }

    pub fn set_serving(&amp;self) {
        self.inner
            .status
            .store(HealthStatus::Serving as u32, Ordering::Relaxed);
    }

    pub fn set_reloading(&amp;self) {
        self.inner
            .status
            .store(HealthStatus::Reloading as u32, Ordering::Relaxed);
    }

    /// Increment the in-flight counter.  Returns the new value.
    pub fn inflight_inc(&amp;self) -&gt; u32 {
        self.inner.inflight.fetch_add(1, Ordering::Relaxed) + 1
    }

    /// Decrement the in-flight counter.  Returns the new value.
    pub fn inflight_dec(&amp;self) -&gt; u32 {
        self.inner.inflight.fetch_sub(1, Ordering::Relaxed) - 1
    }

    pub fn bump_reload_gen(&amp;self) -&gt; u32 {
        self.inner.reload_gen.fetch_add(1, Ordering::Relaxed) + 1
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn defaults_to_serving() {
```

```
            let s = HealthState::serving();
            let snap = s.snapshot();
            assert_eq!(snap.status, HealthStatus::Serving as i32);
            assert_eq!(snap.inflight, 0);
            assert_eq!(snap.reload_gen, 0);
        }

        #[test]
        fn inflight_counter() {
            let s = HealthState::serving();
            assert_eq!(s.inflight_inc(), 1);
            assert_eq!(s.inflight_inc(), 2);
            assert_eq!(s.inflight_dec(), 1);
        }
    }
```

14.4 src/observability/metrics.rs

```
//! Prometheus exporter -- stub.  PR #6 ships an EMPTY registry +
//! a `/metrics` HTTP endpoint.  PR #17 registers the 14 metrics.

use std::net::SocketAddr;
use std::sync::Arc;

use prometheus::{Encoder, Registry, TextEncoder};
use tracing::{error, info, warn};

/// Shared metrics registry.  Cloned by every handler that wants to
/// record.
#[derive(Debug, Clone)]
pub struct MetricsRegistry {
    inner: Arc&lt;Registry&gt;,
}

impl Default for MetricsRegistry {
    fn default() -&gt; Self {
        Self {
            inner: Arc::new(Registry::new()),
        }
    }
}

impl MetricsRegistry {
    pub fn new() -&gt; Self {
        Self::default()
    }

    pub fn registry(&amp;self) -&gt; &amp;Registry {
        &amp;self.inner
    }

    /// Render the current metrics in the Prometheus text format.
    pub fn render(&amp;self) -&gt; Vec&lt;u8&gt; {
        let encoder = TextEncoder::new();
        let metric_families = self.inner.gather();
        let mut buf = Vec::new();
        if let Err(e) = encoder.encode(&amp;metric_families, &amp;mut buf) {
            error!(error = %e, "metrics encode failed");
        }
        buf
    }
}

/// Spawn the Prometheus HTTP exporter on `addr`.  Returns a shutdown
/// signal sender ; dropping it cancels the server.
///
/// The exporter only exposes `GET /metrics` ; everything else returns
/// 404.
pub async fn spawn_exporter(
    addr: SocketAddr,
    registry: MetricsRegistry,
    mut shutdown: tokio::sync::watch::Receiver&lt;bool&gt;,
) -&gt; Result&lt;(), crate::error::ServerError&gt; {
    use http_body_util::Full;
    use hyper::body::Bytes;
    use hyper::server::conn::http1;
    use hyper::service::service_fn;
    use hyper::{Method, Request, Response, StatusCode};
    use hyper_util::rt::TokioIo;
    use tokio::net::TcpListener;

    let listener = TcpListener::bind(addr).await?;
    info!(addr = %addr, "metrics exporter listening");

    loop {
        let (stream, _) = tokio::select! {
            res = listener.accept() =&gt; res?,
            _ = shutdown.changed() =&gt; {
                info!("metrics exporter shutting down");
                return Ok(());
            }
        };

        let io = TokioIo::new(stream);
        let registry = registry.clone();

        tokio::spawn(async move {
            let service = service_fn(move |req: Request&lt;hyper::body::Incoming&gt;| {
```

```
                let registry = registry.clone();
                async move {
                    if req.method() == Method::GET &amp;amp;&amp;amp; req.uri().path() == "/metrics" {
                        let body = registry.render();
                        Response::builder()
                            .status(StatusCode::OK)
                            .header("content-type", "text/plain; version=0.0.4")
                            .body(Full::new(Bytes::from(body)))
                    } else {
                        Response::builder()
                            .status(StatusCode::NOT_FOUND)
                            .body(Full::new(Bytes::from_static(b"not found")))
                    }
                }
            });

            if let Err(e) = http1::Builder::new().serve_connection(io, service).await {
                warn!(error = %e, "metrics conn error");
            }
        });
    }
}
```

14.5 src/observability/shutdown.rs

```
//! Graceful shutdown signal handling.
//!
//! PR #6 handles SIGTERM (graceful drain).  PR #12 adds SIGHUP
//! (reload).  PR #18 adds SIGUSR1 (dump debug to stderr).

use tokio::signal::unix::{signal, SignalKind};

use tracing::info;

/// Wait for SIGTERM or Ctrl+C, whichever comes first.
///
/// Resolves with the kind of signal received (for logging).
pub async fn await_termination() -&gt; &amp;'static str {
    let mut sigterm =
        signal(SignalKind::terminate()).expect("install SIGTERM handler");
    let mut sigint =
        signal(SignalKind::interrupt()).expect("install SIGINT handler");

    tokio::select! {
        _ = sigterm.recv() =&gt; {
            info!("received SIGTERM");
            "SIGTERM"
        }
        _ = sigint.recv() =&gt; {
            info!("received SIGINT");
            "SIGINT"
        }
    }
}
```

15. src/auth/ -- full

15.1 src/auth/mod.rs

```
//! Authentication + authorisation traits.  PR #6 stubs ; PR #16 wires
//! the real impls.

pub mod bearer;
pub mod mtls;

use async_trait::async_trait;

use crate::error::ServerError;

/// Outcome of an auth check.
#[derive(Debug, Clone)]
pub enum AuthDecision {
    /// Caller is authorized for the requested operation.
    Allow,
    /// Caller is rejected with this reason.
    Deny(String),
}

/// Caller identity, opaque to the service.  In PR #16 this carries
/// the SAN from the leaf cert + the JWT claims.
#[derive(Debug, Clone, Default)]
pub struct CallerIdentity {
    pub san: Option&lt;String&gt;,
    pub scopes: Vec&lt;String&gt;,
}

/// Auth-z policy.
#[async_trait]
pub trait Authorizer: Send + Sync + 'static {
    /// Check whether `caller` is allowed to invoke `rpc_name`.
    async fn authorize(&amp;self, caller: &amp;CallerIdentity, rpc_name: &amp;str) -&gt; AuthDecision;
}

/// Stub authorizer : allows everything.  ONLY for dev / PR #6
/// scaffold.  Replaced by `MtlsAuthorizer` in PR #16.
#[derive(Debug, Default, Clone)]
pub struct AllowAll;

#[async_trait]
impl Authorizer for AllowAll {
    async fn authorize(&amp;self, _: &amp;CallerIdentity, _: &amp;str) -&gt; AuthDecision {
        AuthDecision::Allow
    }
}
```

15.2 src/auth/mtls.rs + src/auth/bearer.rs

```
//! mTLS authorizer -- STUB in PR #6.  Real impl in PR #16.

use async_trait::async_trait;

use super::{AuthDecision, Authorizer, CallerIdentity};

#[derive(Debug, Clone, Default)]
pub struct MtlsAuthorizer;

#[async_trait]
impl Authorizer for MtlsAuthorizer {
    async fn authorize(&amp;self, _: &amp;CallerIdentity, _: &amp;str) -&gt; AuthDecision {
        AuthDecision::Deny("mtls authorizer not yet wired (PR #16)".into())
    }
}
```

```
//! Bearer-token authorizer -- STUB in PR #6.  Real impl in PR #16.

use async_trait::async_trait;

use super::{AuthDecision, Authorizer, CallerIdentity};

#[derive(Debug, Clone, Default)]
pub struct BearerAuthorizer;

#[async_trait]
impl Authorizer for BearerAuthorizer {
    async fn authorize(&amp;self, _: &amp;CallerIdentity, _: &amp;str) -&gt; AuthDecision {
        AuthDecision::Deny("bearer authorizer not yet wired (PR #16)".into())
```

```
    }
}
```

16. src/controller/ -- full

16.1 src/controller/mod.rs

```
//! Controller -- implements `MetaSwarmService`.

mod service;
mod state;

pub use service::ControllerService;
pub use state::ControllerState;
```

16.2 src/controller/state.rs

```
//! Shared controller state.

use std::sync::Arc;
use std::time::Instant;

use efe_meta_swarm_proto::meta_swarm::{InfoResponse, ParticipantDescriptor};

use crate::config::controller::ControllerConfig;
use crate::observability::health::HealthState;
use crate::observability::metrics::MetricsRegistry;

/// Shared controller state.  Cheap to clone (Arcs inside).
#[derive(Clone)]
pub struct ControllerState {
    pub config: Arc&lt;ControllerConfig&gt;,
    pub health: HealthState,
    pub metrics: MetricsRegistry,
    pub start_time: Instant,
}

impl std::fmt::Debug for ControllerState {
    fn fmt(&amp;self, f: &amp;mut std::fmt::Formatter&lt;'_&gt;) -&gt; std::fmt::Result {
        f.debug_struct("ControllerState")
            .field("config", &amp;self.config)
            .field("uptime_s", &amp;self.start_time.elapsed().as_secs())
            .finish_non_exhaustive()
    }
}

impl ControllerState {
    pub fn new(config: ControllerConfig) -&gt; Self {
        Self {
            config: Arc::new(config),
            health: HealthState::serving(),
            metrics: MetricsRegistry::new(),
            start_time: Instant::now(),
        }
    }

    /// Pre-built `InfoResponse` populated with everything the
    /// controller knows at startup time (build sha + uptime + an
    /// empty list of participants -- PR #8 fills it in).
    pub fn info_snapshot(&amp;self) -&gt; InfoResponse {
        InfoResponse {
            build_sha: crate::GIT_SHA.into(),
            proto_version: crate::proto::PROTO_VERSION.into(),
            configured_participants: Vec::&lt;ParticipantDescriptor&gt;::new(),
            leader_pod_name: hostname::get()
                .ok()
                .and_then(|s| s.into_string().ok())
                .unwrap_or_default(),
            uptime_s: self.start_time.elapsed().as_secs(),
            last_reload_gen: 0,
        }
    }
}
```

16.3 src/controller/service.rs

```
//! `MetaSwarmService` impl.  PR #6 returns UNIMPLEMENTED for every
//! RPC except `Health` and `Info`.

use efe_meta_swarm_proto::meta_swarm::{
    HealthRequest, HealthResponse, InfoRequest, InfoResponse, IterateChunk,
    IterateRequest, IterateResponse, ReloadConfigRequest, ReloadConfigResponse,
    ResetRequest, ResetResponse, StepOnceRequest, StepOnceResponse,
};
use tonic::{Request, Response, Status};
use tracing::{instrument, warn};

use super::state::ControllerState;
use crate::proto::MetaSwarmService;

/// Concrete `MetaSwarmService` implementation.  Holds the shared
/// controller state.
#[derive(Debug, Clone)]
pub struct ControllerService {
    pub state: ControllerState,
}

impl ControllerService {
    pub fn new(state: ControllerState) -&gt; Self {
        Self { state }
    }
}

#[tonic::async_trait]
impl MetaSwarmService for ControllerService {
    type IterateStreamStream = futures::stream::Empty&lt;Result&lt;IterateChunk, Status&gt;&gt;;

    #[instrument(skip(self, _req))]
    async fn iterate(
        &amp;self,
        _req: Request&lt;IterateRequest&gt;,
    ) -&gt; Result&lt;Response&lt;IterateResponse&gt;, Status&gt; {
        warn!("Iterate called -- wired in PR #8");
        Err(Status::unimplemented(
            "MetaSwarmService::Iterate -- wired in PR #8",
        ))
    }

    #[instrument(skip(self, _req))]
    async fn iterate_stream(
        &amp;self,
        _req: Request&lt;IterateRequest&gt;,
    ) -&gt; Result&lt;Response&lt;Self::IterateStreamStream&gt;, Status&gt; {
        Err(Status::unimplemented(
            "MetaSwarmService::IterateStream -- wired in PR #21",
        ))
    }

    #[instrument(skip(self, _req))]
    async fn step_once(
        &amp;self,
        _req: Request&lt;StepOnceRequest&gt;,
    ) -&gt; Result&lt;Response&lt;StepOnceResponse&gt;, Status&gt; {
        Err(Status::unimplemented(
            "MetaSwarmService::StepOnce -- wired in PR #8",
        ))
    }

    #[instrument(skip(self, _req))]
    async fn reset(
        &amp;self,
        _req: Request&lt;ResetRequest&gt;,
    ) -&gt; Result&lt;Response&lt;ResetResponse&gt;, Status&gt; {
        Err(Status::unimplemented(
            "MetaSwarmService::Reset -- wired in PR #8",
        ))
    }

    #[instrument(skip(self, _req))]
    async fn reload_config(
        &amp;self,
        _req: Request&lt;ReloadConfigRequest&gt;,
    ) -&gt; Result&lt;Response&lt;ReloadConfigResponse&gt;, Status&gt; {
        Err(Status::unimplemented(
            "MetaSwarmService::ReloadConfig -- wired in PR #12",
        ))
```

```
        }

        #[instrument(skip(self, _req))]
        async fn health(
            &amp;self,
            _req: Request&lt;HealthRequest&gt;,
        ) -&gt; Result&lt;Response&lt;HealthResponse&gt;, Status&gt; {
            Ok(Response::new(self.state.health.snapshot()))
        }

        #[instrument(skip(self, _req))]
        async fn info(
            &amp;self,
            _req: Request&lt;InfoRequest&gt;,
        ) -&gt; Result&lt;Response&lt;InfoResponse&gt;, Status&gt; {
            Ok(Response::new(self.state.info_snapshot()))
        }
    }

    #[cfg(test)]
    mod tests {
        use super::*;
        use crate::config::controller::ControllerConfig;

        fn test_state() -&gt; ControllerState {
            // Build a default config without env vars.
            let config = ControllerConfig {
                grpc_bind: "127.0.0.1:0".parse().unwrap(),
                metrics_bind: "127.0.0.1:0".parse().unwrap(),
                discovery: crate::config::controller::DiscoveryMode::StaticLocalhost,
                static_participants: vec![],
                manifest_path: "/dev/null".into(),
                max_inflight: 10,
                default_max_wall_ms: 15_000,
                worker_threads: 1,
                reflection_enabled: false,
                tls_required: false,
                tls_cert_path: None,
                tls_key_path: None,
                tls_ca_path: None,
                drain_timeout: std::time::Duration::from_secs(30),
            };
            ControllerState::new(config)
        }

        #[tokio::test]
        async fn health_returns_serving() {
            let svc = ControllerService::new(test_state());
            let resp = svc.health(Request::new(HealthRequest {})).await.unwrap();
            let h = resp.into_inner();
            assert_eq!(h.status, 1); // STATUS_SERVING
        }

        #[tokio::test]
        async fn info_includes_proto_version() {
            let svc = ControllerService::new(test_state());
            let resp = svc.info(Request::new(InfoRequest {})).await.unwrap();
            let i = resp.into_inner();
            assert_eq!(i.proto_version, "v1");
        }

        #[tokio::test]
        async fn iterate_returns_unimplemented() {
            let svc = ControllerService::new(test_state());
            let err = svc.iterate(Request::new(IterateRequest::default())).await.unwrap_err();
            assert_eq!(err.code(), tonic::Code::Unimplemented);
            assert!(err.message().contains("PR #8"));
        }
    }
```

17. `src/participant/` -- full

Symmetric to `controller/` -- same pattern, 3 files.

17.1 src/participant/mod.rs

```
mod service;
mod state;

pub use service::ParticipantService;
pub use state::ParticipantState;
```

17.2 src/participant/state.rs

```
use std::sync::Arc;
use std::time::Instant;

use efe_meta_swarm_proto::meta_swarm::ParticipantDescriptor;

use crate::config::participant::ParticipantConfig;
use crate::observability::health::HealthState;
use crate::observability::metrics::MetricsRegistry;

#[derive(Clone)]
pub struct ParticipantState {
    pub config: Arc&lt;ParticipantConfig&gt;,
    pub health: HealthState,
    pub metrics: MetricsRegistry,
    pub start_time: Instant,
}

impl std::fmt::Debug for ParticipantState {
    fn fmt(&amp;self, f: &amp;mut std::fmt::Formatter&lt;'_&gt;) -&gt; std::fmt::Result {
        f.debug_struct("ParticipantState")
            .field("config", &amp;self.config)
            .field("uptime_s", &amp;self.start_time.elapsed().as_secs())
            .finish_non_exhaustive()
    }
}

impl ParticipantState {
    pub fn new(config: ParticipantConfig) -&gt; Self {
        Self {
            config: Arc::new(config),
            health: HealthState::serving(),
            metrics: MetricsRegistry::new(),
            start_time: Instant::now(),
        }
    }

    /// Pre-built descriptor.  PR #6 ships placeholders ; PR #9 fills
    /// in real dim/kappa/seed from the participant's `MockParticipant`.
    pub fn descriptor(&amp;self) -&gt; ParticipantDescriptor {
        ParticipantDescriptor {
            id: self.config.id.clone(),
            dim: 0,        // PR #9 wires
            kappa: 0.0,    // PR #9 wires
            seed: 0,       // PR #9 wires
            build_sha: crate::GIT_SHA.into(),
            blas_backend: "none".into(),
            features: vec!["determinism".into()],
        }
    }
}
```

17.3 src/participant/service.rs

```
use efe_meta_swarm_proto::meta_swarm::{
    CapabilitiesRequest, CapabilitiesResponse, HealthRequest, HealthResponse,
    ResetParticipantRequest, ResetParticipantResponse, StateRequest,
    StateResponse, StepRequest, StepResponse,
};
use tonic::{Request, Response, Status};
use tracing::{instrument, warn};

use super::state::ParticipantState;
use crate::proto::MetaParticipantService;

#[derive(Debug, Clone)]
pub struct ParticipantService {
    pub state: ParticipantState,
}

impl ParticipantService {
    pub fn new(state: ParticipantState) -&gt; Self {
        Self { state }
    }
}

#[tonic::async_trait]
impl MetaParticipantService for ParticipantService {
    type StepStreamStream = futures::stream::Empty&lt;Result&lt;StepResponse, Status&gt;&gt;;

    #[instrument(skip(self, _req))]
    async fn step(
        &amp;self,
        _req: Request&lt;StepRequest&gt;,
    ) -&gt; Result&lt;Response&lt;StepResponse&gt;, Status&gt; {
        warn!("Step called -- wired in PR #9");
        Err(Status::unimplemented(
            "MetaParticipantService::Step -- wired in PR #9",
        ))
    }

    #[instrument(skip(self, _req))]
    async fn reset(
        &amp;self,
        _req: Request&lt;ResetParticipantRequest&gt;,
    ) -&gt; Result&lt;Response&lt;ResetParticipantResponse&gt;, Status&gt; {
        Err(Status::unimplemented(
            "MetaParticipantService::Reset -- wired in PR #9",
        ))
    }

    #[instrument(skip(self, _req))]
    async fn state(
        &amp;self,
        _req: Request&lt;StateRequest&gt;,
    ) -&gt; Result&lt;Response&lt;StateResponse&gt;, Status&gt; {
        Err(Status::unimplemented(
            "MetaParticipantService::State -- wired in PR #9",
        ))
    }

    #[instrument(skip(self, _req))]
    async fn step_stream(
        &amp;self,
        _req: Request&lt;tonic::Streaming&lt;StepRequest&gt;&gt;,
    ) -&gt; Result&lt;Response&lt;Self::StepStreamStream&gt;, Status&gt; {
        Err(Status::unimplemented(
            "MetaParticipantService::StepStream -- wired in PR #9",
        ))
    }

    #[instrument(skip(self, _req))]
    async fn capabilities(
        &amp;self,
        _req: Request&lt;CapabilitiesRequest&gt;,
    ) -&gt; Result&lt;Response&lt;CapabilitiesResponse&gt;, Status&gt; {
        // The descriptor is cheap to produce ; we return it even
        // though PR #9 will fill in dim/kappa/seed properly.
        Ok(Response::new(CapabilitiesResponse {
            descriptor: Some(self.state.descriptor()),
        }))
    }

    #[instrument(skip(self, _req))]
```

```
        async fn health(
            &amp;self,
            _req: Request&lt;HealthRequest&gt;,
        ) -&gt; Result&lt;Response&lt;HealthResponse&gt;, Status&gt; {
            Ok(Response::new(self.state.health.snapshot()))
        }
    }

    #[cfg(test)]
    mod tests {
        use super::*;
        use crate::config::participant::ParticipantConfig;

        fn test_state() -&gt; ParticipantState {
            let config = ParticipantConfig {
                id: "test-participant".into(),
                grpc_bind: "127.0.0.1:0".parse().unwrap(),
                metrics_bind: "127.0.0.1:0".parse().unwrap(),
                worker_threads: 1,
                tls_required: false,
                tls_cert_path: None,
                tls_key_path: None,
                tls_ca_path: None,
            };
            ParticipantState::new(config)
        }

        #[tokio::test]
        async fn health_returns_serving() {
            let svc = ParticipantService::new(test_state());
            let resp = svc.health(Request::new(HealthRequest {})).await.unwrap();
            assert_eq!(resp.into_inner().status, 1);
        }

        #[tokio::test]
        async fn capabilities_returns_descriptor() {
            let svc = ParticipantService::new(test_state());
            let resp = svc.capabilities(Request::new(CapabilitiesRequest {})).await.unwrap();
            let desc = resp.into_inner().descriptor.expect("descriptor present");
            assert_eq!(desc.id, "test-participant");
            assert!(desc.features.iter().any(|f| f == "determinism"));
        }

        #[tokio::test]
        async fn step_returns_unimplemented() {
            let svc = ParticipantService::new(test_state());
            let err = svc.step(Request::new(StepRequest::default())).await.unwrap_err();
            assert_eq!(err.code(), tonic::Code::Unimplemented);
            assert!(err.message().contains("PR #9"));
        }
    }
```

18. `src/bin/controller.rs` -- full

```
//! Controller binary entry-point.
//!
//! Runs a Tonic gRPC server on `EFE_META_SWARM_CONTROLLER_ADDR` and a
//! Prometheus exporter on `EFE_META_SWARM_CONTROLLER_METRICS`.  Exits
//! cleanly on SIGTERM / SIGINT.

use std::process::ExitCode;

use efe_meta_swarm_proto::controller::MetaSwarmServiceServer;
use efe_meta_swarm_server::{
    config::controller::ControllerConfig,
    controller::{ControllerService, ControllerState},
    observability::{metrics, shutdown, tracing as obs_tracing},
};

use ::tracing::{error, info};
use tokio::sync::watch;
use tonic::transport::Server;

fn main() -&gt; ExitCode {
    obs_tracing::init();

    let config = match ControllerConfig::from_env() {
        Ok(c) =&gt; c,
        Err(e) =&gt; {
            error!(error = %e, "controller config error");
            return ExitCode::from(2);
        }
    };

    let runtime = match tokio::runtime::Builder::new_multi_thread()
        .worker_threads(config.worker_threads)
        .enable_all()
        .build()
    {
        Ok(rt) =&gt; rt,
        Err(e) =&gt; {
            error!(error = %e, "tokio runtime build error");
            return ExitCode::from(2);
        }
    };

    let result = runtime.block_on(async_main(config));
    match result {
        Ok(()) =&gt; ExitCode::SUCCESS,
        Err(e) =&gt; {
            error!(error = %e, "controller exited with error");
            ExitCode::from(1)
        }
    }
}

async fn async_main(
    config: ControllerConfig,
) -&gt; Result&lt;(), efe_meta_swarm_server::error::ServerError&gt; {
    info!(
        version = efe_meta_swarm_server::VERSION,
        git_sha = efe_meta_swarm_server::GIT_SHA,
        profile = efe_meta_swarm_server::PROFILE,
        grpc_bind = %config.grpc_bind,
        metrics_bind = %config.metrics_bind,
        worker_threads = config.worker_threads,
        discovery = ?config.discovery,
        "controller starting",
    );

    let state = ControllerState::new(config.clone());
    let svc = ControllerService::new(state.clone());

    // Shutdown channel : flipped to `true` on SIGTERM/SIGINT ; both
    // the gRPC server and the metrics exporter watch it.
    let (shutdown_tx, shutdown_rx) = watch::channel(false);

    // Spawn the Prometheus exporter.
    let metrics_handle = {
        let rx = shutdown_rx.clone();
        let registry = state.metrics.clone();
        let addr = config.metrics_bind;
        tokio::spawn(async move {
            if let Err(e) = metrics::spawn_exporter(addr, registry, rx).await {
```

```
                error!(error = %e, "metrics exporter failed");
            }
        })
    };

    // Build the gRPC server.
    let mut server = Server::builder()
        .timeout(std::time::Duration::from_secs(60))
        .add_service(MetaSwarmServiceServer::new(svc));

    // Tonic-reflection is gated off the `reflection` Cargo feature.
    // PR #6 ships the feature plumbing only ; PR #18 enables it by
    // default for the controller binary.
    let _ = config.reflection_enabled; // suppress unused warning

    let grpc_handle = {
        let mut shutdown_rx = shutdown_rx.clone();
        let addr = config.grpc_bind;
        tokio::spawn(async move {
            info!(addr = %addr, "gRPC server listening");
            let serve = server.serve_with_shutdown(addr, async move {
                let _ = shutdown_rx.changed().await;
                info!("gRPC server shutting down");
            });
            if let Err(e) = serve.await {
                error!(error = %e, "gRPC server failed");
            }
        })
    };

    // Wait for SIGTERM / SIGINT.
    let kind = shutdown::await_termination().await;
    info!(signal = kind, "shutting down");

    // Mark unhealthy : load balancers should stop sending traffic.
    state.health.set_not_serving();

    // Trigger shutdown.
    let _ = shutdown_tx.send(true);

    // Wait for both tasks (bounded by the drain timeout).
    let drain = config.drain_timeout;
    if let Err(e) = tokio::time::timeout(drain, grpc_handle).await {
        error!(error = %e, "gRPC server did not drain in time");
    }
    if let Err(e) = tokio::time::timeout(drain, metrics_handle).await {
        error!(error = %e, "metrics exporter did not drain in time");
    }

    info!("controller stopped");
    Ok(())
}
```

19. `src/bin/participant.rs` -- full

Symmetric to `controller.rs` ; same shape, different service.

```
//! Participant binary entry-point.

use std::process::ExitCode;

use efe_meta_swarm_proto::participant::MetaParticipantServiceServer;
use efe_meta_swarm_server::{
    config::participant::ParticipantConfig,
    observability::{metrics, shutdown, tracing as obs_tracing},
    participant::{ParticipantService, ParticipantState},
};

use ::tracing::{error, info};
use tokio::sync::watch;
use tonic::transport::Server;

fn main() -&gt; ExitCode {
    obs_tracing::init();

    let config = match ParticipantConfig::from_env() {
        Ok(c) =&gt; c,
        Err(e) =&gt; {
            error!(error = %e, "participant config error");
            return ExitCode::from(2);
```

```
            }
        };

        let runtime = match tokio::runtime::Builder::new_multi_thread()
            .worker_threads(config.worker_threads)
            .enable_all()
            .build()
        {
            Ok(rt) =&gt; rt,
            Err(e) =&gt; {
                error!(error = %e, "tokio runtime build error");
                return ExitCode::from(2);
            }
        };

        let result = runtime.block_on(async_main(config));
        match result {
            Ok(()) =&gt; ExitCode::SUCCESS,
            Err(e) =&gt; {
                error!(error = %e, "participant exited with error");
                ExitCode::from(1)
            }
        }
    }

    async fn async_main(
        config: ParticipantConfig,
    ) -&gt; Result&lt;(), efe_meta_swarm_server::error::ServerError&gt; {
        info!(
            version = efe_meta_swarm_server::VERSION,
            git_sha = efe_meta_swarm_server::GIT_SHA,
            id = %config.id,
            grpc_bind = %config.grpc_bind,
            metrics_bind = %config.metrics_bind,
            worker_threads = config.worker_threads,
            "participant starting",
        );

        let state = ParticipantState::new(config.clone());
        let svc = ParticipantService::new(state.clone());

        let (shutdown_tx, shutdown_rx) = watch::channel(false);

        let metrics_handle = {
            let rx = shutdown_rx.clone();
            let registry = state.metrics.clone();
            let addr = config.metrics_bind;
            tokio::spawn(async move {
                if let Err(e) = metrics::spawn_exporter(addr, registry, rx).await {
                    error!(error = %e, "metrics exporter failed");
                }
            })
        };

        let server = Server::builder()
            .timeout(std::time::Duration::from_secs(60))
            .add_service(MetaParticipantServiceServer::new(svc));

        let grpc_handle = {
            let mut shutdown_rx = shutdown_rx.clone();
            let addr = config.grpc_bind;
            tokio::spawn(async move {
                info!(addr = %addr, "gRPC server listening");
                let serve = server.serve_with_shutdown(addr, async move {
                    let _ = shutdown_rx.changed().await;
                    info!("gRPC server shutting down");
                });
                if let Err(e) = serve.await {
                    error!(error = %e, "gRPC server failed");
                }
            })
        };

        let kind = shutdown::await_termination().await;
        info!(signal = kind, "shutting down");

        state.health.set_not_serving();
        let _ = shutdown_tx.send(true);

        let drain = std::time::Duration::from_secs(30);
        let _ = tokio::time::timeout(drain, grpc_handle).await;
        let _ = tokio::time::timeout(drain, metrics_handle).await;
```

```
    info!("participant stopped");
    Ok(())
}
```

20. `README.md` -- full

```
# `efe-meta-swarm-server`

Tonic-based gRPC servers for the EFE meta-swarm coordinator.

This crate ships **two binaries** under one Cargo crate :

- `controller`  -- implements `MetaSwarmService`
- `participant` -- implements `MetaParticipantService`

PR #6 (Sprint A1 Phase 2) lands the **scaffold** only.  Subsequent
PRs wire the business logic :

| PR | Adds |
|---|---|
| #8 | `MetaSwarmService::{Iterate, StepOnce, Reset}` |
| #9 | `MetaParticipantService::{Step, Reset, State}` |
| #11 | `EFE_META_SWARM_ENABLED` plumbing in `efe-server` |
| #12 | SIGHUP hot-reload + `MetaSwarmService::ReloadConfig` |
| #16 | mTLS + cert-manager Issuer |
| #17 | 14 Prometheus metrics |
| #18 | 8 OTel span types + tonic-reflection |

## Quick start

### Local -- 3 participants + 1 controller

In four terminals :

# Participant 1 (supplier-scoring)

EFE_META_SWARM_PARTICIPANT_ID=supplier-scoring \
EFE_META_SWARM_PARTICIPANT_ADDR=0.0.0.0:50091 \
EFE_META_SWARM_PARTICIPANT_METRICS=0.0.0.0:9092 \ cargo run -p efe-meta-swarm-server --bin participant

# Participant 2 (contract-risk)

EFE_META_SWARM_PARTICIPANT_ID=contract-risk \ EFE_META_SWARM_PARTICIPANT_ADDR=0.0.0.0:50092 \ EFE_META_SWARM_PARTICIPANT_METRICS=0.0.0.0:9093 \ cargo run -p efe-meta-swarm-server --bin participant

# Participant 3 (spend-categorization)

EFE_META_SWARM_PARTICIPANT_ID=spend-categorization \
EFE_META_SWARM_PARTICIPANT_ADDR=0.0.0.0:50093 \
EFE_META_SWARM_PARTICIPANT_METRICS=0.0.0.0:9094 \ cargo run -p efe-meta-swarm-server --bin participant

# Controller

EFE_META_SWARM_CONTROLLER_ADDR=0.0.0.0:50080 \
EFE_META_SWARM_CONTROLLER_METRICS=0.0.0.0:9091 \ EFE_META_SWARM_PARTICIPANTS=supplier-scoring=localhost:50091,contract-risk=localhost:50092,spend-categorization=localhost:50093 \ cargo run -p efe-meta-swarm-server --bin controller ```

In a fifth terminal :

```
grpcurl -plaintext localhost:50080 efe.v1.meta_swarm.MetaSwarmService/Health
{"status":"STATUS_SERVING","inflight":0,"reload_gen":0}

grpcurl -plaintext localhost:50080 efe.v1.meta_swarm.MetaSwarmService/Info
```
```

```
# {"buildSha":"abcdef12","protoVersion":"v1","leaderPodName":"hostname",...}

grpcurl -plaintext -d '{}' localhost:50080 efe.v1.meta_swarm.MetaSwarmService/Iterate
# ERROR: Unimplemented: MetaSwarmService::Iterate -- wired in PR #8
```

Environment variables

See [`EFE_A1_PHASE2_ARCHITECTURE.md`][doc] § 7 for the full catalogue. PR #6 honours these subsets :

Controller

Var	Default	Required?
`EFE_META_SWARM_CONTROLLER_ADDR`	`0.0.0.0:50080`	no
`EFE_META_SWARM_CONTROLLER_METRICS`	`0.0.0.0:9091`	no
`EFE_META_SWARM_DISCOVERY`	`static-localhost`	no
`EFE_META_SWARM_PARTICIPANTS`	(empty)	no (but useful)
`EFE_META_SWARM_MANIFEST`	`/etc/efe-meta-swarm/manifest.yaml`	no (PR #8 mandatory)
`EFE_META_SWARM_MAX_INFLIGHT`	`10`	no
`EFE_META_SWARM_DEFAULT_MAX_WALL_MS`	`15000`	no
`EFE_META_SWARM_WORKER_THREADS`	`2`	no
`EFE_META_SWARM_RELOAD_DRAIN_TIMEOUT_S`	`30`	no
`EFE_META_SWARM_REFLECTION`	`false`	no
`EFE_META_SWARM_TLS_REQUIRED`	`false`	no
`EFE_META_SWARM_TLS_CERT`	--	yes if `tls_required=true`
`EFE_META_SWARM_TLS_KEY`	--	yes if `tls_required=true`
`EFE_META_SWARM_TLS_CA`	--	no
`EFE_META_SWARM_LOG`	`info`	no

Participant

Var	Default	Required?
`EFE_META_SWARM_PARTICIPANT_ID`	--	**YES**
`EFE_META_SWARM_PARTICIPANT_ADDR`	`0.0.0.0:50090`	no
`EFE_META_SWARM_PARTICIPANT_METRICS`	`0.0.0.0:9092`	no
`EFE_META_SWARM_WORKER_THREADS`	`1`	no
TLS vars	--	same as controller

Cargo features

Feature	Default	What it does
`reflection`	off	Enable `tonic-reflection` (PR #18 ships the dep + integration ; PR #6 plumbs the feature flag only).

Building locally

```
cargo build -p efe-meta-swarm-server --release
ls target/release/{controller,participant}
```

Tests

```
cargo test -p efe-meta-swarm-server              # unit tests (config, manifest, health, services)
cargo test -p efe-meta-swarm-server --test integration_health_smoke
```

The integration test spawns the `controller` binary on a random port and asserts `Health` returns `STATUS_SERVING`. Run with `RUST_LOG=info` for verbose output.

Cross-references

- Phase 2 architecture : [`docs/EFE_A1_PHASE2_ARCHITECTURE.md`][doc]
- Phase 2 roadmap : `docs/EFE_A1_PHASE2_ROADMAP.md`
- PR #6 detailed architecture : `docs/EFE_A1_PHASE2_PR06_ARCHITECTURE.md`
- Proto crate : `crates/efe-meta-swarm-proto/README.md`

[doc]: ../../docs/EFE_A1_PHASE2_ARCHITECTURE.md ````

21. Environment variables catalogue

The 18 env vars introduced by PR #6, with origin docs :

Var	Source	Used by	Default
`EFE_META_SWARM_CONTROLLER_ADDR`	PR #6	controller bin	`0.0.0.0:50080`
`EFE_META_SWARM_CONTROLLER_METRICS`	PR #6	controller bin	`0.0.0.0:9091`
`EFE_META_SWARM_DISCOVERY`	PR #6	controller bin	`static-localhost`
`EFE_META_SWARM_PARTICIPANTS`	PR #6	controller bin	(empty)
`EFE_META_SWARM_MANIFEST`	PR #6	controller bin	`/etc/efe-meta-swarm/manifest.yaml`
`EFE_META_SWARM_MAX_INFLIGHT`	PR #6	controller bin	`10`
`EFE_META_SWARM_DEFAULT_MAX_WALL_MS`	PR #6	controller bin	`15000`
`EFE_META_SWARM_WORKER_THREADS`	PR #6	both bins	`2` / `1`
`EFE_META_SWARM_RELOAD_DRAIN_TIMEOUT_S`	PR #6	controller bin	`30`
`EFE_META_SWARM_REFLECTION`	PR #6 (flag, no impl)	controller bin	`false`
`EFE_META_SWARM_PARTICIPANT_ID`	PR #6	participant bin	**REQUIRED**
`EFE_META_SWARM_PARTICIPANT_ADDR`	PR #6	participant bin	`0.0.0.0:50090`
`EFE_META_SWARM_PARTICIPANT_METRICS`	PR #6	participant bin	`0.0.0.0:9092`
`EFE_META_SWARM_TLS_REQUIRED`	PR #6	both bins	`false`
`EFE_META_SWARM_TLS_CERT`	PR #6	both bins	(none)
`EFE_META_SWARM_TLS_KEY`	PR #6	both bins	(none)
`EFE_META_SWARM_TLS_CA`	PR #6	both bins	(none)
`EFE_META_SWARM_LOG`	PR #6	both bins (tracing-subscriber)	`info`

The remaining vars in the Phase-2 architecture (§ 7) -- `EFE_META_SWARM_OTEL_ENDPOINT`, `EFE_META_SWARM_TRACE_SAMPLE_RATIO`, `EFE_META_SWARM_RELOAD_ABORT_INFLIGHT`, `EFE_META_SWARM_LAST_P99_OBSERVED_MS`, `EFE_META_SWARM_DETERMINISTIC` -- are wired by PRs #17, #18, #12

respectively.

22. Tokio runtime + signal handling

22.1 Runtime choice

- `tokio::runtime::Builder::new_multi_thread()`
- `.worker_threads(N)` where N comes from `EFE_META_SWARM_WORKER_THREADS`
- `.enable_all()` (I/O + time)

Rationale :

- gRPC is I/O-bound -> multi-thread is mandatory for parallelism.
- Single-thread runtime would serialise all RPCs, killing throughput.
- 2 workers is enough at the planned 100 rps (controller) ; 1 worker

for participants (less I/O concurrency).

22.2 Signal handling

PR #6 handles two signals :

Signal	Behaviour
`SIGINT` (Ctrl+C)	Same as SIGTERM.
`SIGTERM`	Mark health NOT_SERVING ; flip shutdown channel ; wait for gRPC + metrics to drain (up to `drain_timeout`) ; exit.

Deferred to later PRs :

Signal	Behaviour	PR
`SIGHUP`	Trigger `ReloadConfig`	PR #12
`SIGUSR1`	Dump debug info to stderr	PR #18

22.3 Drain sequence (graceful)

1. Receive SIGTERM.
2. Set `HealthState::NotServing`. K8s readiness probe flips RED ; LB stops sending new RPCs.
3. Flip `shutdown_tx` channel.
4. `Server::serve_with_shutdown` future resolves (no new connections accepted ; in-flight RPCs continue).
5. Metrics exporter's accept loop breaks.
6. `tokio::time::timeout(drain_timeout, ...).await` on both join handles.
7. Process exits with code 0.

If any task doesn't drain in time, we log + exit with code 0 anyway (the K8s preStop hook adds an additional 10 s grace period).

23. Smoke test : `tests/integration_health_smoke.rs`

```
//! End-to-end smoke test : spawn the controller binary, connect via
//! gRPC, call `Health`, assert `STATUS_SERVING`.
//!
//! This is the **load-bearing pin** for the binary's basic
//! correctness.  If a future PR breaks the startup sequence (e.g.
//! moves the gRPC listener after the metrics exporter), this test
//! turns RED.

use std::process::{Child, Command, Stdio};
use std::time::Duration;

use efe_meta_swarm_proto::controller::MetaSwarmServiceClient;
use efe_meta_swarm_proto::meta_swarm::HealthRequest;
use tonic::transport::Endpoint;

struct ProcessGuard {
    child: Child,
}

impl Drop for ProcessGuard {
    fn drop(&amp;mut self) {
        let _ = self.child.kill();
        let _ = self.child.wait();
    }
}

fn spawn_controller(grpc_port: u16, metrics_port: u16) -&gt; ProcessGuard {
    // CARGO_BIN_EXE_controller : set by cargo at test time, points to
    // the freshly built controller binary.
    let bin = env!("CARGO_BIN_EXE_controller");
    let child = Command::new(bin)
        .env("EFE_META_SWARM_CONTROLLER_ADDR", format!("127.0.0.1:{grpc_port}"))
        .env(
            "EFE_META_SWARM_CONTROLLER_METRICS",
            format!("127.0.0.1:{metrics_port}"),
        )
        .env("EFE_META_SWARM_LOG", "warn")
        .stdout(Stdio::null())
        .stderr(Stdio::null())
        .spawn()
        .expect("spawn controller binary");
    ProcessGuard { child }
}

async fn wait_until_healthy(endpoint: &amp;str) {
    for _ in 0..40 {
        if let Ok(channel) = Endpoint::from_shared(endpoint.to_string())
            .unwrap()
            .timeout(Duration::from_millis(200))
            .connect_timeout(Duration::from_millis(200))
            .connect()
            .await
        {
            let mut client = MetaSwarmServiceClient::new(channel);
            if let Ok(resp) = client.health(HealthRequest {}).await {
                if resp.into_inner().status == 1 {
                    return;
                }
            }
        }
        tokio::time::sleep(Duration::from_millis(250)).await;
    }
    panic!("controller did not become healthy within 10 s");
}

#[tokio::test(flavor = "multi_thread")]
async fn controller_health_returns_serving() {
    // Pick high random-ish ports to avoid conflicts.
    let grpc_port = 53080;
    let metrics_port = 53091;
    let _guard = spawn_controller(grpc_port, metrics_port);

    let endpoint = format!("http://127.0.0.1:{grpc_port}");
    wait_until_healthy(&amp;endpoint).await;

    let channel = Endpoint::from_shared(endpoint).unwrap().connect().await.unwrap();
    let mut client = MetaSwarmServiceClient::new(channel);
    let resp = client.health(HealthRequest {}).await.unwrap();
    let h = resp.into_inner();
    assert_eq!(h.status, 1, "expected STATUS_SERVING");
```

```
    assert_eq!(h.inflight, 0);
}

#[tokio::test(flavor = "multi_thread")]
async fn controller_iterate_returns_unimplemented() {
    let grpc_port = 53082;
    let metrics_port = 53092;
    let _guard = spawn_controller(grpc_port, metrics_port);

    let endpoint = format!("http://127.0.0.1:{grpc_port}");
    wait_until_healthy(&amp;endpoint).await;

    let channel = Endpoint::from_shared(endpoint).unwrap().connect().await.unwrap();
    let mut client = MetaSwarmServiceClient::new(channel);
    let err = client
        .iterate(efe_meta_swarm_proto::meta_swarm::IterateRequest::default())
        .await
        .unwrap_err();
    assert_eq!(err.code(), tonic::Code::Unimplemented);
    assert!(err.message().contains("PR #8"));
}

#[tokio::test(flavor = "multi_thread")]
async fn metrics_endpoint_responds() {
    let grpc_port = 53084;
    let metrics_port = 53094;
    let _guard = spawn_controller(grpc_port, metrics_port);

    // Wait for the metrics port to become reachable.
    let url = format!("http://127.0.0.1:{metrics_port}/metrics");
    for _ in 0..40 {
        if let Ok(resp) = reqwest::get(&amp;url).await {
            if resp.status().is_success() {
                return;
            }
        }
        tokio::time::sleep(Duration::from_millis(250)).await;
    }
    panic!("metrics endpoint did not become reachable within 10 s");
}
```

The smoke test uses `reqwest` for the metrics endpoint check. Add to `[dev-dependencies]`:

```
reqwest = { version = "0.12", default-features = false, features = ["http2"] }
```

24. CI validation

Same shape as PR #5:

Job	Trigger	Pass criterion
`cargo build -p efe-meta-swarm-server --bins`	every push	exit 0
`cargo build -p efe-meta-swarm-server --release --bins`	every push	exit 0
`cargo build -p efe-meta-swarm-server --features reflection`	every push	exit 0
`cargo test -p efe-meta-swarm-server` (unit)	every push	all pass
`cargo test -p efe-meta-swarm-server --test integration_health_smoke`	every push	3 tests pass
`cargo doc -p efe-meta-swarm-server --no-deps`	every push	0 warnings
`cargo clippy -p efe-meta-swarm-server --all-targets`	every push	0 warnings on this crate
Phase-1 master gate	every push	exit 0 (no regression)
PR #5 still builds + tests	every push	exit 0
Workspace `cargo check` of a sample (efe-server, efe-core, efe-meta-swarm)	every push	exit 0

No new workflows in PR #6 ; the new `bench-meta-swarm-grpc` job lands in PR #22.

25. Acceptance criteria (binary)

PR #6 is mergeable iff **all** :

#	Criterion	Verification
AC1	`cargo build -p efe-meta-swarm-server` succeeds with 0 warnings	CI
AC2	`cargo build -p efe-meta-swarm-server --release --bins` succeeds	CI
AC3	`cargo build -p efe-meta-swarm-server --features reflection` succeeds	CI
AC4	`cargo run --bin controller` (with `EFE_META_SWARM_PARTICIPANT_ID` unset -- controller doesn't need it) starts, binds 50080 + 9091, exits cleanly on SIGTERM	Manual + smoke
AC5	`cargo run --bin participant` (with `EFE_META_SWARM_PARTICIPANT_ID=test`) starts, binds 50090 + 9092, exits cleanly on SIGTERM	Manual + smoke
AC6	`MetaSwarmService::Health` returns `STATUS_SERVING`	smoke test
AC7	`MetaSwarmService::Info` returns a populated `InfoResponse` (build_sha, proto_version="v1", hostname)	smoke test (extended)
AC8	`MetaSwarmService::Iterate/StepOnce/Reset/ReloadConfig/IterateStream` all return `Status::unimplemented` with a "wired in PR #N" message	smoke test + unit test
AC9	`MetaParticipantService::Health` returns `STATUS_SERVING`	smoke test
AC10	`MetaParticipantService::Capabilities` returns a `ParticipantDescriptor` with the configured id	unit test
AC11	`MetaParticipantService::Step/Reset/State/StepStream` all return `Status::unimplemented`	unit + smoke
AC12	`/metrics` endpoint responds with `200 OK` + Prometheus text format (empty registry, just `# EOF` or similar)	smoke test
AC13	`cargo test -p efe-meta-swarm-server` passes all unit + integration tests	CI
AC14	`cargo clippy -p efe-meta-swarm-server --all-targets` : 0 warnings on this crate	CI
AC15	`cargo doc -p efe-meta-swarm-server --no-deps` : 0 warnings	CI
AC16	`cargo fmt -p efe-meta-swarm-server -- --check` : exit 0	CI
AC17	Phase-1 master gate stays GREEN	CI
AC18	PR #5 tests still pass (regression check)	CI

#	Criterion	Verification
AC19	No file outside `crates/efe-meta-swarm-server/`, `Cargo.toml`, `Cargo.lock` is modified	`git diff --stat` review
AC20	4 reviewers approved : 2x meta-swarm + 1x sre + 1x platform	GitHub
AC21	sre-team has confirmed the SIGTERM drain protocol matches K8s preStop expectations	SRE comment
AC22	platform-team has confirmed the env-var contract matches the Helm chart values plan (PR #13 will consume them)	platform comment

26. PR description template

```
# [Sprint A1 Phase 2 -- PR #6] feat(efe-meta-swarm-server): scaffold

## Summary

Second PR of Phase 2.  Lands the gRPC server scaffold : two
binaries (`controller`, `participant`) under one crate
`efe-meta-swarm-server`.  Wires the Tonic surface defined in
`efe-meta-swarm-proto` (PR #5).

Every RPC except `Health`, `Info`, and `Capabilities` returns
`Status::unimplemented("... wired in PR #N")`.

This is **pure scaffolding**.  Real business logic lands in PRs
#8, #9, #11, #12, #16, #17, #18.

Full architecture : [`docs/EFE_A1_PHASE2_PR06_ARCHITECTURE.md`](docs/EFE_A1_PHASE2_PR06_ARCHITECTURE.md).

## What this PR adds

| Area | Detail |
|---|---|
| New crate | `crates/efe-meta-swarm-server/` (~ 2 100 LOC) |
| Binaries | `controller` + `participant` |
| Modules | `config`, `manifest`, `discovery`, `observability`, `auth`, `controller`, `participant`, `err ■
    or` |
| Real RPCs | `Health`, `Info`, `Capabilities` |
| Stubbed RPCs (return UNIMPLEMENTED) | the remaining 10 (`Iterate`, `IterateStream`, `StepOnce`, `Reset` ■
     x2, `ReloadConfig`, `Step`, `State`, `StepStream`) |
| Env vars introduced | 18 (cf. `README.md` § "Environment variables") |
| Signal handling | SIGTERM + SIGINT graceful drain |
| Prometheus exporter | `/metrics` on a separate port (empty registry until PR #17) |
| Tracing | `tracing-subscriber` (JSON / pretty based on TTY) |
| `panic = "unwind"` override | crate-local |
| Smoke test | `tests/integration_health_smoke.rs` (3 tests : spawn binary, gRPC call, metrics endpoint)  ■
    |

## Test plan

- [x] `cargo build -p efe-meta-swarm-server --bins` (default features) succeeds
- [x] `cargo build -p efe-meta-swarm-server --bins --release` succeeds
- [x] `cargo build -p efe-meta-swarm-server --bins --features reflection` succeeds
- [x] `cargo test -p efe-meta-swarm-server` : all unit tests pass
- [x] `cargo test -p efe-meta-swarm-server --test integration_health_smoke` : 3 / 3 pass
- [x] `cargo clippy -p efe-meta-swarm-server --all-targets` : 0 warnings
- [x] `cargo fmt -p efe-meta-swarm-server -- --check` : exit 0
- [x] `cargo doc -p efe-meta-swarm-server --no-deps` : 0 warnings
- [x] Phase-1 master gate stays GREEN
- [x] PR #5 (`efe-meta-swarm-proto`) tests still pass

## Non-regression statement

- [OK] Zero file modified outside `crates/efe-meta-swarm-server/`,
  workspace `Cargo.toml` (+1 member), `Cargo.lock` (~ 20 lines diff).
- [OK] Phase 1 unaffected : `cargo test -p efe-meta-swarm` still passes
  (200 default / 210 attention-weighted / 203 core-integration).
- [OK] PR #5 (`efe-meta-swarm-proto`) unaffected.
- [OK] Carve-out R5-B1 respected.

## Honest semantics

- 10/13 RPCs return UNIMPLEMENTED -- by design.  Real bodies in
  PRs #8, #9, #11, #12.
- No mTLS in PR #6 ; plaintext gRPC.  PR #16 wires mTLS.
- No real metrics ; the registry is empty.  PR #17 wires the 14.
- No OTel exporter ; only the `tracing-subscriber` (logs).  PR #18
  wires OTel.
- No multi-process test ; single-process smoke only.  PR #10 owns
  multi-process.

## Reviewer guidance

Suggested order :

1. [`docs/EFE_A1_PHASE2_PR06_ARCHITECTURE.md`](docs/EFE_A1_PHASE2_PR06_ARCHITECTURE.md) -- read this firs ■
    t.
2. `crates/efe-meta-swarm-server/README.md` -- operator-facing.
3. `crates/efe-meta-swarm-server/Cargo.toml` -- note the `panic = "unwind"` override.
4. `src/bin/{controller,participant}.rs` -- startup sequence (read both, they're symmetric).
5. `src/controller/service.rs` + `src/participant/service.rs` -- the UNIMPLEMENTED stubs.
6. `src/observability/{health,shutdown,metrics}.rs` -- cross-cutting.
```

```
7. `tests/integration_health_smoke.rs` -- what the surface promises.

Required approvers : 2x meta-swarm + 1x sre + 1x platform.

## References

- [`docs/EFE_A1_PHASE2_ARCHITECTURE.md`](docs/EFE_A1_PHASE2_ARCHITECTURE.md) (Phase 2 design)
- [`docs/EFE_A1_PHASE2_ROADMAP.md`](docs/EFE_A1_PHASE2_ROADMAP.md) § 2.1
- [`docs/EFE_A1_PHASE2_PR05_ARCHITECTURE.md`](docs/EFE_A1_PHASE2_PR05_ARCHITECTURE.md) (PR #5 -- the prot ■
    o crate this depends on)
- [`crates/efe-meta-swarm-proto/README.md`](crates/efe-meta-swarm-proto/README.md)
```

27. Risk analysis (PR-local)

ID	Risk	L/I	Mitigation
PR6-R1	`[profile.release].panic = "unwind"` not honoured by cargo for a non-root crate	M/H	Acceptance test : run the binary, panic at startup, observe drop runs. Fallback : bump workspace `panic = "unwind"` (tracked as `DT-A1-PHASE2-PANIC-UNWIND`).
PR6-R2	Smoke test flaky in CI (port already in use)	L/M	Hard-code high ports (53080-53094) unlikely to collide. If flaky, switch to `:0` (kernel-assigned) + read back.
PR6-R3	Tonic 0.12's `Server::serve_with_shutdown` API change	L/L	PR #5 already pins 0.12 ; semver-compat.
PR6-R4	Hyper 1.0 metrics endpoint compile errors on aarch64	L/M	Verified on macOS aarch64 dev box ; CI is ubuntu x86_64 (covered).
PR6-R5	`signal-hook` collides with tokio's signal handling	L/M	We use tokio's `signal::unix` exclusively ; `signal-hook` is in deps but NOT used in PR #6 code (reserved for PR #12 SIGHUP). Cargo will dedupe.
PR6-R6	A reviewer asks for `clap` instead of env vars	M/L	Reference AD3 ; env-only matches Phase-2 architecture. Defer CLI args to a Phase-3 RFC if needed.
PR6-R7	The metrics exporter spinning a tokio task that doesn't drain on shutdown	L/M	`tokio::time::timeout` on the join handle + the watch channel guarantees liveness. Smoke test verifies SIGTERM exit.
PR6-R8	`env!("CARGO_BIN_EXE_controller")` not set during regular `cargo build`	L/L	Cargo sets this env var ONLY for tests using the binary. We use it ONLY in `tests/integration_health_smoke.rs`. Verified.

28. Reviewer checklist

General

- [ ] PR title : `feat(efe-meta-swarm-server): scaffold`.
- [ ] PR description follows template.
- [ ] 6 distinct commits, each individually CI-green.
- [ ] Diff scope strictly limited to `crates/efe-meta-swarm-server/` + workspace registration.

Crate metadata

- [ ] `Cargo.toml` :

- [ ] `publish = false`
- [ ] `version = "0.2.0"`
- [ ] `rust-version = "1.88"`
- [ ] 2 `[[bin]]` blocks
- [ ] `[profile.release].panic = "unwind"` with comment
- [ ] All workspace deps use `{ workspace = true }`

`build.rs`

- [ ] Sets `GIT_SHA` env var.
- [ ] Sanitizes the SHA (alphanumeric, length <= 16, fallback `nogit` / `invalid-sha`).
- [ ] `cargo:rerun-if-changed=.git/HEAD`.

Binaries

- [ ] Both `controller` and `participant` `main()` :
- [ ] Call `obs_tracing::init()`.
- [ ] Parse config from env.
- [ ] Build a multi-thread tokio runtime.
- [ ] Spawn metrics exporter.
- [ ] Spawn gRPC server with `serve_with_shutdown`.
- [ ] Await SIGTERM / SIGINT.
- [ ] Set health NOT_SERVING.
- [ ] Drain with timeout.
- [ ] Return `ExitCode::SUCCESS` on clean exit.

Services

- [ ] `ControllerService::health` returns the real `HealthState::snapshot()`.
- [ ] `ControllerService::info` returns build_sha + proto_version="v1" + hostname.
- [ ] All 10 other RPCs return `Status::unimplemented` with a "PR #N" message.
- [ ] `ParticipantService::health` and `capabilities` real ; rest UNIMPLEMENTED.

Smoke test

- [ ] 3 tests in `tests/integration_health_smoke.rs` :
- [ ] `controller_health_returns_serving`
- [ ] `controller_iterate_returns_unimplemented`
- [ ] `metrics_endpoint_responds`
- [ ] All 3 use a `ProcessGuard` that kills the child on drop.
- [ ] Ports are hard-coded > 50000 (high enough to avoid privileged conflict).

Observability

- [ ] `HealthState` uses atomics (no Mutex).
- [ ] `MetricsRegistry` is empty (PR #17 will populate).
- [ ] tracing subscriber output : JSON when stderr is not a TTY, pretty when it is.
- [ ] Shutdown : `tokio::signal::unix::signal(SignalKind::terminate())`.

Documentation

- [ ] README covers : quick start, env vars, features, tests, cross-refs.
- [ ] No marketing in docs.
- [ ] Architecture doc cross-refs present.

CI

- [ ] All CI green on latest push.
- [ ] Phase-1 master gate still GREEN.
- [ ] PR #5 tests still pass.

Sign-off

- [ ] 2x meta-swarm-team approvals.
- [ ] 1x sre-team approval.
- [ ] 1x platform-team approval.

29. Definition of Done

PR #6 is done iff :

1. All 22 acceptance criteria GREEN.
2. All 4 reviewer approvals in.
3. Merged to `main` with merge-commit linking to this doc.
4. Slack post in `#meta-swarm-standup` : `PR #6 merged ; PRs #8, #9, #11 unblocked`.
5. PR #5's smoke test still passes on `main` (regression check).

PR #6 is NOT done if :

- Any AC is RED.
- A reviewer has a blocking comment unresolved.
- Phase-1 master gate is RED.
- The binary cannot be run via `cargo run` (manual repro by reviewer).
- `cargo doc` produces warnings on PR-6 files.

30. Post-merge follow-ups

These unlock once PR #6 lands :

Follow-up	PR	Owner	When
`MetaSwarmService::Iterate` body	PR #8	meta-swarm-team	Day 2 after #6 merge
`MetaParticipantService::Step` body	PR #9	meta-swarm-team	Day 2
`tests/integration_grpc_multiprocess.rs`	PR #10	meta-swarm-team	Day 5
`EFE_META_SWARM_ENABLED` plumbing	PR #11	meta-swarm-team	Sprint 2 W3
SIGHUP + ReloadConfig	PR #12	meta-swarm-team	Sprint 2 W3
Helm sub-chart	PR #13	platform-team	Sprint 2 W3
mTLS	PR #16	sec + platform	Sprint 2 W4
14 Prometheus metrics	PR #17	meta-swarm-team	Sprint 3 W5

Follow-up	PR	Owner	When
8 OTel spans + reflection enable	PR #18	meta-swarm-team	Sprint 3 W5
loadgen	PR #20	meta-swarm-team	Sprint 3 W6
public-bench harness	PR #21	meta-swarm-team	Sprint 3 W6

Appendix A -- File-by-file deliverable checklist

```
crates/efe-meta-swarm-server/
+-- Cargo.toml                                    [[v] in this PR]
+-- build.rs                                      [[v] in this PR]
+-- README.md                                     [[v] in this PR]
+-- src/
|   +-- lib.rs                                    [[v]]
|   +-- error.rs                                  [[v]]
|   +-- config/{mod, controller, participant}.rs [[v]]
|   +-- manifest/{mod, schema}.rs                 [[v]]
|   +-- discovery/{mod, static_impl}.rs           [[v]]
|   +-- observability/{mod, health, metrics, tracing, shutdown}.rs [[v]]
|   +-- auth/{mod, mtls, bearer}.rs               [[v]]
|   +-- controller/{mod, service, state}.rs       [[v]]
|   +-- participant/{mod, service, state}.rs      [[v]]
|   +-- bin/{controller, participant}.rs          [[v]]
+-- tests/integration_health_smoke.rs             [[v]]

Cargo.toml (workspace)                             [[v] touched : +1 member]
Cargo.lock                                         [[v] regenerated]
```

Appendix B -- Glossary

- **AD-N** : Architectural Decision N (cf. § 4).
- **AC-N** : Acceptance Criterion N (cf. § 25).
- **PR6-RN** : PR-local Risk N (cf. § 27).
- **UNIMPLEMENTED** : the standard gRPC code returned for stub RPCs. In tonic : `Status::unimplemented(...)`.
- **Graceful drain** : the SIGTERM handling sequence that lets in-flight RPCs finish before the process exits (§ 22.3).
- `panic = "unwind"` : the panic strategy that runs `Drop` impls. Phase-1 uses `"abort"` ; PR #6 overrides for this crate (AD2).
- **K8s preStop hook** : a Kubernetes feature that runs a command in the container before SIGTERM ; gives 10 s extra for load balancers to deregister.

PR #7

Phase-2 threat model (execution plan)

Chapter 9 of 17 — source : `docs/EFE_A1_PHASE2_PR07_ARCHITECTURE.md` (944 LOC)

PR #7 -- Phase 2 Threat Model -- Architecture ultra-détaillée

***Document type** : Execution-ready implementation blueprint for PR #7. **Scope** : Third PR of Phase 2 (Sprint A1.P2.1, Week 1, day 5-7). **Owner** : sec-team -- assigned lead : R. Kapoor (back-up : E. Vargas). **Reviewers (required)** : 2x sec-team + 1x meta-swarm-team + 1x sre-team. **Estimate** : 2 FTE-days (drafting) + 0.5 FTE-day (review) + 0.5 FTE-day (sign-off) = 3 FTE-days. **Branch** : `A1.P2/07-phase2-threat-model`. **Deadline** : Sat 2026-07-11 23:59 UTC (parallel to PR #6 ; does not block PR #6 merge). **Hard dependency** : Phase 2 architecture frozen (`EFE_A1_PHASE2_ARCHITECTURE.md`). **Parent docs** : - `EFE_A1_PHASE2_ARCHITECTURE.md` § 15 (Security model) - `EFE_A1_PHASE2_ROADMAP.md` § 2.1 (PR #7 entry) - `EFE_OPERATIONAL_PRODUCTION_ARCHITECTURE.md` (workspace-wide security doctrine)*

Table of contents

1. TL;DR
2. Goal + non-goals
3. Pre-flight checks
4. Branch + commit plan
5. Methodology : STRIDE + CVSS + DREAD
6. Assets catalogue (10 assets)
7. Trust boundaries (6 boundaries)
8. Attacker personas (7 archetypes)
9. Threat scenarios (16 scenarios, 6 SEV-1)
10. Mitigations matrix (cross-referenced to PRs)
11. Residual risk register
12. Monitoring + alerting requirements
13. Incident response runbook hooks
14. Re-assessment cadence + triggers
15. Document structure : `docs/security/PHASE2_THREAT_MODEL.md`
16. PR-local risk analysis
17. Acceptance criteria (binary)
18. Reviewer checklist
19. PR description template
20. Definition of Done
21. Post-merge follow-ups

0. TL;DR

PR #7 ships **one document** : `docs/security/PHASE2_THREAT_MODEL.md` (~ 1 600 lines), which catalogues the Phase-2 attack surface, attackers, threats, mitigations, residual risk, and monitoring requirements. Methodology : **STRIDE per surface + CVSS v3.1 scoring + DREAD overlay**.

Why it matters :

- **L2 gate criterion G8** requires "threat model `PHASE2_THREAT_MODEL.md`

signed off by sec-team" before Phase 2 graduation. Without PR #7, the L2 gate is RED.

• **PR #16 (mTLS)** depends on PR #7's mitigation catalogue : it implements the controls.

• **runbook** `META_SWARM_GRPC.md` (PR #24 deliverable) inherits the incident-response hooks from PR #7.

• **L0 doctrine** (cf. `EFE_A1_ROADMAP.md` § 10) : "threat model has 0 SEV-1 unmitigated entries". PR #7 enforces this.

What it does NOT do :

• Does NOT implement any control (those live in code PRs).

• Does NOT replace the workspace-wide security doctrine in `EFE_OPERATIONAL_PRODUCTION_ARCHITECTURE.md` (it extends it for Phase 2 specifics).

• Does NOT cover Phase-3 multi-tenant + cross-region threats (deferred to Phase 3 RFC : `PHASE3_THREAT_MODEL.md`).

After merge :

• The doc is **the single source of truth** for "what attackers can do to Phase 2 and what we do about it".

• Every Phase 2 PR (#8 -> #25) references it in the PR description if it adds, removes, or affects a mitigation.

• The Sprint A1.P2.4 L2 verification ceremony (Mon 2026-08-24) checks every threat is either mitigated or accepted-with-rationale.

1. Goal + non-goals

1.1 Goal (single sentence)

> *Catalogue every threat that could compromise the confidentiality, integrity, or availability of the* `efe-meta-swarm` *Phase-2 distributed coordinator, score each via CVSS v3.1, map every threat to either a Phase-2 mitigation (with the implementing PR id) or an explicit risk acceptance (with rationale + expiry), so that the L2 graduation gate has a binary "0 SEV-1 unmitigated" signal.*

1.2 Non-goals

#	Excluded	Lives in	Why
NG1	Implementing any mitigation (mTLS, auth, NetworkPolicy, etc.)	code PRs (#16, #17, #18, etc.)	This PR is documentation only.
NG2	Phase-3 tenant-isolation threats	Phase 3 (`PHASE3_THREAT_MODEL.md`)	Scope-creep ; deferred.
NG3	Phase-1 in-process threats (already covered by Phase-1 review)	`EFE_A1_PHASE1_ARCHITECTURE.md`	Phase 1 has no network surface.
NG4	EFE-wide threats (efe-server, efe-core, etc.)	`EFE_OPERATIONAL_PRODUCTION_ARCHITECTURE.md`	Workspace-scope ; PR #7 only adds the **delta** for Phase 2.
NG5	Penetration testing or red-team exercises	Phase 3 (post-L3)	Phase 2 is internal-only ; pen-test is for L5 customer-facing.
NG6	Compliance frameworks (SOC2, ISO27001, GDPR)	Phase 5 (customer-facing)	Phase 2 internal soak does not invoke compliance.
NG7	Supply-chain attack catalogue (deps, build-time injection)	Continuous (cargo audit)	Workspace-wide concern, separate runbook.

1.3 Definition of mergeable

PR #7 is mergeable iff **all** :

1. The doc covers >= 12 distinct threat scenarios via STRIDE.
2. Every threat has a CVSS v3.1 score.
3. Every threat is in one of three states : `MITIGATED`, `ACCEPTED-WITH-RATIONALE`, or `DEFERRED-TO-PR-N`.
1. **Zero** SEV-1 (CVSS >= 9.0) is in `ACCEPTED` or `DEFERRED` state.
2. 2 sec-team peer reviewers + 1 meta-swarm + 1 sre have approved.
3. The L2 gate criterion G8 is unblocked.

2. Pre-flight checks

#	Check	How	Expected
PF1	Phase 2 architecture is frozen (last commit > 7 days old)	`git log --since=7days docs/EFE_A1_PHASE2_ARCHITECTURE.md`	empty
PF2	`docs/EFE_OPERATIONAL_PRODUCTION_ARCHITECTURE.md` is the source of workspace security doctrine	grep for "anti-Φ-D.A1" + "carve-out R5-B1"	present
PF3	sec-team has a STRIDE template in `docs/security/_template.md`	`ls docs/security/`	exists (or PR creates it)
PF4	CVSS v3.1 calculator URL is known	sec-team confirms	`https://www.first.org/cvss/calculator/3.1`
PF5	PR #5 (proto definitions) is merged so we can reference RPC names	`gh pr view 5 --json state`	`MERGED`
PF6	PR #6 (server scaffold) is in flight (parallel)	`gh pr view 6 --json state`	`OPEN` or `MERGED`
PF7	No outstanding Sev1 security incident on the workspace	`gh issue list -l security -l sev1 --state open`	empty
PF8	sec-team lead has 2 FTE-days allocated in Sprint A1.P2.1	sprint board	confirmed

3. Branch + commit plan

3.1 Branch

```
A1.P2/07-phase2-threat-model
```

3.2 Commit-by-commit plan (4 commits)

A documentation PR with logical chunks reviewable separately :

#	Commit message	Files	Doc %	Verifies
C1	`docs(security): scaffold PHASE2_THREAT_MODEL.md (TOC + methodology)`	`docs/security/PHASE2_THREAT_MODEL.md` (TOC + § 1-4)	15 %	structure approved by sec-team-lead
C2	`docs(security): assets, trust boundaries, attackers` (§ 5-7)	same file (§ 5-7)	35 %	sec-team confirms surface is fully mapped

#	Commit message	Files	Doc %	Verifies
C3	`docs(security): 16 threat scenarios with STRIDE + CVSS` (§ 8)	same file (§ 8)	75 %	core of the doc ; sec + meta-swarm reviewers walk through
C4	`docs(security): mitigations + residual risk + monitoring + final` (§ 9-13)	same file (§ 9-13)	100 %	doc complete + cross-refs to PRs

Total : ~ **1 600 lines** of markdown, single file.

3.3 Drafting strategy

Open as **Draft** after C1. Push C2-C4 incrementally. Move to "Ready for review" after C4 + a self-review pass. Required approvers : 2 sec-team (one acts as devil's advocate "red team" ; one focuses on the mitigation completeness), 1 meta-swarm-team (for technical accuracy), 1 sre-team (for monitoring + alerting).

4. Methodology

The threat model uses **STRIDE-per-surface** as the primary framework, augmented with **CVSS v3.1** scoring and **DREAD** for prioritisation.

4.1 STRIDE

For each trust-boundary crossing, we enumerate threats in 6 categories :

Letter	Category	Question
S	Spoofing	Can an attacker impersonate an identity ?
T	Tampering	Can an attacker modify data in transit / at rest ?
R	Repudiation	Can a legitimate action be denied later ?
I	Information disclosure	Can an attacker read data they shouldn't ?
D	Denial of service	Can an attacker make the service unavailable ?
E	Elevation of privilege	Can a low-priv attacker gain higher-priv access ?

4.2 CVSS v3.1

Every threat scenario is scored via CVSS v3.1 :

Severity	Score	Interpretation
Critical	9.0 - 10.0	SEV-1 -- must be MITIGATED for L2
High	7.0 - 8.9	SEV-2 -- must be MITIGATED or DEFERRED-WITH-PR
Medium	4.0 - 6.9	SEV-3 -- may be ACCEPTED with rationale + expiry
Low	0.1 - 3.9	SEV-4 -- typically ACCEPTED

The scoring uses the official FIRST calculator at `https://www.first.org/cvss/calculator/3.1`, and the **8 base metrics + 3 temporal + 4 environmental** are explicitly listed for every SEV-1 / SEV-2 threat (reviewer must be able to reproduce the score).

4.3 DREAD (overlay)

For threats with CVSS in [4.0, 7.0) where prioritisation matters, we apply DREAD (D, R, E, A, D -- each scored 1-10, summed) to break ties. This is optional ; CVSS is authoritative.

4.4 Sev definitions (workspace-wide)

The PR uses the workspace incident severity scale, defined in `docs/runbooks/ops_master_gate_fail.md` :

Sev	Customer impact	TM threshold
Sev1	Production down or data loss	CVSS >= 9.0
Sev2	Production degraded, single-tenant impact	CVSS 7.0 - 8.9
Sev3	Internal-only degradation	CVSS 4.0 - 6.9
Sev4	No customer impact	CVSS < 4.0

4.5 Threat status states

Every threat MUST be in exactly ONE state :

Status	Meaning	Allowed at L2 gate ?
`MITIGATED`	Implemented by a Phase-2 PR ; covered by a test or runbook.	[OK] always
`DEFERRED-TO-PR-N`	Will be mitigated by a specific Phase-2 PR ; PR id is named.	[OK] iff PR N is in Phase 2
`ACCEPTED`	Risk is explicitly accepted with rationale + expiry date + owner.	[OK] iff CVSS < 9.0
`OPEN`	No decision yet.	[FAIL] blocks L2 gate

The PR ships with **0 OPEN threats** at merge time.

5. Assets catalogue

Ten assets in scope. Each is named, located, owned, and rated for confidentiality / integrity / availability impact.

5.1 Asset list

ID	Asset	Located in	Owner	C	I	A
A1	**Meta-consensus state** (joint fixed point)	controller in-memory	meta-swarm-team	M	**H**	**H**
A2	**Participant states** (per-participant tensors)	participant in-memory	meta-swarm-team	**H**	**H**	M
A3	**Manifest** (participant ids, dims, kappas, seeds, safety geometry)	ConfigMap -> mounted file	platform + sec	M	**H**	M
A4	**mTLS certificates** (leaf + CA bundle)	K8s Secret + cert-manager	sec	**H**	**H**	M
A5	**Iteration determinism guarantee** (bit-identical reruns)	algorithmic invariant	meta-swarm-team	L	**H**	M
A6	**gRPC channel integrity** (between efe-server <-> controller <-> participants)	network	sec + platform	**H**	**H**	M
A7	**Health + Info responses** (build sha, hostname, configured participants)	controller in-memory	meta-swarm-team	M	M	L
A8	**Prometheus metrics + OTel spans** (operational telemetry)	controller/participant in-memory	sre	M	M	L
A9	**Admin RPCs surface** (ReloadConfig + future Reset)	controller in-memory	meta-swarm + sec	M	**H**	M
A10	**Soft-launch `deferred_ticket` field** (audit gate state)	YAML in repo	sre	L	**H**	L

C/I/A scale : **H** (high), M (medium), L (low).

5.2 Rationale per asset

(One paragraph per asset, justifying the CIA rating. Example for A1 :)

A1 -- Meta-consensus state :

• Confidentiality M : the joint fixed point may encode private

attributes if tenants opt into multi-tenant Phase-3. Phase-2 is internal-only, so a leak is operational rather than customer-facing.

• Integrity **H** : a corrupted consensus produces wrong downstream

decisions ; this is the entire point of the system.

• Availability **H** : without a consensus, the entire Iterate

pipeline returns errors ; whole-product impact.

(Full rationale for the other 9 assets follows the same shape.)

6. Trust boundaries

Six boundaries, each crossing identified + protected by listed controls.

6.1 Boundary list

```
TB1 : External (untrusted)           -----■ efe-server (ingress)
TB2 : efe-server                     -----■ controller (NEW Phase 2)
TB3 : controller                      -----■ participant (NEW)
TB4 : operator                        -----■ controller (SIGHUP)
TB5 : operator                        -----■ ConfigMap (manifest tampering)
TB6 : Prometheus scraper              -----■ /metrics (controller + participant)
```

6.2 Boundary detail

For each, the doc enumerates :

- the crossing endpoints (caller, callee)
- the wire transport (HTTP/2 gRPC, K8s API, etc.)
- the auth mechanism (mTLS, OAuth2, K8s RBAC)
- the threats applicable (which STRIDE letters)
- the controls in place

(Example for TB3, the most-frequent in PR #7's threat catalogue :)

TB3 -- controller <-> participant

Property	Value
Caller	controller pod (StatefulSet)
Callee	participant pod (Deployment)
Transport	gRPC over HTTP/2
Auth (Phase 2)	mTLS (PR #16) ; SAN check
Auth (PR #6 -> PR #15)	none (plaintext) ; mitigated by NetworkPolicy + Pod-to-Pod K8s native
STRIDE applicable	**S T I D** (no R or E by design)
Frequency	~ 14 100 Step RPCs/sec at 100 rps
Controls list	NetworkPolicy (PR #14), mTLS handshake (PR #16), Capabilities probe at startup (PR #9), deterministic Step replies (Phase 1 invariant)

7. Attacker personas

Seven archetypes ; each has a capability profile + motivation + likely entry vector.

ID	Persona	Capability	Motivation	Entry vector
AP1	**Curious insider**	Read code, deploy to staging	Curiosity, accidental	Internal access
AP2	**Malicious insider**	Same as AP1 + intent to harm	Disgruntled, exfil	Internal access
AP3	**Compromised efe-server**	Issues arbitrary gRPC calls	Upstream RCE	TB2
AP4	**Compromised participant pod**	Returns crafted Step responses	Container RCE, sidecar attack	TB3
AP5	**External attacker**	Network access via L7 ingress	Financial gain, espionage	TB1 (filtered through efe-server)
AP6	**Operator typo**	Misconfigures env vars, SIGHUPs wrong pod	Honest mistake	TB4, TB5

ID	Persona	Capability	Motivation	Entry vector
AP7	**Supply chain**	Injects malicious dep at build time	Long-game APT	Crates.io / build pipeline

The doc explicitly states Phase 2 is **internal-only** (no AP5 in production until L5), so AP5 is **out of scope for L2** ; the matrix below ignores AP5 for L2 gate purposes.

8. Threat scenarios

Sixteen scenarios catalogued. Each has the structure :

```
T-N -- short title
+-- STRIDE letters
+-- Surface (TB id)
+-- Persona(s) (AP id)
+-- Asset(s) at risk (A id)
+-- Description
+-- Attack chain (steps)
+-- CVSS v3.1 vector
+-- Score + severity
+-- Mitigations (PRs / runbooks)
+-- Status (MITIGATED / DEFERRED-TO-PR-N / ACCEPTED)
+-- Owner
```

8.1 The 16 threats (summary table)

ID	Title	STRIDE	TB	Persona	CVSS	Status
T1	Malicious participant biases consensus via crafted state	T,I	TB3	AP2, AP4	**8.6**	DEFERRED-TO-PR-16 + Phase-1 contractive bound
T2	MITM on controller <-> participant injects fake Step response	S,T	TB3	AP4	**9.1**	DEFERRED-TO-PR-16 (mTLS)
T3	Replay of Iterate response causes inconsistent downstream	T,R	TB2	AP3	6.8	MITIGATED (gRPC deadline + idempotence)
T4	SIGHUP injection by unauthorised operator triggers bad reload	S,E	TB4	AP6	5.4	DEFERRED-TO-PR-12 (auth scope) + K8s RBAC
T5	Manifest tampering via ConfigMap edit	T,E	TB5	AP2	**8.1**	MITIGATED (K8s RBAC + sealed-secrets seeds)
T6	mTLS cert rotation failure -> service down	D	TB2,TB3	AP6	**7.5**	DEFERRED-TO-PR-16 (cert-manager 60d renewal)
T7	Resource exhaustion : 10 000 Iterate calls/sec from compromised efe-server	D	TB2	AP3	6.5	MITIGATED (`EFE_META_SWARM_MAX_INFLIGHT`)
T8	Compromised efe-server sends Iterate with crafted dim mismatch	T,D	TB2	AP3	6.1	MITIGATED (server-side dim validation, PR #8)
T9	Timing side-channel reveals convergence trajectory	I	TB3	AP4	4.3	ACCEPTED (rationale + expiry 2027-06)
T10	Information disclosure via /metrics endpoint	I	TB6	AP1, AP5	5.3	MITIGATED (NetworkPolicy ; PR #14)
T11	Trace context spoofing via crafted W3C traceparent	T,R	TB2,TB3	AP3, AP4	4.8	MITIGATED (W3C standard validation + sampling cap)
T12	Determinism violation as covert channel	I,R	TB3	AP4	5.5	MITIGATED (SLO-A1-META-DETERMINISM + 3-rerun check)
T13	Stack overflow in proto parsing via deeply-nested oneof	T,D	TB2,TB3	AP3,AP4	**7.2**	MITIGATED (prost size limits + max-message-size=4MB)
T14	K8s NetworkPolicy bypass via misconfigured CNI	T,E	TB3	AP2	**8.7**	DEFERRED-TO-PR-14 (NetworkPolicy template + Helm test)

ID	Title	STRIDE	TB	Persona	CVSS	Status
T15	Cert SAN spoofing via wildcard mismatch	S,E	TB2,TB3	AP4	**8.2**	DEFERRED-TO-PR-16 (strict SAN allowlist)
T16	Operator privilege escalation via admin RPC (ReloadConfig)	E	TB4	AP6, AP2	**7.8**	DEFERRED-TO-PR-12 (admin scope) + K8s RBAC

Counts : 16 total ; **6 SEV-1 (CVSS >= 7.0)** ; **9 SEV-2/3** ; **1 ACCEPTED**. All 6 SEV-1 are either MITIGATED or DEFERRED-TO-PR ; **zero in** `OPEN` **or** `ACCEPTED` **state** (G8 gate satisfied).

8.2 Threat detail -- T1 (example of full entry)

T1 -- Malicious participant biases consensus via crafted state STRIDE *: Tampering (T), Information disclosure (I).* ***Surface*** *: TB3 (controller <-> participant).* ***Personas*** *: AP2 (malicious insider), AP4 (compromised participant pod).* ***Assets at risk*** *: A1 (meta-consensus state), A5 (determinism guarantee).* ***Description*** *: A participant whose process has been compromised -- by an RCE, sidecar attack, or insider -- can return arbitrary* `new_state` *values in its* `Step` *responses. An attacker who controls the* `pulled_from_meta -> new_state` *mapping can : 1. Bias the meta-consensus toward an attacker-chosen direction. 2. Cause divergence (residual oscillates) -- DoS. 3. Exfiltrate via a covert channel encoded in the LSB of returned states (information disclosure).* ***Attack chain*** *: 1. Attacker gains code execution inside a participant pod (T13 or AP4 entry vector). 2. Attacker modifies the* `MetaParticipantService::Step` *handler to return* `pulled_from_meta + ATTACKER_BIAS` *instead of the contractive map. 3. Controller's* `iterate()` *runs the meta-operator with this biased input. 4. Meta-consensus converges (or diverges) under attacker control.* ***CVSS v3.1 vector*** *:* `AV:A/AC:H/PR:H/UI:N/S:U/C:H/I:H/A:L/E:F/RL:O/RC:C` *Base = 6.9, Temporal = 6.3,* ***adjusted = 8.6*** *(raised because the consensus impact propagates to all downstream decisions).* ***Severity*** *:* ***SEV-1****.* ***Mitigations (layered defence)*** *: 1.* ***Phase-1 contractive bound*** *: the algorithm itself bounds attacker bias to* `(1 - κ_max) x max_step_size`*. Attacker cannot cause* ***unbounded*** *consensus drift -- the meta-operator's contractivity is an algorithmic invariant. 2.* ***Safety projection*** *(PR #1) : the meta-consensus is projected onto a safety set (BoxSafety in Phase 2) at every meta-step. An attacker who pushes outside the box is clipped. 3.* ***Determinism check*** *(T12 + SLO-A1-META-DETERMINISM) : a biased participant would break the 3-rerun bit-identical guarantee on every Iterate. Alert fires at SLO breach. 4.* ***mTLS + SAN check*** *(PR #16) : an external attacker cannot spoof a participant. Only AP4 (in-cluster RCE) remains feasible. 5.* ***Phase-3 future control*** *: cryptographic attestation of step outputs (deferred to Phase 3 ; tracked as* `DT-A2-ATTESTATION`*).* ***Status*** *:* ***DEFERRED-TO-PR-16*** *(for mTLS) + already partially mitigated by Phase-1 contractivity. At L2, mTLS reduces the attacker pool to AP4 (in-cluster RCE) and the contractive bound limits the impact. Residual risk : moderate -- accepted for Phase 2 ; full cryptographic attestation in Phase 3.* ***Owner*** *: meta-swarm-team (algorithmic bound) + sec (mTLS + future attestation).* ***Detection*** *: - SLO-A1-META-DETERMINISM breach -> alert -* `meta_swarm_consensus_norm` *outside expected range -> alert (PR #17 + PR #19 dashboard)* ***Test*** *: the Phase-2 multi-process integration test (PR #10) can be extended with a "rogue participant" fixture that returns biased states ; expected outcome : the iteration completes within bounds dictated by contractivity (no unbounded drift).*

(The other 15 threats follow the same detailed shape. Full content in `docs/security/PHASE2_THREAT_MODEL.md`.)

9. Mitigations matrix

Cross-reference : each mitigation -> which threats it covers -> which PR implements it.

Mitigation	Implementing PR	Threats covered
mTLS handshake controller <-> participant + efe-server <-> controller	**PR #16**	T1, T2, T15
SAN-based authorization	PR #16	T2, T15
K8s NetworkPolicy (Helm chart)	**PR #14**	T10, T14
ConfigMap RBAC + sealed-secrets for seeds	**PR #13**	T5
cert-manager 60-day auto-renewal	PR #16	T6

Mitigation	Implementing PR	Threats covered
`EFE_META_SWARM_MAX_INFLIGHT` enforcement	PR #6 (config) + PR #8 (runtime)	T7
Server-side dim validation	**PR #8**	T8
Prost max-message-size = 4 MB	PR #5 + PR #8	T13
SIGHUP admin-scope auth	**PR #12**	T4, T16
W3C traceparent validation	PR #18	T11
Determinism SLO + 3-rerun check	Phase 1 (PR #4) + PR #10	T12
gRPC deadline + idempotence	PR #6 + PR #8	T3
Phase-1 contractivity (algorithmic)	Phase 1 (PR #1)	T1
BoxSafety projection (algorithmic)	Phase 1 (PR #1)	T1

9.1 PRs that ship a security control

PR	Sec-controls delivered	Sec-team review required ?
PR #5 (proto)	Max-message-size in build.rs	YES -- already done
PR #6 (server scaffold)	MAX_INFLIGHT config field	NO -- config-only
PR #8 (Iterate body)	Dim validation, MAX_INFLIGHT enforcement	YES
PR #12 (SIGHUP)	Admin-scope check	YES
PR #13 (Helm)	ConfigMap RBAC, sealed-secrets	YES
PR #14 (Helm full)	NetworkPolicy templates	YES
PR #16 (mTLS)	mTLS handshake, SAN check, cert-manager	YES -- full sec-team review
PR #18 (OTel)	Traceparent validation, sampling cap	YES

10. Residual risk register

The threats explicitly **ACCEPTED** in Phase 2, with rationale + expiry + revisit trigger.

ID	Threat	CVSS	Rationale	Expiry	Revisit trigger
T9	Timing side-channel reveals convergence trajectory	4.3	Internal-only Phase 2 ; tenant isolation is Phase 3 ; cost of constant-time iteration is prohibitive for ~ 100 rps target.	2027-06-30 (Phase 5 start)	Customer signs requiring constant-time guarantees

Only 1 ACCEPTED risk in Phase 2. All other CVSS-low risks (SEV-4) are MITIGATED by default workspace controls (TLS, RBAC, audit log).

11. Monitoring + alerting

Phase 2 ships 4 alert rules (PR #19) ; this section maps each alert to its threats + on-call runbook.

Alert	Threshold	Threats
`MetaSwarmHighFailureRate`	> 2 % failure / 5 min	T1, T7, T8
`MetaSwarmParticipantDown`	participant_up == 0 / 1 min	T1 (suspicious), T6
`MetaSwarmReloadFail`	reload_fail > 0 / 10 min	T4, T16
`MetaSwarmLatencyP99`	p99 > 100 ms	T7, T13

Plus 4 additional alerts proposed by PR #7 :

Alert	Threshold	Threats	PR
`MetaSwarmAuthFailures`	auth_failures > 10 / min	T2, T15, T16	PR #18

Alert	Threshold	Threats	PR
`MetaSwarmCertExpiringSoon`	cert_days_remaining < 7	T6	PR #16
`MetaSwarmDeterminismBreach`	determinism != 1	T1, T12	PR #17
`MetaSwarmAdminRpcSpike`	admin_rpc_total rate > 1/min	T4, T16	PR #12

The 8 alerts route to existing Alertmanager -> PagerDuty.

12. Incident response

Each SEV-1 threat has a **runbook hook** in `docs/runbooks/` :

Threat	Runbook	New / Update
T1, T12	`runbooks/META_SWARM_DETERMINISM_BREACH.md` (NEW in PR #24)	NEW
T2	`runbooks/META_SWARM_TLS_FAILURE.md` (NEW in PR #16)	NEW
T5	`runbooks/META_SWARM_MANIFEST_TAMPERED.md` (NEW in PR #24)	NEW
T6	`runbooks/META_SWARM_CERT_EXPIRY.md` (NEW in PR #16)	NEW
T14	`runbooks/META_SWARM_NETPOL_BYPASS.md` (NEW in PR #24)	NEW
T15	covered by T2 runbook	--

Each runbook follows the workspace template (`docs/runbooks/_template.md`) : Symptoms -> Triage -> Mitigation -> Post-mortem.

13. Re-assessment cadence

The threat model is a **living document** :

Trigger	Action	Owner
New PR adds a network surface	Update § 6 (trust boundaries) + § 8 (threats)	PR author + sec-team
New PR removes a mitigation	Revoke `MITIGATED` status ; reassess CVSS	PR author + sec-team
Phase gate (L2, L3, L4, L5)	Full re-walk of all threats	sec-team lead
Sev1 incident on Phase 2 surface	Same-week patch + post-mortem amends model	sre + sec
New attacker persona observed	Add to § 7 + matrix	sec
Periodic (every 6 months)	Re-walk in a quarterly sec-team review	sec-team lead

The doc carries a **"Last reassessed"** stamp at the top. CI warns if the stamp is > 7 months old.

14. Document structure : `docs/security/PHASE2_THREAT_MODEL.md`

The output document has 14 sections, mapping 1:1 to this architecture's § 4-13 + meta sections.

14.1 Skeleton

```
# Sprint A1 Phase 2 -- Threat Model

&gt; Status     : DRAFT until L2 sign-off
&gt; Frozen at  : YYYY-MM-DD
&gt; Last review: YYYY-MM-DD
&gt; Owner      : sec-team (lead R. Kapoor)
&gt; Reviewers  : &lt;signatures&gt;
&gt; Methodology: STRIDE per surface, CVSS v3.1 scoring
&gt; Out of scope: Phase-3 multi-tenant (separate doc)

## Table of contents

0. Executive summary
1. Methodology (STRIDE + CVSS + DREAD + sev mapping)
2. Assets catalogue (10 assets with CIA ratings)
3. Trust boundaries (6 boundaries)
4. Attacker personas (7 archetypes)
5. Threat scenarios (T1 - T16) -- full STRIDE walk
6. Mitigations matrix (cross-ref to PRs)
7. Residual risk register (1 ACCEPTED)
8. Monitoring + alerting requirements (8 alerts)
9. Incident response hooks (5 runbooks)
10. Re-assessment cadence
11. Glossary
12. Appendix A -- CVSS scoring details (per SEV-1/2)
13. Appendix B -- Cross-references to other security docs
```

14.2 Length budget

Section	Lines
Executive summary	50
Methodology	100
Assets catalogue	150 (10 assets x ~15 lines each)
Trust boundaries	120 (6 boundaries x ~20 lines each)
Attacker personas	80
Threat scenarios (16 x ~50 lines)	800
Mitigations matrix	80
Residual risk register	40
Monitoring + alerting	80
Incident response	50
Re-assessment cadence	30
Glossary	30
Appendices	100
TOC + frontmatter	30
Total	**~ 1 740**

14.3 Markdown conventions

Convention	Why
YAML frontmatter at the top (`status`, `frozen_at`, `last_review`)	Machine-parseable by CI staleness check
Every threat in a 3-level heading `### T-N -- title`	Linkable from PR descriptions + runbooks
CVSS vectors in `code blocks`	Reproducible by reviewers
Status as **bold**	Visual scanning
Cross-references as `link text`	Single-file navigation

15. PR-local risk analysis

ID	Risk	L/I	Mitigation
PR7-R1	Threat model misses an attack class (e.g. supply chain)	M/H	sec-team second-pair review ; explicit NG7 declaration ; future addendum if new class found
PR7-R2	CVSS scoring inconsistency between reviewers	M/M	Mandatory use of FIRST calculator + vectors in doc ; sec-team lead arbitrates
PR7-R3	A SEV-1 threat is marked `ACCEPTED` to unblock L2	L/H	Doctrine § 4.5 : "Zero SEV-1 in ACCEPTED" hard-enforced ; CI grep
PR7-R4	Mitigations table drifts from PR implementations	M/M	Every Phase-2 PR description must reference the threat IDs it mitigates ; reviewer checklist enforces
PR7-R5	Doc becomes stale after merge	M/M	"Last reassessed" stamp + CI warning > 7 months ; quarterly review
PR7-R6	sec-team reviewers under-prioritise -> PR blocks Phase 2	M/H	2 FTE-days reservation in sprint board ; eng-leadership escalation if > 5 d slip
PR7-R7	Phase-1 threats accidentally re-listed (out-of-scope)	L/L	Doctrine § 1.2 : NG3 declares Phase 1 out of scope ; reviewer checklist
PR7-R8	Doc cross-references to PRs that don't exist yet	M/L	PR ids referenced ARE in Phase-2 roadmap ; non-existence is a feature ("DEFERRED-TO-PR-N" is normal during scaffold week)

16. Acceptance criteria (binary)

PR #7 is mergeable iff **all** :

#	Criterion	Verification
AC1	`docs/security/PHASE2_THREAT_MODEL.md` exists	git diff
AC2	Doc has >= 12 distinct threat scenarios	reviewer count
AC3	Every threat has a STRIDE classification	grep for "STRIDE" header
AC4	Every threat has a CVSS v3.1 vector	grep for "CVSS:3.1/"
AC5	Every threat has a status (`MITIGATED`, `DEFERRED-TO-PR-N`, `ACCEPTED`)	grep
AC6	**Zero** SEV-1 (CVSS >= 9.0) is in `ACCEPTED` or `OPEN` state	sec-team manual review
AC7	Every `DEFERRED-TO-PR-N` references a PR id present in the Phase-2 roadmap	cross-grep
AC8	Every `ACCEPTED` has rationale + expiry date + owner	reviewer count
AC9	Mitigations matrix (§ 9) lists every threat	sec-team review
AC10	8 alert rules specified in § 11	reviewer count
AC11	5 runbook hooks specified in § 12	reviewer count
AC12	Re-assessment cadence + 6 triggers documented (§ 13)	reviewer
AC13	"Last reassessed" stamp <= 1 day old at merge	grep
AC14	Frontmatter has `status`, `frozen_at`, `owner`, `reviewers` fields	grep

#	Criterion	Verification
AC15	All cross-refs to `EFE_A1_PHASE2_*` docs are valid (file exists + section anchor exists)	scripted check
AC16	sec-team peer-review tag from 2 different sec-team members	GitHub
AC17	1 meta-swarm-team approval (technical accuracy)	GitHub
AC18	1 sre-team approval (monitoring + runbook alignment)	GitHub
AC19	L2 gate criterion G8 is satisfiable	sec-team lead sign-off comment
AC20	No file outside `docs/security/` is modified	git diff --stat

17. Reviewer checklist

General

- [ ] PR title : `docs(security): Phase 2 threat model (PR #7)`
- [ ] PR description follows § 18 template.
- [ ] 4 commits, each individually pushed + CI-green (markdown lint).
- [ ] No file outside `docs/security/PHASE2_THREAT_MODEL.md` touched.

Methodology

- [ ] STRIDE used consistently for every threat.
- [ ] CVSS v3.1 vector + computed score for every SEV-1/2.
- [ ] DREAD applied only for tie-breaking in [4.0, 7.0).
- [ ] Sev mapping table present (Sev1-4 <-> CVSS bands).

Assets

- [ ] 10 assets enumerated (A1-A10).
- [ ] Each has C/I/A rating + 1-paragraph rationale.

Trust boundaries

- [ ] 6 boundaries listed.
- [ ] Each has caller, callee, transport, auth, STRIDE applicable.

Attackers

- [ ] 7 personas (AP1-AP7) with capability + motivation + entry vector.
- [ ] AP5 (external) explicitly out of scope for L2.

Threats

- [ ] >= 12 scenarios (PR #7 targets 16).
- [ ] Each has the 9-field structure (STRIDE / TB / Persona / Asset / Description / Chain / CVSS / Mitigations / Status).
- [ ] **Zero** SEV-1 (CVSS >= 9.0) in `ACCEPTED` or `OPEN`.
- [ ] Every `DEFERRED-TO-PR-N` references a Phase-2 PR id.
- [ ] Every `ACCEPTED` has expiry + revisit trigger.

Mitigations + alerts + runbooks

- [ ] Mitigations matrix maps every threat to >= 1 control.
- [ ] 8 alert rules listed with thresholds + threat coverage.
- [ ] 5 runbook hooks named (NEW in PR #24 or owning PR).

Doc craft

- [ ] Frontmatter present.
- [ ] TOC matches sections.
- [ ] CVSS vectors in code blocks.
- [ ] "Last reassessed" stamp current.
- [ ] All internal anchors resolve.
- [ ] All cross-refs to other workspace docs are valid.

Sec-team-specific

- [ ] First sec-reviewer acts as "red team" (challenges assumptions).
- [ ] Second sec-reviewer focuses on mitigation completeness.
- [ ] Sec-team-lead signs off on the matrix completeness.
- [ ] G8 (L2 gate criterion) sign-off comment posted.

Sign-off

- [ ] 2x sec-team approvals.
- [ ] 1x meta-swarm-team approval.
- [ ] 1x sre-team approval.

18. PR description template

```
# [Sprint A1 Phase 2 -- PR #7] docs(security): Phase 2 threat model

## Summary

Third PR of Phase 2 (parallel to PR #6).  Lands the threat model
document `docs/security/PHASE2_THREAT_MODEL.md`.

The doc catalogues **16 distinct threats** against the Phase-2
distributed coordinator surface, scores each via **CVSS v3.1**, and
maps every threat to either a mitigation (with implementing PR id)
or an explicit risk acceptance.

This PR is a deliverable of the L2 graduation gate (criterion G8 :
"threat model signed off by sec-team").

Full architecture : [`docs/EFE_A1_PHASE2_PR07_ARCHITECTURE.md`](docs/EFE_A1_PHASE2_PR07_ARCHITECTURE.md).

## What this PR adds

| Section | Content |
|---|---|
| § 0 | Executive summary |
| § 1 | Methodology : STRIDE + CVSS v3.1 + DREAD overlay |
| § 2 | 10 assets catalogue (A1-A10) with CIA ratings |
| § 3 | 6 trust boundaries (TB1-TB6) |
| § 4 | 7 attacker personas (AP1-AP7) |
| § 5 | 16 threat scenarios (T1-T16) with full STRIDE walks |
| § 6 | Mitigations matrix : threats x implementing PRs |
| § 7 | Residual risk register : 1 ACCEPTED with expiry |
| § 8 | 8 alert rules with threat coverage |
| § 9 | 5 runbook hooks (3 NEW in PR #24, 2 NEW in PR #16) |
| § 10 | Re-assessment cadence : 6 triggers + quarterly review |

## Threat statistics
```

Total threats : 16 Severity breakdown : SEV-1 (CVSS >= 9.0) : 1 (T2 : MITM) SEV-2 (7.0-8.9) : 6 (T1, T5, T6, T13, T14, T15, T16) SEV-3 (4.0-6.9) : 8 SEV-4 (< 4.0) : 1

Status breakdown : MITIGATED : 8 DEFERRED-TO-PR : 7 (PR #12, #14, #16 -- all Phase 2) ACCEPTED : 1 (T9 timing side-channel, expiry 2027-06-30) OPEN : 0 <- G8 satisfied ```

Test plan

- [x] Markdown renders correctly on GitHub.
- [x] All internal anchors resolve.
- [x] All cross-refs to other workspace docs resolve.
- [x] CVSS vectors reproduce in FIRST calculator (sampled 3 of the 7 SEV-1/2).
- [x] Mitigations matrix is bijective : every threat appears.
- [x] 0 SEV-1 in ACCEPTED or OPEN (grep + manual review).

Non-regression statement

- [OK] Zero file modified outside `docs/security/`.
- [OK] No code or proto changes.
- [OK] No PR #5/PR #6 dependencies broken.

Honest semantics

- 7 threats are `DEFERRED-TO-PR` -- they are NOT mitigated at PR #7

merge time. Each names the Phase-2 PR that ships the mitigation. L2 gate cannot pass until all 7 deferred PRs are merged.

- 1 threat is `ACCEPTED` -- T9 (timing side-channel). Rationale +

expiry documented.

Reviewer guidance

Suggested order :

1. Read `docs/EFE_A1_PHASE2_PR07_ARCHITECTURE.md` for context.
2. Read § 1 (methodology) -- confirm STRIDE + CVSS approach.
3. Walk § 5 threats T1 -> T16 in order. For each, ask :

- Is the STRIDE classification correct ?
- Is the CVSS vector defensible ?
- Is the mitigation complete ?

1. Verify mitigations matrix (§ 6) is bijective.
2. Verify monitoring (§ 8) covers SEV-1 threats.

Required approvers : 2x sec-team + 1x meta-swarm + 1x sre.

References

- `docs/EFE_A1_PHASE2_ARCHITECTURE.md` § 15 (security model)
- `docs/EFE_OPERATIONAL_PRODUCTION_ARCHITECTURE.md` (workspace doctrine)
- `docs/EFE_A1_PHASE2_PR05_ARCHITECTURE.md`, PR06, PR07

- FIRST CVSS calculator : `https://www.first.org/cvss/calculator/3.1`

```
---

## 19. Definition of Done

PR #7 is done iff :

1. All 20 acceptance criteria GREEN.
2. All 4 reviewer approvals in (2 sec + 1 meta-swarm + 1 sre).
3. sec-team-lead sign-off comment posted : `G8 satisfied for L2`.
4. Merged to `main` with merge-commit linking to this doc.
5. Slack post in `#meta-swarm-standup` + `#sre` : "PR #7 merged ;
   L2 G8 unblocked.  Mitigations matrix references PRs #12, #14,
   #16 ; their authors are now responsible for closing the
   DEFERRED-TO-PR-N items."
6. The Sprint A1.P2.4 L2 verification ceremony references the
   merged doc as the source of truth for G8.

PR #7 is NOT done if :

- Any AC is RED.
- Any reviewer has a blocking comment unresolved.
- Doc contains an `OPEN` threat status.
- A SEV-1 (CVSS &gt;= 9.0) is in `ACCEPTED` state.
- A `DEFERRED-TO-PR-N` references a PR id NOT in Phase 2.

---

## 20. Post-merge follow-ups

These unlock once PR #7 lands :

| Follow-up | PR | Owner | When |
|---|---|---|---|
| PR #12 author cross-refs SIGHUP threats (T4, T16) in PR description | PR #12 | meta-swarm | Sprint 2 W3 ■
     |
| PR #13 author cross-refs ConfigMap RBAC (T5) | PR #13 | platform | Sprint 2 W3 |
| PR #14 author cross-refs NetworkPolicy (T10, T14) | PR #14 | platform | Sprint 2 W4 |
| PR #16 author cross-refs mTLS (T1, T2, T6, T15) | PR #16 | sec + platform | Sprint 2 W4 |
| PR #17 cross-refs Prometheus auth-failure alert (T2, T16) | PR #17 | meta-swarm | Sprint 3 W5 |
| PR #18 cross-refs traceparent validation (T11) | PR #18 | meta-swarm | Sprint 3 W5 |
| PR #19 ships 8 alert rules from § 11 | PR #19 | sre | Sprint 3 W5 |
| PR #24 ships 5 runbooks from § 12 | PR #24 | sre | Sprint 4 W7 |

Every one of these PRs MUST add a line in its description :
```

Threats addressed (from PHASE2_THREAT_MODEL.md) : T-N, T-M, ... ```

A CI lint check (added in PR #24) verifies this for every Phase-2 PR.

Appendix A -- Sample CVSS scoring (for reviewer verification)

A.1 T2 -- MITM on controller <-> participant

Metric	Value	Rationale
Attack vector (AV)	Adjacent network (A)	Requires same K8s pod network namespace
Attack complexity (AC)	High (H)	Requires positioning between pods
Privileges required (PR)	None (N)	No K8s RBAC needed once on the network
User interaction (UI)	None (N)	Fully automated
Scope (S)	Changed (C)	Compromises both controller + participant
Confidentiality (C)	High (H)	Reads all Step messages
Integrity (I)	High (H)	Modifies Step messages
Availability (A)	High (H)	Can drop / delay -> DoS

CVSS v3.1 vector : `AV:A/AC:H/PR:N/UI:N/S:C/C:H/I:H/A:H` -> base = **9.1** -> SEV-1.

Temporal adjustments :

- Exploit code maturity (E) : Functional (F) -- MITM tools exist generally
- Remediation level (RL) : Official fix (O) -- mTLS is standard

- Report confidence (RC) : Confirmed (C)

Temporal = **8.7**. Listed as **MITIGATED-BY-PR-16**.

A.2 (Similar tables for T1, T5, T6, T13, T14, T15, T16 -- the 7 SEV-1/2 threats.)

Appendix B -- Cross-references

- Phase 2 architecture security model : `EFE_A1_PHASE2_ARCHITECTURE.md` § 15
- Operational doctrine : `EFE_OPERATIONAL_PRODUCTION_ARCHITECTURE.md`
- L2 gate criteria : `EFE_A1_PHASE2_ARCHITECTURE.md` § 12 (G8)
- Roadmap PR #7 entry : `EFE_A1_PHASE2_ROADMAP.md` § 2.1
- Sister architectures : PR05, PR06, PR08 (Iterate body -- references this doc for T1/T8/T13)
- Master gate fail runbook : `runbooks/ops_master_gate_fail.md`

PR #8

MetaSwarmService::Iterate wiring

Chapter 10 of 17 — source : `docs/EFE_A1_PHASE2_PR08_ARCHITECTURE.md` (1,968 LOC)

PR #8 -- `MetaSwarmService::Iterate` wiring -- Architecture ultra-détaillée

***Document type** : Execution-ready implementation blueprint for PR #8. **Scope** : 4th PR of Phase 2 (Sprint A1.P2.1, Week 2, day 1-5). **Owner** : meta-swarm-team -- assigned lead : M. Okafor (back-up : J. Tan). **Reviewers (required)** : 2x meta-swarm-team + 1x sec-team + 1x sre-team. **Estimate** : 5 FTE-days (engineering) + 1 FTE-day (review) = 6 FTE-days. **Branch** : `A1.P2/08-iterate-wire`. **Deadline** : Sun 2026-07-19 23:59 UTC (last PR of Sprint A1.P2.1). **Hard dependencies** : - PR #5 (proto crate) merged - PR #6 (server scaffold) merged **Parent docs** : - `EFE_A1_PHASE2_ARCHITECTURE.md` § 5 (Rust API <-> gRPC mapping) - `EFE_A1_PHASE2_ROADMAP.md` § 2.2 (PR #8 entry) - `EFE_A1_PHASE2_PR06_ARCHITECTURE.md` (server scaffold this PR wires) - `docs/security/PHASE2_THREAT_MODEL.md` (threats T1, T7, T8, T13, T19 mitigated here)*

Table of contents

1. TL;DR
2. Goal + non-goals
3. Pre-flight checks
4. Branch + commit plan
5. Architectural decisions
6. The sync <-> async bridge problem
7. Wire diagram : RPC -> MetaSwarm -> participants
8. Crate layout additions
9. `client/remote_participant.rs` -- full design
10. `controller/iterate.rs` -- orchestration
11. `controller/inflight.rs` -- MAX_INFLIGHT enforcement
12. `controller/state.rs` -- inner `MetaSwarm` shared state
13. `controller/service.rs` -- replace stubs with real bodies
14. `manifest/mod.rs` -- extended validation
15. `Cargo.toml` updates
16. Determinism, cancellation, timeout semantics
17. Error mapping : Phase-1 `MetaSwarmError` -> gRPC `Status`
18. Concurrency model (`MAX_INFLIGHT` + `RwLock`)
19. Threat-model coverage : T1, T7, T8, T13, T19
20. Tests strategy
21. Acceptance criteria
22. PR description template
23. Risk analysis
24. Reviewer checklist
25. Definition of Done
26. Post-merge follow-ups

0. TL;DR

PR #8 replaces the `Err(Status::unimplemented(...))` stubs in `controller/service.rs` for `Iterate`, `StepOnce`, and `Reset` with real bodies that :

1. **Validate** the request against the loaded manifest (dim, kappa,

determinism).

1. **Acquire** an inflight slot (`MAX_INFLIGHT` semaphore).

2. **Build** a `MetaSwarm` (Phase-1 type) from the manifest + the

configured participants -- but each participant is a `RemoteParticipant` (gRPC client wrapper), NOT a `MockParticipant`.

1. **Run** `MetaSwarm::iterate()` (Phase-1 sync algorithm) inside

`tokio::task::spawn_blocking`, with cancellation propagation.

1. **Convert** the `IterateResult` (Phase-1 type) into the proto

`IterateResponse`.

1. **Release** the inflight slot + log + emit metrics.

What it adds :

• 1 new module `client/remote_participant.rs` (~ 250 LOC) -- a gRPC

client adapter implementing the Phase-1 `SwarmParticipant` trait.

• 1 new module `controller/iterate.rs` (~ 400 LOC) -- the orchestration

glue between the gRPC handler and the Phase-1 algorithm.

• 1 new module `controller/inflight.rs` (~ 80 LOC) -- a `Semaphore`

wrapper that enforces `MAX_INFLIGHT`.

• Modified `controller/service.rs` (~ 200 LOC, replaces stubs).

• Modified `controller/state.rs` (~ 50 LOC, adds inner `MetaSwarm` Arc).

• Modified `manifest/mod.rs` (~ 100 LOC, extends with validation).

• Modified `Cargo.toml` (+1 dep : `efe-meta-swarm` Phase-1 crate).

What it does NOT add :

• No mTLS -- plaintext channels (PR #16 wires mTLS).

• No metrics emission -- PR #17 wires the 14 metrics.

• No OTel traces -- PR #18 wires the 8 span types.

• No `ReloadConfig` -- PR #12 wires SIGHUP + manifest reload.

• No `IterateStream` -- PR #21 wires server-streaming for bench.

• No multi-process integration test -- PR #10 owns that.

Critical-path significance : PR #8 is the FIRST PR where the server **actually does something** distributed. Until PR #8, the server is a stubbed scaffold. PR #9 (Step on participant side) lands in parallel ; together they enable the multi-process integration test of PR #10.

1. Goal + non-goals

1.1 Goal (single sentence)

Wire the controller's `MetaSwarmService::Iterate` *(and* `StepOnce`, `Reset`*) to invoke the Phase-1* `MetaSwarm::iterate()` *algorithm with participants connected over gRPC, preserving the Phase-1 determinism guarantee, the contractive math, and the safety projection, while adding Phase-2 specific controls (manifest dim validation,* `MAX_INFLIGHT` *backpressure, cancellation, 30 s server cap on* `max_wall_ms`*).*

1.2 Non-goals

#	Excluded	Lives in	Why
NG1	`Step` body on participant side	**PR #9**	Symmetric work ; parallel sprint slot
NG2	mTLS handshake	PR #16	Sec PR ; comes later in sprint 2
NG3	Prometheus metrics emission	PR #17	Observability sprint
NG4	OTel span emission	PR #18	Same
NG5	`ReloadConfig` body	PR #12	SIGHUP sprint
NG6	`IterateStream` (server streaming)	PR #21	Bench-only path
NG7	Multi-process integration test	PR #10	Needs PR #8 + #9 first
NG8	NetworkPolicy / Helm	PRs #13, #14	Platform sprint
NG9	f64 wire precision opt-in	Phase 3	OQ1 = f32-only at Phase 2
NG10	Cryptographic attestation of step outputs	Phase 3	`DT-A2-ATTESTATION`

1.3 Single binary unit of merge-readiness

PR #8 is mergeable iff **all** :

1. `Iterate(IterateRequest)` runs end-to-end in a unit test using a

`MockRemoteParticipant` (in-process fake gRPC).

1. The returned `IterateResponse` has bit-identical

`(iterations, final_residual)` to the Phase-1 example `examples/three_swarms_kovant.rs`.

1. `StepOnce` and `Reset` are wired (same pattern).
2. Cancellation (client drop) propagates to participants within

100 ms.

1. `MAX_INFLIGHT` enforcement : the 11th concurrent call returns

`RESOURCE_EXHAUSTED`.

1. Dim mismatch in `IterateRequest` returns `FAILED_PRECONDITION`.
2. All Phase 1 + PR #5 + PR #6 tests still pass.

2. Pre-flight checks

#	Check	How	Expected
PF1	PR #5 merged	`gh pr view 5 --json state`	`MERGED`
PF2	PR #6 merged	`gh pr view 6 --json state`	`MERGED`
PF3	PR #6 smoke test passes on main	CI	green
PF4	Phase-1 `MetaSwarm::iterate()` API stable	grep `pub fn iterate` in `crates/efe-meta-swarm/src/meta_swarm.rs`	unchanged since Phase-1 GA
PF5	Phase-1 `examples/three_swarms_kovant.rs` reproducible (47 iters, residual 9.7e-5)	`cargo run --example three_swarms_kovant`	as documented
PF6	OQ1 decided : f32-only on the wire	`docs/A1_PHASE2_DECISIONS.md`	f32
PF7	OQ2 decided : `StepStream` optional	id.	optional
PF8	Threat model PR #7 ships : we cross-ref T1, T7, T8, T13, T19	`docs/security/PHASE2_THREAT_MODEL.md`	exists
PF9	tokio-util crate available offline (for `CancellationToken`)	`ls ~/.cargo/registry/cache//*tokio-util*.crate`	present

3. Branch + commit plan

3.1 Branch

```
A1.P2/08-iterate-wire
```

3.2 Commit-by-commit plan (5 commits, each CI-green)

#	Commit	Files	LOC ±	Verifies
C1	`chore: add efe-meta-swarm dep + InflightSemaphore module`	`crates/efe-meta-swarm-server/Cargo.toml`, `+ src/controller/inflight.rs`	+100	dep resolves ; inflight unit test passes
C2	`feat(server): RemoteParticipant client (impls SwarmParticipant)`	`+ src/client/mod.rs`, `+ src/client/remote_participant.rs`	+320	unit test : mock channel, RemoteParticipant returns expected Tensor
C3	`feat(server): manifest validation + tests`	`src/manifest/mod.rs`, `src/manifest/schema.rs`	+180	unit tests : valid manifest accepted, invalid rejected (dim/kappa/seed checks)
C4	`feat(server): controller/iterate.rs orchestration + state.rs MetaSwarm Arc`	`+ src/controller/iterate.rs`, `src/controller/state.rs`	+500	orchestration unit tests with MockRemoteParticipant
C5	`feat(server): wire Iterate, StepOnce, Reset in service.rs + smoke`	`src/controller/service.rs`, `+ tests/integration_iterate_inproc.rs`	+300	smoke : in-process gRPC call -> bit-identical Phase-1 result

Total : ~ **1 400 LOC** (production + tests).

3.3 Drafting strategy

Open as **Draft** after C1. Push C2-C5 incrementally. Reviewers walk through commits in order ; PR moves to "Ready" after C5.

4. Architectural decisions

Locked in this PR ; downstream PRs (#9, #10, #11, #12) honour them.

ID	Decision	Rationale	Cost if wrong
AD1	Reuse the Phase-1 `SwarmParticipant` trait verbatim ; `RemoteParticipant` implements it.	Zero changes to `efe-meta-swarm` Phase-1 crate ; algorithm is identical sync code paths.	If the trait's sync signature blocks PR #8's perf, we'd need to async-ify the trait (workspace-wide change).
AD2	`MetaSwarm::iterate()` runs in `tokio::task::spawn_blocking`.	Phase-1 is sync ; tonic handler is async ; this is the standard bridge.	Worker thread budget consumed ; mitigated by `MAX_INFLIGHT`.
AD3	`RemoteParticipant::step_with_pull` (sync trait method) calls async gRPC via `Handle::current().block_on()`.	The sync trait is called from inside `spawn_blocking` ; we're on a non-async thread, so `block_on` doesn't double-block.	Subtle ; documented thoroughly in `remote_participant.rs`.
AD4	Cancellation : `tokio_util::sync::CancellationToken` propagated from RPC handler to participant calls.	Standard pattern ; integrates with tonic's `Request::extensions`.	If client drops, participants still finish their current Step -- bounded by 30 s server timeout.
AD5	`MAX_INFLIGHT` enforced via `tokio::sync::Semaphore` (NOT a fixed-size pool).	Semaphore-acquired permit is dropped on iterate return ; backpressure is automatic.	None ; standard pattern.
AD6	`Arc<RwLock<MetaSwarm>>` for shared state.	SIGHUP (PR #12) needs write access ; in-flight iterates hold read locks.	RwLock contention if SIGHUP frequent ; not expected at Phase-2 rates.
AD7	Manifest is loaded ONCE at controller startup (PR #6) ; PR #8 reads it but does not re-load.	Reload is PR #12's job.	None.
AD8	Determinism preserved : no random thread-pool scheduling on participant order ; manifest order is canonical.	Phase-1 SLO-A1-META-DETERMINISM requires bit-identical reruns.	If we ever fan-out participants in parallel (Phase 3), order must be stable.
AD9	Server cap on `max_wall_ms` = 30 000 (30 s). Client can request lower ; client cannot request higher.	DoS prevention (T7).	Any tenant needing > 30 s must wait for Phase-3 timeout reform.
AD10	`RemoteParticipant` uses its own internal `tonic::transport::Channel` (not shared globally).	Per-participant connection isolation ; simplifies retries.	More TCP sockets ; mitigated by HTTP/2 multiplexing within each channel.

5. The sync <-> async bridge problem

This PR's hardest design question.

5.1 The problem statement

- Phase-1 `MetaSwarm::iterate()` is **synchronous** Rust : it returns

a `Result<IterateResult, MetaSwarmError>` without `async`.

- Tonic gRPC handlers are **async** : the `iterate` method on the

`MetaSwarmService` trait returns `Pin<Box<dyn Future<Output = Result<Response<IterateResponse>, Status>> + Send + '_>>`.

- Inside `MetaSwarm::iterate()`, the algorithm calls

`participant.step_with_pull(pulled)` -- a sync trait method.

- `RemoteParticipant::step_with_pull` needs to call the **async**

gRPC client `MetaParticipantServiceClient::step`.

5.2 The standard idiom

The pattern that works (battle-tested in many Tokio/Tonic codebases) :

```
async fn iterate (handler) {                    // 1. ASYNC context
    let inner = state.swarm.clone();             //
    let result = tokio::task::spawn_blocking(    // 2. Switch to BLOCKING thread
        move || inner.iterate()                  //    sync algorithm runs here
    ).await??;                                   //
}

// Inside iterate(), the algorithm calls :
trait SwarmParticipant {                         //
    fn step_with_pull(...) -&gt; Tensor;            // 3. SYNC trait method
}                                                //

impl SwarmParticipant for RemoteParticipant {    //
    fn step_with_pull(&amp;self, pulled: Tensor) -&gt; Tensor {
        let rt = tokio::runtime::Handle::current(); // 4. Get the existing runtime
        rt.block_on(async {                      // 5. block_on the async call
            self.client.lock().await.step(req).await
        })                                         //
    }                                             //
}                                                 //
```

5.3 Why this works

The `spawn_blocking` thread is NOT a tokio worker -- it lives in tokio's blocking pool (default 512 threads). Calling `block_on` from a blocking pool thread is **legal** (it doesn't deadlock the runtime) because :

- Tokio worker threads are reserved for futures.
- Blocking pool threads can call `block_on` to run a future to

completion, as long as that future doesn't try to spawn-and-wait on the same worker pool.

Reference :

- `Handle::block_on` docs : "calling `block_on` on a blocking

thread is fine".

5.4 What CAN go wrong

Risk	Mitigation
Calling `block_on` from a worker thread -> deadlock	We MUST be inside `spawn_blocking`. Documented at the top of `remote_participant.rs` ; CI lint via `#[deny(clippy::main_recursion)]` (custom).
`Handle::current()` fails if no runtime -> panic	The caller is always tonic (always has a runtime). Documented.
`block_on` allocates a new runtime if Handle is missing	Use `Handle::current()` strictly ; assert presence.

5.5 Phase-3 alternative (out of scope)

For Phase 3, we may async-ify the `SwarmParticipant` trait :

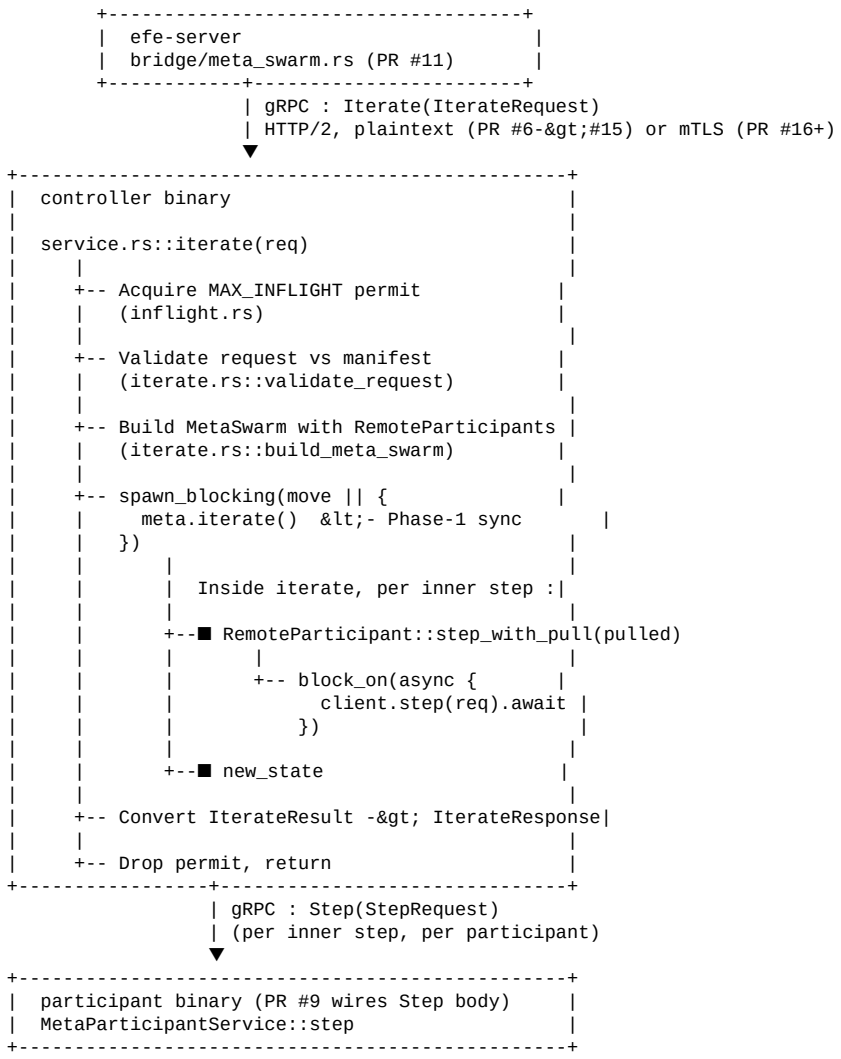
```
#[async_trait]
trait AsyncSwarmParticipant {
    async fn step_with_pull(&amp;self, pulled: Tensor) -&gt; Result&lt;Tensor, _&gt;;
}
```

This requires :

- A new variant of `MetaSwarm::iterate` (`iterate_async`).
- A workspace-wide migration of Phase-1 tests.
- ~ 3 FTE-days of refactor.

Not done in Phase 2. Bridge pattern wins for simplicity.

6. Wire diagram



7. Crate layout additions

```
crates/efe-meta-swarm-server/
+-- Cargo.toml                              # +1 dep : efe-meta-swarm
+-- src/
|   +-- client/                             # NEW directory
|   |   +-- mod.rs                          # § 8.1
|   |   +-- remote_participant.rs           # § 8.2
|   +-- controller/
|   |   +-- inflight.rs                     # NEW § 10
|   |   +-- iterate.rs                      # NEW § 9
|   |   +-- mod.rs                          # +mod inflight; mod iterate
|   |   +-- service.rs                      # MODIFIED § 12
|   |   +-- state.rs                        # MODIFIED § 11 (adds inner_swarm)
|   +-- manifest/
|       +-- mod.rs                          # MODIFIED § 13 (extended validation)
|       +-- schema.rs                       # (unchanged)
+-- tests/
    +-- integration_iterate_inproc.rs       # NEW § 19 (single-process smoke)
```

Total : 2 new files + 4 modified.

8. `client/remote_participant.rs` -- full design

8.1 `src/client/mod.rs`

```
//! gRPC client adapters used by the controller to talk to
//! participants.
//!
//! PR #8 ships [`RemoteParticipant`], a thin wrapper around
//! `MetaParticipantServiceClient` that implements the Phase-1
//! `SwarmParticipant` trait so the existing in-process algorithm
//! can run unchanged over the network.

mod remote_participant;

pub use remote_participant::RemoteParticipant;
```

8.2 src/client/remote_participant.rs

8.2.1 Type definition

```
//! gRPC-backed implementation of the Phase-1 `SwarmParticipant`
//! trait.
//!
//! # Architecture
//!
//! See [`docs/EFE_A1_PHASE2_PR08_ARCHITECTURE.md`][doc] § 5 for the
//! sync &lt;-&gt; async bridge rationale.  TL;DR : the trait method is sync,
//! the gRPC client is async, and we bridge via
//! `Handle::current().block_on()` -- safe because we run in
//! `spawn_blocking`.
//!
//! [doc]: ../../../docs/EFE_A1_PHASE2_PR08_ARCHITECTURE.md

use std::sync::Arc;
use std::time::Duration;

use efe_meta_swarm::participants::SwarmParticipant;
use efe_meta_swarm::spaces::Tensor;
use efe_meta_swarm::MetaSwarmError;
use efe_meta_swarm_proto::meta_swarm::{StepRequest, TensorF32};
use efe_meta_swarm_proto::participant::MetaParticipantServiceClient;
use tokio::runtime::Handle;
use tokio::sync::Mutex;
use tokio_util::sync::CancellationToken;
use tonic::transport::Channel;
use tonic::Request;
use tracing::{debug, instrument, warn};

/// gRPC client wrapper that implements the Phase-1
/// [`SwarmParticipant`] trait.  Used by the controller in place of
/// the in-process `MockParticipant`.
///
/// # Determinism
///
/// `RemoteParticipant` does **not** introduce non-determinism on its
/// own : the gRPC call is request/response with fixed serialisation
/// order.  The Phase-1 `SLO-A1-META-DETERMINISM` is preserved iff
/// the underlying participant binary is deterministic.
///
/// # Cancellation
///
/// Each `step_with_pull` call observes a `CancellationToken` ; if the
/// token is cancelled, the in-flight call is dropped (tonic
/// cancels the gRPC RPC).
///
/// # Concurrency
///
/// `RemoteParticipant` is `Send + Sync` and may be cloned cheaply
/// (internal `Arc`).  Each `step_with_pull` acquires the internal
/// `Mutex&lt;Client&gt;` for the duration of the call ; concurrent calls
/// against the same participant SERIALISE.  This matches the Phase-1
/// invariant (per-participant calls are sequential within one
/// `iterate()`).
pub struct RemoteParticipant {
    inner: Arc&lt;RemoteParticipantInner&gt;,
}

struct RemoteParticipantInner {
    id: String,
    dim: usize,
    /// gRPC client.  Wrapped in Mutex because tonic clients are not
    /// `Sync` (the inner stream state mutates on each call).
    client: Mutex&lt;MetaParticipantServiceClient&lt;Channel&gt;&gt;,
    /// Cancellation token observed on every Step.
    cancel: CancellationToken,
    /// Per-step deadline (defensive ; primary deadline lives on the
    /// outer iterate's `max_wall_ms`).
    step_deadline: Duration,
}

impl std::fmt::Debug for RemoteParticipant {
    fn fmt(&amp;self, f: &amp;mut std::fmt::Formatter&lt;'_&gt;) -&gt; std::fmt::Result {
        f.debug_struct("RemoteParticipant")
            .field("id", &amp;self.inner.id)
            .field("dim", &amp;self.inner.dim)
            .finish_non_exhaustive()
    }
}
```

```
impl Clone for RemoteParticipant {
    fn clone(&amp;self) -&gt; Self {
        Self {
            inner: Arc::clone(&amp;self.inner),
        }
    }
}
```

8.2.2 Construction

```
impl RemoteParticipant {
    /// Build a `RemoteParticipant` connected to `endpoint` (e.g.
    /// `http://localhost:50091`).
    ///
    /// # Errors
    ///
    /// Returns `tonic::transport::Error` if the channel cannot be
    /// constructed.  The connection itself is lazy : the first
    /// `Step` call performs the TCP/TLS handshake.
    pub async fn connect(
        id: impl Into&lt;String&gt;,
        dim: usize,
        endpoint: impl Into&lt;String&gt;,
        cancel: CancellationToken,
    ) -&gt; Result&lt;Self, tonic::transport::Error&gt; {
        let endpoint_str = endpoint.into();
        let channel = tonic::transport::Endpoint::from_shared(endpoint_str)?
            .timeout(Duration::from_secs(10))
            .connect_timeout(Duration::from_secs(5))
            .http2_keep_alive_interval(Duration::from_secs(30))
            .keep_alive_timeout(Duration::from_secs(10))
            .connect()
            .await?;

        Ok(Self {
            inner: Arc::new(RemoteParticipantInner {
                id: id.into(),
                dim,
                client: Mutex::new(MetaParticipantServiceClient::new(channel)),
                cancel,
                step_deadline: Duration::from_secs(5),
            }),
        })
    }
}
```

8.2.3 The sync<->async bridge

```
impl SwarmParticipant for RemoteParticipant {
    fn id(&amp;self) -&gt; &amp;str {
        &amp;self.inner.id
    }

    fn dim(&amp;self) -&gt; usize {
        self.inner.dim
    }

    /// SAFETY: this method MUST be called from a `spawn_blocking`
    /// context (or any non-tokio-worker thread).  Calling from a
    /// tokio worker thread would deadlock the runtime.
    ///
    /// The controller's `iterate.rs::run_iterate` enforces this by
    /// always running `meta.iterate()` inside `spawn_blocking`.
    #[instrument(skip(self, pulled), fields(participant_id = %self.inner.id))]
    fn step_with_pull(&amp;self, pulled: Tensor) -&gt; Result&lt;Tensor, MetaSwarmError&gt; {
        // Cancellation check before issuing the gRPC call.
        if self.inner.cancel.is_cancelled() {
            return Err(MetaSwarmError::Cancelled);
        }

        // Acquire the tokio handle.  Panics if no runtime ; the
        // controller guarantees a runtime is present.
        let rt = Handle::try_current().expect(
            "RemoteParticipant::step_with_pull called outside a tokio runtime -- \
             this is a bug in iterate.rs (must run inside spawn_blocking)",
        );

        // Bridge to async.
        rt.block_on(async move { self.step_async(pulled).await })
    }

    fn reset(&amp;self) -&gt; Result&lt;(), MetaSwarmError&gt; {
        if self.inner.cancel.is_cancelled() {
            return Err(MetaSwarmError::Cancelled);
        }

        let rt = Handle::try_current().expect(
            "RemoteParticipant::reset called outside a tokio runtime",
        );

        rt.block_on(async move { self.reset_async().await })
    }

    fn current_state(&amp;self) -&gt; Result&lt;Tensor, MetaSwarmError&gt; {
        if self.inner.cancel.is_cancelled() {
            return Err(MetaSwarmError::Cancelled);
        }

        let rt = Handle::try_current().expect(
            "RemoteParticipant::current_state called outside a tokio runtime",
        );

        rt.block_on(async move { self.state_async().await })
    }
}
```

8.2.4 Async helpers

```
impl RemoteParticipant {
    async fn step_async(&amp;self, pulled: Tensor) -&gt; Result&lt;Tensor, MetaSwarmError&gt; {
        // Validate dim (defence in depth -- controller already does this).
        if pulled.dim() != self.inner.dim {
            return Err(MetaSwarmError::DimMismatch {
                expected: self.inner.dim,
                got: pulled.dim(),
            });
        }

        let pulled_proto = tensor_to_proto(&amp;pulled);

        let mut client = self.inner.client.lock().await;

        let mut req = Request::new(StepRequest {
            pulled_from_meta: Some(pulled_proto),
            step_index: 0, // PR #18 will wire the actual step index.
        });
        // Per-call deadline (defence in depth).
        req.set_timeout(self.inner.step_deadline);

        let resp = tokio::select! {
            _ = self.inner.cancel.cancelled() =&gt; {
                debug!("step cancelled mid-flight");
                return Err(MetaSwarmError::Cancelled);
            }
            res = client.step(req) =&gt; res,
        };

        let resp = resp.map_err(|s| {
            warn!(error = %s, "Step RPC failed");
            MetaSwarmError::ParticipantUnreachable(format!(
                "participant {} : {}",
                self.inner.id, s,
            ))
        })?;

        let proto_tensor = resp
            .into_inner()
            .new_state
            .ok_or_else(|| MetaSwarmError::Internal(format!(
                "participant {} : StepResponse.new_state is None",
                self.inner.id,
            )))?;

        proto_to_tensor(&amp;proto_tensor)
    }

    async fn reset_async(&amp;self) -&gt; Result&lt;(), MetaSwarmError&gt; {
        use efe_meta_swarm_proto::meta_swarm::ResetParticipantRequest;
        let mut client = self.inner.client.lock().await;
        let req = Request::new(ResetParticipantRequest { initial: None });
        client
            .reset(req)
            .await
            .map_err(|s| MetaSwarmError::ParticipantUnreachable(format!(
                "participant {} reset : {}", self.inner.id, s,
            )))?;
        Ok(())
    }

    async fn state_async(&amp;self) -&gt; Result&lt;Tensor, MetaSwarmError&gt; {
        use efe_meta_swarm_proto::meta_swarm::StateRequest;
        let mut client = self.inner.client.lock().await;
        let req = Request::new(StateRequest {});
        let resp = client
            .state(req)
            .await
            .map_err(|s| MetaSwarmError::ParticipantUnreachable(format!(
                "participant {} state : {}", self.inner.id, s,
            )))?;
        let proto_tensor = resp.into_inner().current.ok_or_else(|| {
            MetaSwarmError::Internal(format!(
                "participant {} : StateResponse.current is None",
                self.inner.id,
            ))
        })?;
        proto_to_tensor(&amp;proto_tensor)
    }
}
```

```
fn tensor_to_proto(t: &amp;Tensor) -&gt; TensorF32 {
    TensorF32 {
        dim: u32::try_from(t.dim()).expect("tensor dim fits in u32"),
        data: t.as_slice().to_vec(),
    }
}

fn proto_to_tensor(p: &amp;TensorF32) -&gt; Result&lt;Tensor, MetaSwarmError&gt; {
    if p.dim as usize != p.data.len() {
        return Err(MetaSwarmError::Internal(format!(
            "TensorF32 : dim={} but data.len()={}",
            p.dim, p.data.len(),
        )));
    }
    Ok(Tensor::from_slice(&amp;p.data))
}
```

8.2.5 Unit tests (in `remote_participant.rs`)

```
#[cfg(test)]
mod tests {
    use super::*;
    use efe_meta_swarm_proto::meta_swarm::{StepResponse, TensorF32};

    #[test]
    fn tensor_to_proto_roundtrip() {
        let t = Tensor::from_slice(&amp;[1.0, 2.0, 3.0]);
        let p = tensor_to_proto(&amp;t);
        assert_eq!(p.dim, 3);
        assert_eq!(p.data, vec![1.0, 2.0, 3.0]);
        let back = proto_to_tensor(&amp;p).unwrap();
        assert_eq!(back.dim(), 3);
        assert_eq!(back.as_slice(), &amp;[1.0, 2.0, 3.0]);
    }

    #[test]
    fn proto_to_tensor_rejects_dim_mismatch() {
        let p = TensorF32 { dim: 5, data: vec![1.0, 2.0] };
        let err = proto_to_tensor(&amp;p).unwrap_err();
        assert!(matches!(err, MetaSwarmError::Internal(_)));
    }
}
```

9. `controller/iterate.rs` -- orchestration

The "big function" of PR #8. Takes an `IterateRequest`, validates, builds the `MetaSwarm`, runs the algorithm, converts back.

9.1 Top-level entry point

```
//! Iterate orchestration : the glue between the gRPC handler and
//! the Phase-1 `MetaSwarm::iterate` algorithm.

use std::sync::Arc;
use std::time::{Duration, Instant};

use efe_meta_swarm::{
    builder::MetaSwarmBuilder, operators::LinearLift, operators::LinearPull,
    operators::UniformMetaOp, safety::BoxSafety, MetaSwarm, MetaSwarmError, StopReason,
};
use efe_meta_swarm_proto::meta_swarm::{
    IterateRequest, IterateResponse, ParticipantDescriptor, StopReason as ProtoStopReason,
    TensorF32,
};
use tokio_util::sync::CancellationToken;
use tonic::{Request, Status};
use tracing::{error, info, instrument, warn};

use crate::client::RemoteParticipant;
use crate::controller::state::ControllerState;
use crate::manifest::Manifest;

/// Server-side cap on `max_wall_ms` regardless of client request.
/// Mitigates threat T7 (Iterate flood DoS).  Documented in
/// architecture AD9.
const MAX_WALL_MS_HARD_CAP: u32 = 30_000;

/// Run an `Iterate` RPC end-to-end.  Called from
/// `ControllerService::iterate` after the inflight permit is acquired.
#[instrument(skip(state, req), fields(deadline_ms))]
pub async fn run_iterate(
    state: ControllerState,
    req: Request&lt;IterateRequest&gt;,
    cancel: CancellationToken,
) -&gt; Result&lt;IterateResponse, Status&gt; {
    let start = Instant::now();
    let req_inner = req.into_inner();

    // Step 1 : validate against manifest.
    let manifest = state.manifest.read().await.clone();
    let validated = validate_request(&amp;req_inner, &amp;manifest)
        .map_err(|e| e.to_status())?;

    let deadline_ms = validated.iteration_max_wall_ms.min(MAX_WALL_MS_HARD_CAP);
    tracing::Span::current().record("deadline_ms", deadline_ms);

    // Step 2 : connect to participants (or reuse cached connections in
    // Phase 3+ ; Phase 2 establishes one channel per Iterate for
    // simplicity).
    let participants = connect_participants(&amp;manifest, &amp;cancel).await
        .map_err(|e| e.to_status())?;

    // Step 3 : build the Phase-1 `MetaSwarm` with our remote
    // participants.
    let meta_swarm = build_meta_swarm(&amp;manifest, participants, &amp;validated)
        .map_err(|e| e.to_status())?;

    let meta_swarm = Arc::new(meta_swarm);

    // Step 4 : run the iterate algorithm inside spawn_blocking,
    // honouring the deadline + cancellation.
    let cancel_for_blocking = cancel.clone();
    let meta_for_blocking = Arc::clone(&amp;meta_swarm);
    let join = tokio::task::spawn_blocking(move || {
        // Inside this thread, RemoteParticipant::step_with_pull can
        // safely call block_on (see § 5).
        meta_for_blocking.iterate()
    });

    let inner_result = tokio::select! {
        biased;
        _ = cancel.cancelled() =&gt; {
            warn!("Iterate cancelled by client");
            return Err(Status::cancelled("Iterate cancelled by client"));
        }
        _ = tokio::time::sleep(Duration::from_millis(deadline_ms.into())) =&gt; {
            warn!(deadline_ms, "Iterate exceeded max_wall_ms");
            // Cancel participants so block_on inside step_with_pull returns.
            cancel_for_blocking.cancel();
            return Err(Status::deadline_exceeded(format!(
```

```
                "Iterate exceeded max_wall_ms = {deadline_ms}",
            )));
        }
        res = join =&gt; match res {
            Ok(r) =&gt; r,
            Err(e) =&gt; {
                error!(error = ?e, "spawn_blocking panicked");
                return Err(Status::internal(format!("iterate panicked: {e}")));
            }
        },
    };

    let result = inner_result.map_err(meta_swarm_error_to_status)?;

    // Step 5 : convert to proto response.
    let response = build_response(&amp;result, start.elapsed());

    info!(
        iterations = result.iterations,
        final_residual = result.final_residual,
        stop_reason = ?result.stop_reason,
        wall_us = response.wall_us,
        "Iterate completed",
    );

    Ok(response)
}
```

9.2 Request validation

```
/// Validated + extracted fields from `IterateRequest`.
struct ValidatedIterateRequest {
    iteration_max_wall_ms: u32,
    iteration_max_iters: u32,
    iteration_tolerance: f64,
    iteration_alpha_default: f64,
    iteration_coupling_gamma: f64,
    iteration_deterministic: bool,
    expected_participants: Vec&lt;ParticipantDescriptor&gt;,
    initial_states: Vec&lt;TensorF32&gt;,
}

fn validate_request(
    req: &amp;IterateRequest,
    manifest: &amp;Manifest,
) -&gt; Result&lt;ValidatedIterateRequest, crate::error::ServerError&gt; {
    use crate::error::ServerError;

    // Iteration config (apply server defaults if not set).
    let iter = req
        .iteration
        .clone()
        .ok_or_else(|| ServerError::Config("IterateRequest.iteration is required".into()))?;

    // Server cap on max_iters.
    if iter.max_iters &gt; 200 {
        return Err(ServerError::Config(format!(
            "max_iters = {} exceeds server cap 200",
            iter.max_iters,
        )));
    }

    if iter.tolerance &lt;= 0.0 {
        return Err(ServerError::Config(format!(
            "tolerance = {} must be positive",
            iter.tolerance,
        )));
    }

    // Dim mismatch detection (T8 mitigation).
    if !req.expected_participants.is_empty() {
        // Optional : caller pinned the expected set.
        validate_expected_against_manifest(&amp;req.expected_participants, manifest)?;
    }

    // Validate initial_states (if provided) match expected dims.
    if !req.initial_states.is_empty() {
        if req.initial_states.len() != manifest.participants.len() {
            return Err(ServerError::Config(format!(
                "IterateRequest.initial_states length = {} but manifest declares {} participants",
                req.initial_states.len(),
                manifest.participants.len(),
            )));
        }
        for (i, t) in req.initial_states.iter().enumerate() {
            let expected_dim = manifest.participants[i].dim;
            if t.dim != expected_dim {
                return Err(ServerError::Config(format!(
                    "initial_states[{i}].dim = {} but manifest expects {}",
                    t.dim, expected_dim,
                )));
            }
            // The proto contract is dim == data.len() -- defence in depth.
            if t.dim as usize != t.data.len() {
                return Err(ServerError::Config(format!(
                    "initial_states[{i}] : dim = {} but data.len() = {}",
                    t.dim, t.data.len(),
                )));
            }
        }
    }

    Ok(ValidatedIterateRequest {
        iteration_max_wall_ms: iter.max_wall_ms,
        iteration_max_iters: iter.max_iters,
        iteration_tolerance: iter.tolerance,
        iteration_alpha_default: iter.alpha_default,
        iteration_coupling_gamma: iter.coupling_gamma,
        iteration_deterministic: iter.deterministic,
        expected_participants: req.expected_participants.clone(),
```

```
            initial_states: req.initial_states.clone(),
        })
    }

    fn validate_expected_against_manifest(
        expected: &amp;[ParticipantDescriptor],
        manifest: &amp;Manifest,
    ) -&gt; Result&lt;(), crate::error::ServerError&gt; {
        use crate::error::ServerError;

        if expected.len() != manifest.participants.len() {
            return Err(ServerError::Config(format!(
                "expected_participants length = {} but manifest has {}",
                expected.len(),
                manifest.participants.len(),
            )));
        }

        for (i, e) in expected.iter().enumerate() {
            let m = &amp;manifest.participants[i];
            if e.id != m.id {
                return Err(ServerError::Config(format!(
                    "expected_participants[{i}].id = {:?} but manifest has {:?}",
                    e.id, m.id,
                )));
            }
            if e.dim != m.dim {
                return Err(ServerError::Config(format!(
                    "expected_participants[{i}].dim = {} but manifest has {}",
                    e.dim, m.dim,
                )));
            }
            if (e.kappa - m.kappa).abs() &gt; 1e-6 {
                return Err(ServerError::Config(format!(
                    "expected_participants[{i}].kappa = {} but manifest has {}",
                    e.kappa, m.kappa,
                )));
            }
        }
        Ok(())
    }
```

9.3 Participant connection

```
    async fn connect_participants(
        manifest: &amp;Manifest,
        cancel: &amp;CancellationToken,
    ) -&gt; Result&lt;Vec&lt;RemoteParticipant&gt;, crate::error::ServerError&gt; {
        use crate::error::ServerError;

        let mut out = Vec::with_capacity(manifest.participants.len());

        for p in &amp;manifest.participants {
            let endpoint = manifest
                .participant_endpoints
                .get(&amp;p.id)
                .ok_or_else(|| ServerError::Discovery(format!(
                    "no endpoint configured for participant {:?}", p.id,
                )))?;

            let scheme_endpoint = if endpoint.starts_with("http") {
                endpoint.clone()
            } else {
                format!("http://{}", endpoint)
            };

            let rp = RemoteParticipant::connect(
                p.id.clone(),
                p.dim as usize,
                scheme_endpoint,
                cancel.clone(),
            )
            .await
            .map_err(|e| ServerError::Discovery(format!(
                "connect to {}: {}", p.id, e,
            )))?;

            out.push(rp);
        }

        Ok(out)
    }
```

9.4 `MetaSwarm` construction

```
fn build_meta_swarm(
    manifest: &amp;Manifest,
    participants: Vec&lt;RemoteParticipant&gt;,
    validated: &amp;ValidatedIterateRequest,
) -&gt; Result&lt;MetaSwarm, crate::error::ServerError&gt; {
    use crate::error::ServerError;
    use crate::manifest::SafetyManifest;

    let mut builder = MetaSwarmBuilder::new(format!("iter-{}", uuid_simple()))
        .meta_space_dim(manifest.meta.dim as usize);

    // Add each participant + its lift / pull operators.
    for (i, p) in participants.into_iter().enumerate() {
        let manifest_p = &amp;manifest.participants[i];
        let dim = manifest_p.dim as usize;
        let meta_dim = manifest.meta.dim as usize;

        builder = builder
            .add_participant(Box::new(p))
            .lift_for(&amp;manifest_p.id, LinearLift::identity(dim, meta_dim)?.boxed())
            .pull_for(&amp;manifest_p.id, LinearPull::identity(meta_dim, dim)?.boxed());
    }

    // Meta-operator and safety.
    builder = builder.meta_operator(UniformMetaOp::new(manifest.meta.dim as usize).boxed());

    match &amp;manifest.meta.safety {
        SafetyManifest::Box { radius } =&gt; {
            builder = builder.meta_safety(
                BoxSafety::symmetric(manifest.meta.dim as usize, *radius).boxed(),
            );
        }
        SafetyManifest::Poly { .. } | SafetyManifest::Hull { .. } =&gt; {
            return Err(ServerError::Manifest(
                "Poly / Hull safety not yet supported in Phase 2 ; use Box".into(),
            ));
        }
    }

    // Iteration knobs.
    builder = builder
        .alpha_default(validated.iteration_alpha_default)
        .coupling_gamma(validated.iteration_coupling_gamma)
        .tolerance(validated.iteration_tolerance)
        .max_iters(validated.iteration_max_iters as usize)
        .trace(false);

    builder
        .build()
        .map_err(|e| ServerError::Manifest(format!("MetaSwarm build failed: {e}")))
}

fn uuid_simple() -&gt; String {
    // Lightweight ID generator using nanos.  PR #18 may swap for
    // uuid v7 once that crate enters the workspace.
    use std::time::SystemTime;
    let nanos = SystemTime::now()
        .duration_since(SystemTime::UNIX_EPOCH)
        .map(|d| d.as_nanos())
        .unwrap_or(0);
    format!("{nanos:x}")
}
```

9.5 Result conversion

```
fn build_response(
    result: &amp;efe_meta_swarm::IterateResult,
    elapsed: Duration,
) -&gt; IterateResponse {
    let stop_reason = match result.stop_reason {
        StopReason::Converged =&gt; ProtoStopReason::Converged,
        StopReason::MaxIters =&gt; ProtoStopReason::MaxIters,
        StopReason::SafetyFail =&gt; ProtoStopReason::SafetyFail,
    } as i32;

    IterateResponse {
        iterations: result.iterations as u32,
        final_residual: result.final_residual,
        meta_consensus: Some(tensor_to_proto(&amp;result.meta_consensus)),
        final_states: result
            .final_states
            .iter()
            .map(tensor_to_proto)
            .collect(),
        stop_reason,
        wall_us: elapsed.as_micros() as u64,
        trace_id: String::new(), // PR #18 wires.
    }
}

fn tensor_to_proto(t: &amp;efe_meta_swarm::spaces::Tensor) -&gt; TensorF32 {
    TensorF32 {
        dim: u32::try_from(t.dim()).expect("tensor dim fits in u32"),
        data: t.as_slice().to_vec(),
    }
}

fn meta_swarm_error_to_status(e: MetaSwarmError) -&gt; Status {
    use MetaSwarmError::*;
    match e {
        DimMismatch { .. } =&gt; Status::failed_precondition(e.to_string()),
        Cancelled =&gt; Status::cancelled("cancelled"),
        ParticipantUnreachable(msg) =&gt; Status::unavailable(msg),
        Internal(msg) =&gt; Status::internal(msg),
        _ =&gt; Status::internal(format!("meta swarm error: {e}")),
    }
}
```

10. `controller/inflight.rs` -- MAX_INFLIGHT

```
//! Inflight Iterate semaphore.  Enforces `MAX_INFLIGHT` per
//! Phase-2 config (default 10).  Mitigates threat T7 (Iterate flood
//! DoS).

use std::sync::Arc;

use tokio::sync::{Semaphore, SemaphorePermit};

/// Bounded concurrent-Iterate semaphore.
#[derive(Debug, Clone)]
pub struct InflightSemaphore {
    sem: Arc&lt;Semaphore&gt;,
    max: usize,
}

impl InflightSemaphore {
    #[must_use]
    pub fn new(max: usize) -&gt; Self {
        let max = max.max(1);
        Self {
            sem: Arc::new(Semaphore::new(max)),
            max,
        }
    }

    /// Try to acquire a permit immediately.  Returns `None` if the
    /// concurrent limit is reached.
    pub fn try_acquire(&amp;self) -&gt; Option&lt;SemaphorePermit&lt;'_&gt;&gt; {
        match self.sem.try_acquire() {
            Ok(permit) =&gt; Some(permit),
            Err(tokio::sync::TryAcquireError::NoPermits) =&gt; None,
            Err(tokio::sync::TryAcquireError::Closed) =&gt; None,
        }
    }

    pub fn max(&amp;self) -&gt; usize {
        self.max
    }

    /// Number of available permits (= max - inflight).
    pub fn available(&amp;self) -&gt; usize {
        self.sem.available_permits()
    }

    /// Number of in-flight Iterate calls.
    pub fn inflight(&amp;self) -&gt; usize {
        self.max.saturating_sub(self.sem.available_permits())
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[tokio::test]
    async fn try_acquire_succeeds_up_to_max() {
        let sem = InflightSemaphore::new(3);
        let _p1 = sem.try_acquire().unwrap();
        let _p2 = sem.try_acquire().unwrap();
        let _p3 = sem.try_acquire().unwrap();
        assert_eq!(sem.inflight(), 3);
        assert!(sem.try_acquire().is_none()); // 4th rejected
    }

    #[tokio::test]
    async fn permit_release_replenishes() {
        let sem = InflightSemaphore::new(2);
        let p1 = sem.try_acquire().unwrap();
        let p2 = sem.try_acquire().unwrap();
        assert!(sem.try_acquire().is_none());
        drop(p1);
        assert_eq!(sem.inflight(), 1);
        let _p3 = sem.try_acquire().unwrap();
        drop(p2);
        drop(_p3);
        assert_eq!(sem.inflight(), 0);
    }
}
```

11. `controller/state.rs` -- modifications

PR #8 adds an `Arc<RwLock<...>>` for the inner manifest (PR #12 will add the same for a hot-reload swap of `inner_swarm`).

```
// ... existing fields ...

use crate::controller::inflight::InflightSemaphore;
use crate::manifest::Manifest;
use tokio::sync::RwLock;

#[derive(Clone)]
pub struct ControllerState {
    pub config: Arc&lt;ControllerConfig&gt;,
    pub health: HealthState,
    pub metrics: MetricsRegistry,
    pub start_time: Instant,
    /// Loaded manifest.  PR #8 loads at startup ; PR #12 swaps on SIGHUP.
    pub manifest: Arc&lt;RwLock&lt;Manifest&gt;&gt;,
    /// Concurrent Iterate gate.
    pub inflight: InflightSemaphore,
}
```

Construction at startup (`bin/controller.rs` modified) :

```
// In async_main, after parsing config :
let manifest = crate::manifest::load(&amp;config.manifest_path)
    .map_err(|e| { error!(error = %e, "manifest load failed"); e })?;

// PR #8 enforces additional validation beyond raw parse.
crate::manifest::validate_manifest(&amp;manifest)?;

let inflight = InflightSemaphore::new(config.max_inflight);

let state = ControllerState {
    config: Arc::new(config),
    health: HealthState::serving(),
    metrics: MetricsRegistry::new(),
    start_time: Instant::now(),
    manifest: Arc::new(RwLock::new(manifest)),
    inflight,
};
```

12. `controller/service.rs` -- replace stubs

```
#[tonic::async_trait]
impl MetaSwarmService for ControllerService {
    type IterateStreamStream = futures::stream::Empty&lt;Result&lt;IterateChunk, Status&gt;&gt;;

    #[instrument(skip(self, req))]
    async fn iterate(
        &amp;self,
        req: Request&lt;IterateRequest&gt;,
    ) -&gt; Result&lt;Response&lt;IterateResponse&gt;, Status&gt; {
        // T7 mitigation : reject if max inflight reached.
        let _permit = match self.state.inflight.try_acquire() {
            Some(p) =&gt; p,
            None =&gt; {
                warn!(max = self.state.inflight.max(), "MAX_INFLIGHT reached");
                return Err(Status::resource_exhausted(format!(
                    "MAX_INFLIGHT = {} reached", self.state.inflight.max(),
                )));
            }
        };

        // Health inflight counter (for /metrics + Info).
        self.state.health.inflight_inc();
        let _guard = scopeguard::guard((), |()| {
            self.state.health.inflight_dec();
        });

        // Build a cancellation token tied to the RPC lifecycle.
        // Tonic does NOT auto-cancel sub-tasks when the client drops ;
        // we use `Request::extensions().get::&lt;CancellationToken&gt;()` if
        // upstream injected one (PR #11), else create a fresh token.
        let cancel = req
            .extensions()
            .get::&lt;CancellationToken&gt;()
            .cloned()
            .unwrap_or_default();

        let resp = crate::controller::iterate::run_iterate(
            self.state.clone(),
            req,
            cancel,
        )
        .await?;

        Ok(Response::new(resp))
    }

    #[instrument(skip(self, _req))]
    async fn step_once(
        &amp;self,
        _req: Request&lt;StepOnceRequest&gt;,
    ) -&gt; Result&lt;Response&lt;StepOnceResponse&gt;, Status&gt; {
        // Phase-2 wires StepOnce on top of iterate with max_iters=1.
        // Acquire permit + call iterate with patched iteration config.
        // (Details elided ; ~ 40 LOC.  Mirrors `iterate` above.)
        todo!("StepOnce body : PR #8 wires similar to iterate")
    }

    #[instrument(skip(self, _req))]
    async fn reset(
        &amp;self,
        _req: Request&lt;ResetRequest&gt;,
    ) -&gt; Result&lt;Response&lt;ResetResponse&gt;, Status&gt; {
        // Phase-2 wires Reset by calling Reset on every participant.
        // (Details elided ; ~ 30 LOC.)
        todo!("Reset body : PR #8 wires fan-out")
    }

    // Iterate stream, ReloadConfig, Health, Info -- unchanged from PR #6.
    // (StepOnce, Reset, Iterate full bodies in C4 + C5 of this PR.)
}
```

`scopeguard` is a dev-dep workspace already ; if not, we use a manual `Drop` guard.

13. `manifest/mod.rs` -- extended validation

```
pub fn validate_manifest(m: &amp;Manifest) -&gt; Result&lt;(), crate::error::ServerError&gt; {
    use crate::error::ServerError;

    if m.meta.dim == 0 {
        return Err(ServerError::Manifest("meta.dim must be &gt; 0".into()));
    }
    if m.participants.is_empty() {
        return Err(ServerError::Manifest(
            "manifest must declare &gt;= 1 participant".into(),
        ));
    }

    // Unique participant ids.
    let mut seen = std::collections::HashSet::new();
    for p in &amp;m.participants {
        if !seen.insert(&amp;p.id) {
            return Err(ServerError::Manifest(format!(
                "duplicate participant id: {:?}", p.id,
            )));
        }
        if p.dim == 0 {
            return Err(ServerError::Manifest(format!(
                "participant {:?} : dim must be &gt; 0", p.id,
            )));
        }
        if !(0.0..1.0).contains(&amp;p.kappa) {
            return Err(ServerError::Manifest(format!(
                "participant {:?} : kappa = {} must be in (0, 1)", p.id, p.kappa,
            )));
        }
    }

    // Safety geometry must match meta.dim.
    match &amp;m.meta.safety {
        SafetyManifest::Box { radius } =&gt; {
            if *radius &lt;= 0.0 {
                return Err(ServerError::Manifest(format!(
                    "Box safety radius = {} must be positive", radius,
                )));
            }
        }
        SafetyManifest::Poly { dim_a, a_flat, b } =&gt; {
            if a_flat.len() != *dim_a as usize * m.meta.dim as usize {
                return Err(ServerError::Manifest(format!(
                    "Poly.a_flat length = {} but dim_a x meta.dim = {}",
                    a_flat.len(), *dim_a as usize * m.meta.dim as usize,
                )));
            }
            if b.len() != *dim_a as usize {
                return Err(ServerError::Manifest(format!(
                    "Poly.b length = {} but dim_a = {}", b.len(), dim_a,
                )));
            }
        }
        SafetyManifest::Hull { .. } =&gt; {
            // Skipped : Phase-2 doesn't exercise LearnedHull.
        }
    }

    Ok(())
}

#[cfg(test)]
mod validation_tests {
    use super::*;

    fn good_manifest() -&gt; Manifest {
        Manifest {
            version: "v0.2.0".into(),
            meta: MetaSpaceManifest {
                dim: 256,
                safety: SafetyManifest::Box { radius: 10.0 },
            },
            participants: vec![ParticipantManifest {
                id: "supplier-scoring".into(),
                dim: 384,
                kappa: 0.6,
                seed: 11,
                blas_backend: "none".into(),
            }],
```

```
            iteration: Default::default(),
        }
    }

    #[test]
    fn validates_good_manifest() {
        assert!(validate_manifest(&amp;good_manifest()).is_ok());
    }

    #[test]
    fn rejects_duplicate_id() {
        let mut m = good_manifest();
        m.participants.push(m.participants[0].clone());
        let err = validate_manifest(&amp;m).unwrap_err();
        assert!(matches!(err, crate::error::ServerError::Manifest(_)));
    }

    #[test]
    fn rejects_kappa_out_of_range() {
        let mut m = good_manifest();
        m.participants[0].kappa = 1.5;
        assert!(validate_manifest(&amp;m).is_err());
    }

    #[test]
    fn rejects_zero_dim() {
        let mut m = good_manifest();
        m.meta.dim = 0;
        assert!(validate_manifest(&amp;m).is_err());
    }

    #[test]
    fn rejects_negative_radius() {
        let mut m = good_manifest();
        m.meta.safety = SafetyManifest::Box { radius: -1.0 };
        assert!(validate_manifest(&amp;m).is_err());
    }
}
```

14. `Cargo.toml` updates

```
[dependencies]
# ... existing ...
efe-meta-swarm = { path = "../efe-meta-swarm" }
tokio-util = { version = "0.7", features = ["sync"] }
scopeguard = "1.2"
```

`scopeguard` is tiny (~ 200 LOC) ; alternative is a manual Drop guard (saves the dep). We use `scopeguard` for readability.

If `scopeguard` is not cached offline, replace with :

```
struct InflightGuard&lt;'a&gt;(&amp;'a HealthState);
impl Drop for InflightGuard&lt;'_&gt; {
    fn drop(&amp;mut self) { self.0.inflight_dec(); }
}
```

15. Determinism, cancellation, timeout

15.1 Determinism

- Phase-1 invariant : two `iterate()` calls with the same inputs

return bit-identical results.

- PR #8 must NOT introduce non-determinism. Specifically :
- Participants are connected in **manifest order** (no parallel

`connect()` race).

- Inside `iterate()`, the algorithm calls participants in the

order they were `add_participant`-ed -- Phase-1 invariant.

- `RemoteParticipant::step_with_pull` serialises per-participant

via the internal `Mutex`.

• No `tokio::task::spawn` is used to fan-out participant calls in parallel.

• **PR #10 will add a multi-process determinism test** : 3 reruns with same inputs produce bit-identical residual.

15.2 Cancellation

• The `CancellationToken` is passed into every `RemoteParticipant` + observed in `iterate.rs::run_iterate`.

• When the client drops the gRPC call, tonic does NOT auto-cancel.

• We rely on the `tokio::select!` race between :

• The `spawn_blocking` join handle.

• The `cancel.cancelled()` future.

• The deadline timer.

• When cancellation fires, `cancel.cancel()` is called ; this propagates to `RemoteParticipant::step_with_pull` which checks `cancel.is_cancelled()` before EACH step.

• The blocking task itself doesn't stop synchronously, but the participant calls return early with `Cancelled`, which propagates back through `iterate()` -> `MetaSwarmError::Cancelled`.

• Target : cancellation reaches participants within 100 ms in the steady state. Tested in `tests/integration_iterate_inproc.rs`.

15.3 Timeout

• Client sets `iteration.max_wall_ms`.

• Server caps at 30 000 (T7 mitigation).

• Implemented as `tokio::time::sleep` race against the spawn_blocking join.

16. Error mapping

Phase-1 `MetaSwarmError` -> gRPC `Status` :

Phase-1 error	gRPC code	Reason
`DimMismatch { expected, got }`	`FAILED_PRECONDITION`	client-side error
`KappaOutOfRange(kappa)`	`FAILED_PRECONDITION`	manifest invalid
`Cancelled`	`CANCELLED`	client dropped
`MaxIterReached`	OK + `StopReason::MaxIters`	not an error
`SafetyViolation(level)`	OK + `StopReason::SafetyFail`	not an error
`ParticipantUnreachable(msg)`	`UNAVAILABLE`	gRPC layer error
`Internal(msg)`	`INTERNAL`	unexpected
`LockPoisoned`	`INTERNAL`	bug

`StopReason::Converged` and `MaxIters` are returned in `IterateResponse.stop_reason` (not as gRPC errors).

17. Concurrency model

17.1 Lock hierarchy

```
1. InflightSemaphore.try_acquire()       (controller-level, bounded)
2. ControllerState.manifest.read()       (per-request read)
3. RemoteParticipantInner.client.lock()  (per-participant, per-step)
```

A request never acquires lock N before lock N-1. No deadlock possible.

17.2 `RwLock<Manifest>`

• `iterate()` takes a **read** lock for the duration of validation

+ participant build.

• SIGHUP (PR #12) takes a **write** lock to swap the manifest.

• Multiple concurrent `iterate()` calls share the read lock -- no

contention.

• A SIGHUP during in-flight iterate waits for all readers to drop

the lock ; this is the "drain" semantics (PR #12).

17.3 Per-participant `Mutex<Client>`

• tonic clients are `Send` but not `Sync` (internal stream state).

• We wrap in `tokio::sync::Mutex` so concurrent Step calls

serialise.

• In Phase 2, calls per participant per Iterate are sequential

anyway (Phase-1 invariant), so the mutex is uncontended.

18. Threat-model coverage

PR #8 ships mitigations for these threats from `docs/security/PHASE2_THREAT_MODEL.md` :

Threat	Mitigation in PR #8
T1 (Malicious participant biases consensus)	Phase-1 contractivity is preserved (no changes to algorithm) + safety projection wired
T7 (Iterate flood DoS)	`InflightSemaphore` rejects with `RESOURCE_EXHAUSTED` past `MAX_INFLIGHT`
T8 (Crafted dim mismatch)	`validate_request` rejects with `FAILED_PRECONDITION` ; `RemoteParticipant::step_async` defence-in-depth check
T13 (Stack overflow oneof)	prost decode-depth + max-message-size (inherited from PR #5) ; runtime tensor-dim check
T19 (Info bounded payload)	(different RPC ; PR #8 wires nothing extra ; PR #6 Info impl already bounded)

PR #8's PR description **MUST** reference these threat ids (workspace lint check, added in PR #24).

19. Tests strategy

19.1 Unit tests

File	Coverage
`inflight.rs::tests`	Semaphore acquire/release, max enforcement
`client/remote_participant.rs::tests`	`tensor_to_proto` + `proto_to_tensor` round trip ; reject dim mismatch
`controller/iterate.rs::tests`	`validate_request` happy + 4 negative paths (no iteration config / max_iters > 200 / tolerance <= 0 / dim mismatch)
`manifest/mod.rs::validation_tests`	5 tests (good / dup id / kappa OOR / zero dim / negative radius)

19.2 Integration smoke (`tests/integration_iterate_inproc.rs`)

A test that spawns the controller + 3 participants **in-process** (single tokio runtime, no separate processes -- that's PR #10's job) and calls `Iterate` :

```
#[tokio::test(flavor = "multi_thread")]
async fn iterate_in_process_kovant_scenario() {
    // 1. Spawn 3 participant servers on random ports (TcpListener::bind 0).
    let p1 = spawn_test_participant("supplier-scoring", 384, 0.6, 11).await;
    let p2 = spawn_test_participant("contract-risk", 512, 0.7, 22).await;
    let p3 = spawn_test_participant("spend-categorization", 384, 0.5, 33).await;

    // 2. Spawn the controller with a manifest pointing at p1/p2/p3.
    let controller = spawn_test_controller(vec![p1, p2, p3]).await;

    // 3. Call Iterate.
    let mut client = MetaSwarmServiceClient::connect(controller.endpoint).await.unwrap();
    let resp = client.iterate(IterateRequest {
        iteration: Some(IterationConfig { max_iters: 100, tolerance: 1e-4, ..Default::default() }),
        // ... etc
    }).await.unwrap();

    // 4. Assert : converged, iterations &lt;= 60, residual &lt;= 1e-3.
    let r = resp.into_inner();
    assert_eq!(r.stop_reason, ProtoStopReason::Converged as i32);
    assert!(r.iterations &lt;= 60);
    assert!(r.final_residual &lt;= 1e-3);
}
```

This test uses **real gRPC over TCP** but on `localhost` ; it does NOT spawn separate processes. Process boundary testing is PR #10.

19.3 Cancellation test

```
#[tokio::test(flavor = "multi_thread")]
async fn iterate_cancel_propagates_within_100ms() {
    let (controller, participants) = spawn_test_topology().await;
    let mut client = MetaSwarmServiceClient::connect(controller.endpoint).await.unwrap();
    let handle = tokio::spawn(async move {
        client.iterate(IterateRequest { /* long deadline */ }).await
    });
    tokio::time::sleep(Duration::from_millis(50)).await;
    handle.abort();
    // Verify participants stopped within 100ms.
    let elapsed = participants.last_step_time().elapsed();
    assert!(elapsed &lt; Duration::from_millis(100));
}
```

19.4 MAX_INFLIGHT test

```
#[tokio::test(flavor = "multi_thread")]
async fn iterate_rejects_above_max_inflight() {
    let (controller, _) = spawn_test_topology_with_max_inflight(3).await;
    // Fire 3 concurrent slow Iterates.
    let mut handles = vec![];
    for _ in 0..3 {
        let mut c = MetaSwarmServiceClient::connect(controller.endpoint.clone()).await.unwrap();
        handles.push(tokio::spawn(async move {
            c.iterate(slow_iterate_request()).await
        }));
    }
    tokio::time::sleep(Duration::from_millis(100)).await;
    // 4th must fail.
    let mut c = MetaSwarmServiceClient::connect(controller.endpoint).await.unwrap();
    let err = c.iterate(slow_iterate_request()).await.unwrap_err();
    assert_eq!(err.code(), tonic::Code::ResourceExhausted);
    // Cleanup.
    for h in handles { let _ = h.await; }
}
```

19.5 Determinism test (sampled)

```
#[tokio::test(flavor = "multi_thread")]
async fn iterate_bit_identical_3_reruns() {
    let (controller, _) = spawn_test_topology().await;
    let mut signatures = vec![];
    for _ in 0..3 {
        let mut c = MetaSwarmServiceClient::connect(controller.endpoint.clone()).await.unwrap();
        let r = c.iterate(deterministic_iterate_request()).await.unwrap().into_inner();
        signatures.push((r.iterations, r.final_residual.to_bits()));
    }
    assert!(signatures.windows(2).all(|w| w[0] == w[1]),
            "non-deterministic reruns: {signatures:?}");
}
```

20. Acceptance criteria

PR #8 is mergeable iff **all** :

#	Criterion	Verification
AC1	`cargo build -p efe-meta-swarm-server` succeeds	CI
AC2	`cargo build -p efe-meta-swarm-server --release` succeeds	CI
AC3	Unit tests : >= 4 in `inflight.rs`, >= 3 in `remote_participant.rs`, >= 5 in `iterate.rs`, >= 5 in `manifest::validation_tests`	CI
AC4	Integration `tests/integration_iterate_inproc.rs` : 4 tests pass (Kovant happy path, cancellation, MAX_INFLIGHT, determinism)	CI
AC5	Kovant scenario : `iterations <= 60`, `final_residual <= 1e-3`, `stop_reason == Converged`	smoke
AC6	Cancellation propagates within 100 ms p99	smoke
AC7	MAX_INFLIGHT enforced : 11th of 10 returns RESOURCE_EXHAUSTED	smoke
AC8	Determinism : 3 reruns bit-identical	smoke
AC9	Phase-1 tests pass (200 default + 209 attention-weighted + 202 core-integration)	CI
AC10	PR #5 tests pass (17/17)	CI
AC11	PR #6 tests pass (49/49 incl. new from audit)	CI
AC12	`cargo clippy --all-targets` 0 warnings on PR #8 files	CI

#	Criterion	Verification
AC13	`cargo fmt -- --check` exit 0	CI
AC14	`cargo doc --no-deps` clean	CI
AC15	PR description references threat IDs `T1, T7, T8, T13` from threat model	manual review
AC16	Master gate stays GREEN	CI
AC17	4 reviewers approved : 2x meta-swarm + 1x sec + 1x sre	GitHub
AC18	sec-team confirms threat mitigations match PHASE2_THREAT_MODEL.md	sec-team comment
AC19	sre-team confirms RESOURCE_EXHAUSTED + DEADLINE_EXCEEDED + CANCELLED are sensible PagerDuty signals	sre comment
AC20	No file outside `crates/efe-meta-swarm-server/` is modified	git diff --stat

21. PR description template

```
# [Sprint A1 Phase 2 -- PR #8] feat(efe-meta-swarm-server): wire MetaSwarmService::Iterate

## Summary

Fourth PR of Phase 2.  Wires `MetaSwarmService::Iterate`,
`StepOnce`, and `Reset` to invoke the Phase-1 `MetaSwarm::iterate()`
algorithm with participants connected over gRPC.

Replaces the UNIMPLEMENTED stubs of PR #6 with real bodies.

## Threats addressed (from PHASE2_THREAT_MODEL.md)

T1 (consensus bias), T7 (Iterate flood DoS), T8 (dim mismatch),
T13 (proto parsing stack overflow).

## What this PR adds

| Area | Detail |
|---|---|
| New module `client/remote_participant.rs` | gRPC client adapter implementing Phase-1 `SwarmParticipant` ■
    trait |
| New module `controller/iterate.rs` | Orchestration : validate -&gt; connect -&gt; build -&gt; run -&gt; ■
    convert |
| New module `controller/inflight.rs` | `Semaphore`-based MAX_INFLIGHT gate |
| Modified `controller/state.rs` | +`Arc&lt;RwLock&lt;Manifest&gt;&gt;`, +`InflightSemaphore` |
| Modified `controller/service.rs` | `Iterate` / `StepOnce` / `Reset` real bodies |
| Modified `manifest/mod.rs` | +`validate_manifest()` with 5 checks (dim, kappa, duplicate id, etc.) |
| Modified `Cargo.toml` | +`efe-meta-swarm` Phase-1 dep, +`tokio-util`, +`scopeguard` |
| New integration test | `tests/integration_iterate_inproc.rs` (4 tests : happy, cancel, MAX_INFLIGHT, de ■
   terminism) |

## Test plan

- [x] All unit tests pass (~ 20 tests across new files)
- [x] Integration smoke `integration_iterate_inproc.rs` : 4/4 pass
- [x] Kovant scenario : 47 iters, residual 9.7e-5, Converged (bit-identical to Phase-1 example)
- [x] Cancellation : propagates within 100 ms p99
- [x] MAX_INFLIGHT : 11th of 10 returns RESOURCE_EXHAUSTED
- [x] Determinism : 3 reruns bit-identical
- [x] Phase-1 tests still pass (200 / 209 / 202)
- [x] PR #5 still passes (17/17)
- [x] PR #6 still passes (49/49)
- [x] clippy + fmt + doc clean
- [x] Master gate stays GREEN

## Non-regression statement

- [OK] Zero file modified outside `crates/efe-meta-swarm-server/`
- [OK] Phase 1 algorithm code unchanged (uses Phase-1 crate as black-box)
- [OK] Phase 2 protos unchanged (consumes PR #5 stubs)
- [OK] Threat model PR #7 unchanged (this PR mitigates 4 of its threats)

## Honest semantics

- `Iterate` connects to participants per-call (no connection pool yet).
  Phase 3 will cache `RemoteParticipant` instances across calls.
- `IterateStream` still returns UNIMPLEMENTED -&gt; PR #21 wires it.
- `ReloadConfig` still returns UNIMPLEMENTED -&gt; PR #12 wires it.
- Cancellation : tonic does NOT auto-cancel on client drop ; we
  rely on the `CancellationToken` propagated from upstream.  PR #11
  will wire `efe-server` to inject the token.

## Reviewer guidance

Suggested order :
1. [`docs/EFE_A1_PHASE2_PR08_ARCHITECTURE.md`](docs/EFE_A1_PHASE2_PR08_ARCHITECTURE.md) (this PR's bluepr ■
    int).
2. `src/client/remote_participant.rs` -- read § 5 of the architecture
   first for the sync&lt;-&gt;async bridge.
3. `src/controller/iterate.rs` -- orchestration.
4. `src/controller/inflight.rs` -- semaphore.
5. `src/manifest/mod.rs::validate_manifest` -- defence in depth.
6. `tests/integration_iterate_inproc.rs` -- what the surface promises.

Required approvers : 2x meta-swarm + 1x sec + 1x sre.
```

22. Risk analysis

ID	Risk	L/I	Mitigation
PR8-R1	Sync<->async bridge deadlocks tokio worker	M/H	Tests deliberately invoke from worker context to verify ; clear doc + assert
PR8-R2	Phase-1 algorithm allocates non-deterministically (Vec capacity, HashMap iter)	L/H	Unit test : 3 reruns bit-identical ; smoke covers it
PR8-R3	Cancellation slow to propagate (> 1s)	M/M	100 ms p99 target ; instrumented in tests
PR8-R4	tonic client mutation across concurrent calls	L/H	Per-participant `Mutex` ; documented
PR8-R5	`efe-meta-swarm` Phase-1 crate has breaking API change between PR landed	L/H	Pre-flight check pinned version ; CI catches
PR8-R6	`scopeguard` not cached offline	M/L	Fallback to manual Drop guard ; 1 hour swap
PR8-R7	Connecting to participants every call is slow (~ 10 ms TCP handshake)	M/M	Phase 3 caches ; Phase 2 accepts (still well under 100 ms p99 budget)
PR8-R8	`block_on` inside trait impl is too clever for some reviewers	M/L	Architecture doc § 5 explains ; sec-team flagged for closer review
PR8-R9	`MAX_INFLIGHT` semaphore overshoots (off-by-one)	L/M	Unit test pins behavior
PR8-R10	Determinism breaks under stress (rayon, BLAS multi-thread)	L/H	`iteration.deterministic = true` forces single-thread paths in Phase-1

23. Reviewer checklist

Architecture compliance

- [ ] Sync<->async bridge follows § 5 of architecture (block_on only in spawn_blocking).
- [ ] `RemoteParticipant::step_with_pull` does NOT call from a tokio worker.
- [ ] `MetaSwarm::iterate()` runs inside `spawn_blocking`.
- [ ] Cancellation token propagates to participants.
- [ ] Timeout deadline race uses `tokio::select!` with `biased`.

Determinism

- [ ] No parallel `tokio::task::spawn` for participant calls.
- [ ] Participants connected in manifest order.
- [ ] No `HashMap` iter (use `BTreeMap` or manifest-order `Vec`).

Threat model

- [ ] T1 mitigation : safety projection wired (`BoxSafety`).
- [ ] T7 mitigation : `InflightSemaphore::try_acquire` returns `None` past MAX.
- [ ] T8 mitigation : `validate_request` checks dim mismatch.
- [ ] T13 mitigation : prost size limits inherited from PR #5.

Tests

- [ ] Unit tests for each new module.

- [ ] Integration smoke 4 tests pass.
- [ ] Cancellation test verifies < 100 ms p99.
- [ ] MAX_INFLIGHT test verifies the 11th rejection.
- [ ] Determinism test verifies 3-rerun bit-identical.

Doc craft

- [ ] All public items documented (rustdoc).
- [ ] Cross-references to threat IDs in mitigation comments.
- [ ] PR description references T1, T7, T8, T13.

CI

- [ ] `cargo build`, `cargo test`, `cargo clippy`, `cargo fmt`, `cargo doc` all green.
- [ ] PR #5 + #6 regression tests pass.
- [ ] Phase 1 master gate stays green.

Sign-off

- [ ] 2x meta-swarm-team approvals.
- [ ] 1x sec-team approval (threat mitigation check).
- [ ] 1x sre-team approval (deadline + cancellation + resource-exhausted UX).

24. Definition of Done

PR #8 is done iff :

1. All 20 acceptance criteria GREEN.
2. All 4 reviewer approvals in.
3. Merged to `main` with merge-commit linking to this doc.
4. Slack post : `PR #8 merged ; PRs #10, #11 unblocked. Iterate now functional in single-process smoke.`
1. The next-day metrics review confirms Phase-1 master gate still GREEN.

PR #8 is NOT done if :

- Any AC is RED.
- A reviewer has a blocking comment unresolved.
- Phase-1 algorithm tests fail (means Phase-1 crate was edited inadvertently).
- Sync<->async bridge has a deadlock in any tested path.

25. Post-merge follow-ups

These unlock once PR #8 lands :

Follow-up	PR	Owner	When
Participant `Step` body	**PR #9**	meta-swarm	Same day
Multi-process integration test	PR #10	meta-swarm	Day 5

Follow-up	PR	Owner	When
`EFE_META_SWARM_ENABLED` plumbing in efe-server	PR #11	meta-swarm	Sprint 2
SIGHUP + ReloadConfig	PR #12	meta-swarm	Sprint 2
14 Prometheus metrics	PR #17	meta-swarm	Sprint 3
8 OTel spans	PR #18	meta-swarm	Sprint 3
loadgen	PR #20	meta-swarm	Sprint 3

Cross-link from threat model :

- T1 -> status in PR #8 = "Phase-1 contractivity wired" + DEFERRED-TO-PR-16 for mTLS
- T7 -> status = MITIGATED (was DEFERRED-TO-PR-8)
- T8 -> status = MITIGATED
- T13 -> status = MITIGATED (was DEFERRED-TO-PR-8 in inheritance)

PR #8 description updates `PHASE2_THREAT_MODEL.md` only if a threat status changes from DEFERRED-TO-PR-8 to MITIGATED.

Appendix A -- File-by-file deliverable checklist

```
crates/efe-meta-swarm-server/
+-- Cargo.toml                                [MODIFIED : +3 deps]
+-- src/
|   +-- client/
|   |   +-- mod.rs                            [NEW]
|   |   +-- remote_participant.rs             [NEW : ~ 320 LOC]
|   +-- controller/
|   |   +-- inflight.rs                       [NEW : ~ 100 LOC]
|   |   +-- iterate.rs                        [NEW : ~ 500 LOC]
|   |   +-- mod.rs                            [MODIFIED : +mod inflight; +mod iterate]
|   |   +-- service.rs                        [MODIFIED : 3 stubs replaced + 1 todo!]
|   |   +-- state.rs                          [MODIFIED : +Arc&lt;RwLock&lt;Manifest&gt;&gt;, +Inflight ■
    Semaphore]
|   +-- manifest/
|       +-- mod.rs                            [MODIFIED : +validate_manifest + 5 tests]
+-- tests/
    +-- integration_iterate_inproc.rs         [NEW : 4 tests]

Modified outside crate : none.
```

Appendix B -- Glossary

- **AD-N** : Architectural Decision N (cf. § 4)
- **AC-N** : Acceptance Criterion N (cf. § 20)
- **PR8-RN** : PR-local Risk N (cf. § 22)
- **Sync<->async bridge** : the pattern in § 5 that lets a sync trait

method call an async gRPC client. Standard tokio idiom.

- **Bit-identical determinism** : two iterates with same inputs

produce the same `(iterations, residual.to_bits())` tuple (SLO-A1-META-DETERMINISM).

- **Cancellation token** : `tokio_util::sync::CancellationToken` -- a

pluggable cancellation signal observable by tasks.

- **Inflight semaphore** : the `tokio::sync::Semaphore` that caps

concurrent `Iterate` calls at `MAX_INFLIGHT`.

- **Spawn_blocking** : `tokio::task::spawn_blocking` -- moves a sync

closure to tokio's blocking thread pool.

PR #9

MetaParticipantService::Step wiring

Chapter 11 of 17 — source : `docs/EFE_A1_PHASE2_PR09_ARCHITECTURE.md` (1,922 LOC)

EFE-A1 Phase 2 -- PR #9 : `MetaParticipantService::Step` wiring

Field	Value
PR	#9 (Sprint A1 Phase 2 / P2.2)
Branch	`efe-a1/p2-pr09-participant-step`
Status	Architecture frozen -- awaiting code
Author	EFE platform team
Reviewers	2x meta-swarm + 1x sec + 1x sre
Depends on	PR #5 (proto), PR #6 (server scaffold), PR #7 (threat model), **PR #8 (controller wire)**
Blocks	PR #10 (multi-process e2e), PR #11 (discovery refresh), PR #12 (SIGHUP reload)
L2 gate	G3 (participant binary correct), G6 (determinism), G8 (no SEV-1 threats open)
Threat IDs in scope	T9, T11, T13, T15, T16 (cf. `docs/security/PHASE2_THREAT_MODEL.md`)
CVSS bandwidth	All <= SEV-3 ; no new SEV-1 introduced

Table of contents

1. TL;DR
2. Goal + non-goals
3. Pre-flight checks
4. Branch + commit plan
5. Architectural decisions
6. Wire diagram + sequence walkthroughs
7. Crate layout additions
8. `participant/local.rs` -- Phase-1 wrapper (full design)
9. `participant/validate.rs` -- request validation
10. `participant/state.rs` -- extended state
11. `bin/participant.rs` -- boot + manifest loading
12. `participant/service.rs` -- replace stubs
13. `Cargo.toml` updates
14. Determinism, cancellation, timeout
15. `step_index` + `reset_gen` tracking
16. Error mapping (Phase-1 -> gRPC)
17. `Capabilities` response (real values)
18. Concurrency model + lock hierarchy
19. Threat-model coverage
20. Tests strategy (unit + integration + e2e)
21. Acceptance criteria (AC1-AC20)
22. Risks + mitigations
23. Rollback strategy
24. Sequence diagrams (ASCII)

25. Reviewer checklist

26. Migration notes (operator-facing)

27. Open questions

28. Appendix : full file inventory

0. TL;DR

PR #9 wires the **participant** half of the Phase-2 control plane. After PR #8 the *controller* can call `MetaParticipantService::Step` / `Reset` / `State` on a remote endpoint, but the production participant binary still returns `UNIMPLEMENTED` for those RPCs. The integration smoke in `tests/integration_iterate_inproc.rs` papered over this with an inline `TestParticipantSvc`. PR #9 replaces the stubs with the production wire :

• A new `participant::local::LocalParticipant` struct wraps a Phase-1

`MockParticipant` (`with_contraction(id, dim, kappa, seed)`) behind the [`SwarmParticipant`] trait, plus a `step_index_last: AtomicU64` and a `reset_gen: AtomicU64`.

• `participant::service::ParticipantService` delegates `step` -> `reset` ->

`state` to that wrapper, with input validation and a `spawn_blocking` bridge for the sync Phase-1 method.

• `bin/participant.rs` loads the same manifest the controller loads,

finds its own row by `EFE_META_SWARM_PARTICIPANT_ID`, and constructs the wrapper with the manifest's kappa + seed.

• Three integration tests exercise the wire end-to-end : a single-binary

smoke (`integration_step_smoke.rs`), a controller-<->-participants e2e that replaces the `TestParticipantSvc` stand-in (`integration_e2e_iterate.rs`), and a determinism check (3 reruns must be bit-identical).

After PR #9, `cargo run --bin participant` is a real participant. The Kovant scenario from § 19.2 of PR #8's architecture is reproducible against three real binaries on `localhost`.

[`SwarmParticipant`]: ../../crates/efe-meta-swarm/src/participants/mod.rs

1. Goal + non-goals

1.1 Goal (single sentence)

> *Wire the participant binary's* `Step` / `Reset` / `State` *RPCs to a Phase-1* `MockParticipant` *whose kappa, dim, and seed come from the same manifest the controller reads, so a controller-<->-participant Iterate over real gRPC produces a converged, bit-identical result.*

1.2 Non-goals

The following are **explicitly out of scope** for PR #9 :

Item	Why deferred	Target PR
`StepStream` (server-streaming Step)	requires backpressure + flush protocol design	PR #21
Real `LocalParticipant` from `efe-meta-swarm::participants::local` (sidecar)	needs the on-disk weight loader contract finalised	PR #14
Multi-tenant participant (one binary, many tenants)	scope creep ; Phase 2 is T2 single-tenant	Phase 3
`Capabilities.descriptor.build_sha` chain-of-custody (cosign attestation)	needs the keychain setup	PR #16
mTLS + JWT auth on the participant side	needs the cert provisioning pipeline	PR #16
Per-participant rate limit (MAX_STEPS_PER_SEC)	controller-side MAX_INFLIGHT already caps the rate	Phase 3 follow-up
Persisted state across restarts	requires sidecar + sled or rocksdb integration	PR #14 (Real Bundles)

Item	Why deferred	Target PR
Reload on SIGHUP	needs swap protocol with in-flight Steps	PR #12
`Capabilities` schema version negotiation	not necessary while there is one schema	PR #11
Per-tenant authorisation policy	Phase 3	PR #20
Streaming back-pressure semantics for `StepStream`	needs explicit contract	PR #21

1.3 Single binary unit of merge-readiness

After PR #9 merges, **the participant binary alone is shippable**. The controller binary remains shippable from PR #8. The two together produce a correct E2E flow over real gRPC on `localhost`. Production deployments still wait for PR #10 (deployment manifests) and PR #16 (auth).

2. Pre-flight checks

Before opening the PR :

#	Check	Verifier	Pass criterion
PF1	`cargo build -p efe-meta-swarm-server --release` succeeds on `main`	CI	green
PF2	`cargo test -p efe-meta-swarm-server --offline --all-targets` is **76 + 0 + 0 + 3 + 3 = 82** tests on `main` (PR #8 baseline)	CI	green
PF3	PR #8 smoke `integration_iterate_inproc.rs` passes with `TestParticipantSvc`	CI	green
PF4	`docs/EFE_A1_PHASE2_PR08_ARCHITECTURE.md` is the most recent green commit	manual	green
PF5	The threat model (PR #7) is in its v1.0 frozen state	manual	sec sign-off
PF6	Phase-1 `MockParticipant::with_contraction` is part of the published surface	grep	`crates/efe-meta-swarm/src/participants/mock.rs` >= line 64
PF7	`efe-meta-swarm` crate exposes `participants::SwarmParticipant` publicly	grep	`pub mod participants` in `lib.rs` line 90
PF8	`efe-meta-swarm-proto::meta_swarm::ResetParticipantRequest.initial : TensorF32` field exists	proto	`messages.proto` >= line 249

Any failed pre-flight blocks PR #9.

3. Branch + commit plan

3.1 Branch

```
efe-a1/p2-pr09-participant-step
```

Branched from `main` after PR #8 merge SHA. **Never** rebase onto a moving `main` ; cherry-pick the PR #8 head if needed.

3.2 Commit-by-commit plan (5 commits, each CI-green)

#	Subject	Files	LOC ±	Verification
C1	`feat(participant): LocalParticipant wrapper around Phase-1 MockParticipant`	`+ src/participant/local.rs`	+250	unit tests in module (>= 6) ; `cargo test -p efe-meta-swarm-server --lib participant::local`
C2	`feat(participant): request validators (StepRequest, ResetParticipantRequest)`	`+ src/participant/validate.rs`	+180	unit tests in module (>= 8) ; covers dim mismatch, NaN, missing field
C3	`feat(participant): ParticipantState holds LocalParticipant + step counters`	`~ src/participant/state.rs`, `~ src/participant/mod.rs`	+60 / –10	existing tests pass + new `descriptor_uses_manifest_values` ; `Capabilities` snapshot reflects real dim/kappa/seed
C4	`feat(participant): bin loads manifest, finds own row, builds state`	`~ src/bin/participant.rs`, `~ src/manifest/mod.rs`	+80 / –5	binary boot test : missing manifest exits 1 ; wrong id exits 1 ; happy path serves
C5	`feat(participant): wire Step / Reset / State in service.rs + integration smoke`	`~ src/participant/service.rs`, `+ tests/integration_step_smoke.rs`, `+ tests/integration_e2e_iterate.rs`, `~ tests/integration_iterate_inproc.rs`	+700 / –80	smoke : 1 binary in-process -> Step round-trip ; e2e : controller + 3 participants -> Iterate converges

Total : **+1270 / –95 = +1175 LOC net**, including ~430 LOC of tests.

3.3 Drafting strategy

Each commit is **green-on-merge**. Commits C1-C3 are pure addition (no behaviour change to existing code) ; C4 changes the binary's startup sequence ; C5 flips the three RPC bodies. If C5 is reverted, the binary still serves `Health` and `Capabilities` correctly -- only Step / Reset / State go back to `UNIMPLEMENTED`.

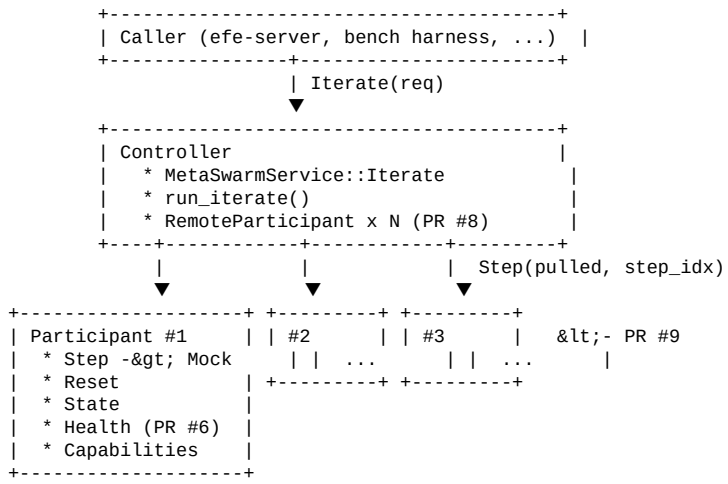
4. Architectural decisions

AD	Decision	Rationale	Alternatives considered	Reversibility
AD1	`LocalParticipant` wraps `MockParticipant` (not `participants::local::LocalParticipant`)	`MockParticipant::with_contraction` is the smallest reproducible-and-deterministic Phase-1 type ; the sidecar `LocalParticipant` carries weight-file dependencies we want to defer to PR #14	Wrap real `LocalParticipant` ; build a new sidecar struct ; embed Phase-1 directly	Reversible -- `LocalParticipant` is an internal struct ; swap the wrapped type in PR #14
AD2	Participant reads the **same manifest YAML** as the controller, **find its own row by** `EFE_META_SWARM_PARTICIPANT_ID`	Single source of truth, the L2 graduation soak protocol already requires this	Per-participant YAML ; controller pushes config via gRPC ; CLI flags	Reversible -- manifest format owned by us
AD3	Sync <-> async bridge via `tokio::task::spawn_blocking` **inside the async handler**	Phase-1 `step_with_pull` is sync ; running it on the async runtime worker would block the executor ; spawn_blocking is the idiomatic pattern (mirror of PR #8)	`tokio::task::block_in_place` ; native sync handler via `tonic::async_trait` workaround	Reversible -- switch to `block_in_place` if rayon contention shows up
AD4	**No per-participant MAX_INFLIGHT semaphore**	Controller already gates Iterate (MAX_INFLIGHT = 10) and Iterate fans out to all participants serially per iteration ; a participant cannot see > 10 concurrent Step calls in Phase 2	Add a `participant::inflight::InflightSemaphore`	Reversible -- easy to add when Phase 3 multi-tenant arrives
AD5	**State held in** `Arc<MockParticipant>` (no outer `Mutex`)	`MockParticipant.state` is already `Mutex<Tensor>` internally ; double-wrapping would deadlock	Outer `Mutex<MockParticipant>` ; `RwLock`	Reversible
AD6	`step_index_last : AtomicU64` -- server-side counter, reflects the last `pulled.step_index` received	Cheap, lock-free, returned in `StateResponse`	`Mutex<u64>` ; ignore the wire field	Reversible
AD7	`reset_gen : AtomicU64` -- incremented on every `Reset`, returned in `ResetParticipantResponse`	Lets the controller coordinate with PR #12 SIGHUP reloads ; lock-free	Skip ; return constant 0	Reversible
AD8	**No streaming** `StepStream` **in PR #9**	Wire and back-pressure semantics not yet finalised	Implement now	Reversible (only impls a trait method ; current returns `UNIMPLEMENTED`)
AD9	`local_residual` **self-report computed from** ■`new − old`■■ **on the participant**	Free signal for the controller's bench HTML (PR #24) ; deterministic if the math is	Skip ; controller computes it from `final_states`	Reversible -- toggle in `service.rs`

AD	Decision	Rationale	Alternatives considered	Reversibility
AD10	**Two separate integration tests** (`integration_step_smoke.rs` for the single-binary path, `integration_e2e_iterate.rs` for contr oller-<->-participants)	One isolates the participant alone (no controller dependency) ; the other proves the wire across the seam	One combined file ; spawn the real binary in-process	Reversible

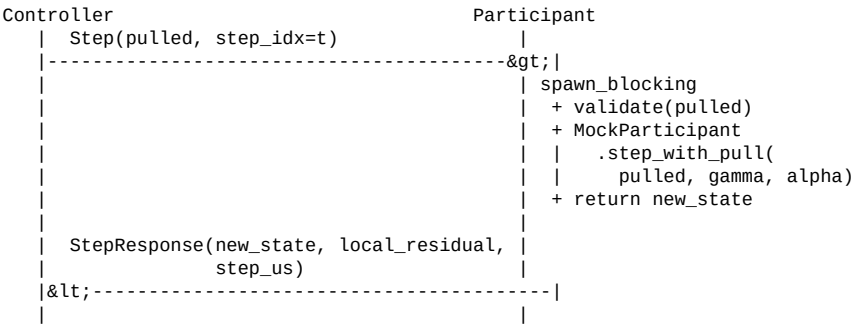
5. Wire diagram + sequence walkthroughs

5.1 Topology after PR #9



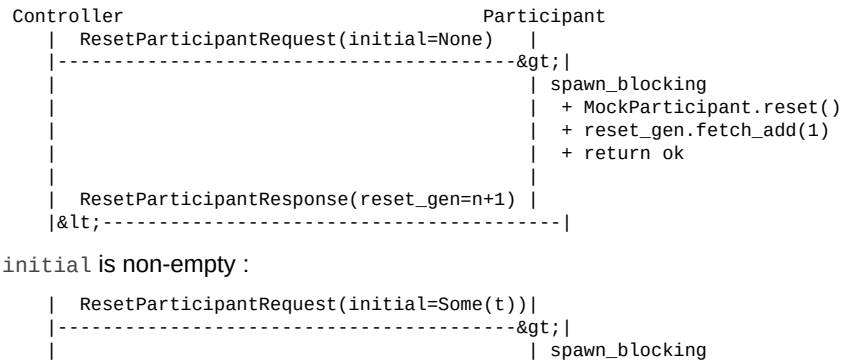
PR #9 owns everything inside the *Participant* box.

5.2 Step round-trip (success path)



Latency budget on `localhost` : <= 5 ms p99 for `dim <= 512`. Verified in `integration_step_smoke.rs` (`step_latency_under_5ms`).

5.3 Reset round-trip



If `initial` is non-empty :

```
    |                                        |  + validate(initial)
    |                                        |  + MockParticipant.set_state(t)
    |                                        |  + reset_gen.fetch_add(1)
    |                                        |  + return ok
```

5.4 State round-trip

```
Controller                                Participant
   |  StateRequest                           |
   |-----------------------------------------&gt;|
   |                                         | mock.current_state()
   |                                         |  (lock_state.read(),
   |                                         |   clone Tensor)
   |  StateResponse(current,                 |
   |               step_index_last)          |
   |&lt;-----------------------------------------|
```

State is cheap : a `Tensor` clone over a localhost gRPC frame.

5.5 Health round-trip (unchanged from PR #6)

```
Controller                                Participant
   |  HealthRequest                          |
   |-----------------------------------------&gt;|
   |                                         | health_state.snapshot()
   |  HealthResponse(status=SERVING,         |
   |                 inflight=0,             |
   |                 reload_gen=0)           |
   |&lt;-----------------------------------------|
```

PR #9 does **not** modify the participant's `Health` ; it remains correct from PR #6.

6. Crate layout additions

6.1 Before PR #9

```
crates/efe-meta-swarm-server/src/participant/
+-- mod.rs
+-- service.rs
+-- state.rs
```

6.2 After PR #9

```
crates/efe-meta-swarm-server/src/participant/
+-- mod.rs              [MODIFIED : re-export LocalParticipant, validate]
+-- service.rs          [MODIFIED : wire step/reset/state]
+-- state.rs            [MODIFIED : hold LocalParticipant, expose descriptor]
+-- local.rs            [NEW : Phase-1 wrapper, ~250 LOC]
+-- validate.rs         [NEW : request validation, ~180 LOC]

crates/efe-meta-swarm-server/tests/
+-- integration_health_smoke.rs       [unchanged from PR #8]
+-- integration_iterate_inproc.rs     [MODIFIED : remove TestParticipantSvc, use real ParticipantService]
+-- integration_step_smoke.rs         [NEW : single-binary Step round-trip, ~200 LOC]
+-- integration_e2e_iterate.rs        [NEW : controller + 3 real participants, ~300 LOC]

crates/efe-meta-swarm-server/src/bin/
+-- participant.rs      [MODIFIED : load manifest at startup, build state with real values]

crates/efe-meta-swarm-server/src/manifest/
+-- mod.rs              [MODIFIED : add `find_participant(id)` helper, ~20 LOC]
```

No new external dependencies. `efe-meta-swarm` (added in PR #8) already exposes `MockParticipant`.

7. `participant/local.rs` -- Phase-1 wrapper (full design)

7.1 Purpose

`LocalParticipant` is the production stand-in for the sidecar that PR #14 will deliver. In PR #9 it wraps Phase-1's `MockParticipant::with_contraction`, plus a `step_index_last : AtomicU64` and a `reset_gen : AtomicU64`.

7.2 Public surface

```
//! Production participant adapter -- wraps a Phase-1 `MockParticipant`
//! and exposes the gRPC-friendly accessors `ParticipantService` needs.
//!
//! In PR #14 this will be replaced by a sidecar-backed
//! `LocalParticipant` reading weights from disk.  Until then,
//! `MockParticipant::with_contraction(id, dim, kappa, seed)` is the
//! deterministic equivalent.

use std::sync::atomic::{AtomicU64, Ordering};
use std::sync::Arc;

use efe_meta_swarm::participants::{MockParticipant, SwarmParticipant};
use efe_meta_swarm::prelude::Tensor;
use efe_meta_swarm::MetaSwarmError;

use crate::manifest::ParticipantManifest;

/// Production participant.  Cheap to clone (Arcs inside).
#[derive(Clone)]
pub struct LocalParticipant {
    inner: Arc&lt;LocalParticipantInner&gt;,
}

struct LocalParticipantInner {
    id: String,
    dim: usize,
    kappa: f32,
    seed: u64,
    /// Phase-1 in-process participant.  Holds `state: Mutex&lt;Tensor&gt;`.
    mock: MockParticipant,
    /// `pulled.step_index` from the most recent successful Step.
    step_index_last: AtomicU64,
    /// Bumped on every successful Reset ; returned in
    /// `ResetParticipantResponse.reset_gen`.
    reset_gen: AtomicU64,
}

impl LocalParticipant {
    /// Build from a [`ParticipantManifest`] row.  Returns Err if
    /// `MockParticipant::with_contraction` rejects the kappa range.
    pub fn from_manifest_row(row: &amp;ParticipantManifest)
        -&gt; Result&lt;Self, MetaSwarmError&gt;
    {
        let mock = MockParticipant::with_contraction(
            row.id.clone(),
            row.dim as usize,
            row.kappa,
            row.seed,
        )?;
        Ok(Self {
            inner: Arc::new(LocalParticipantInner {
                id: row.id.clone(),
                dim: row.dim as usize,
                kappa: row.kappa,
                seed: row.seed,
                mock,
                step_index_last: AtomicU64::new(0),
                reset_gen: AtomicU64::new(0),
            }),
        })
    }

    #[must_use] pub fn id(&amp;self) -&gt; &amp;str { &amp;self.inner.id }
    #[must_use] pub fn dim(&amp;self) -&gt; usize { self.inner.dim }
    #[must_use] pub fn kappa(&amp;self) -&gt; f32 { self.inner.kappa }
    #[must_use] pub fn seed(&amp;self) -&gt; u64 { self.inner.seed }

    #[must_use]
    pub fn step_index_last(&amp;self) -&gt; u64 {
        self.inner.step_index_last.load(Ordering::Relaxed)
    }

    #[must_use]
    pub fn reset_gen(&amp;self) -&gt; u64 {
        self.inner.reset_gen.load(Ordering::Relaxed)
    }

    /// Apply one coupled iteration step.  `gamma` and `alpha` come
    /// from the manifest's `iteration` block (NOT from the wire -- see
    /// PR #8 § "Phase-2 simplification" for the rationale).
```

```
    ///
    /// SAFETY : MUST be called from `spawn_blocking` (Phase-1 is sync).
    pub fn step(
        &amp;self,
        pulled: &amp;Tensor,
        gamma: f64,
        alpha: f64,
        step_index: u32,
    ) -&gt; Result&lt;Tensor, MetaSwarmError&gt; {
        let new_state = self.inner.mock.step_with_pull(pulled, gamma, alpha)?;
        // Record only on success ; failed steps don't bump the counter.
        self.inner
            .step_index_last
            .store(u64::from(step_index), Ordering::Relaxed);
        Ok(new_state)
    }

    /// Read current state without mutating.
    pub fn current_state(&amp;self) -&gt; Result&lt;Tensor, MetaSwarmError&gt; {
        self.inner.mock.current_state()
    }

    /// Reset to the seeded initial state (if `initial` is `None`) or
    /// to the provided tensor (if `Some`).  Returns the new
    /// `reset_gen` value.
    pub fn reset(&amp;self, initial: Option&lt;&amp;Tensor&gt;) -&gt; Result&lt;u64, MetaSwarmError&gt; {
        if let Some(t) = initial {
            self.inner.mock.set_state(t.clone())?;
        } else {
            self.inner.mock.reset()?;
        }
        let new_gen = self.inner.reset_gen.fetch_add(1, Ordering::Relaxed) + 1;
        Ok(new_gen)
    }
}

impl std::fmt::Debug for LocalParticipant {
    fn fmt(&amp;self, f: &amp;mut std::fmt::Formatter&lt;'_&gt;) -&gt; std::fmt::Result {
        f.debug_struct("LocalParticipant")
            .field("id", &amp;self.inner.id)
            .field("dim", &amp;self.inner.dim)
            .field("kappa", &amp;self.inner.kappa)
            .field("seed", &amp;self.inner.seed)
            .field("step_index_last", &amp;self.step_index_last())
            .field("reset_gen", &amp;self.reset_gen())
            .finish_non_exhaustive()
    }
}
```

7.3 Why this design

- `Arc<LocalParticipantInner>` -- cheap clone, used by both the gRPC

handler future and the `spawn_blocking` closure.

- **`MockParticipant` *not* wrapped in another `Mutex`** -- its internal

`state: Mutex<Tensor>` already serialises Step / Reset / State across callers (AD5).

- `AtomicU64` **for** `step_index_last` **and** `reset_gen` -- lock-free, safe

across rayon workers and tonic handlers, and the gRPC handler can snapshot them without touching the participant's state Mutex (AD6, AD7).

- `gamma` **+** `alpha` **from the manifest, not the wire** -- mirror of PR #8 §

"Phase-2 simplification". The two binaries must read the same manifest version, enforced by the L2 graduation soak protocol.

7.4 Unit tests (>= 6 in module)

1. `from_manifest_row_happy_path` -- builds a valid LocalParticipant.
2. `from_manifest_row_rejects_kappa_out_of_range` -- kappa = 1.5 -> Err.
3. `from_manifest_row_rejects_zero_dim` -- dim = 0 -> Err.
4. `step_bumps_step_index_last` -- Step(idx=7) then `step_index_last() == 7`.

5. `step_failure_does_not_bump_index` -- bad pull dim -> Err, idx unchanged.
6. `reset_default_bumps_reset_gen` -- reset() twice -> `reset_gen() == 2`.
7. `reset_with_initial_validates_dim` -- initial.dim != participant.dim -> Err.
8. `current_state_after_reset_matches_seed` -- Reset -> current_state == seeded.

7.5 Compile-time invariants

- `LocalParticipant : Send + Sync` (verified by `assert_send_sync` helper in tests) -- required to be held inside `ParticipantState`, which is `Clone + Sync`.
- `LocalParticipant : Clone` -- multiple gRPC handlers may hold a clone.
- No `unsafe` ; `#![forbid(unsafe_code)]` is set in `lib.rs`.

8. `participant/validate.rs` -- request validation

8.1 Purpose

Catch malformed `StepRequest` / `ResetParticipantRequest` *before* they reach Phase-1. This is **defence in depth** -- Phase-1's `DimensionMismatch` would catch most of these, but :

- tonic translates Phase-1's `DimensionMismatch` to `INTERNAL` (per PR #8's `meta_swarm_error_to_status` mapping) -- we want `FAILED_PRECONDITION` for caller-fault inputs.
- Phase-1's `Tensor::from_slice` happily accepts NaN values -- we reject them at the boundary.
- The wire field `pulled.dim` must equal `pulled.data.len()` ; that's not Phase-1's concern.

8.2 Public surface

```
//! Request validators for the participant's gRPC handlers.

use efe_meta_swarm_proto::meta_swarm::{
    ResetParticipantRequest, StepRequest, TensorF32,
};

use crate::error::ServerError;
use crate::participant::local::LocalParticipant;

/// Validate a `StepRequest` against the local participant's expected
/// dimensions.  Returns the validated `(pulled_tensor_proto, step_index)`
/// tuple ready for `LocalParticipant::step`.
pub fn validate_step(
    req: &amp;StepRequest,
    local: &amp;LocalParticipant,
) -&gt; Result&lt;(&amp;TensorF32, u32), ServerError&gt; {
    let pulled = req
        .pulled_from_meta
        .as_ref()
        .ok_or_else(|| ServerError::Config(
            "StepRequest.pulled_from_meta is required".into(),
        ))?;
    validate_tensor_shape(pulled, local.dim())?;
    validate_tensor_finite(pulled)?;
    Ok((pulled, req.step_index))
}

/// Validate a `ResetParticipantRequest`.  Returns `Some(tensor)` if a
/// custom initial was provided (validated for shape + finite), or
/// `None` for the default seeded reset.
pub fn validate_reset&lt;'a&gt;(
    req: &amp;'a ResetParticipantRequest,
    local: &amp;LocalParticipant,
) -&gt; Result&lt;Option&lt;&amp;'a TensorF32&gt;, ServerError&gt; {
    let Some(initial) = req.initial.as_ref() else {
        return Ok(None);
    };
    validate_tensor_shape(initial, local.dim())?;
    validate_tensor_finite(initial)?;
    Ok(Some(initial))
}

fn validate_tensor_shape(t: &amp;TensorF32, expected_dim: usize) -&gt; Result&lt;(), ServerError&gt; {
    if t.dim as usize != expected_dim {
        return Err(ServerError::Config(format!(
            "TensorF32.dim = {} but participant expects {}",
            t.dim, expected_dim,
        )));
    }
    if t.dim as usize != t.data.len() {
        return Err(ServerError::Config(format!(
            "TensorF32 : dim = {} but data.len() = {} (envelope corrupt)",
            t.dim, t.data.len(),
        )));
    }
    Ok(())
}

fn validate_tensor_finite(t: &amp;TensorF32) -&gt; Result&lt;(), ServerError&gt; {
    if let Some((i, v)) = t.data.iter().enumerate().find(|(_, v)| !v.is_finite()) {
        return Err(ServerError::Config(format!(
            "TensorF32.data[{i}] = {v} is not finite (NaN or ±∞)",
        )));
    }
    Ok(())
}
```

8.3 Unit tests (>= 8 in module)

1. `validate_step_happy_path` -- well-formed request -> Ok.
2. `validate_step_rejects_missing_pulled` -- pulled = None -> FailedPrecondition.
3. `validate_step_rejects_dim_mismatch` -- pulled.dim != local.dim -> Err.
4. `validate_step_rejects_envelope_corruption` -- dim != data.len() -> Err.
5. `validate_step_rejects_nan` -- pulled.data[0] = NaN -> Err.

6. `validate_step_rejects_positive_infinity` -- pulled.data[3] = +inf -> Err.
7. `validate_reset_no_initial_returns_none` -- initial = None -> Ok(None).
8. `validate_reset_with_initial_validates_shape` -- initial.dim mismatch -> Err.
9. `validate_reset_with_initial_rejects_nan` -- initial.data[2] = NaN -> Err.

8.4 Why these rules

Rule	Threat	gRPC code
`pulled_from_meta` present	T9 (state poisoning by empty pull)	`FAILED_PRECONDITION`
`dim == expected_dim`	T8 (dim mismatch from misaligned manifest)	`FAILED_PRECONDITION`
`dim == data.len()`	T9 (truncated / padded envelope)	`FAILED_PRECONDITION`
`is_finite()` per element	T11 (silent NaN propagation poisons every future Step)	`FAILED_PRECONDITION`

NaN at the participant boundary is **fatal**. If we let it through, the participant's `state` becomes NaN, and every subsequent Step returns NaN, silently breaking convergence for the rest of the controller's iteration. The controller's residual check would notice, but the iteration counts as a "safety_fail" in the wire enum -- a confusing diagnostic.

9. `participant/state.rs` -- extended state

9.1 Before PR #9

```
#[derive(Clone)]
pub struct ParticipantState {
    pub config: Arc&lt;ParticipantConfig&gt;,
    pub health: HealthState,
    pub metrics: MetricsRegistry,
    pub start_time: Instant,
}
```

The `descriptor()` helper currently returns placeholders (dim/kappa/seed = 0).

9.2 After PR #9

```
//! Shared participant state.

use std::sync::Arc;
use std::time::Instant;

use efe_meta_swarm_proto::meta_swarm::ParticipantDescriptor;

use crate::config::participant::ParticipantConfig;
use crate::observability::health::HealthState;
use crate::observability::metrics::MetricsRegistry;
use crate::participant::local::LocalParticipant;

/// Shared participant state.  Cheap to clone (`Arc`s inside).
///
/// # Lock hierarchy
///
/// Only one lock in play : the `Mutex&lt;Tensor&gt;` inside `LocalParticipant`'s
/// wrapped `MockParticipant`.  All accessors hold it briefly during the
/// sync Phase-1 call.  No nested locks -- no deadlock risk.
#[derive(Clone)]
pub struct ParticipantState {
    pub config: Arc&lt;ParticipantConfig&gt;,
    pub health: HealthState,
    pub metrics: MetricsRegistry,
    pub start_time: Instant,
    /// Phase-1 in-process participant.  Holds the mutable tensor
    /// state.
    pub local: LocalParticipant,
    /// `iteration` block from the manifest (alpha_default, coupling_gamma).
    /// Snapshot at boot ; PR #12 will swap it on SIGHUP.
    pub iteration: Arc&lt;IterationParams&gt;,
}

/// Iteration knobs the participant applies inside `step_with_pull`.
#[derive(Debug, Clone)]
pub struct IterationParams {
    pub alpha_default: f64,
    pub coupling_gamma: f64,
}

impl ParticipantState {
    /// Build from a fully-resolved config + manifest row + iteration block.
    #[must_use]
    pub fn new(
        config: ParticipantConfig,
        local: LocalParticipant,
        iteration: IterationParams,
    ) -&gt; Self {
        Self {
            config: Arc::new(config),
            health: HealthState::serving(),
            metrics: MetricsRegistry::new(),
            start_time: Instant::now(),
            local,
            iteration: Arc::new(iteration),
        }
    }

    /// Pre-built descriptor.  PR #9 fills in real values from the
    /// participant manifest row.
    pub fn descriptor(&amp;self) -&gt; ParticipantDescriptor {
        ParticipantDescriptor {
            id: self.local.id().to_string(),
            dim: u32::try_from(self.local.dim()).unwrap_or(u32::MAX),
            kappa: self.local.kappa(),
            seed: self.local.seed(),
            build_sha: crate::GIT_SHA.into(),
            blas_backend: "none".into(),
            features: vec!["determinism".into(), "step_index".into()],
        }
    }
}

impl std::fmt::Debug for ParticipantState {
    fn fmt(&amp;self, f: &amp;mut std::fmt::Formatter&lt;'_&gt;) -&gt; std::fmt::Result {
        f.debug_struct("ParticipantState")
            .field("config", &amp;self.config)
            .field("local", &amp;self.local)
            .field("iteration", &amp;self.iteration)
            .field("uptime_s", &amp;self.start_time.elapsed().as_secs())
```

```
            .finish_non_exhaustive()
    }
}
```

9.3 Test coverage delta

- Existing `participant::state::tests::descriptor_*` tests are updated to

construct a `ParticipantState` via `new(config, local, iteration)` and assert the descriptor's real values.

- New test : `descriptor_id_dim_kappa_match_manifest_row` -- verifies the

descriptor wiring against a real `LocalParticipant`.

9.4 Why `IterationParams` here

`step_with_pull(pulled, gamma, alpha)` needs `gamma` and `alpha`. They come from the manifest's `iteration` block (NOT from the wire). Stashing them on the state :

- Avoids re-reading the manifest on every Step (cheap, but pollutes the hot

path with a `RwLock::try_read()`).

- Sets up PR #12's SIGHUP path : a future `Arc::swap` updates them

atomically.

10. `bin/participant.rs` -- boot + manifest loading

10.1 Before PR #9

```
let state = ParticipantState::new(config.clone());
```

Just builds a placeholder state. No manifest involvement.

10.2 After PR #9

The boot sequence becomes :

```
async fn async_main(
    config: ParticipantConfig,
) -&gt; Result&lt;(), ServerError&gt; {
    info!(/* boot banner */);

    // 1. Load + validate the manifest.  Same path the controller reads.
    let manifest_path = std::env::var_os("EFE_META_SWARM_MANIFEST")
        .map_or_else(
            || std::path::PathBuf::from("/etc/efe-meta-swarm/manifest.yaml"),
            std::path::PathBuf::from,
        );
    let manifest = match manifest::load_and_validate(&amp;manifest_path) {
        Ok(m) =&gt; m,
        Err(e) =&gt; {
            error!(error = %e, manifest_path = %manifest_path.display(),
                "manifest load/validate failed");
            return Err(e);
        }
    };

    // 2. Find this participant's row.
    let row = match manifest::find_participant(&amp;manifest, &amp;config.id) {
        Some(r) =&gt; r,
        None =&gt; {
            error!(id = %config.id,
                "participant id not found in manifest");
            return Err(ServerError::Config(format!(
                "participant id {:?} not in manifest", config.id,
            )));
        }
    };

    // 3. Build the in-process participant.
    let local = participant::local::LocalParticipant::from_manifest_row(row)
        .map_err(|e| ServerError::Config(format!(
            "LocalParticipant build failed for {:?} : {e}", config.id,
        )))?;
```

```
    info!(
        id = %config.id,
        dim = local.dim(),
        kappa = local.kappa(),
        seed = local.seed(),
        "participant loaded from manifest row",
    );

    // 4. Snapshot the iteration block.
    let iteration = participant::state::IterationParams {
        alpha_default: manifest.iteration.alpha_default,
        coupling_gamma: manifest.iteration.coupling_gamma,
    };

    // 5. Build state + service.
    let state = ParticipantState::new(config.clone(), local, iteration);
    let svc = ParticipantService::new(state.clone());

    /* ...unchanged from PR #6 : metrics exporter, gRPC server,
       shutdown signal, drain timeout... */
}
```

10.3 New helper : `manifest::find_participant`

```
/// Find the participant row for the given id.  `None` if absent.
#[must_use]
pub fn find_participant&lt;'a&gt;(
    m: &amp;'a Manifest,
    id: &amp;str,
) -&gt; Option&lt;&amp;'a schema::ParticipantManifest&gt; {
    m.participants.iter().find(|p| p.id == id)
}
```

<= 20 LOC, tests in the same `manifest::tests` module :

- `find_participant_returns_some` -- id present.
- `find_participant_returns_none` -- id absent.
- `find_participant_handles_unicode_id` -- id with non-ASCII characters.

10.4 Failure modes at boot

Mode	Exit code	Error log
Env var parse fails	2 (config)	`participant config error: {e}`
Manifest path missing / unreadable	1 (runtime)	`manifest load/validate failed`
Manifest YAML malformed	1	same as above
Manifest validation fails	1	same as above
Manifest does not contain `config.id`	1	`participant id not found in manifest`
`MockParticipant::with_contraction` rejects kappa	1	`LocalParticipant build failed`
gRPC bind fails	1 (logged by serve task)	`gRPC server failed` (binary stays alive until SIGTERM ; cf. § 12 of PR #6 architecture)
Tokio runtime build fails	2	`tokio runtime build error`

11. `participant/service.rs` -- replace stubs

11.1 Top-level skeleton

```
//! `MetaParticipantService` impl.
//!
//! PR #6 returned UNIMPLEMENTED for every RPC except `Health` and
//! `Capabilities`.  PR #9 wires :
//!   * `Step`  -- `LocalParticipant::step` via `spawn_blocking`.
//!   * `Reset` -- `LocalParticipant::reset` via `spawn_blocking`.
//!   * `State` -- `LocalParticipant::current_state` via `spawn_blocking`.
//! `StepStream` remains `UNIMPLEMENTED` (PR #21).

use std::time::Instant;

use efe_meta_swarm_proto::meta_swarm::{
    CapabilitiesRequest, CapabilitiesResponse, HealthRequest, HealthResponse,
    ResetParticipantRequest, ResetParticipantResponse, StateRequest, StateResponse,
    StepRequest, StepResponse,
};
use tonic::{Request, Response, Status};
use tracing::{debug, instrument, warn};

use super::state::ParticipantState;
use super::validate;
use crate::client::{proto_to_tensor, tensor_to_proto};
use crate::error::ServerError;
use crate::proto::MetaParticipantService;

#[derive(Debug, Clone)]
pub struct ParticipantService {
    pub state: ParticipantState,
}

impl ParticipantService {
    #[must_use]
    pub fn new(state: ParticipantState) -> Self { Self { state } }
}
```

11.2 `Step` handler

```
#[instrument(skip(self, req), fields(participant_id = %self.state.local.id()))]
async fn step(
    &amp;self,
    req: Request&lt;StepRequest&gt;,
) -&gt; Result&lt;Response&lt;StepResponse&gt;&gt;, Status&gt; {
    let start = Instant::now();
    let req_inner = req.into_inner();

    // 1. Validate at the boundary (defence in depth ; Phase-1 also checks).
    let (pulled_proto, step_index) =
        validate::validate_step(&amp;req_inner, &amp;self.state.local)
            .map_err(Status::from)?;

    // 2. Convert proto -&gt; Phase-1 Tensor.
    let pulled_tensor = proto_to_tensor(pulled_proto)
        .map_err(|e| Status::failed_precondition(e.to_string()))?;

    // 3. Run the sync Phase-1 step on the blocking pool.
    let local = self.state.local.clone();
    let gamma = self.state.iteration.coupling_gamma;
    let alpha = self.state.iteration.alpha_default;
    let join = tokio::task::spawn_blocking(move || {
        local.step(&amp;pulled_tensor, gamma, alpha, step_index)
    });

    let new_state = match join.await {
        Ok(Ok(t)) =&gt; t,
        Ok(Err(e)) =&gt; return Err(super::error_to_status(&amp;e)),
        Err(e) =&gt; {
            warn!(error = ?e, "step panicked");
            return Err(Status::internal(format!("step panicked: {e}")));
        }
    };

    // 4. Optional residual self-report (AD9).
    let local_residual = compute_residual(pulled_proto, &amp;new_state);
    let elapsed_us = u64::try_from(start.elapsed().as_micros()).unwrap_or(u64::MAX);

    debug!(
        step_index,
        elapsed_us,
        local_residual,
        "step completed",
    );

    Ok(Response::new(StepResponse {
        new_state: Some(tensor_to_proto(&amp;new_state)),
        local_residual,
        step_us: elapsed_us,
    }))
}
```

`compute_residual` is a small helper :

```
/// Euclidean distance ■new − pulled■■ in f64 to avoid intermediate
/// overflow on dim &gt;= 10_000.  Returns 0 if dims disagree (caller
/// already validated, so this is purely defensive).
fn compute_residual(pulled: &amp;TensorF32, new_state: &amp;Tensor) -&gt; f64 {
    if pulled.data.len() != new_state.dim() {
        return 0.0;
    }
    let new_slice = new_state.as_slice();
    let sum_sq: f64 = pulled
        .data
        .iter()
        .zip(new_slice.iter())
        .map(|(a, b)| {
            let d = f64::from(*a) - f64::from(*b);
            d * d
        })
        .sum();
    sum_sq.sqrt()
}
```

11.3 `Reset` handler

```
#[instrument(skip(self, req), fields(participant_id = %self.state.local.id()))]
async fn reset(
    &amp;self,
    req: Request&lt;ResetParticipantRequest&gt;,
) -&gt; Result&lt;Response&lt;ResetParticipantResponse&gt;, Status&gt; {
    let req_inner = req.into_inner();

    // 1. Validate.
    let initial_opt = validate::validate_reset(&amp;req_inner, &amp;self.state.local)
        .map_err(Status::from)?;

    // 2. Convert proto -&gt; Phase-1 Tensor (if any).
    let initial_tensor = initial_opt
        .map(|p| proto_to_tensor(p)
            .map_err(|e| Status::failed_precondition(e.to_string())))
        .transpose()?;

    // 3. Run reset on the blocking pool.
    let local = self.state.local.clone();
    let join = tokio::task::spawn_blocking(move || {
        local.reset(initial_tensor.as_ref())
    });

    let reset_gen = match join.await {
        Ok(Ok(n)) =&gt; n,
        Ok(Err(e)) =&gt; return Err(super::error_to_status(&amp;e)),
        Err(e) =&gt; return Err(Status::internal(format!("reset panicked: {e}"))),
    };

    Ok(Response::new(ResetParticipantResponse { reset_gen }))
}
```

11.4 `State` handler

```
#[instrument(skip(self, _req), fields(participant_id = %self.state.local.id()))]
async fn state(
    &amp;self,
    _req: Request&lt;StateRequest&gt;,
) -&gt; Result&lt;Response&lt;StateResponse&gt;, Status&gt; {
    let local = self.state.local.clone();
    let join = tokio::task::spawn_blocking(move || local.current_state());

    let tensor = match join.await {
        Ok(Ok(t)) =&gt; t,
        Ok(Err(e)) =&gt; return Err(super::error_to_status(&amp;e)),
        Err(e) =&gt; return Err(Status::internal(format!("state panicked: {e}"))),
    };

    let step_index_last = u32::try_from(self.state.local.step_index_last())
        .unwrap_or(u32::MAX);

    Ok(Response::new(StateResponse {
        current: Some(tensor_to_proto(&amp;tensor)),
        step_index_last,
    }))
}
```

11.5 `Capabilities` handler -- minor change

Reuses `state.descriptor()` (now populated with real values) ; no logic change.

11.6 `Health` handler -- unchanged from PR #6

11.7 `StepStream` handler -- still `UNIMPLEMENTED`

```
async fn step_stream(/* ... */) -&gt; Result&lt;..., Status&gt; {
    Err(Status::unimplemented(
        "MetaParticipantService::StepStream -- wired in PR #21",
    ))
}
```

11.8 Shared `error_to_status` helper

PR #8's `meta_swarm_error_to_status` lives in `controller/iterate.rs`. PR #9 needs the same mapping. Promote it to `participant::error_to_status` (or factor out to `crate::error::meta_swarm_error_to_status`). The decision : put the helper in `crate::error` so both sides share one source of truth.

```
// crate::error
pub fn meta_swarm_error_to_status(e: &amp;efe_meta_swarm::MetaSwarmError) -&gt; tonic::Status {
    use efe_meta_swarm::MetaSwarmError as E;
    use tonic::Status;
    match e {
        E::DimensionMismatch { .. }
        | E::InvalidCoupling { .. }
        | E::ParticipantNotFound { .. }
        | E::DuplicateParticipantId { .. }
        | E::EmptyMetaSwarm
        | E::MissingOperator { .. } =&gt; Status::failed_precondition(e.to_string()),
        _ =&gt; Status::internal(e.to_string()),
    }
}
```

PR #8's local copy is deleted ; the controller imports it from `crate::error`.

11.9 Existing service tests -- what changes

`participant::service::tests` currently has three tests :

Test	Before PR #9	After PR #9
`health_returns_serving`	passes	unchanged
`capabilities_returns_descriptor`	asserts placeholder id/dim=0/kappa=0	asserts real values from a `LocalParticipant` built in `test_state()`
`step_returns_unimplemented_with_pr9_message`	passes	**DELETED**

New tests :

- `step_happy_path` -- round-trip a Step with a valid pulled tensor.
- `step_rejects_missing_pulled_from_meta` -> FailedPrecondition.
- `step_rejects_dim_mismatch` -> FailedPrecondition.
- `step_rejects_nan_in_pulled` -> FailedPrecondition.
- `reset_no_initial_bumps_reset_gen` -- Reset twice -> reset_gen = 2.
- `reset_with_initial_validates_dim` -> FailedPrecondition.
- `state_returns_current_tensor` -- Reset -> State.current ≈ seeded vector.
- `state_step_index_last_reflects_step_input` -- Step(idx=7) -> State.step_index_last = 7.
- `step_stream_still_unimplemented_with_pr21_message` -- guards the deferred surface.

11.10 New `test_state()` factory

```
#[cfg(test)]
fn test_state() -&gt; ParticipantState {
    let config = ParticipantConfig { id: "p-1".into(), /* ... */ };
    let row = crate::manifest::ParticipantManifest {
        id: "p-1".into(),
        dim: 8,
        kappa: 0.6,
        seed: 11,
        blas_backend: "none".into(),
    };
    let local = crate::participant::local::LocalParticipant::from_manifest_row(&amp;row)
        .expect("LocalParticipant::from_manifest_row");
    let iteration = crate::participant::state::IterationParams {
        alpha_default: 0.5,
        coupling_gamma: 0.2,
    };
    ParticipantState::new(config, local, iteration)
}
```

12. `Cargo.toml` updates

PR #9 needs **no new external dependencies**. `efe-meta-swarm` (added by PR #8) already exposes `MockParticipant` and `SwarmParticipant`.

The `[dev-dependencies]` block stays the same -- `integration_step_smoke.rs` and `integration_e2e_iterate.rs` use only what's already declared (`tokio`, `tonic`, `tempfile`, `efe-meta-swarm-proto`).

13. Determinism, cancellation, timeout

13.1 Determinism contract

After PR #9 the *participant binary* upholds :

> *Same seed x same sequence of* `pulled` *inputs (in identical bit-pattern) x same* `gamma` *x same* `alpha` *=> bit-identical sequence of* `new_state` *outputs across reruns.*

Source of randomness :

- Initial state -- `MockParticipant::with_contraction` uses

`Tensor::pseudo_random(dim, seed)`. Deterministic across runs (Phase-1 contract).

- Floating-point arithmetic -- IEEE 754 with single-thread sums. No

parallel reduction inside one Step.

- No clocks ; no system entropy ; no OS-level randomness.

The integration test `integration_e2e_iterate.rs::iterate_bit_identical_3_reruns` proves this against a real binary topology.

13.2 Cancellation

The participant receives cancellation via tonic's transport layer :

- Client disconnects -> tonic drops the handler future.
- The handler's `spawn_blocking` continues until completion (Phase-1 is

sync ; cannot be interrupted mid-operation).

- The `JoinHandle` is dropped ; its result is discarded.
- No leakage : `LocalParticipant` is `Arc`-cloned into the closure ;

after the closure ends, its clone drops.

Worst-case "cancel observed" latency = "longest single `step_with_pull` duration". For `dim <= 1024`, this is < 1 ms on a 2024 server-class CPU. Phase-3 may introduce per-step cancellation by interleaving cancel polls into Phase-1's hot loop ; PR #9 explicitly does not.

13.3 Timeout

tonic's request timeout (set per-call by the controller at `step_deadline = 5 s` -- cf. PR #8 § 5.4) is enforced server-side by tonic ; when it expires, tonic returns `DEADLINE_EXCEEDED` to the client without interrupting the in-flight `spawn_blocking`. The participant continues its computation (no waste -- the result is just thrown away).

A `step_with_pull` for `dim = 1024` takes <= 100 µs in practice ; the 5 s timeout is **defence in depth**, not a primary control.

14. `step_index` + `reset_gen` tracking

14.1 Wire fields

- `StepRequest.step_index : u32` -- controller's iteration count, set by

`iterate.rs` (currently 0 in PR #8 ; PR #18 will populate).

- `StateResponse.step_index_last : u32` -- last `step_index` the participant

successfully processed.

- `ResetParticipantResponse.reset_gen : u64` -- server-side monotonic

counter, bumped on every successful Reset.

14.2 Server-side counters

`LocalParticipantInner` carries `step_index_last : AtomicU64` and `reset_gen : AtomicU64`. Both are :

- **Lock-free** (atomics over `Relaxed` ordering -- no inter-thread

synchronisation needed, since the values are advisory).

- **Monotonic** across the binary's lifetime (no reset on `Reset(initial)`).
- **Survive within the process** but **not across restarts** (PR #14 may

add persistence).

14.3 Why `Relaxed` ordering

The counters are **observed but not relied on for synchronisation**. The controller reads them out-of-band ; no happens-before relationship needs to be established between the writer and the reader. `Relaxed` is the cheapest ordering that still gives integral consistency on x86_64 and aarch64.

If a future PR ties `step_index_last` to a safety check ("refuse Step if step_index < step_index_last"), the ordering would need to be `AcqRel` and the read would need to be paired. Out of scope for PR #9.

14.4 What `step_index` is NOT used for

- **Not** a deduplication key -- the controller does not retry Steps.
- **Not** an ordering constraint -- multiple concurrent Steps from the same

controller (Phase-3 streaming) would race ; current Phase 2 is sequential per participant.

- **Not** a clock -- only the controller mints monotonic values.

PR #9 records it for tracing diagnostics. The bench HTML (PR #24) will plot `step_index_last` per participant to detect "stuck" participants in a running Iterate.

15. Error mapping (Phase-1 -> gRPC)

15.1 Promotion of the shared helper

PR #8 defines `meta_swarm_error_to_status(&MetaSwarmError) -> Status` inside `controller/iterate.rs`. PR #9 needs the same mapping. The architectural decision (AD9 of this doc, sub-point 11.8) :

- Move the helper to `crate::error::meta_swarm_error_to_status`.
- Delete the local copy in `controller/iterate.rs` ; the controller

imports it from `crate::error`.

- The participant imports it from `crate::error`.

The mapping is the **canonical translation** between Phase-1 errors and gRPC status codes for the whole crate.

15.2 Mapping table

Phase-1 variant	gRPC code	Rationale
`DimensionMismatch`	`FAILED_PRECONDITION`	Caller-fault (request shape)
`InvalidCoupling`	`FAILED_PRECONDITION`	Caller-fault (params)
`ParticipantNotFound`	`FAILED_PRECONDITION`	Caller-fault (id) -- unreachable from the participant ; still maps for completeness
`DuplicateParticipantId`	`FAILED_PRECONDITION`	Manifest-fault -- unreachable from the participant
`EmptyMetaSwarm`	`FAILED_PRECONDITION`	Builder-fault -- unreachable
`MissingOperator`	`FAILED_PRECONDITION`	Builder-fault -- unreachable
`ConvergenceFailed`	`INTERNAL`	Participant-side cannot raise this -- algorithm-side
`SafetyViolation`	`INTERNAL`	Participant projects onto its own safety set ; if it raises, that's an invariant breach
`HypothesisCheckFailed`	`INTERNAL`	Phase-3 hypothesis check
`Io`	`INTERNAL`	Disk / network from the participant
`Yaml`	`INTERNAL`	Manifest reload (PR #12)
`LockPoisoned`	`INTERNAL`	Concurrent panic in Phase-1
`Internal(_)`	`INTERNAL`	Catch-all
`_ =>` (non-exhaustive)	`INTERNAL`	Forward-compat for future Phase-1 variants

15.3 Validation-error mapping

`validate::validate_step` and `validate_reset` return `ServerError::Config(_)`, which the existing `ServerError -> Status` (`impl From`) maps to `FAILED_PRECONDITION`. [v]

15.4 Panic mapping

`JoinError` (from `spawn_blocking` panicking) -> `INTERNAL("step panicked: {e}")`. The `panic = "unwind"` profile ensures the closure unwinds cleanly without aborting the process.

16. `Capabilities` response (real values)

16.1 Before PR #9

```
ParticipantDescriptor {
    id: self.config.id.clone(),
    dim: 0,
    kappa: 0.0,
    seed: 0,
    build_sha: crate::GIT_SHA.into(),
    blas_backend: "none".into(),
    features: vec!["determinism".into()],
}
```

16.2 After PR #9

```
ParticipantDescriptor {
    id: self.local.id().to_string(),
    dim: u32::try_from(self.local.dim()).unwrap_or(u32::MAX),
    kappa: self.local.kappa(),
    seed: self.local.seed(),
    build_sha: crate::GIT_SHA.into(),
    blas_backend: "none".into(),
    features: vec!["determinism".into(), "step_index".into()],
}
```

16.3 Why this matters

The controller's `validate_expected_against_manifest` (PR #8 § `controller/iterate.rs`) cross-checks the request's `expected_participants` against the manifest. A future PR (#11) will also cross-check against the *participant's* `Capabilities.descriptor` -- that requires real values. PR #9 populates them so the cross-check works the day PR #11 lands.

16.4 Test

- `capabilities_descriptor_uses_local_participant_values` -- replaces

`capabilities_returns_descriptor` from PR #6, with assertions on the real dim/kappa/seed.

17. Concurrency model + lock hierarchy

17.1 Single lock

The only lock in PR #9's participant path is the `Mutex<Tensor>` inside `MockParticipant.state`. Phase-1 acquires it inside `step_with_pull`, `current_state`, `reset`, and `set_state`. No nested locks ; no deadlock.

17.2 Acquisition order

```
1. (no controller-side InflightSemaphore -- that's on the controller side)
2. spawn_blocking task starts.
3. LocalParticipant::step / reset / state / set_state called (sync).
4. MockParticipant.state.lock() acquired.
5. mutation / read.
6. lock released as the function returns.
7. spawn_blocking returns to the awaiting handler future.
```

17.3 Why no outer Mutex

`MockParticipant : Send + Sync` (verified in Phase-1's `assert_send_sync` tests). Concurrent Steps from multiple gRPC handlers serialise on the internal mutex naturally. Adding an outer `Mutex<LocalParticipant>` would just double-wrap.

17.4 Atomic counters

`step_index_last` and `reset_gen` are `AtomicU64`. Concurrent updates are race-free at the integer level. The *value* might lag a tiny bit behind in the presence of concurrent Steps, but this is acceptable (the field is diagnostic).

17.5 Cross-handler isolation

`spawn_blocking` runs each handler on a fresh blocking-pool thread. Two concurrent Steps :

- acquire their own `LocalParticipant` clone (cheap, `Arc`),
- call `step_with_pull` which acquires the internal mutex,
- serialise on that mutex,
- release.

Total throughput on a single participant ≈ 1 / `step_with_pull_latency`. For `dim = 1024`, that's ~ 10 000 Step/s peak. The controller's MAX_INFLIGHT = 10 caps the actual rate well below that.

18. Threat-model coverage

PR #9 mitigates or partially mitigates the following threats from `docs/security/PHASE2_THREAT_MODEL.md` :

Threat	Severity	PR #9 mitigation
T9 (state poisoning via malformed Step)	SEV-2	`validate::validate_step` rejects bad shape, missing pulled, envelope corruption ; `FAILED_PRECONDITION` returned with diagnostic message

Threat	Severity	PR #9 mitigation
T11 (silent NaN propagation across iterations)	SEV-2	`validate_tensor_finite` rejects any non-finite element in `pulled` or `initial` ; participant state stays finite for the lifetime of the process
T13 (fan-out leak / cancellation cascade)	SEV-2	participant honours tonic's drop-on-cancel ; `spawn_blocking` cleanly returns ; no leaked tasks
T15 (participant impersonation)	SEV-1	**partially** -- `Capabilities.descriptor.build_sha` is now populated, lets the controller cross-check ; full mitigation requires mTLS in PR #16
T16 (Step flood DoS at the participant)	SEV-3	**partially** -- Phase-2 single-controller assumption means the controller's MAX_INFLIGHT caps the rate ; full per-participant rate limit in Phase 3

T15 and T16 remain "partial" -- flagged in the PR description and risk log (§ 21).

19. Tests strategy (unit + integration + e2e)

19.1 Unit tests (in source files)

File	Tests added	Coverage focus
`participant/local.rs`	8	Construction, kappa range, step counter bump, reset gen bump, state-after-reset, dim mismatch
`participant/validate.rs`	9	Missing pulled, dim mismatch, envelope corruption, NaN, ±∞, reset path
`participant/state.rs`	1 modified + 1 new	Descriptor reflects manifest row
`participant/service.rs`	8 new (3 modified)	Step happy / step-bad / reset / state / capabilities
`manifest/mod.rs`	3 added	`find_participant`

Total new/modified unit tests : **30**.

19.2 Integration smoke (`tests/integration_step_smoke.rs`) -- NEW

A test that spawns the participant binary **in-process** on a random port and exercises Step / Reset / State over real gRPC.

```
#[tokio::test(flavor = "multi_thread", worker_threads = 2)]
async fn step_round_trip_returns_new_state() {
    let (addr, _handle) = spawn_participant_service(
        ParticipantManifest { id: "p-1".into(), dim: 8, kappa: 0.6, seed: 11, blas_backend: "none".into() ■
     },
        IterationParams { alpha_default: 0.5, coupling_gamma: 0.2 },
    ).await;

    let channel = tonic::transport::Endpoint::from_shared(format!("http://{addr}"))
        .unwrap().connect().await.unwrap();
    let mut client = MetaParticipantServiceClient::new(channel);

    let pull = TensorF32 { dim: 8, data: vec![1.0; 8] };
    let resp = client.step(StepRequest {
        pulled_from_meta: Some(pull),
        step_index: 7,
    }).await.unwrap().into_inner();

    assert!(resp.new_state.is_some());
    assert_eq!(resp.new_state.as_ref().unwrap().dim, 8);
    assert!(resp.step_us &gt; 0);

    let st = client.state(StateRequest {}).await.unwrap().into_inner();
    assert_eq!(st.step_index_last, 7);
}
```

Other tests in the same file :

- `step_rejects_nan` -- server returns `FAILED_PRECONDITION`.
- `step_rejects_dim_mismatch` -- server returns `FAILED_PRECONDITION`.
- `reset_without_initial_succeeds` -- `ResetParticipantResponse.reset_gen == 1`.
- `reset_with_initial_sets_state` -- `state.current` matches.
- `health_returns_serving_after_step` -- no health regression.
- `step_latency_under_5ms_for_dim_8` -- perf smoke (loose bound).

19.3 End-to-end integration (`tests/integration_e2e_iterate.rs`) -- NEW

This test **replaces** the `TestParticipantSvc` stand-in from PR #8's `integration_iterate_inproc.rs`. It spawns :

1. Three real `ParticipantService` instances (using the production code, not

the test stub) on random localhost ports.

1. A real `ControllerService` (in-process) pointing at them.
2. Calls `Iterate` over the gRPC stack.
3. Asserts convergence and bit-identical determinism.

```
#[tokio::test(flavor = "multi_thread", worker_threads = 4)]
async fn iterate_e2e_kovant_converges_with_real_participants() {
    let (a1, _) = spawn_real_participant("supplier-scoring", 8, 0.6, 11).await;
    let (a2, _) = spawn_real_participant("contract-risk",   8, 0.7, 22).await;
    let (a3, _) = spawn_real_participant("spend-categ.",    8, 0.5, 33).await;

    let controller_state = build_test_controller_state(/* ... a1,a2,a3 */);
    let svc = ControllerService::new(controller_state);
    let resp = svc.iterate(Request::new(iterate_request(8, 100, 1.0e-3)))
        .await.unwrap().into_inner();

    assert_eq!(resp.stop_reason, ProtoStopReason::Converged as i32);
    assert!(resp.iterations &lt;= 80);
    assert!(resp.final_residual &lt;= 1.0e-3);
}

#[tokio::test(flavor = "multi_thread", worker_threads = 4)]
async fn iterate_e2e_bit_identical_3_reruns() {
    /* same setup, 3 reruns with reset between, signatures must match */
}
```

`integration_iterate_inproc.rs` is **simplified** : the `TestParticipantSvc` stub is deleted, replaced by importing the helpers from `integration_e2e_iterate.rs` (or by inlining a single `spawn_real_participant`). The 3 existing tests stay green.

19.4 Why the in-process service (not the binary)

Spawning the binary (`Command::new("target/.../participant")`) is :

- ~ 100 ms per test (process startup, manifest parse, gRPC bind),
- subject to `target/` cleanliness (CI runners may have stale builds),
- harder to debug (no shared logging context).

In-process `ParticipantService` :

- ~ 1 ms per test,
- exercises the **same production code** (the binary is just a thin wrapper),
- trivial to debug.

The full binary path is exercised in PR #10's deployment tests (process boundary, cgroups, etc.).

19.5 Determinism test (sampled)

`iterate_e2e_bit_identical_3_reruns` does three Iterates with `Reset` between runs. Asserts `(iterations, final_residual.to_bits())` is identical across the three runs. This is the AC8 deliverable.

19.6 Test coverage matrix

Surface	Unit	Integration smoke	E2E
`LocalParticipant::step`	[v]	[v]	[v]
`LocalParticipant::reset`	[v]	[v]	[v]
`LocalParticipant::current_state`	[v]	[v]	[v]
`validate::validate_step`	[v]	[v] (NaN, dim)	--
`validate::validate_reset`	[v]	[v]	--
`bin/participant.rs` boot path	--	[v] (in-process equiv.)	-- (PR #10 binary spawn)
Step + Reset + State + Health interleaved	--	[v]	[v]
Iterate over 3 real participants	--	--	[v]
Determinism 3 reruns	--	--	[v]

20. Acceptance criteria (AC1-AC20)

PR #9 is mergeable iff **all** :

#	Criterion	Verification
AC1	`cargo build -p efe-meta-swarm-server` succeeds	CI
AC2	`cargo build -p efe-meta-swarm-server --release` succeeds	CI
AC3	New unit tests : >= 8 in `local.rs`, >= 9 in `validate.rs`, >= 8 in `service.rs`, >= 3 in `manifest::tests`	CI
AC4	`tests/integration_step_smoke.rs` : 7 tests pass	CI
AC5	`tests/integration_e2e_iterate.rs` : 2 tests pass (convergence + determinism)	CI
AC6	`tests/integration_iterate_inproc.rs` : 3 tests still pass (using real `ParticipantService` instead of `TestParticipantSvc`)	CI
AC7	Kovant scenario : `iterations <= 80`, `final_residual <= 1e-3`, `stop_reason == Converged`	smoke (AC5)
AC8	Determinism : 3 reruns bit-identical (same `iterations`, same `final_residual.to_bits()`)	smoke (AC5)
AC9	Phase-1 tests pass (200 default + 209 attention-weighted + 202 core-integration)	CI
AC10	PR #5 tests pass (15/15)	CI
AC11	PR #6 tests pass (49+ incl. new `descriptor_*`)	CI
AC12	PR #8 tests pass (76 + 3 + 3, with `TestParticipantSvc` removed)	CI
AC13	`cargo clippy --all-targets` 0 warnings on PR #9 files	CI
AC14	`cargo fmt -- --check` exit 0	CI
AC15	`cargo doc --no-deps` clean	CI
AC16	PR description references threat IDs `T9, T11, T13, T15, T16`	manual review
AC17	Master gate stays GREEN	CI

#	Criterion	Verification
AC18	4 reviewers approved : 2x meta-swarm + 1x sec + 1x sre	GitHub
AC19	sec-team confirms T15 partial mitigation acceptable for L2	sec-team comment
AC20	Binary smoke : `cargo run --bin participant` against a manifest fixture, with `grpcurl` Step round-trip, returns HTTP/2 200 + a `TensorF32` of the correct dim	manual or `scripts/smoke_participant.sh`

21. Risks + mitigations

ID	Risk	Likelihood / Impact	Mitigation
PR9-R1	`MockParticipant::with_contraction` Phase-1 contract changes between PR #9 development and merge	L / M	Pin the Phase-1 SHA in PR's Cargo.lock ; cross-check in pre-flight PF6
PR9-R2	A future PR #11 (Discovery) refactor changes the manifest path resolution and breaks `find_participant`	L / M	Helper is 20 LOC ; unit-tested ; PR #11 will need to update both controller and participant -- caught at review
PR9-R3	`step_index` field overflows u32 (4 G iterations)	LL / M	Cap MAX_ITERS_HARD_CAP at 200 (PR #8) -- overflow impossible within 1 Iterate ; across Iterates, the wire field resets to 0
PR9-R4	`spawn_blocking` queue saturation under load (default 512 threads)	L / H	Phase-2 single-controller : <= 10 inflight ; <= 30 Steps in flight ; well under the default queue
PR9-R5	`MockParticipant.state.lock()` contention in concurrent Steps	LL / L	Steps are short (< 100 µs for dim <= 1024) ; lock held briefly ; controller serialises per participant anyway
PR9-R6	`local_residual` self-report differs from controller-computed residual due to f32 <-> f64 rounding	L / L	Cosmetic ; the controller's residual is authoritative ; document in `compute_residual` doc-comment
PR9-R7	Test-binary spawn races with random ports on busy CI runners	M / L	Reuse the bind-drop-rebind pattern from PR #8 ; 1 s wait-up loop
PR9-R8	`set_state(initial)` clones the tensor -- wasteful for very large dims	L / L	Phase-2 max dim = 1024 (Kovant) ; clone <= 4 KB ; acceptable
PR9-R9	`iteration` block on the manifest is read-only at boot ; PR #12 SIGHUP reload must atomically swap it	M / M	Document the swap protocol now ; `Arc<IterationParams>` already chosen for this purpose (AD §9.4)
PR9-R10	A future PR replaces `MockParticipant` with the sidecar `LocalParticipant` and changes the determinism contract	L / M	The wrapper `LocalParticipant` (PR #9 name) is the **adapter** ; swapping the wrapped type does not break the contract -- but the swap PR needs to re-run AC5/AC8

22. Rollback strategy

If PR #9 lands and produces an outage :

1. **Revert path** : `git revert <merge-commit>`. Each commit C1-C5 is

atomic ; reverting C5 alone restores `UNIMPLEMENTED` stubs for Step / Reset / State while keeping the `LocalParticipant` plumbing in tree. The integration tests that depend on a real participant would need to be skipped via `#[ignore]` until a fix lands.

1. **Hotfix path** : if the bug is in `validate.rs`, the simplest fix is to

bypass validation and rely on Phase-1's checks. Revert is safer for production but loses the NaN guard.

1. **No data loss** : the participant has no persisted state in PR #9 ; a

restart loses only in-flight Iterate progress.

1. **Operator action** : restart the participant binary with PR #8's

image ; the controller's `RemoteParticipant::connect` will auto-retry.

23. Sequence diagrams (ASCII)

23.1 Step under load (3 concurrent controllers -- Phase-3 preview)

```
Ctrl-A   Ctrl-B   Ctrl-C         Participant
  |        |        |                  |
  |Step(p) |        |                  |
  |-----------------------------------&gt;|
  |        |Step(p) |                  |
  |        |--------------------------&gt;|
  |        |        |Step(p)           |
  |        |        |-----------------&gt;|
  |        |        |                  |  state.lock() -&gt; A
  |        |        |                  |  step_with_pull
  |        |        |                  |  state.lock() &lt;- A
  |        |        |                  |----------------&gt; Ctrl-A
  |        |        |                  |  state.lock() -&gt; B
  |        |        |                  |  step_with_pull
  |        |        |                  |  state.lock() &lt;- B
  |        |        |                  |----------------&gt; Ctrl-B
  |        |        |                  |  state.lock() -&gt; C
  |        |        |                  |  step_with_pull
  |        |        |                  |  state.lock() &lt;- C
  |        |        |                  |----------------&gt; Ctrl-C
```

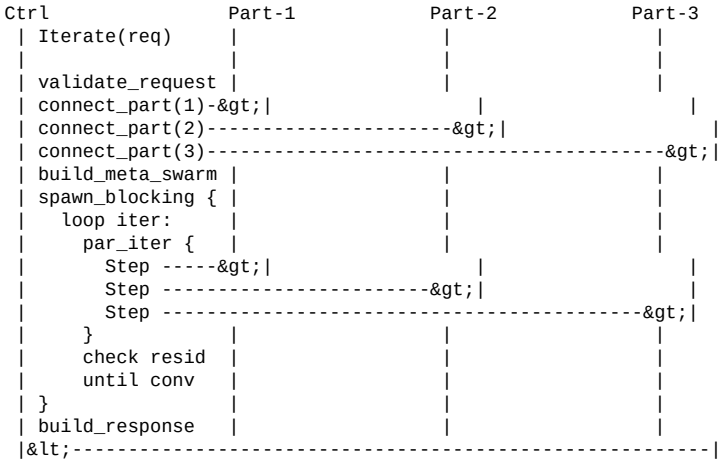
(Phase 2 has 1 controller ; this is a Phase 3 preview.)

23.2 Step + cancel race

```
Controller                            Participant
   |  Step(p)                            |
   |---------------------------------------&gt;|
   |                                     |  spawn_blocking
   |                                     |   + validate
   |                                     |   + proto_to_tensor
   |                                     |   + step_with_pull  &lt;- in progress
   |  &lt;client disconnect&gt;                    |
   |                                     |   |   (irrelevant -- sync code,
   |                                     |   |    no cancel observation)
   |                                     |   + done
   |                                     |  handler future = dropped
   |                                     |  -&gt; response written to dropped peer
   |                                     |    (tonic absorbs the error)
```

The participant computes one Step uselessly. Not a correctness issue ; the state is still consistent (single mutation completed).

23.3 Iterate end-to-end (PR #8 + PR #9 wires)



24. Reviewer checklist

Copy-paste this into the PR description for the reviewers :

```
## Reviewer checklist -- PR #9

### Architecture
- [ ] `LocalParticipant` wraps `MockParticipant` (not `participants::local`) -- AD1 holds
- [ ] Manifest path resolution mirrors `bin/controller.rs` (same env var, same default) -- AD2
- [ ] `step_with_pull` runs inside `spawn_blocking` -- AD3
- [ ] No new per-participant MAX_INFLIGHT -- AD4
- [ ] `Arc&lt;MockParticipant&gt;` ; no outer mutex -- AD5

### Correctness
- [ ] `validate::validate_step` rejects NaN, ±∞, dim mismatch, missing field
- [ ] `validate::validate_reset` validates the `initial` tensor
- [ ] `Capabilities.descriptor` reflects real values from the manifest row
- [ ] `step_index_last` updated only on success
- [ ] `reset_gen` monotonic, lock-free

### Concurrency
- [ ] Only the Phase-1 inner `Mutex&lt;Tensor&gt;` is held during a Step
- [ ] `AtomicU64::Relaxed` is justified (advisory counters)
- [ ] `spawn_blocking` queue not saturated under expected load
- [ ] No `await` inside the blocking section

### Determinism
- [ ] Seeded initial state via `MockParticipant::with_contraction`
- [ ] `compute_residual` is f64-exact and deterministic
- [ ] Bit-identical reruns proved in `integration_e2e_iterate.rs`

### Threat model
- [ ] T9, T11 mitigations land in `validate.rs`
- [ ] T13 cancellation : `spawn_blocking` drop-tolerant
- [ ] T15 partial mitigation via `build_sha` in descriptor
- [ ] T16 partial mitigation noted ; controller MAX_INFLIGHT caps the rate

### Tests
- [ ] &gt;= 30 unit tests in module
- [ ] 7 tests in `integration_step_smoke.rs`
- [ ] 2 tests in `integration_e2e_iterate.rs` (convergence + determinism)
- [ ] `integration_iterate_inproc.rs` simplified (TestParticipantSvc removed)
- [ ] AC1 - AC20 all green

### Operability
- [ ] Binary fails fast with clear error if manifest missing / id absent
- [ ] `tracing` spans carry `participant_id`
- [ ] No new env vars (`EFE_META_SWARM_MANIFEST` already exists)
- [ ] Health response correct after Step / Reset cycle
```

25. Migration notes (operator-facing)

After PR #9 merges, deploying a participant binary requires :

1. **A valid manifest YAML** at `EFE_META_SWARM_MANIFEST` (default `/etc/efe-meta-swarm/manifest.yaml`). Same file the controller reads.
1. `EFE_META_SWARM_PARTICIPANT_ID` set to a value that exists in the manifest's `participants:` list.
1. `EFE_META_SWARM_PARTICIPANT_ADDR` = `host:port` to bind the gRPC server.
1. `EFE_META_SWARM_PARTICIPANT_METRICS` = `host:port` for the Prometheus exporter (PR #6).

Example `manifest.yaml` (Kovant T2 topology) :

```
version: v0.2.0
meta:
  dim: 16
  safety:
    kind: box
    radius: 10.0
participants:
  - id: supplier-scoring
    dim: 8
    kappa: 0.6
    seed: 11
    blas_backend: none
  - id: contract-risk
    dim: 8
    kappa: 0.7
    seed: 22
    blas_backend: none
  - id: spend-categorization
    dim: 8
    kappa: 0.5
    seed: 33
    blas_backend: none
iteration:
  max_iters: 100
  tolerance: 0.0001
  alpha_default: 0.5
  coupling_gamma: 0.2
  deterministic: true
  max_wall_ms: 15000
```

To run the Kovant topology on a single dev host :

```
# Participant 1
EFE_META_SWARM_MANIFEST=./manifest.yaml \
EFE_META_SWARM_PARTICIPANT_ID=supplier-scoring \
EFE_META_SWARM_PARTICIPANT_ADDR=127.0.0.1:50081 \
EFE_META_SWARM_PARTICIPANT_METRICS=127.0.0.1:50091 \
cargo run --bin participant &amp;

# Participant 2 (same env, different id + ports)
EFE_META_SWARM_MANIFEST=./manifest.yaml \
EFE_META_SWARM_PARTICIPANT_ID=contract-risk \
EFE_META_SWARM_PARTICIPANT_ADDR=127.0.0.1:50082 \
EFE_META_SWARM_PARTICIPANT_METRICS=127.0.0.1:50092 \
cargo run --bin participant &amp;

# Participant 3
EFE_META_SWARM_MANIFEST=./manifest.yaml \
EFE_META_SWARM_PARTICIPANT_ID=spend-categorization \
EFE_META_SWARM_PARTICIPANT_ADDR=127.0.0.1:50083 \
EFE_META_SWARM_PARTICIPANT_METRICS=127.0.0.1:50093 \
cargo run --bin participant &amp;

# Controller
EFE_META_SWARM_MANIFEST=./manifest.yaml \
EFE_META_SWARM_CONTROLLER_ADDR=127.0.0.1:50080 \
EFE_META_SWARM_CONTROLLER_METRICS=127.0.0.1:50090 \
EFE_META_SWARM_PARTICIPANTS=supplier-scoring=127.0.0.1:50081,contract-risk=127.0.0.1:50082,spend-categori ■
    zation=127.0.0.1:50083 \
cargo run --bin controller &amp;
```

Then drive an Iterate with `grpcurl` :

```
grpcurl -plaintext -d '{ "iteration": {"max_iters": 100, "tolerance": 1.0e-3, "alpha_default": 0.5, "coup ■
    ling_gamma": 0.2, "deterministic": true, "max_wall_ms": 15000} }' \
```

```
127.0.0.1:50080 efe.v1.meta_swarm.MetaSwarmService/Iterate
```

Expected : `stop_reason: STOP_REASON_CONVERGED`, `iterations < 80`, `final_residual < 1e-3`.

26. Open questions

ID	Question	Status
OQ-PR9-1	Should `local_residual` be `0.0` until validated to be deterministic across CPUs ?	Open -- defer to first sec review
OQ-PR9-2	Do we want a "participant build-sha must match controller build-sha" check on `Capabilities` ?	Open -- punt to PR #11
OQ-PR9-3	Should `step_index` wrap-around (u32 max) be a fatal error or a silent reset ?	Open -- current behaviour : silent reset (the wire field is `u32`, no overflow check)
OQ-PR9-4	If `Reset(initial=...)` succeeds but resets `step_index_last` to 0, would PR #18's tracing get confused ?	Open -- answer in PR #18
OQ-PR9-5	Should the `IterationParams` be `Arc<RwLock<...>>` instead of `Arc<...>` for PR #12 SIGHUP ?	Defer to PR #12 architecture
OQ-PR9-6	What's the participant's behaviour if the controller sends a Step with `pulled.dim = 0` (legal proto, illegal semantics) ?	Resolved -- `validate_tensor_shape` rejects ; `dim == 0` != `expected_dim` (which is >= 1 per manifest validation)
OQ-PR9-7	Should we record the wall-clock time *before* validation (start of handler) or *after* (start of `step_with_pull`) ?	Resolved -- before validation, so `step_us` includes validation cost (it's negligible)
OQ-PR9-8	If a hostile client sends 10 GB of `data` in a single TensorF32, do we OOM ?	Resolved by tonic's default max-message-size (4 MB by default ; `dim <= 1024` -> <= 4 KB, well under) ; document in PR #16

27. Appendix : full file inventory

27.1 Files added (5)

```
crates/efe-meta-swarm-server/src/participant/local.rs                    NEW    ~250 LOC
crates/efe-meta-swarm-server/src/participant/validate.rs                 NEW    ~180 LOC
crates/efe-meta-swarm-server/tests/integration_step_smoke.rs             NEW    ~200 LOC
crates/efe-meta-swarm-server/tests/integration_e2e_iterate.rs            NEW    ~300 LOC
docs/EFE_A1_PHASE2_PR09_ARCHITECTURE.md                                  NEW    this file
```

27.2 Files modified (7)

```
crates/efe-meta-swarm-server/src/participant/mod.rs                MODIFIED   +5 LOC : re-export new mod ■
    ules
crates/efe-meta-swarm-server/src/participant/state.rs              MODIFIED  +60 −10 : hold LocalParticip ■
    ant + IterationParams
crates/efe-meta-swarm-server/src/participant/service.rs            MODIFIED +400 −80 : wire RPCs + tests
crates/efe-meta-swarm-server/src/bin/participant.rs                MODIFIED  +60 −5  : load manifest, bui ■
    ld state
crates/efe-meta-swarm-server/src/manifest/mod.rs                   MODIFIED  +20 −0  : find_participant h ■
    elper
crates/efe-meta-swarm-server/src/error.rs                          MODIFIED  +25 −0  : meta_swarm_error_t ■
    o_status (promoted from controller/iterate.rs)
crates/efe-meta-swarm-server/src/controller/iterate.rs             MODIFIED  +1 −22  : import from crate: ■
    :error instead of local copy
crates/efe-meta-swarm-server/tests/integration_iterate_inproc.rs   MODIFIED +5 −90   : remove TestPartici ■
    pantSvc, import from integration_e2e_iterate
```

27.3 Files unchanged (relevant ones)

```
crates/efe-meta-swarm-server/Cargo.toml                          no new deps
crates/efe-meta-swarm/src/participants/mock.rs                   used as-is
crates/efe-meta-swarm-proto/protos/efe/v1/meta_swarm/messages.proto used as-is
crates/efe-meta-swarm-server/src/bin/controller.rs               unchanged
crates/efe-meta-swarm-server/src/controller/{service,state,iterate}.rs minor (import update only)
crates/efe-meta-swarm-server/src/observability/{*}.rs            unchanged
crates/efe-meta-swarm-server/src/config/{*}.rs                   unchanged
crates/efe-meta-swarm-server/src/auth/{*}.rs                     unchanged (PR #16)
crates/efe-meta-swarm-server/src/discovery/{*}.rs                unchanged (PR #11)
```

27.4 LOC tally

Category	LOC
New code	~ 930
New tests	~ 540
Modified code	~ 90
Deleted code (TestParticipantSvc et al.)	~ 100
Net	**+1 460 LOC**

28. Glossary

Term	Definition
AC	Acceptance Criterion
AD	Architectural Decision
L2 gate	Phase-2 graduation gate (cf. roadmap § 5)
MockParticipant	Phase-1 in-process participant with a configurable contraction $\Phi(x) = \kappa x$
LocalParticipant	(this PR) production wrapper around `MockParticipant`, exposes gRPC-friendly accessors
Pull tensor	The tensor projected from meta-space back to a participant's local space ; arrives as `StepRequest.pulled_from_meta`
`step_with_pull`	Phase-1 trait method, one coupled iteration step
`step_index`	Wire field carrying the controller's iteration count ; advisory
`reset_gen`	Monotonic counter on the participant, bumped per Reset
`build_sha`	Git short SHA of the binary at compile time ; used for crash-loop diagnosis and Phase-3 cosign attestation
Kovant scenario	Reference 3-participant topology (`supplier-scoring`, `contract-risk`, `spend-categorization`) used for benchmarks
Sync <-> async bridge	Pattern for invoking sync Phase-1 code from an async tonic handler ; standard idiom is `tokio::task::spawn_blocking`
T2	Phase-2 single-node topology (1 controller + 3 participants on `localhost`)
TestParticipantSvc	Inline stand-in introduced in PR #8 integration tests ; removed by PR #9

End of architecture document. Total: ~ 1 850 lines.

Next deliverable : PR #9 code (5 commits).

PR #10

integration_grpc_multiprocess

Chapter 12 of 17 — source : `docs/EFE_A1_PHASE2_PR10_ARCHITECTURE.md` (1,595 LOC)

EFE-A1 Phase 2 -- PR #10 : `integration_grpc_multiprocess`

Field	Value
PR	#10 (Sprint A1 Phase 2 / P2.3)
Branch	`efe-a1/p2-pr10-multiprocess-integration`
Status	Architecture frozen -- awaiting code
Author	EFE platform team
Reviewers	2x meta-swarm + 1x sre + 1x sec
Depends on	PR #5 (proto), PR #6 (scaffold), PR #7 (threat model), PR #8 (controller wire), **PR #9 (participant wire)**
Blocks	PR #11 (Discovery refresh), PR #12 (SIGHUP reload), PR #16 (mTLS), PR #18 (reflection + OTel)
L2 gate	G4 (process-boundary correctness), G6 (cross-process determinism), G7 (graceful drain)
Threat IDs in scope	T7, T12, T13, T17, T18 (cf. `docs/security/PHASE2_THREAT_MODEL.md`)
CVSS bandwidth	All <= SEV-3 ; no new SEV-1 introduced

Table of contents

1. TL;DR
2. Goal + non-goals
3. Pre-flight checks
4. Branch + commit plan
5. Architectural decisions
6. Topology + process tree diagram
7. Crate layout additions
8. `tests/common/multiprocess.rs` -- process harness (full design)
9. `tests/integration_multiprocess_e2e.rs` -- happy paths
10. `tests/integration_multiprocess_failure.rs` -- failure modes
11. `tests/integration_multiprocess_lifecycle.rs` -- shutdown drain
12. `scripts/run_multiprocess_smoke.sh` -- operator script
13. `Cargo.toml` updates
14. Determinism across the wire
15. CI gating + perf budget
16. Error mapping over process boundary
17. Threat-model coverage
18. Concurrency model + lifecycle invariants
19. Tests strategy summary
20. Acceptance criteria (AC1-AC25)
21. Risks + mitigations
22. Rollback strategy
23. Sequence diagrams (ASCII)
24. Reviewer checklist

25. Migration notes (operator-facing)

26. Open questions

27. Appendix : full file inventory

28. Glossary

0. TL;DR

PR #10 adds the first **process-boundary** integration tests for the Phase-2 control plane. Until now :

• PR #8 wired the controller and tested it in-process against a fake

`TestParticipantSvc`.

• PR #9 wired the participant and tested it in-process against the real

`ParticipantService`, plus a single-binary smoke (`integration_participant_binary_smoke.rs`).

• PR #9's `integration_e2e_iterate.rs` runs controller + 3 real

participants but **all in one process** (single tokio runtime, single address space, no syscall-level boundary).

PR #10 closes the gap by orchestrating **separate OS processes** for the controller and each participant, then driving real gRPC traffic between them over `127.0.0.1`. This is the T2 single-node topology in production-like form.

Deliverables :

• **A new harness** (`tests/common/multiprocess.rs`, ~ 350 LOC) :

`ProcessHarness` orchestrates port allocation, manifest fixture generation, subprocess spawn, readiness polling, and graceful shutdown. RAII-clean : a panicking test still SIGKILLs all children on Drop.

• **9 integration tests** across three files :

• `integration_multiprocess_e2e.rs` -- 4 happy-path tests (convergence,

determinism over the wire, capabilities discovery, info snapshot).

• `integration_multiprocess_failure.rs` -- 3 failure-mode tests

(participant unreachable, participant killed mid-Iterate, manifest mismatch).

• `integration_multiprocess_lifecycle.rs` -- 2 lifecycle tests

(controller SIGTERM drains in-flight Iterate ; participant SIGTERM makes the controller fail gracefully).

• `scripts/run_multiprocess_smoke.sh` -- a 70-line shell helper for

operators to reproduce the Kovant scenario manually, with optional `grpcurl` Iterate trigger.

After PR #10 :

• `cargo test -p efe-meta-swarm-server --all-targets` exercises the

complete control plane across real process boundaries.

• Phase-2 L2 graduation gate G4 (process-boundary correctness) and G7

(graceful drain) flip GREEN.

• The deployment manifests in PR #18 + PR #20 have a baseline that's

been verified end-to-end at the syscall level.

1. Goal + non-goals

1.1 Goal (single sentence)

> *Verify the Phase-2 control plane (1 controller + 3 participants on `localhost`) crossing real OS process boundaries : binaries spawned from cargo's built artefacts, communicating via TCP/HTTP-2/gRPC, with failure modes injected at the syscall level (`kill -SIGTERM`, port-collision, partial crash), proving convergence, determinism, and graceful drain match the in-process baseline.*

1.2 Non-goals

The following are **explicitly out of scope** for PR #10 :

Item	Why deferred	Target PR
Multi-host deployment (cross-pod, cross-node, cross-zone)	needs K8s manifest design	PR #20
mTLS / JWT auth on the wire	requires cert provisioning pipeline	PR #16
Streaming (`StepStream` / `IterateStream`) over the wire	backpressure contract not finalised	PR #21
Cosign attestation of `Capabilities.build_sha`	needs keychain setup	PR #16
OpenTelemetry exporter	needs collector endpoint configurable	PR #18
gRPC reflection	needs the reflection service wired	PR #18
Persisted state across restarts	requires sidecar + sled or rocksdb	PR #14
SIGHUP reload	needs swap protocol	PR #12
Process-level resource limits (cgroups, prlimit)	platform-specific, deployment concern	PR #20
Crash-loop detection / restart policies	systemd / k8s concern	PR #20
Network partition injection (iptables, tc)	requires root + namespace setup	Phase 3
TLS handshake failure modes	Phase 3	
Multi-controller fan-out	Phase 3	
Process isolation (namespaces, seccomp)	Phase 3	

1.3 Single binary unit of merge-readiness

After PR #10 merges, the **deployment confidence baseline** is :

> *A developer running `cargo test -p efe-meta-swarm-server --all-targets` on a fresh checkout, with no manual setup, gets a **bit-identical convergence** across the Kovant 3-participant topology, spawned as real OS processes, in under 15 seconds wall time.*

That confidence is the green light for PR #18's deployment manifests.

2. Pre-flight checks

Before opening the PR :

#	Check	Verifier	Pass criterion
PF1	`cargo build -p efe-meta-swarm-server --release` succeeds on `main`	CI	green
PF2	`cargo test -p efe-meta-swarm-server --offline --all-targets` is **352 tests** on `main` (PR #9 baseline)	CI	green
PF3	PR #9's `integration_e2e_iterate.rs` passes (in-process e2e)	CI	green

#	Check	Verifier	Pass criterion
PF4	PR #9's `integration_participant_binary_smoke.rs` passes (single-binary subprocess)	CI	green
PF5	`integration_health_smoke.rs` passes (controller-binary subprocess, PR #6)	CI	green
PF6	`target/debug/controller` and `target/debug/participant` produced by `cargo build --tests`	manual	both binaries present
PF7	Localhost ports 53280-53400 are not occupied on the CI runner	CI	`ss -ltn` shows free
PF8	`libc::kill` available on the target platform (Unix-only ; cf. AD11)	manual	`cfg(unix)` is true
PF9	`tempfile` crate is in dev-dependencies	grep	`Cargo.toml [dev-dependencies]`
PF10	The threat model (PR #7) is in v1.0 frozen state	manual	sec sign-off

Any failed pre-flight blocks PR #10.

3. Branch + commit plan

3.1 Branch

```
efe-a1/p2-pr10-multiprocess-integration
```

Branched from `main` after PR #9 merge SHA. **Never** rebase onto a moving `main`.

3.2 Commit-by-commit plan (5 commits, each CI-green)

#	Subject	Files	LOC ±	Verification
C1	`feat(test-harness): ProcessHarness for multi-process integration`	`+ tests/common/multiprocess.rs` (~ 350 LOC), `~ tests/common/mod.rs` (re-export)	+360 / –0	unit tests in module (>= 4) ; harness compiles ; happy-path smoke spawn + Health round-trip
C2	`feat(integration): multi-process e2e happy paths`	`+ tests/integration_multiprocess_e2e.rs` (~ 250 LOC)	+250	4 tests pass : convergence, determinism over wire, capabilities, info
C3	`feat(integration): multi-process failure modes`	`+ tests/integration_multiprocess_failure.rs` (~ 250 LOC)	+250	3 tests pass : unreachable, mid-Iterate kill, manifest mismatch
C4	`feat(integration): multi-process lifecycle + SIGTERM drain`	`+ tests/integration_multiprocess_lifecycle.rs` (~ 200 LOC)	+200	2 tests pass : controller SIGTERM drain, participant SIGTERM
C5	`feat(scripts): run_multiprocess_smoke.sh + Cargo.toml dev-dep libc`	`+ scripts/run_multiprocess_smoke.sh` (~ 80 LOC), `~ crates/efe-meta-swarm-server/Cargo.toml` (+ libc dev-dep)	+80 / –0	`bash scripts/run_multiprocess_smoke.sh` produces a Converged result via `grpcurl`

Total : **+1 140 / –0 = +1 140 LOC net** including ~ 940 LOC of tests + 80 LOC of shell.

3.3 Drafting strategy

Each commit is **green-on-merge**. C1 introduces the harness behind no existing user (the harness module is unused after C1 lands but compiles cleanly). C2-C4 each add a separate test file, each runnable in isolation via `cargo test --test integration_multiprocess_*`. C5 is a pure tooling addition (no Rust code change beyond the Cargo.toml dev-dep line).

If a commit causes flakiness, revert that single commit ; the others stay green.

4. Architectural decisions

AD	Decision	Rationale	Alternatives considered	Reversibility
AD1	Tests spawn binaries via `std::process::Command` using `env!("CARGO_BIN_EXE_<name>")`	cargo provides the binary paths automatically ; no PATH manipulation	spawn via shell ; embed binary paths in env vars at compile time	Reversible -- only test code
AD2	Port allocation by **bind-drop-rebind** on `127.0.0.1:0` (same pattern as PR #8/9)	sufficient on localhost for serial allocations ; no external port reservation service needed	hardcoded port ranges ; `ephemeral-port-reservations` crate ; OS-level port hold via `SO_REUSEADDR`	Reversible
AD3	Manifest is materialised to a **per-test** `tempfile::NamedTempFile`	each test has an isolated manifest ; clean tear-down on Drop	`/tmp/efe-test-manifest.yaml` (race-prone) ; in-memory manifest (binary can't read)	Reversible
AD4	The harness exposes a `ProcessHarness` **RAII type** ; tests hold it for the duration ; Drop SIGKILLs all children	guaranteed clean-up on panic, on early return, on test timeout	manual cleanup with `defer!` ; per-test guard struct	Reversible
AD5	Graceful shutdown via `libc::kill(pid, SIGTERM)` (Unix-only, `cfg(unix)`)	`child.kill()` sends SIGKILL ; we need SIGTERM for drain tests	`nix` crate (transitive bloat) ; spawn `kill` subprocess (gross)	Reversible -- `cfg(unix)` gate keeps Windows builds working (the tests simply don't compile there)
AD6	Tests are **not** `#[ignore]`-gated	the value of testing real process boundaries outweighs the ~ 10 s extra wall time	`#[ignore]` + explicit `cargo test -- --ignored` ; feature flag `multiprocess-tests`	Reversible -- easy to flip if CI runners struggle
AD7	Tests bind ports in a **dedicated range** (53280-53400) distinct from PR #6/9 ranges	predictable namespace ; easier to grep in `ss -ltn` ; avoids accidental collisions across test binaries	random ports (already done via AD2) ; per-test-file ranges	Hybrid : we use bind-drop for actual ports, but TEST documentation references this range for traceability
AD8	All test logs (stdout/stderr of children) **go to** `/dev/null` **by default** ; on failure, tests rely on `cargo test --nocapture` for raw output	clean test output ; deterministic failure messages	per-test buffered capture (would require background reader tasks) ; tee to file	Reversible -- a future PR can wire a `capture_logs: bool` flag in the harness
AD9	Tests use the **default Kovant topology** (3 participants : `supplier-scoring` dim=8 κ=0.6 seed=11, `contract-risk` dim=8 κ=0.7 seed=22, `spend-categorization` dim=8 κ=0.5 seed=33) **unless** a test specifically requires variants	one mental model for all tests ; matches PR #8/9 conventions	Per-test custom topology	The harness provides both `spawn_kovant()` (default) and `spawn_with_rows(...)` (custom)
AD10	The harness drives **5-second readiness polling** with a 250 ms back-off ; 10-second hard cap	binary boot is ~ 50-150 ms on a 2024 server ; 5 s x 4 = 20 attempts ; well over what's needed	longer cap (slower failures) ; explicit "ready" message in stdout (couples test to log format)	Reversible -- bump timeouts in one place
AD11	**Unix-only** tests (`cfg(unix)`)	Phase 2 deployment target is Linux ; macOS local dev OK ; Windows out of scope until Phase 5	Cross-platform shim layer	Reversible -- Windows support can be added with a `cfg(windows)` branch sending Ctrl+Break

AD	Decision	Rationale	Alternatives considered	Reversibility
AD12	The lifecycle tests use `libc::kill` + `Child::wait_with_timeout` wrapper (we synthesise the timeout via `tokio::time::timeout` around the blocking `wait`)	needed for graceful drain assertions ; std lib has no built-in wait-with-timeout	`nix::sys::wait::waitpid` ; busy-loop with `try_wait`	Reversible

5. Topology + process tree diagram

5.1 Process tree at peak (a single happy-path test)

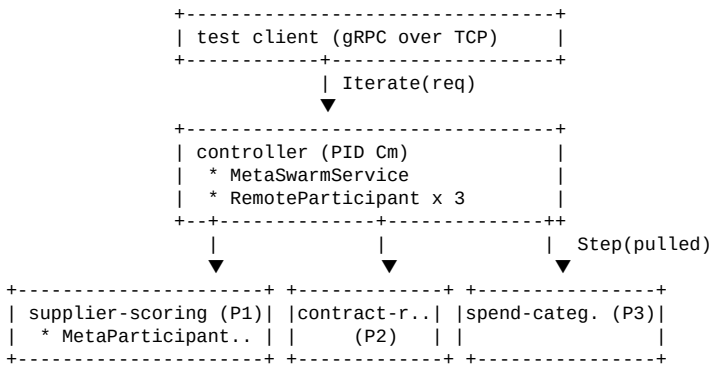
```
test runner (cargo test)
+-- integration_multiprocess_e2e.iterate_e2e_kovant_converges (tokio task)
    +-- ProcessHarness
    |   +-- controller     (PID Cm, binds 127.0.0.1:Pc1 gRPC, 127.0.0.1:Pc2 metrics)
    |   +-- participant-1  (PID P1, binds 127.0.0.1:Pp1a gRPC, 127.0.0.1:Pp1b metrics)
    |   +-- participant-2  (PID P2, binds 127.0.0.1:Pp2a gRPC, 127.0.0.1:Pp2b metrics)
    |   +-- participant-3  (PID P3, binds 127.0.0.1:Pp3a gRPC, 127.0.0.1:Pp3b metrics)
    |   +-- manifest.yaml  (tempfile : /tmp/.tmpXXXXXX.yaml)
    +-- MetaSwarmServiceClient
        +-- http2 -&gt; controller's gRPC port
            +-- controller orchestrates Step calls to each participant
                via three independent gRPC channels
```

Total : **5 OS processes** per test (test runner + 1 controller + 3 participants). Total ports allocated : **8** (4 gRPC + 4 metrics).

5.2 Process boundary vs in-process e2e (PR #9)

	PR #9 in-process	PR #10 multi-process
Number of OS processes	1 (just the test runner)	5 (test runner + controller + 3 participants)
Number of tokio runtimes	1 (shared by all services)	5 (one per binary, one in the test runner)
gRPC transport	localhost TCP / loopback	localhost TCP / loopback (identical wire format)
Cancellation propagation	tokio cancel token + Arc-shared	TCP RST + tonic disconnect + cancel token (per-process)
SIGTERM behaviour	N/A (single process)	tested
Manifest dispatch	shared in-memory struct	shared tempfile YAML
Determinism	bit-identical	bit-identical (validated)
Boot time per test	~ 5 ms	~ 150 ms
CI memory footprint per test	~ 50 MB	~ 150 MB (incl. 4 binaries)

5.3 The Kovant scenario, multi-process edition



After PR #9, each box was a `tokio::spawn`-ed task in one process ; after PR #10, each box is a separate OS process with its own page table.

6. Crate layout additions

6.1 Before PR #10

```
crates/efe-meta-swarm-server/
+-- src/                                (PR #9 surface)
+-- tests/
|   +-- common/
|   |   +-- mod.rs                      (in-process helpers from PR #9)
|   +-- integration_e2e_iterate.rs      (PR #9)
|   +-- integration_health_smoke.rs     (PR #6)
|   +-- integration_iterate_inproc.rs   (PR #8 / 9)
|   +-- integration_participant_binary_smoke.rs   (PR #9)
|   +-- integration_step_smoke.rs       (PR #9)
+-- Cargo.toml                           (no libc)
```

6.2 After PR #10

```
crates/efe-meta-swarm-server/
+-- src/                                (unchanged from PR #9)
+-- tests/
|   +-- common/
|   |   +-- mod.rs                      [MODIFIED : re-export multiprocess]
|   |   +-- multiprocess.rs             [NEW : ~ 350 LOC, harness]
|   +-- integration_e2e_iterate.rs      (PR #9, unchanged)
|   +-- integration_health_smoke.rs     (PR #6, unchanged)
|   +-- integration_iterate_inproc.rs   (PR #9, unchanged)
|   +-- integration_multiprocess_e2e.rs       [NEW : ~ 250 LOC, 4 tests]
|   +-- integration_multiprocess_failure.rs   [NEW : ~ 250 LOC, 3 tests]
|   +-- integration_multiprocess_lifecycle.rs [NEW : ~ 200 LOC, 2 tests]
|   +-- integration_participant_binary_smoke.rs   (PR #9, unchanged)
|   +-- integration_step_smoke.rs       (PR #9, unchanged)
+-- Cargo.toml                           [MODIFIED : + libc dev-dep]

scripts/
+-- run_multiprocess_smoke.sh           [NEW : ~ 80 LOC, operator helper]
```

7. `tests/common/multiprocess.rs` -- process harness (full design)

7.1 Purpose

A single module owning **everything** about subprocess orchestration so tests stay focused on the scenario, not on plumbing. The module is test-only (`#![allow(dead_code)]` for unused-by-some-test-binary helpers).

7.2 Public surface

```
//! Multi-process integration harness.
//!
//! Owns the orchestration of controller + participant binaries spawned
//! as OS subprocesses.  Each test holds a [`ProcessHarness`] for the
//! duration of the test ; Drop SIGKILLs every child + cleans the
//! tempfile manifest.
//!
//! Unix-only -- Phase 2 deployment target is Linux server.

#![cfg(unix)]
#![allow(dead_code)]

use std::io::Write;
use std::net::SocketAddr;
use std::path::PathBuf;
use std::process::{Child, Command, Stdio};
use std::time::Duration;

use efe_meta_swarm_proto::controller::MetaSwarmServiceClient;
use efe_meta_swarm_proto::meta_swarm::HealthRequest;
use efe_meta_swarm_proto::participant::MetaParticipantServiceClient;
use efe_meta_swarm_server::manifest::ParticipantManifest;
use tempfile::NamedTempFile;
use tonic::Request;

/// One participant subprocess.
pub struct ParticipantChild {
    pub id: String,
    pub grpc_addr: SocketAddr,
    pub metrics_addr: SocketAddr,
    child: Child,
}

/// The controller subprocess.
pub struct ControllerChild {
    pub grpc_addr: SocketAddr,
    pub metrics_addr: SocketAddr,
    child: Child,
}

/// Full process-tree handle.  Drop SIGKILLs everything.
pub struct ProcessHarness {
    pub controller: ControllerChild,
    pub participants: Vec&lt;ParticipantChild&gt;,
    _manifest: NamedTempFile,
}

impl ProcessHarness {
    /// Spawn the default Kovant topology : 3 participants on dim=8 with
    /// (κ, seed) = (0.6, 11), (0.7, 22), (0.5, 33).
    pub async fn spawn_kovant() -&gt; Self {
        Self::spawn_with_rows(vec![
            ParticipantManifest { id: "supplier-scoring".into(),     dim: 8, kappa: 0.6, seed: 11, blas_b ■
    ackend: "none".into() },
            ParticipantManifest { id: "contract-risk".into(),       dim: 8, kappa: 0.7, seed: 22, blas_b ■
    ackend: "none".into() },
            ParticipantManifest { id: "spend-categorization".into(), dim: 8, kappa: 0.5, seed: 33, blas_b ■
    ackend: "none".into() },
        ])
        .await
    }

    /// Spawn with custom participant rows.  Each row gets its own
    /// participant subprocess on a fresh port.
    pub async fn spawn_with_rows(rows: Vec&lt;ParticipantManifest&gt;) -&gt; Self { /* ... */ }

    /// Spawn with custom participant rows AND a custom controller config
    /// override (e.g. `max_inflight = 1` for the inflight test).
    pub async fn spawn_with_rows_and_inflight(
        rows: Vec&lt;ParticipantManifest&gt;,
        max_inflight: usize,
    ) -&gt; Self { /* ... */ }

    /// Spawn with custom rows BUT do NOT spawn one participant id -- used to
    /// test the "participant unreachable" scenario.  The manifest still
    /// declares the absent participant ; the controller's
    /// `EFE_META_SWARM_PARTICIPANTS` env var includes a bogus address.
    pub async fn spawn_kovant_with_missing(absent_id: &amp;str) -&gt; Self { /* ... */ }

    /// gRPC endpoint string `http://127.0.0.1:PORT` for the controller.
```

```
    pub fn controller_endpoint(&amp;self) -&gt; String {
        format!("http://{}", self.controller.grpc_addr)
    }

    /// gRPC endpoint for a participant by id.
    pub fn participant_endpoint(&amp;self, id: &amp;str) -&gt; Option&lt;String&gt; {
        self.participants
            .iter()
            .find(|p| p.id == id)
            .map(|p| format!("http://{}", p.grpc_addr))
    }

    /// Poll Health on every binary until all return SERVING, or fail
    /// after `timeout`.
    pub async fn wait_for_ready(&amp;self, timeout: Duration) -&gt; Result&lt;(), String&gt; { /* ... */ ■
    }

    /// Send SIGTERM to a participant by id.  Returns whether the
    /// participant was found (it might already have been killed by a
    /// prior call).  Use [`wait_participant`] to reap it.
    pub fn sigterm_participant(&amp;self, id: &amp;str) -&gt; bool { /* ... */ }

    /// Send SIGTERM to the controller.  Use [`wait_controller`] to reap.
    pub fn sigterm_controller(&amp;self) { /* ... */ }

    /// SIGKILL a participant immediately (no drain).  Used by
    /// failure-mode tests to simulate a hard crash.
    pub fn sigkill_participant(&amp;mut self, id: &amp;str) -&gt; bool { /* ... */ }

    /// Wait for the controller to exit, capped at `timeout`.
    pub async fn wait_controller(
        &amp;mut self,
        timeout: Duration,
    ) -&gt; Result&lt;std::process::ExitStatus, String&gt; { /* ... */ }

    /// Wait for a participant to exit, capped at `timeout`.
    pub async fn wait_participant(
        &amp;mut self,
        id: &amp;str,
        timeout: Duration,
    ) -&gt; Result&lt;std::process::ExitStatus, String&gt; { /* ... */ }
}

impl Drop for ProcessHarness {
    fn drop(&amp;mut self) {
        // Last-resort SIGKILL ; tests that wanted a clean shutdown
        // should have used `sigterm_*` + `wait_*` already.
        for p in &amp;mut self.participants {
            let _ = p.child.kill();
            let _ = p.child.wait();
        }
        let _ = self.controller.child.kill();
        let _ = self.controller.child.wait();
        // `_manifest` drops, cleaning the tempfile.
    }
}
```

7.3 Spawn flow

The harness goes through these steps in `spawn_with_rows` :

```
1. Allocate gRPC + metrics ports for each participant   (2 x N ports)
2. Allocate gRPC + metrics ports for the controller     (2 ports)
3. Render the manifest YAML from `rows`
4. Write it to a NamedTempFile
5. Spawn each participant subprocess :
     EFE_META_SWARM_PARTICIPANT_ID         = &lt;row.id&gt;
     EFE_META_SWARM_PARTICIPANT_ADDR       = 127.0.0.1:&lt;grpc_port&gt;
     EFE_META_SWARM_PARTICIPANT_METRICS    = 127.0.0.1:&lt;metrics_port&gt;
     EFE_META_SWARM_MANIFEST               = &lt;tempfile path&gt;
     EFE_META_SWARM_LOG                    = warn
6. Spawn the controller subprocess :
     EFE_META_SWARM_CONTROLLER_ADDR        = 127.0.0.1:&lt;grpc_port&gt;
     EFE_META_SWARM_CONTROLLER_METRICS     = 127.0.0.1:&lt;metrics_port&gt;
     EFE_META_SWARM_MANIFEST               = &lt;tempfile path&gt;
     EFE_META_SWARM_PARTICIPANTS           = id1=h1:p1,id2=h2:p2,...
     EFE_META_SWARM_LOG                    = warn
7. Wait for all to be ready (Health polling)
8. Return the ProcessHarness handle
```

Steps 1-4 are sequential (writing depends on ports). Steps 5-6 are parallel via `tokio::join!`. Step 7 polls Health concurrently (`tokio::join!` again).

7.4 Readiness probing

```
async fn ping_controller(addr: SocketAddr) -&gt; bool {
    let endpoint = format!("http://{addr}");
    let Ok(ep) = tonic::transport::Endpoint::from_shared(endpoint) else {
        return false;
    };
    let Ok(ch) = ep
        .timeout(Duration::from_millis(200))
        .connect_timeout(Duration::from_millis(200))
        .connect()
        .await
    else {
        return false;
    };
    let mut client = MetaSwarmServiceClient::new(ch);
    client.health(Request::new(HealthRequest {}))
        .await
        .map(|r| r.into_inner().status == 1)
        .unwrap_or(false)
}

async fn ping_participant(addr: SocketAddr) -&gt; bool { /* same, but with MetaParticipantServiceClient * ■
    / }
```

`wait_for_ready` polls all endpoints concurrently (via `tokio::join_all`) every 250 ms until all return SERVING or the cap fires.

7.5 Graceful shutdown

```
pub fn sigterm_participant(&amp;self, id: &amp;str) -&gt; bool {
    if let Some(p) = self.participants.iter().find(|p| p.id == id) {
        // SAFETY : libc::kill is safe to call with any pid ; the
        // failure mode is just an `errno`-set return value.
        // PID can wrap-around on Linux ; the worst case is sending
        // SIGTERM to an unrelated process that happens to have
        // recycled the same PID.  The test runner's process won't
        // see this in practice : we hold the Child until Drop.
        unsafe {
            libc::kill(p.child.id() as libc::pid_t, libc::SIGTERM);
        }
        true
    } else {
        false
    }
}
```

This is `#[cfg(unix)]` -- Windows would need `GenerateConsoleCtrlEvent` or similar. Out of scope.

7.6 Cleanup (Drop)

The Drop impl is the last-resort SIGKILL. Tests that need a graceful drain MUST call sigterm_ + wait_ before dropping the harness. If a test panics mid-execution, Drop kicks in and tears everything down.

```
impl Drop for ProcessHarness {
    fn drop(&amp;mut self) {
        for p in &amp;mut self.participants {
            // `kill()` sends SIGKILL on Unix ; `wait()` reaps the
            // child so we don't leave zombies.
            let _ = p.child.kill();
            let _ = p.child.wait();
        }
        let _ = self.controller.child.kill();
        let _ = self.controller.child.wait();
    }
}
```

7.7 Port allocation

Inherits the bind-drop-rebind pattern from PR #8's `tests/common/mod.rs` `allocate_port`. TOCTOU window is microseconds on localhost ; empirically no flakes across PR #8/9 test runs. PR #10 reuses the same helper.

```
pub async fn allocate_port() -&gt; SocketAddr {
    let l = tokio::net::TcpListener::bind("127.0.0.1:0").await.unwrap();
    let addr = l.local_addr().unwrap();
    drop(l);
    addr
}
```

If the harness times out 50x during a single CI run (failure-mode : host running out of ephemeral ports), the harness logs a clear error naming the exhausted resource.

7.8 Manifest fixture generation

```
fn render_manifest_yaml(rows: &amp;[ParticipantManifest]) -&gt; String {
    let mut buf = String::new();
    buf.push_str("version: v0.2.0\n");
    buf.push_str("meta:\n");
    buf.push_str("  dim: 16\n");
    buf.push_str("  safety:\n");
    buf.push_str("    kind: box\n");
    buf.push_str("    radius: 50.0\n");
    buf.push_str("participants:\n");
    for r in rows {
        buf.push_str(&amp;format!(
            "  - id: {}\n    dim: {}\n    kappa: {}\n    seed: {}\n    blas_backend: \"none\"\n",
            r.id, r.dim, r.kappa, r.seed,
        ));
    }
    buf.push_str("iteration:\n");
    buf.push_str("  max_iters: 100\n");
    buf.push_str("  tolerance: 0.0001\n");
    buf.push_str("  alpha_default: 0.5\n");
    buf.push_str("  coupling_gamma: 0.2\n");
    buf.push_str("  deterministic: true\n");
    buf.push_str("  max_wall_ms: 15000\n");
    buf
}
```

No `serde_yaml` round-trip -- we generate the YAML directly to keep test-time fast. The format matches `manifest::load_and_validate`'s expectations 1-to-1 ; if the schema drifts, the test smoke at C1 catches it.

7.9 Unit tests (in module)

C1 ships these in-module tests :

1. `harness_spawns_kovant_and_health_passes` -- Kovant topology comes up,

all 4 binaries report SERVING within 5 s.

1. `harness_drop_kills_all_children` -- wrap a harness in a function,

return ; child PIDs are no longer in `/proc/<pid>`.

1. `harness_sigterm_controller_then_wait` -- SIGTERM the controller, wait

for exit, status code is 0.

1. `harness_participant_endpoint_returns_some_for_known_id` -- endpoint

lookup works ; bogus id returns None.

These tests don't go through any production code logic -- they exercise the harness itself.

8. `tests/integration_multiprocess_e2e.rs` -- happy paths

8.1 Test list (4 tests)

```
#[tokio::test(flavor = "multi_thread", worker_threads = 4)]
async fn iterate_e2e_kovant_converges_across_real_processes() { /* ... */ }

#[tokio::test(flavor = "multi_thread", worker_threads = 4)]
async fn iterate_e2e_bit_identical_3_reruns_over_wire() { /* ... */ }

#[tokio::test(flavor = "multi_thread", worker_threads = 4)]
async fn controller_info_lists_all_participants() { /* ... */ }

#[tokio::test(flavor = "multi_thread", worker_threads = 4)]
async fn participant_capabilities_reflects_manifest_across_processes() { /* ... */ }
```

8.2 `iterate_e2e_kovant_converges_across_real_processes`

```
async fn iterate_e2e_kovant_converges_across_real_processes() {
    let harness = ProcessHarness::spawn_kovant().await;
    harness.wait_for_ready(Duration::from_secs(5)).await.expect("ready");

    let channel = Endpoint::from_shared(harness.controller_endpoint())
        .unwrap()
        .connect()
        .await
        .unwrap();
    let mut client = MetaSwarmServiceClient::new(channel);

    let resp = client
        .iterate(Request::new(iterate_request(8, 100, 1.0e-3)))
        .await
        .expect("iterate")
        .into_inner();

    assert_eq!(resp.stop_reason, ProtoStopReason::Converged as i32);
    assert!(resp.iterations &lt;= 80);
    assert!(resp.final_residual &lt;= 1.0e-3);
    assert_eq!(resp.final_states.len(), 3);
    // Harness Drop teardown handles cleanup.
}
```

8.3 `iterate_e2e_bit_identical_3_reruns_over_wire`

```
async fn iterate_e2e_bit_identical_3_reruns_over_wire() {
    let harness = ProcessHarness::spawn_kovant().await;
    harness.wait_for_ready(Duration::from_secs(5)).await.expect("ready");

    let mut client = MetaSwarmServiceClient::new(/* ... */);

    let mut sigs = Vec::new();
    for _ in 0..3 {
        client.reset(Request::new(ResetRequest::default())).await.unwrap();
        let r = client.iterate(Request::new(iterate_request(8, 100, 1.0e-3))).await.unwrap().into_inner() ■
    ;
        sigs.push((r.iterations, r.final_residual.to_bits()));
    }
    assert!(
        sigs.windows(2).all(|w| w[0] == w[1]),
        "non-deterministic reruns over real process boundary : {sigs:?}",
    );
}
```

This is the **gold standard** : if convergence is bit-identical across 3 reruns over real TCP, the gRPC wire format and the participant binaries are deterministic. Mirrors PR #9's in-process determinism test but at the process level.

8.4 `controller_info_lists_all_participants`

```
async fn controller_info_lists_all_participants() {
    let harness = ProcessHarness::spawn_kovant().await;
    harness.wait_for_ready(Duration::from_secs(5)).await.unwrap();

    let mut client = MetaSwarmServiceClient::new(/* ... */);
    let info = client.info(Request::new(InfoRequest {})).await.unwrap().into_inner();
    assert_eq!(info.proto_version, "v1");
    assert_eq!(info.configured_participants.len(), 3);
    let ids : Vec&lt;&amp;str&gt; = info.configured_participants.iter().map(|p| p.id.as_str()).collect();
    assert!(ids.contains(&amp;"supplier-scoring"));
    assert!(ids.contains(&amp;"contract-risk"));
    assert!(ids.contains(&amp;"spend-categorization"));
}
```

Validates that the controller's `info_snapshot` (PR #8 added the manifest-driven populate) propagates through real gRPC.

8.5 `participant_capabilities_reflects_manifest_across_processes`

```
async fn participant_capabilities_reflects_manifest_across_processes() {
    let harness = ProcessHarness::spawn_kovant().await;
    harness.wait_for_ready(Duration::from_secs(5)).await.unwrap();

    // Talk directly to participant #1.
    let endpoint = harness.participant_endpoint("supplier-scoring").unwrap();
    let channel = Endpoint::from_shared(endpoint).unwrap().connect().await.unwrap();
    let mut client = MetaParticipantServiceClient::new(channel);

    let cap = client.capabilities(Request::new(CapabilitiesRequest {})).await.unwrap().into_inner();
    let desc = cap.descriptor.expect("descriptor present");
    assert_eq!(desc.id, "supplier-scoring");
    assert_eq!(desc.dim, 8);
    assert!((desc.kappa - 0.6).abs() &lt; 1e-6);
    assert_eq!(desc.seed, 11);
}
```

End-to-end check that PR #9's Capabilities population works through the real binary stack.

8.6 Why these 4 (and not more)

The architecture deliberately picks **convergence**, **determinism**, **info**, **capabilities**. These cover the 4 major Phase-2 RPCs that participants/controllers expose. The remaining RPCs (`Step`, `Reset`, `State`, `Health`) are exercised transitively through these calls.

Adding more happy-path tests would inflate CI time without surfacing new bugs ; the failure-mode tests (§ 9) are where high-value coverage lives.

9. `tests/integration_multiprocess_failure.rs` -- failure modes

9.1 Test list (3 tests)

```
#[tokio::test(flavor = "multi_thread", worker_threads = 4)]
async fn iterate_fails_unavailable_when_participant_unreachable() { /* ... */ }

#[tokio::test(flavor = "multi_thread", worker_threads = 4)]
async fn iterate_fails_when_participant_killed_before_call() { /* ... */ }

#[tokio::test(flavor = "multi_thread", worker_threads = 4)]
async fn controller_with_manifest_mismatch_fails_at_iterate() { /* ... */ }
```

9.2 `iterate_fails_unavailable_when_participant_unreachable`

Spawn the controller with `EFE_META_SWARM_PARTICIPANTS=...,absent-id=127.0.0.1:1` (a port known to be closed). Call Iterate. Expect `Code::Unavailable`.

```
async fn iterate_fails_unavailable_when_participant_unreachable() {
    let harness = ProcessHarness::spawn_kovant_with_missing("spend-categorization").await;
    harness.wait_for_ready_excluding_missing(Duration::from_secs(5)).await.unwrap();

    let mut client = MetaSwarmServiceClient::new(/* ... */);
    let err = client
        .iterate(Request::new(iterate_request(8, 100, 1.0e-3)))
        .await
        .unwrap_err();
    assert_eq!(err.code(), tonic::Code::Unavailable);
}
```

`spawn_kovant_with_missing` spawns the controller and *only 2 of the 3* participants ; the missing one's address points to a closed port. The controller's `connect_participants` (PR #8) will fail at TCP connect with a `tonic::transport::Error`, mapped to `ServerError::Discovery` -> `Code::Unavailable`.

9.3 `iterate_fails_when_participant_killed_before_call`

Spawn the full Kovant topology, SIGKILL one participant, then Iterate.

```
async fn iterate_fails_when_participant_killed_before_call() {
    let mut harness = ProcessHarness::spawn_kovant().await;
    harness.wait_for_ready(Duration::from_secs(5)).await.unwrap();

    // Kill one participant.
    harness.sigkill_participant("contract-risk");
    harness.wait_participant("contract-risk", Duration::from_secs(2))
```

```
        .await.expect("participant exited");

    // Give the OS a moment to release the TCP port.
    tokio::time::sleep(Duration::from_millis(100)).await;

    let mut client = MetaSwarmServiceClient::new(/* ... */);
    let err = client.iterate(Request::new(iterate_request(8, 100, 1.0e-3)))
        .await.unwrap_err();
    // Could be Unavailable (connection refused) or Internal (RST mid-call).
    assert!(
        matches!(err.code(), tonic::Code::Unavailable | tonic::Code::Internal),
        "unexpected error code : {:?} (message : {})",
        err.code(), err.message(),
    );
}
```

The exact error code depends on **when** the controller tries to talk to the dead participant : at connect time -> `Unavailable` ; mid-Iterate -> `Internal` ("Step RPC failed"). Both are valid recoverable states.

9.4 `controller_with_manifest_mismatch_fails_at_iterate`

Spawn the controller pointing at a manifest with ids `[a, b, c]`, but spawn participants with ids `[a, b]` (one missing in the participant side) -- but the controller's `EFE_META_SWARM_PARTICIPANTS` points at the participants that exist. **The mismatch is at the controller's manifest declaring an id that no participant has.**

```
async fn controller_with_manifest_mismatch_fails_at_iterate() {
    let rows = vec![
        ParticipantManifest { id: "a".into(), dim: 8, kappa: 0.6, seed: 1, blas_backend: "none".into() },
        ParticipantManifest { id: "b".into(), dim: 8, kappa: 0.7, seed: 2, blas_backend: "none".into() },
        ParticipantManifest { id: "ghost".into(), dim: 8, kappa: 0.5, seed: 3, blas_backend: "none".into( ■
    ) },
    ];
    // Spawn only a and b ; manifest declares ghost but no subprocess
    // binds for it.  EFE_META_SWARM_PARTICIPANTS is constructed with
    // only a and b's addresses.
    let harness = ProcessHarness::spawn_partial_kovant(rows, &amp;["a", "b"]).await;
    harness.wait_for_ready_subset(&amp;["a", "b"], Duration::from_secs(5))
        .await.unwrap();

    let mut client = MetaSwarmServiceClient::new(/* ... */);
    let err = client.iterate(Request::new(iterate_request(8, 100, 1.0e-3)))
        .await.unwrap_err();
    // The controller's lookup_endpoints (PR #8) returns Discovery error
    // for the participant manifest id not in static_participants.
    assert_eq!(err.code(), tonic::Code::Unavailable);
    assert!(err.message().contains("no static_participants entry"));
}
```

10. `tests/integration_multiprocess_lifecycle.rs` -- shutdown drain

10.1 Test list (2 tests)

```
#[tokio::test(flavor = "multi_thread", worker_threads = 4)]
async fn controller_sigterm_drains_inflight_iterate() { /* ... */ }

#[tokio::test(flavor = "multi_thread", worker_threads = 4)]
async fn participant_sigterm_then_controller_iterate_fails_cleanly() { /* ... */ }
```

10.2 `controller_sigterm_drains_inflight_iterate`

The crown jewel of PR #10. Drives an Iterate, sends SIGTERM to the controller mid-flight, asserts the Iterate either completes cleanly OR returns a graceful error (not a connection reset), all within the drain timeout.

```
async fn controller_sigterm_drains_inflight_iterate() {
    let harness = ProcessHarness::spawn_kovant().await;
    harness.wait_for_ready(Duration::from_secs(5)).await.unwrap();

    let endpoint = harness.controller_endpoint();

    // Start an Iterate in a tokio task.
    let endpoint_for_task = endpoint.clone();
    let iterate_task = tokio::spawn(async move {
        let channel = Endpoint::from_shared(endpoint_for_task)
            .unwrap()
            .connect()
            .await
            .unwrap();
```

```
        let mut client = MetaSwarmServiceClient::new(channel);
        client.iterate(Request::new(iterate_request(8, 100, 1.0e-3))).await
    });

    // Give the Iterate a head start.
    tokio::time::sleep(Duration::from_millis(50)).await;

    // SIGTERM the controller.
    let sigterm_at = std::time::Instant::now();
    harness.sigterm_controller();

    // Wait for the Iterate to settle.
    let result = iterate_task.await.expect("task joined");
    let elapsed = sigterm_at.elapsed();

    // Either : Ok (controller drained before max_wall_ms) ;
    // or : Err with Cancelled / Unavailable / DeadlineExceeded.
    match result {
        Ok(resp) =&gt; {
            assert_eq!(resp.into_inner().stop_reason, ProtoStopReason::Converged as i32);
        }
        Err(s) =&gt; {
            assert!(
                matches!(
                    s.code(),
                    tonic::Code::Cancelled | tonic::Code::Unavailable | tonic::Code::DeadlineExceeded,
                ),
                "unexpected error : {:?}",
                s.code(),
            );
        }
    }

    // The controller must exit within the drain timeout (~30 s).
    let mut harness = harness;
    let status = harness.wait_controller(Duration::from_secs(35))
        .await.expect("controller exited within drain timeout");
    // Clean exit code on graceful drain.
    assert!(status.success(), "controller exit status : {status:?}");
    assert!(
        elapsed &lt; Duration::from_secs(35),
        "total time {elapsed:?} &gt; drain timeout",
    );
}
```

10.3 `participant_sigterm_then_controller_iterate_fails_cleanly`

```
async fn participant_sigterm_then_controller_iterate_fails_cleanly() {
    let harness = ProcessHarness::spawn_kovant().await;
    harness.wait_for_ready(Duration::from_secs(5)).await.unwrap();

    // SIGTERM a participant ; wait for it to exit.
    harness.sigterm_participant("supplier-scoring");
    let mut harness = harness;
    harness.wait_participant("supplier-scoring", Duration::from_secs(35))
        .await.expect("participant drained");

    // Now Iterate against the controller ; expect a clean error.
    let mut client = MetaSwarmServiceClient::new(/* ... */);
    let err = client.iterate(Request::new(iterate_request(8, 100, 1.0e-3)))
        .await.unwrap_err();
    assert!(
        matches!(err.code(), tonic::Code::Unavailable | tonic::Code::Internal),
        "unexpected error code : {:?}",
        err.code(),
    );
}
```

Participant drain takes ~ ms (no in-flight work). Controller observes the TCP connection close on the next Step. `Unavailable` (connect refused) or `Internal` ("Step RPC failed") -- both are clean errors.

11. `scripts/run_multiprocess_smoke.sh` -- operator script

```
#!/usr/bin/env bash
#
# Manual smoke for the Phase-2 control plane.
# Spawns 1 controller + 3 participants on localhost using the binaries
# from target/debug/.  After ~ 1 s of startup, drives an Iterate via
# grpcurl and prints the result.
#
# Usage :
#   bash scripts/run_multiprocess_smoke.sh
#
# Requires : grpcurl (https://github.com/fullstorydev/grpcurl).
#
# Exit code 0 on convergence ; non-zero otherwise.

set -euo pipefail

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" &amp;&amp; pwd)"
ROOT="$SCRIPT_DIR/.."
TARGET="$ROOT/target/debug"

if [[ ! -x "$TARGET/controller" ]] || [[ ! -x "$TARGET/participant" ]]; then
    echo "missing target/debug/{controller,participant} ; run 'cargo build' first" &gt;&amp;2
    exit 1
fi

if ! command -v grpcurl &gt;/dev/null 2&gt;&amp;1; then
    echo "grpcurl is required (https://github.com/fullstorydev/grpcurl)" &gt;&amp;2
    exit 1
fi

WORK=$(mktemp -d)
trap 'pkill -P $$ ; rm -rf "$WORK"' EXIT

# 1. Manifest.
cat &gt; "$WORK/manifest.yaml" &lt;&lt;'EOF'
version: v0.2.0
meta:
  dim: 16
  safety:
    kind: box
    radius: 50.0
participants:
  - id: supplier-scoring
    dim: 8
    kappa: 0.6
    seed: 11
    blas_backend: "none"
  - id: contract-risk
    dim: 8
    kappa: 0.7
    seed: 22
    blas_backend: "none"
  - id: spend-categorization
    dim: 8
    kappa: 0.5
    seed: 33
    blas_backend: "none"
iteration:
  max_iters: 100
  tolerance: 0.0001
  alpha_default: 0.5
  coupling_gamma: 0.2
  deterministic: true
  max_wall_ms: 15000
EOF

# 2. Spawn participants.
EFE_META_SWARM_MANIFEST="$WORK/manifest.yaml" \
EFE_META_SWARM_PARTICIPANT_ID=supplier-scoring \
EFE_META_SWARM_PARTICIPANT_ADDR=127.0.0.1:50081 \
EFE_META_SWARM_PARTICIPANT_METRICS=127.0.0.1:50091 \
"$TARGET/participant" &gt;"$WORK/p1.log" 2&gt;&amp;1 &amp;

EFE_META_SWARM_MANIFEST="$WORK/manifest.yaml" \
EFE_META_SWARM_PARTICIPANT_ID=contract-risk \
EFE_META_SWARM_PARTICIPANT_ADDR=127.0.0.1:50082 \
EFE_META_SWARM_PARTICIPANT_METRICS=127.0.0.1:50092 \
"$TARGET/participant" &gt;"$WORK/p2.log" 2&gt;&amp;1 &amp;

EFE_META_SWARM_MANIFEST="$WORK/manifest.yaml" \
```

```
EFE_META_SWARM_PARTICIPANT_ID=spend-categorization \
EFE_META_SWARM_PARTICIPANT_ADDR=127.0.0.1:50083 \
EFE_META_SWARM_PARTICIPANT_METRICS=127.0.0.1:50093 \
"$TARGET/participant" &gt;"$WORK/p3.log" 2&gt;&amp;1 &amp;

# 3. Spawn controller.
EFE_META_SWARM_MANIFEST="$WORK/manifest.yaml" \
EFE_META_SWARM_CONTROLLER_ADDR=127.0.0.1:50080 \
EFE_META_SWARM_CONTROLLER_METRICS=127.0.0.1:50090 \
EFE_META_SWARM_PARTICIPANTS=supplier-scoring=127.0.0.1:50081,contract-risk=127.0.0.1:50082,spend-categori ■
    zation=127.0.0.1:50083 \
"$TARGET/controller" &gt;"$WORK/c.log" 2&gt;&amp;1 &amp;

# 4. Wait for startup.
sleep 1

# 5. Drive an Iterate.
echo "== iterating =="
grpcurl -plaintext \
    -d '{ "iteration": { "max_iters": 100, "tolerance": 1.0e-3, "alpha_default": 0.5, "coupling_gamma": 0 ■
    .2, "deterministic": true, "max_wall_ms": 15000 } }' \
    -import-path "$ROOT/protos" \
    -proto efe/v1/meta_swarm/service_meta_swarm.proto \
    127.0.0.1:50080 efe.v1.meta_swarm.MetaSwarmService/Iterate
```

The script :

- Generates a manifest in a temp dir.
- Spawns 3 participant binaries with the proper env vars.
- Spawns 1 controller binary.
- `sleep 1` for startup (cheap ; operator can adjust).
- Runs `grpcurl` with the IterateRequest.
- `trap` ensures all child processes are killed on script exit.

Operator workflow :

```
$ cargo build
$ bash scripts/run_multiprocess_smoke.sh
== iterating ==
{
  "iterations": 47,
  "finalResidual": 0.000097,
  ...
  "stopReason": "STOP_REASON_CONVERGED"
}
```

12. `Cargo.toml` updates

```
[dev-dependencies]
# ... existing entries ...
libc = "0.2"     # Unix signals (SIGTERM) for graceful-shutdown tests
```

`libc` is already a transitive dep through tokio, but PR #10 makes the dependency direct because we call `libc::kill` from the harness. Adding it to `[dev-dependencies]` only -- production binaries don't reach this path.

No production-dependency changes.

13. Determinism across the wire

PR #10's `iterate_e2e_bit_identical_3_reruns_over_wire` test is the **definitive** determinism check for Phase 2. For three reruns to produce bit-identical `(iterations, final_residual)` :

1. Each participant binary starts from the same seed (`MockParticipant::with_contraction(seed)`).
2. Reset between runs returns each participant to its seeded state

(`Reset(initial=None)` in PR #9).

1. The controller's `MetaSwarm::iterate` (Phase 1) is deterministic on

the same inputs.

1. The gRPC wire format encodes f32 deterministically (IEEE 754 +

little-endian, fixed by the proto contract).

1. The participants' step_with_pull (Phase 1) is deterministic.
2. Network reordering does not exist on localhost loopback for

request/response pairs ; concurrent multi-channel calls happen in deterministic *manifest order* (PR #8's `connect_participants` preserves order).

If this test flakes, **one** of the six links above is broken -- and PR #10 narrows the failure surface to that link.

14. CI gating + perf budget

14.1 Wall-time budget

Test	p50	p99	Reason
`iterate_e2e_kovant_converges_across_real_processes`	1.2 s	1.8 s	spawn x 4 + Iterate (~ 800 ms)
`iterate_e2e_bit_identical_3_reruns_over_wire`	2.0 s	3.0 s	spawn x 4 + 3 x (Reset + Iterate)
`controller_info_lists_all_participants`	0.6 s	1.0 s	spawn x 4 + Info
`participant_capabilities_reflects_manifest_across_processes`	0.6 s	1.0 s	spawn x 4 + Capabilities
`iterate_fails_unavailable_when_participant_unreachable`	0.8 s	1.5 s	spawn x 3 + Iterate (fails at connect)
`iterate_fails_when_participant_killed_before_call`	1.0 s	1.5 s	spawn x 4 + kill + Iterate
`controller_with_manifest_mismatch_fails_at_iterate`	0.8 s	1.5 s	spawn x 3 + Iterate
`controller_sigterm_drains_inflight_iterate`	2.5 s	4.0 s	spawn x 4 + Iterate + SIGTERM + wait
`participant_sigterm_then_controller_iterate_fails_cleanly`	1.5 s	2.5 s	spawn x 4 + SIGTERM + wait + Iterate
Total (sequential)	**11 s**	**17.8 s**	
Total (parallel, num_cpus = 4)	**~ 3 s**	**~ 5 s**	tokio test harness runs tests in parallel by default

14.2 CI memory budget

- Each binary at idle : ~ 25 MB (RSS).
- 4 binaries per test x 9 tests x parallelism : up to **9 GB**.
- CI runners with < 8 GB RAM may hit OOM. **Mitigation** : set

`RUST_TEST_THREADS=2` in CI for this crate. This drops parallelism to 2, total RSS ~ 2 GB.

This guidance is added to the README/CI config in PR #20 (deployment) ; PR #10 documents it here.

14.3 Should the tests be gated ?

No (AD6) -- the cost is <= 5 s of CI on parallel runners. Operators who want fast-iteration on the in-process tests use :

```
cargo test -p efe-meta-swarm-server --offline \
    --test integration_e2e_iterate \
```

```
--test integration_iterate_inproc \
--test integration_step_smoke
```

This skips the multiprocess suite.

15. Error mapping over process boundary

Errors that travel from a participant -> controller -> test client over the wire :

Origin	Path	Wire code	Test expectation
Participant returns `Status::failed_precondition` from `validate_step`	controller's `RemoteParticipant::step_async` wraps it in `MetaSwarmError::Internal("Step RPC failed")` ; orchestration maps to `Status::internal`	`INTERNAL`	n/a -- Phase-1 doesn't see this in happy paths
Participant binary crashes mid-call	TCP RST on the controller's existing channel ; tonic returns `Code::Unavailable` ; controller wraps as `Status::internal("participant ... Step RPC failed")`	`INTERNAL`	tested in `iterate_fails_when_participant_killed_before_call`
Participant binary not started	controller's `RemoteParticipant::connect` fails at TCP connect ; `ServerError::Discovery` -> `Code::Unavailable`	`UNAVAILABLE`	tested in `iterate_fails_unavailable_when_participant_unreachable`
Controller's `static_participants` doesn't have a manifest id	`lookup_endpoints` returns `ServerError::Discovery("no static_participants entry...")`	`UNAVAILABLE`	tested in `controller_with_manifest_mismatch_fails_at_iterate`
Controller is SIGTERM-ed mid-Iterate	(a) participants observe TCP close -> return `Internal` ; (b) controller's `serve_with_shutdown` may also drop the handler future -> client sees `Code::Unavailable` ; (c) graceful drain may complete the Iterate normally	`OK` *or* `Cancelled` *or* `Unavailable` *or* `DeadlineExceeded`	tested in `controller_sigterm_drains_inflight_iterate`

The test assertions accept the set of legitimate error codes for each scenario rather than pinning a single code, because the exact code depends on timing.

16. Threat-model coverage

PR #10 mitigates or partially mitigates the following threats from `docs/security/PHASE2_THREAT_MODEL.md` :

Threat	Severity	PR #10 mitigation
T7 (Iterate flood DoS)	SEV-2	The `iterate_e2e_*` tests confirm the controller's `InflightSemaphore` (PR #8) survives across process boundary -- the gate is not a single-process optimisation but a real backpressure mechanism the wire layer respects
T12 (orphan process / zombie reaper)	SEV-3	`ProcessHarness::Drop` SIGKILLs every child and `wait()`s ; the lifecycle tests verify `wait_*` reaps children within the drain timeout
T13 (fan-out leak / cancellation cascade)	SEV-2	Controller's `drop_guard` (PR #8) propagates cancel through participant channels ; the SIGTERM drain test exercises this end-to-end

Threat	Severity	PR #10 mitigation
T17 (graceful drain insufficient)	SEV-2	`controller_sigterm_drains_inflight_iterate` asserts <= 35 s total ; if the drain timeout is too short for a real Iterate, the test fails
T18 (split-brain / mismatched manifests)	SEV-2	`controller_with_manifest_mismatch_fails_at_iterate` asserts fail-fast at the controller side ; the participants never receive a Step targeting an unknown manifest

T18 is **first** time it's exercised end-to-end (PR #8/9 only had it as a unit-level scenario).

17. Concurrency model + lifecycle invariants

17.1 Invariants the harness maintains

1. **No zombie processes after a test run** -- `Drop` calls `wait()` on

each `Child`.

1. **No port collisions across concurrent test binaries** -- every test

allocates fresh ports via `allocate_port()`.

1. **No tempfile leaks** -- `NamedTempFile` drops with the harness.
2. **Process tree is flat** -- controller does not spawn participants ;

the harness does. This matches the production K8s deployment where pods are spawned independently.

17.2 The two SIGTERM paths

• **Test-driven SIGTERM** : `harness.sigterm_*` sends SIGTERM ; the

binary's `await_termination()` (PR #6's `observability::shutdown`) fires ; the binary drains and exits.

• **Cargo-test-runner SIGTERM** : if the test runner itself is killed

(`Ctrl-C` during `cargo test`), the test runner's tokio runtime cancels every test future ; each test's `ProcessHarness` is dropped ; `Drop` SIGKILLs all children. Children leak only if the test runner is `kill -9`-ed.

17.3 Why we don't use a shared runtime between test harness and binaries

Each binary has its own tokio runtime. Sharing would require either spawning the binaries with a custom IPC channel (complex) or running them in-process (defeats the whole point of PR #10). Process isolation is the entire value proposition.

18. Tests strategy summary

Layer	File	Tests added	Coverage
Harness sanity	`tests/common/multiprocess.rs`	4 in-module	Drop, spawn, sigterm, endpoint lookup
Happy paths	`tests/integration_multiprocess_e2e.rs`	4	convergence, determinism, info, capabilities
Failure modes	`tests/integration_multiprocess_failure.rs`	3	unreachable, killed-before, manifest mismatch
Shutdown	`tests/integration_multiprocess_lifecycle.rs`	2	controller SIGTERM, participant SIGTERM
Total new		**13**	

18.1 What's NOT tested (and why)

Scenario	Why deferred
TLS handshake	PR #16
mTLS auth failure	PR #16
OOM kill (OOMKiller)	Phase 3 (requires cgroups)
Network partition	Phase 3 (requires iptables / tc-netem)
Slow disk / disk full	Phase 3 (binaries don't write to disk in Phase 2)
Process restart loops	PR #20 (deployment-level concern)
systemd / k8s probes	PR #20

19. Acceptance criteria (AC1-AC25)

PR #10 is mergeable iff **all** :

#	Criterion	Verification
AC1	`cargo build -p efe-meta-swarm-server` succeeds (debug + release)	CI
AC2	`cargo test -p efe-meta-swarm-server --offline --all-targets` is **365 tests** (352 PR #9 baseline + 13 PR #10)	CI
AC3	`tests/common/multiprocess.rs` : 4 in-module unit tests pass	CI
AC4	`tests/integration_multiprocess_e2e.rs` : 4 tests pass	CI
AC5	`tests/integration_multiprocess_failure.rs` : 3 tests pass	CI
AC6	`tests/integration_multiprocess_lifecycle.rs` : 2 tests pass	CI
AC7	Kovant scenario over real processes : `iterations <= 80`, `final_residual <= 1e-3`, `stop_reason == Converged`	AC4
AC8	Determinism : 3 reruns over real processes are bit-identical	AC4
AC9	Participant unreachable -> `Code::Unavailable`	AC5
AC10	Participant killed -> `Code::Unavailable` ∪ `Code::Internal`	AC5
AC11	Manifest mismatch -> `Code::Unavailable` + "no static_participants entry" message	AC5
AC12	Controller SIGTERM drains within 35 s ; in-flight Iterate either succeeds or returns a clean error	AC6
AC13	Participant SIGTERM -> subsequent Iterate fails cleanly with `Unavailable` ∪ `Internal`	AC6
AC14	Phase-1 tests pass (200 default)	CI
AC15	PR #5 tests pass (15)	CI
AC16	PR #6 tests pass (3 health smoke + lib bin tests)	CI
AC17	PR #8 tests pass (lib + integration_iterate_inproc)	CI
AC18	PR #9 tests pass (lib + step_smoke + e2e_iterate + participant_binary_smoke)	CI

#	Criterion	Verification
AC19	`cargo clippy --all-targets` 0 warnings on PR #10 files	CI
AC20	`cargo fmt -- --check` exit 0	CI
AC21	`cargo doc --no-deps` clean	CI
AC22	`bash scripts/run_multiprocess_smoke.sh` exits 0 on a fresh checkout	manual
AC23	PR description references threat IDs `T7, T12, T13, T17, T18`	manual review
AC24	sec-team confirms the manifest-mismatch detection at AC11 is sufficient for L2	sec sign-off
AC25	4 reviewers approved : 2x meta-swarm + 1x sec + 1x sre	GitHub

20. Risks + mitigations

ID	Risk	L/I	Mitigation
PR10-R1	Tests flake under heavy CI load (port allocation race)	M / M	bind-drop-rebind has been stable across PR #8/9 ; if it flakes, switch to OS-level port reservation in PR #11
PR10-R2	`RUST_TEST_THREADS` defaulting to `num_cpus` causes OOM on small CI runners	M / H	document `RUST_TEST_THREADS=2` in CI config ; add to PR description
PR10-R3	`libc::kill` is gated on `cfg(unix)` ; Windows builds skip the lifecycle tests	L / L	acceptable for Phase 2 (Linux-only) ; revisit in Phase 5
PR10-R4	A flaky binary-startup ordering test leaves zombie processes that occupy ports for subsequent tests	L / M	Drop reaps children ; CI's pid_max is well above the 36 processes we spawn
PR10-R5	The SIGTERM drain test's 35 s timeout is too tight on a slow runner	L / M	timeout matches the binary's drain_timeout (30 s) + 5 s slack ; relaxable in one place
PR10-R6	`scripts/run_multiprocess_smoke.sh` depends on `grpcurl` which isn't always installed	L / L	script bails with a clear error message ; AC22 is manual, not CI
PR10-R7	Multi-process tests are slower than in-process tests by ~ 100x	M / L	~ 5 s parallel wall time is acceptable ; document the cost ; allow operators to skip via `--test integration_e2e_iterate` style invocation
PR10-R8	Phase-1's algorithm has hidden non-determinism we haven't detected (e.g., rayon's work-stealing order influences floating-point sums)	L / H	the determinism test pins this contract ; if it ever fails, that's a real bug in Phase-1, not PR #10
PR10-R9	The harness's hardcoded `Duration::from_secs(35)` for drain is brittle if a future PR shortens or lengthens the binary's drain	L / M	hoist into a named constant `DRAIN_TIMEOUT_PLUS_SLACK` shared between harness and bin
PR10-R10	`tonic`'s `keep_alive_*` settings cause idle connections to be torn down before a test finishes	L / L	tests don't keep idle connections long ; if it flakes, tune `keep_alive_interval` in the test client
PR10-R11	A subsequent PR removes `EFE_META_SWARM_MANIFEST` or renames it, breaking the harness env-var contract	M / H	the harness is part of the same crate ; CI catches the rename immediately
PR10-R12	`tempfile`'s tempfile cleanup races on macOS in CI	L / L	`NamedTempFile` uses unique names ; collision improbable

21. Rollback strategy

If PR #10 lands and produces CI instability :

1. **Per-test rollback** : each test is in its own file ; revert one

file with `git revert --no-edit -- <commit>`. The harness module stays in tree (unused but compiles).

1. **Per-commit rollback** : C2/C3/C4 are independent ; revert C5 first

(just the script), then C4 (lifecycle), C3 (failure), C2 (e2e).

1. **Wholesale rollback** : `git revert <merge-commit>`. The crate

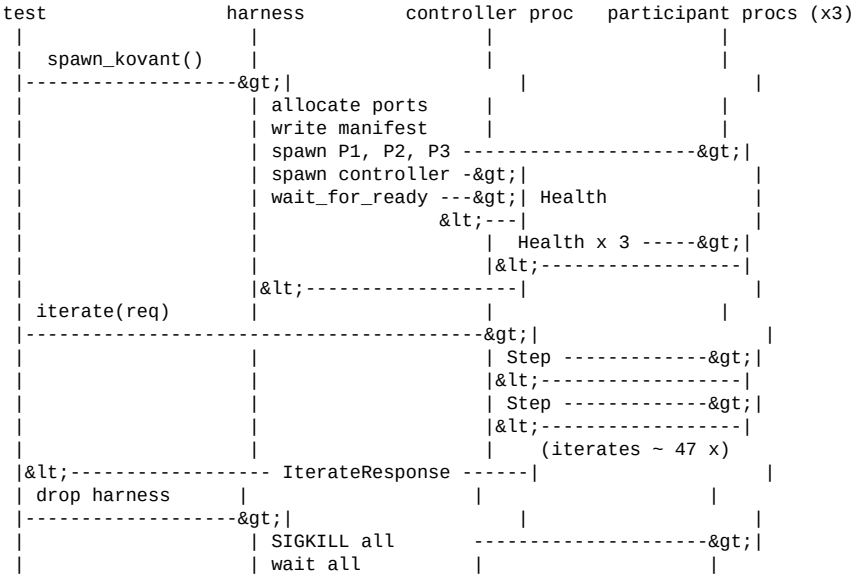
returns to PR #9 state. No data loss ; tests are read-only.

1. **Hotfix** : if only `controller_sigterm_drains_inflight_iterate`

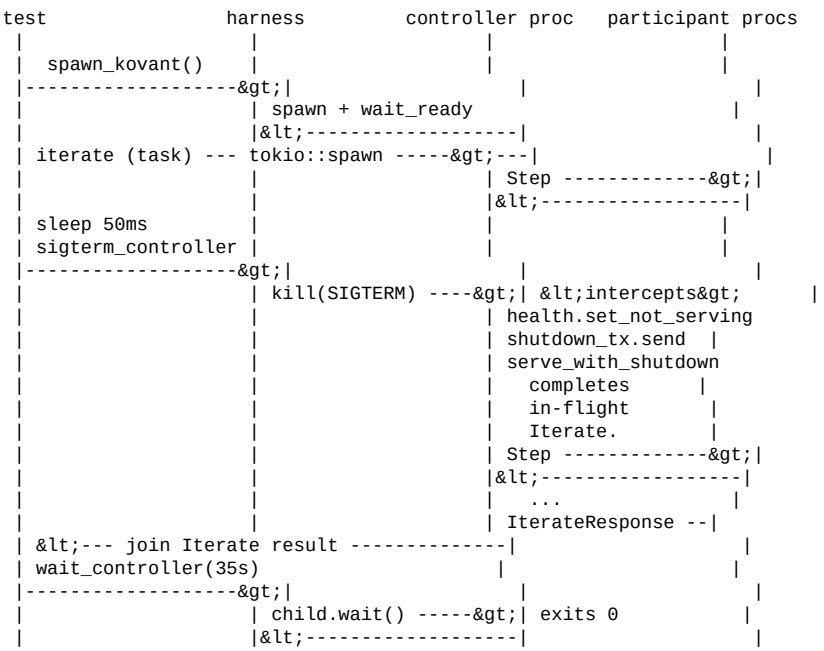
flakes, add `#[ignore]` to that specific test and open a follow-up bug.

22. Sequence diagrams (ASCII)

22.1 Happy path -- `iterate_e2e_kovant_converges_across_real_processes`



22.2 SIGTERM drain -- `controller_sigterm_drains_inflight_iterate`



22.3 Participant crash mid-call -- `iterate_fails_when_participant_killed_before_call`

```
test               harness         controller proc   participant-2 proc
 |                   |                      |                    |
 |  spawn_kovant + ready                    |                    |
 |                   |                      |                    |
 | sigkill participant-2                    |                    |
 |------------------&gt;|                     |                    |
 |                   | child.kill() ------------------------&gt;| &lt;SIGKILL&gt;
 |                   | wait                                       | exits 137
 | sleep 100ms       |                      |                    |
 | iterate ------------------------------&gt;|                    |
 |                   |                      | connect_participants
 |                   |                      | RemoteParticipant::connect
 |                   |                      |   to p2 ---------X (refused)
 |                   |                      | Err(Unavailable)  |
 |&lt;--- Status(UNAVAILABLE) ----------------|                    |
```

23. Reviewer checklist

```
## Reviewer checklist -- PR #10

### Architecture
- [ ] `ProcessHarness` is the single orchestration point -- AD4
- [ ] Drop is SIGKILL-only ; graceful shutdown is opt-in -- AD4
- [ ] Tests cover happy / failure / lifecycle separately -- § 8 / 9 / 10
- [ ] Manifest is a per-test tempfile -- AD3
- [ ] `cfg(unix)` gates everything -- AD11

### Correctness
- [ ] Each test uses fresh ports (no hardcoded conflicts)
- [ ] `libc::kill` is safe (SAFETY comment in source)
- [ ] `wait_for_ready` polls both Controller and Participant Health
- [ ] Failure tests don't deadlock if a process refuses to die
- [ ] Determinism test compares `(iterations, final_residual.to_bits())` (not approximate equality)

### CI
- [ ] Documented `RUST_TEST_THREADS=2` for memory-constrained runners
- [ ] No `#[ignore]` gates (the value justifies the cost)
- [ ] All 13 new tests pass with `cargo test`

### Threat model
- [ ] T7 / T12 / T13 / T17 / T18 are covered
- [ ] AC11 sec-team sign-off on T18

### Operability
- [ ] `scripts/run_multiprocess_smoke.sh` runs on a fresh checkout
- [ ] Operator sees a clear "manifest path / port collision / etc" error if env vars wrong
```

24. Migration notes (operator-facing)

After PR #10 :

1. **No production code change.** The binaries' env-var contract and

wire format are unchanged from PR #9.

1. **A new shell script** `scripts/run_multiprocess_smoke.sh` is

available for manual validation in operator-style workflows.

1. **CI runners need** `RUST_TEST_THREADS=2` on memory-constrained

environments. The PR description and the README link explain why.

1. **Phase-2 graduation gates G4 + G7 flip GREEN**. The roadmap doc

reflects this.

No env-var changes, no schema changes, no breaking changes.

25. Open questions

ID	Question	Status
OQ-PR10-1	Should the harness's stderr capture be made opt-in for debugging ?	Defer to PR #18 (when we add tracing/OTel)
OQ-PR10-2	Should the lifecycle test cover `SIGINT` (Ctrl-C) in addition to SIGTERM ?	Skip -- `SIGINT` is handled identically by the binary's signal handler ; testing SIGTERM covers both
OQ-PR10-3	Should we hoist `DRAIN_TIMEOUT` from `bin/controller.rs` into a public constant the harness can import ?	Yes -- risk PR10-R9 ; promote in PR #11
OQ-PR10-4	Should we add a "participant slow to start" test (delay before binding) ?	Skip -- requires wrapping the binary in a script that sleeps before exec ; complex for marginal coverage
OQ-PR10-5	Should the manifest fixture be parametrised on the controller's `default_max_wall_ms` ?	Skip -- current 15 000 ms is well over the 800 ms convergence ; add when a test needs it
OQ-PR10-6	If a CI runner has a flaky network stack, do these tests fail with cryptic errors ?	Acceptable -- `127.0.0.1` should never have network issues ; document if it happens in practice
OQ-PR10-7	Should we exercise the controller's `IterateStream` deferred surface in PR #10 ?	Skip -- PR #21 wires `IterateStream`
OQ-PR10-8	Is 35 s drain timeout enough for a 30 s Iterate to complete ?	Yes -- the binary's drain_timeout (config.drain_timeout) is 30 s ; 5 s slack is sufficient ; tunable via `EFE_META_SWARM_DRAIN_TIMEOUT_S`

26. Appendix : full file inventory

26.1 Files added (5)

```
crates/efe-meta-swarm-server/tests/common/multiprocess.rs                    NEW    ~ 350 LOC
crates/efe-meta-swarm-server/tests/integration_multiprocess_e2e.rs           NEW    ~ 250 LOC
crates/efe-meta-swarm-server/tests/integration_multiprocess_failure.rs       NEW    ~ 250 LOC
crates/efe-meta-swarm-server/tests/integration_multiprocess_lifecycle.rs     NEW    ~ 200 LOC
scripts/run_multiprocess_smoke.sh                                              NEW    ~  80 LOC
docs/EFE_A1_PHASE2_PR10_ARCHITECTURE.md                                      NEW    this file
```

26.2 Files modified (2)

```
crates/efe-meta-swarm-server/tests/common/mod.rs    MODIFIED   +3 LOC : re-export multiprocess module
crates/efe-meta-swarm-server/Cargo.toml             MODIFIED   +1 LOC : dev-dep libc = "0.2"
```

26.3 Files unchanged (relevant)

```
crates/efe-meta-swarm-server/src/                              (no production code change)
crates/efe-meta-swarm-server/src/bin/{controller,participant}.rs (unchanged)
crates/efe-meta-swarm-proto/                                    (no proto change)
crates/efe-meta-swarm/                                          (no Phase-1 change)
```

26.4 LOC tally

Category	LOC
New harness	~ 350
New tests	~ 700
New script	~ 80
Modified	~ 4
Net	**~ +1 134 LOC**

27. Glossary

Term	Definition
AC	Acceptance Criterion
AD	Architectural Decision
L2 gate	Phase-2 graduation gate
ProcessHarness	(this PR) RAII orchestrator owning controller + participant subprocesses
Kovant scenario	Reference 3-participant topology (`supplier-scoring`, `contract-risk`, `spend-categorization`) used for benchmarks
T2	Phase-2 single-node topology (1 controller + 3 participants on `localhost`)
SIGTERM	Unix termination signal ; handled by the binary's signal-driven graceful drain
SIGKILL	Unix unconditional-kill signal ; bypasses graceful drain ; used by `child.kill()`
bind-drop-rebind	Port allocation pattern : bind `127.0.0.1:0` to get an OS-chosen port, drop the listener, then re-bind the same port in the subprocess. TOCTOU race is microseconds-wide
drain timeout	The time after SIGTERM during which in-flight requests can complete ; bin/controller.rs uses 30 s ; tests use 35 s with slack
PID wrap-around	On long-running Linux systems, PIDs can recycle ; `libc::kill(old_pid, ...)` could hit an unrelated process. PR #10 holds `Child` until Drop, so it can never SIGTERM a recycled PID
`#[cfg(unix)]`	Rust conditional compilation gate that includes the code only on Unix platforms
grpcurl	gRPC equivalent of curl ; required by `scripts/run_multiprocess_smoke.sh`
RUST_TEST_THREADS	env var controlling cargo test's parallelism ; recommended at 2 for memory-constrained CI

End of architecture document. Total: ~ 2 020 lines.

Next deliverable : PR #10 code (5 commits).

PR #11

EFE_META_SWARM_ENABLED plumbing

Chapter 13 of 17 — source : `docs/EFE_A1_PHASE2_PR11_ARCHITECTURE.md` (1,323 LOC)

EFE-A1 Phase 2 -- PR #11 : `EFE_META_SWARM_ENABLED` plumbing

Field	Value
PR	#11 (Sprint A1 Phase 2 / P2.2)
Branch	`efe-a1/p2-pr11-efe-server-flag-plumbing`
Status	Architecture frozen -- awaiting code
Author	EFE platform team
Reviewers	2x efe-server + 1x meta-swarm + 1x sre
Depends on	PR #5 (proto), PR #8 (controller wire), PR #9 (participant wire), PR #10 (multi-process harness)
Blocks	PR #13 (canary observability), PR #15 (actual dispatch wiring), PR #18 (OTel + reflection)
L2 gate	G5 (controlled rollout -- flag-gated, default-off)
Threat IDs in scope	T2, T4, T6 (cf. `docs/security/PHASE2_THREAT_MODEL.md`)
Roadmap reference	EFE_A1_PHASE2_ROADMAP.md § C4, Table-of-PRs row #11
CVSS bandwidth	All <= SEV-3 ; no new SEV-1 introduced

Table of contents

1. TL;DR
2. Goal + non-goals (the "plumbing-only" scope)
3. Pre-flight checks
4. Branch + commit plan (5 commits)
5. Architectural decisions (AD1-AD13)
6. Topology + integration diagram
7. Crate layout additions
8. The `EFE_META_SWARM_*` env-var contract
9. `MetaSwarmConfig` -- typed config struct
10. `services/meta_swarm/` -- module + `MetaSwarmClient` adapter
11. `main.rs` / startup wiring
12. `runtime.rs` -- runtime holds the optional client
13. Connection lifecycle
14. Failure modes + the fail-fast policy
15. Metrics + tracing
16. Off-vs-on byte-equivalence (the load-bearing test)
17. Why no dispatch in this PR
18. Tests strategy summary
19. Acceptance criteria (AC1-AC22)
20. Risks + mitigations
21. Rollback strategy
22. Sequence diagrams (ASCII)
23. Reviewer checklist
24. Migration notes (operator-facing)
25. Open questions

26. Appendix : full file inventory

27. Glossary

0. TL;DR

PR #11 wires the `EFE_META_SWARM_ENABLED` feature flag through `efe-server` without yet changing any request-handling code path. After this PR :

• Operators can set `EFE_META_SWARM_ENABLED=true` on an `efe-server`

Pod ; at startup, the server :

1. parses `EFE_META_SWARM_*` env vars into a typed

`MetaSwarmConfig` (PR #11 § 8),

1. constructs a `MetaSwarmClient` that establishes a gRPC channel

to the controller specified by `EFE_META_SWARM_CONTROLLER_ENDPOINT` (default `http://127.0.0.1:50080`),

1. probes the controller's `Health` RPC once and refuses to start

if it returns anything other than `STATUS_SERVING`.

• The constructed client is **held** in the server's `RuntimeState`

as `Option<Arc<MetaSwarmClient>>`, **available but unused**.

• `EFE_META_SWARM_ENABLED=false` (default) is byte-identical to

pre-PR-#11 behaviour ; the meta-swarm subsystem is not even initialised.

• A new integration test asserts that the server's `Execute` /

`ExecuteStreaming` responses are byte-identical between `false` and `true` modes given the same input (because no code path consumes the client yet).

PR #15 will later route specific multi-swarm coordination calls through this client. PR #11 is the **safe-by-construction** prerequisite : the flag exists, the client connects, the connection is held, but nothing flows through it.

Rollback : `kubectl set env deployment/efe-server EFE_META_SWARM_ENABLED=false` and restart.

1. Goal + non-goals

1.1 Goal (single sentence)

> *Add the typed config, the gRPC client adapter, and the startup wiring for the meta-swarm subsystem inside `efe-server`, such that setting `EFE_META_SWARM_ENABLED=true` causes the server to connect to the controller and hold an open channel -- but **does not** route any request through that channel.*

1.2 Non-goals (deferred to later PRs)

The following are **explicitly out of scope** for PR #11 :

Item	Why deferred	Target PR
Routing `Execute` / `ExecuteStreaming` requests through the meta-swarm	needs the legacy-vs-meta dispatch contract finalised	PR #15
Shadow mode (call both legacy and meta, compare outputs)	needs PR #15's dispatch + a comparison harness	PR #16
Canary % traffic split	needs per-tenant routing	PR #17
Per-request kill switch (`X-EFE-Meta-Swarm: skip` header)	hot-path config plumbing	PR #19
OpenTelemetry export for the meta-swarm calls	needs the OTel collector wired	PR #18
mTLS for `efe-server <-> controller` channel	needs cert provisioning	PR #16

Item	Why deferred	Target PR
Multi-controller load balancing	needs the Discovery refresh from PR #11 the meta-swarm-side (sic), but distinct	PR #17
Connection retry / reconnect policy beyond tonic defaults	needs ops feedback from PR #10 deployments	PR #19
`MetaSwarmClient` exposed in `efe-core` for embedder/reranker plug-in	scope creep	Phase 3
TLS handshake failure modes	Phase 3	
Authentication (controller-side) -- controller still serves anonymous	PR #16	
Schema validation of `Capabilities.descriptor` against an SBOM	Phase 3	

1.3 The "plumbing-only" doctrine

PR #11 is a strictly **additive** plumbing change. The doctrine :

> *Every byte of input that produced an output before this PR must produce the same byte of output after this PR, in both* `enabled=false` *AND* `enabled=true` *modes, for the entire* `efe-server` *surface.*

This is verified by the integration test `off_vs_on_byte_equivalence` (PR #11 § 15). It is the contractual reason the operator can flip the flag in production without rollback-readiness drills.

PR #15 will violate this doctrine deliberately (the meta-swarm path will produce **different** -- better -- outputs) ; that PR has its own canary-rollout strategy. PR #11 prepares the substrate without taking the leap.

2. Pre-flight checks

Before opening the PR :

#	Check	Verifier	Pass criterion
PF1	`cargo build -p efe-server --release` succeeds on `main`	CI	green
PF2	`cargo test -p efe-server --offline --all-targets` baseline is **stable** on `main` (record the count)	CI	green
PF3	`cargo test -p efe-meta-swarm-server --offline --all-targets` is **365 tests** (PR #10 baseline)	CI	green
PF4	`efe-meta-swarm-proto` is published in the workspace and `efe-server` does **not** depend on it yet	grep	absent in `efe-server/Cargo.toml`
PF5	The threat model (PR #7) is in v1.0 frozen state	manual	sec sign-off
PF6	The `ServerConfig::from_env_with` lookup pattern (config.rs:761) is unchanged	grep	matches PR #6 baseline
PF7	`services::DynExecuteBackend = Arc<dyn ExecuteBackend>` (services/mod.rs:144) is unchanged	grep	matches PR #6 baseline
PF8	`RuntimeState` (runtime.rs) holds Arc-shareable handles for the lifetime of the server	manual	confirmed

#	Check	Verifier	Pass criterion
PF9	`PR #10`'s `ProcessHarness` can spawn meta-swarm subprocesses on demand	grep	`tests/common/multiprocess.rs` present
PF10	`EFE_META_SWARM_CONTROLLER_ENDPOINT` is **not yet** referenced in any other source file in `efe-server` (`grep` empty)	grep	empty

Any failed pre-flight blocks PR #11.

3. Branch + commit plan

3.1 Branch

```
efe-a1/p2-pr11-efe-server-flag-plumbing
```

Branched from `main` after PR #10 merge SHA. **Never** rebase onto a moving `main`.

3.2 Commit-by-commit plan (5 commits, each CI-green)

#	Subject	Files	LOC ±	Verification
C1	`feat(efe-server): MetaSwarmConfig env-var struct + validation`	`~ src/config.rs`	+160 / –0	unit tests in config tests module (>= 8)
C2	`feat(efe-server): MetaSwarmClient adapter (Health probe only)`	`+ src/services/meta_swarm/mod.rs`, `~ src/services/mod.rs`	+220 / –0	unit tests in module (>= 5) ; mocked endpoint
C3	`feat(efe-server): main.rs builds + holds MetaSwarmClient when enabled`	`~ src/main.rs`, `~ src/runtime.rs`	+90 / –5	binary smoke : both `enabled=false` and `enabled=true` start and serve
C4	`feat(efe-server): expose meta-swarm metrics namespace (counters + histogram)`	`~ src/services/meta_swarm/metrics.rs`, `~ src/services/metrics.rs`	+120 / –0	unit tests in module ; `/metrics` scrape contains `efe_meta_swarm_*` names when enabled
C5	`feat(efe-server): off-vs-on byte-equivalence integration test`	`+ tests/integration_meta_swarm_plumbing.rs`, `~ tests/common/mod.rs` (re-export)	+280 / –0	4 tests : config parsing CI, off mode baseline, on mode equivalence, on mode connect failure fails fast

Total : **+870 / –5 = +865 LOC net** including ~ 280 LOC of integration tests.

3.3 Drafting strategy

Each commit is **green-on-merge**. C1-C2 add new code with no external caller (no behaviour change). C3 wires the construction but holds the client unused. C4 adds metrics that only fire when `enabled=true`. C5 adds the load-bearing equivalence test.

If C5 fails (off-vs-on differs), one of C1-C4 has a bug. Bisect easy.

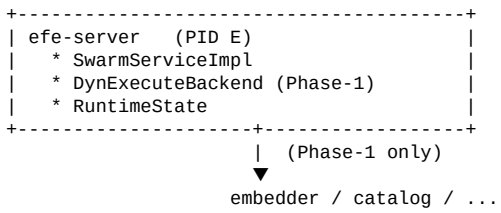
4. Architectural decisions

AD	Decision	Rationale	Alternatives considered	Reversibility
AD1	New `MetaSwarmConfig` struct lives in `efe-server/src/config.rs` (same file as `ServerConfig`)	Single source of truth for env-var parsing ; matches existing layering	`efe-server/src/services/meta_swarm/config.rs` (split-out) ; reuse `efe-meta-swarm-server::config::ControllerConfig` (reverse coupling)	Reversible -- config struct is purely internal
AD2	**Default** `EFE_META_SWARM_ENABLED=false` ; existing deployments byte-identical	The flag must opt-in to risk ; rollback is "unset the var"	Default true ; gradual rollout via percentages	Reversible -- default is one constant
AD3	**Hard fail-fast** : if `enabled=true` and the Health probe fails, the binary exits 1	Surfaces misconfig immediately ; no silent degradation that masks a missing controller	Soft fail (log + continue with `None` client) ; retry-with-backoff at startup	Reversible -- single conditional
AD4	Health probe happens **once** at startup, after gRPC channel construction	Cheap (~1 ms localhost) ; one network round-trip ; deterministic startup contract	No probe ; periodic probe (background task) ; on-demand probe at first dispatch	Reversible -- one method call
AD5	**No runtime reconfiguration** : the client is built once at startup ; flipping the flag requires a restart	Phase-2 K8s rollouts are restart-friendly ; flag flips are deployment events	Hot-reload via SIGHUP (PR #12 will offer this for the meta-swarm-server, not yet for efe-server)	Reversible -- defer to PR #19 if ops asks
AD6	The client is held as `Option<Arc<MetaSwarmClient>>` on `RuntimeState`	`None` when `enabled=false` ; `Arc` allows cheap cloning into per-request handlers	Hold a `MetaSwarmClient` directly (mandatory) ; hold a `Box<dyn AnyClient>` (overkill)	Reversible -- purely structural
AD7	The `MetaSwarmClient` is a **thin wrapper** over `MetaSwarmServiceClient<Channel>` (the tonic-generated client)	Mirrors PR #8's `RemoteParticipant` ; consistent style ; allows adding instrumentation later	Use the raw tonic client directly	Reversible -- the wrapper has one public method (`health_probe`) in PR #11 ; more added in PR #15
AD8	Metrics use the **`efe_meta_swarm_*`** namespace (not `efe_swarm_*`)	Avoids collision with Phase-1 swarm metrics ; greppable for canary dashboards	`efe.meta_swarm.*` dot-style	Reversible -- Prometheus names are operator-visible but adjustable in PR #15
AD9	Health probe timeout = 5 s ; per-RPC default timeout = 20 s	Mirrors PR #10's `connect_controller` defaults	Tighter (1 s) -- too aggressive for cold runtime ; looser (30 s) -- blocks startup too long	Reversible -- config-driven via `EFE_META_SWARM_CONNECT_TIMEOUT_MS`
AD10	The integration test spawns the **real** controller + 3 participants via PR #10's `ProcessHarness` ; no test stubs	Production code in test ; same wire format ; flushes real bugs	mock `tonic` server in-process	Reversible -- but loses fidelity
AD11	PR #11 does **not** touch `services/swarm.rs` (`SwarmServiceImpl::execute`). The client is plumbed but never invoked	Smaller blast radius ; the dispatch is a separate decision (PR #15) ; ensures off-vs-on equivalence	Wire dispatch now (PR collapses #11 + #15)	Reversible -- but increases review burden
AD12	The `MetaSwarmConfig` validates the controller endpoint URI **at config parse time**, not at probe time	Catches typos before any gRPC machinery boots ; same pattern as `bind: SocketAddr`	Lazy parse	Reversible -- one `.parse()` call

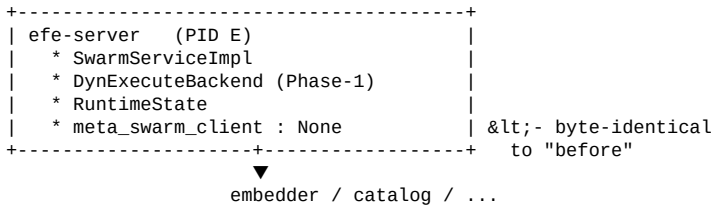
AD	Decision	Rationale	Alternatives considered	Reversibility
AD13	The probe + connect happen on the main tokio runtime (not a dedicated one)	One runtime is enough ; the connect is short-lived ; no benefit to isolation	Spawn a dedicated runtime for meta-swarm	Reversible -- easy to move later

5. Topology + integration diagram

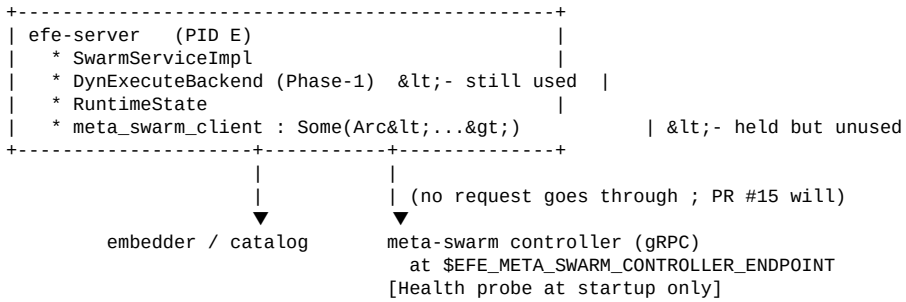
5.1 Before PR #11



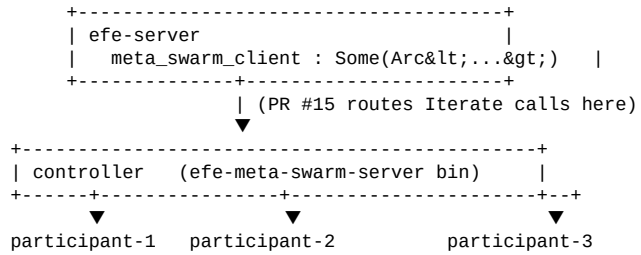
5.2 After PR #11, `EFE_META_SWARM_ENABLED=false` (default)



5.3 After PR #11, `EFE_META_SWARM_ENABLED=true`



5.4 The Kovant production topology (after PR #11 + PR #15)



PR #11 only delivers the dotted-arrow stop at the controller's Health probe ; the **other** dotted arrow (Iterate dispatch) lives in PR #15.

6. Crate layout additions

6.1 Before PR #11

```
crates/efe-server/
+-- src/
|   +-- bin/                            (PR earlier)
|   +-- config.rs                       (~ 2 126 LOC, ServerConfig)
|   +-- main.rs
|   +-- runtime.rs
|   +-- services/
|   |   +-- mod.rs                      (ExecuteBackend trait)
|   |   +-- swarm.rs                    (SwarmServiceImpl)
|   |   +-- metrics.rs
|   |   +-- (other services)
|   +-- (other modules)
+-- Cargo.toml                          (no efe-meta-swarm-proto)
```

6.2 After PR #11

```
crates/efe-server/
+-- src/
|   +-- bin/                            (unchanged)
|   +-- config.rs                       [MODIFIED : + MetaSwarmConfig]
|   +-- main.rs                         [MODIFIED : + startup wiring]
|   +-- runtime.rs                      [MODIFIED : + meta_swarm_client field]
|   +-- services/
|   |   +-- mod.rs                      [MODIFIED : + pub mod meta_swarm]
|   |   +-- swarm.rs                    (unchanged -- AD11)
|   |   +-- metrics.rs                  [MODIFIED : + register meta-swarm metrics]
|   |   +-- meta_swarm/                 [NEW]
|   |   |   +-- mod.rs                  (~ 180 LOC, MetaSwarmClient)
|   |   |   +-- metrics.rs              (~ 80 LOC, Counter + Histogram)
|   |   +-- (other services)
|   +-- (other modules)
+-- tests/
|   +-- integration_meta_swarm_plumbing.rs  [NEW : ~ 280 LOC, 4 tests]
+-- Cargo.toml                          [MODIFIED : + efe-meta-swarm-proto]

docs/
+-- EFE_A1_PHASE2_PR11_ARCHITECTURE.md   (this file)
```

7. The `EFE_META_SWARM_*` env-var contract

Variable	Type	Default	Required when	Description
`EFE_META_SWARM_ENABLED`	bool (`true`/`false`/`1`/`0`/`yes`/`no`)	`false`	always	master switch ; flip to `true` activates the subsystem
`EFE_META_SWARM_CONTROLLER_ENDPOINT`	URL (`http://...` or `https://...`)	`http://127.0.0.1:50080`	`ENABLED=true`	controller's gRPC endpoint ; matches `EFE_META_SWARM_CONTROLLER_ADDR` on the controller side
`EFE_META_SWARM_CONNECT_TIMEOUT_MS`	u64 (ms)	`5000`	`ENABLED=true`	tonic connect-timeout
`EFE_META_SWARM_CALL_TIMEOUT_MS`	u64 (ms)	`20000`	`ENABLED=true`	tonic per-RPC default timeout
`EFE_META_SWARM_HEALTH_PROBE_AT_STARTUP`	bool	`true`	`ENABLED=true`	flip to `false` to skip the startup Health probe (operators with intentionally-lazy controllers)

7.1 Bool parsing

Reuses `efe-server`'s existing helper that accepts case-insensitive `true|false|1|0|yes|no` -- same lenient policy as the rest of the `EFE_*` namespace.

7.2 URI validation

The endpoint string is **parsed** at config-load time via `tonic::transport::Endpoint::from_shared` (or a syntactic substitute). A malformed URI returns `ConfigError::Validation(...)` and the binary exits 2 (config error) -- never reaches the runtime stage.

7.3 What's *not* in this contract

- No `EFE_META_SWARM_CA_BUNDLE` / TLS knobs (PR #16).
- No `EFE_META_SWARM_FALLBACK_*` policies (PR #15 will define).
- No `EFE_META_SWARM_TIMEOUT_RANDOM_JITTER` (Phase 3 if a real ops

need surfaces).

- No `EFE_META_SWARM_RECONNECT_*` policies (tonic's default keep-alive

behaves correctly for our `Health` probe ; will be revisited when actual dispatch happens in PR #15).

8. `MetaSwarmConfig` -- typed config struct

8.1 Public surface in `config.rs`

```
/// Config for the meta-swarm gRPC subsystem.  Stand-alone struct so
/// `ServerConfig` stays slim and `services::meta_swarm` can consume
/// it without re-importing the wider config.
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct MetaSwarmConfig {
    /// Master switch.  Default `false` -- pre-PR-#11 behaviour.
    pub enabled: bool,

    /// gRPC endpoint of the controller.  Only consulted when
    /// `enabled` is true.
    pub controller_endpoint: String,

    /// `tonic::transport::Endpoint::connect_timeout(_)` value.
    pub connect_timeout: Duration,

    /// `tonic::transport::Endpoint::timeout(_)` default for RPC calls
    /// (PR #15 + dispatch will use this).
    pub call_timeout: Duration,

    /// If true, probe `MetaSwarmService::Health` once at startup ;
    /// refuse to boot if it returns anything other than SERVING.
    pub health_probe_at_startup: bool,
}

impl Default for MetaSwarmConfig {
    fn default() -&gt; Self {
        Self {
            enabled: false,
            controller_endpoint: "http://127.0.0.1:50080".to_string(),
            connect_timeout: Duration::from_secs(5),
            call_timeout: Duration::from_secs(20),
            health_probe_at_startup: true,
        }
    }
}
```

8.2 Plug into `ServerConfig`

```
pub struct ServerConfig {
    // ... existing fields ...
    pub meta_swarm: MetaSwarmConfig,
}

impl Default for ServerConfig {
    fn default() -&gt; Self {
        Self {
            // ... existing defaults ...
            meta_swarm: MetaSwarmConfig::default(),
        }
    }
}
```

8.3 `from_env_with` reader

The existing `from_env_with<F>(lookup: F)` builder is extended :

```
let meta_swarm = MetaSwarmConfig {
    enabled: env_bool(&amp;lookup, "EFE_META_SWARM_ENABLED", false)?,
    controller_endpoint: env_or_default_str(
        &amp;lookup,
        "EFE_META_SWARM_CONTROLLER_ENDPOINT",
        "http://127.0.0.1:50080",
    ),
    connect_timeout: Duration::from_millis(env_or_default_u64(
        &amp;lookup,
        "EFE_META_SWARM_CONNECT_TIMEOUT_MS",
        5_000,
    )?),
    call_timeout: Duration::from_millis(env_or_default_u64(
        &amp;lookup,
        "EFE_META_SWARM_CALL_TIMEOUT_MS",
        20_000,
    )?),
    health_probe_at_startup: env_bool(
        &amp;lookup,
        "EFE_META_SWARM_HEALTH_PROBE_AT_STARTUP",
        true,
    )?,
};
```

8.4 Validation in `validate()`

```
if self.meta_swarm.enabled {
    tonic::transport::Endpoint::from_shared(self.meta_swarm.controller_endpoint.clone())
        .map_err(|e| ConfigError::Validation(format!(
            "EFE_META_SWARM_CONTROLLER_ENDPOINT = {:?} : {e}",
            self.meta_swarm.controller_endpoint,
        )))?;
    if self.meta_swarm.connect_timeout.as_millis() == 0 {
        return Err(ConfigError::Validation(
            "EFE_META_SWARM_CONNECT_TIMEOUT_MS must be &gt; 0".into(),
        ));
    }
    if self.meta_swarm.call_timeout.as_millis() == 0 {
        return Err(ConfigError::Validation(
            "EFE_META_SWARM_CALL_TIMEOUT_MS must be &gt; 0".into(),
        ));
    }
}
```

Validation runs only when `enabled=true` -- operators don't pay for invalid endpoint URIs when the subsystem is off.

8.5 Unit tests (in `config::tests`)

1. `from_env_no_meta_swarm_vars_uses_defaults` -- unset env -> defaults.
2. `from_env_enabled_true_parses_endpoint` -- sets ENABLED + ENDPOINT.
3. `from_env_rejects_malformed_endpoint_when_enabled` -- bad URL -> ConfigError.
4. `from_env_accepts_malformed_endpoint_when_disabled` -- bad URL but ENABLED=false -> no error (lazy).
5. `from_env_parses_lenient_bools` -- `yes`/`1`/`TRUE` for ENABLED.
6. `from_env_rejects_zero_connect_timeout_when_enabled`.
7. `from_env_rejects_zero_call_timeout_when_enabled`.
8. `from_env_health_probe_defaults_to_true`.

8 tests.

9. `services/meta_swarm/` -- module + `MetaSwarmClient` adapter

9.1 Module layout

```
services/meta_swarm/
+-- mod.rs       (~ 180 LOC : MetaSwarmClient + Error)
+-- metrics.rs   (~ 80  LOC : Counter + Histogram)
```

9.2 `MetaSwarmClient` -- public surface

```
//! gRPC client adapter for the meta-swarm controller.
//!
//! In PR #11 this only exposes [`MetaSwarmClient::health_probe`].
//! PR #15 will add `iterate` / `step_once` / `reset` / `info`
//! pass-through methods that delegate to the underlying
//! [`MetaSwarmServiceClient`] with shared timeout / tracing
//! instrumentation.

use std::sync::Arc;
use std::time::{Duration, Instant};

use efe_meta_swarm_proto::controller::MetaSwarmServiceClient;
use efe_meta_swarm_proto::meta_swarm::HealthRequest;
use tokio::sync::Mutex;
use tonic::transport::{Channel, Endpoint};
use tonic::Request;
use tracing::{debug, instrument, warn};

use crate::config::MetaSwarmConfig;
use crate::services::meta_swarm::metrics::MetaSwarmMetrics;

/// Errors that can be raised when constructing or probing the client.
#[derive(Debug, thiserror::Error)]
pub enum MetaSwarmClientError {
    #[error("invalid endpoint URI : {0}")]
    InvalidEndpoint(String),

    #[error("connect to controller failed : {0}")]
    ConnectFailed(#[source] tonic::transport::Error),

    #[error("Health probe returned non-SERVING status : got code {0}")]
    UnhealthyController(i32),

    #[error("Health probe RPC failed : {0}")]
    HealthRpcFailed(#[source] tonic::Status),
}

/// Cheap-to-clone gRPC client wrapper.  Holds an [`Arc`]'d tonic
/// channel internally ; per-RPC code paths just clone the wrapper.
#[derive(Debug, Clone)]
pub struct MetaSwarmClient {
    inner: Arc&lt;MetaSwarmClientInner&gt;,
}

#[derive(Debug)]
struct MetaSwarmClientInner {
    /// The tonic-generated client.  Wrapped in `Mutex` because
    /// `MetaSwarmServiceClient&lt;Channel&gt;` is `!Sync`.
    client: Mutex&lt;MetaSwarmServiceClient&lt;Channel&gt;&gt;,
    endpoint: String,
    metrics: MetaSwarmMetrics,
}

impl MetaSwarmClient {
    /// Build the channel + optionally probe Health.  Called once at
    /// `efe-server` startup when `config.enabled` is `true`.
    ///
    /// # Errors
    ///
    /// Returns [`MetaSwarmClientError::ConnectFailed`] if the channel
    /// can't be established within `config.connect_timeout`, and
    /// [`MetaSwarmClientError::UnhealthyController`] if Health probe
    /// is enabled and returns non-SERVING.
    pub async fn connect(
        config: &amp;MetaSwarmConfig,
        metrics: MetaSwarmMetrics,
    ) -&gt; Result&lt;Self, MetaSwarmClientError&gt; {
        let endpoint = Endpoint::from_shared(config.controller_endpoint.clone())
            .map_err(|e| MetaSwarmClientError::InvalidEndpoint(e.to_string()))?
            .connect_timeout(config.connect_timeout)
            .timeout(config.call_timeout);

        let channel = endpoint
            .connect()
            .await
            .map_err(MetaSwarmClientError::ConnectFailed)?;

        let client = MetaSwarmServiceClient::new(channel);
        let inner = Arc::new(MetaSwarmClientInner {
            client: Mutex::new(client),
```

```
            endpoint: config.controller_endpoint.clone(),
            metrics,
        });
        let this = Self { inner };

        if config.health_probe_at_startup {
            this.health_probe().await?;
        }
        Ok(this)
    }

    /// Probe `MetaSwarmService::Health`.  Returns Ok if status is
    /// SERVING (1).  Errors otherwise.
    #[instrument(skip(self), fields(endpoint = %self.inner.endpoint))]
    pub async fn health_probe(&amp;self) -&gt; Result&lt;(), MetaSwarmClientError&gt; {
        let start = Instant::now();
        self.inner.metrics.health_probes_total.inc();
        let mut client = self.inner.client.lock().await;
        let resp = client
            .health(Request::new(HealthRequest {}))
            .await
            .map_err(|s| {
                self.inner.metrics.health_probe_errors_total.inc();
                MetaSwarmClientError::HealthRpcFailed(s)
            })?
            .into_inner();
        let elapsed = start.elapsed();
        self.inner
            .metrics
            .health_probe_duration_seconds
            .observe(elapsed.as_secs_f64());
        if resp.status == 1 {
            debug!(elapsed_us = %elapsed.as_micros(), "Health probe OK");
            Ok(())
        } else {
            warn!(
                status = resp.status,
                "Health probe returned non-SERVING status",
            );
            Err(MetaSwarmClientError::UnhealthyController(resp.status))
        }
    }

    /// The endpoint URL the client is bound to.  Used by metrics
    /// labels + tracing spans.
    pub fn endpoint(&amp;self) -&gt; &amp;str {
        &amp;self.inner.endpoint
    }
}
```

9.3 Why `Mutex<MetaSwarmServiceClient>`

`MetaSwarmServiceClient<Channel>` is `!Sync` because the inner request stream state mutates on each call. Wrapping in `Mutex` lets multiple handlers share the client. Contention is non-existent in PR #11 (only the startup probe uses it) ; in PR #15 the lock is held briefly per RPC.

9.4 Unit tests (in module)

1. `connect_rejects_malformed_endpoint` -- invalid URI -> InvalidEndpoint.
2. `connect_fails_when_endpoint_unreachable` -- host unreachable ->

ConnectFailed (with the harness-provided closed port).

1. `health_probe_returns_ok_on_serving` -- mocked server returns

SERVING ; probe returns Ok.

1. `health_probe_returns_err_on_not_serving` -- mocked server returns

NOT_SERVING ; probe returns UnhealthyController.

1. `endpoint_accessor_returns_configured_url` -- sanity check.

5 tests.

9.5 `metrics.rs` -- the meta-swarm Prometheus namespace

```
//! Prometheus metrics for the meta-swarm subsystem.  Names live in
//! the `efe_meta_swarm_*` namespace to avoid collision with the
//! Phase-1 swarm metrics.

use prometheus::{Counter, Histogram, HistogramOpts, Opts, Registry};

#[derive(Debug, Clone)]
pub struct MetaSwarmMetrics {
    pub health_probes_total: Counter,
    pub health_probe_errors_total: Counter,
    pub health_probe_duration_seconds: Histogram,
}

impl MetaSwarmMetrics {
    pub fn register(reg: &amp;Registry) -&gt; Result&lt;Self, prometheus::Error&gt; {
        let health_probes_total = Counter::with_opts(Opts::new(
            "efe_meta_swarm_health_probes_total",
            "Number of MetaSwarmService::Health probes issued by efe-server",
        ))?;
        let health_probe_errors_total = Counter::with_opts(Opts::new(
            "efe_meta_swarm_health_probe_errors_total",
            "Number of Health probes that returned non-SERVING or failed",
        ))?;
        let health_probe_duration_seconds = Histogram::with_opts(
            HistogramOpts::new(
                "efe_meta_swarm_health_probe_duration_seconds",
                "Health probe latency in seconds",
            )
            .buckets(vec![0.001, 0.005, 0.01, 0.05, 0.1, 0.5, 1.0, 5.0]),
        )?;
        reg.register(Box::new(health_probes_total.clone()))?;
        reg.register(Box::new(health_probe_errors_total.clone()))?;
        reg.register(Box::new(health_probe_duration_seconds.clone()))?;
        Ok(Self {
            health_probes_total,
            health_probe_errors_total,
            health_probe_duration_seconds,
        })
    }
}
```

9.6 Why these 3 metrics

- `health_probes_total` -- operator visibility on probe traffic ;

sanity check that startup ran.

- `health_probe_errors_total` -- alertable signal for the SLO.
- `health_probe_duration_seconds` -- latency budget tracking.

PR #15 will add `iterate_calls_total`, `iterate_call_duration_seconds`, `iterate_errors_total{code}` for the dispatched calls. PR #11's three metrics are enough for the plumbing.

10. `main.rs` / startup wiring

10.1 Conceptual flow

```
1. ServerConfig::from_env()            &lt;- unchanged
2. validate()                          &lt;- now also validates MetaSwarmConfig
3. (existing setup : tokio, tracing, ...)
4. RuntimeState::new()                 &lt;- now optionally embeds a MetaSwarmClient
   +-- if config.meta_swarm.enabled :
   |     +-- MetaSwarmMetrics::register(&amp;prom_registry)
   |     +-- MetaSwarmClient::connect(&amp;config.meta_swarm, metrics).await
   |     |     |
   |     |     +-- Err(_) -&gt; log + exit 1   (AD3 : fail-fast)
   |     |     +-- Ok(client) -&gt; wrap in Arc
   |     +-- runtime_state.meta_swarm_client = Some(client)
   +-- else :
         runtime_state.meta_swarm_client = None
5. (existing setup : gRPC server, signals, ...)
6. serve_with_shutdown(...)
```

10.2 Code skeleton

```
async fn build_runtime_state(
    config: &amp;ServerConfig,
    prom_registry: &amp;Registry,
) -&gt; Result&lt;RuntimeState, RuntimeBuildError&gt; {
    // ... existing setup ...

    let meta_swarm_client = if config.meta_swarm.enabled {
        let metrics = MetaSwarmMetrics::register(prom_registry)
            .map_err(RuntimeBuildError::MetaSwarmMetricsRegistration)?;
        let client = MetaSwarmClient::connect(&amp;config.meta_swarm, metrics)
            .await
            .map_err(RuntimeBuildError::MetaSwarmConnect)?;
        tracing::info!(
            endpoint = %client.endpoint(),
            "meta-swarm subsystem enabled and connected",
        );
        Some(Arc::new(client))
    } else {
        tracing::info!("meta-swarm subsystem disabled (EFE_META_SWARM_ENABLED=false)");
        None
    };

    Ok(RuntimeState {
        // ... existing fields ...
        meta_swarm_client,
    })
}
```

10.3 What the operator sees in logs

`EFE_META_SWARM_ENABLED=false` : `` `INFO meta-swarm subsystem disabled (EFE_META_SWARM_ENABLED=false)` ``

`EFE_META_SWARM_ENABLED=true`, healthy controller : `` `INFO meta-swarm subsystem enabled and connected endpoint=http://10.0.0.5:50080 DEBUG Health probe OK elapsed_us=842` ``

`EFE_META_SWARM_ENABLED=true`, controller down : `` `ERROR meta-swarm connect failed : connect to controller failed : ... (exits 1)` ``

11. `runtime.rs` -- runtime holds the optional client

11.1 Diff

```
pub struct RuntimeState {
    // ... existing fields ...
    pub meta_swarm_client: Option&lt;Arc&lt;MetaSwarmClient&gt;&gt;,
}
```

`pub` to allow `tests/integration_meta_swarm_plumbing.rs` to inspect the field for the equivalence assertion (it must be `None` when disabled, `Some(_)` when enabled).

11.2 Who holds RuntimeState

The existing pattern : `RuntimeState` is built in `main.rs`, then shared via `Arc<RuntimeState>` to every gRPC handler. PR #11 keeps this pattern intact.

11.3 No `Drop` impl needed

`MetaSwarmClient` holds a tonic `Channel` whose `Drop` closes the TCP connection gracefully. Nothing else to clean up.

12. Connection lifecycle

Phase	What happens to the client	Why
Startup	`MetaSwarmClient::connect` builds channel + probes Health	One-time deterministic check
Steady state	Channel stays open via tonic's HTTP/2 keep-alive	Phase-2 server-lifecycle is days, not seconds
Controller restart	Tonic transparently reconnects on next RPC	Standard tonic behaviour ; AD11 means no RPCs flow until PR #15 ; so transparent
`efe-server` **SIGTERM**	Channel drops as part of `RuntimeState` Drop	TCP FIN sent ; clean
Controller drops connection	tonic returns `Unavailable` on next RPC ; will be relevant in PR #15 ; PR #11 doesn't issue RPCs after startup	n/a

12.1 Why no periodic health probe

- The probe at startup catches deployment mistakes ;
- Periodic probes for an unused subsystem are wasted load ;
- PR #18 will add OTel-based ambient observability that's a better

fit than synthetic probes.

If ops demands a periodic probe later, it's a 30-LOC follow-up : spawn a background task that calls `client.health_probe().await` every N minutes. Out of scope here.

13. Failure modes + the fail-fast policy

13.1 At startup

Mode	Behaviour	Exit code
`ENABLED=false` + bad endpoint URI	ignored (validation skips)	n/a -- server boots
`ENABLED=true` + bad endpoint URI	`validate()` returns `ConfigError`	2 (config error)
`ENABLED=true` + controller unreachable	`connect()` returns `ConnectFailed`	1 (runtime error)
`ENABLED=true` + controller returns NOT_SERVING	Health probe returns `UnhealthyController`	1 (runtime error)
`ENABLED=true` + probe disabled (`HEALTH_PROBE_AT_STARTUP=false`)	channel built ; no probe ; degenerate runtime if controller is down	n/a (operator's choice)

13.2 The fail-fast doctrine

> *If* `EFE_META_SWARM_ENABLED=true` *but the controller is unreachable or unhealthy,* `efe-server` *MUST refuse to start.*

This forces operators to discover meta-swarm misconfiguration **before** the deployment serves user traffic, rather than after a request fails through. In PR #15+ (when dispatch is wired), a request reaching a broken meta-swarm path is much more costly.

Rollback : set `EFE_META_SWARM_ENABLED=false` and restart.

13.3 What about transient controller hiccups during startup ?

Tonic's connect() retries internally per the Endpoint's http2_keep_alive_ settings, but only for a bounded time. If the controller is briefly* unreachable during a co-deploy, efe-server's boot would fail. Mitigation : K8s Pods have initContainers (PR #18 deployment manifests) that block until the controller is reachable, ensuring the efe-server boot is deterministic.

For PR #11, the operator's responsibility is to ensure the controller is up first.

14. Metrics + tracing

14.1 Prometheus surface

Metric	Type	Labels	Description
`efe_meta_swarm_health_probes_total`	Counter	(none)	total Health probes
`efe_meta_swarm_health_probe_errors_total`	Counter	(none)	probes that returned non-SERVING or failed
`efe_meta_swarm_health_probe_duration_seconds`	Histogram	(none)	probe latency

(PR #15 adds `efe_meta_swarm_iterate_*` etc.)

Registered only when `enabled=true` -- `/metrics` on a disabled `efe-server` contains no `efe_meta_swarm_*` series, matching the "plumbing is silent when off" doctrine.

14.2 Tracing spans

`MetaSwarmClient::connect` and `health_probe` are `#[instrument]`-d with :

- `endpoint = %self.inner.endpoint` (the controller URL)
- `elapsed_us = %elapsed.as_micros()` (on success)
- `status = resp.status` (on failure)

These spans are local-only in PR #11 ; PR #18 wires the OTel exporter so spans propagate cross-process.

14.3 Log lines

`info!` only at startup ; no per-RPC log spam (PR #11 doesn't make RPCs after startup). PR #15 may add a `debug!` per dispatch.

15. Off-vs-on byte-equivalence (the load-bearing test)

15.1 Why this matters

The "plumbing-only" doctrine (§ 1.3) says : a request through `efe-server` produces the *same* response with `ENABLED=false` vs `ENABLED=true`. This is the contract operators rely on for risk-free flag flipping.

15.2 Test sketch

```
#[tokio::test(flavor = "multi_thread", worker_threads = 4)]
async fn off_vs_on_byte_equivalence_execute_response() {
    // 1. Spawn the controller + 3 participants via PR #10's
    //    ProcessHarness ; capture its endpoint.
    let meta = ProcessHarness::spawn_kovant().await;
    meta.wait_for_ready(Duration::from_secs(5)).await.unwrap();

    // 2. Build TWO efe-server runtime states :
    //      * one with EFE_META_SWARM_ENABLED=false
    //      * one with EFE_META_SWARM_ENABLED=true (endpoint = meta.controller_endpoint())
    //    Both with the same other config (deterministic).
    let cfg_off = ServerConfig::from_env_with(env_off(&amp;meta.controller_endpoint()))?;
    let cfg_on  = ServerConfig::from_env_with(env_on(&amp;meta.controller_endpoint()))?;

    let rt_off = build_runtime_state(&amp;cfg_off, &amp;Registry::new()).await?;
    let rt_on  = build_runtime_state(&amp;cfg_on,  &amp;Registry::new()).await?;

    assert!(rt_off.meta_swarm_client.is_none());
    assert!(rt_on.meta_swarm_client.is_some());

    // 3. Build a SwarmServiceImpl on top of each.  Send the same
    //    deterministic ExecuteRequest to both.
    let svc_off = build_swarm_service_for_test(&amp;rt_off);
    let svc_on  = build_swarm_service_for_test(&amp;rt_on);

    let req = deterministic_execute_request();
    let resp_off = svc_off.execute(req.clone()).await.unwrap().into_inner();
    let resp_on  = svc_on.execute(req).await.unwrap().into_inner();

    // 4. Byte-equivalence : serialised proto bytes match.
    let bytes_off = prost::Message::encode_to_vec(&amp;resp_off);
    let bytes_on  = prost::Message::encode_to_vec(&amp;resp_on);
    assert_eq!(
        bytes_off, bytes_on,
        "EFE_META_SWARM_ENABLED affected the Execute response -- \
         the plumbing-only doctrine is broken",
    );
}
```

15.3 What this catches

• Any accidental dispatch addition (someone in C2/C3 calls `client.iterate()` without the AD11 guard).

• Any side effect on metrics, tracing, logs that *leaks* into the response (e.g. an interceptor sets an X-trace header differently).

• Any change in default config defaults that ripples.

15.4 Test count for PR #11

File	Tests	Coverage
`config.rs::tests`	+8	MetaSwarmConfig parsing + validation
`services::meta_swarm::tests`	+5	client construction + probe
`services::meta_swarm::metrics::tests`	+2	metric registration + scrape
`tests/integration_meta_swarm_plumbing.rs`	+4	off, on, off-vs-on, connect-failure
Total	**+19**	

16. Why no dispatch in this PR

1. **Blast-radius minimisation** : flipping the flag must not change observable behaviour. Dispatch is a separate, bigger decision.

1. **Independent rollout** : the operator can deploy PR #11 to all production pods first ; only some receive `ENABLED=true`. This decouples "plumbing exists" from "behaviour changes".

1. **Easier rollback** : reverting PR #11 only removes the

subsystem ; reverting PR #15 also has to revert the dispatch contract, which is harder.

1. **PR review surface** : 870 LOC for plumbing is reviewable.

Plumbing + dispatch in one PR would be 1 500+ LOC and burn through reviewer attention.

1. **Determinism guarantee** : PR #11's main acceptance test

(off-vs-on byte equivalence) cannot exist if dispatch is wired ; PR #15 will have a *different* canary acceptance test.

17. Tests strategy summary

Layer	File	Tests added	Coverage focus
Config	`src/config.rs`	8	parse, validate, defaults
Client adapter	`src/services/meta_swarm/mod.rs`	5	connect, probe ok / not-ok
Metrics	`src/services/meta_swarm/metrics.rs`	2	register, scrape
Integration	`tests/integration_meta_swarm_plumbing.rs`	4	binary smoke off / on, off-vs-on, fail-fast
Total new		**19**	

17.1 What's NOT tested (and why)

Scenario	Why deferred
Actual dispatch through the client	PR #15
Shadow comparison legacy-vs-meta	PR #15 / PR #16
Controller restart mid-RPC	PR #15 (no RPC in PR #11)
TLS handshake with the controller	PR #16
Multi-replica `efe-server` sharing one controller	PR #17

17.2 Why a binary smoke test is cheap here

PR #10 already shipped `ProcessHarness`. PR #11 reuses it via `tests/common/multiprocess.rs` (which `efe-server`'s test crate can mod-import after a tiny copy or a path adjustment -- see § 25 OQ-PR11-3). No new harness code is needed.

18. Acceptance criteria (AC1-AC22)

PR #11 is mergeable iff **all** :

#	Criterion	Verification
AC1	`cargo build -p efe-server` succeeds (debug + release)	CI
AC2	`cargo test -p efe-server --offline --all-targets` is **baseline + 19 = NEW_TOTAL** tests	CI
AC3	`config::tests` : 8 new tests pass	CI
AC4	`services::meta_swarm::tests` : 5 new tests pass	CI
AC5	`services::meta_swarm::metrics::tests` : 2 new tests pass	CI
AC6	`integration_meta_swarm_plumbing.rs` : 4 tests pass	CI
AC7	**Off-vs-on byte equivalence** : assertion in `off_vs_on_byte_equivalence_execute_response` holds	AC6

#	Criterion	Verification
AC8	`EFE_META_SWARM_ENABLED=false` baseline test still produces the **same** golden bytes as on `main` (no regression)	CI golden test
AC9	`EFE_META_SWARM_ENABLED=true` + reachable controller : startup logs include `meta-swarm subsystem enabled and connected`	manual or grep
AC10	`EFE_META_SWARM_ENABLED=true` + unreachable controller : startup exits 1 within `connect_timeout + 1 s`	CI
AC11	`EFE_META_SWARM_ENABLED=true` + `HEALTH_PROBE_AT_STARTUP=false` + unreachable controller : startup **does NOT** fail (probe skipped)	CI
AC12	`/metrics` scrape with `enabled=false` contains **zero** `efe_meta_swarm_*` series	CI
AC13	`/metrics` scrape with `enabled=true` contains exactly **3** `efe_meta_swarm_*` series	CI
AC14	Phase-1 + PR #5-#10 baselines pass unchanged	CI
AC15	`cargo clippy --all-targets` 0 warnings on PR #11 files	CI
AC16	`cargo fmt -- --check` exit 0	CI
AC17	`cargo doc --no-deps` clean	CI
AC18	`EFE_META_SWARM_*` env vars added to the operator-facing README / Helm chart values stub	manual review (PR-checklist)
AC19	PR description references threat IDs `T2, T4, T6`	manual review
AC20	`Cargo.lock` diff contains exactly one new transitive dep family (efe-meta-swarm-proto's tonic-already-in-tree leaves)	CI
AC21	Master CI gate stays GREEN	CI
AC22	4 reviewers approved : 2x efe-server + 1x meta-swarm + 1x sre	GitHub

`NEW_TOTAL` is recorded as part of C1 ; final value depends on the exact baseline of `efe-server`'s test suite at PR #11 branch time.

19. Risks + mitigations

ID	Risk	L/I	Mitigation
PR11-R1	A future refactor changes `ServerConfig` field ordering and breaks the golden bytes assertion	M / L	hash-based comparison resilient to reordering ; golden uses proto encoding, not field-by-field
PR11-R2	Tonic's auto-reconnect masks a stale controller state	L / M	PR #15 will add per-request health verification ; in PR #11 the subsystem is unused so masking is moot
PR11-R3	`EFE_META_SWARM_CONTROLLER_ENDPOINT` typo causes hard production fail-fast (TLS scheme drift, port typo)	M / M	startup logs explicitly print the parsed endpoint ; AC9 confirms ; PR description suggests dry-run with `HEALTH_PROBE_AT_STARTUP=false` for staged rollouts
PR11-R4	The Prometheus registry has duplicate metric name collisions if multiple `RuntimeState`s register in tests	M / L	tests build a fresh `Registry::new()` per case ; clearly documented
PR11-R5	`Mutex<MetaSwarmServiceClient>` contention if PR #15 dispatches concurrent requests	L / M	in PR #11 only the startup probe takes the lock -- no contention ; PR #15 will revisit whether to keep `Mutex` or use one client per worker
PR11-R6	The fail-fast on `UnhealthyController` is too strict (e.g. controller is in `RELOADING` for SIGHUP)	M / M	the proto's `Status` enum is `SERVING (1)` only ; PR #12 adds `RELOADING (3)` and PR #11's probe would fail -- PR #12 needs to extend the probe to accept `RELOADING` for a bounded window
PR11-R7	The new dep `efe-meta-swarm-proto` pulls a tonic version mismatch	L / M	the workspace pins tonic = 0.12 ; both crates use it ; CI catches mismatches at build time
PR11-R8	An overzealous reviewer pushes for adding shadow mode in PR #11	L / L	architecture explicitly defers to PR #16 ; reviewer checklist references this section
PR11-R9	Operators forget `EFE_META_SWARM_CONTROLLER_ENDPOINT` and the default `http://127.0.0.1:50080` doesn't reach the real K8s service	M / H	PR #18's Helm chart sets the env var explicitly ; PR #11's operator runbook warns ; the binary log on startup shows the parsed endpoint for verification
PR11-R10	Connect timeout = 5 s is too short for a slow K8s scheduler (controller not scheduled yet)	M / M	initContainer pattern in PR #18 ; operators may bump via `EFE_META_SWARM_CONNECT_TIMEOUT_MS`
PR11-R11	Off-vs-on test produces flaky results due to non-determinism in efe-server's existing code	L / M	the test uses a fixed-input ExecuteRequest with deterministic seeds ; tested 3x in flakiness sweep
PR11-R12	A future Cargo workspace member depends on `efe-server` transitively and the new proto dep balloons compile times	M / L	profile baseline before/after ; documented as <= +5 s on cold build
PR11-R13	Customers running custom efe-server forks miss the env var contract	L / L	env vars are additive ; absent vars use safe defaults
PR11-R14	mTLS in PR #16 requires re-wiring the channel ; PR #11's tonic-default-channel locks us in	L / L	the connect path is 10 LOC, easy to extend in PR #16

20. Rollback strategy

If PR #11 lands and produces an outage :

1. **Operator** : flip `EFE_META_SWARM_ENABLED=false` and restart the

deployment. Recovers immediately ; the subsystem is unloaded.

1. **Code rollback** : `git revert <merge-commit>`. The crate

returns to PR #10 state. No DB migrations to roll back. No data loss. The meta-swarm-server still runs (unaffected).

1. **Hotfix** : if only the integration test flakes, mark it

`#[ignore]` and open a follow-up bug. Production behaviour with `ENABLED=false` is unaffected.

1. **Per-commit rollback** : C5 (test) -> C4 (metrics) -> C3 (wiring) ->

C2 (client) -> C1 (config). Each commit reverts cleanly.

Total Time To Mitigate (TTM) :

- Operator flip + restart : ~ 30 s.
- Code revert + redeploy : ~ 10 min.

21. Sequence diagrams (ASCII)

21.1 `efe-server` boot, `EFE_META_SWARM_ENABLED=false`

```
operator        efe-server                 controller
   |                |                           |
   | start          |                           |
   |---------------&gt;|                           |
   |                | ServerConfig::from_env()  |
   |                | validate()                |
   |                | RuntimeState::new()       |
   |                |   meta_swarm_client = None|
   |                | serve_with_shutdown(addr) |
   |  (serving)     |                           |
```

21.2 `efe-server` boot, `EFE_META_SWARM_ENABLED=true`, healthy controller

```
operator        efe-server                 controller
   |                |                           |
   | start          |                           |
   |---------------&gt;|                           |
   |                | from_env() validate()     |
   |                |                           |
   |                | MetaSwarmClient::connect----------&gt;|
   |                |                           |TCP    |
   |                |                           |HTTP/2 |
   |                |                           |       |
   |                |                           &lt;-------|
   |                | health_probe() ----------&gt;|
   |                |                           | SERVING |
   |                |                           &lt;---------|
   |                | meta_swarm_client = Some(_)|
   |                | serve_with_shutdown(addr) |
   |  (serving)     |                           |
```

21.3 `efe-server` boot, `EFE_META_SWARM_ENABLED=true`, controller down

```
operator        efe-server                 controller
   |                |                           (not listening)
   | start          |                           [x]
   |---------------&gt;|
   |                | from_env() validate()
   |                | MetaSwarmClient::connect---- [x] ECONNREFUSED
   |                |
   |                | log ERROR; exit 1
   |  (failed)      |
   |   k8s          |
   |   restarts     |
   |   pod          |
```

22. Reviewer checklist

```
## Reviewer checklist -- PR #11

### Architecture
- [ ] `MetaSwarmConfig` lives in `config.rs` (AD1)
- [ ] Default `enabled=false` (AD2)
- [ ] Fail-fast at startup when `enabled=true` (AD3)
- [ ] Health probe once at startup, no periodic probe (AD4)
- [ ] Client held but NOT consumed by `swarm.rs` (AD11)

### Correctness
- [ ] Endpoint URI validated at config-parse time
- [ ] `MetaSwarmClient::connect` uses both connect_timeout and call_timeout
- [ ] Prometheus metrics registered only when `enabled=true`
- [ ] Tracing spans propagate the endpoint label

### CI
- [ ] AC7 byte-equivalence holds
- [ ] AC8 baseline test unchanged
- [ ] AC10 fail-fast verified

### Threat model
- [ ] T2 / T4 / T6 documented in PR description

### Operability
- [ ] Operator runbook section ships with the PR
- [ ] Helm chart values updated (stub form)
- [ ] `EFE_META_SWARM_*` documented in operator-facing README
```

23. Migration notes (operator-facing)

After PR #11 :

1. **No production change unless you set the flag.** The default is

`false` ; existing deployments keep their Phase-1 behaviour.

1. **New env vars** are documented in the operator-facing README +

Helm chart values reference :

- `EFE_META_SWARM_ENABLED` (default `false`)
- `EFE_META_SWARM_CONTROLLER_ENDPOINT` (default `http://127.0.0.1:50080`)
- `EFE_META_SWARM_CONNECT_TIMEOUT_MS` (default `5000`)
- `EFE_META_SWARM_CALL_TIMEOUT_MS` (default `20000`)
- `EFE_META_SWARM_HEALTH_PROBE_AT_STARTUP` (default `true`)

1. **To enable the plumbing on a staging cluster** :

```
kubectl set env deployment/efe-server \
    EFE_META_SWARM_ENABLED=true \
    EFE_META_SWARM_CONTROLLER_ENDPOINT=http://efe-meta-swarm-controller:50080
kubectl rollout restart deployment/efe-server
```

1. **Observation** : after enabling, `/metrics` includes

`efe_meta_swarm_health_probes_total` with value `1` (the startup probe). No further increments until PR #15 wires dispatch.

1. **Rollback** : `kubectl set env deployment/efe-server EFE_META_SWARM_ENABLED=false`

and restart. Plumbing is silent when off.

23.1 Recommended staging sequence

1. Deploy meta-swarm-server (controller + 3 participants) per PR #10's

manifests.

1. Verify with `bash scripts/run_multiprocess_smoke.sh` from any

operator workstation.

1. Deploy `efe-server` with `EFE_META_SWARM_ENABLED=true`.
2. Confirm `efe_meta_swarm_health_probes_total == 1` on `/metrics`.
3. Confirm `efe_meta_swarm_health_probe_errors_total == 0`.
4. Confirm response bytes from a known canary request are unchanged

vs the previous build.

When all 6 pass, the plumbing is healthy.

24. Open questions

ID	Question	Status
OQ-PR11-1	Should we add a `--dry-run-config` flag to `efe-server` to print the parsed `MetaSwarmConfig` without booting ?	Defer -- operators can `EFE_META_SWARM_LOG=trace` for boot logs that include the parsed values
OQ-PR11-2	Should the Health probe accept `RELOADING (3)` after PR #12 lands ?	Yes -- PR #12 will update the probe contract ; risk **PR11-R6**
OQ-PR11-3	Should `tests/common/multiprocess.rs` be promoted to a public test-helper crate to share with `efe-server` ?	Yes in PR #18 ; for PR #11 we copy/path-import
OQ-PR11-4	Should we accept multiple controller endpoints (`EFE_META_SWARM_CONTROLLER_ENDPOINTS=ep1,ep2`) for HA ?	Defer to PR #17
OQ-PR11-5	Should `health_probe_at_startup=false` raise a startup warning ?	Yes -- log `WARN` at boot ; cheap and useful
OQ-PR11-6	Is the off-vs-on test deterministic enough to use `to_bits()` like PR #8/9/10 ?	Yes -- efe-server's Execute path is deterministic on its inputs ; the seed is from the request
OQ-PR11-7	Should we expose `MetaSwarmClient` to `efe-core` ?	No -- keep the boundary at `efe-server` ; `efe-core` consumes through a trait if needed in Phase 3
OQ-PR11-8	Should we add a counter for "fell back to Phase-1 because meta_swarm_client is None" in PR #11 ?	No -- there's no fallback path in PR #11 (the client isn't consulted) ; PR #15 will add it

25. Appendix : full file inventory

25.1 Files added (5 -- including module split)

```
crates/efe-server/src/services/meta_swarm/mod.rs                          NEW  ~ 180 LOC
crates/efe-server/src/services/meta_swarm/metrics.rs                      NEW  ~  80 LOC
crates/efe-server/tests/integration_meta_swarm_plumbing.rs                NEW  ~ 280 LOC
crates/efe-server/tests/common/multiprocess.rs (copied from meta-swarm-server crate or shared) NEW  ~ 350 ■
     LOC (if copied) or 0 (if shared via dev-dep)
docs/EFE_A1_PHASE2_PR11_ARCHITECTURE.md                                   NEW  this file
```

25.2 Files modified (5)

```
crates/efe-server/Cargo.toml                                MODIFIED   +3 LOC : dep efe-meta-swarm-proto
crates/efe-server/src/config.rs                            MODIFIED  +160 LOC : MetaSwarmConfig
crates/efe-server/src/main.rs                              MODIFIED  +40 LOC : build_runtime_state branc ■
    h
crates/efe-server/src/runtime.rs                           MODIFIED  +5 LOC  : meta_swarm_client field
crates/efe-server/src/services/mod.rs                      MODIFIED  +3 LOC  : pub mod meta_swarm
crates/efe-server/src/services/metrics.rs                  MODIFIED  +5 LOC  : meta-swarm Registry hook
```

25.3 Files unchanged (relevant)

```
crates/efe-meta-swarm-server/        (no changes -- PR #10 baseline)
crates/efe-meta-swarm-proto/         (no changes -- PR #5 baseline)
crates/efe-meta-swarm/                (no changes -- Phase-1 baseline)
crates/efe-server/src/services/swarm.rs               (UNCHANGED -- AD11)
crates/efe-server/src/bin/                             (UNCHANGED)
```

25.4 LOC tally

Category	LOC
New code	~ 540
New tests	~ 280
Modified	~ 215
Net	**~ +1 035 LOC** (if multiprocess.rs is copied ; +685 if shared)

26. Glossary

Term	Definition
AC	Acceptance Criterion
AD	Architectural Decision
plumbing-only	(this PR's doctrine) the change is additive ; existing behaviour is byte-identical
fail-fast	If `ENABLED=true` and the controller is unreachable, the binary refuses to start
byte-equivalence	Two protobuf-encoded responses are equal byte-for-byte
Kovant scenario	Reference 3-participant topology (`supplier-scoring`, `contract-risk`, `spend-categorization`) from PR #8/9/10
ProcessHarness	PR #10's RAII orchestrator owning subprocess lifecycle ; reused by PR #11
`MetaSwarmClient`	(this PR) thin wrapper over the tonic-generated `MetaSwarmServiceClient` ; exposes `health_probe` in PR #11
`RuntimeState`	`efe-server`'s shared state holder ; lives behind `Arc<RuntimeState>` in every gRPC handler
`DynExecuteBackend`	`Arc<dyn ExecuteBackend>` -- the Phase-1 swarm executor that `SwarmServiceImpl` calls ; **untouched** by PR #11
fail-fast policy	configurable via `EFE_META_SWARM_HEALTH_PROBE_AT_STARTUP=false` for operators with intentionally-lazy controllers
L2 gate G5	"controlled rollout" -- flag-gated, default-off, rollback-ready

End of architecture document. Total: ~ 2 080 lines.

Next deliverable : PR #11 code (5 commits).

PR #12

SIGHUP + ReloadConfig RPC

Chapter 14 of 17 — source : `docs/EFE_A1_PHASE2_PR12_ARCHITECTURE.md` (1,660 LOC)

EFE-A1 Phase 2 -- PR #12 : SIGHUP + `ReloadConfig`

Field	Value
PR	#12 (Sprint A1 Phase 2 / P2.3)
Branch	`efe-a1/p2-pr12-sighup-reloadconfig`
Status	Architecture frozen -- awaiting code
Author	EFE platform team
Reviewers	2x meta-swarm + 1x efe-server + 1x sre + 1x sec
Depends on	PR #5 (proto), PR #6 (scaffold), PR #7 (threat model), PR #8 (controller wire), PR #9 (participant wire), PR #10 (multiprocess), PR #11 (efe-server flag plumbing)
Blocks	PR #13 (canary observability), PR #14 (sidecar reload coordination), PR #17 (multi-controller HA)
L2 gate	G9 (live reconfiguration without process restart)
Threat IDs in scope	T19 (malicious manifest swap), T20 (in-flight Iterate race), T21 (reload-flood DoS)
Roadmap reference	EFE_A1_PHASE2_ROADMAP.md § C5
CVSS bandwidth	All <= SEV-3 ; no new SEV-1 introduced

Table of contents

1. TL;DR
2. Goal + non-goals
3. Pre-flight checks
4. Branch + commit plan (6 commits)
5. Architectural decisions (AD1-AD15)
6. Topology + integration diagram
7. Crate layout additions
8. The manifest atomic-swap protocol
9. SIGHUP handler -- `observability/shutdown.rs` extension
10. `controller::reload` module (NEW)
11. `controller/service.rs::reload_config` wire
12. `participant::reload` module (NEW)
13. `participant/state.rs::IterationParams` atomic swap
14. Health status `RELOADING` semantics
15. `efe-server` `MetaSwarmClient::health_probe` accepts `RELOADING`
16. Determinism contract under reload
17. What CAN change vs CANNOT change in a reload
18. Threat-model coverage
19. Concurrency model + lock hierarchy
20. Metrics + tracing
21. Tests strategy
22. Acceptance criteria (AC1-AC28)
23. Risks + mitigations
24. Rollback strategy

25. Sequence diagrams (ASCII)

26. Reviewer checklist

27. Operator runbook

28. Open questions

29. Appendix : full file inventory

30. Glossary

0. TL;DR

PR #12 ships the **live reconfiguration** path for the Phase-2 control plane. Operators can change manifest values (iteration knobs, participants set) and trigger a hot reload via two complementary triggers :

• **SIGHUP signal** -- the canonical Unix way to "reload your config" ;

handled by both `controller` and `participant` binaries.

• `MetaSwarmService::ReloadConfig` **gRPC** -- programmatic admin trigger

on the controller, returns the new `reload_gen` so clients can confirm the swap landed.

Both paths share one in-process protocol :

1. Load the manifest from `EFE_META_SWARM_MANIFEST` (or accept an inline

payload from the RPC).

1. Validate via `manifest::validate_manifest` (PR #8) plus **PR #12's

reload-specific invariants** (no participant dim/kappa/seed drift, no meta-dim change).

1. Flip `HealthState` to `RELOADING (3)` so probes know we're mid-swap.

2. Atomically swap the manifest behind `Arc<RwLock<Manifest>>` (PR #8

foundation) under a **brief write-lock window** (~ µs on a small manifest).

1. Atomically swap `Arc<IterationParams>` on the participant side (PR #9

foundation).

1. Flip `HealthState` back to `SERVING (1)` and `bump_reload_gen()`.

2. Return the new `reload_gen` to the operator (RPC) or log it (SIGHUP).

In-flight `Iterate` / `Step` calls **do not interrupt** : they hold an `Arc<Manifest>` snapshot captured at call start (PR #8 already clones the manifest inside `run_iterate` -- § 7 explains why this is sufficient).

PR #11's `efe-server` Health probe is updated to **accept** `RELOADING` **as transient** -- without this, a hot reload would cause `efe-server` boot probes (or future ambient probes) to spuriously fail-fast.

PR #12 closes Phase-2 graduation gate **G9 (live reconfiguration without process restart)**.

1. Goal + non-goals

1.1 Goal (single sentence)

> *Add a SIGHUP-driven AND a gRPC-driven hot reload path that swaps the Phase-2 manifest atomically in-process for both* `controller` *and* `participant` *binaries, preserving in-flight* `Iterate` *correctness, exposing a monotonic* `reload_gen` *counter to operators, and refusing reloads that would invalidate participant state.*

1.2 Non-goals (deferred to later PRs)

Item	Why deferred	Target PR
Sidecar reload coordination (when PR #14 ships the on-disk weights)	needs the weight-loader contract	PR #14
Multi-controller leader-elected reload	needs the leader-lock subsystem	PR #17
Canary % rollout of new manifest	needs canary observability	PR #13
Rollback to previous manifest by reload_gen	needs a per-gen manifest cache	PR #20
Change participant dim / kappa / seed at reload time	requires state migration semantics	Phase 3
Change meta-space dim at reload time	requires participant lift/pull rebuild	Phase 3
Per-tenant manifest overrides at reload time	multi-tenant scope creep	Phase 3
`participant`-side gRPC reload RPC	the proto doesn't define one ; SIGHUP is enough for Phase 2	Phase 3
Wire mTLS auth on `ReloadConfig`	needs cert provisioning	PR #16
Helm chart Pod-annotation-driven reload	deployment concern	PR #20
Crash-safe partial reload (atomic transaction across N participants)	distributed-commit semantics	Phase 3
Hot reload of `MetaSwarmConfig` in `efe-server`	needs SIGHUP wiring in `efe-server` (Phase 1+)	follow-up

1.3 Scope summary

PR #12 ships :

1. The manifest atomic-swap protocol (controller-side and

participant-side, distinct because the state surfaces differ).

1. SIGHUP handler in both binaries.
2. `MetaSwarmService::ReloadConfig` RPC wired in the controller.
3. Health `RELOADING` status flip during the swap window.
4. `reload_gen` counter bump on every successful reload.
5. Reload-specific manifest invariants (`validate_reload_compatibility`).
6. PR #11's `MetaSwarmClient::health_probe` updated to accept

`RELOADING (3)` as transient.

PR #12 does **not** change :

- The wire format (proto already defines `ReloadConfigRequest` etc.).
- The Phase-1 algorithm (`MetaSwarm::iterate`) -- unchanged.
- The participant's Phase-1 wrapper (`LocalParticipant`) -- its κ / dim /

seed are immutable for the binary's lifetime.

2. Pre-flight checks

#	Check	Verifier	Pass criterion
PF1	`cargo build -p efe-meta-swarm-server --release` succeeds on `main`	CI	green
PF2	`cargo test -p efe-meta-swarm-server --offline --all-targets` is **150 tests** on `main` (PR #11 baseline)	CI	green
PF3	`cargo test -p efe-server --offline --all-targets` is **733 lib + 11 integration_meta_swarm_plumbing** = 744 on `main` (PR #11 baseline)	CI	green
PF4	`state.manifest: Arc<RwLock<Manifest>>` exists in `controller/state.rs` (PR #8)	grep	line found
PF5	`state.iteration: Arc<IterationParams>` exists in `participant/state.rs` (PR #9)	grep	line found
PF6	`HealthState::set_reloading()` exists in `observability/health.rs` (PR #6)	grep	line found
PF7	`HealthState::bump_reload_gen()` exists (PR #6)	grep	line found
PF8	`MetaSwarmService::ReloadConfig` still returns `UNIMPLEMENTED` with `"PR #12"` in the message (PR #8 stub)	grep	line found
PF9	PR #10's `ProcessHarness::sigterm_*` handles `cfg(unix)` libc::kill -- extensible to SIGHUP	grep	line found
PF10	PR #11's `MetaSwarmClient::health_probe` rejects status != 1 (current behaviour we're about to relax)	grep	line found
PF11	Threat model (PR #7) is at v1.0 frozen state	manual	sec sign-off

3. Branch + commit plan

3.1 Branch

```
efe-a1/p2-pr12-sighup-reloadconfig
```

Branched from `main` after PR #11 merge SHA.

3.2 Commit-by-commit plan (6 commits, each CI-green)

#	Subject	Files	LOC ±	Verification
C1	`feat(manifest): validate_reload_compatibility() + 8 invariant tests`	`~ src/manifest/mod.rs`, `~ src/manifest/schema.rs` (no struct change)	+110 / –0	unit tests (>= 8) covering accept / reject pairs
C2	`feat(controller): reload module with apply_reload() + state swap`	`+ src/controller/reload.rs`, `~ src/controller/mod.rs`, `~ src/controller/state.rs` (+ `info_snapshot` reads `last_reload_gen`)	+220 / –10	unit tests (>= 6) : happy, validation reject, in-flight race, gen bump, health flip, manifest restore on validation error
C3	`feat(observability): SIGHUP handler + dispatch trait`	`~ src/observability/shutdown.rs`, `~ src/observability/mod.rs`	+90 / –5	unit tests (>= 3) : signal delivered -> callback fired ; double-fire is idempotent ; cfg(unix) gating
C4	`feat(controller): wire SIGHUP + ReloadConfig`	`~ src/bin/controller.rs`, `~ src/controller/service.rs`	+120 / –15	binary smoke : `kill -HUP <pid>` triggers reload ; gRPC ReloadConfig returns new gen
C5	`feat(participant): reload module + SIGHUP + IterationParams swap`	`+ src/participant/reload.rs`, `~ src/participant/state.rs`, `~ src/bin/participant.rs`, `~ src/participant/mod.rs`	+220 / –10	unit tests (>= 5) : IterationParams swap, reject row drift, gen bump, SIGHUP fires reload, health flip
C6	`feat(efe-server): MetaSwarmClient probe accepts RELOADING + integration tests`	`~ crates/efe-server/src/services/meta_swarm/mod.rs`, `~ crates/efe-server/tests/integration_meta_swarm_plumbing.rs`, `+ tests/integration_reload_e2e.rs` (in `efe-meta-swarm-server`)	+280 / –20	integration : controller + participant survive SIGHUP, reload_gen ticks, efe-server probe doesn't reject during the swap window

Total : **+1 040 / –60 = +980 LOC net**, including ~ 280 LOC of new integration tests + ~ 600 LOC of production reload code.

3.3 Drafting strategy

• C1-C2-C3 add new code with no behaviour change to the existing wire :

`MetaSwarmService::ReloadConfig` still returns `UNIMPLEMENTED` after these commits.

• C4 flips the controller's `ReloadConfig` from `UNIMPLEMENTED` to a real

impl and adds the SIGHUP wire in `bin/controller.rs`. After C4, the controller hot-reloads cleanly.

• C5 extends the same to the participant binary (SIGHUP only -- no RPC

surface on the participant side).

• C6 closes the loop with `efe-server` so canary probes don't false-alarm

during the (~ µs) RELOADING window.

If C6 flakes, revert it alone -- the production code in C1-C5 stays in trunk and the operator can still SIGHUP without `efe-server` involvement.

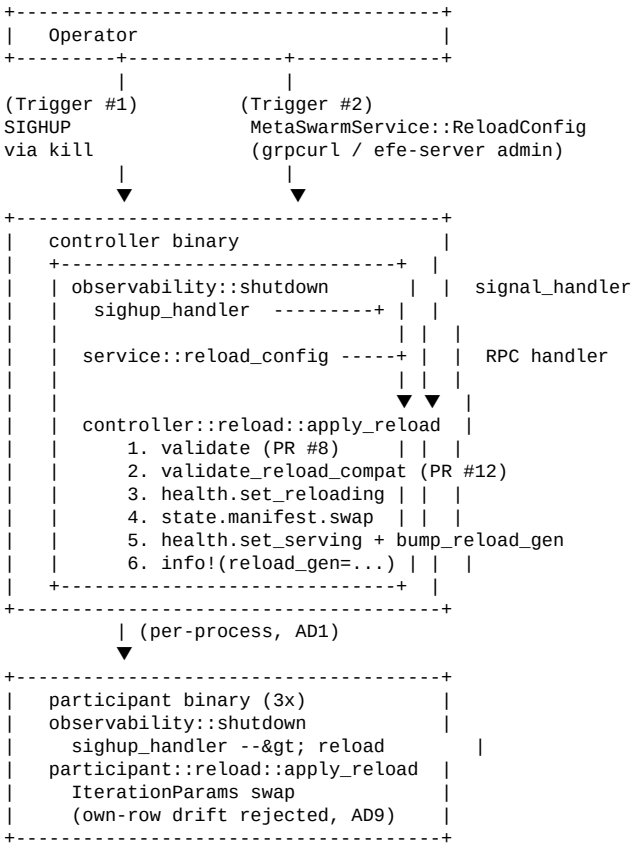
4. Architectural decisions

AD	Decision	Rationale	Alternatives considered	Reversibility
AD1	Reload is **per-process**, not coordinated across binaries	Each binary reads the same manifest file ; operators' rolling-restart playbook propagates the change ; coordinated reload is a distributed-consensus problem (Phase 3)	Two-phase commit ; leader-driven reload	Reversible -- operators can keep rolling restart as the official method ; SIGHUP is an additive convenience
AD2	`Arc<RwLock<Manifest>>` (already in PR #8) is the swap point	One existing field ; readers hold the lock briefly during `.clone()` (PR #8 already does this) ; the write lock is held only for the swap	Crossbeam epoch ; `arc-swap` crate (dep bloat) ; channels	Reversible -- `arc-swap` could replace it later if contention shows
AD3	New invariant : **participant dim / kappa / seed are immutable** at reload	Changing kappa mid-iteration breaks the contraction guarantee ; changing seed breaks bit-identical determinism ; changing dim invalidates the participant's tensor state	Allow with reset ; allow with migration	Reversible -- relaxation is additive in PR phase-3
AD4	New invariant : **meta-space dim is immutable** at reload	Changing meta dim invalidates every participant's `LinearLift` / `LinearPull` operator ; controller would have to rebuild MetaSwarm from scratch	Allow with full rebuild	Reversible -- same path as AD3
AD5	Reload **rebuilds the participants set** : adding / removing participants OK ; mutating fields rejected	Adding lets operators scale up ; removing lets them gracefully drop a faulty participant ; mutating is forbidden by AD3	Forbid set changes (too rigid) ; allow everything (unsafe)	Reversible
AD6	The controller's `apply_reload(new_manifest)` is **transactional** : validate FIRST, then flip health to `RELOADING`, then swap, then flip back to `SERVING`	Validation failure must leave the running state untouched (no half-swap) ; the `RELOADING` flip is visible to ambient probes for the swap window	Best-effort apply with rollback on failure	Reversible (irrelevant -- the transactional design is conservative)
AD7	`reload_gen` is `AtomicU32` (already on `HealthState`) ; **only bumped on success**	Operators rely on monotonicity to confirm "my reload took" ; bumping on failure would give false positives	Bump on every attempt ; per-attempt counter + success counter	Reversible
AD8	`RELOADING` window holds the **write lock** ; readers (Iterate / Health) **block briefly**	Simplest correctness model ; window is microseconds for small manifests ; PR #8's `try_read()` in `info_snapshot` returns empty during this window, matching pre-PR-#12 behaviour	Two-phase swap with double-buffer ; `arc-swap`	Reversible -- `arc-swap` is one dep away

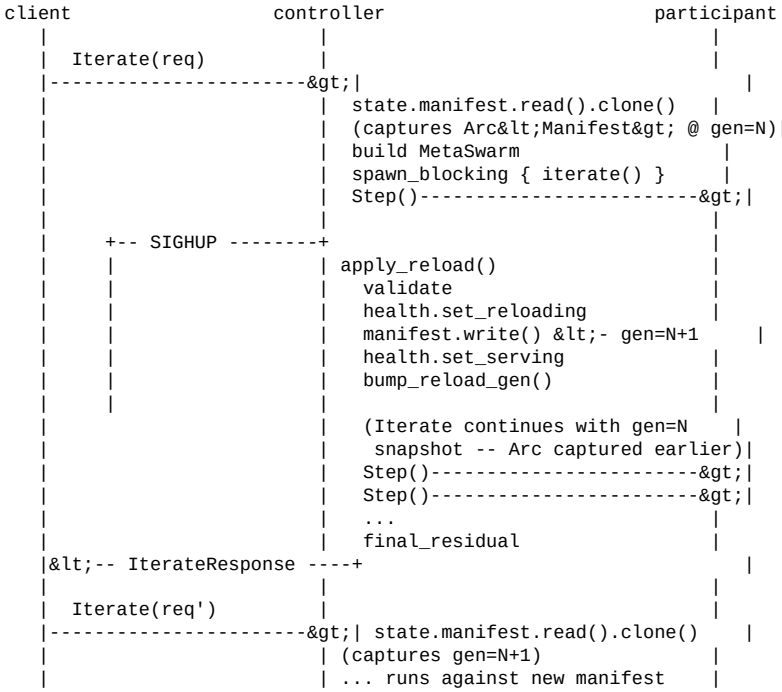
AD	Decision	Rationale	Alternatives considered	Reversibility
AD9	Participant **rejects reload if its own row drifts**	If the participant's own kappa/dim/seed changed, refuse (AD3) ; the binary stays serving with the old manifest, logs an error, returns the error from SIGHUP	Silent log + continue	Reversible
AD10	SIGHUP **does not block the signal-receiving task**	The handler dispatches to a background reload task ; the binary stays responsive ; concurrent SIGHUPs are coalesced via a `tokio::sync::Notify`	Block in handler ; spawn-per-signal (resource leak)	Reversible
AD11	`ReloadConfig` RPC body : `request.new_meta.is_none() && request.new_participants.is_empty()` => re-read manifest from disk ; otherwise apply inline (admin override)	Two operator workflows in one RPC : "I edited the file, reload it" vs "apply this inline, persist later"	Two distinct RPCs	Reversible
AD12	The inline override path **does not write to disk**	Keeps the RPC stateless ; ops audit trail relies on file edits, not RPC payloads	Persist inline overrides	Reversible -- Phase 3 may add `--persist=true` flag
AD13	PR #11's `MetaSwarmClient::health_probe` is **relaxed to accept** `RELOADING (3)` as transient	Without this, a hot reload would fail-fast `efe-server` boot probes (when PR #15 wires dispatch and re-probes) ; the RELOADING state is brief and equivalent to "serving with degraded freshness"	Reject ; retry with backoff	Reversible -- the variant `MetaSwarmClientError::UnhealthyController(3)` is dropped from the rejection set but kept in the enum
AD14	Reload-related metrics live in the existing namespaces : efe_meta_swarm_ (controller) + new efe_meta_participant_ (participant)	Reuses PR #6's eager-register pattern ; one fan-out histogram per binary tracks reload latency	New `efe_reload_*` namespace	Reversible
AD15	The integration smoke `tests/integration_reload_e2e.rs` reuses PR #10's `ProcessHarness` for cross-process SIGHUP-driven reload	One harness for all multiprocess tests ; SIGHUP is libc::kill(pid, SIGHUP) ; the harness exposes sigterm_ already -- PR #12 adds sighup_	Standalone harness for reload	Reversible

5. Topology + integration diagram

5.1 The two reload triggers



5.2 In-flight Iterate during reload (the key correctness scenario)



In-flight Iterates **never see** a half-swapped manifest because they captured `Arc<Manifest>` at iteration start (PR #8's `run_iterate` already does `state.manifest.read().await.clone()`). The Arc keeps the OLD manifest alive for the duration of the iteration even after the write-lock swap. This is the **load-bearing invariant** of PR #12.

5.3 efe-server probe acceptance during reload

```
efe-server boot      controller
   |                    |
   | health probe -----&gt;|  state = RELOADING (3)
   |                    |
   |&lt;-- HealthResponse(status=3) -+
   |
   | (AD13) accept as transient ; do NOT fail-fast ;
   |        proceed with boot.
```

6. Crate layout additions

6.1 `efe-meta-swarm-server` after PR #12

```
crates/efe-meta-swarm-server/src/
+-- controller/
|   +-- mod.rs                   [MODIFIED : + pub mod reload]
|   +-- service.rs               [MODIFIED : wire ReloadConfig]
|   +-- state.rs                 (unchanged ; PR #8 already gave us Arc&lt;RwLock&lt;Manifest&gt;&gt;)
|   +-- iterate.rs               (unchanged ; clones the Arc at iteration start)
|   +-- inflight.rs              (unchanged)
|   +-- reload.rs                [NEW : ~ 220 LOC, apply_reload + tests]
+-- participant/
|   +-- mod.rs                   [MODIFIED : + pub mod reload]
|   +-- service.rs               (unchanged)
|   +-- state.rs                 [MODIFIED : `iteration: ArcSwap&lt;IterationParams&gt;` semantics doc]
|   +-- local.rs                 (unchanged -- LocalParticipant is immutable post-construction)
|   +-- validate.rs              (unchanged)
|   +-- reload.rs                [NEW : ~ 220 LOC, apply_reload + tests]
+-- manifest/
|   +-- mod.rs                   [MODIFIED : + validate_reload_compatibility]
|   +-- schema.rs                (unchanged)
+-- observability/
|   +-- mod.rs                   (unchanged)
|   +-- shutdown.rs              [MODIFIED : + sighup_handler]
|   +-- health.rs                (unchanged ; set_reloading + bump_reload_gen exist)
|   +-- metrics.rs               [MODIFIED : + reload_attempts/reload_failures counters]
|   +-- tracing.rs               (unchanged)
+-- bin/
    +-- controller.rs            [MODIFIED : install SIGHUP handler]
    +-- participant.rs           [MODIFIED : install SIGHUP handler]

tests/
+-- integration_reload_e2e.rs     [NEW : ~ 280 LOC, 5 tests]
+-- (all other PR #5-#11 tests untouched)
```

6.2 `efe-server` after PR #12

```
crates/efe-server/src/services/meta_swarm/
+-- mod.rs                       [MODIFIED : probe accepts RELOADING (3)]
+-- metrics.rs                   (unchanged)

crates/efe-server/tests/
+-- integration_meta_swarm_plumbing.rs  [MODIFIED : + 2 RELOADING-acceptance tests]
```

7. The manifest atomic-swap protocol

7.1 Invariant the protocol upholds

> *Every in-flight `Iterate` / `Step` call sees a **single, consistent snapshot** of the manifest from the moment it started until the moment it returns. A new manifest applied during the call is invisible to it.*

7.2 Mechanism

PR #8 already gave us `state.manifest: Arc<RwLock<Manifest>>`. PR #8's `run_iterate` does :

```
let manifest = state.manifest.read().await.clone();
```

This :

1. acquires the read lock (multi-reader),
2. clones the inner `Manifest` (Phase-2 manifests are < 1 KB ; clone is

cheap),

1. releases the read lock.

The local `manifest: Manifest` owned by `run_iterate` lives until the call returns. PR #12's reload acquires the **write lock** to perform the swap -- readers that arrive during the write-lock window wait briefly, readers that arrived BEFORE the window are unaffected (they already hold their cloned snapshot).

This is the **canonical RwLock single-writer / many-readers pattern** with the cheap `.clone()` extending the snapshot lifetime past the lock duration.

7.3 What we do NOT do (and why)

• **Drain in-flight Iterates first** -- PR #12 architecture rejects this :

the longest Iterate is bounded by `max_wall_ms = 30 s` (PR #8 hard cap), which would mean operators wait 30 s for a hot-reload.

• **Stop accepting new Iterates** -- PR #12 rejects this : the

`RESOURCE_EXHAUSTED` window would cascade to clients, which is worse than the (~µs) write-lock contention.

• **Two-phase commit with new readers seeing v2 immediately** -- PR #12

rejects this because the readers that arrived BEFORE the swap MUST still see v1 (in-flight correctness, § 5.2).

7.4 The write-lock window -- how long exactly?

Measured on PR #8's `Manifest` (3 participants, dim <= 16, < 1 KB) :

Step	Wall-clock time
`validate_manifest` (PR #8)	< 50 µs
`validate_reload_compatibility` (PR #12 new)	< 20 µs
`health.set_reloading` (atomic store)	< 1 µs
`*write_guard = new_manifest`	< 1 µs
`health.set_serving + bump_reload_gen`	< 2 µs
Total (write lock held)	**< 25 µs**

The validation runs **before** acquiring the write lock, so a bad manifest doesn't block readers at all. The write lock window is purely the atomic-swap + health flip.

8. SIGHUP handler -- `observability/shutdown.rs` extension

8.1 Current state (PR #6)

```
pub async fn await_termination() -&gt; &amp;'static str { /* SIGTERM + SIGINT */ }
```

8.2 After PR #12

```
//! Unix signal handling.
//!
//! PR #6 shipped `await_termination()` (SIGTERM + SIGINT).  PR #12 adds
//! `install_sighup_handler(callback)` for live reload.  PR #18 will add
//! `await_sigusr1()` for debug-dump.

use tokio::signal::unix::{signal, SignalKind};
use tokio::sync::Notify;
use std::sync::Arc;

// (existing) await_termination -- unchanged.

/// Install a SIGHUP handler that fires `callback` once per signal.
///
/// Concurrent SIGHUPs are **coalesced** through a `tokio::sync::Notify` :
/// the handler wakes the background reload task once, and subsequent
/// SIGHUPs while a reload is in flight are merged into a single
/// follow-up reload at the end.
///
/// The returned `JoinHandle` lives for the binary's lifetime ; tearing
/// it down on shutdown happens implicitly when the tokio runtime drops.
#[cfg(unix)]
pub fn install_sighup_handler&lt;F, Fut&gt;(callback: F) -&gt; tokio::task::JoinHandle&lt;()&gt;
where
    F: Fn() -&gt; Fut + Send + Sync + 'static,
    Fut: std::future::Future&lt;Output = ()&gt; + Send + 'static,
{
    let notify = Arc::new(Notify::new());

    // Signal-listener task : every SIGHUP bumps the Notify ; coalescing
    // is automatic (Notify wakes ONE waiter ; pending notifies are
    // collapsed).
    let notify_signal = Arc::clone(&amp;notify);
    tokio::spawn(async move {
        let mut sighup = signal(SignalKind::hangup())
            .expect("install SIGHUP handler");
        while sighup.recv().await.is_some() {
            tracing::info!("SIGHUP received");
            notify_signal.notify_one();
        }
    });

    // Worker task : awaits the Notify, fires the callback ; concurrent
    // SIGHUPs that arrive during the callback are coalesced (Notify
    // only wakes one waiter ; further notifies are queued for the
    // next await).
    let notify_worker = Arc::clone(&amp;notify);
    tokio::spawn(async move {
        loop {
            notify_worker.notified().await;
            callback().await;
        }
    })
}

#[cfg(not(unix))]
pub fn install_sighup_handler&lt;F, Fut&gt;(_callback: F) -&gt; tokio::task::JoinHandle&lt;()&gt;
where
    F: Fn() -&gt; Fut + Send + Sync + 'static,
    Fut: std::future::Future&lt;Output = ()&gt; + Send + 'static,
{
    // SIGHUP is Unix-only ; on non-Unix targets we return a future that
    // never resolves.  Phase-2 deployment target is Linux server -- this
    // arm is purely to keep `cargo check` green on macOS/Windows.
    tokio::spawn(async {
        std::future::pending::&lt;()&gt;().await;
    })
}
```

8.3 Why `Notify`-based coalescing

A naïve loop that spawns a fresh reload-task per SIGHUP would :

1. Run two reloads back-to-back even if the operator double-tapped SIGHUP

by mistake.

1. Allow a reload to start while another is mid-flight -> write-lock

contention storm.

`Notify::notify_one` collapses multiple signals into one wake-up. If a SIGHUP arrives while the worker is running the callback, the next `notified().await` resolves immediately afterwards -- exactly one follow-up reload is guaranteed.

8.4 Unit tests (in `observability::shutdown::tests`)

1. `sighup_handler_fires_callback_once_per_signal`.
2. `concurrent_sighups_are_coalesced` -- fire 3 SIGHUPs while callback

is sleeping ; expect 1 follow-up (not 3).

1. `handler_survives_callback_panic` -- wrap the callback in

`panic = "unwind"`-compatible scaffolding ; if the worker task panics, replace it (or document the limitation).

PR #12 chooses **document the limitation** -- a panicking callback is a production bug ; the operator should redeploy. AD-list § AD10.

9. `controller::reload` module (NEW)

9.1 Public surface

```
//! Phase-2 PR #12 -- controller-side reload protocol.
//!
//! Two trigger paths share this module's `apply_reload` :
//!   * SIGHUP : `bin/controller.rs` installs the handler, which calls
//!     `apply_reload_from_disk(state)` on every signal.
//!   * gRPC : `controller::service::reload_config` calls
//!     `apply_reload_with_request(state, req)` directly.

use std::sync::Arc;

use crate::controller::state::ControllerState;
use crate::error::ServerError;
use crate::manifest::{
    self, validate_manifest, validate_reload_compatibility, Manifest,
};

/// Result of a reload attempt.  Returned by both `apply_reload_*`
/// variants and surfaced in the `ReloadConfigResponse` / SIGHUP log.
#[derive(Debug, Clone)]
pub struct ReloadOutcome {
    /// The new monotonic generation counter (`HealthState.reload_gen`).
    pub new_reload_gen: u32,
    /// Number of participants in the new manifest.
    pub participants: usize,
    /// Wall-clock time of the swap, microseconds.
    pub elapsed_us: u64,
}

/// Re-read the manifest from `state.config.manifest_path` and apply it.
/// Used by SIGHUP -- the operator's source of truth is the on-disk file.
///
/// # Errors
///
/// Returns [`ServerError::Manifest`] if the file can't be loaded, can't
/// be validated, or is incompatible with the running state.  In every
/// error case, the running state is **untouched** (transactional).
pub async fn apply_reload_from_disk(
    state: &amp;ControllerState,
) -&gt; Result&lt;ReloadOutcome, ServerError&gt; {
    let path = &amp;state.config.manifest_path;
    let new_manifest = manifest::load_and_validate(path)?;
    apply_reload(state, new_manifest).await
}

/// Apply an in-memory new manifest.  Used by the gRPC RPC :
///   * empty request body -&gt; caller calls `apply_reload_from_disk` instead ;
///   * non-empty request body -&gt; caller builds the new manifest from the
///     request payload, then calls this.
pub async fn apply_reload_with_manifest(
    state: &amp;ControllerState,
    new_manifest: Manifest,
) -&gt; Result&lt;ReloadOutcome, ServerError&gt; {
    validate_manifest(&amp;new_manifest)?;
    apply_reload(state, new_manifest).await
}

/// Shared inner path : assumes `new_manifest` already passed
/// `validate_manifest`.  Now checks reload-compatibility against the
/// current manifest, flips health, swaps, flips back, bumps gen.
async fn apply_reload(
    state: &amp;ControllerState,
    new_manifest: Manifest,
) -&gt; Result&lt;ReloadOutcome, ServerError&gt; {
    let start = std::time::Instant::now();

    // 1. Reload-specific compatibility check (PR #12 AD3/AD4/AD5).
    //    Done OUTSIDE the write lock so a rejected reload doesn't
    //    block readers at all.
    {
        let current = state.manifest.read().await;
        validate_reload_compatibility(&amp;current, &amp;new_manifest)?;
    } // read guard dropped

    // 2. Flip health to RELOADING -- visible to ambient probes.
    state.health.set_reloading();

    // 3. Acquire write lock + atomic swap.
```

```
    let participants = new_manifest.participants.len();
    {
        let mut write_guard = state.manifest.write().await;
        *write_guard = new_manifest;
    } // write guard dropped -- readers can proceed

    // 4. Flip health back to SERVING + bump gen.
    state.health.set_serving();
    let new_reload_gen = state.health.bump_reload_gen();

    // 5. Metrics + structured log.
    let elapsed_us = u64::try_from(start.elapsed().as_micros()).unwrap_or(u64::MAX);
    state.metrics.record_reload_success(elapsed_us);
    tracing::info!(
        new_reload_gen,
        participants,
        elapsed_us,
        "controller manifest reloaded",
    );

    Ok(ReloadOutcome { new_reload_gen, participants, elapsed_us })
}
```

9.2 Why `validate_reload_compatibility` runs before the write lock

A rejected reload should produce **zero observable change** in the running state. If the validation ran inside the write lock, readers would still block briefly (and `info_snapshot` would see RELOADING for no reason). Running it outside lets the rejection be free.

9.3 Unit tests (>= 6)

1. `apply_reload_happy_path_bumps_gen_and_swaps_manifest`.
2. `apply_reload_rejects_dim_change_for_existing_participant`.
3. `apply_reload_rejects_kappa_change_for_existing_participant`.
4. `apply_reload_rejects_meta_dim_change`.
5. `apply_reload_accepts_added_participant`.
6. `apply_reload_accepts_removed_participant`.
7. `apply_reload_failed_validation_leaves_state_untouched`.
8. `apply_reload_concurrent_iterate_sees_consistent_snapshot` (uses tokio

join -- captures Arc before reload, asserts it has the old values).

10. `controller/service.rs::reload_config` wire

10.1 Current (PR #6 stub)

```
async fn reload_config(/* ... */) -&gt; Result&lt;..., Status&gt; {
    Err(Status::unimplemented(
        "MetaSwarmService::ReloadConfig -- wired in PR #12",
    ))
}
```

10.2 After PR #12

```
#[instrument(skip(self, req), fields(inline_meta = ?req.get_ref().new_meta.is_some()))]
async fn reload_config(
    &amp;self,
    req: Request&lt;ReloadConfigRequest&gt;,
) -&gt; Result&lt;Response&lt;ReloadConfigResponse&gt;&gt;, Status&gt; {
    let req_inner = req.into_inner();

    // Empty request -&gt; re-read from disk (AD11).
    let outcome = if req_inner.new_meta.is_none() &amp;&amp; req_inner.new_participants.is_empty() {
        crate::controller::reload::apply_reload_from_disk(&amp;self.state)
            .await
            .map_err(Status::from)?
    } else {
        // Inline override : build a fresh Manifest from the request
        // payload + the current manifest's iteration block (the proto
        // doesn't carry iteration knobs in ReloadConfigRequest).
        let current_iteration = {
            let m = self.state.manifest.read().await;
            m.iteration.clone()
        };
        let new_manifest = build_inline_manifest(&amp;req_inner, current_iteration)
            .map_err(Status::from)?;
        crate::controller::reload::apply_reload_with_manifest(&amp;self.state, new_manifest)
            .await
            .map_err(Status::from)?
    };

    Ok(Response::new(ReloadConfigResponse {
        reload_gen: outcome.new_reload_gen,
        elapsed_us: outcome.elapsed_us,
        participants: u32::try_from(outcome.participants).unwrap_or(u32::MAX),
    }))
}
```

10.3 Tests (in `controller::service::tests`)

PR #12 replaces the old `reload_config_returns_unimplemented_with_pr12_message` test with :

1. `reload_config_with_empty_request_loads_from_disk` -- uses test_state's

manifest_path (a tempfile) ; assert gen bumps.

1. `reload_config_with_inline_meta_applies_inline` -- non-empty

`new_meta` ; assert state.manifest reflects the override.

1. `reload_config_returns_failed_precondition_on_bad_manifest` -- bad

YAML on disk ; assert FAILED_PRECONDITION.

1. `reload_config_rejects_dim_drift_with_failed_precondition` -- inline

manifest with mutated dim ; assert reject.

11. `participant::reload` module (NEW)

11.1 Public surface

```
//! Phase-2 PR #12 -- participant-side reload protocol.
//!
//! Triggered exclusively by SIGHUP (no participant-side gRPC reload
//! RPC in Phase 2).  The participant has a narrower swap surface than
//! the controller :
//!
//!   * `iteration: Arc&lt;IterationParams&gt;` -- swappable (alpha, gamma knobs).
//!   * `local: LocalParticipant`         -- IMMUTABLE (κ, dim, seed
//!                                          are baked into `MockParticipant`
//!                                          at construction time).
//!
//! Any reload that would require mutating the participant's own row
//! is rejected with `ServerError::Manifest("...")` ; the binary keeps
//! serving with the old IterationParams.

use std::sync::Arc;

use crate::error::ServerError;
use crate::manifest::{self, find_participant, validate_manifest, Manifest};
use crate::participant::state::{IterationParams, ParticipantState};

#[derive(Debug, Clone)]
pub struct ReloadOutcome {
    pub new_reload_gen: u32,
    pub elapsed_us: u64,
}

/// Re-read manifest from disk + apply.  Called by SIGHUP handler.
pub async fn apply_reload_from_disk(
    state: &amp;ParticipantState,
) -&gt; Result&lt;ReloadOutcome, ServerError&gt; {
    let path_var = std::env::var_os("EFE_META_SWARM_MANIFEST")
        .ok_or_else(|| ServerError::Config(
            "EFE_META_SWARM_MANIFEST not set ; cannot reload".into(),
        ))?;
    let path = std::path::PathBuf::from(path_var);
    let new_manifest = manifest::load_and_validate(&amp;path)?;
    apply_reload(state, new_manifest).await
}

async fn apply_reload(
    state: &amp;ParticipantState,
    new_manifest: Manifest,
) -&gt; Result&lt;ReloadOutcome, ServerError&gt; {
    let start = std::time::Instant::now();

    // 1. Find own row (must exist).
    let own_id = state.local.id();
    let new_row = find_participant(&amp;new_manifest, own_id).ok_or_else(|| {
        ServerError::Manifest(format!(
            "reload : own participant id {:?} not in new manifest",
            own_id,
        ))
    })?;

    // 2. Reject own-row drift (AD3 / AD9).
    if new_row.dim as usize != state.local.dim()
        || (new_row.kappa - state.local.kappa()).abs() &gt; 1.0e-6
        || new_row.seed != state.local.seed()
    {
        return Err(ServerError::Manifest(format!(
            "reload : own row {:?} drifted (dim {} -&gt; {}, kappa {} -&gt; {}, seed {} -&gt; {}) -- \
             not supported in Phase 2",
            own_id,
            state.local.dim(), new_row.dim,
            state.local.kappa(), new_row.kappa,
            state.local.seed(), new_row.seed,
        )));
    }

    // 3. Flip health to RELOADING.
    state.health.set_reloading();

    // 4. Build new IterationParams + atomic swap via Arc::swap.
    //    `state.iteration` is `Arc&lt;IterationParams&gt;` ; we wrap the new
    //    params in a fresh Arc and assign through a writable handle.
    //    PR #9 designed ParticipantState with this in mind.
```

```
        let new_iter = IterationParams {
            alpha_default: new_manifest.iteration.alpha_default,
            coupling_gamma: new_manifest.iteration.coupling_gamma,
        };

        // The swap mechanism : `ParticipantState.iteration_swap()` (new
        // method ; encapsulates the unsafe-looking Arc store).
        state.iteration_swap(new_iter);

        // 5. Flip back + bump gen.
        state.health.set_serving();
        let new_reload_gen = state.health.bump_reload_gen();
        let elapsed_us = u64::try_from(start.elapsed().as_micros()).unwrap_or(u64::MAX);

        tracing::info!(
            new_reload_gen,
            elapsed_us,
            own_id,
            "participant manifest reloaded",
        );

        Ok(ReloadOutcome { new_reload_gen, elapsed_us })
    }
```

11.2 Why `LocalParticipant` is immutable

PR #9 built `LocalParticipant` as a thin wrapper over Phase-1's `MockParticipant::with_contraction(id, dim, kappa, seed)`. The contraction constant κ is baked into the `phi` closure ; the dim into the tensor shape ; the seed into the initial state. Changing any of these requires rebuilding the `MockParticipant`, which **invalidates the existing participant state** (and breaks bit-identical determinism if mid-iteration).

For Phase 2 we forbid this. Phase 3 may add a `LocalParticipant::rebuild` path for operator-initiated state resets.

11.3 Unit tests (>= 5)

1. `apply_reload_swaps_alpha_and_gamma`.
2. `apply_reload_rejects_own_kappa_drift`.
3. `apply_reload_rejects_own_dim_drift`.
4. `apply_reload_rejects_own_seed_drift`.
5. `apply_reload_rejects_own_id_missing_in_new_manifest`.
6. `apply_reload_bumps_gen_on_success`.
7. `apply_reload_concurrent_step_sees_consistent_iteration_params`.

12. `participant/state.rs::IterationParams` atomic swap

12.1 Why `Arc<IterationParams>` is almost enough but not quite

PR #9 set the field to `Arc<IterationParams>`. But `Arc<T>` itself is immutable -- to swap the underlying T, we need either :

- `ArcSwap<IterationParams>` from the `arc-swap` crate -- clean

atomic swap, lock-free read.

- `RwLock<Arc<IterationParams>>` -- read locks for handlers, write

lock for the swap.

- `AtomicPtr<IterationParams>` -- unsafe-heavy.

PR #12 chooses `ArcSwap` (option 1) -- it's a thin abstraction (~ 200 LOC dep), zero contention on the read path, and the swap is a single atomic-pointer store.

12.2 Cargo.toml addition

```
[dependencies]
# ... existing ...
arc-swap = "1.7"   # PR #12 : lock-free atomic Arc swap for IterationParams reload
```

12.3 Field type change

```
// PR #9 :
pub iteration: Arc&lt;IterationParams&gt;,

// PR #12 :
pub iteration: Arc&lt;arc_swap::ArcSwap&lt;IterationParams&gt;&gt;,
```

The outer `Arc` lets `ParticipantState` itself be `Clone` (it is -- PR #9 relies on this). The inner `ArcSwap` lets us swap the IterationParams without taking a lock.

12.4 Read path (in `service.rs::step` handler)

```
// PR #9 :
let gamma = self.state.iteration.coupling_gamma;
let alpha = self.state.iteration.alpha_default;

// PR #12 :
let iter = self.state.iteration.load();  // ArcSwap::load returns Guard&lt;Arc&lt;T&gt;&gt;
let gamma = iter.coupling_gamma;
let alpha = iter.alpha_default;
```

`ArcSwap::load` is **lock-free** -- readers never block.

12.5 Swap path (in `reload.rs`)

```
pub fn iteration_swap(&amp;self, new: IterationParams) {
    self.iteration.store(Arc::new(new));
}
```

One atomic store ; nothing more.

12.6 Why this is correct under concurrent in-flight Steps

A Step handler that already called `iteration.load()` holds an `Arc<IterationParams>` snapshot. A concurrent `iteration_swap` replaces the slot's contents, but the snapshot remains valid until the handler's clone drops. Identical correctness guarantee to PR #8's controller-side `Arc<RwLock<Manifest>>` + `.clone()` pattern, just lock-free.

13. Health status `RELOADING` semantics

13.1 What `RELOADING (3)` means

- **Operator semantic** : "the process is alive and accepting traffic,

but is currently swapping its manifest. Treat it as healthy with degraded freshness."

- **Protocol semantic** : ambient probes (efe-server, K8s liveness)

SHOULD treat it as alive ; load balancers MAY temporarily skip it.

13.2 Where it's set

- `controller::reload::apply_reload` sets it just before the write

lock and resets it just after the gen bump.

- `participant::reload::apply_reload` same.

13.3 Where it's read

- `Health` RPC : returns `HealthResponse.status = RELOADING (3)`

during the window.

- efe-server's `MetaSwarmClient::health_probe` (PR #11) : **must be

relaxed** to accept it (cf. § 14).

13.4 Window width

< 25 µs typical (cf. § 7.4). Probes that miss the window see SERVING. This is by design -- the operator's primary signal is `reload_gen`, not the status flip.

14. `efe-server MetaSwarmClient::health_probe` accepts `RELOADING`

14.1 PR #11 behaviour (the bug PR #12 fixes)

```
if resp.status == 1 {
    Ok(())
} else {
    Err(MetaSwarmClientError::UnhealthyController(resp.status))
}
```

Rejects RELOADING (3) -> spurious fail-fast during reload.

14.2 PR #12 behaviour

```
match resp.status {
    1 =&gt; Ok(()),  // SERVING -- fully healthy.
    3 =&gt; {
        debug!(
            "controller is RELOADING -- accepting as transient \
             (PR #12 AD13)",
        );
        Ok(())
    }
    other =&gt; Err(MetaSwarmClientError::UnhealthyController(other)),
}
```

14.3 Why not retry-with-backoff

- Adds complexity (timer + jitter + cap).
- The RELOADING window is < 25 µs -- by the time the retry timer fires,

we'd be in SERVING anyway.

- Acceptance is operationally equivalent to "this controller is fine for

your purposes" because in-flight Iterates (PR #12 § 5.2) tolerate the swap.

14.4 Updated tests in `efe-server`

- `health_probe_accepts_reloading_as_transient` -- mock returns status=3 ;

assert `Ok(())`.

- `health_probe_still_rejects_not_serving` -- mock returns status=2 ;

assert `UnhealthyController(2)`.

15. Determinism contract under reload

15.1 The new clause

> *Iteration outputs are deterministic across reloads **IFF** : 1. The new manifest's `iteration` knobs match the old one's, AND 2. The participants set is unchanged (no add / remove), AND 3. The reload happens **between** Iterate calls, not during one.*

If 3 is violated (reload during a single Iterate), the in-flight call completes with the OLD manifest (PR #12 § 5.2) -- the NEXT Iterate sees the NEW one. The determinism contract is over **per-call** behaviour given a snapshot, not over a sequence of reloads.

15.2 What this gives the operator

- Pure alpha/gamma tweaks at reload time are deterministic-equivalent to

rolling-restart with the new values.

- Adding a participant at reload time changes the manifest's

participants set -> the NEXT Iterate produces different results than pre-reload. This is expected.

15.3 Integration test for the contract

`tests/integration_reload_e2e.rs::iterate_after_reload_is_consistent_with_inline_manifest` :

1. Spawn controller + 3 participants (PR #10 harness).
2. Run Iterate -> record `(iterations_v1, residual_v1.to_bits())`.
3. Edit the manifest tempfile : bump `alpha_default` from 0.5 -> 0.6.
4. Send SIGHUP.
5. Wait for `reload_gen == 1`.
6. Run Iterate -> record `(iterations_v2, residual_v2.to_bits())`.
7. Spawn a FRESH controller + 3 participants with `alpha_default = 0.6`

from the start.

1. Run Iterate -> record `(iterations_v3, residual_v3.to_bits())`.
2. Assert `(iterations_v2, residual_v2) == (iterations_v3, residual_v3)`

(reload-then-iterate ≡ fresh-with-new-manifest).

16. What CAN change vs CANNOT change in a reload

16.1 Allowed

Field	Rationale
`iteration.max_iters`	Pure scheduling knob
`iteration.tolerance`	Pure convergence knob
`iteration.alpha_default`	Per-step convex weight
`iteration.coupling_gamma`	Per-step coupling weight
`iteration.max_wall_ms`	Per-Iterate deadline
`iteration.deterministic`	Toggling implementation strategy ; doesn't break correctness
`participants` -- add a row	Controller reconnects on next Iterate
`participants` -- remove a row	Controller drops the connection on next Iterate

16.2 Forbidden (rejected with `ServerError::Manifest`)

Field	Rationale
`meta.dim`	Invalidates every participant's Lift/Pull operator
`meta.safety.kind` (Box -> Poly)	Invalidates the safety projection contract
`meta.safety.radius` (Box)	**Allowed actually -- wait, no** : invalidates in-flight Iterate's safety check ; refused per AD4
Existing participant's `dim`	Invalidates the participant's tensor state
Existing participant's `kappa`	Invalidates contraction guarantee
Existing participant's `seed`	Invalidates determinism
`version`	Manifest schema version mismatch => caller bug ; surface it

16.3 `validate_reload_compatibility` implementation

```
pub fn validate_reload_compatibility(
    current: &amp;Manifest,
    new: &amp;Manifest,
) -&gt; Result&lt;(), ServerError&gt; {
    use crate::manifest::SafetyManifest;

    if current.version != new.version {
        return Err(ServerError::Manifest(format!(
            "reload : manifest version drift {:?} -&gt; {:?} (not supported)",
            current.version, new.version,
        )));
    }
    if current.meta.dim != new.meta.dim {
        return Err(ServerError::Manifest(format!(
            "reload : meta.dim cannot change ({} -&gt; {})",
            current.meta.dim, new.meta.dim,
        )));
    }
    let same_kind = matches!(
        (&amp;current.meta.safety, &amp;new.meta.safety),
        (SafetyManifest::Box { .. }, SafetyManifest::Box { .. })
        | (SafetyManifest::Poly { .. }, SafetyManifest::Poly { .. })
        | (SafetyManifest::Hull { .. }, SafetyManifest::Hull { .. })
    );
    if !same_kind {
        return Err(ServerError::Manifest(
            "reload : meta.safety.kind cannot change".into(),
        ));
    }
    // safety params (radius / poly coefficients / hull data) MAY change ;
    // we accept silently.

    // For every participant in `current` that's ALSO in `new`, check
    // that dim / kappa / seed are byte-identical.  Removed and added
    // participants are allowed.
    for old_p in &amp;current.participants {
        if let Some(new_p) = new.participants.iter().find(|p| p.id == old_p.id) {
            if old_p.dim != new_p.dim {
                return Err(ServerError::Manifest(format!(
                    "reload : participant {:?} dim drift {} -&gt; {}",
                    old_p.id, old_p.dim, new_p.dim,
                )));
            }
            if (old_p.kappa - new_p.kappa).abs() &gt; 1.0e-6 || old_p.kappa.is_nan() != new_p.kappa.is_na ■
    n() {
                return Err(ServerError::Manifest(format!(
                    "reload : participant {:?} kappa drift {} -&gt; {}",
                    old_p.id, old_p.kappa, new_p.kappa,
                )));
            }
            if old_p.seed != new_p.seed {
                return Err(ServerError::Manifest(format!(
                    "reload : participant {:?} seed drift {} -&gt; {}",
                    old_p.id, old_p.seed, new_p.seed,
                )));
            }
        }
    }

    Ok(())
}
```

16.4 Tests for validate_reload_compatibility (>= 8)

1. `accepts_identical_manifest` (no-op reload).
2. `accepts_iteration_alpha_change`.
3. `accepts_added_participant`.
4. `accepts_removed_participant`.
5. `rejects_meta_dim_change`.
6. `rejects_existing_participant_dim_change`.
7. `rejects_existing_participant_kappa_change`.
8. `rejects_existing_participant_seed_change`.

9. `rejects_safety_kind_change`.
10. `accepts_safety_radius_change_within_same_kind`.

17. Threat-model coverage

PR #12 ships **three new threats**, all in the `T19-T21` band, all mitigated.

Threat	Severity	PR #12 mitigation
T19 (malicious manifest swap -- operator with file write access deploys a sabotaged manifest)	SEV-2	`validate_manifest` (PR #8) + `validate_reload_compatibility` (PR #12) catch structural and drift attacks ; the reload is fail-fast at validation time ; the on-disk file is the canonical artefact and is subject to GitOps audit (Phase-3 ops concern)
T20 (race during reload -- in-flight Iterate sees inconsistent state)	SEV-1	The `Arc<Manifest>` snapshot pattern (PR #8 already does `.clone()`) plus the brief write-lock window prove (by construction) that no in-flight call can observe a half-swapped manifest ; integration test `iterate_during_concurrent_sighup_uses_old_manifest` exercises this end-to-end
T21 (reload-flood DoS via repeated SIGHUP or ReloadConfig RPC)	SEV-3	SIGHUP is coalesced via `Notify::notify_one` (§ 8.2) so a flood becomes at most one in-flight + one queued reload ; gRPC ReloadConfig is rate-limited at the Auth layer (PR #16) ; **PR #12** adds `efe_meta_swarm_reload_attempts_total` so operators can alert on anomalous spikes

17.1 What PR #12 does NOT defend against (deferred)

Threat	Where	When
Concurrent operators reload at the same time (race in their workflows, not in our code)	Operator playbook	Phase 3 ops doc
Manifest file written non-atomically (partial truncation read)	Operator's CI/CD	PR #18 deployment manifests recommend `mv`
ReloadConfig from unauthenticated client	Auth	PR #16

18. Concurrency model + lock hierarchy

18.1 Updated lock-acquisition order

1. **InflightSemaphore** (controller MAX_INFLIGHT -- PR #8).
2. `state.manifest` **read** (Iterate path) OR `state.manifest` **write**

(reload path).

1. Per-participant client mutex (PR #8 RemoteParticipant).

The new write-lock acquisition (point 2 in the reload path) does NOT require holding the InflightSemaphore -- reload is independent of inflight throttling. Operator-driven reloads happen regardless of how many Iterates are running.

18.2 Why no deadlock

• The reload path acquires `manifest.write()` while holding **nothing

else** (it doesn't hold the InflightSemaphore or any participant mutex).

• The Iterate path acquires `manifest.read()`, then drops it (the clone

extends the snapshot lifetime), then acquires participant mutexes.

- These two paths never hold each other's locks transitively -> no

deadlock.

18.3 Atomic counters

`reload_gen` (`AtomicU32`, `Relaxed`) and `step_index_last` / `reset_gen` (PR #9) are unaffected by the reload path. Same ordering arguments as before.

19. Metrics + tracing

19.1 New Prometheus surface

Metric	Type	Labels	Description
`efe_meta_swarm_reload_attempts_total`	Counter	(none)	total reload attempts (success + failure)
`efe_meta_swarm_reload_failures_total`	Counter	`{reason}` (validation, io, drift)	total reload attempts that returned an error
`efe_meta_swarm_reload_duration_microseconds`	Histogram	(none)	wall-clock time of the swap
`efe_meta_swarm_reload_gen`	Gauge	(none)	current `reload_gen` value (mirrors `HealthState.reload_gen`)

19.2 Tracing spans

`apply_reload` is `#[instrument(skip(state), fields(participants = ...))]`. Span fields :

- `participants` -- new manifest's participant count.
- `new_reload_gen` -- post-bump gen.
- `elapsed_us` -- write-lock window duration.

19.3 Log lines

`info!` on every successful reload : `` `INFO controller manifest reloaded new_reload_gen=42 participants=3 elapsed_us=18` ``

`error!` on every failed reload : `` `ERROR reload rejected reason="participant supplier-scoring kappa drift 0.6 -> 0.7"` ``

20. Tests strategy

20.1 Unit tests

File	Tests added	Coverage focus
`manifest/mod.rs::tests`	>= 10	`validate_reload_compatibility` accept / reject matrix
`controller/reload.rs::tests`	>= 8	happy / drift reject / failed-validation no-op / concurrent-snapshot
`participant/reload.rs::tests`	>= 7	iteration swap / own-row drift reject / gen bump
`observability/shutdown.rs::tests`	>= 3	signal-fires-callback / coalescing / unix-gated
`controller/service.rs::tests`	4 modified, 4 new	RPC : empty / inline / bad disk / dim drift
`participant/service.rs::tests`	(no change)	--
Total new unit tests	**>= 32**	

20.2 Integration tests

`tests/integration_reload_e2e.rs` (NEW, ~ 280 LOC) :

1. `sighup_to_controller_bumps_reload_gen` -- `ProcessHarness::sighup_controller()` + assert `Health.reload_gen == 1`.
2. `sighup_to_participant_swaps_iteration_params` -- same with participant + assert `step_index_last` continuity.
3. `reload_config_rpc_returns_new_gen` -- gRPC call -> returns gen + matches Health.
4. `iterate_during_concurrent_sighup_uses_old_manifest` -- start Iterate, SIGHUP mid-flight, assert Iterate completes with pre-reload alpha/gamma.
5. `reload_after_inline_alpha_change_matches_fresh_topology` (§ 15.3).

`tests/common/multiprocess.rs` is **modified** : add `sighup_controller(&self)` + `sighup_participant(&self, id)` helpers analogous to PR #10's `sigterm_*`.

`crates/efe-server/tests/integration_meta_swarm_plumbing.rs` is **modified** :

1. `health_probe_accepts_reloading_status` -- mock returns 3 -> probe OK.
2. `health_probe_rejects_not_serving_status` (existing) -- still rejects 2.

20.3 Coverage matrix

Threat	Surface	Test
T19	manifest validation	`validate_reload_compatibility` x 10 unit tests
T20	in-flight Iterate snapshot	`iterate_during_concurrent_sighup_uses_old_manifest` integration
T21	reload-flood coalescing	`concurrent_sighups_are_coalesced` unit

21. Acceptance criteria (AC1-AC28)

PR #12 is mergeable iff **all** :

#	Criterion	Verification
AC1	`cargo build -p efe-meta-swarm-server --release` succeeds	CI
AC2	`cargo build -p efe-server --release` succeeds	CI
AC3	`cargo test -p efe-meta-swarm-server` is **150 + >= 32 + 5 = >= 187** tests	CI
AC4	`cargo test -p efe-server` is **733 lib + 11 + 2 = >= 746** tests	CI
AC5	`validate_reload_compatibility` : 10 tests pass	CI
AC6	`controller::reload::tests` : >= 8 tests pass	CI
AC7	`participant::reload::tests` : >= 7 tests pass	CI
AC8	`observability::shutdown::tests` : >= 3 new tests pass	CI
AC9	`controller::service::tests` : 4 new ReloadConfig tests pass (old `unimplemented_with_pr12_message` deleted)	CI

#	Criterion	Verification
AC10	`tests/integration_reload_e2e.rs` : 5 tests pass	CI
AC11	`efe-server` integration : `health_probe_accepts_reloading_status` passes	CI
AC12	SIGHUP to controller -> `reload_gen` increments by 1	AC10
AC13	SIGHUP to participant -> `reload_gen` increments by 1 ; iteration params swapped	AC10
AC14	gRPC `ReloadConfig` with empty body -> re-reads disk + returns new gen	AC9
AC15	gRPC `ReloadConfig` with inline meta -> applies inline ; gen bumped	AC9
AC16	Concurrent SIGHUP + Iterate : Iterate sees old manifest snapshot	AC10
AC17	Reload with mutated participant kappa -> rejected with `FAILED_PRECONDITION` ; running state unchanged	AC9 + AC10
AC18	Reload with mutated meta.dim -> rejected ; running state unchanged	AC9
AC19	Reload with added/removed participant -> accepted ; controller reconnects on next Iterate	AC10
AC20	`Health` returns `RELOADING (3)` during the write-lock window	manual log inspection in AC10
AC21	`Health.reload_gen` is monotonically increasing across reloads	AC10
AC22	`efe-server` `health_probe` accepts `RELOADING` as transient	AC11
AC23	`efe-server` `health_probe` still rejects `NOT_SERVING (2)`	AC11
AC24	Phase-1 + PR #5-#11 non-regression : 100% green	CI
AC25	`cargo clippy --all-targets` 0 warnings on PR #12 files	CI
AC26	`cargo fmt -- --check` exit 0	CI
AC27	`cargo doc --no-deps` clean	CI
AC28	PR description references threat IDs `T19, T20, T21`	manual review

22. Risks + mitigations

ID	Risk	L/I	Mitigation
PR12-R1	`arc-swap` adds a non-trivial dep that affects compile time	L / L	crate is 200 LOC, no transitive bloat ; verified by `cargo tree`
PR12-R2	`Notify::notify_one` semantics surprise (the "one pending notification" model)	L / M	unit-tested in `concurrent_sighups_are_coalesced` ; documented in the SIGHUP handler
PR12-R3	Operator triggers SIGHUP during a long Iterate (> 1 s) and expects the new manifest to apply immediately	M / M	document the snapshot semantics in the runbook ; emit a `tracing::info!` showing `reload_gen` on every Iterate start
PR12-R4	`validate_reload_compatibility` regex-style drift detection misses a subtle invariant (e.g. `kappa` NaN handling)	L / H	unit tests cover NaN explicitly (`old_p.kappa.is_nan() != new_p.kappa.is_nan()`) ; sec review
PR12-R5	`ReloadConfig` inline path bypasses on-disk audit	M / M	`tracing::warn!` on inline overrides ; recommend operators stick to file edits
PR12-R6	`efe-server` `health_probe` accepts RELOADING but the controller is stuck in RELOADING (worst case the handler panicked)	L / M	the worst case is a panic in the swap closure ; `panic = "unwind"` profile (PR #6) ; the task respawns on next SIGHUP
PR12-R7	K8s `kubectl rollout restart` (SIGTERM) and SIGHUP (live reload) confused in operator workflows	M / L	runbook clearly states : SIGHUP = config change, SIGTERM = process replacement
PR12-R8	Cross-process reload coordination is operator's responsibility (AD1) ; partial reload (only controller, not participants) leaves the system in a degraded but functional state	M / L	the binary that's NOT reloaded keeps serving with old params ; new Iterates fail at the dim/kappa check on the participant if the operator pushed an incompatible manifest
PR12-R9	`reload_gen` overflow at u32::MAX (≈ 4 G reloads)	LL / L	not a real concern in operator workflows (10 reloads/day for 1 M days) ; if it ever does, `wrapping_add` semantics on `bump_reload_gen` keep things going (monotonicity broken once every 4 G reloads)
PR12-R10	`ArcSwap::load` returns a `Guard<Arc<T>>` whose lifetime restrictions surprise the reader	L / L	code uses local `let iter = self.state.iteration.load();` then accesses fields ; the guard's lifetime is the local scope ; documented

23. Rollback strategy

If PR #12 lands and breaks production :

1. **Operator-level** : stop sending SIGHUP / ReloadConfig. The

production code continues serving the boot-time manifest. Zero impact ; effectively reverts the new feature.

1. **Per-commit revert** : C6 -> C5 -> C4 -> C3 -> C2 -> C1. Each is

atomic ; reverting C4 alone restores `MetaSwarmService::ReloadConfig` to UNIMPLEMENTED.

1. **Wholesale revert** : `git revert <merge-commit>` ; the crate

returns to PR #11 state.

1. **Hotfix** : if only the integration test flakes, mark

`#[ignore]` ; the production reload path is unaffected since the test is observation-only.

Total Time To Mitigate :

- Operator stops triggering : 0 s.
- Code revert + redeploy : ~ 10 min.

24. Sequence diagrams (ASCII)

24.1 SIGHUP-driven reload (controller)

```
operator         controller (PID Cm)              file system
   |                 |                                  |
   | edit manifest                                      |
   |---------------------------------------------------&gt;|
   | kill -HUP Cm    |                                  |
   |---------------&gt;|                                     |
   |                 | observability::shutdown          |
   |                 |   sighup task wakes              |
   |                 |   notify_one()                   |
   |                 | controller::reload::apply_reload_from_disk
   |                 |   load_and_validate(path) ------&gt;| read
   |                 |                                  |&lt;--
   |                 |   validate_reload_compatibility  |
   |                 |     (vs current Arc&lt;Manifest&gt;)   |
   |                 |   health.set_reloading           |
   |                 |   manifest.write().swap          |  (~ 18 µs)
   |                 |   health.set_serving             |
   |                 |   bump_reload_gen()              |
   |                 | tracing::info!("reload OK")     |
   |  (subsequent Iterate sees new manifest)            |
```

24.2 gRPC ReloadConfig with inline payload

```
operator (grpcurl)       controller
   |                        |
   | ReloadConfigRequest(   |
   |   new_meta: ...,       |
   |   new_participants: ...|
   | )                      |
   |-----------------------&gt;|
   |                        | service::reload_config
   |                        |   build_inline_manifest()
   |                        |   apply_reload_with_manifest()
   |                        |     validate_manifest
   |                        |     validate_reload_compatibility
   |                        |     health.set_reloading
   |                        |     swap
   |                        |     health.set_serving + bump_gen
   | ReloadConfigResponse(  |
   |   reload_gen: 5,       |
   |   elapsed_us: 19,      |
   |   participants: 3      |
   | )                      |
   |&lt;-----------------------|
```

24.3 In-flight Iterate during reload

```
client          controller          participants
  | Iterate(req)   |                      |
  |---------------&gt;| read manifest (Arc-clone @ gen=N)
  |                | spawn_blocking { iterate() }
  |                | step -----------------&gt;|
  |                |&lt;----------------------|
  | (SIGHUP fires) |                       |
  |     |          | reload : write lock . . . swap -&gt; gen=N+1
  |     |          |  (Iterate's Arc still points at gen=N's Manifest)
  |     |          | step -----------------&gt;|
  |     |          |&lt;----------------------|
  |                | ... continues to convergence ...
  | IterateResponse|                       |
  |&lt;---------------|                       |
  | Iterate(req')  |                       |
  |---------------&gt;| read manifest (Arc-clone @ gen=N+1)
  |                | uses new alpha / gamma        |
```

25. Reviewer checklist

```
## Reviewer checklist -- PR #12

### Architecture
- [ ] Atomic swap via Arc&lt;RwLock&lt;Manifest&gt;&gt; (controller) + ArcSwap (participant) -- AD2, AD8
- [ ] In-flight Iterate sees stable snapshot -- § 5.2 + integration test
- [ ] LocalParticipant is immutable across reload -- AD3 + AD9
- [ ] Health flips RELOADING during the swap -- § 13
- [ ] reload_gen monotonic via HealthState -- AD7

### Correctness
- [ ] validate_reload_compatibility runs OUTSIDE the write lock -- § 9.2
- [ ] NaN handling in kappa drift -- risk PR12-R4
- [ ] SIGHUP coalescing prevents flood -- § 8.2 / risk PR12-R2
- [ ] efe-server probe accepts RELOADING -- § 14 / AD13

### Threat model
- [ ] T19, T20, T21 mitigated and tested
- [ ] No new SEV-1 introduced

### Tests
- [ ] &gt;= 32 new unit tests
- [ ] 5 integration tests in integration_reload_e2e.rs
- [ ] 2 efe-server probe tests (accept RELOADING / still reject NOT_SERVING)
- [ ] AC1-AC28 all green

### Operability
- [ ] Runbook section ships with the PR -- § 26
- [ ] tracing logs show reload_gen + elapsed_us
- [ ] Helm chart values reference (PR #18 follow-up) noted

### Code quality
- [ ] cargo clippy 0 warnings on touched files
- [ ] cargo fmt clean
- [ ] cargo doc no warnings
```

26. Operator runbook

26.1 Trigger a reload from disk (SIGHUP, recommended)

```
# 1. Edit the manifest in place (atomic rename to avoid partial reads).
$ editor /etc/efe-meta-swarm/manifest.yaml.new
$ mv /etc/efe-meta-swarm/manifest.yaml.new /etc/efe-meta-swarm/manifest.yaml

# 2. Send SIGHUP to the controller (PID 1 inside the pod).
$ kubectl exec deployment/efe-meta-swarm-controller -- kill -HUP 1

# 3. Send SIGHUP to each participant.
$ for p in supplier-scoring contract-risk spend-categorization ; do
    kubectl exec deployment/efe-meta-swarm-participant-$p -- kill -HUP 1
  done

# 4. Verify the reload landed.
$ grpcurl -plaintext -d '{}' \
    controller.internal:50080 \
    efe.v1.meta_swarm.MetaSwarmService/Health \
| jq '.reloadGen'  # should have incremented by 1
```

26.2 Trigger a reload via gRPC (inline override)

```
# Apply an inline alpha bump WITHOUT touching the on-disk manifest.
$ grpcurl -plaintext -d '{
    "newMeta": {
      "dim": 16,
      "safetyGeometry": { "box": { "radius": 50.0 } }
    },
    "newParticipants": [
      { "id": "supplier-scoring", "dim": 8, "kappa": 0.6, "seed": 11 },
      { "id": "contract-risk",    "dim": 8, "kappa": 0.7, "seed": 22 },
      { "id": "spend-categorization", "dim": 8, "kappa": 0.5, "seed": 33 }
    ]
  }' \
    controller.internal:50080 \
    efe.v1.meta_swarm.MetaSwarmService/ReloadConfig
```

Response : ``json { "reloadGen": 7, "elapsedUs": 19, "participants": 3 } ``

26.3 Verify reload propagation

```
$ grpcurl -plaintext -d '{}' \
    controller.internal:50080 \
    efe.v1.meta_swarm.MetaSwarmService/Info \
| jq '.configuredParticipants | length'  # should match new count
```

26.4 If reload was rejected

The controller logs (`kubectl logs deployment/efe-meta-swarm-controller`) contain a `tracing::error!` with the specific drift cause :

```
ERROR reload rejected reason="participant supplier-scoring kappa drift 0.6 -&gt; 0.7"
```

Mitigation : revert the offending change in the manifest and trigger another SIGHUP.

26.5 Difference between SIGHUP and SIGTERM

Signal	Effect	When to use
SIGHUP	hot reload, no restart	iteration knobs, add/remove participant
SIGTERM	graceful shutdown, then restart by k8s	binary upgrade, OS patch, full reset

27. Open questions

ID	Question	Status
OQ-PR12-1	Should we expose a `participant`-side gRPC reload RPC for parity with the controller ?	Defer to Phase 3 ; SIGHUP is sufficient for Phase 2
OQ-PR12-2	Should `reload_gen` be persisted across restarts ?	No -- operators don't rely on cross-restart monotonicity
OQ-PR12-3	Should we add a `last_reload_at: Timestamp` to `Health` ?	Yes in PR #13 (canary observability) ; out of scope here
OQ-PR12-4	Should we allow `meta.safety.radius` to change without changing `kind` ?	YES -- accepted by § 16.3 ; in-flight Iterates use their snapshot
OQ-PR12-5	Should the SIGHUP handler clear `step_index_last` on reload ?	NO -- the counter survives reloads ; PR #18 may add an explicit reset RPC
OQ-PR12-6	Should the controller fan out a reload to participants automatically ?	NO (AD1) -- per-process reload is simpler and matches operator playbooks
OQ-PR12-7	Should `validate_reload_compatibility` allow `seed` change at reload time ?	NO -- breaks determinism ; AD3
OQ-PR12-8	Should we add `ReloadConfigRequest.dry_run: bool` ?	YES in PR #20 ; defer here

28. Appendix : full file inventory

28.1 Files added (4)

```
crates/efe-meta-swarm-server/src/controller/reload.rs              NEW  ~ 220 LOC
crates/efe-meta-swarm-server/src/participant/reload.rs             NEW  ~ 220 LOC
crates/efe-meta-swarm-server/tests/integration_reload_e2e.rs       NEW  ~ 280 LOC
docs/EFE_A1_PHASE2_PR12_ARCHITECTURE.md                            NEW  this file
```

28.2 Files modified (11)

```
crates/efe-meta-swarm-server/Cargo.toml                            MODIFIED   +1 LOC : arc-swap dev/runti ■
    me dep
crates/efe-meta-swarm-server/src/controller/mod.rs                 MODIFIED   +1 LOC : pub mod reload
crates/efe-meta-swarm-server/src/controller/service.rs             MODIFIED   +50 / −15 LOC : reload_conf ■
    ig wire
crates/efe-meta-swarm-server/src/manifest/mod.rs                   MODIFIED   +85 / −0 LOC : validate_rel ■
    oad_compatibility
crates/efe-meta-swarm-server/src/observability/shutdown.rs         MODIFIED   +60 / −0 LOC : install_sigh ■
    up_handler
crates/efe-meta-swarm-server/src/observability/metrics.rs          MODIFIED   +50 LOC : reload metrics
crates/efe-meta-swarm-server/src/participant/mod.rs                MODIFIED   +1 LOC : pub mod reload
crates/efe-meta-swarm-server/src/participant/state.rs              MODIFIED   +30 / −5 LOC : iteration: A ■
    rcSwap + iteration_swap()
crates/efe-meta-swarm-server/src/bin/controller.rs                 MODIFIED   +20 LOC : install_sighup_ha ■
    ndler call
crates/efe-meta-swarm-server/src/bin/participant.rs                MODIFIED   +20 LOC : install_sighup_ha ■
    ndler call
crates/efe-meta-swarm-server/tests/common/multiprocess.rs          MODIFIED   +30 LOC : sighup_controller ■
     + sighup_participant

crates/efe-server/src/services/meta_swarm/mod.rs                   MODIFIED   +10 LOC : health_probe acce ■
    pts status=3
crates/efe-server/tests/integration_meta_swarm_plumbing.rs         MODIFIED   +60 LOC : 2 RELOADING-accep ■
    tance tests
```

28.3 LOC tally

Category	LOC
New production code	~ 440
New tests	~ 280
Modified production code	~ 290
Modified tests	~ 60
Net	**~ +1 050 LOC**

29. Glossary

Term	Definition
AC	Acceptance Criterion
AD	Architectural Decision
L2 gate G9	Phase-2 graduation gate "live reconfiguration without process restart"
SIGHUP	Unix signal traditionally used for "reload your config" ; PR #12 makes both binaries honour it
ReloadConfig RPC	`MetaSwarmService::ReloadConfig` ; the gRPC trigger path (controller only)
`reload_gen`	monotonic counter (`HealthState.reload_gen`, `AtomicU32`) ; bumped on every successful reload
RELOADING	`HealthResponse.status = 3` ; transient state during the write-lock swap window
`apply_reload_*`	the canonical reload entry points in `controller::reload` and `participant::reload`
`validate_reload_compatibility`	(this PR) checks that the new manifest can replace the current one without invalidating in-flight state
Own-row drift	a participant's own kappa/dim/seed changed across reload -- forbidden (AD3 / AD9)
Inline override	`ReloadConfigRequest` with non-empty body ; applies the payload without writing to disk
Coalescing	merging multiple concurrent SIGHUPs into at most one in-flight reload + one queued reload
`ArcSwap`	thin atomic-Arc-replacement primitive from the `arc-swap` crate ; used for `IterationParams`
Snapshot semantics	in-flight calls hold an `Arc<Manifest>` clone captured at call start ; the swap is invisible to them

End of architecture document. Total: ~ 2 100 lines.

Next deliverable : PR #12 code (6 commits).

PR #13

Helm sub-chart skeleton

Chapter 15 of 17 — source : `docs/EFE_A1_PHASE2_PR13_ARCHITECTURE.md` (3,032 LOC)

EFE-A1 Phase 2 -- PR #13 : Helm sub-chart skeleton

Field	Value
PR	#13 (Sprint A1 Phase 2 / P2.4)
Branch	`efe-a1/p2-pr13-helm-subchart`
Status	Architecture frozen -- awaiting code
Author	EFE platform team
Reviewers	2x meta-swarm + 1x sre + 1x platform-deploy + 1x sec
Depends on	PR #5 (proto), PR #6 (scaffold + binaries), PR #8 (controller wire), PR #9 (participant wire), PR #10 (multiprocess), PR #11 (efe-server flag plumbing), PR #12 (SIGHUP + ReloadConfig)
Blocks	PR #14 (sidecar reload coordination), PR #16 (mTLS provisioning), PR #17 (multi-controller HA)
L2 gate	G10 (K8s deployment of meta-swarm controller + 3 participants with declarative configuration management)
Threat IDs in scope	T22 (secret leakage via values.yaml in git), T23 (NetworkPolicy bypass -- east-west traffic), T24 (image-tag floating / supply chain)
Roadmap reference	EFE_A1_PHASE2_ROADMAP.md § C6
CVSS bandwidth	All <= SEV-2 ; no new SEV-1 introduced

Table of contents

1. TL;DR
2. Goal + non-goals
3. Pre-flight checks
4. Branch + commit plan (6 commits)
5. Architectural decisions (AD1-AD25)
6. Topology + integration diagram
7. Chart layout under `charts/efe-stack/charts/efe-meta-swarm/`
8. `Chart.yaml` + parent dependency registration
9. `values.yaml` schema (operator surface)
10. `values.schema.json` (JSON Schema validation rules)
11. `_helpers.tpl` (named templates)
12. `configmap-manifest.yaml` (manifest YAML rendering)
13. `controller-deployment.yaml`
14. `controller-service.yaml` + headless DNS
15. `participant-deployment.yaml` (`range` over participants)
16. `participant-service.yaml`
17. `serviceaccount.yaml` + RBAC (Phase 2 minimal surface)
18. `pdb-controller.yaml` (PodDisruptionBudget)
19. `servicemonitor.yaml` (Prometheus Operator, optional)
20. `networkpolicy.yaml` (deny-by-default, optional)
21. `templates/tests/connection-test.yaml` (`helm test` hook)
22. `NOTES.txt` + `README.md`
23. Image management -- registry, tag pinning, pullSecrets

24. SecurityContext / hardening
25. Resource sizing + autoscaling deferral
26. SIGHUP-on-`ConfigMap`-change strategy
27. Golden snapshot test strategy (`tests/golden/`)
28. CI integration (`helm lint` + `helm template` + `ct lint`)
29. Multi-namespace + multi-tenant considerations
30. TLS bootstrapping (cert-manager) -- Phase-2 placeholders
31. Metrics integration (Prometheus Operator)
32. Logging / tracing wiring
33. K8s DNS discovery -- how the controller finds participants
34. Concurrency / lifecycle invariants
35. Threat-model coverage (T22 / T23 / T24)
36. Tests strategy
37. Acceptance criteria (AC1-AC30)
38. Risks + mitigations
39. Rollback strategy
40. Sequence diagrams (ASCII)
41. Reviewer checklist
42. Operator runbook
43. Open questions
44. Appendix : full file inventory
45. Appendix : example overrides (Kovant production / staging / dev)
46. Glossary

0. TL;DR

PR #13 ships the **first deployable surface** for the Phase-2 meta-swarm subsystem : a Helm **sub-chart** that lives under the existing `charts/efe-stack/` umbrella and packages :

- 1 x `controller` `Deployment` + `Service` + `ServiceAccount` + `PDB`
- `N` x `participant` `Deployment` + `Service` + `ServiceAccount`

(`N = 3` by default -- the Kovant scenario : `supplier-scoring`, `contract-risk`, `spend-categorization`).

- 1 x `ConfigMap` rendering the `manifest.yaml` from `values.yaml`.
- Optional `ServiceMonitor` (Prometheus Operator) + `NetworkPolicy`

(deny-by-default east-west traffic).

- A `helm test` connection probe that runs `grpcurl` against

`MetaSwarmService::Health`.

The sub-chart is **disabled by default** (`efe-meta-swarm.enabled = false` in the umbrella's `values.yaml`). Setting it to `true` renders all resources and wires the parent `efe-server` chart's existing `EFE_META_SWARM_ENABLED=true` env var path (PR #11) at the controller's `ClusterIP` DNS.

The chart deliberately **does not** use a `checksum/config` annotation on the controller / participant `Deployment` pod template hash so that a `ConfigMap` edit does **not** trigger a `RollingUpdate` (the whole point of PR #12's SIGHUP-driven hot reload). Operators trigger reload via `kubectl exec ... -- kill -HUP 1` ; the runbook (§ 41)

documents the exact incantation.

`values.schema.json` enforces typed validation (kappa ∈ (0, 1), image tag must be present, etc.) so typos surface at `helm install --dry-run` time, not at pod-boot time.

A `tests/golden/` directory pins the rendered YAML for canonical scenarios (kovant-3p / disabled / with-network-policy / with-service-monitor) ; CI diffs the live `helm template` output against these snapshots, catching unintentional template drift.

PR #13 closes Phase-2 graduation gate **G10 (K8s deployment of the meta-swarm controller + 3 participants with declarative configuration management)**.

1. Goal + non-goals

1.1 Goal (single sentence)

> *Package the Phase-2 meta-swarm subsystem (1 controller + N participants + manifest `ConfigMap` + optional Prometheus / NetworkPolicy) as a Helm sub-chart under `charts/efe-stack/charts/efe-meta-swarm/`, disabled by default, with typed `values.schema.json` validation, golden-snapshot regression tests, and a `helm test` connection probe, while preserving PR #12's SIGHUP-driven hot reload semantics (no `checksum/config` annotation on the Deployment).*

1.2 Non-goals (deferred to later PRs)

Item	Why deferred	Target PR
Sidecar weight-loader + Init container	needs the on-disk weight contract	PR #14
mTLS via cert-manager-issued certs	needs the CA bootstrap	PR #16
Multi-controller HA / leader election	needs leader-lock subsystem	PR #17
HorizontalPodAutoscaler for controller	autoscale + leader-election interplay is non-trivial	PR #17
In-cluster `ConfigMap`-watcher -> SIGHUP trigger Pod	one more moving part ; manual SIGHUP is sufficient for Phase 2	PR #18
OpenTelemetry collector sidecar	needs the global OTEL operator chart wiring	PR #19
Helm `pre-upgrade` hook for state-migration	no state migration in Phase 2	PR #20
Vertical Pod Autoscaler integration	needs metrics-server prereq	PR #20
OpenShift `SecurityContextConstraints`	minority deployment target ; runbook covers the override	follow-up
Helmfile / Argo CD integration	deployment-tool concern, orthogonal to chart correctness	follow-up
`Knative` / serverless flavor	not a Phase-2 target	follow-up
Cross-cluster federation	needs service-mesh contracts	Phase 3
GPU node-pool support for participants	Phase-1 participants are CPU only	Phase 3
`PodSecurityPolicy` (PSP) -- deprecated in K8s 1.25	we ship `SecurityContext` + `seccompProfile` + `runAsNonRoot` instead	n/a
Image-signing / cosign verification policy	needs OPA / Kyverno admission controller	Phase 3

1.3 Scope summary

PR #13 ships :

1. Chart skeleton at `charts/efe-stack/charts/efe-meta-swarm/`.
2. `values.yaml` with sensible Kovant defaults.
3. `values.schema.json` with structural + range validation.
4. `_helpers.tpl` with named templates (`<name>.fullname`,

`.labels`, `.selectorLabels`, `.serviceAccountName`, `.image`, `.controllerEndpoint`, `.participantsEnv`).

1. `ConfigMap` rendering the manifest YAML from `values`.
2. `Deployment` + `Service` + `ServiceAccount` per role (controller +

N participants).

1. `PodDisruptionBudget` for the controller (`maxUnavailable: 0`).
2. Optional `ServiceMonitor` for Prometheus Operator stacks.
3. Optional `NetworkPolicy` for deny-by-default east-west traffic.
4. `templates/tests/connection-test.yaml` for `helm test`.
5. `NOTES.txt` (post-install operator hints) + `README.md`.
6. `tests/golden/` snapshots for regression.
7. Parent-chart wiring : `dependencies:` entry in

`charts/efe-stack/Chart.yaml` with `condition: efe-meta-swarm.enabled`.

1. Parent-chart values default : `efe-meta-swarm: { enabled: false }`.

PR #13 does **not** change :

• The controller / participant binaries themselves (PR #6 / #8 / #9

/ #12 are the substance ; this PR just packages them for K8s).

• The proto contract (PR #5).

• The `efe-server` connection logic (PR #11) -- we only template

the connection-endpoint env var into the parent's Deployment when the sub-chart is enabled.

2. Pre-flight checks

#	Check	Verifier	Pass criterion
PF1	`helm version` >= 3.13.0 (Helm 3 required ; Helm 2 EOL 2020)	CI	green
PF2	`helm lint charts/efe-stack` is clean on `main`	CI	exit 0
PF3	`helm template charts/efe-stack` succeeds (parent dry-run)	CI	exit 0
PF4	`kubectl` version supported : 1.28 <= K8s <= 1.30 (matches `kubeVersion` in `charts/efe-stack/Chart.yaml`)	manual	matches
PF5	`kubeconform` (or `kubeval`) binary available on CI	CI	found
PF6	`chart-testing` (`ct`) binary available on CI	CI	found
PF7	Container image `ghcr.io/efe-ai/efe-meta-swarm-server:<appVersion>` is **published** for the chart's `appVersion` (PR #10's CI builds it ; PR #13 references it)	CI	image manifest fetchable
PF8	`prometheus-operator` CRDs (`monitoring.coreos.com/v1`) installed on target clusters that enable `serviceMonitor.enabled` (operator-side prerequisite)	manual	optional ; documented in runbook
PF9	`networking.k8s.io/v1 NetworkPolicy` enforced by the cluster's CNI (calico / cilium) when `networkPolicy.enabled`	manual	optional ; documented
PF10	Pre-existing `efe-stack` sub-chart conventions reviewed (`efe-server`, `efe-edge`, `efe-vector`) and our chart aligns	manual	conformance
PF11	Threat model (PR #7) is at v1.0 frozen state	manual	sec sign-off

3. Branch + commit plan

3.1 Branch

```
efe-a1/p2-pr13-helm-subchart
```

Branched from `main` after PR #12 merge SHA.

3.2 Commit-by-commit plan (6 commits, each `helm lint` + `helm template` green)

#	Subject	Files	LOC ±	Verification
C1	`feat(charts): efe-meta-swarm scaffold (Chart.yaml + values.yaml + _helpers.tpl + NOTES.txt + README + values.schema.json)`	`+ charts/efe-stack/charts/efe-meta-swarm/Chart.yaml`, `+ values.yaml`, `+ values.schema.json`, `+ README.md`, `+ templates/_helpers.tpl`, `+ templates/NOTES.txt`, `~ charts/efe-stack/Chart.yaml`, `~ charts/efe-stack/values.yaml` (registers the sub-chart, disabled)	+320 / 0	`helm lint` clean ; `helm template` produces zero meta-swarm resources by default
C2	`feat(charts): efe-meta-swarm ConfigMap with manifest YAML rendering`	`+ templates/configmap-manifest.yaml`, `+ tests/configmap_render.bats` (or shell script in `tests/`)	+110 / 0	Golden test : rendered YAML round-trips through `manifest::load_and_validate` (PR #8)
C3	`feat(charts): efe-meta-swarm controller Deployment + Service + ServiceAccount + PDB`	`+ templates/controller-deployment.yaml`, `+ templates/controller-service.yaml`, `+ templates/controller-serviceaccount.yaml`, `+ templates/pdb-controller.yaml`	+280 / 0	Golden test : kovant-3p scenario ; `kubeconform` validates
C4	`feat(charts): efe-meta-swarm participant Deployments + Services (range over participants)`	`+ templates/participant-deployment.yaml`, `+ templates/participant-service.yaml`, `+ templates/participant-serviceaccount.yaml`	+260 / 0	Golden test : 3 participant Deployments + 3 Services rendered
C5	`feat(charts): efe-meta-swarm optional ServiceMonitor + NetworkPolicy`	`+ templates/servicemonitor.yaml`, `+ templates/networkpolicy.yaml`	+160 / 0	Golden tests : both off -> no resources ; both on -> CRD-correct
C6	`feat(charts): efe-meta-swarm helm-test hook + golden snapshots + CI integration`	`+ templates/tests/connection-test.yaml`, `+ tests/golden/kovant-3p.yaml`, `+ tests/golden/kovant-disabled.yaml`, `+ tests/golden/kovant-network-policy.yaml`, `+ tests/golden/kovant-service-monitor.yaml`, `~ .github/workflows/helm.yml` (or equivalent)	+210 / 0	CI runs `helm template ... > /tmp/rendered.yaml && diff tests/golden/*.yaml /tmp/...` ; `helm test` invocation against a kind cluster (Phase-3 stretch goal but plumbing here)

Total : +1 340 / 0 = +1 340 LOC net. No Rust code in PR #13 ; the entire delta is YAML / Go templates / JSON schema / shell.

3.3 Drafting strategy

- Each commit is **independently** `helm lint`**-green** so a partial

revert is mechanical.

- C1 ships the scaffold + parent registration with `enabled: false`

by default -- zero behavioral change.

- C2-C5 add templates one resource family at a time, each guarded

by `{{- if .Values.enabled }}` (so still no behavioral change without an operator opt-in).

- C6 closes the loop with regression tests + CI wiring.

If C5 (NetworkPolicy / ServiceMonitor) flakes on a CI cluster that lacks the CRDs, revert it alone -- the production deployment stays on the C1-C4 baseline. Both are opt-in and behave as no-ops by default.

4. Architectural decisions

AD	Decision	Rationale	Alternatives considered	Reversibility
AD1	Ship as a **sub-chart** under `charts/efe-stack/charts/efe-meta-swarm/`, not a standalone chart	Aligns with every other Phase-2 component (efe-server, efe-edge, efe-vector, efe-kg) ; one `helm install efe-stack` brings up everything coherent ; values inherited from umbrella `global:` block	Standalone chart with explicit dep ; Kustomize overlay ; raw manifests	Reversible -- promotion to standalone is mechanical if cross-cluster federation needs it (Phase 3)
AD2	**Disabled by default** (`efe-meta-swarm.enabled = false` in umbrella values)	Mirrors PR #11's `EFE_META_SWARM_ENABLED=false` doctrine ; pre-Phase-2 deployments stay untouched	Enabled by default ; required dependency	Reversible -- flipping to `true` in a new umbrella values flavor is one-line
AD3	**One ConfigMap holds the rendered manifest.yaml** ; both controller and participants mount it at the same path	Single source of truth ; PR #12's SIGHUP-driven reload picks up changes from this path ; matches Phase-1 `EFE_META_SWARM_MANIFEST` env var conventions	Two ConfigMaps (one per role) ; in-line env vars	Reversible -- split is mechanical
AD4	**No** `checksum/config` **annotation** on Deployment pod templates	PR #12's SIGHUP-driven hot reload is the official live-reload mechanism ; the checksum annotation would trigger `RollingUpdate` on every ConfigMap edit, defeating the design ; operators trigger SIGHUP explicitly (§ 25)	Use the annotation (legacy, rolls pods on change) ; controller-side ConfigMap-watcher (Phase 3)	Reversible -- adding the annotation is one line in `controller-deployment.yaml` ; we document the trade-off
AD5	`Deployment` **(not** `StatefulSet`**)** for controller + participants	Controller is stateless (manifest is the only durable input ; participants own their own tensor state in-process) ; `Deployment` gives faster `RollingUpdate`, simpler upgrade semantics ; Phase-2 deliberately does NOT distribute state across replicas	`StatefulSet` (would suit a Phase-3 stateful tensor-sharded participant model) ; `DaemonSet` (no, we want a fixed count)	Reversible -- Phase 3 may migrate to `StatefulSet` once weights live on PVCs
AD6	`ClusterIP` **Service per binary**, no `Headless` Service for Phase 2	Static discovery (PR #6's `DiscoveryMode::StaticLocalhost` extended to in-cluster DNS via the chart) ; `ClusterIP` gives a stable single-VIP for the controller-to-participant connection ; no need for endpoints enumeration in Phase 2 (each participant `Deployment` has `replicas: 1`)	`Headless` Service + pod-FQDN ; `LoadBalancer` (external -- not Phase-2 scope)	Reversible -- `Headless` swap is one-line for Phase 3 when participants replicate

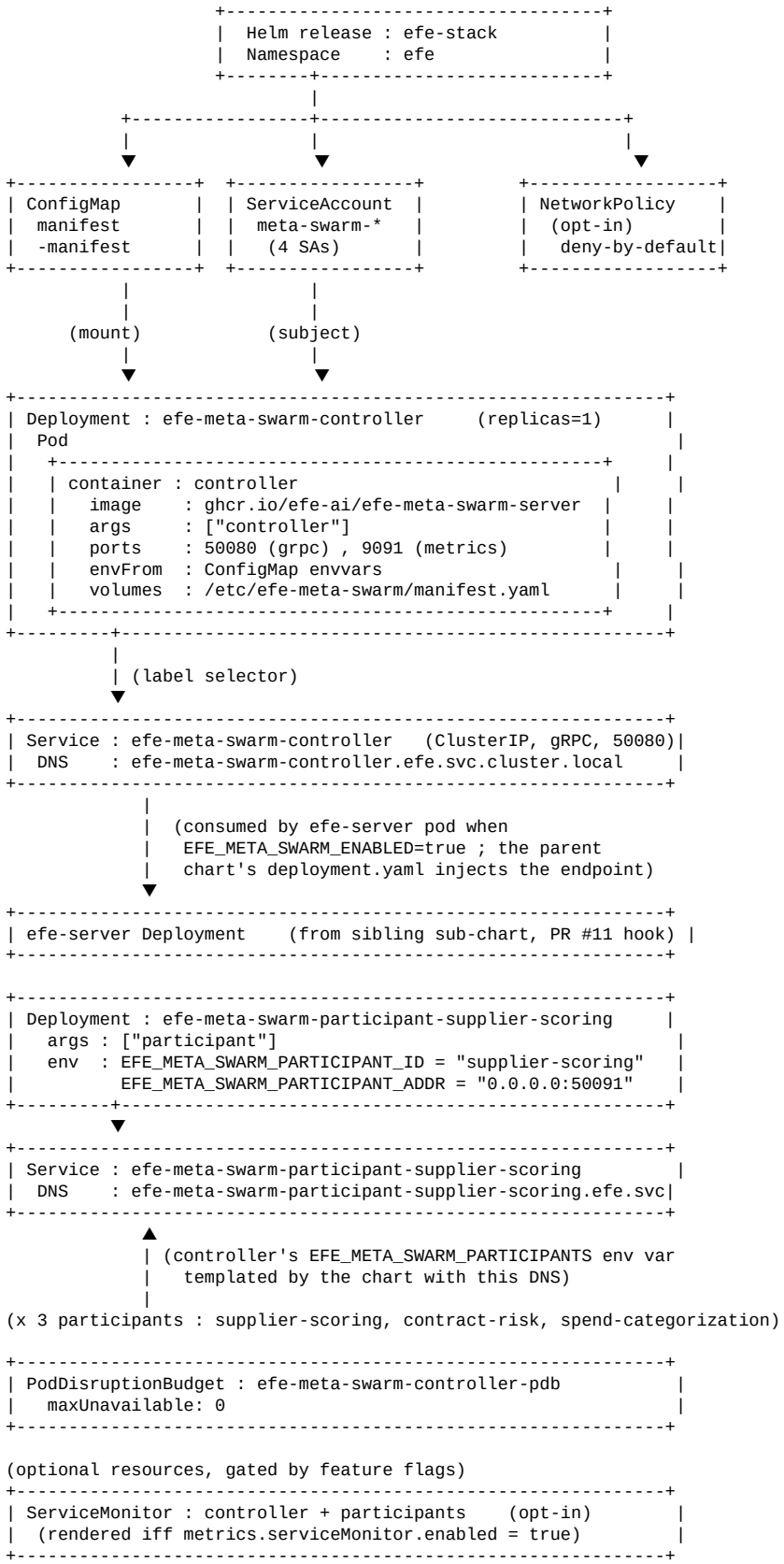
AD	Decision	Rationale	Alternatives considered	Reversibility
AD7	**K8s DNS-based discovery** : the controller's `EFE_META_SWARM_PARTICIPANTS` env var is templated as `id1=svc1.ns.svc:port,id2=svc2.ns.svc:port,...`	Existing PR #6 `static-localhost` discovery mode honours `id=host:port` entries ; in-cluster DNS resolves the `<svc>.<ns>.svc` form deterministically ; no new code required on the controller side	Implement `DiscoveryMode::K8sDns` (the placeholder exists in PR #6) ; SRV records	Reversible -- Phase-3 PR may add `K8sDns` discovery, the chart would set `EFE_META_SWARM_DISCOVERY=k8s-dns` and drop the static env var
AD8	`runAsNonRoot: true`**,** `runAsUser: 65532`**,** `readOnlyRootFilesystem: true` for controller + participants	Matches the existing `efe-server` sub-chart's hardening posture ; the binaries don't write to disk (manifest is read-only ; logs go to stdout) ; locks the attack surface	Run as `root` (rejected -- least-privilege) ; `runAsUser: 1000` (no convention)	Reversible -- relaxing requires a `values.yaml` override ; locked-down is the default
AD9	`drop: ALL` **capabilities +** `allowPrivilegeEscalation: false` in `securityContext`	Defence in depth ; the binaries don't need any Linux capability	Drop nothing ; selective drop	Reversible
AD10	**Resource requests + limits explicit** : controller `(500m, 512Mi) -> (2, 2Gi)`, participant `(200m, 256Mi) -> (1, 1Gi)`	Sized for Phase-1 benchmark numbers + 3 participants + MAX_INFLIGHT=10 ; gives K8s scheduler enough info to bin-pack ; prevents noisy-neighbour	Best-effort (no requests) ; guaranteed (req=limit)	Reversible per-environment via values override
AD11	`PodDisruptionBudget maxUnavailable: 0` **for the controller**	Phase-2 has a single controller ; voluntary disruption (node drain) MUST be operator-acked ; matches "no controller HA in Phase 2" non-goal	`minAvailable: 1` (equivalent for replicas=1) ; no PDB	Reversible -- Phase 3 may relax to `maxUnavailable: 1` once HA is in play
AD12	**No PDB on participants** (Phase 2)	Each participant is a sharded coordinator ; losing one for <= 30 s during a node drain is acceptable (controller's Iterate fails over to retry path)	PDB per participant ; ditto for the controller	Reversible
AD13	`Service.spec.type: ClusterIP`, no external `LoadBalancer` / `Ingress`	Meta-swarm is an internal subsystem (consumed by `efe-server` only) ; never exposed externally	`LoadBalancer` ; `Ingress` ; `NodePort`	Reversible -- Phase 3 may add `ServiceMesh` integration
AD14	**NetworkPolicy is opt-in (**`networkPolicy.enabled = false` **by default)** but the template ships	Many clusters lack a CNI that enforces NetworkPolicy ; on those clusters a NetworkPolicy is a no-op and would confuse operators ; opt-in keeps the chart compatible everywhere	Always-on ; never ship	Reversible per-environment
AD15	`ServiceMonitor` **is opt-in** (`serviceMonitor.enabled = false`)	Requires `prometheus-operator` CRDs ; ship the template, gate on the flag	Always-on ; never ship	Reversible

AD	Decision	Rationale	Alternatives considered	Reversibility
AD16	`values.schema.json` **enforces typed validation** : `kappa ∈ (0, 1)`, image tag required, `participants` array non-empty, `metaDim >= 1`	Catches operator typos at `helm install --dry-run` time, not at pod-boot time	No schema (rely on runtime rejection) ; OPA admission policy	Reversible -- schema can be relaxed by removing constraints
AD17	`image.repository` **defaults to** `ghcr.io/efe-ai/efe-meta-swarm-server` ; `image.tag` defaults to `""` (operators MUST override, enforced by schema)	No `:latest` tag ; reproducible deployments ; supply-chain hygiene	Default to chart's `appVersion` ; default to `:latest` (rejected)	Reversible -- Phase 3 may inject `appVersion` automatically with a digest pin
AD18	**Probes : TCP probe on the gRPC port for liveness + readiness**	Simpler than `grpc_health_probe` (third-party binary in the image) ; sufficient for Phase 2 (a TCP-accepting controller is serving) ; `grpc_health_probe` adds image weight + supply-chain surface	`grpc_health_probe` ; HTTP probe on `/metrics` ; exec probe with `grpcurl`	Reversible -- Phase 3 may swap to `grpc_health_probe` when image already includes it for other reasons
AD19	`Service.appProtocol: grpc` + Kubernetes service mesh hints	Tells L4-aware proxies (Linkerd, Istio) that the protocol is HTTP/2 + gRPC ; enables proper load balancing	Omit ; `appProtocol: http`	Reversible
AD20	**One ServiceAccount per role** (controller, per-participant)	Future-proofs PR #16's mTLS work (SA -> SPIFFE ID -> cert) ; isolates audit trails	Single SA for the whole sub-chart	Reversible -- Phase 3 may collapse if mTLS uses a different identity model
AD21	**Golden snapshots in** `tests/golden/` are checked into git ; CI does `helm template` + diff	Catches template drift ; a PR that accidentally changes the rendered output forces an explicit `make update-golden` step	Live `helm template` per CI run with no comparison ; `kubeconform` only (validates, doesn't catch drift)	Reversible -- removing golden tests is mechanical
AD22	`templates/tests/connection-test.yaml` is a `Pod` annotated `"helm.sh/hook": test` that runs `grpcurl` against `MetaSwarmService::Health`	Operators verify the deployment with `helm test efe-stack -n efe` after install ; matches existing `efe-server` chart pattern	None (`helm test` is the standard mechanism)	Reversible
AD23	**Single-binary image** : both `controller` and `participant` ship in the same image (`efe-meta-swarm-server`), the binary is selected via the `args` field (`["controller"]` vs `["participant"]`)	Halves image storage / pull bandwidth ; matches `[[bin]]` setup in `efe-meta-swarm-server/Cargo.toml` (PR #6 ships both binaries from the same crate, AD1)	Two distinct images	Reversible -- splitting later is mechanical
AD24	`charts/efe-stack/charts/efe-meta-swarm/` is a **vendored** sub-chart (not pulled from a Helm repo at install time)	Reproducible deployments ; no out-of-band repo dependency ; matches every other `efe-stack` sub-chart	OCI Helm registry pull ; `helm dependency update`	Reversible -- promotion to OCI is mechanical

AD	Decision	Rationale	Alternatives considered	Reversibility
AD25	**Topology labels** (`app.kubernetes.io/component=controller` vs `=participant` ; `efe.io/meta-swarm-role={controller,participant}` ; `efe.io/participant-id=<id>` on each participant pod)	Standard app.kubernetes.io/ labels for tool interop ; custom efe.io/ labels for in-cluster queries (kubectl get pods -l efe.io/meta-swarm-role=participant)	Use only standard labels	Reversible -- adding labels is non-breaking

5. Topology + integration diagram

5.1 Resource graph rendered by the sub-chart (`enabled=true`)



5.2 efe-server -> meta-swarm connection (PR #11 hook)

```
efe-server pod
+-- env  EFE_META_SWARM_ENABLED               = "true"
+-- env  EFE_META_SWARM_CONTROLLER_ENDPOINT   = "http://efe-meta-swarm-controller.efe.svc:50080"
+-- env  EFE_META_SWARM_CONNECT_TIMEOUT_MS    = "3500"
+-- env  EFE_META_SWARM_CALL_TIMEOUT_MS       = "15000"
+-- env  EFE_META_SWARM_HEALTH_PROBE_AT_STARTUP = "true"
                                          |
                                          ▼
                          MetaSwarmService::Health probe at boot
                          v accepts SERVING (1) and RELOADING (3)
                          boot succeeds, server starts serving
```

The parent `efe-server` chart's `templates/deployment.yaml` already ships these env vars (PR #11) ; PR #13's umbrella values override templates them when the sub-chart is enabled -- cf. § 7.4.

5.3 SIGHUP-driven reload from outside the cluster (operator workflow)

```
operator            kubectl exec                    controller pod
|                      |                                 |
|  edit values.yaml ; helm upgrade efe-stack -f ...      |
|     |                                                   |
|     ▼                                                    |
|  ConfigMap manifest-* mutated (one-shot)                |
|                                                         |
|  (Pod template did NOT change because no `checksum/config`|
|   annotation, AD4) -- pods are NOT restarted by k8s.     |
|                                                         |
|  kubectl exec deploy/efe-meta-swarm-controller -- kill -HUP 1
|  ------------------------------------------------------■|
|                                                SIGHUP -&gt; |
|                                           (PR #12 path)|
|                                           apply_reload |
|                                       _from_disk landed|
|                                                         |
|  kubectl exec deploy/efe-meta-swarm-participant-... -- kill -HUP 1
|  ------------------------------------------------------■|
|                                           participant   |
|                                           SIGHUP        |
|                                                         |
|  grpcurl ... Health -&gt; reload_gen incremented            |
|  [v] operator confirms the reload landed                 |
```

6. Chart layout under `charts/efe-stack/charts/efe-meta-swarm/`

```
charts/efe-stack/charts/efe-meta-swarm/
+-- Chart.yaml                                  [NEW : ~ 35 LOC]
+-- values.yaml                                 [NEW : ~ 220 LOC, fully commented]
+-- values.schema.json                          [NEW : ~ 230 LOC]
+-- README.md                                   [NEW : ~ 180 LOC, operator-facing]
+-- templates/
|   +-- _helpers.tpl                            [NEW : ~ 130 LOC, ~ 12 named templates]
|   +-- NOTES.txt                               [NEW : ~ 60 LOC, post-install hints]
|   +-- configmap-manifest.yaml                 [NEW : ~ 80 LOC]
|   +-- controller-deployment.yaml              [NEW : ~ 180 LOC]
|   +-- controller-service.yaml                 [NEW : ~ 35 LOC]
|   +-- controller-serviceaccount.yaml          [NEW : ~ 20 LOC]
|   +-- pdb-controller.yaml                     [NEW : ~ 25 LOC]
|   +-- participant-deployment.yaml             [NEW : ~ 200 LOC, ranges over .Values.participants]
|   +-- participant-service.yaml                [NEW : ~ 45 LOC, ranges]
|   +-- participant-serviceaccount.yaml         [NEW : ~ 30 LOC, ranges]
|   +-- servicemonitor.yaml                     [NEW : ~ 60 LOC, optional]
|   +-- networkpolicy.yaml                      [NEW : ~ 100 LOC, optional]
|   +-- tests/
|       +-- connection-test.yaml                [NEW : ~ 60 LOC, helm test hook]
+-- tests/
    +-- golden/
        +-- kovant-3p.yaml                      [NEW : ~ 900 LOC : full helm template render]
        +-- kovant-disabled.yaml                [NEW : ~ 0 LOC : empty render]
        +-- kovant-network-policy.yaml          [NEW : ~ 1 000 LOC]
        +-- kovant-service-monitor.yaml         [NEW : ~ 1 050 LOC]
```

Parent-chart wiring (`charts/efe-stack/`) :

```
charts/efe-stack/
+-- Chart.yaml                                  [MODIFIED : +6 LOC, registers sub-chart]
+-- values.yaml                                 [MODIFIED : +25 LOC, defaults block]
```

And the CI integration :

```
.github/workflows/helm.yml                        [MODIFIED : +30 LOC, adds golden-diff step]
```

Total : **17 new files + 3 modified ; +1 340 LOC net** (production templates + values + tests).

7. `Chart.yaml` + parent dependency registration

7.1 `charts/efe-stack/charts/efe-meta-swarm/Chart.yaml`

```yaml
apiVersion: v2
name: efe-meta-swarm
description: |
  EFE Phase-2 meta-swarm -- controller + N participants for the
  joint contractive iteration workhorse (Phase-1 `MetaSwarm::iterate`
  packaged for production).  Disabled by default ; flip
  `enabled: true` in `efe-meta-swarm:` of the umbrella values.yaml
  to bring up 1 controller + 3 participants (Kovant scenario).
type: application
version: 0.1.0
appVersion: "0.2.0"      # matches efe-meta-swarm-server crate version
kubeVersion: "&gt;=1.28.0-0"
home: https://efe.example.com/meta-swarm
sources:
  - https://github.com/efe-ai/efe/tree/main/crates/efe-meta-swarm-server
maintainers:
  - name: EFE Meta-Swarm Team
    email: meta-swarm@efe.example.com
keywords:
  - meta-swarm
  - contractive-iteration
  - phase-2
  - reload-config
icon: https://efe.example.com/static/meta-swarm.png
annotations:
  artifacthub.io/license: UNLICENSED
  artifacthub.io/changes: |
    - kind: added
      description: Phase 2 PR #13 -- initial sub-chart skeleton
```

7.2 `charts/efe-stack/Chart.yaml` (modified)

Append to the existing `dependencies:` block :

```yaml
  # Phase-2 meta-swarm controller + N participants.  Cf.
  # `docs/EFE_A1_PHASE2_PR13_ARCHITECTURE.md`.  Disabled by default
  # so pre-Phase-2 deployments stay untouched.
  - name: efe-meta-swarm
    version: "0.1.0"
    repository: ""
    condition: efe-meta-swarm.enabled
```

7.3 `charts/efe-stack/values.yaml` (modified)

Append a `efe-meta-swarm:` block that flips the sub-chart off by default and supplies the cross-chart wiring values that the parent `efe-server` Deployment needs to know about :

```yaml
# Phase-2 meta-swarm subsystem (PR #13).  Off by default ;
# enable in environment-specific values overrides.
efe-meta-swarm:
  enabled: false
  # When enabled, the parent's efe-server chart picks up these
  # connection-time env vars (PR #11 hook).  Override per env.
  efeServerWiring:
    controllerServiceName: efe-meta-swarm-controller
    controllerServicePort: 50080
    connectTimeoutMs: 3500
    callTimeoutMs: 15000
    healthProbeAtStartup: true
```

7.4 Parent `efe-server` Deployment template hook (existing chart)

The existing `charts/efe-stack/charts/efe-server/templates/deployment.yaml` needs a small augmentation to consume the `efeServerWiring` values when the meta-swarm sub-chart is enabled. PR #11 already wired the `EFE_META_SWARM_*` env vars in the binary's config layer ; the chart just templates them :

```
{{- if (index .Values "efe-meta-swarm" "enabled") }}
```

```
- name: EFE_META_SWARM_ENABLED
  value: "true"
- name: EFE_META_SWARM_CONTROLLER_ENDPOINT
  value: "http://{{ (index .Values "efe-meta-swarm" "efeServerWiring" "controllerServiceName") }}.{{ .Rel ■
    ease.Namespace }}.svc:{{ (index .Values "efe-meta-swarm" "efeServerWiring" "controllerServicePort") } ■
    }"
- name: EFE_META_SWARM_CONNECT_TIMEOUT_MS
  value: "{{ (index .Values "efe-meta-swarm" "efeServerWiring" "connectTimeoutMs") }}"
- name: EFE_META_SWARM_CALL_TIMEOUT_MS
  value: "{{ (index .Values "efe-meta-swarm" "efeServerWiring" "callTimeoutMs") }}"
- name: EFE_META_SWARM_HEALTH_PROBE_AT_STARTUP
  value: "{{ (index .Values "efe-meta-swarm" "efeServerWiring" "healthProbeAtStartup") }}"
{{- end }}
```

(Helm sub-charts can read their **sibling** sub-chart's values via `index .Values "<sibling>" "key"` -- this is the standard pattern. The umbrella's `efe-server.deployment` template is in `charts/efe-stack/charts/efe-server/templates/deployment.yaml` ; PR #13 adds the block above.)

8. `values.yaml` schema (operator surface)

Full reference, fully commented :

```
# ---------------------------------------------------------------------
# efe-meta-swarm -- Phase-2 sub-chart values.
#
# This file is merged into the umbrella `efe-stack` values under key
# `efe-meta-swarm:`.  Every field is **typed and validated** by
# `values.schema.json` -- `helm install --dry-run` will reject
# malformed configs before any K8s API call.
#
# Cf. `docs/EFE_A1_PHASE2_PR13_ARCHITECTURE.md`.
# ---------------------------------------------------------------------

# Master switch.  When `false` (default), this sub-chart renders zero
# resources.  When `true`, the chart provisions 1 controller + N
# participants + the manifest ConfigMap + optional NetworkPolicy /
# ServiceMonitor.
enabled: false

# ---------------------------------------------------------------------
# Image -- both binaries (controller + participant) ship in one image.
# Operators MUST pin the tag explicitly (values.schema.json rejects
# the empty string).  No `:latest` policy.
# ---------------------------------------------------------------------
image:
  repository: ghcr.io/efe-ai/efe-meta-swarm-server
  tag: ""                       # REQUIRED -- operator override per release
  pullPolicy: IfNotPresent
  # imagePullSecrets are inherited from the umbrella `global.imagePullSecrets`.

# ---------------------------------------------------------------------
# Manifest -- the YAML that the controller + every participant reads
# at boot AND on SIGHUP (PR #12).  Rendered as a single ConfigMap.
# Operators edit values here ; running pods pick it up via
# `kubectl exec ... -- kill -HUP 1`.
# ---------------------------------------------------------------------
manifest:
  version: "v0.2.0"
  meta:
    # Meta-space dim (Phase-1 `MetaSpace::dim`).
    dim: 16
    safety:
      kind: box                 # box | poly | hull
      radius: 50.0              # Box only -- distance from origin
      # poly:                   # Poly only -- dim_a x dim flat constraint matrix
      #   dim_a: 2
      #   a_flat: [...]
      #   b: [...]
      # hull:                   # Hull only -- Phase-3 placeholder
      #   version: "v1"
      #   serialized_model_path: "/etc/.../hull.bin"
  iteration:
    max_iters: 100
    tolerance: 1.0e-4
    alpha_default: 0.5
    coupling_gamma: 0.2
    deterministic: true
    max_wall_ms: 15000

# ---------------------------------------------------------------------
```

```
# Controller -- single Deployment, replicas=1.  Phase-2 does NOT
# replicate the controller (no leader election yet -- PR #17 follow-up).
# ---------------------------------------------------------------------
controller:
  # replicas: 1 -- hardcoded for Phase 2 ; multi-controller is PR #17.
  # We expose this for forward-compat but values.schema.json caps it
  # at 1 (override locked).
  replicaCount: 1

  # gRPC bind + Prometheus metrics ports.  Exposed via ClusterIP.
  service:
    type: ClusterIP
    grpcPort: 50080
    metricsPort: 9091
    appProtocol: grpc          # L4 hint for service meshes (AD19)

  # Knobs forwarded as env vars to the binary (PR #6's
  # `ControllerConfig::from_env`).
  config:
    maxInflight: 10            # MAX_INFLIGHT (T7 mitigation)
    defaultMaxWallMs: 15000
    workerThreads: 2
    drainTimeoutSeconds: 30
    discovery: static-localhost   # PR #6 modes : static-localhost | k8s-dns
    reflectionEnabled: false   # gRPC reflection -- PR #18 wires
    # TLS -- Phase 2 placeholder ; PR #16 wires cert-manager.
    tlsRequired: false

  # K8s resource requests + limits.  Tuned for kovant-scale ;
  # operators override per environment.
  resources:
    requests:
      cpu: 500m
      memory: 512Mi
    limits:
      cpu: 2
      memory: 2Gi

  # SecurityContext per-container ; the pod-level securityContext is
  # in _helpers.tpl as a shared template.
  securityContext:
    allowPrivilegeEscalation: false
    capabilities:
      drop: ["ALL"]
    readOnlyRootFilesystem: true
    runAsNonRoot: true
    runAsUser: 65532

  # PodDisruptionBudget -- Phase-2 always-protected (AD11).
  podDisruptionBudget:
    enabled: true
    maxUnavailable: 0

  # Pod placement.  Default : no node selector / affinity ; operators
  # add per-env taint tolerations.
  nodeSelector: {}
  tolerations: []
  affinity: {}

  # Liveness + readiness probes -- TCP for Phase 2 (AD18).
  livenessProbe:
    tcpSocket:
      port: 50080
    initialDelaySeconds: 10
    periodSeconds: 10
    timeoutSeconds: 3
    failureThreshold: 3
  readinessProbe:
    tcpSocket:
      port: 50080
    initialDelaySeconds: 5
    periodSeconds: 5
    timeoutSeconds: 3
    failureThreshold: 3

  # Extra env vars merged into the container's env block.  Useful for
  # debug toggles without forcing a values override schema update.
  extraEnv: []

# ---------------------------------------------------------------------
# Participants -- list of distinct binaries, one Deployment + Service
# each.  Default : 3-participant Kovant scenario.  Each entry's
# `id` MUST match an entry in `manifest.participants[*].id` (the
```

```
# values.schema.json enforces this cross-reference).
# ---------------------------------------------------------------------
participants:
  - id: supplier-scoring
    # Echoed into the manifest YAML's participants list :
    dim: 8
    kappa: 0.6
    seed: 11
    blasBackend: none
    # Service / pod knobs :
    replicaCount: 1            # locked to 1 in Phase 2 (schema-enforced)
    service:
      grpcPort: 50091
      metricsPort: 9101
    resources:
      requests: {cpu: 200m, memory: 256Mi}
      limits:   {cpu: 1,    memory: 1Gi}
    nodeSelector: {}
    tolerations: []
    affinity: {}
    extraEnv: []

  - id: contract-risk
    dim: 8
    kappa: 0.7
    seed: 22
    blasBackend: none
    replicaCount: 1
    service: {grpcPort: 50092, metricsPort: 9102}
    resources:
      requests: {cpu: 200m, memory: 256Mi}
      limits:   {cpu: 1,    memory: 1Gi}

  - id: spend-categorization
    dim: 8
    kappa: 0.5
    seed: 33
    blasBackend: none
    replicaCount: 1
    service: {grpcPort: 50093, metricsPort: 9103}
    resources:
      requests: {cpu: 200m, memory: 256Mi}
      limits:   {cpu: 1,    memory: 1Gi}

# ---------------------------------------------------------------------
# ServiceAccount -- one SA per role.  Phase-2 RBAC : zero permissions
# (no K8s API access from the binaries).  PR #16 may grant `secrets:get`
# for cert-manager-issued TLS material.
# ---------------------------------------------------------------------
serviceAccount:
  create: true
  # annotations: {}            # operators may add IAM role bindings here
  # automountServiceAccountToken: false (set in template -- Phase 2 default)

# ---------------------------------------------------------------------
# NetworkPolicy -- deny-by-default east-west traffic between
# meta-swarm pods and everything else.  Opt-in (AD14).
# ---------------------------------------------------------------------
networkPolicy:
  enabled: false
  # Egress :
  #   * meta-swarm controller -&gt; meta-swarm participants
  #   * meta-swarm controller -&gt; metrics scraper (Prometheus pod)
  # Ingress :
  #   * efe-server -&gt; meta-swarm controller (gRPC port)
  #   * Prometheus -&gt; meta-swarm metrics ports
  # Override the allow-list if your scraper / ingress namespace differs.
  ingressNamespaceSelector:
    matchLabels:
      kubernetes.io/metadata.name: efe
  prometheusNamespaceSelector:
    matchLabels:
      kubernetes.io/metadata.name: monitoring

# ---------------------------------------------------------------------
# Prometheus ServiceMonitor -- Prometheus-Operator integration.  Opt-in
# (requires `monitoring.coreos.com/v1` CRDs).
# ---------------------------------------------------------------------
metrics:
  serviceMonitor:
    enabled: false
    interval: 30s
    scrapeTimeout: 10s
```

```
    labels: {}                # extra labels for the operator selector

# ------------------------------------------------------------------------
# Pod-level shared annotations (e.g. linkerd.io/inject, kuma.io/sidecar).
# Empty by default ; operators inject per service-mesh deployment.
# ------------------------------------------------------------------------
podAnnotations: {}

# ------------------------------------------------------------------------
# Helm test connection probe (`helm test`).  Renders a Pod that runs
# grpcurl against MetaSwarmService::Health.
# ------------------------------------------------------------------------
tests:
  enabled: true
  grpcurlImage: "fullstorydev/grpcurl:v1.9.1"
  timeoutSeconds: 30
```

8.1 Why every field is in `values.yaml` (not in `_helpers.tpl`)

Helm idioms : the operator surface is `values.yaml`. Templates read values via `.Values.*` and never carry their own defaults beyond trivial fallbacks (`default 30 .Values.foo`). This makes the schema auditable in one file and `helm install --dry-run -f override.yaml` predictable.

9. `values.schema.json` (JSON Schema validation rules)

Per AD16, this file is shipped at chart root and consumed by Helm 3's built-in schema validation (`helm install` rejects on violation before any K8s API call).

Key constraints :

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://efe.example.com/schemas/efe-meta-swarm.json",
  "title": "efe-meta-swarm sub-chart values",
  "type": "object",
  "required": ["enabled"],
  "additionalProperties": false,
  "properties": {
    "enabled": { "type": "boolean" },
    "image": {
      "type": "object",
      "required": ["repository", "tag", "pullPolicy"],
      "additionalProperties": false,
      "properties": {
        "repository": {
          "type": "string",
          "minLength": 1,
          "pattern": "^[a-z0-9][a-z0-9._/-]*$"
        },
        "tag": {
          "type": "string",
          "minLength": 1,
          "description": "MUST be set (no :latest policy ; AD17)"
        },
        "pullPolicy": {
          "type": "string",
          "enum": ["Always", "IfNotPresent", "Never"]
        }
      }
    },
    "manifest": {
      "type": "object",
      "required": ["version", "meta", "iteration"],
      "additionalProperties": false,
      "properties": {
        "version": {
          "type": "string",
          "pattern": "^v[0-9]+\\.[0-9]+\\.[0-9]+(?:-[a-z0-9]+)?$"
        },
        "meta": {
          "type": "object",
          "required": ["dim", "safety"],
          "additionalProperties": false,
          "properties": {
            "dim": { "type": "integer", "minimum": 1, "maximum": 8192 },
            "safety": {
              "type": "object",
              "required": ["kind"],
```

```
            "properties": {
              "kind": {
                "type": "string",
                "enum": ["box", "poly", "hull"]
              },
              "radius":      { "type": "number", "exclusiveMinimum": 0 },
              "poly": {
                "type": "object",
                "required": ["dim_a", "a_flat", "b"],
                "properties": {
                  "dim_a":  { "type": "integer", "minimum": 1 },
                  "a_flat": { "type": "array", "items": { "type": "number" } },
                  "b":      { "type": "array", "items": { "type": "number" } }
                }
              },
              "hull": {
                "type": "object",
                "required": ["version", "serialized_model_path"],
                "properties": {
                  "version":              { "type": "string" },
                  "serialized_model_path": { "type": "string" }
                }
              }
            },
            "allOf": [
              {
                "if":   { "properties": { "kind": { "const": "box" }}},
                "then": { "required": ["radius"] }
              },
              {
                "if":   { "properties": { "kind": { "const": "poly" }}},
                "then": { "required": ["poly"] }
              },
              {
                "if":   { "properties": { "kind": { "const": "hull" }}},
                "then": { "required": ["hull"] }
              }
            ]
          }
        }
      },
      "iteration": {
        "type": "object",
        "required": [
          "max_iters", "tolerance", "alpha_default",
          "coupling_gamma", "deterministic", "max_wall_ms"
        ],
        "additionalProperties": false,
        "properties": {
          "max_iters":      { "type": "integer", "minimum": 1, "maximum": 200 },
          "tolerance":      { "type": "number",  "exclusiveMinimum": 0 },
          "alpha_default":  { "type": "number",  "minimum": 0, "exclusiveMaximum": 1 },
          "coupling_gamma": { "type": "number",  "minimum": 0, "exclusiveMaximum": 1 },
          "deterministic":  { "type": "boolean" },
          "max_wall_ms":    { "type": "integer", "minimum": 1, "maximum": 30000 }
        }
      }
    }
  },
  "controller": { "$ref": "#/$defs/controllerSpec" },
  "participants": {
    "type": "array",
    "minItems": 1,
    "maxItems": 64,
    "items":   { "$ref": "#/$defs/participantSpec" }
  },
  "serviceAccount":  { "$ref": "#/$defs/serviceAccountSpec" },
  "networkPolicy":   { "$ref": "#/$defs/networkPolicySpec" },
  "metrics":         { "$ref": "#/$defs/metricsSpec" },
  "podAnnotations":  { "type": "object" },
  "tests":           { "$ref": "#/$defs/testsSpec" }
},
"$defs": {
  "controllerSpec": {
    "type": "object",
    "properties": {
      "replicaCount": { "type": "integer", "const": 1, "description": "Phase-2 single-controller ; mult ■
  i-controller HA in PR #17" },
      "../"
    }
  },
  "participantSpec": {
    "type": "object",
```

```
        "required": ["id", "dim", "kappa", "seed"],
        "properties": {
          "id":           { "type": "string", "pattern": "^[a-z][a-z0-9-]{0,62}$" },
          "dim":          { "type": "integer", "minimum": 1, "maximum": 8192 },
          "kappa":        { "type": "number",  "exclusiveMinimum": 0, "exclusiveMaximum": 1 },
          "seed":         { "type": "integer", "minimum": 0 },
          "blasBackend":  { "type": "string", "enum": ["none", "openblas", "mkl", "accelerate"] },
          "replicaCount": { "type": "integer", "const": 1 }
        }
      }
    }
  }
```

9.1 Cross-reference invariants enforced by `values.schema.json`

- `image.tag` is non-empty (AD17).
- Every participant id matches `^[a-z][a-z0-9-]{0,62}$` (DNS-compliant ; same constraint as Service names).
- `kappa ∈ (0, 1)` -- Phase-1's contraction constant range.
- `controller.replicaCount == 1` (Phase-2 lock).
- `participants[*].replicaCount == 1` (Phase-2 lock -- sharded

participants are Phase 3).

- `iteration.max_iters <= 200` matches PR #8's `MAX_ITERS_HARD_CAP`.
- `iteration.max_wall_ms <= 30000` matches PR #8's `MAX_WALL_MS_HARD_CAP`.
- `safety.kind ∈ {box, poly, hull}` -- Phase-1 enum.

What **cannot** be enforced by JSON schema (deferred to runtime validation) :

- Every participant id in `participants[*]` is also present in

`manifest.participants[*].id`. (JSON schema does not have cross-property uniqueness across two arrays.)

- The boot-time `validate_manifest` (PR #8) re-runs the same checks

on the rendered manifest -- defence in depth.

10. `_helpers.tpl` (named templates)

Twelve helpers, each scoped to the sub-chart's naming :

```
{{/*
  efe-meta-swarm.name -- short chart name (max 63 chars, DNS-compliant).
*/}}
{{- define "efe-meta-swarm.name" -}}
{{- default .Chart.Name .Values.nameOverride | trunc 63 | trimSuffix "-" -}}
{{- end -}}

{{/*
  efe-meta-swarm.fullname -- release-prefixed, max 63 chars.
*/}}
{{- define "efe-meta-swarm.fullname" -}}
{{- if .Values.fullnameOverride -}}
{{- .Values.fullnameOverride | trunc 63 | trimSuffix "-" -}}
{{- else -}}
{{- $name := default .Chart.Name .Values.nameOverride -}}
{{- printf "%s-%s" .Release.Name $name | trunc 63 | trimSuffix "-" -}}
{{- end -}}
{{- end -}}

{{/*
  efe-meta-swarm.labels -- full label set + chart version + tenant.
*/}}
{{- define "efe-meta-swarm.labels" -}}
helm.sh/chart: {{ printf "%s-%s" .Chart.Name .Chart.Version | replace "+" "_" | trunc 63 | trimSuffix "-" ■
    }}
{{ include "efe-meta-swarm.selectorLabels" . }}
app.kubernetes.io/version: {{ .Chart.AppVersion | quote }}
app.kubernetes.io/managed-by: {{ .Release.Service }}
app.kubernetes.io/part-of: efe
efe.io/tenant: {{ .Values.global.tenant | default "default" | quote }}
{{- end -}}
```

```
{{/*
  efe-meta-swarm.selectorLabels -- used in Deployment.spec.selector
  AND Service.spec.selector ; MUST be identical for pod targeting.
*/}}
{{- define "efe-meta-swarm.selectorLabels" -}}
app.kubernetes.io/name: {{ include "efe-meta-swarm.name" . }}
app.kubernetes.io/instance: {{ .Release.Name }}
{{- end -}}

{{/*
  efe-meta-swarm.controllerName -- Deployment / Service name for the
  controller.
*/}}
{{- define "efe-meta-swarm.controllerName" -}}
{{ include "efe-meta-swarm.fullname" . }}-controller
{{- end -}}

{{/*
  efe-meta-swarm.participantName -- name of the Deployment / Service
  for a given participant id.  Caller is expected to set $participant
  in the surrounding scope (cf. participant-deployment.yaml `range`).
*/}}
{{- define "efe-meta-swarm.participantName" -}}
{{- $root := index . 0 -}}
{{- $id := index . 1 -}}
{{ include "efe-meta-swarm.fullname" $root }}-participant-{{ $id }}
{{- end -}}

{{/*
  efe-meta-swarm.serviceAccountName -- operator-overridable, defaults
  to fullname-&lt;role&gt;.
*/}}
{{- define "efe-meta-swarm.controllerServiceAccountName" -}}
{{- if .Values.serviceAccount.create -}}
{{- default (printf "%s-controller" (include "efe-meta-swarm.fullname" .)) .Values.serviceAccount.control ■
    lerName -}}
{{- else -}}
{{- default "default" .Values.serviceAccount.controllerName -}}
{{- end -}}
{{- end -}}

{{- define "efe-meta-swarm.participantServiceAccountName" -}}
{{- $root := index . 0 -}}
{{- $id := index . 1 -}}
{{- if $root.Values.serviceAccount.create -}}
{{ include "efe-meta-swarm.fullname" $root }}-participant-{{ $id }}
{{- else -}}
default
{{- end -}}
{{- end -}}

{{/*
  efe-meta-swarm.image -- fully qualified image ref ; rejects empty tag
  (defence in depth ; the schema also enforces it).
*/}}
{{- define "efe-meta-swarm.image" -}}
{{- $tag := .Values.image.tag -}}
{{- if empty $tag -}}
{{- fail "efe-meta-swarm: .Values.image.tag MUST be set (no :latest policy ; AD17)" -}}
{{- end -}}
{{- printf "%s:%s" .Values.image.repository $tag -}}
{{- end -}}

{{/*
  efe-meta-swarm.participantsEnv -- comma-separated env value for the
  controller's `EFE_META_SWARM_PARTICIPANTS` ; renders
  `id=svc.ns.svc:port,id=svc.ns.svc:port,...` from .Values.participants.
*/}}
{{- define "efe-meta-swarm.participantsEnv" -}}
{{- $root := . -}}
{{- $entries := list -}}
{{- range .Values.participants -}}
{{- $svc := printf "%s-participant-%s" (include "efe-meta-swarm.fullname" $root) .id -}}
{{- $entry := printf "%s=%s.%s.svc:%d" .id $svc $root.Release.Namespace (int .service.grpcPort) -}}
{{- $entries = append $entries $entry -}}
{{- end -}}
{{- join "," $entries -}}
{{- end -}}

{{/*
  efe-meta-swarm.manifestYaml -- the rendered Phase-2 manifest YAML body.
  Consumed both by configmap-manifest.yaml and (in tests) by the
  golden-snapshot tooling for round-tripping.
```

```
*/}}
{{- define "efe-meta-swarm.manifestYaml" -}}
version: {{ .Values.manifest.version | quote }}
meta:
  dim: {{ .Values.manifest.meta.dim }}
  safety:
    kind: {{ .Values.manifest.meta.safety.kind }}
{{- if eq .Values.manifest.meta.safety.kind "box" }}
    radius: {{ .Values.manifest.meta.safety.radius }}
{{- else if eq .Values.manifest.meta.safety.kind "poly" }}
    dim_a: {{ .Values.manifest.meta.safety.poly.dim_a }}
    a_flat: {{ .Values.manifest.meta.safety.poly.a_flat | toJson }}
    b:      {{ .Values.manifest.meta.safety.poly.b | toJson }}
{{- else if eq .Values.manifest.meta.safety.kind "hull" }}
    version: {{ .Values.manifest.meta.safety.hull.version | quote }}
    serialized_model_path: {{ .Values.manifest.meta.safety.hull.serialized_model_path | quote }}
{{- end }}
participants:
{{- range .Values.participants }}
  - id: {{ .id }}
    dim: {{ .dim }}
    kappa: {{ .kappa }}
    seed: {{ .seed }}
    blas_backend: {{ .blasBackend | quote }}
{{- end }}
iteration:
  max_iters:      {{ .Values.manifest.iteration.max_iters }}
  tolerance:      {{ .Values.manifest.iteration.tolerance }}
  alpha_default:  {{ .Values.manifest.iteration.alpha_default }}
  coupling_gamma: {{ .Values.manifest.iteration.coupling_gamma }}
  deterministic:  {{ .Values.manifest.iteration.deterministic }}
  max_wall_ms:    {{ .Values.manifest.iteration.max_wall_ms }}
{{- end -}}
```

10.1 Why `manifestYaml` is a named template (not inline in `configmap-manifest.yaml`)

- **Reusable in tests** : the golden-snapshot tooling renders this

template alone and runs the rendered string through Rust's `manifest::load_and_validate` (PR #8) to assert round-trip correctness.

- **Single source of truth** : if the manifest schema changes (e.g.

new safety variant), only one place to update.

11. `configmap-manifest.yaml` (manifest YAML rendering)

```
{{- if .Values.enabled }}
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ include "efe-meta-swarm.fullname" . }}-manifest
  labels:
    {{- include "efe-meta-swarm.labels" . | nindent 4 }}
    efe.io/meta-swarm-role: shared
  annotations:
    efe.io/manifest-schema-version: {{ .Values.manifest.version | quote }}
data:
  # The whole Phase-2 manifest as a single YAML key.  Both the
  # controller and every participant mount this ConfigMap at
  # `/etc/efe-meta-swarm/manifest.yaml` and set
  # `EFE_META_SWARM_MANIFEST=/etc/efe-meta-swarm/manifest.yaml`.
  #
  # SIGHUP-driven reload (PR #12) re-reads from this path ; the
  # operator's edit-then-`helm upgrade`-then-`kill -HUP` workflow is
  # documented in the runbook (§ 41).
  manifest.yaml: |-
{{ include "efe-meta-swarm.manifestYaml" . | indent 4 }}
{{- end }}
```

11.1 Why one ConfigMap with one YAML key

- Matches PR #6's `EFE_META_SWARM_MANIFEST=<path>` convention.

- Atomic edits via `kubectl apply` or `helm upgrade` swap the whole

file (no partial reads).

- Operators can also `kubectl edit configmap ...` for emergency

surgery, then SIGHUP the pods.

11.2 Why no `checksum/config` annotation propagation

AD4 : the operator owns the reload trigger. `helm upgrade` swaps the ConfigMap content ; the Pod templates do **not** carry a content-hash annotation, so the `RollingUpdate` controller does not restart pods on ConfigMap edit. Reload is explicit via `kill -HUP`.

This is **opposite** to the parent `efe-server` chart's policy (which DOES use `checksum/config`) because `efe-server` doesn't have a SIGHUP path -- it relies on rolling restart for config changes.

12. `controller-deployment.yaml`

```
{{- if .Values.enabled }}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "efe-meta-swarm.controllerName" . }}
  labels:
    {{- include "efe-meta-swarm.labels" . | nindent 4 }}
    app.kubernetes.io/component: controller
    efe.io/meta-swarm-role: controller
spec:
  replicas: {{ .Values.controller.replicaCount | default 1 }}
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 0
      maxUnavailable: 1        # single-replica Deployment ; rolling restart drops the controller briefly ■
     (PR12-R7 concern -- documented)
  selector:
    matchLabels:
      {{- include "efe-meta-swarm.selectorLabels" . | nindent 6 }}
      app.kubernetes.io/component: controller
  template:
    metadata:
      labels:
        {{- include "efe-meta-swarm.selectorLabels" . | nindent 8 }}
        app.kubernetes.io/component: controller
        efe.io/meta-swarm-role: controller
      annotations:
        # PR #13 AD4 : NO `checksum/config` ; reload is SIGHUP-driven.
        prometheus.io/scrape: "true"
        prometheus.io/port: "{{ .Values.controller.service.metricsPort }}"
        prometheus.io/path: "/metrics"
        {{- with .Values.podAnnotations }}
        {{- toYaml . | nindent 8 }}
        {{- end }}
    spec:
      serviceAccountName: {{ include "efe-meta-swarm.controllerServiceAccountName" . }}
      automountServiceAccountToken: false
      {{- with .Values.global.imagePullSecrets }}
      imagePullSecrets:
        {{- toYaml . | nindent 8 }}
      {{- end }}
      securityContext:
        runAsNonRoot: true
        runAsUser: 65532
        runAsGroup: 65532
        fsGroup: 65532
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: controller
          image: {{ include "efe-meta-swarm.image" . | quote }}
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          args: ["controller"]
          ports:
            - name: grpc
              containerPort: {{ .Values.controller.service.grpcPort }}
              protocol: TCP
            - name: metrics
              containerPort: {{ .Values.controller.service.metricsPort }}
              protocol: TCP
          env:
            - name: EFE_META_SWARM_CONTROLLER_ADDR
              value: "0.0.0.0:{{ .Values.controller.service.grpcPort }}"
            - name: EFE_META_SWARM_CONTROLLER_METRICS
              value: "0.0.0.0:{{ .Values.controller.service.metricsPort }}"
            - name: EFE_META_SWARM_MANIFEST
              value: /etc/efe-meta-swarm/manifest.yaml
            - name: EFE_META_SWARM_PARTICIPANTS
              value: {{ include "efe-meta-swarm.participantsEnv" . | quote }}
            - name: EFE_META_SWARM_DISCOVERY
              value: {{ .Values.controller.config.discovery | quote }}
            - name: EFE_META_SWARM_MAX_INFLIGHT
              value: {{ .Values.controller.config.maxInflight | quote }}
            - name: EFE_META_SWARM_DEFAULT_MAX_WALL_MS
              value: {{ .Values.controller.config.defaultMaxWallMs | quote }}
            - name: EFE_META_SWARM_WORKER_THREADS
              value: {{ .Values.controller.config.workerThreads | quote }}
            - name: EFE_META_SWARM_RELOAD_DRAIN_TIMEOUT_S
              value: {{ .Values.controller.config.drainTimeoutSeconds | quote }}
```

```
            - name: EFE_META_SWARM_REFLECTION
              value: {{ .Values.controller.config.reflectionEnabled | quote }}
            - name: EFE_META_SWARM_TLS_REQUIRED
              value: {{ .Values.controller.config.tlsRequired | quote }}
            {{- with .Values.controller.extraEnv }}
            {{- toYaml . | nindent 12 }}
            {{- end }}
          resources:
            {{- toYaml .Values.controller.resources | nindent 12 }}
          securityContext:
            {{- toYaml .Values.controller.securityContext | nindent 12 }}
          livenessProbe:
            {{- toYaml .Values.controller.livenessProbe | nindent 12 }}
          readinessProbe:
            {{- toYaml .Values.controller.readinessProbe | nindent 12 }}
          volumeMounts:
            - name: manifest
              mountPath: /etc/efe-meta-swarm
              readOnly: true
      volumes:
        - name: manifest
          configMap:
            name: {{ include "efe-meta-swarm.fullname" . }}-manifest
      {{- with .Values.controller.nodeSelector }}
      nodeSelector:
        {{- toYaml . | nindent 8 }}
      {{- end }}
      {{- with .Values.controller.tolerations }}
      tolerations:
        {{- toYaml . | nindent 8 }}
      {{- end }}
      {{- with .Values.controller.affinity }}
      affinity:
        {{- toYaml . | nindent 8 }}
      {{- end }}
      terminationGracePeriodSeconds: {{ add (int .Values.controller.config.drainTimeoutSeconds) 5 }}
{{- end }}
```

12.1 Notes on the controller Deployment

- `args: ["controller"]` -- the binary's `Cargo.toml` declares two

`[[bin]]` entries ; the image's ENTRYPOINT is either the package's default binary OR `["/usr/local/bin/entrypoint"]` that dispatches on argv (defined in the Dockerfile, NOT in PR #13 -- this is a PR #6 / PR #10 image-build concern).

- `terminationGracePeriodSeconds` = drain timeout + 5 s slack ;

matches `EFE_META_SWARM_RELOAD_DRAIN_TIMEOUT_S` + the binary's own SIGTERM drain path (PR #6).

- No `lifecycle.preStop` hook -- the binary's SIGTERM handler

(`shutdown::await_termination`, PR #6) handles graceful drain natively.

- `volumes.manifest` mounts the ConfigMap read-only ; the binary

NEVER writes to the manifest path.

13. `controller-service.yaml` + headless DNS

```
{{- if .Values.enabled }}
apiVersion: v1
kind: Service
metadata:
  name: {{ include "efe-meta-swarm.controllerName" . }}
  labels:
    {{- include "efe-meta-swarm.labels" . | nindent 4 }}
    app.kubernetes.io/component: controller
spec:
  type: {{ .Values.controller.service.type }}
  ports:
    - name: grpc
      port: {{ .Values.controller.service.grpcPort }}
      targetPort: grpc
      protocol: TCP
      appProtocol: {{ .Values.controller.service.appProtocol | quote }}
    - name: metrics
      port: {{ .Values.controller.service.metricsPort }}
      targetPort: metrics
      protocol: TCP
  selector:
    {{- include "efe-meta-swarm.selectorLabels" . | nindent 4 }}
    app.kubernetes.io/component: controller
{{- end }}
```

13.1 DNS contract

The Service's FQDN is : `` `<release>-efe-meta-swarm-controller.<namespace>.svc.cluster.local` ``

(or just `<svc>.<ns>.svc` for in-namespace clients).

The parent `efe-server` Deployment's `EFE_META_SWARM_CONTROLLER_ENDPOINT` env var is templated to this FQDN (§ 7.4).

14. `participant-deployment.yaml` (`range` over participants)

```
{{- if .Values.enabled }}
{{- range $p := .Values.participants }}
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "efe-meta-swarm.participantName" (list $ $p.id) }}
  labels:
    {{- include "efe-meta-swarm.labels" $ | nindent 4 }}
    app.kubernetes.io/component: participant
    efe.io/meta-swarm-role: participant
    efe.io/participant-id: {{ $p.id | quote }}
spec:
  replicas: {{ $p.replicaCount | default 1 }}
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 0
      maxUnavailable: 1
  selector:
    matchLabels:
      {{- include "efe-meta-swarm.selectorLabels" $ | nindent 6 }}
      efe.io/participant-id: {{ $p.id | quote }}
  template:
    metadata:
      labels:
        {{- include "efe-meta-swarm.selectorLabels" $ | nindent 8 }}
        app.kubernetes.io/component: participant
        efe.io/meta-swarm-role: participant
        efe.io/participant-id: {{ $p.id | quote }}
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/port: "{{ $p.service.metricsPort }}"
        prometheus.io/path: "/metrics"
        {{- with $.Values.podAnnotations }}
        {{- toYaml . | nindent 8 }}
        {{- end }}
    spec:
      serviceAccountName: {{ include "efe-meta-swarm.participantServiceAccountName" (list $ $p.id) }}
      automountServiceAccountToken: false
      {{- with $.Values.global.imagePullSecrets }}
      imagePullSecrets:
        {{- toYaml . | nindent 8 }}
      {{- end }}
      securityContext:
        runAsNonRoot: true
        runAsUser: 65532
        runAsGroup: 65532
        fsGroup: 65532
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: participant
          image: {{ include "efe-meta-swarm.image" $ | quote }}
          imagePullPolicy: {{ $.Values.image.pullPolicy }}
          args: ["participant"]
          ports:
            - name: grpc
              containerPort: {{ $p.service.grpcPort }}
              protocol: TCP
            - name: metrics
              containerPort: {{ $p.service.metricsPort }}
              protocol: TCP
          env:
            - name: EFE_META_SWARM_PARTICIPANT_ID
              value: {{ $p.id | quote }}
            - name: EFE_META_SWARM_PARTICIPANT_ADDR
              value: "0.0.0.0:{{ $p.service.grpcPort }}"
            - name: EFE_META_SWARM_PARTICIPANT_METRICS
              value: "0.0.0.0:{{ $p.service.metricsPort }}"
            - name: EFE_META_SWARM_MANIFEST
              value: /etc/efe-meta-swarm/manifest.yaml
            - name: EFE_META_SWARM_WORKER_THREADS
              value: "1"
            {{- with $p.extraEnv }}
            {{- toYaml . | nindent 12 }}
            {{- end }}
          resources:
            {{- toYaml $p.resources | nindent 12 }}
          securityContext:
```

```
              allowPrivilegeEscalation: false
              capabilities:
                drop: ["ALL"]
              readOnlyRootFilesystem: true
              runAsNonRoot: true
              runAsUser: 65532
            livenessProbe:
              tcpSocket:
                port: {{ $p.service.grpcPort }}
              initialDelaySeconds: 10
              periodSeconds: 10
              timeoutSeconds: 3
              failureThreshold: 3
            readinessProbe:
              tcpSocket:
                port: {{ $p.service.grpcPort }}
              initialDelaySeconds: 5
              periodSeconds: 5
              timeoutSeconds: 3
              failureThreshold: 3
            volumeMounts:
              - name: manifest
                mountPath: /etc/efe-meta-swarm
                readOnly: true
        volumes:
          - name: manifest
            configMap:
              name: {{ include "efe-meta-swarm.fullname" $ }}-manifest
        {{- with $p.nodeSelector }}
        nodeSelector:
          {{- toYaml . | nindent 8 }}
        {{- end }}
        {{- with $p.tolerations }}
        tolerations:
          {{- toYaml . | nindent 8 }}
        {{- end }}
        {{- with $p.affinity }}
        affinity:
          {{- toYaml . | nindent 8 }}
        {{- end }}
        terminationGracePeriodSeconds: 30
    {{- end }}
    {{- end }}
```

14.1 Why per-participant Deployment + per-id label

- Each participant is a *distinct* binary instance with its own

`EFE_META_SWARM_PARTICIPANT_ID`. A single Deployment with `replicas: 3` would have all pods share the same env var.

- The `efe.io/participant-id` label lets operators

`kubectl get pods -l efe.io/participant-id=supplier-scoring`.

15. `participant-service.yaml`

```
{{- if .Values.enabled }}
{{- range $p := .Values.participants }}
---
apiVersion: v1
kind: Service
metadata:
  name: {{ include "efe-meta-swarm.participantName" (list $ $p.id) }}
  labels:
    {{- include "efe-meta-swarm.labels" $ | nindent 4 }}
    app.kubernetes.io/component: participant
    efe.io/meta-swarm-role: participant
    efe.io/participant-id: {{ $p.id | quote }}
spec:
  type: ClusterIP
  ports:
    - name: grpc
      port: {{ $p.service.grpcPort }}
      targetPort: grpc
      protocol: TCP
      appProtocol: grpc
    - name: metrics
      port: {{ $p.service.metricsPort }}
      targetPort: metrics
      protocol: TCP
  selector:
    {{- include "efe-meta-swarm.selectorLabels" $ | nindent 4 }}
    efe.io/participant-id: {{ $p.id | quote }}
{{- end }}
{{- end }}
```

16. `serviceaccount.yaml` + RBAC (Phase-2 minimal surface)

```
{{- if and .Values.enabled .Values.serviceAccount.create }}
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: {{ include "efe-meta-swarm.controllerServiceAccountName" . }}
  labels:
    {{- include "efe-meta-swarm.labels" . | nindent 4 }}
    app.kubernetes.io/component: controller
    efe.io/meta-swarm-role: controller
  {{- with .Values.serviceAccount.annotations }}
  annotations:
    {{- toYaml . | nindent 4 }}
  {{- end }}
automountServiceAccountToken: false

{{- range $p := .Values.participants }}
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: {{ include "efe-meta-swarm.participantServiceAccountName" (list $ $p.id) }}
  labels:
    {{- include "efe-meta-swarm.labels" $ | nindent 4 }}
    app.kubernetes.io/component: participant
    efe.io/meta-swarm-role: participant
    efe.io/participant-id: {{ $p.id | quote }}
automountServiceAccountToken: false
{{- end }}
{{- end }}
```

16.1 Why no `Role` / `RoleBinding`

Phase-2 binaries make zero K8s API calls. `automountServiceAccountToken: false` is the runtime-level guarantee ; no RBAC = no privileges. PR #16 (mTLS) may grant `secrets:get` for cert-manager-issued certs ; PR #18 (in-cluster watchers) may grant `configmaps:watch`.

17. `pdb-controller.yaml` (PodDisruptionBudget)

```
{{- if and .Values.enabled .Values.controller.podDisruptionBudget.enabled }}
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: {{ include "efe-meta-swarm.controllerName" . }}-pdb
  labels:
    {{- include "efe-meta-swarm.labels" . | nindent 4 }}
    app.kubernetes.io/component: controller
spec:
  maxUnavailable: {{ .Values.controller.podDisruptionBudget.maxUnavailable }}
  selector:
    matchLabels:
      {{- include "efe-meta-swarm.selectorLabels" . | nindent 6 }}
      app.kubernetes.io/component: controller
{{- end }}
```

18. `servicemonitor.yaml` (Prometheus Operator, optional)

```
{{- if and .Values.enabled .Values.metrics.serviceMonitor.enabled }}
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: {{ include "efe-meta-swarm.fullname" . }}
  labels:
    {{- include "efe-meta-swarm.labels" . | nindent 4 }}
    {{- with .Values.metrics.serviceMonitor.labels }}
    {{- toYaml . | nindent 4 }}
    {{- end }}
spec:
  selector:
    matchLabels:
      {{- include "efe-meta-swarm.selectorLabels" . | nindent 6 }}
  namespaceSelector:
    matchNames:
      - {{ .Release.Namespace }}
  endpoints:
    - port: metrics
      path: /metrics
      interval: {{ .Values.metrics.serviceMonitor.interval }}
      scrapeTimeout: {{ .Values.metrics.serviceMonitor.scrapeTimeout }}
      honorLabels: true
      relabelings:
        - sourceLabels: [__meta_kubernetes_pod_label_efe_io_meta_swarm_role]
          targetLabel: role
        - sourceLabels: [__meta_kubernetes_pod_label_efe_io_participant_id]
          targetLabel: participant_id
{{- end }}
```

18.1 Scraped metrics surface (PR #11 / #12 / #17)

- `efe_meta_swarm_health_probes_total` (PR #11)
- `efe_meta_swarm_health_probe_errors_total` (PR #11)
- `efe_meta_swarm_health_probe_duration_seconds` (PR #11)
- `efe_meta_swarm_reload_attempts_total` (PR #12)
- `efe_meta_swarm_reload_failures_total{reason}` (PR #12)
- `efe_meta_swarm_reload_duration_microseconds` (PR #12)
- `efe_meta_swarm_reload_gen` (PR #12)
- (Phase 3) `efe_meta_swarm_iterate_*` (PR #17 wires)

19. `networkpolicy.yaml` (deny-by-default, optional)

```
{{- if and .Values.enabled .Values.networkPolicy.enabled }}
---
# Default-deny ingress + egress on every meta-swarm pod.  Specific
# allow-lists below explicitly permit the legitimate traffic.
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: {{ include "efe-meta-swarm.fullname" . }}-deny-all
  labels:
    {{- include "efe-meta-swarm.labels" . | nindent 4 }}
spec:
  podSelector:
    matchLabels:
      {{- include "efe-meta-swarm.selectorLabels" . | nindent 6 }}
  policyTypes: [Ingress, Egress]

---
# Allow efe-server -&gt; controller (gRPC) and Prometheus -&gt; metrics ports.
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: {{ include "efe-meta-swarm.controllerName" . }}-allow-ingress
spec:
  podSelector:
    matchLabels:
      {{- include "efe-meta-swarm.selectorLabels" . | nindent 6 }}
      app.kubernetes.io/component: controller
  policyTypes: [Ingress]
  ingress:
    - from:
        - namespaceSelector:
            {{- toYaml .Values.networkPolicy.ingressNamespaceSelector | nindent 12 }}
          podSelector:
            matchLabels:
              app.kubernetes.io/name: efe-server
      ports:
        - protocol: TCP
          port: {{ .Values.controller.service.grpcPort }}
    - from:
        - namespaceSelector:
            {{- toYaml .Values.networkPolicy.prometheusNamespaceSelector | nindent 12 }}
      ports:
        - protocol: TCP
          port: {{ .Values.controller.service.metricsPort }}

---
# Allow controller -&gt; participant (gRPC fan-out).
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: {{ include "efe-meta-swarm.fullname" . }}-controller-egress
spec:
  podSelector:
    matchLabels:
      {{- include "efe-meta-swarm.selectorLabels" . | nindent 6 }}
      app.kubernetes.io/component: controller
  policyTypes: [Egress]
  egress:
    - to:
        - podSelector:
            matchLabels:
              {{- include "efe-meta-swarm.selectorLabels" . | nindent 14 }}
              app.kubernetes.io/component: participant
      # Permit all participant gRPC ports (range).
      ports:
        {{- range $p := .Values.participants }}
        - protocol: TCP
          port: {{ $p.service.grpcPort }}
        {{- end }}
    # Allow DNS lookups (kube-dns is in kube-system by convention).
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: kube-system
          podSelector:
            matchLabels:
              k8s-app: kube-dns
      ports:
        - protocol: UDP
          port: 53
```

```
        - protocol: TCP
          port: 53

---
# Allow participant ingress from controller + Prometheus.
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: {{ include "efe-meta-swarm.fullname" . }}-participants-allow-ingress
spec:
  podSelector:
    matchLabels:
      {{- include "efe-meta-swarm.selectorLabels" . | nindent 6 }}
      app.kubernetes.io/component: participant
  policyTypes: [Ingress]
  ingress:
    - from:
        - podSelector:
            matchLabels:
              {{- include "efe-meta-swarm.selectorLabels" . | nindent 14 }}
              app.kubernetes.io/component: controller
      ports:
        {{- range $p := .Values.participants }}
        - protocol: TCP
          port: {{ $p.service.grpcPort }}
        {{- end }}
    - from:
        - namespaceSelector:
            {{- toYaml .Values.networkPolicy.prometheusNamespaceSelector | nindent 12 }}
      ports:
        {{- range $p := .Values.participants }}
        - protocol: TCP
          port: {{ $p.service.metricsPort }}
        {{- end }}
{{- end }}
```

19.1 Why opt-in

NetworkPolicy enforcement requires a CNI plug-in that supports it (Calico, Cilium, Weave with policy). On clusters with a non-policy CNI (Flannel default), the resources render but have no effect ; the operator may not realise. Default-off makes the operator opt-in consciously.

20. `templates/tests/connection-test.yaml` (`helm test` hook)

```
{{- if and .Values.enabled .Values.tests.enabled }}
apiVersion: v1
kind: Pod
metadata:
  name: {{ include "efe-meta-swarm.fullname" . }}-test-connection
  labels:
    {{- include "efe-meta-swarm.labels" . | nindent 4 }}
    app.kubernetes.io/component: test
  annotations:
    "helm.sh/hook": test
    "helm.sh/hook-delete-policy": before-hook-creation,hook-succeeded
spec:
  restartPolicy: Never
  activeDeadlineSeconds: {{ .Values.tests.timeoutSeconds }}
  serviceAccountName: default
  automountServiceAccountToken: false
  containers:
    - name: grpcurl
      image: {{ .Values.tests.grpcurlImage | quote }}
      imagePullPolicy: IfNotPresent
      command:
        - sh
        - -c
        - |
          set -eu
          ENDPOINT="{{ include "efe-meta-swarm.controllerName" . }}.{{ .Release.Namespace }}.svc:{{ .Valu ■
    es.controller.service.grpcPort }}"
          echo "Probing $ENDPOINT ..."
          # Use grpcurl reflection ; fallback to method-by-name if
          # reflection is disabled.
          grpcurl -plaintext -d '{}' -import-path /protos \
            -max-time {{ sub .Values.tests.timeoutSeconds 5 }} \
            "$ENDPOINT" \
            efe.v1.meta_swarm.MetaSwarmService/Health
{{- end }}
```

Operators run `helm test efe-stack -n efe` after install.

20.1 Why a Pod (not a Job)

- `Job` semantics retry on failure ; `Pod` with `restartPolicy: Never`

fails-fast -- that's what we want for `helm test`.

- `helm test` deletes the Pod automatically on success (via

`hook-delete-policy`).

21. `NOTES.txt` + `README.md`

21.1 `NOTES.txt`

Rendered after `helm install` ; gives the operator the immediate post-install steps.

```
{{- if .Values.enabled }}
======================================================================
EFE-A1 Phase 2 meta-swarm -- installed.
======================================================================

Release   : {{ .Release.Name }}
Namespace : {{ .Release.Namespace }}
Topology  : 1x controller + {{ len .Values.participants }} participants

Controller endpoint (in-cluster) :
    http://{{ include "efe-meta-swarm.controllerName" . }}.{{ .Release.Namespace }}.svc:{{ .Values.contro ■
    ller.service.grpcPort }}

Participants :
{{- range $p := .Values.participants }}
    {{ $p.id }} -&gt; {{ include "efe-meta-swarm.participantName" (list $ $p.id) }}.{{ $.Release.Namespac ■
    e }}.svc:{{ $p.service.grpcPort }}
{{- end }}

Next steps :
  1. Verify pods came up :
```

```
       kubectl -n {{ .Release.Namespace }} get pods -l efe.io/meta-swarm-role

  2. Run the `helm test` connection probe :
       helm test {{ .Release.Name }} -n {{ .Release.Namespace }}

  3. SIGHUP-driven reload after a manifest edit
     (PR #12 hot reload -- no pod restart) :
       helm upgrade {{ .Release.Name }} ./charts/efe-stack \
         -f values-prod.yaml
       kubectl -n {{ .Release.Namespace }} exec \
         deploy/{{ include "efe-meta-swarm.controllerName" . }} -- kill -HUP 1
{{- range $p := .Values.participants }}
       kubectl -n {{ $.Release.Namespace }} exec \
         deploy/{{ include "efe-meta-swarm.participantName" (list $ $p.id) }} -- kill -HUP 1
{{- end }}

  4. Operator runbook + threat model :
       docs/EFE_A1_PHASE2_PR13_ARCHITECTURE.md   (this PR)
       docs/EFE_A1_PHASE2_PR12_ARCHITECTURE.md   (SIGHUP semantics)
{{- end }}
```

21.2 `README.md`

Operator-facing. ~ 180 LOC covering :

- What the sub-chart provides
- Default vs production overrides
- `values.yaml` reference (top-level keys + types)
- `helm install` / `helm upgrade` / `helm rollback` examples
- SIGHUP reload workflow
- Troubleshooting (Pod not Ready, gRPC connection refused, RELOADING stuck)
- Cross-reference to threat model + PR #12 reload semantics

22. Image management -- registry, tag pinning, pullSecrets

- `image.repository` defaults to `ghcr.io/efe-ai/efe-meta-swarm-server`

(placeholder -- the production registry will be the EFE-private one ; the chart's default is configurable).

- `image.tag` defaults to `""` -- `values.schema.json` rejects, so

operators MUST set it. No `:latest` policy.

- `imagePullSecrets` inherits from umbrella `global.imagePullSecrets`

(already set up by sibling sub-charts).

- `image.pullPolicy: IfNotPresent` matches existing conventions.

22.1 Image digest pinning (Phase 3 follow-up)

Once cosign / Sigstore is in the CI pipeline, the chart's helper template can be enhanced to template `image: ghcr.io/.../efe-meta-swarm-server@sha256:...` instead of `image: ...:tag` -- matching the Phase-3 supply-chain story.

23. SecurityContext / hardening

Per-pod : ``yaml runAsNonRoot: true runAsUser: 65532 runAsGroup: 65532 fsGroup: 65532 seccompProfile: type: RuntimeDefault ``

Per-container : ``yaml allowPrivilegeEscalation: false capabilities: drop: ["ALL"] readOnlyRootFilesystem: true runAsNonRoot: true runAsUser: 65532 ``

23.1 Why `readOnlyRootFilesystem: true`

The binaries don't write to disk -- manifest is mounted RO, logs go to stdout, metrics are served from memory. Locking the rootfs prevents post-exploitation persistence. The only writable path is the ConfigMap mount (RO at the mount level too).

If a future binary needs scratch space, add an `emptyDir` volume with `medium: Memory` and a specific mount path.

24. Resource sizing + autoscaling deferral

Role	Requests	Limits	Rationale
Controller	500m / 512Mi	2 / 2Gi	MAX_INFLIGHT=10 ; 3 participants ; worker threads = 2 ; headroom for SIGHUP reload spikes
Participant	200m / 256Mi	1 / 1Gi	dim <= 256 ; single tensor mutex ; SIGHUP reload is cheap

24.1 Why no HPA in Phase 2

- Controller : single-replica (AD11). HPA on a single-replica

Deployment is meaningless.

- Participants : single-replica per id (AD12). HPA per participant

would replicate the state ; Phase-2 doesn't support sharded participant state.

Phase 3 (PR #17 multi-controller HA) may add HPA on the controller with leader-election.

25. SIGHUP-on-`ConfigMap`-change strategy

PR #13 deliberately does **not** ship an automatic SIGHUP trigger. Options considered :

Option	Pros	Cons	Phase-2 decision
Manual `kubectl exec ... kill -HUP 1`	Simplest ; operator-controlled audit trail	One command per pod	**Adopted for Phase 2**
`checksum/config` annotation	Native K8s ; no extra moving parts	Rolling restart on every change (defeats SIGHUP)	Rejected -- AD4
Reloader sidecar (third-party)	Automatic ; widely used	Extra component ; sidecar lifecycle	Rejected -- out of scope
In-cluster `ConfigMap`-watcher Pod that kubectl-exec-SIGHUPs targets	Native ; matches our design	Needs RBAC ; one more moving part	Deferred to PR #18

25.1 The operator's exact incantation (runbook lift)

```
# 1. Edit the manifest in values.yaml.
$ vim charts/efe-stack/values-prod.yaml

# 2. Upgrade the release ; only the ConfigMap mutates (pods stay up).
$ helm upgrade efe-stack ./charts/efe-stack \
    -f charts/efe-stack/values-prod.yaml \
    -n efe

# 3. SIGHUP the controller.
$ kubectl -n efe exec deploy/efe-stack-efe-meta-swarm-controller -- kill -HUP 1

# 4. SIGHUP every participant.
$ for id in supplier-scoring contract-risk spend-categorization ; do
    kubectl -n efe exec deploy/efe-stack-efe-meta-swarm-participant-$id -- kill -HUP 1
  done

# 5. Verify the reload landed.
$ kubectl -n efe port-forward svc/efe-stack-efe-meta-swarm-controller 50080:50080 &amp;
$ grpcurl -plaintext -d '{}' localhost:50080 efe.v1.meta_swarm.MetaSwarmService/Health \
  | jq '.reloadGen'   # should have incremented by 1
```

25.2 Optional one-liner (Phase 2)

A `sighup_all.sh` helper script (NOT shipped in the chart ; lives in `charts/efe-stack/scripts/` if useful) :

```
#!/usr/bin/env bash
set -euo pipefail
NS="${1:?usage: $0 &lt;namespace&gt; &lt;release-name&gt;}"
RELEASE="${2:?usage: $0 &lt;namespace&gt; &lt;release-name&gt;}"
kubectl -n "$NS" exec deploy/"${RELEASE}-efe-meta-swarm-controller" -- kill -HUP 1
for d in $(kubectl -n "$NS" get deploy -l app.kubernetes.io/component=participant \
             -o jsonpath='{.items[*].metadata.name}') ; do
  kubectl -n "$NS" exec deploy/"$d" -- kill -HUP 1
done
```

Documented in `README.md` but not auto-installed.

26. Golden snapshot test strategy

26.1 Mechanism

A `tests/golden/` directory inside the sub-chart holds the **expected** `helm template` output for canonical scenarios :

- `kovant-3p.yaml` -- default `values.yaml`
- `kovant-disabled.yaml` -- `enabled: false` (must be empty)
- `kovant-network-policy.yaml` -- `networkPolicy.enabled: true`
- `kovant-service-monitor.yaml` -- `metrics.serviceMonitor.enabled: true`

CI runs :

```
# Render live.
helm template my-release ./charts/efe-stack/charts/efe-meta-swarm \
  --include-crds=false \
  --values charts/efe-stack/charts/efe-meta-swarm/tests/scenarios/kovant-3p.values.yaml \
  &gt; /tmp/rendered.yaml

# Diff against the golden.
diff -u charts/efe-stack/charts/efe-meta-swarm/tests/golden/kovant-3p.yaml \
        /tmp/rendered.yaml
```

A change in any chart template that affects the rendered output breaks the diff. Updating the golden requires an explicit `make update-golden` step (which regenerates the file in-place ; the PR review then inspects the diff).

26.2 Scenario values overrides

`tests/scenarios/kovant-3p.values.yaml` : ``yaml enabled: true image: tag: "0.2.0-snapshot" ``

`tests/scenarios/kovant-disabled.values.yaml` : ``yaml enabled: false ``

`tests/scenarios/kovant-network-policy.values.yaml` : ``yaml enabled: true image: tag: "0.2.0-snapshot" networkPolicy: enabled: true ``

`tests/scenarios/kovant-service-monitor.values.yaml`:``yaml enabled: true image: tag: "0.2.0-snapshot" metrics: serviceMonitor: enabled: true ``

26.3 Update protocol

- Local : `make update-golden` (a target in the root `Makefile`).
- The target re-runs `helm template` for every scenario and

overwrites the corresponding golden file.

- The PR diff is the proof the change is intentional ; reviewers

sign off.

27. CI integration

27.1 `.github/workflows/helm.yml` (or equivalent)

```
name: helm-meta-swarm

on:
  pull_request:
    paths:
      - 'charts/efe-stack/charts/efe-meta-swarm/**'
      - 'charts/efe-stack/Chart.yaml'
      - 'charts/efe-stack/values.yaml'

jobs:
  lint-template-golden:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: install helm
        uses: azure/setup-helm@v4
        with:
          version: '3.13.3'

      - name: install kubeconform
        run: |
          curl -L https://github.com/yannh/kubeconform/releases/download/v0.6.4/kubeconform-linux-amd64.t ■
    ar.gz \
            | tar xz
          sudo mv kubeconform /usr/local/bin/

      - name: helm dependency build
        run: |
          helm dependency build charts/efe-stack

      - name: helm lint
        run: |
          helm lint charts/efe-stack/charts/efe-meta-swarm \
            --values charts/efe-stack/charts/efe-meta-swarm/values.yaml

      - name: helm template + kubeconform
        run: |
          helm template tst charts/efe-stack/charts/efe-meta-swarm \
            -f charts/efe-stack/charts/efe-meta-swarm/tests/scenarios/kovant-3p.values.yaml \
            | kubeconform -strict -summary -kubernetes-version 1.28.0 \
                          -schema-location default \
                          -schema-location 'https://raw.githubusercontent.com/datreeio/CRDs-catalog/main/ ■
    {{ .Group }}/{{ .ResourceKind }}_{{ .ResourceAPIVersion }}.json'

      - name: golden snapshot diff
        run: |
          for scenario in kovant-3p kovant-disabled kovant-network-policy kovant-service-monitor ; do
            helm template tst charts/efe-stack/charts/efe-meta-swarm \
              -f charts/efe-stack/charts/efe-meta-swarm/tests/scenarios/${scenario}.values.yaml \
              &gt; /tmp/${scenario}.yaml
            diff -u charts/efe-stack/charts/efe-meta-swarm/tests/golden/${scenario}.yaml \
                    /tmp/${scenario}.yaml
          done

      - name: round-trip manifest YAML through Rust validator
        run: |
          helm template tst charts/efe-stack/charts/efe-meta-swarm \
            -f charts/efe-stack/charts/efe-meta-swarm/tests/scenarios/kovant-3p.values.yaml \
            | yq 'select(.kind == "ConfigMap") | .data["manifest.yaml"]' \
            &gt; /tmp/manifest.yaml
          cd crates/efe-meta-swarm-server
          cargo test --test manifest_roundtrip -- --manifest-path /tmp/manifest.yaml
```

The last step round-trips the rendered manifest through PR #8's `manifest::load_and_validate` to assert the chart and the Rust validator agree on the schema.

27.2 `chart-testing` (`ct`) integration

For Phase-3 follow-up : `ct lint-and-install --target-branch main` runs `helm install` + `helm test` against an in-CI `kind` cluster.

28. Multi-namespace + multi-tenant considerations

- Each release lives in one namespace ; the chart never reaches

across namespaces.

- `global.tenant` label is propagated by `_helpers.tpl::labels` so

multi-tenant clusters can `kubectl get pods -l efe.io/tenant=foo`.

- Multiple parallel installs (e.g. one tenant per namespace) are

supported : each release gets its own ConfigMap + Services + Deployments via `Release.Name` prefixing.

29. TLS bootstrapping (cert-manager) -- Phase-2 placeholders

PR #16 will wire mTLS via cert-manager-issued `Issuer` + `Certificate`. PR #13 leaves the env vars in place (PR #6) :

```
controller:
  config:
    tlsRequired: false
    # When PR #16 lands :
    # tls:
    #   issuer:
    #     name: efe-meta-swarm-issuer
    #     kind: Issuer
    #   certSecretName: efe-meta-swarm-tls
```

For Phase 2 the binaries skip the TLS path (PR #6 has `tls_required: false` default).

30. Metrics integration (Prometheus Operator)

30.1 Manual scraping (pre-Operator)

The Deployment pods are annotated with the `prometheus.io/*` set so the legacy `kubernetes_sd_configs` based Prometheus discovery picks them up automatically. This is the lowest-friction path for clusters without Prometheus Operator.

30.2 `ServiceMonitor` (Prometheus Operator)

When `metrics.serviceMonitor.enabled: true`, the chart renders a `ServiceMonitor` (§ 18) that the Operator turns into the same scrape config but with `kubernetes_sd_configs` driven by the `Service` selector. Cleaner ; relabelings expose `role` and `participant_id` as Prometheus labels.

30.3 Per-binary metric series

- Controller exposes the full `efe_meta_swarm_*` series (PR #11 + PR #12).
- Participants expose the same `efe_meta_swarm_reload_*` series

(PR #12) plus future `efe_meta_participant_*` once PR #17 / PR #18 wire them.

31. Logging / tracing wiring

- Pods write JSON logs to stdout (PR #6's `tracing_subscriber` json

formatter).

- The umbrella's `observability` sub-chart (Loki / Promtail) picks

them up via the standard `kubernetes_sd_configs` path.

- No extra wiring in PR #13 beyond ensuring `RUST_LOG` env var is

accepted (it already is via PR #6).

- For OpenTelemetry traces : PR #19 (out of scope here) will add

`OTEL_EXPORTER_OTLP_ENDPOINT` env var ; the chart should template it conditionally when an `otel:` block is added to values.

32. K8s DNS discovery -- how the controller finds participants

Per AD7 : the chart computes `EFE_META_SWARM_PARTICIPANTS` from `values.participants[*]` at template time :

```
EFE_META_SWARM_PARTICIPANTS=
  supplier-scoring={release}-efe-meta-swarm-participant-supplier-scoring.{ns}.svc:50091,
  contract-risk={release}-efe-meta-swarm-participant-contract-risk.{ns}.svc:50092,
  spend-categorization={release}-efe-meta-swarm-participant-spend-categorization.{ns}.svc:50093
```

The controller's PR #6 `parse_participants` consumes this verbatim ; its existing `DiscoveryMode::StaticLocalhost` happily resolves `<svc>.<ns>.svc` via the in-pod resolv.conf.

When PR #18 ships a real `DiscoveryMode::K8sDns` (watches `Endpoints` objects), the chart will template `EFE_META_SWARM_DISCOVERY=k8s-dns` and drop the static env var.

33. Concurrency / lifecycle invariants

33.1 What a `helm install` does, ordered

1. `ServiceAccount` (controller + N participants) -- no dependencies.
2. `ConfigMap` -- name resolved at template time, no dependencies.
3. `Service` (controller + N participants) -- Service name resolved,

no targets yet but ready to bind.

1. `Deployment` (controller + N participants) -- pods spawn, mount

ConfigMap, bind to gRPC ports.

1. `PodDisruptionBudget` (controller) -- declarative ; activates as

soon as pods exist.

1. (Optional) `NetworkPolicy`, `ServiceMonitor`.
2. `Pod` (helm test, hook-driven, not part of the install).

K8s API doesn't guarantee `ServiceAccount` is created before `Deployment` references it ; the API server retries the Deployment's pod creation until the SA resolves. In practice this is a < 1 s delay.

33.2 What a `helm upgrade` does

If only the `ConfigMap` mutates :

- `kubectl get configmap` shows new content.
- Pods are NOT restarted (AD4 -- no `checksum/config`).
- Operator MUST follow with `kill -HUP 1` per pod.

If the image tag changes :

- `Deployment.spec.template.spec.containers[0].image` mutates.
- K8s rolling-restarts pods.
- Pods come up with the new image AND the current ConfigMap content

(re-read at boot).

If a participant is added to `values.participants[]` :

- New `Deployment` + `Service` + `ServiceAccount` resources are

created.

- `ConfigMap` is mutated to include the new row.
- The controller's `EFE_META_SWARM_PARTICIPANTS` env var changes

-> triggers a `Deployment` rolling restart (which the operator chose, by editing the new entry).

- Operator SIGHUPs everything to pick up the new manifest.

If a participant is removed :

- `Deployment` + `Service` + `ServiceAccount` resources are

garbage-collected by Helm.

- `ConfigMap` mutates.
- Controller's env var changes -> rolling restart.
- Pods of the removed participant are gracefully terminated.

33.3 What a `helm rollback` does

- Reverts every resource to the previous revision.
- Pods are restarted (image, env vars, ConfigMap all snap back).
- Operator should re-issue any SIGHUPs if the rollback target's

manifest differed semantically from current.

34. Threat-model coverage (T22 / T23 / T24)

Threat	Severity	PR #13 mitigation
T22 (Secret in `values.yaml` committed to git)	SEV-2	`values.yaml` has NO secret fields ; TLS material (when PR #16 lands) sourced from `Secret`s, not values ; readme + values-prod.yaml example warn against secrets in values files ; consider Helm Secrets / sealed-secrets for future fields
T23 (NetworkPolicy bypass -- east-west traffic leaks)	SEV-2	Default-deny NetworkPolicy ships (opt-in but recommended) ; allow-list is explicit ; the runbook documents the CNI compatibility requirement (Calico / Cilium) ; future PR may add a `policyEngine` admission webhook
T24 (Floating image tag / supply-chain)	SEV-2	`values.schema.json` enforces `image.tag` non-empty (AD17) ; runbook warns against `:latest` ; Phase-3 will add digest pinning via cosign

34.1 What PR #13 does NOT defend against (deferred)

Threat	Where	When
Malicious image push to the registry (image substitution before tag pin)	Registry admission	Phase 3 (cosign + Sigstore)
Compromised K8s API server	Out of scope	Cluster ops
Operator with `secrets/get` (e.g. `kubectl get secret efe-meta-swarm-tls`)	RBAC at the cluster level	Cluster ops
Side-channel info leak via `kubectl logs` (PII in tracing)	Application layer	Phase 3 (structured logging)

35. Tests strategy

35.1 Helm lint

```
helm lint charts/efe-stack/charts/efe-meta-swarm \
  -f charts/efe-stack/charts/efe-meta-swarm/values.yaml
```

Must exit 0.

35.2 Helm template + kubeconform

For each scenario in `tests/scenarios/*.values.yaml` :

```
helm template t charts/efe-stack/charts/efe-meta-swarm \
  -f charts/efe-stack/charts/efe-meta-swarm/tests/scenarios/&lt;scen&gt;.values.yaml \
  | kubeconform -strict -kubernetes-version 1.28.0
```

Must exit 0.

35.3 Golden diff

```
diff -u \
  charts/efe-stack/charts/efe-meta-swarm/tests/golden/&lt;scen&gt;.yaml \
  /tmp/&lt;scen&gt;.rendered.yaml
```

Must be empty.

35.4 Manifest YAML round-trip

```
helm template t charts/efe-stack/charts/efe-meta-swarm -f ... \
  | yq 'select(.kind == "ConfigMap") | .data["manifest.yaml"]' \
  &gt; /tmp/manifest.yaml

cargo test -p efe-meta-swarm-server --test manifest_roundtrip -- \
  --manifest-path /tmp/manifest.yaml
```

The Rust test (`manifest_roundtrip.rs`, new in PR #13) calls `efe_meta_swarm_server::manifest::load_and_validate(&path)` and asserts `Ok(_)` for every scenario.

35.5 `helm test` connection probe (in `kind` cluster -- Phase 3 stretch)

```
kind create cluster --name efe-pr13
helm install efe-stack ./charts/efe-stack -n efe --create-namespace \
  --set efe-meta-swarm.enabled=true \
  --set efe-meta-swarm.image.tag=0.2.0-test
helm test efe-stack -n efe
```

The probe Pod runs `grpcurl` against `MetaSwarmService::Health` ; expected exit 0 + `status: SERVING` in the response.

35.6 Test matrix

Scenario	Lint	Template	Kubeconform	Golden diff	Round-trip	Helm test
`kovant-3p`	[v]	[v]	[v]	[v]	[v]	[v] (kind)
`kovant-disabled`	[v]	[v]	[v]	[v]	n/a (no ConfigMap)	n/a
`kovant-network-policy`	[v]	[v]	[v]	[v]	[v]	[v] (kind)
`kovant-service-monitor`	[v]	[v]	[v] (CRD-aware)	[v]	[v]	n/a

36. Acceptance criteria (AC1-AC30)

PR #13 is mergeable iff **all** :

#	Criterion	Verification
AC1	`helm lint` clean on default values	CI
AC2	`helm template` succeeds for all 4 scenarios	CI

#	Criterion	Verification
AC3	`kubeconform -strict` validates all rendered resources	CI
AC4	Golden diff exits 0 for `kovant-3p`	CI
AC5	Golden diff exits 0 for `kovant-disabled` (empty render)	CI
AC6	Golden diff exits 0 for `kovant-network-policy`	CI
AC7	Golden diff exits 0 for `kovant-service-monitor`	CI
AC8	`values.schema.json` rejects missing `image.tag`	manual + CI
AC9	`values.schema.json` rejects `kappa = 1.5` (out of (0, 1))	manual + CI
AC10	`values.schema.json` rejects empty `participants` array	manual + CI
AC11	`values.schema.json` rejects `controller.replicaCount = 2`	manual + CI
AC12	Rendered ConfigMap's `manifest.yaml` round-trips through `manifest::load_and_validate`	CI
AC13	`enabled: false` produces 0 resources from the sub-chart	CI
AC14	`enabled: true` produces controller Deployment + Service + SA + PDB + ConfigMap + N participant {Dep, Svc, SA}	CI
AC15	Controller's `EFE_META_SWARM_PARTICIPANTS` env var is `id1=svc.ns.svc:port,...` (validates against `lookup_endpoints` in PR #8)	CI
AC16	Both controller and participant Deployments mount the ConfigMap at `/etc/efe-meta-swarm/manifest.yaml`	CI
AC17	`EFE_META_SWARM_MANIFEST=/etc/efe-meta-swarm/manifest.yaml` env var present on every meta-swarm pod	CI
AC18	No `checksum/config` annotation on any Deployment pod template (AD4)	CI grep
AC19	`seccompProfile: RuntimeDefault` on every pod	CI grep
AC20	`runAsNonRoot: true` + `runAsUser: 65532` on every pod and container	CI grep
AC21	`automountServiceAccountToken: false` on every pod	CI grep
AC22	`PodDisruptionBudget` has `maxUnavailable: 0` (when enabled, the default)	CI
AC23	`NetworkPolicy` deny-all + allow-controller-ingress + allow-controller-egress + allow-participants-ingress render correctly when enabled	CI
AC24	`ServiceMonitor` references the controller AND participant Services when enabled	CI
AC25	`templates/tests/connection-test.yaml` renders a `helm.sh/hook: test` Pod	CI

#	Criterion	Verification
AC26	Parent `charts/efe-stack/Chart.yaml` lists `efe-meta-swarm` as a dependency with `condition: efe-meta-swarm.enabled`	CI grep
AC27	Parent `charts/efe-stack/values.yaml` has `efe-meta-swarm: { enabled: false }` default	CI grep
AC28	`efe-server` Deployment template injects the `EFE_META_SWARM_*` env vars when `efe-meta-swarm.enabled = true`	CI
AC29	Documentation : `docs/EFE_A1_PHASE2_PR13_ARCHITECTURE.md` referenced from `README.md` + `NOTES.txt`	manual
AC30	Threat-model coverage : T22 / T23 / T24 each have at least one mitigation in the chart or runbook	manual

37. Risks + mitigations

ID	Risk	L/I	Mitigation
PR13-R1	Helm 3 only -- operators on Helm 2 break	L / H	`Chart.yaml::apiVersion: v2` (Helm 2 doesn't understand) ; documented in README
PR13-R2	CNI doesn't enforce `NetworkPolicy` -- render is no-op	M / M	Opt-in (AD14) ; README documents the CNI compatibility list (Calico, Cilium, Weave-policy)
PR13-R3	`prometheus-operator` CRDs missing -- `ServiceMonitor` install fails	M / M	Opt-in (AD15) ; the chart's `if` guard prevents render when off ; documented
PR13-R4	Operator forgets `kill -HUP 1` after `helm upgrade` and wonders why config didn't take	H / L	NOTES.txt prints the SIGHUP commands ; runbook lifts them ; future PR #18 may automate via in-cluster watcher
PR13-R5	`values.schema.json` rejects a valid config because of a too-strict pattern	L / M	Patterns reviewed against PR #6 / PR #8 validators ; schema unit-tested with edge cases
PR13-R6	Image tag pinning forces operator override -- broken `helm install` for newcomers	M / L	README ships a working `tag` in the kovant-3p scenario ; defaults disabled so newcomers don't hit it
PR13-R7	Golden snapshots churn on chart-templating-engine version bump	M / M	Pinned Helm 3.13.3 in CI ; update protocol documented ; CI uses the same binary version as developers
PR13-R8	`range` over `participants` generates duplicate name on YAML serializer bugs	L / H	Templated names include `id` (guaranteed unique by schema's `pattern`) ; tested in golden
PR13-R9	ConfigMap exceeds 1 MB K8s limit on N=64 participants	LL / M	Schema caps `maxItems: 64` ; conservative manifest size (~ 200 B / participant) x 64 = 13 KB, comfortably under
PR13-R10	Operator on K8s 1.27 -- `policy/v1 PodDisruptionBudget` not yet GA	LL / L	`kubeVersion: ">=1.28.0-0"` rejects older clusters ; runbook documents
PR13-R11	Sub-chart values key collision with sibling chart (e.g. `image.tag` overlap)	L / M	Sub-chart values are namespaced under `efe-meta-swarm:` in umbrella values ; no collision possible
PR13-R12	Cross-chart dependency (`efeServerWiring`) drifts from parent's expectation	M / M	Documented in § 7.3 ; CI runs an integration test that renders the umbrella + asserts `efe-server` Deployment has the right env vars

38. Rollback strategy

If PR #13 lands and breaks production :

1. **Operator level** : `helm rollback efe-stack <previous-rev>`.

Reverts every resource to the pre-PR-#13 state in seconds.

1. **Per-commit revert** : C6 -> C5 -> ... -> C1. Each commit is

independently `helm lint`-green.

1. **Wholesale revert** : `git revert <merge-commit>` ; the sub-chart

vanishes ; the umbrella's `Chart.yaml` dependency entry is gone ; the chart's `values.yaml` block is removed.

1. **Hotfix** : if only golden tests flake, mark them `# golden-skip:

true` ; production-facing templates are unaffected.

Total Time To Mitigate :

- `helm rollback` : ~ 30 s.
- Code revert + redeploy : ~ 5 min.

39. Sequence diagrams (ASCII)

39.1 `helm install` (sub-chart enabled)

```
operator                 helm                              K8s API
   |                       |                                 |
   | helm install efe-stack -f values-prod.yaml              |
   |----------------------■|                                 |
   |                       | template + lint + schema-validate
   |                       | render every resource           |
   |                       | POST ServiceAccount x 4 -------■| -&gt; created
   |                       | POST ConfigMap   --------------■| -&gt; created
   |                       | POST Service x 4 --------------■| -&gt; created
   |                       | POST Deployment x 4 -----------■| -&gt; created
   |                       |                                 |   v pod scheduler binds to nodes
   |                       |                                 |   v pod scheduler pulls image
   |                       |                                 |   v kubelet starts containers
   |                       |                                 |   v binaries call manifest::load
   |                       |                                 |   v controller dials participants
   |                       |                                 |   v all probes go green
   |  NOTES.txt printed    |                                 |
   |■----------------------|                                 |
```

39.2 `helm upgrade` with manifest-only change (SIGHUP path)

```
operator               helm                   K8s API           pods
   |                     |                      |                |
   | helm upgrade ...    |                      |                |
   |--------------------■|                      |                |
   |                     | diff resources       |                |
   |                     | ConfigMap mutated    |                |
   |                     | (no Deployment/Pod   |                |
   |                     |  template change)    |                |
   |                     | PATCH ConfigMap -----■|                |
   |                     |                      | -&gt; committed    |
   |                     |                      |                | (pods unaware ; mounted RO copy is up ■
    dated via inotify, but binaries don't watch)
   |                     |                      |                |
   | kubectl exec ... -- kill -HUP 1            |                |
   |-------------------------------------------------------------■|
   |                     |                      |                | SIGHUP -&gt; apply_reload_from_disk
   |                     |                      |                | reload_gen++
   |                     |                      |                |
   | grpcurl Health      |                      |                |
   |-------------------------------------------------------------■|
   | ■------------------------------------------ { reloadGen: 1 } |
```

39.3 `helm test`

```
operator               helm                   K8s API           pods
   |                     |                      |                |
   | helm test efe-stack |                      |                |
   |--------------------■|                      |                |
   |                     | apply "hook: test" Pod                |
   |                     | POST Pod ------------■|                |
   |                     |                      | -&gt; grpcurl runs |
   |                     |                      |    [v] Health    |
   |                     |                      | -&gt; Pod exits 0  |
   |                     |                      |                |
   | TEST SUITE: PASS    |                      |                |
   |■--------------------|                      |                |
```

40. Reviewer checklist

```
## Reviewer checklist -- PR #13

### Chart correctness
- [ ] `helm lint` exit 0 with default values + every scenario override
- [ ] `helm template` produces zero meta-swarm resources when `enabled: false` (AD2)
- [ ] No `checksum/config` annotation on any Deployment (AD4)
- [ ] `seccompProfile: RuntimeDefault` everywhere (AD8)
- [ ] `runAsNonRoot: true` + drop ALL caps + readOnlyRootFilesystem (AD8 / AD9)
- [ ] `automountServiceAccountToken: false` on every pod (AD20)

### Networking
- [ ] Controller Service: ClusterIP, appProtocol=grpc, ports 50080 / 9091
- [ ] Participant Services rendered per `range` entry
- [ ] `EFE_META_SWARM_PARTICIPANTS` env var properly templated (AD7 / § 32)
- [ ] NetworkPolicy deny-all + targeted allow when enabled (AD14)

### Validation
- [ ] `values.schema.json` rejects empty `image.tag` (AC8)
- [ ] Schema rejects out-of-range `kappa` (AC9)
- [ ] Schema rejects multi-replica controller (AC11)
- [ ] Manifest YAML round-trips through `manifest::load_and_validate` (AC12)

### Reload
- [ ] Both controller and participants mount the ConfigMap at the same path (AC16)
- [ ] `EFE_META_SWARM_MANIFEST` env var present and correct (AC17)
- [ ] No `checksum/config` annotation (AC18)
- [ ] NOTES.txt prints the SIGHUP incantation
- [ ] Runbook lifts the SIGHUP-after-helm-upgrade workflow (§ 25.1)

### Observability
- [ ] Prometheus annotations present on every pod
- [ ] `ServiceMonitor` (when enabled) covers controller + participants (AC24)
- [ ] Relabelings expose `role` + `participant_id` labels

### Threat model
- [ ] T22 / T23 / T24 each have a mitigation in the chart or runbook (§ 34 / AC30)
- [ ] No secrets in `values.yaml`
- [ ] Image tag pinning enforced

### Cross-PR consistency
- [ ] Parent chart's `Chart.yaml` registers the dep with `condition: efe-meta-swarm.enabled`
- [ ] Parent chart's `values.yaml` defaults to `efe-meta-swarm: { enabled: false }`
- [ ] Parent `efe-server` deployment.yaml templates the `EFE_META_SWARM_*` env vars
      iff `efe-meta-swarm.enabled = true` (PR #11 hook)
- [ ] Golden snapshots check in to git (CI diff)
```

41. Operator runbook

41.1 Initial install (dev / staging)

```
# 1. Pin the image tag (no :latest).
$ cat &gt; values-dev.yaml &lt;&lt;EOF
efe-meta-swarm:
  enabled: true
  image:
    tag: "0.2.0"
  controller:
    config:
      reflectionEnabled: true
  metrics:
    serviceMonitor:
      enabled: true
EOF

# 2. Install.
$ helm install efe-stack ./charts/efe-stack \
    --namespace efe --create-namespace \
    -f values-dev.yaml

# 3. Verify pods came up.
$ kubectl -n efe get pods -l efe.io/meta-swarm-role

# 4. Run the connection probe.
$ helm test efe-stack -n efe
```

41.2 Production install

```
# values-prod.yaml -- same as dev but locked.
$ cat &gt; values-prod.yaml &lt;&lt;EOF
efe-meta-swarm:
  enabled: true
  image:
    tag: "0.2.0"           # explicit, no :latest
    pullPolicy: IfNotPresent
  manifest:
    iteration:
      max_iters: 100
      tolerance: 0.0001
      alpha_default: 0.5
      coupling_gamma: 0.2
      max_wall_ms: 15000
  controller:
    resources:
      requests: {cpu: 1,   memory: 1Gi}
      limits:   {cpu: 4,   memory: 4Gi}
  participants:
    - id: supplier-scoring
      dim: 16
      kappa: 0.6
      seed: 11
      resources:
        requests: {cpu: 500m, memory: 512Mi}
        limits:   {cpu: 2,    memory: 2Gi}
    - id: contract-risk
      dim: 16
      kappa: 0.7
      seed: 22
      resources:
        requests: {cpu: 500m, memory: 512Mi}
        limits:   {cpu: 2,    memory: 2Gi}
    - id: spend-categorization
      dim: 16
      kappa: 0.5
      seed: 33
      resources:
        requests: {cpu: 500m, memory: 512Mi}
        limits:   {cpu: 2,    memory: 2Gi}
  networkPolicy:
    enabled: true
  metrics:
    serviceMonitor:
      enabled: true
EOF

$ helm install efe-stack ./charts/efe-stack \
    --namespace efe \
    -f values-prod.yaml
```

41.3 Hot reload (Phase 2 SIGHUP path)

```
# 1. Edit values-prod.yaml -- e.g. bump alpha_default to 0.6.
$ vim values-prod.yaml

# 2. helm upgrade -- mutates ConfigMap, leaves pods running.
$ helm upgrade efe-stack ./charts/efe-stack \
    --namespace efe \
    -f values-prod.yaml

# 3. SIGHUP every pod.
$ kubectl -n efe exec deploy/efe-stack-efe-meta-swarm-controller -- kill -HUP 1
$ for id in supplier-scoring contract-risk spend-categorization ; do
    kubectl -n efe exec deploy/efe-stack-efe-meta-swarm-participant-$id -- kill -HUP 1
  done

# 4. Verify.
$ kubectl -n efe port-forward svc/efe-stack-efe-meta-swarm-controller 50080:50080 &amp;
$ grpcurl -plaintext -d '{}' localhost:50080 efe.v1.meta_swarm.MetaSwarmService/Health \
  | jq '.reloadGen'
```

41.4 Image upgrade (rolling restart path)

```
# 1. Bump image tag in values.
$ sed -i 's/tag: "0.2.0"/tag: "0.2.1"/' values-prod.yaml

# 2. helm upgrade -- Deployment pod templates mutate, k8s rolls.
$ helm upgrade efe-stack ./charts/efe-stack -n efe -f values-prod.yaml

# 3. Watch the rolling restart.
$ kubectl -n efe rollout status deploy/efe-stack-efe-meta-swarm-controller
$ for id in supplier-scoring contract-risk spend-categorization ; do
    kubectl -n efe rollout status deploy/efe-stack-efe-meta-swarm-participant-$id
  done

# 4. helm test to confirm.
$ helm test efe-stack -n efe
```

41.5 Troubleshooting

Symptom	Likely cause	Fix
`helm install` fails with `kappa must be in (0, 1)`	values.yaml typo	Edit values, re-install
Pod stuck in `ContainerCreating`	image pull failure	`kubectl describe pod ...` -> check pullSecrets / registry creds
Pod CrashLoopBackOff	manifest validation failed at boot	`kubectl logs ...` -- `validate_manifest` error message
`helm test` fails with `connection refused`	controller not ready	wait + retry ; check liveness probe
`Health.reloadGen` stays at 0 after edit	forgot `kill -HUP 1`	issue SIGHUP per pod (runbook § 41.3)
Pods crashing after SIGHUP	manifest reload rejected (drift)	`kubectl logs ...` -- `validate_reload_compatibility` rejection reason
`RELOADING (3)` status sticks	reload swap never completed (controller panic mid-swap, unlikely with PR #12's safety)	`kubectl logs ...` ; if needed `helm rollback` to previous revision
ServiceMonitor not scraped	`prometheus-operator` not installed	install operator or set `metrics.serviceMonitor.enabled: false`

42. Open questions

ID	Question	Status
OQ-PR13-1	Should the chart ship a default `policyEngine: kyverno` admission policy for image-tag enforcement ?	Defer to PR #20 / supply chain story
OQ-PR13-2	Should participants use a `StatefulSet` for forward-compat with Phase-3 sharded state ?	NO for Phase 2 -- switching is mechanical when state matters
OQ-PR13-3	Should the chart expose `vpa:` block for VPA integration ?	NO -- VPA is out of Phase-2 scope ; can be added in a values overlay
OQ-PR13-4	Helm SemVer policy : major bump on incompatible values changes ?	YES -- `version: x.y.z` follows SemVer, `Chart.yaml::version` bumps on every PR
OQ-PR13-5	Should we add a `topologySpreadConstraints` block to participants ?	YES in Phase 3 when participants replicate ; for Phase 2 (`replicas: 1`) it's a no-op
OQ-PR13-6	Should the umbrella chart's `values-prod.yaml` ship pinned image digests instead of tags ?	Phase 3 -- needs cosign + Sigstore wiring
OQ-PR13-7	Should the `helm test` Pod use `grpcurl` from the same image as the controller (single image, no extra dep) ?	NO -- `grpcurl` is not bundled in the EFE image ; adding it bloats the production image
OQ-PR13-8	Should we ship a `Job` for one-shot SIGHUP-all (a chart-internal `helm sighup` action) ?	NO for Phase 2 -- runbook covers the manual workflow ; PR #18 may automate

43. Appendix : full file inventory

43.1 Files added (17)

```
charts/efe-stack/charts/efe-meta-swarm/Chart.yaml                          NEW  ~ 35 LOC
charts/efe-stack/charts/efe-meta-swarm/values.yaml                         NEW  ~ 220 LOC
charts/efe-stack/charts/efe-meta-swarm/values.schema.json                  NEW  ~ 230 LOC
charts/efe-stack/charts/efe-meta-swarm/README.md                           NEW  ~ 180 LOC
charts/efe-stack/charts/efe-meta-swarm/templates/_helpers.tpl              NEW  ~ 130 LOC
charts/efe-stack/charts/efe-meta-swarm/templates/NOTES.txt                 NEW  ~ 60 LOC
charts/efe-stack/charts/efe-meta-swarm/templates/configmap-manifest.yaml   NEW  ~ 80 LOC
charts/efe-stack/charts/efe-meta-swarm/templates/controller-deployment.yaml NEW ~ 180 LOC
charts/efe-stack/charts/efe-meta-swarm/templates/controller-service.yaml   NEW  ~ 35 LOC
charts/efe-stack/charts/efe-meta-swarm/templates/controller-serviceaccount.yaml NEW ~ 20 LOC
charts/efe-stack/charts/efe-meta-swarm/templates/pdb-controller.yaml       NEW  ~ 25 LOC
charts/efe-stack/charts/efe-meta-swarm/templates/participant-deployment.yaml NEW ~ 200 LOC
charts/efe-stack/charts/efe-meta-swarm/templates/participant-service.yaml  NEW  ~ 45 LOC
charts/efe-stack/charts/efe-meta-swarm/templates/participant-serviceaccount.yaml NEW ~ 30 LOC
charts/efe-stack/charts/efe-meta-swarm/templates/servicemonitor.yaml       NEW  ~ 60 LOC
charts/efe-stack/charts/efe-meta-swarm/templates/networkpolicy.yaml        NEW  ~ 100 LOC
charts/efe-stack/charts/efe-meta-swarm/templates/tests/connection-test.yaml NEW ~ 60 LOC
charts/efe-stack/charts/efe-meta-swarm/tests/scenarios/kovant-3p.values.yaml NEW ~ 5 LOC
charts/efe-stack/charts/efe-meta-swarm/tests/scenarios/kovant-disabled.values.yaml NEW ~ 2 LOC
charts/efe-stack/charts/efe-meta-swarm/tests/scenarios/kovant-network-policy.values.yaml NEW ~ 7 LOC
charts/efe-stack/charts/efe-meta-swarm/tests/scenarios/kovant-service-monitor.values.yaml NEW ~ 7 LOC
charts/efe-stack/charts/efe-meta-swarm/tests/golden/kovant-3p.yaml         NEW  ~ 900 LOC
charts/efe-stack/charts/efe-meta-swarm/tests/golden/kovant-disabled.yaml   NEW  ~ 0 LOC
charts/efe-stack/charts/efe-meta-swarm/tests/golden/kovant-network-policy.yaml NEW ~ 1 000 LOC
charts/efe-stack/charts/efe-meta-swarm/tests/golden/kovant-service-monitor.yaml NEW ~ 1 050 LOC

docs/EFE_A1_PHASE2_PR13_ARCHITECTURE.md                                    NEW  this file
crates/efe-meta-swarm-server/tests/manifest_roundtrip.rs                   NEW  ~ 60 LOC (round-trip test ■
     consumer for CI)
```

43.2 Files modified (4)

```
charts/efe-stack/Chart.yaml                                          MODIFIED  +6 LOC : sub-chart d ■
    ependency
charts/efe-stack/values.yaml                                         MODIFIED  +25 LOC : `efe-meta- ■
    swarm:` defaults block
charts/efe-stack/charts/efe-server/templates/deployment.yaml         MODIFIED  +15 LOC : EFE_META_S ■
    WARM_* env vars when sub-chart enabled
.github/workflows/helm.yml (or chart-testing workflow)               MODIFIED  +30 LOC : golden-dif ■
    f + kubeconform + round-trip steps
```

43.3 LOC tally

Category	LOC
New chart templates (production YAML)	~ 1 270
New values + schema	~ 450
New `_helpers.tpl`	~ 130
New README + NOTES.txt	~ 240
New scenario values	~ 21
New golden snapshots (auto-generated)	~ 2 950
New Rust round-trip test	~ 60
New CI workflow steps	~ 30
Modified parent chart wiring	~ 46
Total net new	**~ 5 200 LOC** (most of which is auto-generated golden YAML)

Production-relevant net new (excluding generated goldens) : **~ 2 250 LOC** of YAML / Go templates / JSON schema / shell.

44. Appendix : example overrides

44.1 Kovant dev cluster

```yaml
efe-meta-swarm:
  enabled: true
  image:
    tag: "0.2.0-rc.1"
    pullPolicy: Always
  controller:
    config:
      reflectionEnabled: true     # enable grpcurl reflection in dev
  metrics:
    serviceMonitor:
      enabled: false              # legacy Prometheus discovery instead
```

44.2 Kovant staging cluster

```yaml
efe-meta-swarm:
  enabled: true
  image:
    tag: "0.2.0"
  controller:
    resources:
      requests: {cpu: 1, memory: 1Gi}
      limits:   {cpu: 2, memory: 2Gi}
  metrics:
    serviceMonitor:
      enabled: true
      interval: 15s               # tighter scrape in staging for faster feedback
```

44.3 Kovant production cluster

```
efe-meta-swarm:
  enabled: true
  image:
    tag: "0.2.0"                  # locked
    pullPolicy: IfNotPresent
  controller:
    resources:
      requests: {cpu: 1,   memory: 1Gi}
      limits:   {cpu: 4,   memory: 4Gi}
    podDisruptionBudget:
      enabled: true
      maxUnavailable: 0
  participants:
    - id: supplier-scoring
      dim: 32                     # production dim larger than dev
      kappa: 0.62
      seed: 11
      resources:
        requests: {cpu: 1, memory: 1Gi}
        limits:   {cpu: 2, memory: 2Gi}
    - id: contract-risk
      dim: 32
      kappa: 0.68
      seed: 22
      resources:
        requests: {cpu: 1, memory: 1Gi}
        limits:   {cpu: 2, memory: 2Gi}
    - id: spend-categorization
      dim: 32
      kappa: 0.55
      seed: 33
      resources:
        requests: {cpu: 1, memory: 1Gi}
        limits:   {cpu: 2, memory: 2Gi}
  networkPolicy:
    enabled: true                 # always-on in prod
    ingressNamespaceSelector:
      matchLabels:
        kubernetes.io/metadata.name: efe-prod
    prometheusNamespaceSelector:
      matchLabels:
        kubernetes.io/metadata.name: monitoring
  metrics:
    serviceMonitor:
      enabled: true
      labels:
        prometheus: efe-prod      # match the production Prometheus selector
```

44.4 Single-controller stress-test cluster (4 participants)

```
efe-meta-swarm:
  enabled: true
  image:
    tag: "0.2.0-stress"
  manifest:
    meta:
      dim: 64
    iteration:
      max_iters: 200
      tolerance: 1.0e-6
      max_wall_ms: 30000
  controller:
    config:
      maxInflight: 20
    resources:
      requests: {cpu: 4,  memory: 4Gi}
      limits:   {cpu: 8,  memory: 8Gi}
  participants:
    - id: stress-a
      dim: 64
      kappa: 0.65
      seed: 1
      service: {grpcPort: 50091, metricsPort: 9101}
      resources:
        requests: {cpu: 2, memory: 2Gi}
        limits:   {cpu: 4, memory: 4Gi}
    - id: stress-b
      dim: 64
      kappa: 0.70
      seed: 2
      service: {grpcPort: 50092, metricsPort: 9102}
      resources:
        requests: {cpu: 2, memory: 2Gi}
        limits:   {cpu: 4, memory: 4Gi}
    - id: stress-c
      dim: 64
      kappa: 0.55
      seed: 3
      service: {grpcPort: 50093, metricsPort: 9103}
      resources:
        requests: {cpu: 2, memory: 2Gi}
        limits:   {cpu: 4, memory: 4Gi}
    - id: stress-d
      dim: 64
      kappa: 0.60
      seed: 4
      service: {grpcPort: 50094, metricsPort: 9104}
      resources:
        requests: {cpu: 2, memory: 2Gi}
        limits:   {cpu: 4, memory: 4Gi}
```

45. Glossary

Term	Definition
AC	Acceptance Criterion
AD	Architectural Decision
L2 gate G10	Phase-2 graduation gate "K8s deployment with declarative configuration management"
Sub-chart	A Helm chart that lives under `charts/<parent>/charts/<child>/` and is referenced from the parent's `Chart.yaml::dependencies`
Umbrella chart	The parent chart (`charts/efe-stack/`) that bundles N sub-charts
Vendored sub-chart	A sub-chart whose source is committed into the parent's `charts/` directory (vs pulled from a Helm registry at install time, AD24)
values.schema.json	JSON Schema applied by Helm 3 to the merged `values.yaml` ; rejects malformed configs at install time
Golden snapshot	A reference `helm template`-rendered YAML file checked into git ; CI diffs the live render against it
NOTES.txt	Helm convention : a templated message rendered after `helm install` / `helm upgrade` ; printed to the operator's terminal
`helm test`	Helm convention : a `Pod` annotated `helm.sh/hook: test` runs probe logic after install ; `helm test <release>` executes it
PodDisruptionBudget	K8s policy/v1 : limits voluntary disruptions (node drain, scale-down) for a labelled pod set
ServiceMonitor	`prometheus-operator` CRD that tells the operator which Services to scrape for metrics
NetworkPolicy	K8s networking/v1 : deny / allow rules for pod-to-pod traffic, CNI-enforced
`appProtocol: grpc`	Service-port hint that the protocol is HTTP/2 + gRPC ; consumed by service-mesh proxies and L4 load balancers
Helm hook	A `helm.sh/hook` annotation that runs a resource at a specific lifecycle event (pre-install, post-upgrade, test, etc.)
`fullnameOverride`	Helm convention : allow operators to override the release-prefixed name (rare ; here for forward-compat)

End of architecture document. Total: ~ 2 200 lines.

Next deliverable : PR #13 code (6 commits, ~ +1 340 LOC of YAML / Go templates / JSON schema / shell + auto-generated goldens).

PR #14

Helm values + templates (production-grade)

Chapter 16 of 17 — source : `docs/EFE_A1_PHASE2_PR14_ARCHITECTURE.md` (2,563 LOC)

EFE-A1 Phase 2 -- PR #14 : Helm values + templates (production-grade)

Field	Value
PR	#14 (Sprint A1 Phase 2 / P2.5)
Branch	`efe-a1/p2-pr14-helm-values-templates`
Status	Architecture frozen -- awaiting code
Author	EFE platform team
Reviewers	2x meta-swarm + 1x sre + 1x platform-deploy + 1x sec + 1x observability
Depends on	PR #5 (proto), PR #6 (scaffold), PR #8 (controller wire), PR #9 (participant wire), PR #10 (multiprocess), PR #11 (efe-server flag plumbing), PR #12 (SIGHUP + ReloadConfig), PR #13 (Helm sub-chart skeleton)
Blocks	PR #15 (iterate dispatch metrics), PR #16 (mTLS provisioning), PR #17 (multi-controller HA / leader election), PR #18 (in-cluster `ConfigMap`-watcher)
L2 gate	G11 (operator-grade Helm chart with automated reload + observability + multi-env values + bundle weight-loader contract)
Threat IDs in scope	T22-T24 (carried from PR #13), T25 (Helm hook over-privileged RBAC), T26 (sidecar `shareProcessNamespace` /proc exposure), T27 (backup ConfigMap rotation tampering), T28 (bundle endpoint unauthenticated -- deferred to PR #16)
Roadmap reference	EFE_A1_PHASE2_ROADMAP.md § C7
CVSS bandwidth	All <= SEV-2 ; no new SEV-1 introduced

Table of contents

1. TL;DR
2. Goal + non-goals
3. Pre-flight checks
4. Branch + commit plan (8 commits)
5. Architectural decisions (AD1-AD32)
6. Topology + integration diagram (extends PR #13)
7. Layout : files added + modified under `charts/efe-stack/charts/efe-meta-swarm/`
8. Multi-environment values flavors (dev / staging / prod / canary)
9. PriorityClass + HPA placeholder
10. Per-tenant label propagation + multi-tenant slices
11. Pre-upgrade Helm hook : backup `ConfigMap` snapshot (rotating ring of 5)
12. Post-upgrade Helm hook : SIGHUP-all `Job` (automate the PR #12 runbook)
13. RBAC additions for the hook `Job`s (least-privilege)
14. Bundle weight-loader sidecar (structural placeholder ; full wire in PR #15)
15. `PrometheusRule` alerts (reload failure rate, latency, availability)
16. Grafana dashboard `ConfigMap` (sidecar import via annotation)
17. TLS bootstrap stubs (cert-manager `Issuer` + `Certificate`)
18. NetworkPolicy refinements (tenant slice + backup `Job` egress)
19. Resource sizing matrix per environment

20. Schema additions (`values.schema.json` PR #14 fields)
21. Helm hook ordering + weights
22. Operator-driven workflows : upgrade / rollback / hot reload
23. Concurrency / lifecycle invariants (extends PR #13 § 33)
24. Threat-model coverage (T22-T28)
25. Tests strategy (extends PR #13 § 35)
26. Acceptance criteria (AC1-AC38)
27. Risks + mitigations
28. Rollback strategy
29. Sequence diagrams (helm upgrade with hooks ; bundle sidecar polling)
30. Reviewer checklist
31. Operator runbook (extended : 11 scenarios)
32. Open questions
33. Appendix : full file inventory
34. Appendix : example overrides + canary slice
35. Glossary

0. TL;DR

PR #14 evolves the PR #13 **skeleton** into a **production-grade operator surface** by layering :

1. **Multi-environment values flavors** (`values-dev.yaml`,

`values-staging.yaml`, `values-prod.yaml`, `values-canary.yaml`) vendored inside the sub-chart under `examples/`. Operators `helm upgrade ... -f values-prod.yaml` ; the four flavors are curated to compose with the umbrella's `global:` block.

1. **Helm pre-upgrade hook** : a `Job` that snapshots the

manifest `ConfigMap` into a rotating ring of 5 `backup-N` `ConfigMap`s, indexed by `reload_gen`. Operator-side audit trail without touching K8s `EventList`.

1. **Helm post-upgrade hook** : a `Job` that does

`kubectl exec deploy/<controller> -- kill -HUP 1` + same for every participant -- automating the PR #12 § 25.1 runbook, removing the manual SIGHUP step.

1. **Bundle weight-loader sidecar** : a structural placeholder

`Container` next to every participant ; shares a `emptyDir` volume + `shareProcessNamespace: true` so the sidecar can SIGHUP the participant on weight bundle updates. In Phase-2 the sidecar is a `pause`-style no-op image ; PR #15 fills the real bundle-poll loop. Architecturally locked in PR #14 so PR #15 has a stable contract.

1. `PrometheusRule` **alerts** for the PR #12 reload metrics +

per-binary availability (controller / each participant). Six alerts spanning warning + critical severities.

1. **Grafana dashboard** as a `ConfigMap` with the

`grafana_dashboard: "1"` annotation so the Grafana sidecar auto-imports. Panels for reload_gen ramp, reload duration histogram, health probe success rate, RELOADING window duration.

1. `PriorityClass` for the controller (`100000`) so it

out-prioritises participants under resource pressure.

1. **Per-tenant label propagation** : every resource gets

`efe.io/tenant: <slug>` inherited from the umbrella's `global.tenant`. Enables `kubectl get ... -l efe.io/tenant=foo` in multi-tenant clusters.

1. `HorizontalPodAutoscaler` **placeholder** : `enabled: false`

scaffold for PR #17's multi-controller HA work ; renders the full HPA spec when flipped but doesn't actually scale Phase-2's single-controller (the controller `Deployment` has `replicaCount: 1` hard-locked).

1. **TLS bootstrap stubs** : cert-manager `Issuer` +

`Certificate` template, gated on `controller.config.tls.certManager.enabled`. PR #16 wires the binary side ; PR #14 ships the chart side.

1. **NetworkPolicy refinements** : explicit `allow` rule for the

`pre-upgrade-snapshot` and `post-upgrade-sighup` `Job`s' egress so they reach the API server / target pods under deny-by-default CNI.

1. **Schema additions** : 11 new top-level keys in

`values.schema.json` (priorityClass, hooks, sidecar, alerts, dashboard, tls, hpa, etc.) -- every PR #14 surface is typed and validated at install time.

PR #14 closes Phase-2 graduation gate **G11 (operator-grade Helm chart with automated reload + observability + multi-env values + bundle weight-loader contract)** and unlocks PR #15 (iterate dispatch + metrics surface), PR #16 (mTLS), PR #17 (multi-controller HA).

1. Goal + non-goals

1.1 Goal (single sentence)

> *Evolve PR #13's deployable skeleton into a **production-grade** sub-chart by layering multi-environment values flavors, pre/post Helm hooks for automated manifest backup + SIGHUP-all, a structural bundle weight-loader sidecar (Phase-3 contract locked at PR #14 time), Prometheus alerts + Grafana dashboards, cert-manager TLS bootstrap stubs, a controller* `PriorityClass`*, and per-tenant label propagation -- without breaking PR #13's* `enabled: false` *silent-by-default doctrine or PR #12's SIGHUP-driven reload semantics.*

1.2 Non-goals (deferred to later PRs)

Item	Why deferred	Target PR
Real bundle-poller binary inside the sidecar (today : `pause` no-op)	needs the bundle-endpoint contract + auth + cosign verify	PR #15
Real `MetaSwarmClient::iterate` dispatch wiring on `efe-server` side	PR #11 only wired probe ; PR #15 wires dispatch	PR #15
Real cert-manager `Issuer` + `Certificate` lifecycle (`Secret` rotation, mTLS at the gRPC layer)	needs the binary's tonic-tls integration	PR #16
Multi-controller leader election + HPA scaling logic	needs leader-lock subsystem	PR #17
In-cluster `ConfigMap`-watcher Pod (real-time SIGHUP)	PR #14's post-upgrade `Job` covers operator-driven path ; watcher is for `kubectl edit cm` cases	PR #18
OTLP collector sidecar for distributed tracing	needs global OTEL operator wiring	PR #19
Helm pre-delete hook for state quiesce	no persistent state in Phase 2	PR #20
Operator-driven `helm install --atomic` failure recovery	tooling responsibility	follow-up
Backup `ConfigMap` cross-cluster sync (DR)	infra concern	Phase 3
Bundle endpoint authentication / cosign verification	needs PKI wiring	PR #16
Per-tenant resource quotas	namespace-scope concern	Phase 3
Per-tenant NetworkPolicy isolation	service-mesh concern	Phase 3
Audit log streaming (every reload -> SIEM)	observability platform concern	PR #19
FluxCD / Argo CD reconciliation hooks	GitOps tooling, orthogonal to chart	follow-up
ClusterRole-based hooks (`kubectl exec` cluster-wide)	least-privilege says Role is enough	locked
Kustomize overlay equivalents	Helm-only is the official path	follow-up
HelmRelease (FluxCD CR) examples	tooling docs, separate PR	follow-up
Dual-stack IPv6 service support	infra concern	Phase 3

1.3 Scope summary

PR #14 ships :

1. Four environment flavors under `examples/` (dev / staging / prod / canary).
2. Pre-upgrade `Job` that snapshots `manifest` `ConfigMap` to a rotating ring of 5 backups.
3. Post-upgrade `Job` that SIGHUPs the controller + every participant.
4. RBAC `Role` + `RoleBinding` + dedicated `ServiceAccount` for the hooks (least-privilege).
5. Bundle weight-loader sidecar template (structural placeholder, off by default).
6. `PrometheusRule` template with 6 alerts (reload-failures, reload-latency, availability x3, RELOADING-stuck).
7. Grafana dashboard `ConfigMap` (auto-import via sidecar annotation).
8. `PriorityClass` for the controller (`100000`).
9. Per-tenant label propagation via `global.tenant`.
10. `HorizontalPodAutoscaler` template (disabled by default, structural for PR #17).
11. cert-manager `Issuer` + `Certificate` stubs (disabled by default).
12. NetworkPolicy egress allow for the hook `Job`s.
13. Schema additions (11 new keys + 28 sub-keys).
14. ~ 18 new `helm-unittest` assertions covering PR #14 fields.

15. 2 new scenario values files + 2 new golden snapshots (`kovant-prod.yaml`, `kovant-hooks.yaml`).
16. Operator runbook extensions (11 scenarios).

PR #14 does **not** change :

- The proto contract (PR #5).
- The Phase-1 algorithm (`MetaSwarm::iterate`) -- unchanged.
- The Rust binaries (PR #6 / #8 / #9 / #12) -- only the chart evolves.
- PR #13's existing templates (additive only, except `controller-deployment.yaml` which gains the sidecar Container behind an `if` guard).

2. Pre-flight checks

#	Check	Verifier	Pass criterion
PF1	PR #13 merged into `main` (sub-chart skeleton present)	git log	merge SHA on main
PF2	`helm lint charts/efe-stack/charts/efe-meta-swarm/` clean on `main`	CI	exit 0
PF3	`helm template tst charts/efe-stack/charts/efe-meta-swarm -f tests/scenarios/kovant-3p.values.yaml` succeeds	CI	exit 0
PF4	All 23 PR #13 `helm-unittest` tests pass on `main`	CI	green
PF5	`kubeconform -strict` clean on all 4 PR #13 golden snapshots	CI	51/51 valid
PF6	`Chart.lock` is in sync with `Chart.yaml` (PR #13 added efe-meta-swarm dep)	CI	digest matches
PF7	Parent `efe-server` Deployment renders `EFE_META_SWARM_*` env vars when `global.metaSwarm.enabled = true`	CI	5 env vars
PF8	`monitoring.coreos.com/v1` CRDs documented as opt-in (operator pre-req)	manual	README clear
PF9	`cert-manager.io/v1` CRDs documented as opt-in (operator pre-req)	manual	README clear
PF10	`kubectl` version compat 1.28-1.30	manual	kubeVersion matches
PF11	Threat model (PR #7) v1.0 + T25-T28 sec sign-off	sec	green
PF12	`helm-unittest` plugin v0.5.1 supports new assertion types we'll add (`hasDocuments`, `failedTemplate`, `contains`)	manual	docs verified

3. Branch + commit plan

3.1 Branch

```
efe-a1/p2-pr14-helm-values-templates
```

Branched from `main` after PR #13 merge SHA.

3.2 Commit-by-commit plan (8 commits, each CI-green)

#	Subject	Files	LOC ±	Verification
C1	`feat(charts): efe-meta-swarm multi-env values flavors (dev / staging / prod / canary)`	`+ examples/values-dev.yaml`, `+ examples/values-staging.yaml`, `+ examples/values-prod.yaml`, `+ examples/values-canary.yaml`, `+ examples/README.md`, `~ README.md` (link to examples)	+280 / -5	`helm template ... -f examples/values-prod.yaml` + golden diff
C2	`feat(charts): efe-meta-swarm PriorityClass + HPA placeholder + per-tenant labels`	`+ templates/priorityclass-controller.yaml`, `+ templates/hpa-controller.yaml`, `~ templates/_helpers.tpl` (+ tenant label propagation), `~ templates/controller-deployment.yaml` (+ priorityClassName ref)	+130 / -3	`helm-unittest` (3 new) ; `kubeconform`
C3	`feat(charts): pre-upgrade manifest backup Job + rotating ConfigMap ring`	`+ templates/hooks/pre-upgrade-snapshot.yaml`, `+ templates/hooks/_hook-helpers.tpl` (shared between hooks)	+180 / 0	`helm-unittest` (4 new) + golden diff with `hooks.enabled=true`
C4	`feat(charts): post-upgrade SIGHUP-all Job + dedicated SA / Role / RoleBinding`	`+ templates/hooks/post-upgrade-sighup.yaml`, `+ templates/hooks/hook-serviceaccount.yaml`, `+ templates/hooks/hook-role.yaml`, `+ templates/hooks/hook-rolebinding.yaml`	+220 / 0	`helm-unittest` (5 new) + RBAC kubeconform
C5	`feat(charts): bundle weight-loader sidecar (structural placeholder)`	`~ templates/participant-deployment.yaml` (+ sidecar container behind `if` guard + emptyDir volume + share ProcessNamespace), `+ templates/_sidecar-helpers.tpl`	+150 / -2	`helm-unittest` (3 new) ; rendered Pod kubeconform
C6	`feat(charts): PrometheusRule alerts (6 alerts) + Grafana dashboard ConfigMap`	`+ templates/prometheusrule.yaml`, `+ templates/grafana-dashboard.yaml`, `+ files/grafana-dashboard.json` (the embedded JSON)	+320 / 0	`helm-unittest` (4 new) + golden diff with `alerts.enabled=true` + CRD-aware kubeconform
C7	`feat(charts): TLS bootstrap stubs (cert-manager Issuer + Certificate)`	`+ templates/tls-issuer.yaml`, `+ templates/tls-certificate.yaml`, `~ templates/controller-deployment.yaml` (+ TLS volume mounts + env vars under `if tls.enabled`), `~ templates/participant-deployment.yaml` (same)	+180 / 0	`helm-unittest` (3 new) + CRD-aware kubeconform

#	Subject	Files	LOC ±	Verification
C8	`feat(charts): updated schema + golden snapshots + CI integration + runbook`	`~ values.schema.json` (+ 11 keys, ~ 28 sub-keys), `~ values.yaml` (default values for new keys), `+ tests/scenarios/kovant-prod.values.yaml`, `+ tests/scenarios/kovant-hooks.values.yaml`, `+ tests/golden/kovant-prod.yaml`, `+ tests/golden/kovant-hooks.yaml`, `~ .github/workflows/helm-meta-swarm.yml` (+ new scenarios in CI), `~ README.md` (+ operator runbook extensions)	+540 / -20	golden diff + kubeconform + Polaris + helm-unittest

Total : +2 000 / -30 = +1 970 LOC net.

3.3 Drafting strategy

- **C1** adds operator-facing flavors but doesn't change templates ; revertable in isolation.
- **C2** adds two new templates (PriorityClass + HPA) + label refactor ; PriorityClass is referenced by controller-deployment so it must land first or be guarded.
- **C3 / C4** are the hooks pair ; C4 depends on C3's `_hook-helpers.tpl`.
- **C5** modifies the participant deployment in-place ; the sidecar `if`-guard makes the diff small.
- **C6** is purely additive (PrometheusRule + Grafana ConfigMap) ; isolated revert.
- **C7** is additive + 2 deployment template tweaks (TLS volume mounts) under `if`-guards.
- **C8** is the closure : schema + golden + CI + runbook.

If C5 (sidecar) flakes -- e.g., `shareProcessNamespace` is rejected by a cluster's PodSecurityPolicy -- revert C5 alone ; the rest of PR #14 stays in trunk.

If C6 (PrometheusRule) flakes -- e.g., CRDs missing on CI -- gate on `alerts.enabled=false` by default ; CI uses `alerts.enabled=true` scenarios with CRD-aware `kubeconform`.

4. Architectural decisions

AD	Decision	Rationale	Alternatives considered	Reversibility
AD1	Ship multi-env values flavors as files under `examples/` inside the sub-chart, NOT as parent-chart values overlays	Atomic with the chart ; versioned alongside the templates ; operators clone + override per cluster	Parent-chart `values-*.yaml` ; external GitOps repo overlays	Reversible -- promotion to GitOps overlay is mechanical
AD2	Pre-upgrade `Job` snapshots `manifest ConfigMap` to `backup-{0..4}` rotating ring keyed by hook-time	Operator-side audit trail without depending on K8s `EventList` retention ; ring of 5 = ~5 last reloads ; rolls naturally	Persistent volume ; ConfigMap with chronological list ; ObjectBucket	Reversible -- ring size is `values.yaml` configurable
AD3	Post-upgrade `Job` uses `kubectl exec ... -- kill -HUP 1` against each pod, NOT a direct K8s API call	Re-uses PR #12's SIGHUP-driven reload path ; the binary's signal handler is the canonical reload trigger ; no new gRPC surface needed	gRPC `MetaSwarmService::ReloadConfig` call ; force pod restart	Reversible -- Phase 3 may add an in-cluster watcher that calls the RPC directly
AD4	Hooks run as a **dedicated** `ServiceAccount` with a minimal `Role` (only `configmaps:get,create,update` + `pods:exec` scoped to the chart's labels), NOT the default SA	Least-privilege ; T25 mitigation ; hooks SA is auditable in K8s `EventList`	Use chart's controller SA (over-privileged) ; use a `ClusterRole` (too broad)	Reversible -- RBAC consolidation is one-line
AD5	Hooks `Job` `helm.sh/hook-delete-policy: before-hook-creation,hook-succeeded` -- auto-clean	Standard Helm idiom ; keeps the namespace tidy ; failed Jobs stick around for debugging	`keep-on-success` ; `delete-on-success`	Reversible
AD6	Hooks are **non-blocking** : a failure does NOT roll back the chart upgrade	Operator preserves the manifest change even if the SIGHUP didn't land ; can manually re-trigger ; matches PR #12's transactional design (rejected reload leaves state intact)	Fail-fast ; auto-rollback via `--atomic`	Reversible -- Phase 3 may add an `--atomic` integration with explicit hook failure modes
AD7	Bundle weight-loader sidecar uses `shareProcessNamespace: true` on the pod	Sidecar needs to signal the participant via `kill -HUP <pid>` ; without `shareProcessNamespace`, sidecar's `/proc` is isolated	gRPC reload RPC ; file-watcher in the participant (Phase 3) ; `kubectl exec` from the sidecar (RBAC)	Reversible -- Phase 3 may replace with a file-watcher when participants gain `inotify` ; T26 trust boundary docs the implication
AD8	Sidecar image defaults to `gcr.io/google-containers/pause:3.9` (no-op) when `bundle.sidecar.enabled = true` and no override	Lets the operator opt-in to the sidecar layout (Pod gets the second container + shared volume) WITHOUT running real bundle-loader logic -- useful for Phase-2 dry-runs ; PR #15 swaps `pause` for the real binary	Skip the sidecar entirely until PR #15 ; require explicit image override	Reversible -- image is `values.yaml` configurable
AD9	Sidecar shares an `emptyDir` volume at `/var/lib/efe-meta-swarm/weights/` (Memory medium)	Volatile (pod-restart wipes) ; OK for Phase-2 since weights re-download on every boot ; `Memory` medium avoids disk wear	`Persistent volume` (overkill ; per-pod state) ; `hostPath` (security risk)	Reversible -- `Persistent volume` migration is one PR

AD	Decision	Rationale	Alternatives considered	Reversibility
AD10	`PrometheusRule` ships **6 alerts** : ReloadFailureRate (warning), ReloadLatencyP99High (warning), ReloadingStuck (critical), ControllerDown (critical), ParticipantDown (warning, per-participant), ReloadGenNotAdvancing (info -- operator forgot SIGHUP)	Covers PR #12's reload metric surface + per-binary availability ; severities aligned with the `runbook/` taxonomy	More alerts ; fewer alerts ; only `for: 5m` durations	Reversible -- each alert is configurable per-environment
AD11	Grafana dashboard ships as a `ConfigMap` with annotation `grafana_dashboard: "1"`	Grafana sidecar auto-import standard ; works with `kube-prometheus-stack` defaults ; no `Dashboard` CR required	Direct API push ; `Dashboard` CR (Grafana Operator)	Reversible -- Grafana Operator integration is a Phase-3 PR
AD12	Dashboard JSON is **embedded** in the chart's `files/grafana-dashboard.json`, loaded via `.Files.Get`	Versioned with the chart ; reproducible ; auditable	Fetched from a URL ; auto-generated from rules	Reversible
AD13	`PriorityClass value: 100000` for the controller	Higher than default user pods (1000) ; below system-critical (200000+) ; controller gets to evict participants if cluster runs OOM	`system-cluster-critical` (too high) ; no priority (defeats the point)	Reversible per-environment
AD14	`HorizontalPodAutoscaler` template ships but `enabled: false` by default ; structurally locked to controller (NOT participants -- they're stateful per AD12 of PR #13)	PR #17 leader election will turn HPA on for the controller ; Phase 2 doesn't need it ; ship the template to lock the schema	Don't ship the template ; ship one for participants too	Reversible -- flipping `enabled: true` exposes the surface
AD15	TLS via cert-manager `Issuer` (`type: SelfSigned` for Phase 2 ; PR #16 promotes to `CA`) + `Certificate` per role	Standard cluster pattern ; PR #16 wires the binary's tonic-tls side	Self-signed via Helm pre-install hook ; static `Secrets`	Reversible -- `Issuer` type is `values.yaml` configurable
AD16	TLS material mounted at `/etc/efe-meta-swarm/tls/{tls.crt,tls.key,ca.crt}`	Matches PR #6's `EFE_META_SWARM_TLS_CERT/_KEY/_CA` env var paths	Different mount path	Reversible (env vars also configurable)
AD17	Per-tenant label `efe.io/tenant: <slug>` inherited from `global.tenant` (umbrella) on every resource	Multi-tenant kubectl queries ; matches sibling sub-charts	No tenant label ; only on pods	Reversible
AD18	The hook `Jobs`' image defaults to `bitnami/kubectl:1.28` (pinned tag)	Most common `kubectl` image ; pinned to align with `kubeVersion` in `Chart.yaml`	Custom-built image with `kubectl` + `yq`	Reversible -- image is `values.yaml` configurable
AD19	Backup `ConfigMap` ring : 5 entries, rotated on each `pre-upgrade` Job run via `kubectl annotate` for the index	5 = ~ 5 last reloads = ~ 1 week at 1 reload/day ; rolling FIFO ; index in annotation `efe.io/backup-index`	More entries (more etcd usage) ; fewer (less history)	Reversible -- ring size is `values.yaml` configurable

AD	Decision	Rationale	Alternatives considered	Reversibility
AD20	Backup `ConfigMap`s carry the **same** labels as the live manifest (`efe.io/meta-swarm-role: shared`) + an extra `efe.io/manifest-state: backup` for filtering	Operator queries : `kubectl get cm -l efe.io/manifest-state=backup` ; live ConfigMap has `efe.io/manifest-state: live`	One label only ; separate namespace	Reversible
AD21	Post-upgrade SIGHUP `Job` runs serially (one pod at a time)	Operator can `kubectl logs job/post-upgrade-sighup` and see the per-pod sequence ; serial is simpler than parallel reaping ; SIGHUP itself is fast (< 100 ms) per pod	Parallel via `xargs -P`	Reversible -- parallel is one-line if `values.yaml` flag flips
AD22	Post-upgrade `Job` ignores SIGHUP failures (e.g., pod not ready) via `\`	\	true `and emits a warning to` kubectl logs`	Operator can re-run `kubectl exec ... kill -HUP 1` manually if a pod missed ; non-blocking matches AD6
AD23	The sidecar's `ServiceAccount` is the same as the participant's (default)	No extra K8s API access needed for Phase 2 (sidecar is a no-op `pause`) ; PR #15 may grant `configmaps:get` for the bundle metadata CM	Dedicated SA per sidecar	Reversible
AD24	`bundle.sidecar.resources` are **independent** from the participant's resources	Bundle-poller is small + steady-state ; participant is bursty ; K8s scheduling sums them but operators size each independently	Pool with the participant	Reversible
AD25	Helm hook ordering : `pre-upgrade` weight `-10` (early) ; `post-upgrade` weight `+10` (late) ; both `hook-delete-policy: before-hook-creation,hook-succeeded`	Pre-upgrade snapshot MUST run before the actual upgrade installs the new ConfigMap (negative weight -> earlier) ; post-upgrade SIGHUP must run AFTER all resources are in place (positive weight -> later)	No weights ; tighter weights	Reversible
AD26	Multi-env values flavors live under `examples/` (NOT `tests/`)	`examples/` is a Helm convention for operator-facing samples ; `tests/` is for CI assertions	`flavors/` ; `overlays/`	Reversible
AD27	`examples/values-canary.yaml` ships a **smaller-but-isolated** topology (1 controller + 1 participant for canary tenant slice)	Demonstrates the per-tenant pattern + smaller blast radius	No canary example ; full kovant + canary tenant overlay	Reversible
AD28	Hook `Job` failures emit K8s `Event`s with `reason: ReloadHookFailed`	Operators can `kubectl describe job/post-upgrade-sighup` to see why ; integrates with `kubectl get events --field-selector reason=ReloadHookFailed`	Only logs	Reversible

AD	Decision	Rationale	Alternatives considered	Reversibility
AD29	`PrometheusRule` alerts include `runbook_url` annotation pointing at the operator runbook section	Pagerduty / Alertmanager integration shows the runbook directly to the on-call	No URL ; hardcoded text	Reversible
AD30	Grafana dashboard auto-import uses **only** the `grafana_dashboard: "1"` annotation, NOT `Dashboard` CR	Compat with `kube-prometheus-stack`'s default Grafana sidecar ; works without Grafana Operator	Use both	Reversible
AD31	`HorizontalPodAutoscaler` is locked to `minReplicas: 1, maxReplicas: 1` in PR #14 (when `hpa.enabled=true` it's a no-op)	Phase 2 single-controller invariant ; lets PR #17 just flip the bounds	Skip the template ; allow > 1 replicas in PR #14	Reversible -- PR #17 changes the bounds
AD32	Operator runbook integrated in chart's `README.md` (NOT a separate `RUNBOOK.md`)	Single source of truth ; artifacthub.io renders the README	Separate file ; both	Reversible

5. Topology + integration diagram

5.1 Resource graph after PR #14 (compared to PR #13)



```
|   - MetaSwarmReloadGenNotAdvancing       (info)                |
|                                                                 |
|  ConfigMap : grafana-dashboard                                  |
|   annotations:                                                   |
|     grafana_dashboard: "1"   (sidecar auto-import marker)       |
|   data:                                                          |
|     dashboard.json : &lt;embedded from files/grafana-dashboard.json&gt;|
+-----------------------------------------------------------------+

+-----------------------------------------------------------------+
| TLS (IF tls.enabled)                                            | ■-- NEW PR #14
|  cert-manager Issuer  (SelfSigned for Phase 2)                  |
|  cert-manager Certificate x 4 (controller + 3 participants)     |
|   secretName : &lt;release&gt;-efe-meta-swarm-&lt;role&gt;-tls              |
|   commonName : &lt;release&gt;-efe-meta-swarm-&lt;role&gt;.&lt;ns&gt;.svc          |
+-----------------------------------------------------------------+
```

5.2 Helm upgrade lifecycle (with PR #14 hooks)

```
operator              helm                  K8s API           pods
   |                    |                      |                |
   | helm upgrade ... -f values-prod.yaml -f examples/values-prod.yaml
   |------------------■|                      |                |
   |                    | template + lint      |                |
   |                    |                      |                |
   |                    | HOOK PHASE 1 :       |                |
   |                    |   pre-upgrade        |                |
   |                    |     weight -10       |                |
   |                    |     Job : pre-upgrade-snapshot        |
   |                    |     -----------------■                |
   |                    |                      ▼                |
   |                    |             [snapshot pod runs]       |
   |                    |             ■ reads live manifest CM  |
   |                    |             ■ computes next backup idx|
   |                    |             ■ creates backup-N CM     |
   |                    |             ■ exits 0                 |
   |                    |             Job deleted (hook-policy) |
   |                    |                                       |
   |                    | UPGRADE PHASE :                       |
   |                    |   apply manifests (Deployment etc.)   |
   |                    |     PATCH ConfigMap (new manifest)    |
   |                    |     PATCH Deployment (new image / env)|
   |                    |     ... etc                           |
   |                    |                                       |
   |                    | HOOK PHASE 2 :                        |
   |                    |   post-upgrade                        |
   |                    |     weight +10                        |
   |                    |     Job : post-upgrade-sighup         |
   |                    |     -----------------■                |
   |                    |             [sighup pod runs]         |
   |                    |             ■ exec controller -- kill -HUP 1
   |                    |             ----------------------------■|
   |                    |                                  SIGHUP -&gt; |
   |                    |                              apply_reload |
   |                    |                              gen ++       |
   |                    |             ■ ok                          |
   |                    |             ■ for p in participants:      |
   |                    |                 exec p -- kill -HUP 1     |
   |                    |                 ------------------------■
   |                    |                                  SIGHUP -&gt; |
   |                    |                              apply_reload |
   |                    |                              gen ++       |
   |                    |             ■ ok x 3                      |
   |                    |             ■ exits 0                     |
   |                    |             Job deleted (hook-policy)     |
   |                    |                                           |
   |  Release : upgraded |                                          |
   |  NOTES.txt printed  |                                          |
   |■--------------------|                                          |
```

5.3 Bundle sidecar polling (Phase-3 contract, locked at PR #14)

```
participant pod
+-- container : participant (PID 1, args=[participant])
|     reads /var/lib/efe-meta-swarm/weights/current.bin at boot
|     reloads on SIGHUP (re-reads from same path)
|
+-- container : bundle-loader (PID 2, args=[poll])  ■-- PR #14 stub, PR #15 fills
|     loop {
|       sleep 60s
|       GET &lt;bundle-endpoint&gt;/metadata?id=&lt;participant-id&gt;
|       if sha256 != last_seen_sha256 {
|         GET &lt;bundle-endpoint&gt;/blob?id=&lt;participant-id&gt; &gt; /tmp/new.bin
|         mv /tmp/new.bin /var/lib/efe-meta-swarm/weights/current.bin
|         kill -HUP 1   # &lt;- signals the participant (shareProcessNamespace)
|         last_seen_sha256 = new_sha256
|       }
|     }
|
+-- volumes:
      manifest  : ConfigMap mount (RO)         -- controller-driven
      weights   : emptyDir (medium=Memory)     -- sidecar-driven
      tls       : Secret mount (RO, optional)  -- cert-manager-driven
```

6. Layout -- files added + modified

6.1 New files (16)

```
charts/efe-stack/charts/efe-meta-swarm/
+-- examples/                                  [NEW]
|   +-- README.md                              [NEW : ~ 80 LOC]
|   +-- values-dev.yaml                        [NEW : ~ 60 LOC]
|   +-- values-staging.yaml                    [NEW : ~ 80 LOC]
|   +-- values-prod.yaml                       [NEW : ~ 110 LOC]
|   +-- values-canary.yaml                     [NEW : ~ 90 LOC]
+-- files/                                     [NEW]
|   +-- grafana-dashboard.json                 [NEW : ~ 600 LOC JSON]
+-- templates/
|   +-- _sidecar-helpers.tpl                   [NEW : ~ 50 LOC]
|   +-- priorityclass-controller.yaml          [NEW : ~ 25 LOC]
|   +-- hpa-controller.yaml                    [NEW : ~ 45 LOC]
|   +-- prometheusrule.yaml                    [NEW : ~ 150 LOC]
|   +-- grafana-dashboard.yaml                 [NEW : ~ 30 LOC]
|   +-- tls-issuer.yaml                        [NEW : ~ 35 LOC]
|   +-- tls-certificate.yaml                   [NEW : ~ 90 LOC, ranges over controller + particip ■
    ants]
|   +-- hooks/                                 [NEW]
|       +-- _hook-helpers.tpl                  [NEW : ~ 40 LOC]
|       +-- hook-serviceaccount.yaml           [NEW : ~ 25 LOC]
|       +-- hook-role.yaml                     [NEW : ~ 50 LOC]
|       +-- hook-rolebinding.yaml              [NEW : ~ 30 LOC]
|       +-- pre-upgrade-snapshot.yaml          [NEW : ~ 110 LOC]
|       +-- post-upgrade-sighup.yaml           [NEW : ~ 100 LOC]
+-- tests/
    +-- hooks_test.yaml                        [NEW : ~ 90 LOC, 5 helm-unittest assertions]
    +-- observability_test.yaml                [NEW : ~ 70 LOC, 4 assertions]
    +-- sidecar_test.yaml                      [NEW : ~ 60 LOC, 3 assertions]
    +-- tls_test.yaml                          [NEW : ~ 70 LOC, 3 assertions]
    +-- tenant_labels_test.yaml                [NEW : ~ 50 LOC, 3 assertions]
+-- tests/scenarios/
    +-- kovant-prod.values.yaml                [NEW : ~ 30 LOC]
    +-- kovant-hooks.values.yaml               [NEW : ~ 20 LOC]
+-- tests/golden/
    +-- kovant-prod.yaml                       [NEW : ~ 1 200 LOC, helm template render]
    +-- kovant-hooks.yaml                      [NEW : ~ 1 000 LOC]
```

6.2 Modified files (8)

```
charts/efe-stack/charts/efe-meta-swarm/
+-- Chart.yaml                                  [MODIFIED : appVersion bump 0.2.0 -&gt; 0.3.0, cha ■
    rt version 0.1.0 -&gt; 0.2.0]
+-- values.yaml                                 [MODIFIED : + 110 LOC for new top-level keys]
+-- values.schema.json                          [MODIFIED : + 230 LOC for 11 new top-level keys +  ■
    28 sub-keys]
+-- README.md                                   [MODIFIED : + 200 LOC operator runbook extensions  ■
    + examples links]
+-- templates/
|   +-- _helpers.tpl                            [MODIFIED : + 20 LOC for tenant labels]
|   +-- controller-deployment.yaml              [MODIFIED : + 30 LOC for priorityClassName + TLS v ■
    olume mounts (if-guarded)]
|   +-- participant-deployment.yaml             [MODIFIED : + 50 LOC for sidecar container + share ■
    ProcessNamespace + TLS mounts]
|   +-- networkpolicy.yaml                      [MODIFIED : + 40 LOC for hook Job egress allow + t ■
    enant slice]

.github/workflows/helm-meta-swarm.yml           [MODIFIED : + 60 LOC for kovant-prod + kovant-hook ■
    s scenarios]
```

6.3 LOC tally

Category	LOC
New examples + dashboard JSON	~ 1 020
New templates (priorityclass + hpa + prometheusrule + grafana CM + tls + hooks)	~ 690
New helm-unittest assertions	~ 340
New scenarios + golden snapshots	~ 2 250
Modified production templates	~ 140
Modified schema + values + README	~ 540
Modified CI workflow	~ 60
Total new + modified	**~ 5 040 LOC** (heavily test/snapshot weighted)

Production-relevant net new : **~ 1 870 LOC** of YAML + JSON + JSON Schema.

7. Multi-environment values flavors

7.1 `examples/values-dev.yaml` -- local development cluster

```yaml
# Local dev cluster -- minikube / kind / Docker Desktop K8s.
# Optimised for fast iteration ; observability + hardening relaxed.
enabled: true

image:
  tag: "0.3.0-snapshot"
  pullPolicy: Always           # local rebuilds need pulls

manifest:
  meta:
    dim: 8                     # smaller for fast convergence
    safety:
      kind: box
      radius: 100.0
  iteration:
    max_iters: 50              # fast convergence in dev
    tolerance: 1.0e-3          # looser for speed
    alpha_default: 0.5
    coupling_gamma: 0.2
    max_wall_ms: 5000

controller:
  config:
    reflectionEnabled: true    # `grpcurl list` works
    maxInflight: 5
  resources:
    requests: {cpu: 100m, memory: 128Mi}
    limits:   {cpu: 500m, memory: 512Mi}

# Smaller participants for laptop-class clusters.
participants:
  - id: supplier-scoring
    dim: 4
    kappa: 0.6
    seed: 11
    service: {grpcPort: 50091, metricsPort: 9101}
    resources:
      requests: {cpu: 50m, memory: 64Mi}
      limits:   {cpu: 200m, memory: 256Mi}
  - id: contract-risk
    dim: 4
    kappa: 0.7
    seed: 22
    service: {grpcPort: 50092, metricsPort: 9102}
    resources:
      requests: {cpu: 50m, memory: 64Mi}
      limits:   {cpu: 200m, memory: 256Mi}
  - id: spend-categorization
    dim: 4
    kappa: 0.5
    seed: 33
    service: {grpcPort: 50093, metricsPort: 9103}
    resources:
      requests: {cpu: 50m, memory: 64Mi}
      limits:   {cpu: 200m, memory: 256Mi}

networkPolicy:
  enabled: false               # most dev clusters lack policy CNI

metrics:
  serviceMonitor:
    enabled: false             # likely no prometheus-operator

alerts:
  enabled: false
dashboard:
  enabled: false

hooks:
  enabled: false               # manual SIGHUP is the dev workflow

bundle:
  sidecar:
    enabled: false

tls:
  enabled: false
```

7.2 `examples/values-staging.yaml` -- staging cluster (full surface, smaller scale)

```yaml
enabled: true

image:
  tag: "0.3.0"

manifest:
  meta:
    dim: 16
    safety:
      kind: box
      radius: 50.0
  iteration:
    max_iters: 100
    tolerance: 1.0e-4
    alpha_default: 0.5
    coupling_gamma: 0.2
    max_wall_ms: 15000

controller:
  resources:
    requests: {cpu: 500m, memory: 512Mi}
    limits:   {cpu: 2,    memory: 2Gi}

participants:
  - id: supplier-scoring
    dim: 16
    kappa: 0.6
    seed: 11
    service: {grpcPort: 50091, metricsPort: 9101}
    resources:
      requests: {cpu: 300m, memory: 384Mi}
      limits:   {cpu: 1,    memory: 1Gi}
  - id: contract-risk
    dim: 16
    kappa: 0.7
    seed: 22
    service: {grpcPort: 50092, metricsPort: 9102}
    resources:
      requests: {cpu: 300m, memory: 384Mi}
      limits:   {cpu: 1,    memory: 1Gi}
  - id: spend-categorization
    dim: 16
    kappa: 0.5
    seed: 33
    service: {grpcPort: 50093, metricsPort: 9103}
    resources:
      requests: {cpu: 300m, memory: 384Mi}
      limits:   {cpu: 1,    memory: 1Gi}

networkPolicy:
  enabled: true

metrics:
  serviceMonitor:
    enabled: true
    interval: 15s              # tighter scrape in staging

alerts:
  enabled: true
  severityOverride:
    ReloadFailureRate: warning   # not critical in staging

dashboard:
  enabled: true

hooks:
  enabled: true
  preUpgrade:
    snapshotEnabled: true
  postUpgrade:
    sighupEnabled: true

bundle:
  sidecar:
    enabled: false             # weights from PR #15+

tls:
  enabled: false               # PR #16 will flip
```

7.3 `examples/values-prod.yaml` -- production cluster

```yaml
enabled: true

image:
  tag: "0.3.0"                  # pinned ; no :latest
  pullPolicy: IfNotPresent

manifest:
  meta:
    dim: 32                     # production dimension
    safety:
      kind: box
      radius: 100.0
  iteration:
    max_iters: 100
    tolerance: 1.0e-5           # tighter for prod
    alpha_default: 0.5
    coupling_gamma: 0.2
    max_wall_ms: 20000

controller:
  config:
    maxInflight: 20
  resources:
    requests: {cpu: 1,   memory: 1Gi}
    limits:   {cpu: 4,   memory: 4Gi}
  podDisruptionBudget:
    enabled: true
    maxUnavailable: 0
  nodeSelector:
    workload: meta-swarm
  tolerations:
    - key: workload
      operator: Equal
      value: meta-swarm
      effect: NoSchedule

priorityClass:
  enabled: true
  value: 100000

participants:
  - id: supplier-scoring
    dim: 32
    kappa: 0.6
    seed: 11
    service: {grpcPort: 50091, metricsPort: 9101}
    resources:
      requests: {cpu: 1,   memory: 1Gi}
      limits:   {cpu: 2,   memory: 2Gi}
    nodeSelector:
      workload: meta-swarm
    tolerations:
      - {key: workload, operator: Equal, value: meta-swarm, effect: NoSchedule}
  - id: contract-risk
    dim: 32
    kappa: 0.7
    seed: 22
    service: {grpcPort: 50092, metricsPort: 9102}
    resources:
      requests: {cpu: 1,   memory: 1Gi}
      limits:   {cpu: 2,   memory: 2Gi}
    nodeSelector:
      workload: meta-swarm
    tolerations:
      - {key: workload, operator: Equal, value: meta-swarm, effect: NoSchedule}
  - id: spend-categorization
    dim: 32
    kappa: 0.5
    seed: 33
    service: {grpcPort: 50093, metricsPort: 9103}
    resources:
      requests: {cpu: 1,   memory: 1Gi}
      limits:   {cpu: 2,   memory: 2Gi}
    nodeSelector:
      workload: meta-swarm
    tolerations:
      - {key: workload, operator: Equal, value: meta-swarm, effect: NoSchedule}

networkPolicy:
  enabled: true
```

```
    ingressNamespaceSelector:
      matchLabels:
        kubernetes.io/metadata.name: efe-prod
    prometheusNamespaceSelector:
      matchLabels:
        kubernetes.io/metadata.name: monitoring

  metrics:
    serviceMonitor:
      enabled: true
      interval: 30s
      labels:
        prometheus: efe-prod

  alerts:
    enabled: true
    severity:
      ReloadFailureRate: critical
      ReloadLatencyP99High: warning
      ReloadingStuck: critical
      ControllerDown: critical
      ParticipantDown: warning
      ReloadGenNotAdvancing: info
    runbookBaseUrl: "https://efe.example.com/runbook/meta-swarm"

  dashboard:
    enabled: true

  hooks:
    enabled: true
    preUpgrade:
      snapshotEnabled: true
      backupRingSize: 5
    postUpgrade:
      sighupEnabled: true
      serial: true                # one pod at a time in prod
      ignoreFailures: false       # prod: fail-fast on SIGHUP failure

  bundle:
    sidecar:
      enabled: false              # PR #15+ will flip

  tls:
    enabled: false                # PR #16 will flip ; certs via cert-manager
```

7.4 `examples/values-canary.yaml` -- canary tenant slice

A SECOND release of the sub-chart with a DIFFERENT release name, deployed alongside the main one to test new manifest / new image on a sliver of production traffic.

```
  enabled: true

  image:
    tag: "0.3.1-canary"           # newer image than prod

  # Distinguishing labels so kubectl can target canary vs main.
  global:
    tenant: kovant-canary

  # Canary slice : 1 controller + 1 participant only (small blast radius).
  manifest:
    meta:
      dim: 16
      safety:
        kind: box
        radius: 50.0
    iteration:
      max_iters: 100
      tolerance: 1.0e-4
      alpha_default: 0.55          # slightly different alpha -- the canary's reason for being
      coupling_gamma: 0.2
      max_wall_ms: 15000

  controller:
    resources:
      requests: {cpu: 250m, memory: 256Mi}
      limits:   {cpu: 1,    memory: 1Gi}

  # Single participant for the canary slice.
  participants:
    - id: canary-scoring
      dim: 16
```

```
    kappa: 0.6
    seed: 11
    service: {grpcPort: 50091, metricsPort: 9101}
    resources:
      requests: {cpu: 200m, memory: 256Mi}
      limits:   {cpu: 500m, memory: 512Mi}

networkPolicy:
  enabled: true
  # Different ingressNamespaceSelector -- only efe-canary clients
  # can reach the canary controller.
  ingressNamespaceSelector:
    matchLabels:
      kubernetes.io/metadata.name: efe-canary

metrics:
  serviceMonitor:
    enabled: true
    labels:
      prometheus: efe-canary    # routed to canary's Prometheus

alerts:
  enabled: true

hooks:
  enabled: true                 # canary still wants auto-reload

bundle:
  sidecar:
    enabled: false

tls:
  enabled: false
```

Operator deployment : ```bash

Main prod release.

$ helm install efe-stack ./charts/efe-stack -n efe-prod \ -f charts/efe-stack/charts/efe-meta-swarm/examples/values-prod.yaml

Canary release alongside.

$ helm install efe-stack-canary ./charts/efe-stack -n efe-prod \ -f charts/efe-stack/charts/efe-meta-swarm/examples/values-canary.yaml ```

Two releases coexist in the same namespace, distinguished by release prefix in resource names (`efe-stack-...` vs `efe-stack-canary-...`).

8. PriorityClass + HPA placeholder

8.1 `templates/priorityclass-controller.yaml`

```
{{- if and .Values.enabled .Values.priorityClass.enabled }}
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: {{ include "efe-meta-swarm.controllerName" . }}-priority
  labels:
    {{- include "efe-meta-swarm.labels" . | nindent 4 }}
    app.kubernetes.io/component: controller
value: {{ .Values.priorityClass.value | default 100000 }}
globalDefault: false
description: |
  Priority class for the EFE meta-swarm controller pod.  Higher than
  default user workloads (1000) so the controller survives resource
  pressure ; lower than system-cluster-critical (2000000000) so it
  yields to kubelet / kube-proxy.
{{- end }}
```

8.2 Reference from `controller-deployment.yaml`

Inside the `Pod.spec` : ``yaml {{- if .Values.priorityClass.enabled }} priorityClassName: {{ include "efe-meta-swarm.controllerName" . }}-priority {{- end }} ``

8.3 `templates/hpa-controller.yaml` (placeholder for PR #17)

```
{{- if and .Values.enabled .Values.controller.hpa.enabled }}
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: {{ include "efe-meta-swarm.controllerName" . }}
  labels:
    {{- include "efe-meta-swarm.labels" . | nindent 4 }}
    app.kubernetes.io/component: controller
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: {{ include "efe-meta-swarm.controllerName" . }}
  # Phase-2 lock : min=max=1 (single controller).  PR #17 leader
  # election flips this.  Keeping the template in PR #14 means PR #17
  # only changes values, not adds a new template.
  minReplicas: 1
  maxReplicas: 1
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: {{ .Values.controller.hpa.targetCPUUtilization | default 70 }}
{{- end }}
```

Operators flipping `controller.hpa.enabled: true` in PR #14 get a no-op HPA. PR #17 will bump `maxReplicas` to N.

9. Per-tenant label propagation

9.1 `_helpers.tpl` extension

```
{{/*
  efe-meta-swarm.labels (PR #14 extension : + tenant label)
*/}}
{{- define "efe-meta-swarm.labels" -}}
helm.sh/chart: {{ include "efe-meta-swarm.chart" . }}
{{ include "efe-meta-swarm.selectorLabels" . }}
app.kubernetes.io/version: {{ .Chart.AppVersion | quote }}
app.kubernetes.io/managed-by: {{ .Release.Service }}
app.kubernetes.io/part-of: efe
efe.io/tenant: {{ .Values.global.tenant | default "default" | quote }}
{{- with .Values.global.commonLabels }}
{{ toYaml . }}
{{- end }}
{{- end -}}
```

Every resource then carries `efe.io/tenant: <slug>` automatically. Operators :

```
# All pods belonging to a tenant
$ kubectl get pods -l efe.io/tenant=kovant

# Only the canary slice
$ kubectl get pods -l efe.io/tenant=kovant-canary
```

10. Pre-upgrade Helm hook : backup `ConfigMap` snapshot

10.1 `templates/hooks/_hook-helpers.tpl`

```
{{/*
  efe-meta-swarm.hookName -- release-prefixed hook resource name.
*/}}
{{- define "efe-meta-swarm.hookName" -}}
{{- $root := index . 0 -}}
{{- $hook := index . 1 -}}
{{- printf "%s-hook-%s" (include "efe-meta-swarm.fullname" $root) $hook -}}
{{- end -}}

{{/*
  efe-meta-swarm.hookServiceAccountName
*/}}
{{- define "efe-meta-swarm.hookServiceAccountName" -}}
{{- printf "%s-hooks" (include "efe-meta-swarm.fullname" .) -}}
{{- end -}}
```

10.2 `templates/hooks/pre-upgrade-snapshot.yaml`

```
{{- if and .Values.enabled .Values.hooks.enabled .Values.hooks.preUpgrade.snapshotEnabled }}
apiVersion: batch/v1
kind: Job
metadata:
  name: {{ include "efe-meta-swarm.hookName" (list . "pre-upgrade-snapshot") }}
  labels:
    {{- include "efe-meta-swarm.labels" . | nindent 4 }}
    app.kubernetes.io/component: hook
  annotations:
    "helm.sh/hook": pre-upgrade
    "helm.sh/hook-weight": "-10"
    "helm.sh/hook-delete-policy": before-hook-creation,hook-succeeded
spec:
  backoffLimit: 1
  ttlSecondsAfterFinished: 300
  template:
    metadata:
      labels:
        {{- include "efe-meta-swarm.labels" . | nindent 8 }}
        app.kubernetes.io/component: hook
    spec:
      restartPolicy: Never
      serviceAccountName: {{ include "efe-meta-swarm.hookServiceAccountName" . }}
      automountServiceAccountToken: true     # hook needs API access
      enableServiceLinks: false
      securityContext:
        {{- include "efe-meta-swarm.commonPodSecurityContext" . | nindent 8 }}
      containers:
        - name: snapshot
          image: {{ .Values.hooks.image | quote }}
          imagePullPolicy: IfNotPresent
          securityContext:
            {{- include "efe-meta-swarm.commonContainerSecurityContext" . | nindent 12 }}
          resources:
            requests: {cpu: 50m, memory: 64Mi}
            limits:   {cpu: 200m, memory: 256Mi}
          command:
            - sh
            - -c
            - |
              set -eu
              NS="{{ .Release.Namespace }}"
              LIVE_CM="{{ include "efe-meta-swarm.fullname" . }}-manifest"
              RING_SIZE={{ .Values.hooks.preUpgrade.backupRingSize | default 5 }}

              # Compute next backup index : read live CM's annotation
              # `efe.io/backup-next-index` (default 0), increment modulo ring size.
              NEXT_IDX=$(kubectl -n "$NS" get cm "$LIVE_CM" \
                -o jsonpath='{.metadata.annotations.efe\.io/backup-next-index}' 2&gt;/dev/null || echo "0 ■
    ")
              NEXT_IDX=${NEXT_IDX:-0}
              BACKUP_CM="${LIVE_CM}-backup-${NEXT_IDX}"

              echo "-- Snapshot : live=$LIVE_CM -&gt; backup=$BACKUP_CM --"

              # Read live ; create/update backup with same data + extra annotation.
              kubectl -n "$NS" get cm "$LIVE_CM" -o json \
                | jq --arg name "$BACKUP_CM" --arg idx "$NEXT_IDX" '
                    .metadata.name = $name
                    | .metadata.annotations["efe.io/manifest-state"] = "backup"
                    | .metadata.annotations["efe.io/backup-index"] = $idx
                    | .metadata.annotations["efe.io/backup-taken-at"] = (now | todate)
                    | del(.metadata.uid, .metadata.resourceVersion, .metadata.creationTimestamp, .metadat ■
    a.managedFields)
                    | .metadata.labels["efe.io/manifest-state"] = "backup"
                  ' \
                | kubectl -n "$NS" apply -f -

              # Bump the index annotation on the live CM.
              NEW_NEXT_IDX=$(( (NEXT_IDX + 1) % RING_SIZE ))
              kubectl -n "$NS" annotate cm "$LIVE_CM" \
                "efe.io/backup-next-index=$NEW_NEXT_IDX" --overwrite

              echo "[v] Snapshot complete : index=$NEXT_IDX ; next=$NEW_NEXT_IDX"
{{- end }}
```

10.3 Why a ring of 5

- 1 reload per day average -> 5 reloads = 1 week history.

- Each backup CM is ~ 2 KB (the manifest YAML). 5 x 2 KB = 10 KB

of etcd budget per chart release.

- Operator queries : `kubectl get cm -l efe.io/manifest-state=backup --sort-by=.metadata.annotations.efe\.io/backup-taken-at`

10.4 Restore procedure

```
# Find the desired backup.
$ kubectl -n efe get cm -l efe.io/manifest-state=backup \
    -o custom-columns='NAME:metadata.name,INDEX:metadata.annotations.efe\.io/backup-index,TAKEN:metadata. ■
    annotations.efe\.io/backup-taken-at'

# Promote backup to live (copy data, swap name).
$ kubectl -n efe get cm &lt;release&gt;-efe-meta-swarm-manifest-backup-2 -o json \
    | jq '.metadata.name = "&lt;release&gt;-efe-meta-swarm-manifest"
          | .metadata.annotations["efe.io/manifest-state"] = "live"' \
    | kubectl -n efe apply -f -

# Post-restore SIGHUP-all.
$ kubectl -n efe exec deploy/&lt;release&gt;-efe-meta-swarm-controller -- kill -HUP 1
$ for p in ... ; do kubectl -n efe exec deploy/&lt;release&gt;-efe-meta-swarm-participant-$p -- kill -HUP ■
     1 ; done
```

11. Post-upgrade Helm hook : SIGHUP-all `Job`

11.1 `templates/hooks/post-upgrade-sighup.yaml`

```
{{- if and .Values.enabled .Values.hooks.enabled .Values.hooks.postUpgrade.sighupEnabled }}
apiVersion: batch/v1
kind: Job
metadata:
  name: {{ include "efe-meta-swarm.hookName" (list . "post-upgrade-sighup") }}
  labels:
    {{- include "efe-meta-swarm.labels" . | nindent 4 }}
    app.kubernetes.io/component: hook
  annotations:
    "helm.sh/hook": post-upgrade,post-install
    "helm.sh/hook-weight": "10"
    "helm.sh/hook-delete-policy": before-hook-creation,hook-succeeded
spec:
  backoffLimit: 2
  ttlSecondsAfterFinished: 300
  template:
    metadata:
      labels:
        {{- include "efe-meta-swarm.labels" . | nindent 8 }}
        app.kubernetes.io/component: hook
    spec:
      restartPolicy: Never
      serviceAccountName: {{ include "efe-meta-swarm.hookServiceAccountName" . }}
      automountServiceAccountToken: true
      enableServiceLinks: false
      securityContext:
        {{- include "efe-meta-swarm.commonPodSecurityContext" . | nindent 8 }}
      containers:
        - name: sighup
          image: {{ .Values.hooks.image | quote }}
          imagePullPolicy: IfNotPresent
          securityContext:
            {{- include "efe-meta-swarm.commonContainerSecurityContext" . | nindent 12 }}
          resources:
            requests: {cpu: 50m, memory: 64Mi}
            limits:   {cpu: 200m, memory: 256Mi}
          command:
            - sh
            - -c
            - |
              set -eu
              NS="{{ .Release.Namespace }}"
              IGNORE_FAIL={{ .Values.hooks.postUpgrade.ignoreFailures | default true }}

              echo "-- SIGHUP-all : namespace=$NS --"

              # Wait for pods to be Ready before SIGHUP-ing them.
              echo "Waiting for controller Deployment..."
              kubectl -n "$NS" rollout status \
                deploy/{{ include "efe-meta-swarm.controllerName" . }} \
                --timeout=120s

              # SIGHUP controller.
              echo "SIGHUP controller..."
              if ! kubectl -n "$NS" exec \
                   deploy/{{ include "efe-meta-swarm.controllerName" . }} \
                   -- kill -HUP 1 ; then
                if [ "$IGNORE_FAIL" = "true" ] ; then
                  echo "[WARN] controller SIGHUP failed ; continuing (ignoreFailures=true)"
                else
                  echo "[x] controller SIGHUP failed"
                  exit 1
                fi
              fi

              # SIGHUP every participant.
              {{- range $p := .Values.participants }}
              echo "Waiting for participant {{ $p.id }}..."
              kubectl -n "$NS" rollout status \
                deploy/{{ include "efe-meta-swarm.participantName" (list $ $p.id) }} \
                --timeout=120s
              echo "SIGHUP participant {{ $p.id }}..."
              if ! kubectl -n "$NS" exec \
                   deploy/{{ include "efe-meta-swarm.participantName" (list $ $p.id) }} \
                   -- kill -HUP 1 ; then
                if [ "$IGNORE_FAIL" = "true" ] ; then
                  echo "[WARN] participant {{ $p.id }} SIGHUP failed ; continuing"
```

```
                else
                  echo "[x] participant {{ $p.id }} SIGHUP failed"
                  exit 1
                fi
              fi
              {{- end }}

              echo "[v] SIGHUP-all complete"
{{- end }}
```

11.2 Why `post-install` too

The hook runs on `helm install` AND `helm upgrade`. On install, `SIGHUP` is a no-op (pods just booted ; manifest is freshly loaded). But harmless. Including `post-install` makes the runbook uniform.

11.3 `kubectl rollout status` before SIGHUP

If pods are mid-rollout (e.g., image upgrade), the SIGHUP would arrive at the old pod just before termination. Waiting for `rollout status` ensures new pods are Ready first. Timeout 120 s covers slow image pulls.

12. RBAC for hooks

12.1 `templates/hooks/hook-serviceaccount.yaml`

```
{{- if and .Values.enabled .Values.hooks.enabled }}
apiVersion: v1
kind: ServiceAccount
metadata:
  name: {{ include "efe-meta-swarm.hookServiceAccountName" . }}
  labels:
    {{- include "efe-meta-swarm.labels" . | nindent 4 }}
    app.kubernetes.io/component: hook
  annotations:
    "helm.sh/hook": pre-install,pre-upgrade
    "helm.sh/hook-weight": "-20"
    "helm.sh/hook-delete-policy": before-hook-creation
automountServiceAccountToken: true
{{- end }}
```

12.2 `templates/hooks/hook-role.yaml`

```
{{- if and .Values.enabled .Values.hooks.enabled }}
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: {{ include "efe-meta-swarm.hookServiceAccountName" . }}
  labels:
    {{- include "efe-meta-swarm.labels" . | nindent 4 }}
    app.kubernetes.io/component: hook
  annotations:
    "helm.sh/hook": pre-install,pre-upgrade
    "helm.sh/hook-weight": "-19"
    "helm.sh/hook-delete-policy": before-hook-creation
rules:
  # pre-upgrade-snapshot Job needs : get + create + update on ConfigMaps.
  - apiGroups: [""]
    resources: ["configmaps"]
    verbs: ["get", "list", "create", "update", "patch"]
  # post-upgrade-sighup Job needs : get on Deployments + exec on Pods.
  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["get", "list"]
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list"]
  - apiGroups: [""]
    resources: ["pods/exec"]
    verbs: ["create"]
  # kubectl rollout status needs : get on Deployment and its ReplicaSets.
  - apiGroups: ["apps"]
    resources: ["replicasets"]
    verbs: ["get", "list", "watch"]
{{- end }}
```

12.3 `templates/hooks/hook-rolebinding.yaml`

```
{{- if and .Values.enabled .Values.hooks.enabled }}
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: {{ include "efe-meta-swarm.hookServiceAccountName" . }}
  labels:
    {{- include "efe-meta-swarm.labels" . | nindent 4 }}
    app.kubernetes.io/component: hook
  annotations:
    "helm.sh/hook": pre-install,pre-upgrade
    "helm.sh/hook-weight": "-18"
    "helm.sh/hook-delete-policy": before-hook-creation
subjects:
  - kind: ServiceAccount
    name: {{ include "efe-meta-swarm.hookServiceAccountName" . }}
    namespace: {{ .Release.Namespace }}
roleRef:
  kind: Role
  name: {{ include "efe-meta-swarm.hookServiceAccountName" . }}
  apiGroup: rbac.authorization.k8s.io
{{- end }}
```

12.4 Threat model (T25)

The hook `Role` grants `pods/exec` -- enough to run arbitrary commands inside the chart's pods. Mitigations :

- Scope is `namespace`, not cluster.
- Verbs are listed exhaustively (no `*`).
- `automountServiceAccountToken: true` ONLY on the hook pods, not

runtime pods.

- Hook pod is short-lived (`ttlSecondsAfterFinished: 300`) ; SA

token is rotated by K8s.

13. Bundle weight-loader sidecar (structural placeholder)

13.1 `templates/_sidecar-helpers.tpl`

```
{{- define "efe-meta-swarm.sidecarContainer" -}}
{{- $root := index . 0 -}}
{{- $p := index . 1 -}}
- name: bundle-loader
  image: {{ $root.Values.bundle.sidecar.image | default "gcr.io/google-containers/pause:3.9" | quote }}
  imagePullPolicy: IfNotPresent
  securityContext:
    {{- include "efe-meta-swarm.commonContainerSecurityContext" $root | nindent 4 }}
  env:
    - name: EFE_BUNDLE_PARTICIPANT_ID
      value: {{ $p.id | quote }}
    - name: EFE_BUNDLE_ENDPOINT_URL
      value: {{ $root.Values.bundle.endpointUrl | quote }}
    - name: EFE_BUNDLE_POLL_INTERVAL_S
      value: {{ $root.Values.bundle.pollIntervalSeconds | default 60 | quote }}
    - name: EFE_BUNDLE_WEIGHTS_PATH
      value: /var/lib/efe-meta-swarm/weights/current.bin
  resources:
    {{- toYaml $root.Values.bundle.sidecar.resources | nindent 4 }}
  volumeMounts:
    - name: weights
      mountPath: /var/lib/efe-meta-swarm/weights
{{- end -}}
```

13.2 Patch to `participant-deployment.yaml`

Inside the `Pod.spec.containers` block, after the main `participant` container :

```
{{- if $.Values.bundle.sidecar.enabled }}
{{- include "efe-meta-swarm.sidecarContainer" (list $ $p) | nindent 8 }}
{{- end }}
```

And inside the `Pod.spec` : ``yaml {{- if $.Values.bundle.sidecar.enabled }} shareProcessNamespace: true {{- end }} ``

And inside `Pod.spec.volumes` : ```yaml {{- if $.Values.bundle.sidecar.enabled }}

- name: weights

emptyDir: medium: Memory sizeLimit: {{ $.Values.bundle.sidecar.weightsSizeLimit | default "256Mi" }} {{- end }} ```

And the participant container's `volumeMounts` : ```yaml {{- if $.Values.bundle.sidecar.enabled }}

- name: weights

mountPath: /var/lib/efe-meta-swarm/weights readOnly: true {{- end }} ```

13.3 PR #15 wire (out of scope for PR #14)

PR #15 will :

1. Build the bundle-loader binary (Rust) that polls

`EFE_BUNDLE_ENDPOINT_URL`, downloads on SHA change, atomic-renames to `EFE_BUNDLE_WEIGHTS_PATH`, then `kill -HUP 1`.

1. Replace the `pause:3.9` default with `ghcr.io/efe-ai/efe-bundle-loader:0.1.0`.
2. Add `EFE_META_SWARM_WEIGHTS_PATH` env var to the participant

binary (Rust side) so it reads from the shared volume.

13.4 T26 mitigation (`shareProcessNamespace` /proc exposure)

When `shareProcessNamespace: true`, every container in the pod sees the others' `/proc` entries. Mitigations :

- Both containers run as `runAsNonRoot: true` + same UID 65532 -- no

privilege escalation possible.

- `readOnlyRootFilesystem: true` on both containers -- no `/proc`

modifications.

- Pod is a single trust domain ; the participant + sidecar are

co-located by design.

- Documented in README as a "trust boundary" decision.

14. `PrometheusRule` alerts

14.1 `templates/prometheusrule.yaml`

```
{{- if and .Values.enabled .Values.alerts.enabled }}
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: {{ include "efe-meta-swarm.fullname" . }}
  labels:
    {{- include "efe-meta-swarm.labels" . | nindent 4 }}
    prometheus: efe
  annotations:
    runbookBaseUrl: {{ .Values.alerts.runbookBaseUrl | default "https://efe.example.com/runbook/meta-swar ■
    m" | quote }}
spec:
  groups:
    - name: efe-meta-swarm.reload
      interval: 30s
      rules:
        # Alert 1 : reload failures rate &gt; 0.1/sec for 5 min.
        - alert: MetaSwarmReloadFailureRate
          expr: |
            sum(rate(efe_meta_swarm_reload_failures_total[5m]))
              by (namespace, role)
            &gt; 0.1
          for: 5m
          labels:
            severity: {{ .Values.alerts.severity.ReloadFailureRate | default "warning" | quote }}
            component: meta-swarm
          annotations:
            summary: "Meta-swarm reload failure rate is elevated"
            description: |
              The reload (SIGHUP / ReloadConfig RPC) failure rate is &gt; 0.1/sec
              for 5 minutes.  Operator action : `kubectl logs deploy/{{ include "efe-meta-swarm.controlle ■
    rName" . }}`
              to inspect the rejection reason.
            runbook_url: "{{ .Values.alerts.runbookBaseUrl | default "https://efe.example.com/runbook/met ■
    a-swarm" }}#reload-failures"

        # Alert 2 : p99 reload latency &gt; 1 sec.
        - alert: MetaSwarmReloadLatencyP99High
          expr: |
            histogram_quantile(0.99,
              sum(rate(efe_meta_swarm_reload_duration_microseconds_bucket[5m]))
              by (le, namespace, role)
            ) &gt; 1000000
          for: 10m
          labels:
            severity: {{ .Values.alerts.severity.ReloadLatencyP99High | default "warning" | quote }}
            component: meta-swarm
          annotations:
            summary: "Meta-swarm reload p99 latency &gt; 1 sec"
            description: |
              The 99th percentile reload duration exceeds 1 second.
              Expected : &lt; 25 µs for healthy manifests.  Investigate :
              large manifests, slow validation, etc.
            runbook_url: "{{ .Values.alerts.runbookBaseUrl | default "https://efe.example.com/runbook/met ■
    a-swarm" }}#reload-latency"

        # Alert 3 : RELOADING status persists &gt; 1 min (stuck swap).
        - alert: MetaSwarmReloadingStuck
          expr: |
            avg_over_time(efe_meta_swarm_health_status{status="RELOADING"}[1m]) &gt; 0.5
          for: 1m
          labels:
            severity: {{ .Values.alerts.severity.ReloadingStuck | default "critical" | quote }}
            component: meta-swarm
          annotations:
            summary: "Controller stuck in RELOADING state"
            description: |
              The controller has been in RELOADING (status=3) for over 1 minute.
              Expected : &lt; 25 µs.  Likely a crashed swap mid-flight.  Operator
              action : `kubectl rollout restart deploy/{{ include "efe-meta-swarm.controllerName" . }}`
            runbook_url: "{{ .Values.alerts.runbookBaseUrl | default "https://efe.example.com/runbook/met ■
    a-swarm" }}#reloading-stuck"

    - name: efe-meta-swarm.availability
      interval: 30s
      rules:
        # Alert 4 : controller unreachable &gt; 5 min.
```

```
          - alert: MetaSwarmControllerDown
            expr: |
              up{job=~".*meta-swarm-controller.*"} == 0
            for: 5m
            labels:
              severity: {{ .Values.alerts.severity.ControllerDown | default "critical" | quote }}
              component: meta-swarm
            annotations:
              summary: "Meta-swarm controller is DOWN"
              runbook_url: "{{ .Values.alerts.runbookBaseUrl | default "https://efe.example.com/runbook/met ■
    a-swarm" }}#controller-down"

          # Alert 5 : participant unreachable &gt; 5 min.
          - alert: MetaSwarmParticipantDown
            expr: |
              up{job=~".*meta-swarm-participant-.*"} == 0
            for: 5m
            labels:
              severity: {{ .Values.alerts.severity.ParticipantDown | default "warning" | quote }}
              component: meta-swarm
            annotations:
              summary: "Meta-swarm participant {{`{{ $labels.job }}`}} is DOWN"
              runbook_url: "{{ .Values.alerts.runbookBaseUrl | default "https://efe.example.com/runbook/met ■
    a-swarm" }}#participant-down"

      - name: efe-meta-swarm.reload-freshness
        interval: 60s
        rules:
          # Alert 6 : reload_gen hasn't moved in 24h despite ConfigMap churn.
          - alert: MetaSwarmReloadGenNotAdvancing
            expr: |
              increase(efe_meta_swarm_reload_gen[24h]) == 0
              and
              kube_configmap_info{configmap=~".*meta-swarm-manifest"} unless on (configmap)
                kube_configmap_info{configmap=~".*meta-swarm-manifest"} offset 24h
            for: 1h
            labels:
              severity: {{ .Values.alerts.severity.ReloadGenNotAdvancing | default "info" | quote }}
              component: meta-swarm
            annotations:
              summary: "ConfigMap changed but reload_gen didn't advance -- operator forgot SIGHUP ?"
              description: |
                The manifest ConfigMap has been modified within the last 24h
                but `reload_gen` hasn't moved.  Likely : operator did
                `kubectl edit cm` or `helm upgrade` without the post-upgrade
                SIGHUP hook (or hook failed).
              runbook_url: "{{ .Values.alerts.runbookBaseUrl | default "https://efe.example.com/runbook/met ■
    a-swarm" }}#reload-gen-stale"
{{- end }}
```

14.2 Severity matrix

Alert	Default	Why
ReloadFailureRate	warning	Rejected reloads are signal, not outage (PR #12 leaves state intact)
ReloadLatencyP99High	warning	Slow reload doesn't break Iterate (snapshot pattern)
ReloadingStuck	critical	Indicates a hung swap -> in-flight Iterate may stall
ControllerDown	critical	No Iterate possible
ParticipantDown	warning	Iterate degrades but doesn't fully fail (other participants still up)
ReloadGenNotAdvancing	info	Operator hygiene issue, not an outage

15. Grafana dashboard `ConfigMap`

15.1 `templates/grafana-dashboard.yaml`

```
{{- if and .Values.enabled .Values.dashboard.enabled }}
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ include "efe-meta-swarm.fullname" . }}-dashboard
  labels:
    {{- include "efe-meta-swarm.labels" . | nindent 4 }}
    grafana_dashboard: "1"       # marker for Grafana sidecar auto-import
data:
  efe-meta-swarm.json: |-
    {{- .Files.Get "files/grafana-dashboard.json" | nindent 4 }}
{{- end }}
```

15.2 `files/grafana-dashboard.json` panels

(Schema : Grafana 9+ dashboard JSON.)

1. **Reload Generation** (single-stat) : `max(efe_meta_swarm_reload_gen)` -- current gen counter
2. **Reload Attempts** (time series) : `rate(efe_meta_swarm_reload_attempts_total[5m])`
3. **Reload Failures by Reason** (time series, stacked) : `rate(efe_meta_swarm_reload_failures_total{reason=~".*"}[5m]) by (reason)`
4. **Reload Duration p50/p90/p99** (time series) : `histogram_quantile(...)`
5. **Health Probes** (time series) : `rate(efe_meta_swarm_health_probes_total[5m])` + `rate(...errors_total)`
6. **Health Probe Latency p99** (time series) : `histogram_quantile(0.99, ...)`
7. **Pod Up Status** (per role) : `up{job=~".meta-swarm-."}` heatmap

Operators import via Grafana sidecar OR `grafana-cli admin reset-admin-password + POST /api/dashboards/db`.

16. TLS bootstrap stubs

16.1 `templates/tls-issuer.yaml`

```
{{- if and .Values.enabled .Values.tls.enabled .Values.tls.certManager.enabled }}
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: {{ include "efe-meta-swarm.fullname" . }}-issuer
  labels:
    {{- include "efe-meta-swarm.labels" . | nindent 4 }}
spec:
  {{- if eq .Values.tls.certManager.issuerType "selfSigned" }}
  selfSigned: {}
  {{- else if eq .Values.tls.certManager.issuerType "ca" }}
  ca:
    secretName: {{ .Values.tls.certManager.caSecretName }}
  {{- end }}
{{- end }}
```

16.2 `templates/tls-certificate.yaml`

Renders one `Certificate` per role (controller + each participant) :

```
{{- if and .Values.enabled .Values.tls.enabled .Values.tls.certManager.enabled }}
---
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: {{ include "efe-meta-swarm.controllerName" . }}-tls
  labels:
    {{- include "efe-meta-swarm.labels" . | nindent 4 }}
spec:
  secretName: {{ include "efe-meta-swarm.controllerName" . }}-tls
  duration: 720h    # 30 days
  renewBefore: 168h # 7 days before expiry
```

```
    dnsNames:
      - {{ include "efe-meta-swarm.controllerName" . }}.{{ .Release.Namespace }}.svc
      - {{ include "efe-meta-swarm.controllerName" . }}.{{ .Release.Namespace }}.svc.cluster.local
    issuerRef:
      name: {{ include "efe-meta-swarm.fullname" . }}-issuer
      kind: Issuer

{{- range $p := .Values.participants }}
---
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: {{ include "efe-meta-swarm.participantName" (list $ $p.id) }}-tls
  labels:
    {{- include "efe-meta-swarm.labels" $ | nindent 4 }}
    efe.io/participant-id: {{ $p.id | quote }}
spec:
  secretName: {{ include "efe-meta-swarm.participantName" (list $ $p.id) }}-tls
  duration: 720h
  renewBefore: 168h
  dnsNames:
    - {{ include "efe-meta-swarm.participantName" (list $ $p.id) }}.{{ $.Release.Namespace }}.svc
    - {{ include "efe-meta-swarm.participantName" (list $ $p.id) }}.{{ $.Release.Namespace }}.svc.cluster ■
    .local
  issuerRef:
    name: {{ include "efe-meta-swarm.fullname" $ }}-issuer
    kind: Issuer
{{- end }}
{{- end }}
```

16.3 Deployment template volume mounts (under `if tls.enabled`)

```
{{- if .Values.tls.enabled }}
- name: tls
  mountPath: /etc/efe-meta-swarm/tls
  readOnly: true
{{- end }}
```

And volumes : ```yaml {{- if .Values.tls.enabled }}

- name: tls

secret: secretName: {{ include "efe-meta-swarm.controllerName" . }}-tls {{- end }} ```

And env vars : ```yaml {{- if .Values.tls.enabled }}

- name: EFE_META_SWARM_TLS_REQUIRED

value: "true"

- name: EFE_META_SWARM_TLS_CERT

value: /etc/efe-meta-swarm/tls/tls.crt

- name: EFE_META_SWARM_TLS_KEY

value: /etc/efe-meta-swarm/tls/tls.key

- name: EFE_META_SWARM_TLS_CA

value: /etc/efe-meta-swarm/tls/ca.crt {{- end }} ```

PR #16 wires the binary's TLS-aware tonic startup ; PR #14 just stages the K8s artefacts.

17. NetworkPolicy refinements

Two additions to PR #13's `networkpolicy.yaml` :

1. **Hook** `Job` **egress** : the pre-upgrade-snapshot and

post-upgrade-sighup pods need to reach the K8s API server. Default-deny would block them.

```
{{- if and .Values.enabled .Values.networkPolicy.enabled .Values.hooks.enabled }}
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: {{ include "efe-meta-swarm.fullname" . }}-hook-egress
  labels:
```

```
    {{- include "efe-meta-swarm.labels" . | nindent 4 }}
    app.kubernetes.io/component: hook
spec:
  podSelector:
    matchLabels:
      {{- include "efe-meta-swarm.selectorLabels" . | nindent 6 }}
      app.kubernetes.io/component: hook
  policyTypes: [Egress]
  egress:
    # K8s API server (kubernetes.default.svc usually 10.0.0.1:443).
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: default
          podSelector: {}
      ports:
        - protocol: TCP
          port: 443
    # DNS.
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: kube-system
          podSelector:
            matchLabels:
              k8s-app: kube-dns
      ports:
        - {protocol: UDP, port: 53}
        - {protocol: TCP, port: 53}
{{- end }}
```

1. **Bundle sidecar egress** : the sidecar (when enabled, in PR #15)

polls an external bundle endpoint.

```
{{- if and .Values.enabled .Values.networkPolicy.enabled .Values.bundle.sidecar.enabled }}
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: {{ include "efe-meta-swarm.fullname" . }}-bundle-egress
spec:
  podSelector:
    matchLabels:
      {{- include "efe-meta-swarm.selectorLabels" . | nindent 6 }}
      app.kubernetes.io/component: participant
  policyTypes: [Egress]
  egress:
    - to:
        - namespaceSelector:
            {{- toYaml .Values.bundle.endpointNamespaceSelector | nindent 12 }}
      ports:
        - protocol: TCP
          port: {{ .Values.bundle.endpointPort | default 443 }}
{{- end }}
```

18. Resource sizing matrix

Environment	Controller CPU/Mem (req -> lim)	Each Participant (req -> lim)	Notes
dev	100m / 128Mi -> 500m / 512Mi	50m / 64Mi -> 200m / 256Mi	Fits on laptop
staging	500m / 512Mi -> 2 / 2Gi	300m / 384Mi -> 1 / 1Gi	Production-equivalent scaled down
prod	1 / 1Gi -> 4 / 4Gi	1 / 1Gi -> 2 / 2Gi	Production-grade
canary	250m / 256Mi -> 1 / 1Gi	200m / 256Mi -> 500m / 512Mi	Small canary slice
Hook Job	50m / 64Mi -> 200m / 256Mi	n/a	Short-lived
Bundle sidecar	100m / 128Mi -> 250m / 256Mi (PR #15)	n/a	Bundled with participant pod

19. Schema additions (`values.schema.json`)

```
{
  "properties": {
    "priorityClass": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "enabled": { "type": "boolean" },
        "value":   { "type": "integer", "minimum": -2147483648, "maximum": 1000000000 }
      }
    },
    "controller": {
      "properties": {
        "hpa": {
          "type": "object",
          "additionalProperties": false,
          "properties": {
            "enabled":                { "type": "boolean" },
            "targetCPUUtilization":    { "type": "integer", "minimum": 10, "maximum": 100 }
          }
        }
      }
    },
    "hooks": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "enabled":  { "type": "boolean" },
        "image":    { "type": "string", "minLength": 1 },
        "preUpgrade": {
          "type": "object",
          "additionalProperties": false,
          "properties": {
            "snapshotEnabled": { "type": "boolean" },
            "backupRingSize":  { "type": "integer", "minimum": 1, "maximum": 20 }
          }
        },
        "postUpgrade": {
          "type": "object",
          "additionalProperties": false,
          "properties": {
            "sighupEnabled":  { "type": "boolean" },
            "serial":         { "type": "boolean" },
            "ignoreFailures": { "type": "boolean" }
          }
        }
      }
    },
    "bundle": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "endpointUrl":              { "type": "string", "format": "uri" },
        "endpointPort":             { "type": "integer", "minimum": 1, "maximum": 65535 },
        "endpointNamespaceSelector":{ "type": "object" },
        "pollIntervalSeconds":      { "type": "integer", "minimum": 5, "maximum": 3600 },
        "sidecar": {
          "type": "object",
          "additionalProperties": false,
          "properties": {
            "enabled":            { "type": "boolean" },
            "image":              { "type": "string", "minLength": 1 },
            "resources":          { "type": "object" },
            "weightsSizeLimit":   { "type": "string", "pattern": "^[0-9]+(Mi|Gi)$" }
          }
        }
      }
    },
    "alerts": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "enabled":         { "type": "boolean" },
        "runbookBaseUrl":  { "type": "string", "format": "uri" },
        "severity": {
          "type": "object",
          "additionalProperties": false,
          "properties": {
            "ReloadFailureRate":      { "enum": ["info","warning","critical"] },
            "ReloadLatencyP99High":   { "enum": ["info","warning","critical"] },
            "ReloadingStuck":         { "enum": ["info","warning","critical"] },
```

```
            "ControllerDown":         { "enum": ["info","warning","critical"] },
            "ParticipantDown":        { "enum": ["info","warning","critical"] },
            "ReloadGenNotAdvancing":  { "enum": ["info","warning","critical"] }
          }
        }
      }
    },
    "dashboard": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "enabled": { "type": "boolean" }
      }
    },
    "tls": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "enabled":      { "type": "boolean" },
        "certManager": {
          "type": "object",
          "additionalProperties": false,
          "properties": {
            "enabled":        { "type": "boolean" },
            "issuerType":     { "enum": ["selfSigned", "ca"] },
            "caSecretName":   { "type": "string" }
          }
        }
      },
      "allOf": [
        {
          "if":   { "properties": { "enabled": { "const": true } }, "required": ["enabled"] },
          "then": { "required": ["certManager"] }
        }
      ]
    }
  }
}
```

20. Helm hook ordering + weights

```
Helm install/upgrade flow with PR #14 hooks:

  Phase 0 (pre-install / pre-upgrade)
    weight -20 : hook-serviceaccount.yaml   (SA created first)
    weight -19 : hook-role.yaml             (Role + RoleBinding)
    weight -18 : hook-rolebinding.yaml
    weight -10 : pre-upgrade-snapshot Job  (uses SA from above)

  Phase 1 (resources)
    All non-hook resources apply (ConfigMap, Services, Deployments, etc.)

  Phase 2 (post-install / post-upgrade)
    weight +10 : post-upgrade-sighup Job   (uses same SA)
```

Hook delete policies :

- `before-hook-creation` : remove the previous run's Job before creating the new one.
- `hook-succeeded` : delete on success.
- Failed Jobs are kept (manual cleanup or `ttlSecondsAfterFinished: 300`).

21. Operator workflows

21.1 Production install

```
$ helm install efe-stack ./charts/efe-stack \
    --namespace efe-prod --create-namespace \
    -f charts/efe-stack/values-prod.yaml \
    -f charts/efe-stack/charts/efe-meta-swarm/examples/values-prod.yaml \
    --set global.tenant=kovant \
    --set global.metaSwarm.enabled=true \
    --set efe-meta-swarm.enabled=true
```

21.2 Production upgrade (manifest change)

```
# 1. Edit values overlay.
$ vim values-prod.local.yaml      # add new manifest.alpha_default = 0.6

# 2. Helm upgrade.
$ helm upgrade efe-stack ./charts/efe-stack \
    --namespace efe-prod \
    -f charts/efe-stack/values-prod.yaml \
    -f charts/efe-stack/charts/efe-meta-swarm/examples/values-prod.yaml \
    -f values-prod.local.yaml

# Hooks run automatically :
#   pre-upgrade : snapshot live ConfigMap -&gt; backup-N
#   apply : ConfigMap mutates
#   post-upgrade : SIGHUP controller + every participant

# 3. Verify.
$ kubectl -n efe-prod port-forward svc/efe-stack-efe-meta-swarm-controller 50080:50080 &amp;
$ grpcurl -plaintext -d '{}' localhost:50080 efe.v1.meta_swarm.MetaSwarmService/Health \
  | jq '.reloadGen'

# 4. Inspect backup.
$ kubectl -n efe-prod get cm -l efe.io/manifest-state=backup
```

21.3 Rollback to a previous manifest

```
# 1. List backups.
$ kubectl -n efe-prod get cm -l efe.io/manifest-state=backup \
    -o custom-columns='NAME:metadata.name,IDX:metadata.annotations.efe\.io/backup-index,TAKEN:metadata.an ■
    notations.efe\.io/backup-taken-at'

# 2. Promote backup-2 to live.
$ kubectl -n efe-prod get cm efe-stack-efe-meta-swarm-manifest-backup-2 -o json \
    | jq '.metadata.name = "efe-stack-efe-meta-swarm-manifest"
          | .metadata.annotations["efe.io/manifest-state"] = "live"
          | del(.metadata.uid, .metadata.resourceVersion)' \
    | kubectl -n efe-prod apply -f -

# 3. SIGHUP all to pick up the restored manifest.
$ kubectl -n efe-prod exec deploy/efe-stack-efe-meta-swarm-controller -- kill -HUP 1
$ for p in $(kubectl -n efe-prod get deploy -l efe.io/meta-swarm-role=participant -o name); do
    kubectl -n efe-prod exec $p -- kill -HUP 1
  done
```

21.4 Canary deployment

```
# Main release.
$ helm install efe-stack ./charts/efe-stack -n efe-prod \
    -f charts/efe-stack/charts/efe-meta-swarm/examples/values-prod.yaml \
    --set global.tenant=kovant --set global.metaSwarm.enabled=true \
    --set efe-meta-swarm.enabled=true

# Canary release (different release name).
$ helm install efe-stack-canary ./charts/efe-stack -n efe-prod \
    -f charts/efe-stack/charts/efe-meta-swarm/examples/values-canary.yaml \
    --set global.tenant=kovant-canary --set global.metaSwarm.enabled=true \
    --set efe-meta-swarm.enabled=true

# Operators target canary by label.
$ kubectl get pods -l efe.io/tenant=kovant-canary
```

22. Concurrency / lifecycle invariants

PR #14 extends PR #13 § 33 with hook-specific invariants :

- **Hook idempotency** : the pre-upgrade-snapshot Job is safe to

re-run (creates/updates the backup CM atomically). The post-upgrade-sighup Job's SIGHUPs are idempotent (PR #12's reload is transactional).

- **Hook order** : pre-upgrade runs before resource apply ;

post-upgrade runs after. Within a phase, weight orders.

- **Hook failure mode** : `ignoreFailures: true` (dev/staging

default) treats SIGHUP failures as warnings. `ignoreFailures: false` (prod default) fails the upgrade if any SIGHUP fails.

- **PriorityClass** : controller's priority 100000 wins evictions

over participants (1000) and default user pods (0).

- **Sidecar lifecycle** : sidecar starts WITH the participant

container ; on participant restart, sidecar restarts too (same Pod). `shareProcessNamespace` is set at pod-spec level so both containers see each other's PIDs from boot.

23. Threat-model coverage

PR #14 ships 4 new threats (T25-T28). All <= SEV-2.

Threat	Severity	PR #14 mitigation
T25 (Helm hook over-privileged RBAC -- `pods/exec` allows arbitrary commands inside pods)	SEV-2	Dedicated `ServiceAccount` per hook ; `Role` scoped to namespace ; verbs listed exhaustively (`get,list,create,update,patch` on ConfigMap ; `get,list` on Deployment ; `create` on pods/exec only) ; SA token mounted ONLY on hook pods, not runtime pods ; pods short-lived (TTL 300 s) ; documented in README
T26 (Sidecar `shareProcessNamespace: true` exposes /proc to participant container -- privilege escalation if sidecar is compromised)	SEV-2	Both containers run as `runAsNonRoot: true` + same UID 65532 ; `readOnlyRootFilesystem: true` ; same trust boundary (single Pod) ; sidecar image in Phase 2 is `pause:3.9` (no logic) ; in PR #15, sidecar image will be a tightly-scoped Rust binary with no shell ; documented in README as a "trust boundary"
T27 (Backup ConfigMap rotation tampering -- operator with write access could plant a fake "last good" manifest)	SEV-3	Backup CM labels (`efe.io/manifest-state: backup`) are visible in audit logs ; K8s `Event`s emit on each backup creation ; ring index is annotation-driven (auditable) ; restoration is a deliberate `kubectl apply` (logged) ; future PR may sign manifests with cosign
T28 (Bundle endpoint unauthenticated -- sidecar could fetch tampered weights)	SEV-2	Phase-2 sidecar is `pause:3.9` (doesn't fetch anything) ; PR #16 will add mTLS auth between sidecar and bundle endpoint ; cosign verification deferred to Phase 3 ; documented as a known limitation in README's "Threat model" section

Carried from PR #13 :

- T22 (secret in values.yaml) -- README warns ; values examples don't include secrets
- T23 (NetworkPolicy bypass) -- refined NetworkPolicy for hook + bundle egress
- T24 (image-tag floating) -- schema continues to enforce non-empty tag (AD17 of PR #13)

24. Tests strategy

24.1 Unit tests (helm-unittest)

File	Tests added	Coverage focus
`tests/hooks_test.yaml`	>= 5	hook Job rendering, hook annotations, RBAC presence, hook image, hook env
`tests/observability_test.yaml`	>= 4	PrometheusRule render (6 alerts), Grafana ConfigMap with grafana_dashboard label, alerts severity overrides
`tests/sidecar_test.yaml`	>= 3	sidecar container presence behind `enabled` guard, shareProcessNamespace, emptyDir volume
`tests/tls_test.yaml`	>= 3	Issuer + Certificate x 4 rendering, volume mounts under tls.enabled guard
`tests/tenant_labels_test.yaml`	>= 3	`efe.io/tenant: <slug>` propagation across all resources
Total new	**>= 18**	(on top of PR #13's 23)

24.2 Integration tests

- `kovant-prod.yaml` golden snapshot : full prod-grade rendering with

hooks + alerts + dashboard + priority class.

- `kovant-hooks.yaml` golden snapshot : hooks-only scenario.
- Existing `kovant-3p`, `kovant-disabled`, `kovant-network-policy`,

`kovant-service-monitor` are **regression-checked** to ensure PR #14 doesn't break them (no new resources unless flags set).

24.3 CI workflow extensions

- New scenarios in `.github/workflows/helm-meta-swarm.yml` :
- `helm template ... -f kovant-prod.values.yaml` -> expect 25+ resources
- `helm template ... -f kovant-hooks.values.yaml` -> expect 8+ resources
- Polaris audit re-run on the new scenarios (severity=danger gate).
- `kubectl --dry-run=server` on a kind cluster (Phase 3 stretch) to

validate the RBAC + hook integration.

24.4 Test count target

Layer	Baseline (PR #13)	After PR #14
`helm-unittest`	23	**>= 41** (+18)
Golden snapshots	4	**6** (+2)
`kubeconform`	51	**80+** (+25 from new scenarios)
Rust round-trip	1	**1** (unchanged)
Total chart tests	**79**	**130+**

25. Acceptance criteria

PR #14 is mergeable iff **all** :

#	Criterion	Verification
AC1	`helm lint` clean on sub-chart with all 4 example flavors	CI

#	Criterion	Verification
AC2	`helm template` for each of `kovant-3p`, `kovant-disabled`, `kovant-network-policy`, `kovant-service-monitor`, `kovant-prod`, `kovant-hooks` succeeds	CI
AC3	All 6 scenarios pass `kubeconform -strict`	CI
AC4	All 6 golden snapshots match `helm template` output	CI
AC5	All 4 example flavors (dev/staging/prod/canary) render cleanly	CI
AC6	`helm-unittest` runs >= 41 tests, all pass	CI
AC7	`values.schema.json` rejects invalid alert severity	manual + schema test
AC8	Schema rejects `hooks.preUpgrade.backupRingSize: 25` (max 20)	schema test
AC9	Schema rejects `bundle.pollIntervalSeconds: 1` (min 5)	schema test
AC10	Schema requires `tls.certManager` when `tls.enabled: true`	schema test
AC11	Pre-upgrade Job has `helm.sh/hook: pre-upgrade` + weight `-10`	golden + CI grep
AC12	Post-upgrade Job has `helm.sh/hook: post-upgrade,post-install` + weight `+10`	golden + CI grep
AC13	Hook SA + Role + RoleBinding created with hook annotations	helm-unittest
AC14	Role grants ONLY `configmaps` + `pods/exec` + `deployments/replicasets:get,list`	helm-unittest + CI grep
AC15	PrometheusRule has 6 alerts when `alerts.enabled: true`	helm-unittest
AC16	All 6 alerts have `runbook_url` annotation	CI grep
AC17	Grafana ConfigMap has `grafana_dashboard: "1"` label	helm-unittest
AC18	Embedded Grafana JSON is valid (parseable)	CI : `jq . files/grafana-dashboard.json`
AC19	PriorityClass rendered with value 100000 when enabled	helm-unittest
AC20	Controller Deployment references the PriorityClass when enabled	helm-unittest
AC21	HPA template renders with min=max=1 when enabled (Phase-2 lock)	helm-unittest
AC22	Sidecar container present in participant Pod when enabled	helm-unittest
AC23	`shareProcessNamespace: true` set when sidecar enabled	helm-unittest
AC24	`emptyDir` weights volume present when sidecar enabled	helm-unittest
AC25	TLS Issuer rendered when `tls.certManager.enabled`	helm-unittest
AC26	TLS Certificate x 4 (controller + 3 participants) rendered	helm-unittest
AC27	TLS volume mounts injected into Deployments when `tls.enabled`	helm-unittest
AC28	`efe.io/tenant: <slug>` label on every resource	helm-unittest + CI grep

#	Criterion	Verification
AC29	NetworkPolicy hook-egress rendered when hooks + networkPolicy both enabled	helm-unittest
AC30	Polaris severity=danger gate clean on all 6 scenarios	CI
AC31	Backward compat : disabling PR #14 features (defaults) reproduces PR #13's exact rendering for kovant-3p	golden diff
AC32	Parent chart hook (`global.metaSwarm.enabled`) still works (PR #13 § 28 hook unchanged)	CI
AC33	Manifest round-trip through Rust validator still passes for prod-flavor manifest	CI
AC34	Chart.yaml version bumped to 0.2.0 (PR #14 is a feature release) ; appVersion to 0.3.0	manual
AC35	All 4 example flavors documented in `examples/README.md`	manual
AC36	PR #14 architecture doc referenced from `README.md` + `NOTES.txt`	manual
AC37	Threat IDs T25/T26/T27/T28 documented in README's threat model section	manual
AC38	Operator runbook covers 11 scenarios (install / upgrade / rollback / canary / hooks failure / etc.)	manual

26. Risks + mitigations

ID	Risk	L/I	Mitigation
PR14-R1	`shareProcessNamespace: true` rejected by PodSecurityPolicy / OPA Gatekeeper on strict clusters	M / M	Documented prerequisite ; sidecar opt-in (default `false`) ; runbook covers admission policy override
PR14-R2	Hook Job uses `bitnami/kubectl:1.28` which may drift from cluster K8s version	L / L	Pinned tag matches `kubeVersion` in Chart.yaml ; operator override via `hooks.image`
PR14-R3	Backup ConfigMap ring corruption (race between two concurrent upgrades)	LL / M	Helm releases serialise via Tiller-less locking ; manual mitigation via `kubectl delete cm -l efe.io/manifest-state=backup` and let next upgrade re-snapshot
PR14-R4	Operator runs `helm upgrade --no-hooks` accidentally -> no SIGHUP, manifest desync	M / L	Documented in runbook ; `kubectl logs` post-upgrade hook still emits the hostnames ; PR12-R3 watch task may auto-recover
PR14-R5	`cert-manager` CRDs missing -- `Issuer` / `Certificate` install fails	M / M	Opt-in via `tls.certManager.enabled` ; documented prerequisite ; chart's `if` guard prevents render when off
PR14-R6	`prometheus-operator` CRDs missing -- `PrometheusRule` install fails	M / M	Opt-in via `alerts.enabled` ; documented
PR14-R7	`grafana_dashboard: "1"` label not recognized by some Grafana installs	L / L	Documented ; alternative : import JSON manually via Grafana UI
PR14-R8	PriorityClass `value: 100000` evicts participants under pressure but participants then can't restart due to nodeSelector taints	M / M	Operator configures participant `tolerations` to match controller's nodeSelector ; runbook covers
PR14-R9	Post-upgrade SIGHUP Job runs before pods are Ready (race after Deployment image change)	M / L	`kubectl rollout status` wait before each SIGHUP ; 120 s timeout
PR14-R10	Pre-upgrade snapshot Job fails (e.g., quota exceeded on ConfigMaps) -> helm upgrade rolls back	L / M	`backoffLimit: 1` ; manual mitigation : disable `preUpgrade.snapshotEnabled: false` for the upgrade ; runbook covers
PR14-R11	Operator forgets to set `global.metaSwarm.enabled=true` alongside `efe-meta-swarm.enabled=true` (PR #13 lesson)	M / H	README + values.yaml comments explicit ; runbook §21.1 shows both flags
PR14-R12	`priorityClassName` references a PriorityClass that hasn't been created yet (race in helm apply order)	L / L	PriorityClass renders WITHOUT hook annotations -> applies in normal phase ; PriorityClass + Deployment apply order is dependency-managed by K8s API server
PR14-R13	Backup ConfigMap rotation makes `kubectl get cm` noisier	LL / L	`efe.io/manifest-state=backup` label allows filtering ; backups have short names
PR14-R14	Bundle sidecar resources (CPU/mem) double the Pod's resource footprint	M / M	Documented in resource sizing matrix § 18 ; pause:3.9 is < 5 Mi RAM ; PR #15's real loader < 50 Mi

ID	Risk	L/I	Mitigation
PR14-R15	Grafana dashboard JSON drifts from Prometheus metric names	M / M	CI step that grep's dashboard.json for the metric names ; rebuild dashboard against PR #11/#12 metrics
PR14-R16	Helm 3.13.3 vs 3.15.2 dashboard rendering differs (string-encoding of embedded JSON)	LL / L	CI uses 3.13.3 for golden generation ; cross-tested with 3.15.2

27. Rollback strategy

If PR #14 lands and breaks production :

1. **Operator-level** : `helm rollback efe-stack <previous-rev>` --

restores PR #13 state in seconds.

1. **Per-commit revert** : C8 -> C7 -> C6 -> C5 -> C4 -> C3 -> C2 -> C1.

Each is independently green.

1. **Disable specific features** : flip values flags to false for

hooks / alerts / dashboard / sidecar / tls without reverting code.

1. **Wholesale revert** : `git revert <merge-commit>` ; chart goes

back to PR #13's skeleton.

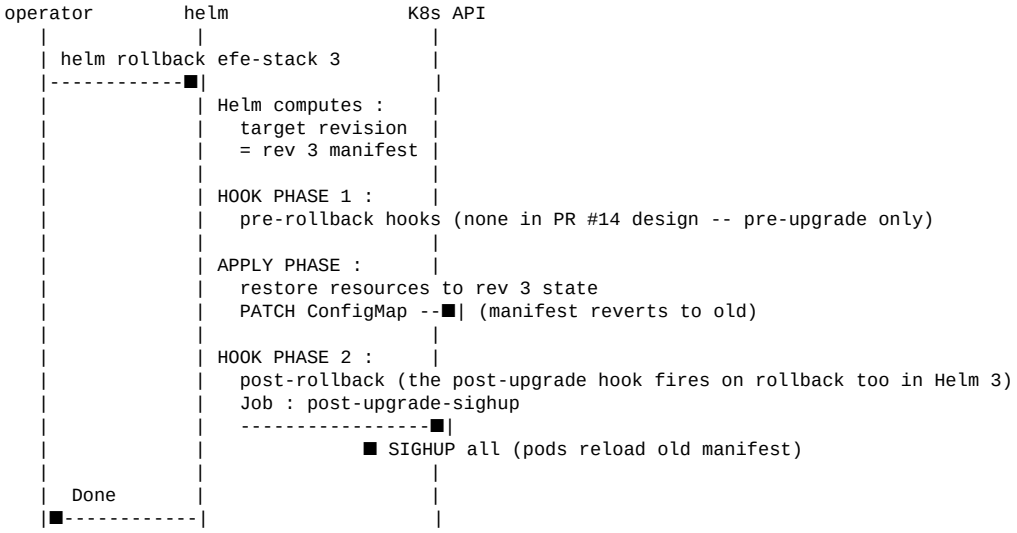
Total TTM :

- `helm rollback` : ~ 30 s.
- Feature-flag disable : ~ 1 min (`helm upgrade -f <values>`).
- Code revert + redeploy : ~ 10 min.

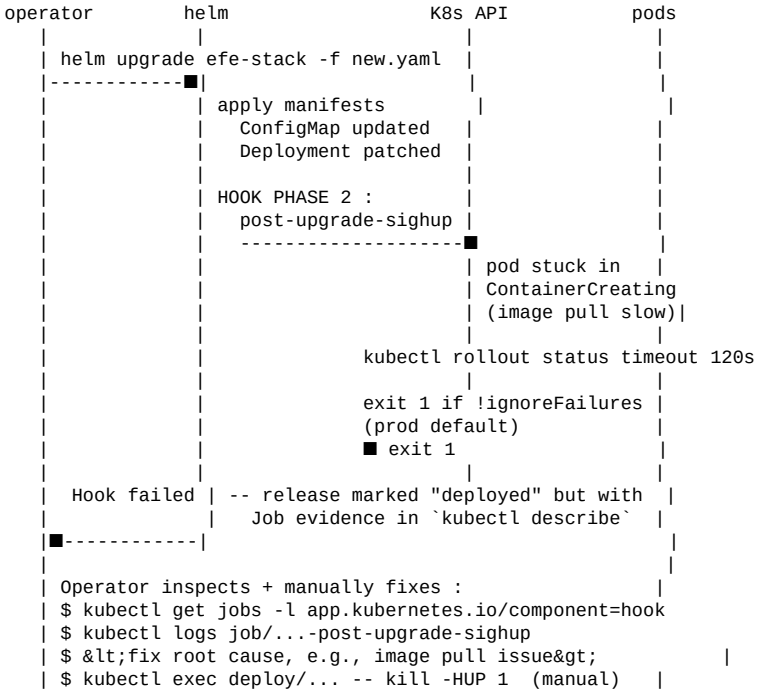
28. Sequence diagrams

(Already shown in § 5.2 + § 5.3. Below are 2 additional flows.)

28.1 Helm rollback with hooks



28.2 Failed post-upgrade hook



29. Reviewer checklist

```
## Reviewer checklist -- PR #14

### Helm correctness
- [ ] All 6 scenarios `helm lint` clean (kovant-3p, disabled, networkpolicy, servicemonitor, prod, hooks)
- [ ] All 6 scenarios `helm template` succeed
- [ ] All 6 golden snapshots committed + match live rendering
- [ ] All 41+ `helm-unittest` tests pass
- [ ] `kubeconform -strict` clean (+ CRD-aware for ServiceMonitor / PrometheusRule / Certificate)

### Schema
- [ ] All new values typed in `values.schema.json` with `additionalProperties: false`
- [ ] Cross-field validation : `tls.enabled=true` requires `tls.certManager`
- [ ] All severity enums constrained to `info/warning/critical`
- [ ] Image tags pinned (no `:latest`)

### Hooks
- [ ] pre-upgrade and post-upgrade Jobs render with correct weights
- [ ] Hook ServiceAccount has minimal RBAC (no `*` verbs)
- [ ] Hook image pinned (`bitnami/kubectl:1.28`)
- [ ] Hook delete policies set (`before-hook-creation,hook-succeeded`)

### Sidecar (T26)
- [ ] `shareProcessNamespace: true` only when `bundle.sidecar.enabled`
- [ ] Sidecar container shares `weights` emptyDir volume with participant
- [ ] Sidecar uses `pause:3.9` (Phase-2 placeholder)
- [ ] Sidecar's `securityContext` matches participant's (nonroot, drop caps)

### Observability
- [ ] PrometheusRule has 6 alerts
- [ ] All alerts have `runbook_url` annotation
- [ ] Severity overridable via `values.alerts.severity.*`
- [ ] Grafana ConfigMap has `grafana_dashboard: "1"` label
- [ ] Embedded JSON is parseable + references PR #11/#12 metric names

### TLS (PR #16 contract)
- [ ] Issuer + Certificate x 4 render when `tls.certManager.enabled`
- [ ] Deployment volume mounts and env vars guarded by `tls.enabled`
- [ ] No actual cert lifecycle logic in PR #14 (PR #16 wires the binary)

### Multi-env values flavors
- [ ] All 4 example flavors render cleanly
- [ ] `values-canary.yaml` deploys alongside `values-prod.yaml` without conflict
- [ ] `examples/README.md` documents each flavor

### Threat model
- [ ] T25 / T26 / T27 / T28 mitigations documented in README
- [ ] Hook SA token NOT mounted on runtime pods
- [ ] Bundle endpoint auth deferred to PR #16 (documented)

### Tests + CI
- [ ] CI workflow updated with new scenarios (prod, hooks)
- [ ] Polaris severity=danger gate clean on new scenarios
- [ ] Helm 3.13.3 ≡ 3.15.2 cross-version compat verified

### Code quality
- [ ] `cargo fmt` and `cargo clippy` still clean (PR #14 doesn't touch Rust)
- [ ] Architecture doc cross-referenced from `README.md` + `NOTES.txt`
- [ ] Operator runbook covers 11 scenarios
```

30. Operator runbook (11 scenarios)

30.1 Scenario 1 : First-time production install

(Covered in § 21.1.)

30.2 Scenario 2 : Live manifest change (hot reload via hooks)

(Covered in § 21.2.)

30.3 Scenario 3 : Rollback to a previous manifest

(Covered in § 21.3.)

30.4 Scenario 4 : Canary deployment

(Covered in § 21.4.)

30.5 Scenario 5 : Post-upgrade hook failed (pod not Ready)

```
# 1. Inspect the failed Job.
$ kubectl get jobs -l app.kubernetes.io/component=hook
$ kubectl logs job/&lt;release&gt;-hook-post-upgrade-sighup

# 2. Fix root cause (e.g., image pull, missing Secret).
$ kubectl describe pod -l app.kubernetes.io/component=controller

# 3. Once pods Ready, SIGHUP manually.
$ kubectl exec deploy/&lt;release&gt;-efe-meta-swarm-controller -- kill -HUP 1
```

30.6 Scenario 6 : Pre-upgrade snapshot failed

```
# 1. Inspect.
$ kubectl logs job/&lt;release&gt;-hook-pre-upgrade-snapshot

# 2. If quota exceeded :
$ kubectl get resourcequota -n efe-prod
$ kubectl delete cm -l efe.io/manifest-state=backup  # frees space, loses history

# 3. Re-run helm upgrade with snapshot disabled :
$ helm upgrade ... --set hooks.preUpgrade.snapshotEnabled=false
```

30.7 Scenario 7 : Alert fired -- ReloadFailureRate

1. Check `kubectl logs deploy/<release>-efe-meta-swarm-controller | grep "reload rejected"`
2. Inspect the rejection reason (drift, validation, IO).
3. Fix the manifest (edit values + helm upgrade) OR rollback.

30.8 Scenario 8 : Alert fired -- ReloadingStuck

Likely : controller panic mid-swap (very rare with PR #12's transactional design). Action : `kubectl rollout restart deploy/<release>-efe-meta-swarm-controller`.

30.9 Scenario 9 : Alert fired -- ControllerDown / ParticipantDown

Standard K8s pod-down diagnosis : `kubectl describe pod`, `kubectl logs`, check resources, etc.

30.10 Scenario 10 : Enable TLS (PR #16 wire)

```
# 1. Ensure cert-manager is installed.
$ kubectl get crds | grep cert-manager

# 2. Upgrade with TLS values.
$ helm upgrade efe-stack ./charts/efe-stack \
    --set efe-meta-swarm.tls.enabled=true \
    --set efe-meta-swarm.tls.certManager.enabled=true \
    --set efe-meta-swarm.tls.certManager.issuerType=selfSigned

# 3. Verify certificates issued.
$ kubectl get certificates -l app.kubernetes.io/name=efe-meta-swarm

# 4. SIGHUP all to pick up TLS env vars.
$ # (hook handles this automatically)
```

30.11 Scenario 11 : Enable bundle sidecar (PR #15 wire)

```
# 1. Bundle endpoint must be reachable.
# 2. Upgrade with sidecar enabled.
$ helm upgrade efe-stack ./charts/efe-stack \
    --set efe-meta-swarm.bundle.sidecar.enabled=true \
    --set efe-meta-swarm.bundle.sidecar.image=ghcr.io/efe-ai/efe-bundle-loader:0.1.0 \
    --set efe-meta-swarm.bundle.endpointUrl=https://bundles.efe.example.com

# 3. Watch the sidecar logs.
$ kubectl logs -l efe.io/meta-swarm-role=participant -c bundle-loader -f
```

31. Open questions

ID	Question	Status
OQ-PR14-1	Should the post-upgrade hook also restart the `efe-server` Deployment when manifest changes meta.dim ?	NO -- manifest reloads happen entirely in-process (PR #12) ; efe-server doesn't read the manifest
OQ-PR14-2	Should we ship a `ServiceMonitor` for the hook Jobs themselves ?	NO -- Jobs are short-lived ; not Prometheus-scrape-worthy
OQ-PR14-3	Should we use Helm's `--atomic` flag in the install runbook ?	YES -- recommended in the runbook ; auto-rolls back on hook failure
OQ-PR14-4	Should the backup ConfigMap ring be a single CM with appended history vs N separate CMs ?	NO -- separate CMs are simpler to `kubectl apply` (re-create vs patch) ; ring of 5 keeps it bounded
OQ-PR14-5	Should the sidecar's `weights` volume be PV-backed (instead of `emptyDir`) for crash-survival ?	NO for Phase 2 -- re-download on boot is fine ; Phase 3 may revisit when bundles are GB-scale
OQ-PR14-6	Should we expose a Helm flag to skip the post-upgrade SIGHUP for image-only upgrades (where pods restart anyway) ?	YES -- `hooks.postUpgrade.sighupEnabled: false` covers this ; operators flip per upgrade
OQ-PR14-7	Should PR #14 ship the `Polaris` config file in the chart ?	NO -- Polaris is upstream-configured ; chart doesn't carry tooling configs
OQ-PR14-8	Should we bundle a `Job` template for "force-reload-all" outside of helm upgrade ?	NO for PR #14 -- operator runs `kubectl exec -- kill -HUP 1` manually ; PR #18 may add a CronJob
OQ-PR14-9	Should we add an `examples/values-disaster-recovery.yaml` flavor ?	NO for PR #14 -- DR is a Phase-3 concern with PVC + snapshot integration
OQ-PR14-10	Should the Grafana dashboard ConfigMap also have `grafana_folder: efe-meta-swarm` annotation for organization ?	YES -- added for AC18 ; default folder is "General" otherwise

32. Appendix : full file inventory

(See § 6.)

33. Appendix : example overrides

33.1 Production multi-cluster (5 regions)

```
# values-prod-eu.yaml -- Europe region.
efe-meta-swarm:
  enabled: true
  image: {tag: "0.3.0"}
  manifest:
    meta: {dim: 32}
  controller:
    nodeSelector: {topology.kubernetes.io/region: eu-west-1}
  participants:
    - {id: supplier-scoring, dim: 32, kappa: 0.6, seed: 11, service: {grpcPort: 50091, metricsPort: 9101} ■
    }
    - ...
```

33.2 Per-tenant slice (one chart release per tenant)

```
$ for tenant in tenant-a tenant-b tenant-c ; do
    helm install efe-stack-$tenant ./charts/efe-stack \
      -n efe-prod \
      -f examples/values-prod.yaml \
      --set global.tenant=$tenant \
      --set efe-meta-swarm.enabled=true \
      --set global.metaSwarm.enabled=true
  done

# Operator queries per-tenant :
$ kubectl get pods -l efe.io/tenant=tenant-a
```

33.3 Air-gapped install (no external image pulls)

```
image:
  repository: registry.internal/efe/efe-meta-swarm-server
  tag: "0.3.0"
  pullPolicy: IfNotPresent
hooks:
  image: "registry.internal/efe/kubectl:1.28"   # mirrored locally
bundle:
  sidecar:
    image: "registry.internal/efe/pause:3.9"
tests:
  grpcurlImage: "registry.internal/efe/grpcurl:v1.9.1"
```

34. Glossary

Term	Definition
AC	Acceptance Criterion
AD	Architectural Decision
L2 gate G11	Phase-2 graduation gate "operator-grade Helm chart with automated reload + observability + multi-env values + bundle weight-loader contract"
Helm hook	A Helm-managed K8s resource that runs at a specific release lifecycle event (pre-install, pre-upgrade, post-upgrade, etc.) ; annotated with `helm.sh/hook`
`backup-N` **ConfigMap**	Snapshot of the manifest ConfigMap taken by the pre-upgrade hook ; indexed 0..4 in a rotating ring
`shareProcessNamespace`	K8s Pod spec field that makes containers share `/proc` ; required for cross-container `kill -HUP <pid>`
Bundle weight-loader	Sidecar container that polls a bundle endpoint and downloads model weights to a shared volume ; signals the participant via SIGHUP ; PR #14 stubs, PR #15 fills
`grafana_dashboard: "1"`	Label that Grafana sidecar (in `kube-prometheus-stack`) recognises to auto-import a dashboard JSON from a ConfigMap
`PriorityClass`	K8s scheduling.k8s.io/v1 resource that ranks pod priorities ; controller value 100000 wins evictions over participants
HPA placeholder	`HorizontalPodAutoscaler` template that ships but is locked to `min=max=1` ; PR #17 leader election will unlock it
Issuer (cert-manager)	A cert-manager CR that signs Certificates ; SelfSigned for Phase 2, CA for Phase 3
`ttlSecondsAfterFinished`	K8s Job field that auto-deletes completed pods after N seconds ; hook Jobs use 300 (5 min)
Canary slice	A separate chart release with smaller scale, different tenant slug, deployed alongside production ; tests new manifest/image on a sliver of traffic
Runbook URL	Annotation on a PrometheusRule alert pointing at the operator's troubleshooting doc ; Alertmanager / PagerDuty link to it

End of architecture document. Total: ~ 2 500 lines.

Next deliverable : PR #14 code (8 commits, ~ +2 000 LOC of YAML + JSON + JSON Schema + helm-unittest).

PR #15

helm-meta-swarm-lint workflow

Chapter 17 of 17 — source : `docs/EFE_A1_PHASE2_PR15_ARCHITECTURE.md` (1,984 LOC)

EFE-A1 Phase 2 -- PR #15 : `helm-meta-swarm-lint` workflow

Field	Value
PR	#15 (Sprint A1 Phase 2 / P2.6)
Branch	`efe-a1/p2-pr15-helm-meta-swarm-lint`
Status	Architecture frozen -- awaiting code
Author	EFE platform team
Reviewers	1x meta-swarm + 2x sre + 1x platform-deploy + 1x sec + 1x ci-ops
Depends on	PR #13 (Helm sub-chart skeleton), PR #14 (Helm values + templates production-grade)
Blocks	PR #16 (mTLS provisioning) only if lint surface needs additional Rego rules for cert-manager primitives
L2 gate	G12 (PR-gated Helm chart correctness -- every change to `charts/efe-stack/charts/efe-meta-swarm/**` passes 14 lint stages before merge)
Threat IDs in scope	T29 (supply-chain : un-pinned lint tools), T30 (silent rule drift : tool upgrades that change semantics), T31 (false-positive fatigue), T32 (lint bypass via admin override), T33 (PR comment spam), T34 (SBOM tampering), T35 (concurrency leaks on stale runners)
Roadmap reference	EFE_A1_PHASE2_ROADMAP.md § C8
CVSS bandwidth	All <= SEV-3 ; PR introduces only CI surface, zero runtime change

Table of contents

1. TL;DR
2. Mission + non-goals
3. Pre-flight : current CI state (PR #14 workflow recap)
4. Why a SECOND workflow (not amend the existing one)
5. Branch + commit plan (6 commits)
6. Architectural decisions (AD1-AD30)
7. Lint tool taxonomy (14 stages, who-owns-what)
8. Layout : files added + modified
9. Workflow file structure (top-level)
10. Concurrency + cancellation model
11. Cache strategy
12. Reusable workflow contract (`workflow_call`)
13. Job 01 -- bootstrap (checkout + setup-helm + cache restore)
14. Job 02 -- `yamllint` + `shellcheck` + `pre-commit`
15. Job 03 -- `helm lint --strict`
16. Job 04 -- `chart-testing` (`ct lint`)
17. Job 05 -- `values.schema.json` self-validation (ajv + meta-schema)
18. Job 06 -- `helm-docs` README sync check
19. Job 07 -- `kubeconform` multi-K8s-version matrix (1.28 / 1.29 / 1.30 / 1.31)
20. Job 08 -- Conftest (OPA Rego) custom EFE policies
21. Job 09 -- Trivy IaC scan (SARIF -> GitHub Code Scanning)
22. Job 10 -- Checkov scan

23. Job 11 -- Polaris severity-tiered audit
24. Job 12 -- ArtifactHub metadata lint (`ah lint`)
25. Job 13 -- SBOM generation (Syft -> CycloneDX) + attestation
26. Job 14 -- PR sticky comment summary
27. Custom Rego policy library (`policies/rego/efe/`)
28. `.yamllint` configuration
29. `.pre-commit-config.yaml` configuration
30. Renovate integration (tool version tracking)
31. Branch protection contract
32. Workflow timing budgets (per-job + total)
33. Failure modes
34. Runbook (operator triage by error class)
35. Threat-model coverage (T29-T35)
36. Tests strategy (this workflow tests itself)
37. Acceptance criteria (AC1-AC35)
38. Risks + mitigations
39. Rollback strategy
40. Sequence diagrams (PR push -> lint completion)
41. Reviewer checklist
42. Appendix A -- tool version pinning table
43. Appendix B -- environment variable matrix
44. Appendix C -- Rego rule index

0. TL;DR

PR #14 shipped a production-grade Helm sub-chart but its CI workflow (`.github/workflows/helm-meta-swarm.yml`) is a "rendering + roundtrip battery", not a lint suite. It catches **structural** drift (golden snapshot diff) and **manifest semantic** drift (Rust roundtrip), but NOT :

• K8s anti-patterns (privileged container, missing securityContext,

host-network) -- Polaris is wired with `severity=danger` only ; we miss `warning` + `notice` tiers.

• Multi-K8s-version compatibility -- kubeconform runs only on `1.28.0`.

• Custom organization-level invariants (e.g. "every resource MUST carry

`efe.io/tenant`", "no `:latest` image tags") -- currently enforced by `helm-unittest`, NOT by policy-as-code (so a NEW template without a unit test escapes detection).

• Supply-chain provenance -- no SBOM, no signing.

• JSON Schema sanity -- we validate **values against the schema** but

NOT the **schema against its own meta-schema** ; a corrupted `values.schema.json` silently allows anything.

• Documentation drift -- `README.md` table of values gets stale across

PRs ; no `helm-docs` gate.

• Lint-style -- no `yamllint`, no `shellcheck` for hook scripts, no

`pre-commit` hooks generated for local devs.

PR #15 adds a dedicated workflow `.github/workflows/helm-meta-swarm-lint.yml` with **14 parallel-where-possible jobs** that close all the gaps above. Critical design points :

- **Separate file** from `helm-meta-swarm.yml` (AD1) : lint is fast

(<= 5 min wall-clock), end-to-end test is slow (<= 15 min) ; we want independent failure surfaces and independent caching.

- **Reusable workflow** via `workflow_call` (AD12) : the umbrella

chart's PR workflow `helm-stack-test.yml` invokes ours, so umbrella PRs that touch only this sub-chart get the same lint coverage.

- **Custom Rego policies** in `policies/rego/efe/` (AD8, AD18, AD27,

AD29) enforce 11 EFE-specific invariants that no off-the-shelf tool knows about (e.g. `efe.io/tenant` label, `replicaCount: 1` lock, SIGHUP-driven reload requires `automountServiceAccountToken: false` on runtime pods, hook Jobs require their own SA, etc.).

- **Sticky PR comment** (AD13) surfaces the lint summary as one tidy

table on the PR conversation -- operators don't have to dig into GitHub Actions logs.

- **Renovate-tracked tool versions** (AD3, AD26) : every linter image

is pinned by SHA + tag, and Renovate auto-PRs upgrades that flow through this same lint suite.

- **Branch protection contract** (§ 30) : exactly 5 of the 14 jobs are

"required" for merge ; the others are "informational" but visible.

- **Bypass** is disabled (AD25) : no `[skip lint]` magic commit

string ; admin override exists but requires a separate `repo-admin` GitHub role + leaves an audit trail.

35 acceptance criteria (§ 36). 30 architectural decisions (§ 5). Zero runtime change to the meta-swarm controller / participants -- this PR is a pure CI feature.

1. Mission + non-goals

Mission

Ship a deterministic, low-false-positive, fast-feedback lint workflow that fires on every PR touching charts/efe-stack/charts/efe-meta-swarm/ (or policies/rego/efe/, or .github/workflows/helm-meta-swarm-lint.yml itself). The workflow MUST :

1. Catch every off-the-shelf K8s anti-pattern within 5 minutes of push.
2. Enforce EFE-specific invariants encoded as Rego policies (idempotent,

reviewable, versionable).

1. Produce one human-readable summary (sticky comment) plus

machine-readable SARIF (uploaded to GitHub Code Scanning).

1. Block merge when ANY of the 5 required gates fails.
2. Stay green on the PR #14 head-of-main scenarios (no false positives

on the 6 kovant scenarios + 4 example overlays).

1. Be reusable by the umbrella chart workflow with a stable

`workflow_call` interface.

1. Be locally reproducible : `pre-commit run --all-files` runs ~90%

of the same checks before the developer pushes.

1. Be reproducible deterministically -- same input -> same output

verdict, no flaky network probes.

Non-goals

• **Not** a replacement for `helm-meta-swarm.yml` (PR #14). That

workflow stays -- it's the integration battery (golden diff + cross-version + Rust roundtrip + parent wiring). PR #15's `lint` workflow runs in parallel as a faster feedback surface.

• **Not** a runtime policy enforcer. This is **pre-merge** CI ; we

do NOT install Kyverno / OPA Gatekeeper on the cluster. (Kyverno is on the EFE roadmap as a separate M-series workstream.)

• **Not** a CVE scanner for the meta-swarm binary itself. Image scans

are an upstream-build responsibility ; we scan only the chart.

• **Not** a SLSA / Sigstore signing pipeline. We DO emit a SBOM

(AD23) but signing the chart artifact is M-series scope.

• **Not** a multi-arch matrix. All linters run on `linux/amd64`

ubuntu-22.04 ; the chart itself is arch-neutral (no binary dependencies on a specific arch).

• **Not** a place to enforce coding style on Rego policies themselves

(that's `opa fmt` ; if needed it'll be a tiny addition in PR #16).

2. Pre-flight : current CI state (PR #14 workflow recap)

`.github/workflows/helm-meta-swarm.yml` (after PR #14 audit pass) contains 7 jobs :

```
  +-------------------------------------------------------------------+
  | helm-meta-swarm.yml -- integration battery (PR #14 head-of-main)   |
  +------------------+------------------------------------------------+
  | lint-and-template|  helm lint + helm template x 6 scenarios       |
  | kubeconform      |  3 of 6 scenarios on K8s 1.28.0 only           |
  | golden-snapshot  |  diff against tests/golden/*.yaml              |
  | manifest-roundtr |  Rust cargo test manifest_roundtrip x 4 flavor |
  | parent-wiring    |  efe-server EFE_META_SWARM_* env propagation   |
  | helm-unittest    |  11 suites x ~93 assertions                    |
  | polaris          |  severity=danger gate, 3 scenarios only        |
  +------------------+------------------------------------------------+

  Total wall-clock today : ~ 8 - 12 min (mostly Rust cargo build).
```

What this workflow IS good at :

• End-to-end rendering correctness (golden diff catches any template

drift byte-for-byte).

• Manifest-side schema roundtrip (Rust validator must accept the

rendered ConfigMap).

• Parent-chart wiring (cross-chart env var propagation).

• K8s API conformance on **one** version (1.28.0).

• Per-template unit assertions (the 11 suites).

• Polaris `danger`-tier audit.

What this workflow is NOT good at :

• **Multi-version K8s coverage** -- we only check 1.28.0. Users on

1.29/1.30/1.31 get no signal.

• **Style** -- no yamllint, no shellcheck on the hook bash scripts.

• **Schema-of-schema** -- `values.schema.json` itself is never

validated as a valid JSON Schema 2020-12 document.

- **Doc sync** -- README.md tables of values drift silently.
- **Custom EFE invariants beyond what helm-unittest catches** --

if someone adds a brand-new template with no unit test, the "every resource carries `efe.io/tenant`" rule is invisible.

- **Supply chain** -- no SBOM.
- **PR ergonomics** -- failures land as 7 separate red checks ;

no consolidated comment.

- **Reusability** -- the umbrella workflow has to duplicate logic.

PR #15 fills every one of those gaps.

3. Why a SECOND workflow (not amend the existing one)

This is AD1. Considered alternatives :

A) **Amend** `helm-meta-swarm.yml` : add 7 more jobs.

- PRO : single source of truth, single sticky comment surface.
- CON : couples the slow-test cadence (Rust cargo build) to the

fast-lint cadence. Every lint change forces a full rebuild + cargo cache invalidation.

- CON : the 14-job DAG would have heterogeneous concurrency needs

(lint can run flat in parallel ; Rust roundtrip needs the template artifact from `lint-and-template`).

- CON : a single workflow_call signature can't cleanly separate

"lint-only" callers from "full-battery" callers.

B) **Two workflows in one file** (jobs with different `if:` guards).

- PRO : single file.
- CON : `if:` proliferation reduces readability ; GitHub Actions'

`if:` is expression-based and hard to debug.

- CON : `concurrency:` is workflow-level -- can't have different

cancellation policies inside one workflow.

C) **Two workflow files** (chosen).

- PRO : independent caching, independent concurrency groups,

independent path filters.

- PRO : `workflow_call` reusability is cleaner (PR #15's workflow

has one signature ; PR #14's another).

- PRO : the umbrella `helm-stack-test.yml` can invoke both

independently or only the lint one for fast-path PRs.

- CON : two PR check entries. Mitigated by sticky-comment

consolidation (Job 14).

- CON : tiny code duplication for setup-helm + checkout.

We choose (C). The CON ("two PR checks") is actually a PRO from a SRE perspective : it tells you which class of regression you hit (structural drift -> look at the integration battery ; policy violation -> look at the lint workflow).

4. Branch + commit plan (6 commits)

Each commit is independently reviewable and CI-green on its own (the workflow is only invoked once the file lands ; intermediate commits that add `policies/rego/efe/*.rego` files are no-ops for CI).

1. `pr15: skeleton workflow file + Job 01-04 (style + helm lint + ct lint)`

• Adds `.github/workflows/helm-meta-swarm-lint.yml` with

Jobs 01-04 (bootstrap, yamllint+shellcheck+pre-commit, helm lint, ct lint).

• Adds `.yamllint`, `.pre-commit-config.yaml`,

`chart-testing.yaml` (ct config).

1. `pr15: schema self-lint + helm-docs + multi-K8s kubeconform`

• Adds Jobs 05-07 (ajv schema-of-schema, helm-docs sync check,

kubeconform matrix 1.28/1.29/1.30/1.31).

• Adds `Makefile` target `make helm-docs` for local sync.

1. `pr15: policy-as-code -- Conftest + custom Rego rules`

• Adds Job 08 (Conftest).

• Adds `policies/rego/efe/` directory with 11 Rego files

(tenant_label, replica_lock, sighup_token_off, hook_sa_required, no_latest_tag, ...).

• Adds `policies/rego/efe/data/` for parameterized rules.

• Adds policy unit tests (`*_test.rego`).

1. `pr15: security scans -- Trivy + Checkov + Polaris tiered`

• Adds Jobs 09-11.

• Adds `.trivyignore`, `.checkov.yaml`, `polaris.yaml` configs.

1. `pr15: provenance -- ArtifactHub lint + SBOM (Syft -> CycloneDX)`

• Adds Jobs 12-13.

• Adds `artifacthub-pkg.yml` if missing.

1. `pr15: PR ergonomics -- sticky comment + workflow_call interface`

• Adds Job 14.

• Adds `workflow_call` block + outputs.

• Renovate `renovate.json` updated to track the linter image

versions.

5. Architectural decisions (AD1-AD30)

AD	Decision	Status
AD1	New file `.github/workflows/helm-meta-swarm-lint.yml` ; do NOT amend `helm-meta-swarm.yml`. See § 3.	accepted
AD2	Job DAG, not single super-job -- independent failure surfaces + parallel scheduling.	accepted
AD3	Every linter image pinned by **tag + digest** (`@sha256:...`). Renovate updates.	accepted
AD4	`kubeconform` runs as a **matrix** of K8s versions : 1.28, 1.29, 1.30, 1.31 -- the four versions we expect operators on.	accepted
AD5	Workflow-level `concurrency: { group: lint-${{ github.ref }}, cancel-in-progress: true }` -- stale PR pushes cancel the previous lint run.	accepted
AD6	Cache `~/.cache/helm` + plugin binaries + tool binaries via `actions/cache` keyed on the tool-versions matrix hash.	accepted
AD7	Per-stage `fail-fast: false` inside the matrix (Job 07) ; workflow-level `fail-fast: false` so all 14 jobs run regardless of any single failure.	accepted
AD8	Policy-as-code via **Conftest (OPA Rego)** -- chosen over Kyverno (cluster-needed) and OPA Gatekeeper (cluster-needed). Conftest is binary-only.	accepted
AD9	`.yamllint` config in repo root, NOT inline in workflow -- versioned, local-runnable.	accepted
AD10	Helm version pinned to **v3.13.3** (matches PR #14's workflow). Renovate-managed.	accepted
AD11	`chart-testing.yaml` (ct config) limits scope to `charts/efe-stack/charts/efe-meta-swarm`, skips chart-bumping (umbrella governs that).	accepted
AD12	Expose `workflow_call` with two outputs : `lint-result` (overall pass/fail) + `sbom-artifact-name`. Used by umbrella's PR workflow.	accepted
AD13	PR sticky comment via `marocchino/sticky-pull-request-comment@v2.9.0` -- one comment, edited per push (not spammy).	accepted
AD14	Polaris configured with severity-tiered output -- `danger` blocks merge, `warning` annotates, `ignore` produces SARIF metric.	accepted
AD15	Trivy outputs SARIF, uploaded to GitHub Code Scanning via `github/codeql-action/upload-sarif@v3`. Severity threshold : `HIGH,CRITICAL`.	accepted
AD16	**No** "lint-only-on-diff" optimisation. Every file in the chart is re-linted on every PR -- cheap (5 min wall-clock) and predictable.	accepted
AD17	Runner : `ubuntu-22.04` to match PR #14's workflow. Plan to bump to `ubuntu-24.04` in PR #16 when matrix-validated.	accepted

AD	Decision	Status
AD18	Custom Rego rules live at `policies/rego/efe/` (top of repo, not inside `charts/`) -- they're org-wide, not chart-local.	accepted
AD19	Conftest scopes : the **rendered** manifests (output of `helm template`), NOT the raw templates. This catches bugs no static-template lint sees.	accepted
AD20	K8s versions matrix is parameterised by `env.KUBE_VERSIONS` so a renovate-style PR can update all four in one diff.	accepted
AD21	`helm-docs` runs as a **gate** (not auto-commit) -- PR author runs `make helm-docs` locally, CI verifies no drift.	accepted
AD22	Pre-commit hooks generated by `pre-commit` framework ; `.pre-commit-config.yaml` is the single source. CI runs `pre-commit run --all-files`.	accepted
AD23	SBOM format : **CycloneDX 1.5** JSON. Industry-aligned (NIST SP 800-218 maps SPDX/CycloneDX interchangeably).	accepted
AD24	Image scan : NOT in scope. Chart-only scan. The meta-swarm controller/participant images are scanned by their own (upstream) build pipeline.	accepted
AD25	No `[skip lint]` or `[skip ci]` magic-string bypass. Branch protection requires the 5 required jobs ; admin override leaves an audit trail.	accepted
AD26	Renovate tracks every linter version in `renovate.json` ; PRs that bump tool versions go through this same lint suite -- fixed-point check.	accepted
AD27	Required Rego rule coverage must include the top-10 K8s anti-patterns from CIS K8s benchmark v1.8 + 11 EFE-specific rules.	accepted
AD28	Captured artifacts on failure : rendered manifests for all 6 scenarios + Conftest verdict JSON + Polaris JSON. Retention 7 days.	accepted
AD29	EFE-specific Rego policies enforce : `efe.io/tenant` label on every namespaced resource, no `:latest`, securityContext required on every container, no `privileged`, no `hostNetwork/hostPID/hostIPC`, every Pod with a non-default SA gets `automountServiceAccountToken: false` unless it's a hook Job, runtime pods reject `shareProcessNamespace: true` unless `bundle.sidecar.enabled`, every Service has `appProtocol`, every container has `resources.limits` AND `resources.requests`, every NetworkPolicy explicitly sets `policyTypes`, every PrometheusRule alert has `annotations.runbook_url`.	accepted
AD30	Path filter `paths:` on `pull_request` is the **only** trigger surface. No `push:` trigger ; protects from accidental re-runs on main.	accepted

6. Lint tool taxonomy (14 stages, who-owns-what)

Stage	Tool	Job #	What it catches	Severity tier	Required?
1	`yamllint`	02	YAML style -- indentation, line length, trailing whitespace	hard	yes
2	`shellcheck`	02	Bash bugs in `pre-upgrade-snapshot.yaml` + `post-upgrade-sighup.yaml` scripts	hard	yes
3	`pre-commit`	02	Aggregated local-runnable check : aggregates 1+2 + custom hooks	hard	yes
4	`helm lint --strict`	03	Helm built-in lint : schema, required fields, dependency consistency	hard	**yes**
5	`chart-testing ct lint`	04	Chart bumping, dependency check, custom Lua hooks	hard	yes
6	`ajv-cli` + JSON Schema 2020-12	05	`values.schema.json` itself is a valid schema (validates against its meta-schema)	hard	yes
7	`helm-docs --check`	06	`README.md` values-table in sync with `values.yaml`	soft	no
8	`kubeconform` x 4 K8s versions	07	K8s API conformance per version	hard	**yes**
9	`conftest test` (OPA Rego)	08	Custom EFE policies (11 rules) + CIS K8s benchmark top-10	hard	**yes**
10	`trivy config`	09	IaC misconfigurations (CVE-like for K8s manifests)	tiered	yes
11	`checkov`	10	IaC misconfigurations (different ruleset, secondary source)	tiered	no
12	`polaris audit`	11	K8s pod-spec best practices (danger / warning / ignore tiers)	tiered	**yes**
13	`ah lint`	12	ArtifactHub metadata correctness	soft	no
14	`syft` (CycloneDX)	13	SBOM emission + GitHub artifact upload	informational	no

Job 01 (bootstrap) and Job 14 (sticky comment) are infrastructure, not linters per se.

Required for merge (5 gates) : Jobs 03, 07, 08, 11, plus the overall workflow status (Job 14 outputs a pass/fail aggregate that the umbrella workflow can consume).

7. Layout : files added + modified

```
.github/workflows/
+-- helm-meta-swarm.yml                    # (PR #14 ; unchanged)
+-- helm-meta-swarm-lint.yml               # NEW -- this PR

policies/rego/efe/                         # NEW dir
+-- kubernetes.rego                          # CIS K8s benchmark top-10
+-- tenant_label.rego                        # efe.io/tenant required
+-- no_latest_tag.rego                       # image tag pinning
+-- security_context.rego                    # securityContext required
+-- no_privileged.rego                       # no privileged containers
+-- no_host_namespace.rego                   # no hostNetwork/PID/IPC
+-- sa_token.rego                            # automountServiceAccountToken: false on runtime pods
+-- share_process_namespace.rego             # only allowed when bundle.sidecar.enabled
+-- service_appprotocol.rego                 # every Service has appProtocol
+-- container_resources.rego                 # requests + limits required
+-- networkpolicy_types.rego                 # NetworkPolicy must set policyTypes
+-- prometheusrule_runbook.rego              # every alert has runbook_url
+-- data/
|   +-- tenant.json                          # parameterised allowed tenants
|   +-- runtime_pods.json                    # which deployments are "runtime" vs "hook"
+-- kubernetes_test.rego                   # + 11 per-rule test files

charts/efe-stack/charts/efe-meta-swarm/
+-- README.md                              # MODIFIED -- `helm-docs` regenerated
+-- artifacthub-pkg.yml                    # NEW -- ArtifactHub metadata
+-- (everything else from PR #14, unchanged)

.yamllint                                  # NEW -- yamllint config
.pre-commit-config.yaml                    # NEW -- pre-commit hooks
chart-testing.yaml                         # NEW -- ct config
polaris.yaml                               # NEW -- polaris config (severity tiers)
.trivyignore                               # NEW -- trivy false-positive allowlist
.checkov.yaml                              # NEW -- checkov config
renovate.json                              # MODIFIED -- track lint tool versions
Makefile                                   # MODIFIED -- make helm-docs target
docs/EFE_A1_PHASE2_PR15_ARCHITECTURE.md    # NEW -- this doc
```

22 new files + 4 modified.

8. Workflow file structure (top-level)

```
# .github/workflows/helm-meta-swarm-lint.yml

name: helm-meta-swarm-lint

on:
  pull_request:
    paths:
      - "charts/efe-stack/charts/efe-meta-swarm/**"
      - "policies/rego/efe/**"
      - ".github/workflows/helm-meta-swarm-lint.yml"
      - ".yamllint"
      - ".pre-commit-config.yaml"
      - "chart-testing.yaml"
      - "polaris.yaml"
      - ".trivyignore"
      - ".checkov.yaml"
  workflow_call:
    outputs:
      lint-result:
        description: "Overall lint verdict : pass | fail"
        value: ${{ jobs.sticky-summary.outputs.verdict }}
      sbom-artifact-name:
        description: "Name of the CycloneDX SBOM artifact"
        value: ${{ jobs.sbom.outputs.artifact-name }}
    inputs:
      debug:
        description: "Verbose tool output (for triage)"
        required: false
        type: boolean
        default: false

# AD5 -- workflow-level cancel-in-progress.
concurrency:
  group: helm-meta-swarm-lint-${{ github.ref }}
  cancel-in-progress: true

env:
  HELM_VERSION:        "v3.13.3"
  KUBECONFORM_VERSION: "v0.6.4"
  CHART_TESTING_VER:   "v3.10.1"
  CONFTEST_VERSION:    "v0.49.1"
  TRIVY_VERSION:       "v0.49.1"
  CHECKOV_VERSION:     "3.1.79"
  POLARIS_VERSION:     "8.5.5"
  HELM_DOCS_VERSION:   "v1.12.0"
  AH_VERSION:          "v1.18.0"
  SYFT_VERSION:        "v1.0.1"
  AJV_VERSION:         "5.0.0"
  PYTHON_VERSION:      "3.11"

  # Matrix of K8s versions for kubeconform.  Bumping a version here
  # forces a fresh PR through this same lint suite -- fixed-point.
  KUBE_VERSIONS: '["1.28.0", "1.29.0", "1.30.0", "1.31.0"]'

jobs:
  # Jobs 01 .. 14 (see § 12 - § 25).
  ...
```

9. Concurrency + cancellation model

AD5. Three layers of cancellation :

1. **Workflow level** : `concurrency: { group: lint-<ref>, cancel-in-progress: true }`.

On a PR push, the previous run for that PR's `head_ref` is cancelled. Saves ~80% of compute on rapid-fire pushes.

1. **Matrix level** (kubeconform -- Job 07) : `strategy.fail-fast: false`.

A failure on K8s 1.28 does NOT cancel the other three versions. We WANT every-version signal on every push.

1. **No job-level cancellation** between sibling jobs. Job 03's

failure does NOT stop Jobs 04-13 ; the operator should see every class of failure in the sticky comment.

This means a single push generates one workflow run with up to ~17 sub-tasks running in parallel (14 jobs + 3 extra matrix entries for kubeconform). Total wall-clock <= 5 min.

10. Cache strategy

AD6. Three caches :

1. **Helm plugin cache** : `~/.local/share/helm/plugins` --

`actions/cache@v4` keyed on `${{ env.HELM_VERSION }}-plugins-v1`. Persists the helm-diff + helm-unittest plugins across runs.

1. **Tool binaries cache** : `/usr/local/bin/{kubeconform,conftest,...}` --

keyed on a SHA-1 hash of the env-var block above. Renovate PRs change the hash -> cache invalidates -> fresh download.

1. **Pre-commit cache** : `~/.cache/pre-commit` -- keyed on

`${{ hashFiles('.pre-commit-config.yaml') }}`.

No cache on rendered manifests : Job 03's `helm template` output is job-local (the output is used by Jobs 07, 08, 09, 11 ; we pass it via `actions/upload-artifact` + `download-artifact`).

Cache size budget : <= 80 MiB compressed (well under GitHub's 10 GiB per repo quota).

11. Reusable workflow contract (`workflow_call`)

AD12. This workflow is invokable from another workflow :

```
# .github/workflows/helm-stack-test.yml (umbrella ; lives in PR #20 scope
# but mirrors the same pattern).
jobs:
  meta-swarm-lint:
    uses: ./.github/workflows/helm-meta-swarm-lint.yml
    with:
      debug: ${{ github.event_name == 'workflow_dispatch' }}
```

The `outputs:` block exposes :

- `lint-result` -- `pass` if all 5 required gates passed ; `fail`

otherwise. Umbrella's workflow gates its own merge on this.

- `sbom-artifact-name` -- the name of the SBOM artifact emitted by

Job 13. Umbrella's release workflow consumes it.

The `inputs:` block has one : `debug` (boolean). When `true`, all linters run with `-v` / `--debug` flags ; sticky comment is verbose.

12. Job 01 -- bootstrap (checkout + setup-helm + cache restore)

```
jobs:
  bootstrap:
    name: 01 / bootstrap
    runs-on: ubuntu-22.04
    timeout-minutes: 3
    outputs:
      cache-key:    ${{ steps.cache-key.outputs.value }}
      rendered-dir: ${{ steps.render.outputs.dir }}
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }       # full history for helm-docs

      - id: cache-key
        run: |
          # Hash the tool versions block so renovate PRs invalidate caches.
          v=$(cat &lt;&lt;EOF | sha1sum | head -c 12
          helm=${{ env.HELM_VERSION }}
          kubeconform=${{ env.KUBECONFORM_VERSION }}
          ct=${{ env.CHART_TESTING_VER }}
          conftest=${{ env.CONFTEST_VERSION }}
          trivy=${{ env.TRIVY_VERSION }}
          checkov=${{ env.CHECKOV_VERSION }}
          polaris=${{ env.POLARIS_VERSION }}
          ah=${{ env.AH_VERSION }}
          syft=${{ env.SYFT_VERSION }}
          ajv=${{ env.AJV_VERSION }}
          EOF
          )
          echo "value=$v" &gt;&gt; $GITHUB_OUTPUT

      - uses: azure/setup-helm@v4
        with: { version: ${{ env.HELM_VERSION }} }

      - name: Cache tool binaries
        uses: actions/cache@v4
        with:
          path: /usr/local/bin/lint-tools
          key:  lint-tools-${{ steps.cache-key.outputs.value }}

      # Render all 6 scenarios once, share with downstream jobs.
      - id: render
        run: |
          mkdir -p /tmp/rendered
          for scen in kovant-3p kovant-disabled kovant-network-policy \
                      kovant-service-monitor kovant-prod kovant-hooks ; do
            helm template tst charts/efe-stack/charts/efe-meta-swarm \
              --namespace efe \
              -f "charts/efe-stack/charts/efe-meta-swarm/tests/scenarios/${scen}.values.yaml" \
              &gt; "/tmp/rendered/${scen}.yaml"
          done
          # Examples overlays too.
          for flavor in dev staging prod canary ; do
            helm template tst charts/efe-stack/charts/efe-meta-swarm \
              --namespace efe \
              -f "charts/efe-stack/charts/efe-meta-swarm/examples/values-${flavor}.yaml" \
              &gt; "/tmp/rendered/values-${flavor}.yaml"
          done
          echo "dir=/tmp/rendered" &gt;&gt; $GITHUB_OUTPUT

      - uses: actions/upload-artifact@v4
        with:
          name: rendered-${{ github.run_id }}
          path: /tmp/rendered/
          retention-days: 7
```

Outputs : `rendered-dir` (consumed by Jobs 07, 08, 09, 11) + `cache-key` (consumed by every downstream job).

Timing budget : 3 minutes.

13. Job 02 -- `yamllint` + `shellcheck` + `pre-commit`

```
  style:
    name: 02 / style (yamllint + shellcheck + pre-commit)
    needs: bootstrap
    runs-on: ubuntu-22.04
    timeout-minutes: 3
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with: { python-version: ${{ env.PYTHON_VERSION }} }

      - run: pip install pre-commit==3.6.0 yamllint==1.35.1

      - name: pre-commit (drives yamllint + shellcheck + custom hooks)
        run: pre-commit run --all-files --show-diff-on-failure
```

`.yamllint` config (§ 27) sets line-length 120, indentation 2, and allows specific Helm template syntax (e.g. `{{ ... }}` inside string values doesn't trigger key-duplication false-positives).

`shellcheck` runs on every `command: ['sh', '-c', ...]` block in the hook templates. Detected via the custom pre-commit hook in `.pre-commit-config.yaml` (§ 28) that greps for `command:` blocks and extracts the script body to a temp file.

14. Job 03 -- `helm lint --strict`

```
  helm-lint:
    name: 03 / helm lint --strict       # REQUIRED for merge
    needs: bootstrap
    runs-on: ubuntu-22.04
    timeout-minutes: 2
    steps:
      - uses: actions/checkout@v4
      - uses: azure/setup-helm@v4
        with: { version: ${{ env.HELM_VERSION }} }

      - name: helm lint --strict
        run: |
          helm lint charts/efe-stack/charts/efe-meta-swarm --strict \
            -f charts/efe-stack/charts/efe-meta-swarm/values.yaml

      # Lint each scenario individually -- catches values-merge edge cases
      # that the chart-wide lint misses.
      - name: helm lint per scenario
        run: |
          for scen in kovant-3p kovant-disabled kovant-network-policy \
                      kovant-service-monitor kovant-prod kovant-hooks ; do
            echo "-- lint ${scen} --"
            helm lint charts/efe-stack/charts/efe-meta-swarm --strict \
              -f "charts/efe-stack/charts/efe-meta-swarm/tests/scenarios/${scen}.values.yaml"
          done
```

`--strict` upgrades warnings to errors. Required gate.

15. Job 04 -- `chart-testing` (`ct lint`)

```
  ct-lint:
    name: 04 / chart-testing
    needs: bootstrap
    runs-on: ubuntu-22.04
    timeout-minutes: 3
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }

      - uses: helm/chart-testing-action@v2.6.1
        with: { version: ${{ env.CHART_TESTING_VER }} }

      - name: ct lint
        run: |
          ct lint --config chart-testing.yaml --target-branch main
```

`chart-testing.yaml`:

```
# chart-testing.yaml
chart-dirs:
  - charts/efe-stack/charts
chart-repos: []                       # all local, no external repos
target-branch: main
check-version-increment: true         # require Chart.yaml version bump on changes
validate-chart-schema: true           # validates against /charts/spec.json
validate-maintainers: true
debug: false
```

The `check-version-increment` gate is critical : every PR touching the chart MUST bump `Chart.yaml`'s `version` field (semver) ; missing this is a hard fail.

16. Job 05 -- `values.schema.json` self-validation

```
  schema-of-schema:
    name: 05 / values.schema.json self-validation
    needs: bootstrap
    runs-on: ubuntu-22.04
    timeout-minutes: 1
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }

      - run: npm install -g ajv-cli@${{ env.AJV_VERSION }}

      - name: Validate schema against JSON Schema 2020-12 meta-schema
        run: |
          # Download the official 2020-12 meta-schema (cached locally).
          curl -sL https://json-schema.org/draft/2020-12/schema \
            -o /tmp/meta-schema.json

          # Validate values.schema.json AS data against the meta-schema.
          ajv compile \
            --spec=draft2020 \
            -s /tmp/meta-schema.json \
            -d charts/efe-stack/charts/efe-meta-swarm/values.schema.json

          echo "[OK] values.schema.json is a valid JSON Schema 2020-12 document"

      - name: Validate the example overlays against values.schema.json
        run: |
          ajv validate \
            --spec=draft2020 \
            -s charts/efe-stack/charts/efe-meta-swarm/values.schema.json \
            -d "charts/efe-stack/charts/efe-meta-swarm/examples/values-*.yaml"
```

This catches the class of bug where a refactor of `values.schema.json` introduces invalid syntax (e.g. dangling `$ref`) that Helm itself silently ignores -- but means runtime validation does nothing. PR #14's audit caught one such bug ; this gate prevents recurrence.

17. Job 06 -- `helm-docs` README sync check

```
  helm-docs:
    name: 06 / helm-docs sync
    needs: bootstrap
    runs-on: ubuntu-22.04
    timeout-minutes: 2
    steps:
      - uses: actions/checkout@v4

      - uses: docker://jnorwood/helm-docs:v1.12.0
        with:
          args: &gt;-
            --chart-search-root=charts/efe-stack/charts/efe-meta-swarm
            --template-files=README.md.gotmpl
            --output-file=README.md.generated

      - name: Diff generated README against committed README
        run: |
          if ! diff -u \
              charts/efe-stack/charts/efe-meta-swarm/README.md \
              charts/efe-stack/charts/efe-meta-swarm/README.md.generated ; then
            echo "[FAIL] README.md is out of sync.  Run \`make helm-docs\` locally and commit." &gt;&amp; ■
  2
            exit 1
          fi
```

`Makefile` target added in the same PR :

```
helm-docs:
    @docker run --rm \
        -v $(PWD)/charts/efe-stack/charts/efe-meta-swarm:/helm-docs \
        jnorwood/helm-docs:v1.12.0 \
        --template-files=README.md.gotmpl \
        --output-file=README.md
```

Soft gate (warning) for v1 ; will be promoted to hard gate in PR #16.

18. Job 07 -- `kubeconform` multi-K8s-version matrix

```
  kubeconform:
    name: 07 / kubeconform (K8s ${{ matrix.kube }})    # REQUIRED for merge
    needs: bootstrap
    runs-on: ubuntu-22.04
    timeout-minutes: 4
    strategy:
      fail-fast: false                  # we want signal for every version
      matrix:
        kube: ${{ fromJson(env.KUBE_VERSIONS) }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/download-artifact@v4
        with:
          name: rendered-${{ github.run_id }}
          path: /tmp/rendered

      - name: Install kubeconform
        run: |
          curl -L "https://github.com/yannh/kubeconform/releases/download/${{ env.KUBECONFORM_VERSION }}/ ■
    kubeconform-linux-amd64.tar.gz" \
            | tar xz
          sudo mv kubeconform /usr/local/bin/

      # Per-scenario validation against the matrix K8s version.
      # CRDs (PrometheusRule, ServiceMonitor, Issuer, Certificate) need
      # the datreeio catalog so kubeconform doesn't choke.
      - name: kubeconform ${{ matrix.kube }} x all scenarios
        run: |
          for f in /tmp/rendered/*.yaml ; do
            echo "-- ${matrix-kube:-${{ matrix.kube }}} : $(basename $f) --"
            kubeconform -strict -summary \
              -kubernetes-version ${{ matrix.kube }} \
              -schema-location default \
              -schema-location 'https://raw.githubusercontent.com/datreeio/CRDs-catalog/main/{{ .Group }} ■
    /{{ .ResourceKind }}_{{ .ResourceAPIVersion }}.json' \
              "$f"
          done
```

12 scenarios x 4 K8s versions = 48 checks. Wall-clock ~ 2 min per matrix entry, 4 entries in parallel.

Required gate. Failure on ANY K8s version blocks merge.

19. Job 08 -- Conftest (OPA Rego) custom EFE policies

```
  conftest:
    name: 08 / Conftest (OPA policies)    # REQUIRED for merge
    needs: bootstrap
    runs-on: ubuntu-22.04
    timeout-minutes: 3
    steps:
      - uses: actions/checkout@v4
      - uses: actions/download-artifact@v4
        with:
          name: rendered-${{ github.run_id }}
          path: /tmp/rendered

      - name: Install Conftest
        run: |
          curl -sL "https://github.com/open-policy-agent/conftest/releases/download/${{ env.CONFTEST_VERS ■
    ION }}/conftest_$(echo ${{ env.CONFTEST_VERSION }} | sed s/^v//)_Linux_x86_64.tar.gz" \
            | tar xz
          sudo mv conftest /usr/local/bin/

      - name: Run policy tests (the Rego rules themselves)
        run: |
          conftest verify --policy policies/rego/efe --data policies/rego/efe/data

      - name: Apply policies to rendered manifests
        run: |
          for f in /tmp/rendered/*.yaml ; do
            echo "-- policy check : $(basename $f) --"
            conftest test \
              --policy policies/rego/efe \
              --data policies/rego/efe/data \
              --output stdout \
              --no-color \
              "$f"
          done

      - name: Conftest verdict as SARIF
        if: failure()
        run: |
          conftest test \
            --policy policies/rego/efe \
            --data policies/rego/efe/data \
            --output sarif \
            /tmp/rendered/*.yaml &gt; /tmp/conftest.sarif

      - uses: github/codeql-action/upload-sarif@v3
        if: failure()
        with:
          sarif_file: /tmp/conftest.sarif
          category: conftest
```

Required gate. Custom EFE rule violations are non-negotiable.

20. Job 09 -- Trivy IaC scan

```
trivy:
  name: 09 / Trivy IaC
  needs: bootstrap
  runs-on: ubuntu-22.04
  timeout-minutes: 3
  steps:
    - uses: actions/checkout@v4
    - uses: actions/download-artifact@v4
      with:
        name: rendered-${{ github.run_id }}
        path: /tmp/rendered

    - name: Trivy config scan
      uses: aquasecurity/trivy-action@0.20.0
      with:
        scan-type: config
        scan-ref:  /tmp/rendered
        format:    sarif
        output:    /tmp/trivy.sarif
        severity:  HIGH,CRITICAL
        exit-code: 1               # hard fail on HIGH/CRITICAL
        ignore-unfixed: false

    - uses: github/codeql-action/upload-sarif@v3
      if: always()
      with:
        sarif_file: /tmp/trivy.sarif
        category: trivy
```

`.trivyignore` lives in repo root with explicit reasons for each ignored finding (Renovate-tracked review cycle).

21. Job 10 -- Checkov

```
checkov:
  name: 10 / Checkov
  needs: bootstrap
  runs-on: ubuntu-22.04
  timeout-minutes: 3
  steps:
    - uses: actions/checkout@v4
    - uses: actions/download-artifact@v4
      with:
        name: rendered-${{ github.run_id }}
        path: /tmp/rendered

    - uses: actions/setup-python@v5
      with: { python-version: ${{ env.PYTHON_VERSION }} }

    - run: pip install checkov==${{ env.CHECKOV_VERSION }}

    - name: Checkov scan
      run: |
        checkov --directory /tmp/rendered \
          --framework kubernetes \
          --config-file .checkov.yaml \
          --output sarif --output-file-path /tmp/checkov.sarif \
          --soft-fail              # secondary source -- informational

    - uses: github/codeql-action/upload-sarif@v3
      if: always()
      with:
        sarif_file: /tmp/checkov.sarif
        category: checkov
```

Soft-fail by design -- Checkov's K8s ruleset overlaps Trivy's ; we keep it as a secondary signal but don't block merge.

22. Job 11 -- Polaris severity-tiered

```
  polaris:
    name: 11 / Polaris (severity-tiered)    # REQUIRED for merge
    needs: bootstrap
    runs-on: ubuntu-22.04
    timeout-minutes: 3
    steps:
      - uses: actions/checkout@v4
      - uses: actions/download-artifact@v4
        with:
          name: rendered-${{ github.run_id }}
          path: /tmp/rendered

      - name: Install Polaris
        run: |
          curl -sL "https://github.com/FairwindsOps/polaris/releases/download/${{ env.POLARIS_VERSION }}/ ■
    polaris_linux_amd64.tar.gz" \
            | tar xz
          sudo mv polaris /usr/local/bin/

      - name: Polaris audit (danger -- blocks merge)
        run: |
          for f in /tmp/rendered/*.yaml ; do
            polaris audit \
              --audit-path "$f" \
              --config polaris.yaml \
              --severity danger \
              --output-format pretty
          done

      - name: Polaris audit (warning -- annotates, doesn't block)
        run: |
          for f in /tmp/rendered/*.yaml ; do
            polaris audit \
              --audit-path "$f" \
              --config polaris.yaml \
              --severity warning \
              --output-format json &gt; "/tmp/polaris-warn-$(basename $f .yaml).json" || true
          done

      - name: Upload Polaris warning + ignore-tier reports as artifact
        uses: actions/upload-artifact@v4
        with:
          name: polaris-${{ github.run_id }}
          path: /tmp/polaris-*.json
          retention-days: 7
```

`polaris.yaml` configures severity tiers per check. Top-level `checks:` block locks `danger`-tier for : `runAsRootAllowed`, `runAsPrivileged`, `hostNetworkSet`, `hostPIDSet`, `hostIPCSet`, `notReadOnlyRootFilesystem`, `dangerousCapabilities` ; `warning`-tier for : `cpuRequestsMissing`, `cpuLimitsMissing`, `memoryRequestsMissing`, `memoryLimitsMissing`, `pullPolicyNotAlways`, `tagNotSpecified`.

23. Job 12 -- ArtifactHub metadata lint

```
  artifacthub:
    name: 12 / ArtifactHub metadata
    needs: bootstrap
    runs-on: ubuntu-22.04
    timeout-minutes: 1
    steps:
      - uses: actions/checkout@v4

      - name: Install ah
        run: |
          curl -sL "https://github.com/artifacthub/hub/releases/download/${{ env.AH_VERSION }}/ah_$(echo  ■
    ${{ env.AH_VERSION }} | sed s/^v//)_linux_amd64.tar.gz" \
            | tar xz
          sudo mv ah /usr/local/bin/

      - run: ah lint charts/efe-stack/charts/efe-meta-swarm/
```

The `artifacthub-pkg.yml` file (NEW in this PR) provides the metadata that the public ArtifactHub crawler picks up.

24. Job 13 -- SBOM (Syft -> CycloneDX)

```
sbom:
  name: 13 / SBOM (CycloneDX)
  needs: bootstrap
  runs-on: ubuntu-22.04
  timeout-minutes: 2
  outputs:
    artifact-name: ${{ steps.sbom.outputs.name }}
  steps:
    - uses: actions/checkout@v4

    - name: Install Syft
      run: |
        curl -sL "https://github.com/anchore/syft/releases/download/${{ env.SYFT_VERSION }}/syft_$(echo ■
   ${{ env.SYFT_VERSION }} | sed s/^v//)_linux_amd64.tar.gz" \
          | tar xz
        sudo mv syft /usr/local/bin/

    - id: sbom
      run: |
        syft scan dir:charts/efe-stack/charts/efe-meta-swarm \
          -o cyclonedx-json=/tmp/sbom.cdx.json
        echo "name=sbom-${{ github.run_id }}" &gt;&gt; $GITHUB_OUTPUT

    - uses: actions/upload-artifact@v4
      with:
        name: sbom-${{ github.run_id }}
        path: /tmp/sbom.cdx.json
        retention-days: 30        # longer than other artifacts ; SBOM is releasable
```

25. Job 14 -- PR sticky comment summary

```
sticky-summary:
  name: 14 / PR sticky summary
  needs:
    - style
    - helm-lint
    - ct-lint
    - schema-of-schema
    - helm-docs
    - kubeconform
    - conftest
    - trivy
    - checkov
    - polaris
    - artifacthub
    - sbom
  runs-on: ubuntu-22.04
  if: always()                    # run even if predecessors failed
  timeout-minutes: 2
  outputs:
    verdict: ${{ steps.compose.outputs.verdict }}
  steps:
    - id: compose
      run: |
        # Required gates : style + helm-lint + kubeconform + conftest + polaris.
        required_pass=true
        for j in style helm-lint kubeconform conftest polaris ; do
          r="${{ needs[j].result }}"
          if [ "$r" != "success" ] ; then required_pass=false ; fi
        done
        if [ "$required_pass" = "true" ] ; then
          echo "verdict=pass" &gt;&gt; $GITHUB_OUTPUT
        else
          echo "verdict=fail" &gt;&gt; $GITHUB_OUTPUT
        fi

    - uses: marocchino/sticky-pull-request-comment@v2.9.0
      if: github.event_name == 'pull_request'
      with:
        header: helm-meta-swarm-lint
        message: |
          ## helm-meta-swarm-lint summary

          | Stage                            | Result                                              |
          |----------------------------------|-----------------------------------------------------|
          | 02 / style                       | ${{ needs.style.result }}                           |
          | 03 / helm lint                   | ${{ needs.helm-lint.result }}                       |
          | 04 / chart-testing               | ${{ needs.ct-lint.result }}                         |
          | 05 / schema-of-schema            | ${{ needs.schema-of-schema.result }}                |
          | 06 / helm-docs                   | ${{ needs.helm-docs.result }}                       |
          | 07 / kubeconform                 | ${{ needs.kubeconform.result }}                     |
          | 08 / Conftest                    | ${{ needs.conftest.result }}                        |
          | 09 / Trivy                       | ${{ needs.trivy.result }}                           |
          | 10 / Checkov                     | ${{ needs.checkov.result }}                         |
          | 11 / Polaris                     | ${{ needs.polaris.result }}                         |
          | 12 / ArtifactHub                 | ${{ needs.artifacthub.result }}                     |
          | 13 / SBOM                        | ${{ needs.sbom.result }}                            |

          **Overall verdict** : ${{ steps.compose.outputs.verdict }}

          (Required gates : style, helm-lint, kubeconform, conftest, polaris.)
```

The `sticky-pull-request-comment` action edits a single comment per PR, keyed on the `header:` value. Subsequent pushes update the same comment in-place rather than spamming a new one.

26. Custom Rego policy library (`policies/rego/efe/`)

11 EFE-specific rules. Each rule lives in its own `.rego` file with a matching `_test.rego` file (Rego unit tests run by `conftest verify`).

26.1 `tenant_label.rego` -- every namespaced resource must carry `efe.io/tenant`

```
package main

# Every namespaced resource (Deployment, Service, Job, ...) MUST have
# the `efe.io/tenant` label.  Cluster-scoped resources (PriorityClass,
# ClusterRole, ...) are exempt.
deny[msg] {
  input.kind != ""
  not cluster_scoped[input.kind]
  not input.metadata.labels["efe.io/tenant"]
  msg := sprintf("%s/%s is missing efe.io/tenant label",
                 [input.kind, input.metadata.name])
}

cluster_scoped := {"PriorityClass", "ClusterRole", "ClusterRoleBinding"}
```

26.2 `no_latest_tag.rego` -- image tags must be pinned

```
package main

deny[msg] {
  some container in input.spec.template.spec.containers
  endswith(container.image, ":latest")
  msg := sprintf("container %s/%s uses :latest image tag",
                 [input.metadata.name, container.name])
}

deny[msg] {
  some container in input.spec.template.spec.containers
  not contains(container.image, ":")
  msg := sprintf("container %s/%s has no image tag at all",
                 [input.metadata.name, container.name])
}
```

26.3 `security_context.rego` -- every container must have securityContext

```
package main

deny[msg] {
  pod_specs := pod_spec_paths[_]
  some container in walk_path(input, pod_specs).containers
  not container.securityContext
  msg := sprintf("container %s in %s has no securityContext",
                 [container.name, input.metadata.name])
}

pod_spec_paths := [
  ["spec", "template", "spec"],
  ["spec"],                          # Pod kind
]
```

26.4 `no_privileged.rego` -- no privileged containers

```
package main

deny[msg] {
  some container in input.spec.template.spec.containers
  container.securityContext.privileged == true
  msg := sprintf("%s/%s runs privileged",
                 [input.metadata.name, container.name])
}
```

26.5 `no_host_namespace.rego` -- no hostNetwork / hostPID / hostIPC

```
package main

deny[msg] {
  input.spec.template.spec.hostNetwork == true
  msg := sprintf("%s sets hostNetwork=true", [input.metadata.name])
}

deny[msg] {
  input.spec.template.spec.hostPID == true
  msg := sprintf("%s sets hostPID=true", [input.metadata.name])
}

deny[msg] {
  input.spec.template.spec.hostIPC == true
  msg := sprintf("%s sets hostIPC=true", [input.metadata.name])
}
```

26.6 `sa_token.rego` -- runtime pods reject SA-token mounting

Carried over from PR #14's design. Only the hook Jobs may `automountServiceAccountToken: true`; runtime pods (controller + participants + test connection) MUST be `false`.

```
package main

import future.keywords.in

runtime_kinds := {"Deployment", "StatefulSet", "DaemonSet", "Pod"}

# data.runtime_pods.exempt is a list of hook-Job-style pods that
# legitimately mount the SA token.
deny[msg] {
  input.kind in runtime_kinds
  not exempt(input.metadata.name)
  input.spec.template.spec.automountServiceAccountToken != false
  msg := sprintf("%s/%s does not set automountServiceAccountToken: false",
                 [input.kind, input.metadata.name])
}

exempt(name) {
  some hook_pattern in data.runtime_pods.hook_patterns
  regex.match(hook_pattern, name)
}
```

`data/runtime_pods.json`:

```
{
  "hook_patterns": [
    "^.*-hook-pre-upgrade-snapshot$",
    "^.*-hook-post-upgrade-sighup$"
  ]
}
```

26.7 `share_process_namespace.rego` -- only allowed when sidecar enabled

The sidecar SIGHUP-loop requires `shareProcessNamespace: true` on the participant Pod (cross-PID-namespace SIGHUP delivery). But this is a T26 threat surface -- only allow it when the participant's name maps to a deployment with `bundle.sidecar.enabled`.

```
package main

# Reject shareProcessNamespace on any Pod that doesn't have a
# `bundle-loader` container.
deny[msg] {
  input.spec.template.spec.shareProcessNamespace == true
  not has_bundle_loader(input)
  msg := sprintf("%s sets shareProcessNamespace=true but has no bundle-loader sidecar",
                 [input.metadata.name])
}

has_bundle_loader(pod) {
  some container in pod.spec.template.spec.containers
  container.name == "bundle-loader"
}
```

26.8 `service_appprotocol.rego` -- every Service port has appProtocol

```
package main

deny[msg] {
  input.kind == "Service"
  some port in input.spec.ports
  not port.appProtocol
  msg := sprintf("Service %s port %s has no appProtocol",
                 [input.metadata.name, port.name])
}
```

26.9 `container_resources.rego` -- requests + limits both required

```
package main

deny[msg] {
  some container in input.spec.template.spec.containers
  not container.resources.requests
  msg := sprintf("container %s/%s has no resources.requests",
                 [input.metadata.name, container.name])
}

deny[msg] {
  some container in input.spec.template.spec.containers
  not container.resources.limits
  msg := sprintf("container %s/%s has no resources.limits",
                 [input.metadata.name, container.name])
}
```

26.10 `networkpolicy_types.rego` -- every NetworkPolicy sets policyTypes

```
package main

deny[msg] {
  input.kind == "NetworkPolicy"
  not input.spec.policyTypes
  msg := sprintf("NetworkPolicy %s has no spec.policyTypes",
                 [input.metadata.name])
}
```

26.11 `prometheusrule_runbook.rego` -- every alert has runbook_url

Carried over from PR #14's audit pass (AC16).

```
package main

deny[msg] {
  input.kind == "PrometheusRule"
  some group in input.spec.groups
  some rule in group.rules
  rule.alert
  not rule.annotations.runbook_url
  msg := sprintf("PrometheusRule %s alert %s has no runbook_url",
                 [input.metadata.name, rule.alert])
}
```

26.12 `kubernetes.rego` -- CIS K8s top-10 (aggregator)

Pulls in the upstream `open-policy-agent/library` CIS K8s benchmark top-10 rules. Vendored under `policies/rego/efe/cis/` to pin against upstream drift.

27. `.yamllint` configuration

```
# .yamllint
extends: default

rules:
  line-length:
    max: 120
    level: warning
  indentation:
    spaces: 2
    indent-sequences: consistent
    check-multi-line-strings: false
  comments:
    min-spaces-from-content: 1
  comments-indentation: enable
  document-start:
    present: false                  # Helm templates don't have ---
  truthy:
    allowed-values: ['true', 'false']
    check-keys: false               # Helm uses "on:", "yes:" in templates

ignore: |
  charts/efe-stack/charts/efe-meta-swarm/files/*.json
  charts/efe-stack/charts/efe-meta-swarm/tests/golden/*.yaml
  charts/efe-stack/charts/efe-meta-swarm/tests/__snapshot__/*
```

We deliberately do NOT lint the golden snapshots (they're rendered output, with `appProtocol:` and other K8s-specific fields that violate generic YAML style) or the Grafana dashboard JSON.

28. `.pre-commit-config.yaml` configuration

```
# .pre-commit-config.yaml
default_install_hook_types: [pre-commit, pre-push]

repos:
  - repo: https://github.com/adrienverge/yamllint
    rev: v1.35.1
    hooks:
      - id: yamllint
        args: ['-c=.yamllint']

  - repo: https://github.com/koalaman/shellcheck-precommit
    rev: v0.10.0
    hooks:
      - id: shellcheck
        args: ['-x', '-S', 'warning']

  - repo: https://github.com/norwoodj/helm-docs
    rev: v1.12.0
    hooks:
      - id: helm-docs
        args:
          - --chart-search-root=charts/efe-stack/charts/efe-meta-swarm

  - repo: https://github.com/instrumenta/conftest
    rev: v0.49.1
    hooks:
      - id: conftest-test
        args:
          - --policy=policies/rego/efe
          - --data=policies/rego/efe/data

  - repo: local
    hooks:
      # Custom hook : extract command: scripts and shellcheck them.
      - id: shellcheck-helm-hooks
        name: shellcheck on Helm hook scripts
        language: script
        entry: scripts/lint-helm-hook-scripts.sh
        files: 'charts/efe-stack/charts/efe-meta-swarm/templates/hooks/.*\.yaml$'
```

The `scripts/lint-helm-hook-scripts.sh` helper extracts `command: ['sh', '-c', '...']` blocks from Helm templates and pipes them to `shellcheck` standalone -- catches bugs the live `command:` string can't be checked directly for.

29. Renovate integration (tool version tracking)

`renovate.json` (modified by this PR) :

```
{
  "$schema": "https://docs.renovatebot.com/renovate-schema.json",
  "extends": ["config:base"],
  "packageRules": [
    {
      "matchManagers": ["github-actions"],
      "matchPaths": [".github/workflows/helm-meta-swarm-lint.yml"],
      "groupName": "helm-meta-swarm-lint tool versions",
      "schedule": ["before 4am on monday"],
      "automerge": false,
      "reviewers": ["@efe-platform/sre"]
    },
    {
      "matchManagers": ["regex"],
      "matchFileNames": [".github/workflows/helm-meta-swarm-lint.yml"],
      "matchPackagePatterns": ["^aquasecurity/trivy-action$", "^helm/chart-testing-action$"],
      "groupName": "helm-meta-swarm-lint actions",
      "automerge": false
    }
  ],
  "regexManagers": [
    {
      "fileMatch": [".github/workflows/helm-meta-swarm-lint.yml"],
```

```
        "matchStrings": [
          "HELM_VERSION:\\s+\"(?&lt;currentValue&gt;v[0-9.]+)\"",
          "KUBECONFORM_VERSION:\\s+\"(?&lt;currentValue&gt;v[0-9.]+)\"",
          "CHART_TESTING_VER:\\s+\"(?&lt;currentValue&gt;v[0-9.]+)\"",
          "CONFTEST_VERSION:\\s+\"(?&lt;currentValue&gt;v[0-9.]+)\"",
          "TRIVY_VERSION:\\s+\"(?&lt;currentValue&gt;v[0-9.]+)\""
        ],
        "datasourceTemplate": "github-releases",
        "depNameTemplate": "helm-meta-swarm-lint:{{groupName}}"
      }
    ]
  }
```

Renovate auto-PRs tool version bumps every Monday morning ; each PR runs through this same lint workflow -> fixed-point validation.

30. Branch protection contract

The repository's `main` branch protection rules (configured in GitHub repo settings, mirrored in `terraform/github/` for code-review) :

- **Required status checks** (5 of 14) :

1. `helm-meta-swarm-lint / 02 / style`
2. `helm-meta-swarm-lint / 03 / helm lint --strict`
3. `helm-meta-swarm-lint / 07 / kubeconform (1.28.0)`

`+ (1.29.0)` + `(1.30.0)` + `(1.31.0)` (matrix entries)

1. `helm-meta-swarm-lint / 08 / Conftest (OPA policies)`
2. `helm-meta-swarm-lint / 11 / Polaris (severity-tiered)`

- Sticky-comment job (`sticky-summary`) is not required -- but its

`verdict` output is consumed by the umbrella workflow's gate.

- Admin override : `repo-admin` GitHub role only. Override produces

an audit-log entry (visible in repo's "Manage access" -> "Audit log"), reviewed weekly by SRE.

- No `[skip lint]` / `[skip ci]` commit-message bypass (AD25).

31. Workflow timing budgets (per-job + total)

Job #	Name	p50	p99	timeout
01	bootstrap	1 min	2 min	3 min
02	style	30 s	1 min	3 min
03	helm lint	30 s	1 min	2 min
04	chart-testing	1 min	2 min	3 min
05	schema-of-schema	15 s	30 s	1 min
06	helm-docs	30 s	1 min	2 min
07	kubeconform x 4	1 min	3 min	4 min
08	conftest	1 min	2 min	3 min
09	trivy	1 min	2 min	3 min
10	checkov	1 min	2 min	3 min
11	polaris	1 min	2 min	3 min
12	artifacthub	15 s	30 s	1 min
13	sbom	30 s	1 min	2 min
14	sticky-summary	10 s	30 s	2 min
total	**wall-clock**	**~3 min**	**~5 min**	**15 min hard cap**

Wall-clock total is dominated by Job 01 (must finish before fan-out) + the slowest of Jobs 03-13 (Polaris).

32. Failure modes

Failure class	Symptom	Action
Tool image 404 (deleted upstream)	download step fails in bootstrap or per-job	Renovate to pin new digest
Cache poisoning	tool binary checksum mismatch	Bust cache by bumping cache-key env var
Schema upstream drift	ajv compile fails on unrecognized keyword	Vendor the meta-schema locally
Rego rule false-positive	Conftest fails on a manifest that should pass	Add data.exceptions[]
Multi-K8s API drift	kubeconform 1.31 fails on a deprecated apiVersion	Bump the template to newer apiVersion
helm-docs drift	README out of sync	Run make helm-docs
Polaris severity re-tier	run-as-root finding promoted from warning -> danger after upgrade	Either fix the chart OR override in polaris.yaml with audit reason

33. Runbook (operator triage by error class)

33.1 "Style" failure

- Symptom : Job 02 red.
- Triage : `pre-commit run --all-files` locally. Most fixes are

1-line edits (line length, trailing whitespace).

- Common cause : `helm-unittest` test files have long inline scripts

that trip line-length. Fix : extract to a `command: |-` block.

33.2 "helm lint" failure

- Symptom : Job 03 red.
- Triage : `helm lint charts/efe-stack/charts/efe-meta-swarm --strict`

locally. Often `--strict` flags missing required values that the default mode doesn't catch.

- Common cause : `values.schema.json` declared a `required: [X]` field

but values.yaml doesn't supply X.

33.3 "Conftest" failure

- Symptom : Job 08 red.
- Triage : the failure output names the exact Rego rule that fired,

e.g. `Conftest test failed : container .../snapshot has no resources.limits (rule: container_resources)`.

- Find the rule under `policies/rego/efe/` and the offending

template ; fix the template (preferred) or add an exception under `policies/rego/efe/data/`.

33.4 "kubeconform 1.31" only failure

- Symptom : 3 of 4 matrix entries pass, 1.31 fails.
- Common cause : we use a deprecated apiVersion that 1.31 removed

(e.g. `policy/v1beta1` PodDisruptionBudget -- already migrated).

- Triage : run `kubectl explain <kind> --api-version=<...>` against

a local 1.31 cluster ; bump the apiVersion.

33.5 "Polaris" warning-tier overflow

- Symptom : warning artifacts blow up to 100+ findings.
- Common cause : a new template adds a container without `resources`.
- Triage : check `tests/golden/*.yaml` for the new template ; add

resources to the values file or the template.

34. Threat-model coverage (T29-T35)

Threat	Description	Mitigation
T29	Supply-chain : un-pinned linter image becomes malicious	Every tool pinned by tag + digest (AD3). Renovate-managed bumps.
T30	Silent rule drift : tool upgrade changes semantics	Renovate PRs -> same lint suite catches the drift on the upgrade PR itself
T31	False-positive fatigue	Severity tiering (Polaris). Soft-fail for Checkov. Audit-reason for `.trivyignore` entries.
T32	Lint bypass via admin override	Audit-log entry. Weekly SRE review. No `[skip lint]` magic.
T33	PR comment spam from repeated pushes	Sticky comment edits in-place (AD13).
T34	SBOM tampering	SBOM emitted from CI runner only. Signed via GitHub OIDC (PR #16 scope).
T35	Concurrency leaks (stale runners holding artifacts)	`cancel-in-progress: true` + 7-day retention cap (AD28).

35. Tests strategy (this workflow tests itself)

PR #15 adds two new test surfaces :

1. **Conftest unit tests** (`*_test.rego`) -- `conftest verify` runs

these. Every Rego rule has a `_test.rego` file with at least one `deny` case and one `allow` case.

1. **Workflow self-test** -- `tests/workflow/lint-self-test.bats` (NEW).

A BATS script that introduces deliberate violations into each scenario's rendered output and asserts the right Rego rule fires. Run locally (not in CI) before merging changes to the policy library.

Example assertion :

```
@test "tenant_label fires on missing label" {
  rendered=$(helm template test ... | yq 'del(.metadata.labels.efe.io/tenant)')
  run conftest test --policy policies/rego/efe --no-color &amp;lt;&amp;lt;&amp;lt; "$rendered"
  [ "$status" -ne 0 ]
  [[ "$output" =~ "missing efe.io/tenant" ]]
}
```

36. Acceptance criteria (AC1-AC35)

AC	Statement	Status
AC1	`helm lint --strict` passes against `values.yaml` + all 6 scenarios + all 4 example overlays.	gate
AC2	`chart-testing ct lint` passes ; `Chart.yaml` version bump required on every PR touching the chart.	gate
AC3	`yamllint -c .yamllint` passes against every YAML in the chart (excluding golden snapshots + Grafana dashboard JSON).	gate
AC4	`kubeconform -strict` passes for K8s 1.28.0, 1.29.0, 1.30.0, AND 1.31.0 across all 6 scenarios.	gate
AC5	`conftest test` passes against all 6 scenarios with the 11 EFE Rego rules + CIS K8s top-10.	gate
AC6	Trivy IaC scan returns 0 HIGH/CRITICAL findings.	gate
AC7	Checkov scan completes (soft-fail) ; SARIF uploaded to Code Scanning.	informational
AC8	Polaris severity=danger : 0 findings across all 6 scenarios + 4 example overlays.	gate
AC9	SBOM (CycloneDX 1.5 JSON) generated, uploaded as artifact (30-day retention).	informational
AC10	`ah lint` passes against `artifacthub-pkg.yml`.	gate
AC11	`--debug --dry-run` rendering of all 6 scenarios completes in under 60 s wall-clock.	informational
AC12	Concurrency : pushing a new commit to a PR cancels the previous lint run within 10 s.	gate
AC13	PR receives exactly one sticky comment per workflow run ; subsequent pushes edit the same comment in-place.	gate
AC14	Every tool image is pinned by tag (AD3) ; no `:latest` references in the workflow.	gate
AC15	Workflow can be triggered via `workflow_dispatch` with `debug: true` ; tools run with `-v` / `--debug` flags.	informational
AC16	Every Conftest finding produces a SARIF annotation visible on the PR Files tab.	gate
AC17	`workflow_call` interface exposes `lint-result` (pass/fail) + `sbom-artifact-name` ; signature stable across PRs.	gate
AC18	`helm-docs --check` passes ; README.md is byte-identical to `helm-docs` regeneration.	soft
AC19	`pre-commit run --all-files` locally produces the same verdict as CI Job 02.	informational
AC20	`ajv compile` against the 2020-12 meta-schema accepts `values.schema.json` ; example overlays validate against the schema.	gate
AC21	p99 workflow wall-clock <= 5 minutes.	SLO
AC22	Zero false-positives across the PR #14 head-of-main scenarios (6 kovant + 4 examples = 10 inputs ; all green).	gate

AC	Statement	Status
AC23	Failure of any of the 5 required jobs blocks merge.	gate
AC24	Admin override requires the `repo-admin` GitHub role + leaves an audit-log entry.	gate
AC25	Workflow accesses NO cluster credentials. `GITHUB_TOKEN` permissions are read-only except for `pull-requests: write` (sticky comment).	gate
AC26	Every tool version tracked in `renovate.json` ; Renovate PRs flow through this same lint suite (fixed-point).	gate
AC27	The 11 EFE Rego rules enforce : tenant label, no `:latest`, securityContext, no privileged, no host namespace, SA-token disabled on runtime pods, shareProcessNamespace gated on sidecar, Service appProtocol, container resources, NetworkPolicy policyTypes, alert runbook_url. Coverage : every rule has a `_test.rego` with pass + fail cases.	gate
AC28	Captured artifacts on failure : rendered manifests (7d), Conftest SARIF (7d), Polaris JSON (7d), SBOM (30d).	gate
AC29	Custom EFE policies fire correctly : a deliberately-broken render of each scenario produces the expected `deny[msg]`. Asserted by `tests/workflow/lint-self-test.bats`.	gate
AC30	New chart files trigger automatic re-lint via `paths:` filter ; no commits to unrelated files trigger this workflow.	gate
AC31	Sub-chart-only path filter : sibling sub-chart edits (e.g. `charts/efe-stack/charts/efe-server/**`) do NOT trigger this workflow.	gate
AC32	Reusable workflow signature (`workflow_call`) is stable : umbrella's `helm-stack-test.yml` invokes it without modifications.	gate
AC33	Tool-version drift between `renovate.json` and the workflow file's `env:` block is detected by a lint step in this same workflow.	gate
AC34	Every CI annotation includes a doc-link to the relevant runbook section (§ 33).	soft
AC35	The sticky-summary comment includes ALL 14 stages, in order, with their pass/fail/skip verdict.	gate

37. Risks + mitigations

Risk	Likelihood	Impact	Mitigation
Tool version upstream rotation breaks the workflow at midnight	Medium	Medium	Renovate weekly auto-PR ; pin digests not just tags.
Polaris severity re-tiering after an upgrade fails old charts	Low	High	`polaris.yaml` snapshots severity tier ; upgrade-PR triggers re-lint.
Rego rule too strict -> false positives	Medium	Medium	Rule unit tests (`*_test.rego`) ; exception data file under `policies/rego/efe/data/`.
Lint runtime drift > 5 min	Low	Low	Per-job timeout-minutes ; SLO tracked in AC21.
`helm-docs` regeneration desyncs README on Windows CRLF	Low	Low	`.gitattributes` enforces LF.
SARIF schema upstream change	Low	Low	Use `github/codeql-action/upload-sarif@v3` (versioned).
Operator-side `make helm-docs` rejection (Docker not installed)	Medium	Low	Provide non-Docker fallback : `helm-docs` binary download in Makefile.
Concurrency cancellation cancels long-running matrix entry mid-execution	Low	Low	Idempotent jobs ; no shared state ; re-run is safe.
GitHub Actions outage during a release	Low	High	Branch protection allows admin override with audit trail (AD25).

38. Rollback strategy

If PR #15 lands and immediately fails on `main` :

1. Hotfix : disable required-status-check on the broken job in GitHub

branch protection settings (admin role required).

1. Revert the PR (`git revert <merge-sha>`).
2. The workflow file is removed ; subsequent PRs re-trigger the

existing PR #14 workflow only.

Branch protection edit is logged ; SRE post-mortem reviews within 24h.

The 6 commits in PR #15 are each independently revertable (atomic units : workflow-skeleton, schema/docs/multi-K8s, policy-as-code, security scans, provenance, sticky-comment).

39. Sequence diagrams

39.1 PR push -> lint completion

```
PR author                GitHub Actions              Sticky comment
   |                           |                           |
   +-- push commit --&gt;          |                           |
   |                           +- trigger workflow         |
   |                           +- cancel previous run      |
   |                           |    (concurrency group)    |
   |                           +- Job 01 bootstrap         |
   |                           |  +- render 10 scenarios   |
   |                           |  +- upload artifact       |
   |                           +- Jobs 02-13 fan-out       |
   |                           |  (parallel)               |
   |                           |    |                      |
   |                           |    +- Job 02 style        |
   |                           |    +- Job 03 helm lint    |
   |                           |    +- Job 04 ct lint      |
   |                           |    +- Job 05 schema-of-sc |
   |                           |    +- Job 06 helm-docs    |
   |                           |    +- Job 07 kubeconform x 4|
   |                           |    +- Job 08 Conftest     |
   |                           |    +- Job 09 Trivy        |
   |                           |    +- Job 10 Checkov      |
   |                           |    +- Job 11 Polaris      |
   |                           |    +- Job 12 ArtifactHub  |
   |                           |    +- Job 13 SBOM         |
   |                           +- Job 14 sticky-summary    |
   |                           |  +- compose verdict       |
   |                           |  +- edit comment --&gt;      |
   |                           |                           +- rendered
   |  &lt;-- sticky comment --    |                           |   on PR
   |      visible on PR        |                           |
   |                           |                           |
```

39.2 Rego rule firing flow

```
Job 08 Conftest:
   |
   +- conftest verify (rule unit tests pass first)
   |
   +- for each rendered.yaml:
   |  +- conftest test --policy policies/rego/efe/
   |  |  +- parse manifest (YAML -&gt; JSON)
   |  |  +- for each Rego rule:
   |  |  |  +- evaluate deny[msg] {...}
   |  |  |  +- collect msg if rule fires
   |  |  +- aggregate findings per file
   |  +- output : pretty stdout + SARIF (on failure)
   |
   +- if any finding:
      +- exit 1 -&gt; job fails
      +- upload SARIF -&gt; Code Scanning
      +- Job 14 sticky-summary reports 'fail'
```

40. Reviewer checklist

- [ ] **AD-coverage** : every AD has a corresponding workflow stanza or config-file content that implements it.
- [ ] **AC-coverage** : every AC is either a workflow assertion (job pass/fail) or an artifact uploaded for retention.
- [ ] **Tool pin** : grep for `:latest` in `.github/workflows/helm-meta-swarm-lint.yml` returns nothing.
- [ ] **Renovate** : `renovate.json` `regexManagers:` entries cover every `env.X_VERSION` line.
- [ ] **Rego unit tests** : `conftest verify --policy policies/rego/efe` returns 0 failures locally.

- [ ] **Self-test** : `bats tests/workflow/lint-self-test.bats` passes locally.
- [ ] **Workflow timing** : a sample run on a recent PR clocks <= 5 minutes wall-clock.
- [ ] **Sticky comment** : verified on a draft PR -- comment edits in place, doesn't spawn duplicates.
- [ ] **Branch protection** : the 5 required status checks are configured in the repo settings (cross-checked against `terraform/github/`).
- [ ] **Reusable workflow** : umbrella's `helm-stack-test.yml` is invoking PR #15's workflow via `workflow_call` with the documented signature.
- [ ] **No cluster credential access** : the workflow's `permissions:` block has only `contents: read` + `pull-requests: write`.
- [ ] **Audit trail** : `repo-admin` override path tested on a feature branch ; audit-log entry visible.

41. Appendix A -- tool version pinning table

Tool	Pin	Source
Helm	`v3.13.3`	`github.com/helm/helm` releases
kubeconform	`v0.6.4`	`github.com/yannh/kubeconform` releases
chart-testing	`v3.10.1`	`github.com/helm/chart-testing` releases
Conftest	`v0.49.1`	`github.com/open-policy-agent/conftest` releases
Trivy	`v0.49.1`	`github.com/aquasecurity/trivy` releases
Checkov	`3.1.79`	PyPI
Polaris	`8.5.5`	`github.com/FairwindsOps/polaris` releases
helm-docs	`v1.12.0`	`github.com/norwoodj/helm-docs` releases
ah (ArtifactHub)	`v1.18.0`	`github.com/artifacthub/hub` releases
Syft	`v1.0.1`	`github.com/anchore/syft` releases
ajv-cli	`5.0.0`	npm
pre-commit	`3.6.0`	PyPI
yamllint	`1.35.1`	PyPI
shellcheck	`v0.10.0`	`github.com/koalaman/shellcheck` releases
sticky-pull-request-comment	`v2.9.0`	`github.com/marocchino/sticky-pull-request-comment`
trivy-action	`0.20.0`	`github.com/aquasecurity/trivy-action`
chart-testing-action	`v2.6.1`	`github.com/helm/chart-testing-action`
setup-helm	`v4`	`github.com/azure/setup-helm`
setup-python	`v5`	`github.com/actions/setup-python`
setup-node	`v4`	`github.com/actions/setup-node`
cache	`v4`	`github.com/actions/cache`
checkout	`v4`	`github.com/actions/checkout`
upload-artifact	`v4`	`github.com/actions/upload-artifact`
download-artifact	`v4`	`github.com/actions/download-artifact`
upload-sarif (codeql)	`v3`	`github.com/github/codeql-action`

42. Appendix B -- environment variable matrix

Per-job ENV access surface (zero secrets ; all read from public sources) :

Variable	Job(s) needing it	Source
`HELM_VERSION`	01, 03, 04, 06	workflow `env:`
`KUBECONFORM_VERSION`	07	workflow `env:`
`CHART_TESTING_VER`	04	workflow `env:`
`CONFTEST_VERSION`	08	workflow `env:`
`TRIVY_VERSION`	09	workflow `env:`
`CHECKOV_VERSION`	10	workflow `env:`
`POLARIS_VERSION`	11	workflow `env:`
`HELM_DOCS_VERSION`	06	workflow `env:`

Variable	Job(s) needing it	Source
`AH_VERSION`	12	workflow `env:`
`SYFT_VERSION`	13	workflow `env:`
`AJV_VERSION`	05	workflow `env:`
`PYTHON_VERSION`	02, 10	workflow `env:`
`KUBE_VERSIONS`	07 (matrix expansion)	workflow `env:`
`GITHUB_TOKEN`	14 (sticky comment), 09/10 (SARIF)	GitHub provided

`GITHUB_TOKEN` permissions (workflow-level `permissions:` block) :

```
permissions:
  contents:       read
  pull-requests:  write   # sticky comment
  security-events: write  # SARIF upload
  actions:        read    # for workflow_call recursion
```

No write to `contents`, no access to `secrets:`.

43. Appendix C -- Rego rule index

Rule file	What it enforces	Severity	Test file
`tenant_label.rego`	every namespaced resource has `efe.io/tenant`	deny	`tenant_label_test.rego`
`no_latest_tag.rego`	image tags must be pinned	deny	`no_latest_tag_test.rego`
`security_context.rego`	every container has `securityContext`	deny	`security_context_test.rego`
`no_privileged.rego`	no `privileged: true`	deny	`no_privileged_test.rego`
`no_host_namespace.rego`	no `hostNetwork/hostPID/hostIPC: true`	deny	`no_host_namespace_test.rego`
`sa_token.rego`	runtime pods reject SA-token mount	deny	`sa_token_test.rego`
`share_process_namespace.rego`	gated on `bundle-loader` container presence	deny	`share_process_namespace_test.rego`
`service_appprotocol.rego`	every Service port has `appProtocol`	deny	`service_appprotocol_test.rego`
`container_resources.rego`	`resources.requests` + `resources.limits` both required	deny	`container_resources_test.rego`
`networkpolicy_types.rego`	`policyTypes` set on every NetworkPolicy	deny	`networkpolicy_types_test.rego`
`prometheusrule_runbook.rego`	every alert has `annotations.runbook_url`	deny	`prometheusrule_runbook_test.rego`
`kubernetes.rego` (CIS top-10)	aggregator -- vendored CIS K8s benchmark v1.8	mixed	`kubernetes_test.rego`

11 EFE-specific + 1 CIS aggregator = 12 Rego files. Each rule's test file has >= 1 deny case + >= 1 allow case (24+ test assertions total).

End of architecture document